

MMALS_Prototype_1_1_RC2M_beta_Dynamic_Boundary_Ambiguity_L

June 7, 2026

1 MMALS Prototype v1.1 — RC2M-beta Dynamic Boundary + Ambiguity-Lock Selector Benchmark Harness

Mycelium-Inspired Mutualistic Adaptive Learning Systems

Status: executable benchmark harness for smoke / evidence / robust runs across MNIST, FashionMNIST, PermutedMNIST, and RotatedMNIST.

This notebook is based on the current **RC2M-alpha PATCHED v3 robust harness** and patches **selector logic only**.

RC2M-beta adds:

1. **Dataset-agnostic dynamic weak-boundary detection** on validation/audit memory.
2. **Ambiguous-context regime lock** that protects the guarded context-gap family when the checkpoint remains foggy/ambiguous.
3. **Regression-suite readiness** for FashionMNIST, MNIST, PermutedMNIST, and RotatedMNIST before external datasets such as Split-CIFAR100 / CORE50.

It does **not** add a new backbone, a new training objective, final-test leakage, Hydro regularization, or any performance-policy change outside the selector.

Final-test evaluation starts only after candidate selection is frozen.

1.1 RC2M-beta benchmark purpose

RC2M-alpha was mechanically validated on FashionMNIST robust: it selected online, did not abstain, and did not use final-test data. However, it was too permissive in the ambiguous FashionMNIST regime and regressed against the guarded context-gap family.

RC2M-beta answers the next experimental question:

Can a boundary-first selector remain dataset-agnostic while protecting ambiguous-context regimes with a conservative ambiguity lock?

Core changes versus RC2M-alpha:

- Replace fixed benchmark-family features such as `weak_boundary_eer_2_4_5`, `weak_boundary_auc_2_4_5`, and `class2_eer` with dynamic validation-memory features:
 - `weak_boundary_classes_dynamic`
 - `weak_boundary_eer_dynamic`

- `weak_boundary_auc_dynamic`
- `worst_class_eer`
- `worst_class_auc`
- `topk_boundary_eer`
- `topk_boundary_auc`
- Before accepting the boundary-first winner, detect whether the checkpoint is still an ambiguous-context regime.
- If `ambiguity` is active, prefer `context_gap_selected` or `context_temp_blend_selected_head_guard_035` unless the challenger shows a large multi-signal no-regression gain.

Recommended regression sequence:

1. `RUN_MODE="smoke", DATASET_NAME="FashionMNIST"`.
2. Then robust one-by-one: `FashionMNIST` → `MNIST` → `PermutedMNIST` → `RotatedMNIST`.
3. Only after MNIST-family regression, open external datasets such as `Split-CIFAR100` / `CORe50`.

2 RC2M-beta patch note — dynamic weak boundaries + ambiguity lock

RC2M-beta is a selector-only patch over the current RC2M-alpha PATCHED v3 benchmark harness.

It keeps:

- no new backbone;
- no new training objective;
- no final-test data in candidate construction or selection;
- Hydro audit-only behavior;
- LSA and Energy Context Probe as diagnostics;
- the RC2JG weak-boundary latent calibration candidate in the candidate pool;
- the existing smoke/evidence/robust run modes and MNIST-family datasets.

It changes only the selector:

1. **Dynamic weak-boundary set:** the weak classes are detected from policy-validation memory, not hard-coded as 2/4/5.
2. **Dataset-agnostic boundary metrics:** selection uses macro/micro and dynamically selected weak-boundary EER/AUC features.
3. **Ambiguity lock:** when validation-memory signals indicate an ambiguous-context regime, the selector protects the guarded context-gap family unless a challenger has a large, multi-signal, no-regression improvement.

This is the bridge toward `Split-CIFAR100` / `CORe50`: the selector no longer depends on class IDs 2/4/5 as selection features.

2.1 1. Why RC2Je exists

RC2J introduced the right scientific question:

Which regime am I in, and therefore which already validated policy family should be deployed?

RC2Jc solved the FashionMNIST evidence failure by adding a final-checkpoint ambiguity lock. RC2Jd solved the complementary MNIST failure by adding a clean-context veto, but the FashionMNIST regression check showed that RC2Jd's veto was too permissive: it selected `proto_global_no_repair` when FashionMNIST still needed the RC1b guarded context-gap family.

RC2Je therefore makes the veto harder:

At the final checkpoint, veto the ambiguity lock only if proto/global no-repair is strong, tri-boundary-safe, not meaningfully worse than context-gap, and the context posterior is **hard-clear**: high top-1, low entropy, small gap surrogate, and low Hydro/context turbulence.

This patch does not add a model, a backbone, a new readout family, or a new training objective. It only makes the final selector decision more conservative in ambiguous regimes.

2.2 RC2Je change log from RC2Jd

- Renamed package/output paths to `MMALS_Prototype_1_1_RC2je_Hard_Clarify_Clean_Veto_Guard_EER_D`
- Default profile is `RC2JE_HARD_CLARITY_CLEAN_VETO_GUARD_HYDRO_AUDIT`.
- Preserved strict no-leakage candidate selection.
- Preserved the RC2Jc final-checkpoint ambiguity lock for ambiguous regimes.
- Preserved the RC2Jd clean-context veto idea, but added **hard-clarity gates**:
 - proto validation accuracy must be strong;
 - proto tri-boundary must be healthy;
 - context-gap gain over proto must be small or negative;
 - proto must not be far behind the raw validation winner;
 - proto context top-1 must be high;
 - proto context entropy must be low;
 - gap surrogate must be small;
 - Hydro/context turbulence must be low when available.
- Kept Hydro as audit-only by default.
- Preserved RC2B/RC2G/RC2H/RC2I/RC2J/RC2Jb/RC2Jc/RC2Jd compatibility profiles in spirit; the default branch is RC2Je.
- Preserved the candidate pool and same-backbone paired evaluation protocol.
- Preserved the fixed biometric-style threshold diagnostics:
 - score dump,
 - ROC-style curves,
 - DET-style curves,
 - FAR/FRR curves,
 - EER by class and method,
 - chapters 9b/9c/9d with displayed plots and tables.
- Added RC2Je audit columns:
 - `rc2je_hard_clarity_clean_veto_applied`;
 - `rc2je_hard_clarity_veto_reason`;
 - `rc2je_hard_clarity_signal_count`;
 - `rc2je_proto_top1_hard_clear`;
 - `rc2je_proto_entropy_hard_clear`;

- rc2je_gap_surrogate_hard_clear;
- rc2je_hydro_context_hard_clear.

2.3 RC2JF patch note — true-guard safe veto for PermutedMNIST

This notebook includes a small selector-only patch over RC2JE. The model, backbone, training objective, candidate pool, Hydro audit, and no-leakage protocol are unchanged.

The patch allows `proto_global_head_ce_kl_guard_035` to veto the final ambiguity lock when, on validation/policy memory only, it is the raw-best deployable candidate, safe, better than the guarded context-gap candidate, and improves weak-boundary/min-class signals. This is intended to correct the PermutedMNIST smoke behavior where the final ambiguity lock selected `context_temp_blend_selected_head_guard_035` although validation memory favored the true global CE/KL guard.

Default run target in this file: `RUN_MODE = "robust", DATASET_NAME = "FashionMNIST"`.

RC2JF-Evidence audit patch: this notebook restores the neutral RC2I dual-anchor audit columns for continuity and uses an RC2JF experiment profile/run ID so exported folders, manifests, and ZIP packages are clearly labeled as RC2JF, not RC2JE.

Panel E / Learning-Signal Alignment patch: this notebook adds a live training audit that compares the expected update allocation across MMALS hosts against the observed gradient/update norm allocation. It exports `learning_signal_alignment_panel_e_<RUN_MODE>.csv`, `learning_signal_alignment_panel_e_summary_<RUN_MODE>.csv`, and a compact Panel-E plot. The metric is diagnostic-only: it does not change training, policy selection, candidate construction, Hydro, or final-test evaluation.

RotatedMNIST final-rotation diagnostic patch: this notebook adds a diagnostic-only gate for the final rotated task (4,5) @ +30deg. It flags class-5 / final-rotation weakness and attributes the likely source to one or more of: route/context posterior, readout boundary 4 5, or latent separability. It exports `rotatedmnist_final_rotation_diagnostic_<RUN_MODE>.csv`, `rotatedmnist_final_rotation_gate_<RUN_MODE>.csv`, and a compact diagnostic plot. The diagnostic does not change training, selector decisions, Hydro, LSA, candidate construction, or final-test evaluation.

RC2JG patch note — generic weak-boundary latent separability calibration: this notebook adds one more deployable candidate to the validation-memory candidate pool: `generic_weak_boundary_latent_calibrated`. The candidate is trained only on repair memory and selected only through policy-validation memory. It does not know about RotatedMNIST, class 5, task 4, or final-test diagnostics. It detects the weakest validation-memory class boundary generically, trains a small residual latent adapter/readout with CE+KL+anchor regularization and weak-boundary margin/separation losses, and is allowed to override RC2JF only if strict no-regression gates pass on validation memory.

RC2JG operational rule: first run the previous RC2JF selector and remember its winner. Then train/evaluate the generic latent calibration candidate in shadow/candidate mode using only repair and policy-validation memory. RC2JG can override RC2JF only if validation-memory evidence shows:

- the route/context posterior is healthy;
- a weak boundary exists in validation memory;

- the weak boundary improves after calibration;
- min-class and old-task validation metrics do not regress;
- global validation score damage remains within tolerance;
- no final-test metric is used in the trigger, repair, or selection decision.

If these gates fail, RC2JG silently falls back to the RC2JF winner.

2.4 RC2jh audit patch — Energy Context Probe

This notebook is the RC2jg RotatedMNIST robust notebook updated with an **audit-only Energy Context Probe**.

The training objective, policy selector, Hydro audit behavior, LSA Panel E logic, final gates, and final-test evaluation remain unchanged. The new probe is executed after each seed’s RC2jg training/policy-selection artifact is available, freezes **z0** and **m0**, optimizes only a continuous context vector **c**, and diagnoses whether the latent/memory energy landscape contains a recoverable basin around the oracle context.

Core energy:

$$E(z, m, c) = ||z - D(m, c)||^2 + ||m - M(c)||^2 + \lambda / (2^2) ||c - (z)||^2$$

Scientific status: diagnostic/audit layer only. A failed probe is still evidence because it documents whether the landscape is flat, unstable, locally wrong, or only partially informative.

2.5 RC2jh selector-hygiene patch — dataset-agnostic true-guard veto control

This notebook is based on the **RC2jh Energy Context Probe** notebook, not the earlier RC2JG-only notebook.

It preserves the audit-only Energy Context Probe and adds the FashionMNIST selector-hygiene correction in a dataset-agnostic way:

```
allow true-guard veto only if it is not below context-gap on:
- validation weak-boundary score,
- weak-class / class-2 sentinel behavior when available,
- min-task validation proxy,
- Hydro turbulence,
- Hydro output drift.
```

The rule remains validation-memory / audit-memory only. It does not use final-test data and does not change training, Hydro, LSA, or the energy probe.

2.6 Historical chapter: from RC1b to RC2JG

The v1.1 line moved from explicit task IDs toward inferred context. RC1b established a strong guarded context-gap candidate on FashionMNIST. RC2B/RC2C/RC2D hardened no-leakage policy selection. RC2E/RC2F/RC2G debugged weak-class and boundary behavior. RC2H introduced a conservative anchor. RC2I generalized that anchor into a dual safe-anchor family.

The RC2I-MQ multi-dataset qualification step showed that the problem was not simply “which one policy is best?” It was “which regime are we in?” FashionMNIST behaved like an

ambiguous-context regime. MNIST, RotatedMNIST, and PermutedMNIST behaved more like clean-context/domain-shift regimes.

RC2J made the regime idea explicit. RC2Jb added late-route caution. RC2Jc added final-checkpoint ambiguity lock and fixed FashionMNIST evidence. The fixed EER diagnostic edition then added threshold-level reliability evidence.

MNIST evidence exposed the opposite selector risk: when the forest is clear, the final lock can become over-protective. RC2Jd added a clean-context veto and correctly recovered `proto_global_no_repair` on MNIST. But the FashionMNIST regression check showed that the veto was too permissive: FashionMNIST still had ambiguous context signals and needed the guarded context-gap family.

RC2Je added the missing strictness: keep the clear-path veto, but only when hard validation-memory clarity is present.

RC2JF added a true-guard safe veto for cases such as PermutedMNIST smoke, where validation memory favored the true global CE/KL guarded head over the final context-gap lock. PermutedMNIST evidence then showed that the clean/domain-shift regime can remain strong without forcing a complex repair everywhere: the selector may still keep a simpler proto/global policy when validation evidence supports it.

RotatedMNIST smoke exposed a new kind of failure. The final-rotation diagnostic found that context routing could be healthy and readout-only repair could be insufficient, while the latent space itself remained weakly separated for the validation/test boundary. This should not be fixed by using final-test information. RC2JG therefore converts the observation into a generic, validation-memory-only hypothesis:

If a weak class boundary is detected in policy-validation memory, and the route is healthy, and the latent boundary is weak, train one small repair-memory latent calibration candidate. Select it only if validation-memory safety gates show improvement without regression.

RC2JG is therefore not “RotatedMNIST class-5 repair.” It is a generic weak-boundary latent separability candidate, added to the candidate pool under strict no-leakage and anti-regression discipline.

2.7 12-year-old story: the mushroom map, the foggy forest, the clear forest, and the weak bridge

Imagine a mushroom network under a forest. The network can send food through different underground paths.

Sometimes the forest is foggy. Several trees look similar. If the mushroom sends food too quickly through the first path it sees, it may feed the wrong tree. In that case, the mushroom needs a guarded path. That is like FashionMNIST, where the RC1b guarded context-gap policy wins.

RC2Jc added a final gate:

At the last gate, if the forest is still foggy and the guarded path is not clearly dangerous, choose the guarded path.

Then MNIST showed another case: sometimes the forest is very clear. The mushroom can see the right tree easily. RC2Jd said:

If the path is clear, do not force the guarded path. Keep the simple path.

But RC2Jd trusted the “clear path” too quickly. FashionMNIST still had fog, but RC2Jd sometimes said it was clear.

RC2Je added a stricter rule:

Only call the forest clear if many signs agree: the simple path is strong, old trees are safe, the map is confident, the fog is low, and the path is calm.

RC2JF then added another rule:

If the guarded global path is clearly better than the fog-guarded path on the mushroom’s own memory checks, allow it to win — but only if it is safe.

Now RC2JG deals with a different problem.

Sometimes the mushroom has the **right map** and chooses the **right path**, but two trees still have roots that are too close together underground. The mushroom is not lost. The problem is that the underground signal for two nearby trees is not separated enough.

That is what RotatedMNIST smoke suggested: the context route was healthy, but the latent space did not separate the weak boundary well enough.

RC2JG therefore says:

Do not use the final exam to decide how to repair the forest. Look only at the mushroom’s training memory. If two tree roots are too mixed there, build one small temporary bridge to separate them. Keep the bridge only if it helps on memory checks and does not hurt the rest of the forest.

So RC2JG does **not** say:

“Always repair class 5.”

It says:

“Find any weak boundary from memory only. Try a small latent repair. Keep it only if it is safe.”

This keeps MMALS honest: the mushroom can learn from its own memory, but it cannot cheat by looking at the final test.

2.8 PhD-level interpretation of RC2JG

RC2JG introduces a **generic weak-boundary latent separability calibration candidate** into the no-leakage RC2JF policy-selection harness.

The scientific motivation is the distinction between three failure modes:

1. **Route/context failure:** the inferred context posterior is wrong, high-entropy, unstable, or inconsistent with memory.
2. **Readout boundary failure:** the latent representation is usable, but the deployed head does not exploit it correctly.

3. **Latent separability failure:** context inference is healthy and readout repair is insufficient because the latent representation itself does not separate a validation weak boundary.

The RotatedMNIST final-rotation diagnostic can reveal the third case after final evaluation, but RC2JG must not use that diagnostic as a trigger. Instead, RC2JG defines a generic repair candidate whose trigger, training, and selection use only repair memory and policy-validation memory.

Operationally, RC2JG:

- runs the previous RC2JF selector first and stores its selected candidate;
- detects the weakest validation-memory class boundary using class accuracy, boundary margins, prototype proximity, and validation separability signals;
- trains a small latent calibration candidate on repair memory only;
- evaluates that candidate on policy-validation memory only;
- allows override of RC2JF only if weak-boundary improvement, min-class safety, old-task safety, and global validation damage gates pass;
- exports audit columns explaining whether the candidate was skipped, rejected, shadow-only, or selected.

RC2JG is therefore a **candidate-pool extension**, not a test-specific intervention. It is designed to test whether smooth geometric failures can be repaired through validation-memory latent calibration without regressing clean-context regimes such as MNIST/PermutedMNIST or ambiguous regimes such as FashionMNIST.

2.9 0. Optional install cell for Colab

Most Colab runtimes already include the required packages. Uncomment only if needed.

```
[48]: # !pip -q install torch torchvision pandas numpy matplotlib
```

2.10 1. Imports and reproducibility

This block imports the scientific Python stack, selects CPU/GPU, and defines reproducibility helpers.

```
[49]: import math
import copy
import random
import gc
import time
import shutil
import sys
import os
import json
import hashlib
import uuid
import platform
import contextlib
from datetime import datetime, timezone
from pathlib import Path
```



```

from typing import Dict, List, Tuple, Optional, Any

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import TensorDataset, DataLoader
from IPython.display import display

# Robust torchvision import. Some runtimes have torch/torchvision NMS mismatch.
try:
    from torchvision import datasets, transforms
except RuntimeError as e:
    if "torchvision::nms" not in str(e):
        raise
    for name in list(sys.modules):
        if name.startswith("torchvision"):
            del sys.modules[name]
    try:
        lib = torch.library.Library("torchvision", "DEF")
        lib.define("nms(Tensor dets, Tensor scores, float iou_threshold) ->␣
↳Tensor")
    except Exception:
        pass
    from torchvision import datasets, transforms

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("DEVICE:", DEVICE)
if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))

def set_seed(seed: int):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

def clear_memory():
    gc.collect()
    if torch.cuda.is_available():
        torch.cuda.empty_cache()

def make_loader(ds, batch_size=128, shuffle=True):

```

```

    return DataLoader(ds, batch_size=batch_size, shuffle=shuffle,
↳num_workers=0, pin_memory=False)

def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

def total_parameters(model):
    return sum(p.numel() for p in model.parameters())

def model_size_mb(model, bytes_per_param=4, trainable_only=True):
    n = count_parameters(model) if trainable_only else total_parameters(model)
    return n * bytes_per_param / (1024 ** 2)

```

DEVICE: cpu

2.11 2. Configuration

Default mode is smoke for first execution checks. Switch to robust after smoke passes.

This notebook sets the active profile to:

RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT

Expected configuration print before running:

```

EXPERIMENT_PROFILE: RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT
POLICY_BRANCH: RC2m_beta
ENABLE_RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK: True
RC2C_METHOD: paired_rc2m_beta_dynamic_boundary_ambiguity_lock_policy
RC2M_BETA_VERSION: rc2m_beta_dynamic_boundary_ambiguity_lock_v1

```

The summary-PDF export remains Drive-safe with run_summary.pdf.

```

[50]: RUN_MODE = "robust" # "smoke", "evidence", "robust", "benchmark", or "paper"
DATASET_NAME = "FashionMNIST" # "MNIST", "FashionMNIST", "RotatedMNIST", or
↳"PermutedMNIST"
DATA_ROOT = "./data"

# =====
# RC2JG branch control
# =====
# Choose exactly one profile.
# Recommended default for this notebook:
↳RC2JG_GENERIC_WEAK_BOUNDARY_LATENT_CALIBRATION_HYDRO_AUDIT.
# This RC2jh file keeps the audit-only Energy Context Probe and adds
↳dataset-agnostic selector hygiene.
#
# 1) RC2B_STRICT
# - Reproduce the previous strict no-leakage policy-selection branch.
# - Hydro disabled.

```

```

# - RC2c risk-aware selector disabled.
#
# 2) RC2J_REGIME_AWARE_NO_HYDRO
# - RC2J regime-aware selector enabled.
# - Hydro disabled.
# - Use this to isolate selector effects.
#
# 3) RC2J_REGIME_AWARE_HYDRO_AUDIT [DEFAULT]
# - RC2J regime-aware selector enabled.
# - Hydro metrics are computed on validation memory only.
# - No Hydro training loss and no Hydro selection penalty.
#
# 4) RC2J_REGIME_AWARE_HYDRO_STABILITY_ABLATION [EXPERIMENTAL]
# - RC2J selector enabled.
# - Hydro audit enabled.
# - Validation selection score includes a small drift/turbulence penalty.
# - Report separately; not the default deployable claim.
EXPERIMENT_PROFILE = "RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT"
VALID_EXPERIMENT_PROFILES = [
    "RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT",
    "RC2B_STRICT",
    "RC2JE_HARD_CLARITY_CLEAN_VETO_GUARD_NO_HYDRO",
    "RC2JE_HARD_CLARITY_CLEAN_VETO_GUARD_HYDRO_AUDIT",
    "RC2JE_HARD_CLARITY_CLEAN_VETO_GUARD_HYDRO_STABILITY_ABLATION",
    "RC2JF_TRUE_GUARD_SAFE_VETO_NO_HYDRO",
    "RC2JF_TRUE_GUARD_SAFE_VETO_HYDRO_AUDIT",
    "RC2JF_TRUE_GUARD_SAFE_VETO_HYDRO_STABILITY_ABLATION",
    "RC2JG_GENERIC_WEAK_BOUNDARY_LATENT_CALIBRATION_NO_HYDRO",
    "RC2JG_GENERIC_WEAK_BOUNDARY_LATENT_CALIBRATION_HYDRO_AUDIT",
    "RC2JG_GENERIC_WEAK_BOUNDARY_LATENT_CALIBRATION_HYDRO_STABILITY_ABLATION",
    "RC2JC_FINAL_CHECKPOINT_AMBIGUITY_LOCK_NO_HYDRO",
    "RC2JC_FINAL_CHECKPOINT_AMBIGUITY_LOCK_HYDRO_AUDIT",
    "RC2JC_FINAL_CHECKPOINT_AMBIGUITY_LOCK_HYDRO_STABILITY_ABLATION",
    "RC2JB_LATE_AMBIGUITY_HYSTERESIS_NO_HYDRO",
    "RC2JB_LATE_AMBIGUITY_HYSTERESIS_HYDRO_AUDIT",
    "RC2JB_LATE_AMBIGUITY_HYSTERESIS_HYDRO_STABILITY_ABLATION",
    "RC2J_REGIME_AWARE_NO_HYDRO",
    "RC2J_REGIME_AWARE_HYDRO_AUDIT",
    "RC2J_REGIME_AWARE_HYDRO_STABILITY_ABLATION",
    "RC2I_DUAL_ANCHOR_CONSERVATIVE_NO_HYDRO",
    "RC2I_DUAL_ANCHOR_CONSERVATIVE_HYDRO_AUDIT",
    "RC2I_DUAL_ANCHOR_CONSERVATIVE_HYDRO_STABILITY_ABLATION",
    "RC2H_RC1B_ANCHORED_CONSERVATIVE_NO_HYDRO",
    "RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT",
    "RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_STABILITY_ABLATION",
    "RC2G_TRI_BOUNDARY_EQUILIBRIUM_NO_HYDRO",
    "RC2G_TRI_BOUNDARY_EQUILIBRIUM_HYDRO_AUDIT",

```

```

"RC2G_TRI_BOUNDARY_EQUILIBRIUM_HYDRO_STABILITY_ABLATION",
"RC2D_GATE_ALIGNED_NO_HYDRO",
"RC2D_GATE_ALIGNED_HYDRO_AUDIT",
"RC2D_GATE_ALIGNED_HYDRO_STABILITY_ABLATION",
"RC2C_RISK_AWARE_NO_HYDRO",
"RC2C_RISK_AWARE_HYDRO_AUDIT",
"RC2C_RISK_AWARE_HYDRO_STABILITY_ABLATION",
# Backward-compatible aliases from the merged Hydro branch.
"RC3_HYDRO_AUDIT",
"RC3_HYDRO_STABILITY_SELECTED",
]
if EXPERIMENT_PROFILE not in VALID_EXPERIMENT_PROFILES:
    raise ValueError(f"EXPERIMENT_PROFILE must be one of {VALID_EXPERIMENT_PROFILES}")

# Low-level manual override.
# Keep AUTO_CONFIGURE_FROM_PROFILE=True unless you are deliberately doing an
# ablation.
AUTO_CONFIGURE_FROM_PROFILE = True

# Manual values are used only if AUTO_CONFIGURE_FROM_PROFILE=False.
POLICY_BRANCH = "RC2j" # "RC2b", "RC2c", "RC2d", "RC2e",
# "RC2g", "RC2h", "RC2i", "RC2j", or "RC2m_alpha"
ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
ACTIVATE_HYDRO_AUDIT = True # compute audit metrics
ACTIVATE_HYDRO_REGULARIZER = False # add viscosity loss during training
ACTIVATE_HYDRO_SELECTION = False # use Hydro penalty in validation
# selection
HYDRO_REGULARIZER_MODE = "off" # "off", "viscous",
# "high_gain_viscous"
HYDRO_POLICY_MODE = "audit_only" # "off", "audit_only", or
# "stability_aware"

if AUTO_CONFIGURE_FROM_PROFILE:
    if EXPERIMENT_PROFILE == "RC2B_STRICT":
        POLICY_BRANCH = "RC2b"
        ACTIVATE_RC2C_RISK_AWARE_SELECTION = False
        ACTIVATE_HYDRO_AUDIT = False
        ACTIVATE_HYDRO_REGULARIZER = False
        ACTIVATE_HYDRO_SELECTION = False
        HYDRO_REGULARIZER_MODE = "off"
        HYDRO_POLICY_MODE = "off"

```

```

    elif EXPERIMENT_PROFILE in ["RC2JE_HARD_CLARITY_CLEAN_VETO_GUARD_NO_HYDRO",
↪ "RC2JF_TRUE_GUARD_SAFE_VETO_NO_HYDRO",
↪ "RC2JG_GENERIC_WEAK_BOUNDARY_LATENT_CALIBRATION_NO_HYDRO",
↪ "RC2JC_FINAL_CHECKPOINT_AMBIGUITY_LOCK_NO_HYDRO",
↪ "RC2JB_LATE_AMBIGUITY_HYSTERESIS_NO_HYDRO"]:
        POLICY_BRANCH = "RC2j"
        ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
        ACTIVATE_HYDRO_AUDIT = False
        ACTIVATE_HYDRO_REGULARIZER = False
        ACTIVATE_HYDRO_SELECTION = False
        HYDRO_REGULARIZER_MODE = "off"
        HYDRO_POLICY_MODE = "off"
    elif EXPERIMENT_PROFILE in
↪ ["RC2JE_HARD_CLARITY_CLEAN_VETO_GUARD_HYDRO_AUDIT",
↪ "RC2JF_TRUE_GUARD_SAFE_VETO_HYDRO_AUDIT",
↪ "RC2JG_GENERIC_WEAK_BOUNDARY_LATENT_CALIBRATION_HYDRO_AUDIT",
↪ "RC2JC_FINAL_CHECKPOINT_AMBIGUITY_LOCK_HYDRO_AUDIT",
↪ "RC2JB_LATE_AMBIGUITY_HYSTERESIS_HYDRO_AUDIT"]:
        POLICY_BRANCH = "RC2j"
        ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
        ACTIVATE_HYDRO_AUDIT = True
        ACTIVATE_HYDRO_REGULARIZER = False
        ACTIVATE_HYDRO_SELECTION = False
        HYDRO_REGULARIZER_MODE = "off"
        HYDRO_POLICY_MODE = "audit_only"
    elif EXPERIMENT_PROFILE in
↪ ["RC2JE_HARD_CLARITY_CLEAN_VETO_GUARD_HYDRO_STABILITY_ABLATION",
↪ "RC2JF_TRUE_GUARD_SAFE_VETO_HYDRO_STABILITY_ABLATION",
↪ "RC2JG_GENERIC_WEAK_BOUNDARY_LATENT_CALIBRATION_HYDRO_STABILITY_ABLATION",
↪ "RC2JC_FINAL_CHECKPOINT_AMBIGUITY_LOCK_HYDRO_STABILITY_ABLATION",
↪ "RC2JB_LATE_AMBIGUITY_HYSTERESIS_HYDRO_STABILITY_ABLATION"]:
        POLICY_BRANCH = "RC2j"
        ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
        ACTIVATE_HYDRO_AUDIT = True
        ACTIVATE_HYDRO_REGULARIZER = False
        ACTIVATE_HYDRO_SELECTION = True
        HYDRO_REGULARIZER_MODE = "off"
        HYDRO_POLICY_MODE = "stability_aware"
    elif EXPERIMENT_PROFILE == "RC2J_REGIME_AWARE_NO_HYDRO":
        POLICY_BRANCH = "RC2j"
        ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
        ACTIVATE_HYDRO_AUDIT = False
        ACTIVATE_HYDRO_REGULARIZER = False
        ACTIVATE_HYDRO_SELECTION = False
        HYDRO_REGULARIZER_MODE = "off"
        HYDRO_POLICY_MODE = "off"
    elif EXPERIMENT_PROFILE == "RC2J_REGIME_AWARE_HYDRO_AUDIT":

```

```

POLICY_BRANCH = "RC2j"
ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
ACTIVATE_HYDRO_AUDIT = True
ACTIVATE_HYDRO_REGULARIZER = False
ACTIVATE_HYDRO_SELECTION = False
HYDRO_REGULARIZER_MODE = "off"
HYDRO_POLICY_MODE = "audit_only"
elif EXPERIMENT_PROFILE == "RC2J_REGIME_AWARE_HYDRO_STABILITY_ABLATION":
    POLICY_BRANCH = "RC2j"
    ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
    ACTIVATE_HYDRO_AUDIT = True
    ACTIVATE_HYDRO_REGULARIZER = False
    ACTIVATE_HYDRO_SELECTION = True
    HYDRO_REGULARIZER_MODE = "off"
    HYDRO_POLICY_MODE = "stability_aware"
elif EXPERIMENT_PROFILE == "RC2I_DUAL_ANCHOR_CONSERVATIVE_NO_HYDRO":
    POLICY_BRANCH = "RC2i"
    ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
    ACTIVATE_HYDRO_AUDIT = False
    ACTIVATE_HYDRO_REGULARIZER = False
    ACTIVATE_HYDRO_SELECTION = False
    HYDRO_REGULARIZER_MODE = "off"
    HYDRO_POLICY_MODE = "off"
elif EXPERIMENT_PROFILE == "RC2I_DUAL_ANCHOR_CONSERVATIVE_HYDRO_AUDIT":
    POLICY_BRANCH = "RC2i"
    ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
    ACTIVATE_HYDRO_AUDIT = True
    ACTIVATE_HYDRO_REGULARIZER = False
    ACTIVATE_HYDRO_SELECTION = False
    HYDRO_REGULARIZER_MODE = "off"
    HYDRO_POLICY_MODE = "audit_only"
elif EXPERIMENT_PROFILE == "
↪"RC2I_DUAL_ANCHOR_CONSERVATIVE_HYDRO_STABILITY_ABLATION":
    POLICY_BRANCH = "RC2i"
    ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
    ACTIVATE_HYDRO_AUDIT = True
    ACTIVATE_HYDRO_REGULARIZER = False
    ACTIVATE_HYDRO_SELECTION = True
    HYDRO_REGULARIZER_MODE = "off"
    HYDRO_POLICY_MODE = "stability_aware"
elif EXPERIMENT_PROFILE == "RC2H_RC1B_ANCHORED_CONSERVATIVE_NO_HYDRO":
    POLICY_BRANCH = "RC2h"
    ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
    ACTIVATE_HYDRO_AUDIT = False
    ACTIVATE_HYDRO_REGULARIZER = False
    ACTIVATE_HYDRO_SELECTION = False
    HYDRO_REGULARIZER_MODE = "off"

```

```

HYDRO_POLICY_MODE = "off"
elif EXPERIMENT_PROFILE == "RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT":
    POLICY_BRANCH = "RC2h"
    ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
    ACTIVATE_HYDRO_AUDIT = True
    ACTIVATE_HYDRO_REGULARIZER = False
    ACTIVATE_HYDRO_SELECTION = False
    HYDRO_REGULARIZER_MODE = "off"
    HYDRO_POLICY_MODE = "audit_only"
elif EXPERIMENT_PROFILE == □
↳"RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_STABILITY_ABLATION":
    POLICY_BRANCH = "RC2h"
    ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
    ACTIVATE_HYDRO_AUDIT = True
    ACTIVATE_HYDRO_REGULARIZER = False
    ACTIVATE_HYDRO_SELECTION = True
    HYDRO_REGULARIZER_MODE = "off"
    HYDRO_POLICY_MODE = "stability_aware"
elif EXPERIMENT_PROFILE == "RC2G_TRI_BOUNDARY_EQUILIBRIUM_NO_HYDRO":
    POLICY_BRANCH = "RC2g"
    ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
    ACTIVATE_HYDRO_AUDIT = False
    ACTIVATE_HYDRO_REGULARIZER = False
    ACTIVATE_HYDRO_SELECTION = False
    HYDRO_REGULARIZER_MODE = "off"
    HYDRO_POLICY_MODE = "off"
elif EXPERIMENT_PROFILE == "RC2G_TRI_BOUNDARY_EQUILIBRIUM_HYDRO_AUDIT":
    POLICY_BRANCH = "RC2g"
    ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
    ACTIVATE_HYDRO_AUDIT = True
    ACTIVATE_HYDRO_REGULARIZER = False
    ACTIVATE_HYDRO_SELECTION = False
    HYDRO_REGULARIZER_MODE = "off"
    HYDRO_POLICY_MODE = "audit_only"
elif EXPERIMENT_PROFILE == □
↳"RC2G_TRI_BOUNDARY_EQUILIBRIUM_HYDRO_STABILITY_ABLATION":
    POLICY_BRANCH = "RC2g"
    ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
    ACTIVATE_HYDRO_AUDIT = True
    ACTIVATE_HYDRO_REGULARIZER = False
    ACTIVATE_HYDRO_SELECTION = True
    HYDRO_REGULARIZER_MODE = "off"
    HYDRO_POLICY_MODE = "stability_aware"
elif EXPERIMENT_PROFILE == "RC2D_GATE_ALIGNED_NO_HYDRO":
    POLICY_BRANCH = "RC2d"
    ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
    ACTIVATE_HYDRO_AUDIT = False

```

```

    ACTIVATE_HYDRO_REGULARIZER = False
    ACTIVATE_HYDRO_SELECTION = False
    HYDRO_REGULARIZER_MODE = "off"
    HYDRO_POLICY_MODE = "off"
elif EXPERIMENT_PROFILE == "RC2D_GATE_ALIGNED_HYDRO_AUDIT":
    POLICY_BRANCH = "RC2d"
    ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
    ACTIVATE_HYDRO_AUDIT = True
    ACTIVATE_HYDRO_REGULARIZER = False
    ACTIVATE_HYDRO_SELECTION = False
    HYDRO_REGULARIZER_MODE = "off"
    HYDRO_POLICY_MODE = "audit_only"
elif EXPERIMENT_PROFILE == "RC2D_GATE_ALIGNED_HYDRO_STABILITY_ABLATION":
    POLICY_BRANCH = "RC2d"
    ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
    ACTIVATE_HYDRO_AUDIT = True
    ACTIVATE_HYDRO_REGULARIZER = False
    ACTIVATE_HYDRO_SELECTION = True
    HYDRO_REGULARIZER_MODE = "off"
    HYDRO_POLICY_MODE = "stability_aware"
elif EXPERIMENT_PROFILE == "RC2C_RISK_AWARE_NO_HYDRO":
    POLICY_BRANCH = "RC2c"
    ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
    ACTIVATE_HYDRO_AUDIT = False
    ACTIVATE_HYDRO_REGULARIZER = False
    ACTIVATE_HYDRO_SELECTION = False
    HYDRO_REGULARIZER_MODE = "off"
    HYDRO_POLICY_MODE = "off"
elif EXPERIMENT_PROFILE == "RC2C_RISK_AWARE_HYDRO_AUDIT":
    POLICY_BRANCH = "RC2c"
    ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
    ACTIVATE_HYDRO_AUDIT = True
    ACTIVATE_HYDRO_REGULARIZER = False
    ACTIVATE_HYDRO_SELECTION = False
    HYDRO_REGULARIZER_MODE = "off"
    HYDRO_POLICY_MODE = "audit_only"
elif EXPERIMENT_PROFILE == "RC2C_RISK_AWARE_HYDRO_STABILITY_ABLATION":
    POLICY_BRANCH = "RC2c"
    ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
    ACTIVATE_HYDRO_AUDIT = True
    ACTIVATE_HYDRO_REGULARIZER = False
    ACTIVATE_HYDRO_SELECTION = True
    HYDRO_REGULARIZER_MODE = "off"
    HYDRO_POLICY_MODE = "stability_aware"
elif EXPERIMENT_PROFILE == "RC3_HYDRO_AUDIT":
    POLICY_BRANCH = "RC2g"
    ACTIVATE_RC2C_RISK_AWARE_SELECTION = True

```



```

    ACTIVATE_HYDRO_AUDIT = True
    ACTIVATE_HYDRO_REGULARIZER = False
    ACTIVATE_HYDRO_SELECTION = False
    HYDRO_REGULARIZER_MODE = "off"
    HYDRO_POLICY_MODE = "audit_only"
elif EXPERIMENT_PROFILE == "RC3_HYDRO_STABILITY_SELECTED":
    POLICY_BRANCH = "RC2g"
    ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
    ACTIVATE_HYDRO_AUDIT = True
    ACTIVATE_HYDRO_REGULARIZER = False
    ACTIVATE_HYDRO_SELECTION = True
    HYDRO_REGULARIZER_MODE = "off"
    HYDRO_POLICY_MODE = "stability_aware"

# Hydro branch constants. Selection weights are used only in STABILITY_SELECTED.
HYDRO_REGULARIZER_COEFF = 0.10
HYDRO_HIGH_GAIN_IMPORTANCE_THRESHOLD = 0.55
HYDRO_SELECTION_TURBULENCE_WEIGHT = 0.05 if ACTIVATE_HYDRO_SELECTION else 0.0
HYDRO_SELECTION_LATENT_DRIFT_WEIGHT = 0.20 if ACTIVATE_HYDRO_SELECTION else 0.0
HYDRO_SELECTION_ROUTE_DRIFT_WEIGHT = 0.05 if ACTIVATE_HYDRO_SELECTION else 0.0

NOTEBOOK_BASENAME = _
↳ "MMALS_Prototype_1_1_RC2jh_Generic_Weak_Boundary_Latent_Calibration_RotatedMNIST_Robust_LSA"
NOTEBOOK_FILENAME = f"{NOTEBOOK_BASENAME}.ipynb"
RESULTS_DIR = Path("./mmals_v11_rc2jf_true_guard_safe_veto_selector_results")
RESULTS_DIR.mkdir(parents=True, exist_ok=True)

if RUN_MODE == "smoke":
    TRAIN_PER_CLASS = 200
    TEST_PER_CLASS = 100
    SEEDS = [0]
    EPOCHS_PER_TASK = 2
    MEMORY_PER_TASK = 96
    MEMORY_PER_CLASS = 96
    BATCH_SIZE = 128
    LABEL_NOISE_RATE = 0.05
    INPUT_NOISE_STD = 0.03
    CONTEXT_NOISE_RATE = 0.15
    PROBE_EPOCHS = 10
    PROBE_MAX_PER_TASK = 600
    READOUT_TRAIN_EPOCHS = 2
    READOUT_TRAIN_STEPS = 10
elif RUN_MODE == "evidence":
    TRAIN_PER_CLASS = 1200
    TEST_PER_CLASS = 400
    SEEDS = [0, 1, 2]
    EPOCHS_PER_TASK = 8

```

```

MEMORY_PER_TASK = 256
MEMORY_PER_CLASS = 256
BATCH_SIZE = 128
LABEL_NOISE_RATE = 0.10
INPUT_NOISE_STD = 0.05
CONTEXT_NOISE_RATE = 0.20
PROBE_EPOCHS = 25
PROBE_MAX_PER_TASK = 2000
READOUT_TRAIN_EPOCHS = 5
READOUT_TRAIN_STEPS = 50
elif RUN_MODE == "robust":
    TRAIN_PER_CLASS = 1200
    TEST_PER_CLASS = 400
    SEEDS = list(range(5))
    EPOCHS_PER_TASK = 8
    MEMORY_PER_TASK = 256
    MEMORY_PER_CLASS = 256
    BATCH_SIZE = 128
    LABEL_NOISE_RATE = 0.10
    INPUT_NOISE_STD = 0.05
    CONTEXT_NOISE_RATE = 0.20
    PROBE_EPOCHS = 25
    PROBE_MAX_PER_TASK = 2000
    READOUT_TRAIN_EPOCHS = 5
    READOUT_TRAIN_STEPS = 50
elif RUN_MODE == "benchmark":
    TRAIN_PER_CLASS = 1200
    TEST_PER_CLASS = 400
    SEEDS = list(range(5))
    EPOCHS_PER_TASK = 8
    MEMORY_PER_TASK = 256
    MEMORY_PER_CLASS = 256
    BATCH_SIZE = 128
    LABEL_NOISE_RATE = 0.10
    INPUT_NOISE_STD = 0.05
    CONTEXT_NOISE_RATE = 0.20
    PROBE_EPOCHS = 25
    PROBE_MAX_PER_TASK = 2000
    READOUT_TRAIN_EPOCHS = 5
    READOUT_TRAIN_STEPS = 50
elif RUN_MODE == "paper":
    TRAIN_PER_CLASS = 3000
    TEST_PER_CLASS = 1000
    SEEDS = list(range(10))
    EPOCHS_PER_TASK = 15
    MEMORY_PER_TASK = 512
    MEMORY_PER_CLASS = 512

```

```

    BATCH_SIZE = 128
    LABEL_NOISE_RATE = 0.10
    INPUT_NOISE_STD = 0.05
    CONTEXT_NOISE_RATE = 0.20
    PROBE_EPOCHS = 50
    PROBE_MAX_PER_TASK = 5000
    READOUT_TRAIN_EPOCHS = 10
    READOUT_TRAIN_STEPS = 120
else:
    raise ValueError("RUN_MODE must be smoke, evidence, robust, benchmark, or_
↳paper")

PAIR_MODE = "overlap_chain"
N_TASKS = 5
ROTATION_ANGLES = [-30, -15, 0, 15, 30]
PERMUTATION_SEEDS = [100, 101, 102, 103, 104]
N_CONTEXTS = N_TASKS
N_HOSTS = 5
N_CLASSES = 6
INPUT_DIM = 28 * 28
HIDDEN_DIM = 256
LATENT_DIM = 64
ROUTE_HIDDEN_DIM = 128
TASK_EMB_DIM = 16
CONTEXT_FEAT_DIM = 32

LR = 1e-3
PROBE_LR = 1e-2
READOUT_LR = 3e-3
TEMPERATURE = 0.7
DISTILL_TEMPERATURE = 2.0
PROBE_DISTILL_TEMPERATURE = 2.0
CONTEXT_TEMPERATURE = 0.45
FRECHET_TEMPERATURE = 0.20
LATENT_FRECHET_TEMPERATURE = 0.35
CENTER_TEMPERATURE = 0.20

# Context calibration inherited from v0.14.x.
CONTEXT_SUP_START = 1.00
CONTEXT_SUP_END = 0.15
CONTEXT_SUP_GAMMA = 1.25
CONTEXT_LABEL_SMOOTHING = 0.03
LAMBDA_CONTEXT_CENTER = 0.35
LAMBDA_CONTEXT_ENTROPY = 0.010
LAMBDA_CONTEXT_MARGIN = 0.020
CONTEXT_MARGIN_TARGET = 0.35
LAMBDA_CONTEXT_REPLAY_CE = 0.50

```

```

LAMBDA_CONTEXT_REPLAY_CENTER = 0.20
LAMBDA_CONTEXT_TEACHER_KL = 0.30
LAMBDA_CONTEXT_FEATURE_DRIFT = 0.10
CONTEXT_DISTILL_TEMPERATURE = 2.0
CONTEXT_REPLAY_BATCH = 64
CONTEXT_TEMP_GRID = [0.25, 0.35, 0.45, 0.60, 0.80, 1.00, 1.25, 1.50, 2.00]
TEMP_CALIB_MAX_ITEMS = 128

# Prototype-first context posterior.
BATCH_BLEND_SINGLE = 0.30
BATCH_BLEND_LATENT = 0.55
BATCH_BLEND_FEATURE = 0.15
PROTO_FIRST_BLEND_SINGLE = 0.05
PROTO_FIRST_BLEND_LATENT = 0.75
PROTO_FIRST_BLEND_FEATURE = 0.20

# Replay and functional-memory controls.
BALANCED_REPLAY_WEIGHT = 1.00
BALANCED_REPLAY_BATCH = 128
BALANCED_REPLAY_USE_EVAL_CONTEXT = True
LAMBDA_ENTROPY = 0.002
LAMBDA_METABOLIC = 1e-4
PROTOTYPE_MAX_ITEMS = 256

# Readout/head audit controls.
HEAD_REPAIR_BATCH = 128
HEAD_MARGIN_TARGET = 0.25
HEAD_MARGIN_LAMBDA = 0.30
HARD_CLASSES = [2]
REFERENCE_CLASS_FOR_MARGIN = 4
READOUT_KL_WEIGHT = 0.35
READOUT_TEMP = 0.25
COSINE_SCALE_INIT = 12.0
VALIDATION_SPLIT_FRACTION = 0.25
VALIDATION_MAX_ITEMS_PER_CLASS = 128
VALIDATION_SCORE_CLASS2_WEIGHT = 0.20
VALIDATION_SCORE_MINCLASS_WEIGHT = 0.20
VALIDATION_SCORE_DAMAGE_WEIGHT = 1.00
VALIDATION_ACCEPTANCE_TOL = 0.005

# RC2c configurable strict no-leakage policy-selection controls.
# Compatibility note: the historical RC2B_* and RC3_* names are retained so
↳ inherited helper
# functions do not need to be renamed.
if POLICY_BRANCH == "RC2b":
    RC2C_METHOD = "paired_rc2b_validation_selected_policy"
elif POLICY_BRANCH == "RC2c":

```

```

    RC2C_METHOD = "paired_rc2c_risk_aware_policy"
elif POLICY_BRANCH == "RC2d":
    RC2C_METHOD = "paired_rc2d_gate_aligned_policy"
elif POLICY_BRANCH == "RC2g":
    RC2C_METHOD = "paired_rc2g_tri_boundary_equilibrium_policy"
elif POLICY_BRANCH == "RC2h":
    RC2C_METHOD = "paired_rc2h_rc1b_anchored_conservative_policy"
elif POLICY_BRANCH == "RC2i":
    RC2C_METHOD = "paired_rc2i_dual_anchor_conservative_policy"
elif POLICY_BRANCH == "RC2j":
    if str(EXPERIMENT_PROFILE).startswith("RC2JG_"):
        RC2C_METHOD = "paired_rc2jg_generic_weak_boundary_latent_calibration_policy"
    elif str(EXPERIMENT_PROFILE).startswith("RC2JF_"):
        RC2C_METHOD = "paired_rc2jf_true_guard_safe_veto_policy"
    else:
        RC2C_METHOD = "paired_rc2je_hard_clarity_clean_veto_guard_policy"
else:
    RC2C_METHOD = "paired_rc2g_tri_boundary_equilibrium_policy"

RC3_METHOD = RC2C_METHOD
RC2B_METHOD = RC2C_METHOD
RC2B_STRICT_NO_FINAL_TEST_SELECTION = True
RC2B_EVALUATE_ALL_CANDIDATES_AFTER_SELECTION = True
RC2B_POLICY_REPAIR_FRACTION = 0.50
RC2B_CONTEXT_SELECTION_FRACTION = 0.25
RC2B_POLICY_VALIDATION_FRACTION = 0.25
RC2B_SELECTION_TIE_TOL = 0.002
RC2B_COMPLEXITY_PENALTY = 0.001
RC2B_CLASS2_FLOOR = 0.80
RC2B_CLASS5_FLOOR = 0.90
RC2B_MIN_CLASS_FLOOR = 0.80
RC2B_VALIDATION_SCORE_CLASS5_WEIGHT = 0.05
RC2B_VALIDATION_SCORE_FLOOR_PENALTY = 2.00

# RC2c risk-aware selector controls.
RC2C_RISK_AWARE_SELECTOR_ENABLED = bool(ACTIVATE_RC2C_RISK_AWARE_SELECTION)
RC2C_CLASS2_FLOOR = 0.82
RC2C_MIN_CLASS_FLOOR = 0.82
RC2C_MIN_TASK_FLOOR = 0.82
RC2C_OLD_TASK_FLOOR = 0.82
RC2C_HARD_CLASS2_FLOOR = 0.80
RC2C_HARD_MIN_TASK_FLOOR = 0.80
RC2C_HARD_MIN_CLASS_FLOOR = 0.80
RC2C_CLASS2_FLOOR_PENALTY = 5.00
RC2C_MIN_CLASS_FLOOR_PENALTY = 4.00
RC2C_MIN_TASK_FLOOR_PENALTY = 4.00

```

```

RC2C_OLD_TASK_FLOOR_PENALTY = 3.00
RC2C_TASK_DISPERSION_PENALTY = 0.25
RC2C_NO_REPAIR_VETO_AFTER_TASK = 2
RC2C_NO_REPAIR_REQUIRED_MARGIN = 0.015
RC2C_NO_REPAIR_SOFT_PENALTY = 0.010
RC2C_GUARDED_HYSTERESIS_MARGIN = 0.003
RC2C_GUARDED_CONTEXT_BONUS = 0.001
RC2C_FLOOR_RISK_FALLBACK_ENABLED = True
RC2C_GUARDED_REFERENCE_VARIANTS = [
    "context_temp_blend_selected_head_guard_035",
    "context_gap_selected",
    "generic_weak_boundary_latent_calibrated",
]

# RC2d gate-alignment selector controls.
# These are validation-memory-only controls; they do not touch final test data.
RC2D_SELECTOR_VERSION = "RC2d_gate_aligned_v1"
RC2D_GATE_ALIGNMENT_ENABLED = True
RC2D_AUX_PROXY_WEIGHT = 0.35          # auxiliary validation split score to
    ↪reduce one-split noise
RC2D_PRIMARY_PROXY_WEIGHT = 0.65
RC2D_PROXY_ACC_FLOOR = 0.68          # smoke guard; robust runs should exceed
    ↪this comfortably
RC2D_PROXY_ACC_FLOOR_PENALTY = 3.00
RC2D_CLASS5_FLOOR = 0.90
RC2D_HARD_CLASS5_FLOOR = 0.80
RC2D_CLASS5_FLOOR_PENALTY = 5.00
RC2D_CLASS2_RESCUE_MARGIN = 0.08
RC2D_CLASS5_RESCUE_MARGIN = 0.08
RC2D_TRUE_GUARD_RESCUE_SCORE_MARGIN = 0.020
RC2D_TRUE_GUARD_CLASS2_MARGIN = 0.05
RC2D_TRUE_GUARD_ACC_MARGIN = 0.010
RC2D_TRUE_GUARD_BONUS = 0.003
RC2D_CONTEXT_GUARD_BONUS = 0.001     # lower than RC2c to avoid
    ↪over-conservative context-gap selection
RC2D_GUARDED_HYSTERESIS_MARGIN = 0.003
RC2D_SELECTOR_FINAL_ALIGNMENT_TOL = 0.005
RC2D_CLASS_BALANCE_WEIGHT = 0.35
RC2D_MIN_TASK_WEIGHT = 0.35
RC2D_OLD_TASK_WEIGHT = 0.20
RC2D_TASK_STD_WEIGHT = 0.25

# RC2g NaN-safe / class-balanced selector controls.
# These are validation-memory / repair-memory-only controls.
RC2G_SELECTOR_VERSION = "RC2g_nan_safe_class_balanced_v1"
RC2G_NAN_SAFE_SCORING_ENABLED = True
RC2G_CLASS_AVAILABILITY_GATING_ENABLED = True

```

```

RC2G_EMERGENCY_SCORE_IF_ALL_NAN = True
RC2G_CLASS_BALANCED_READOUT_CALIBRATION_ENABLED = True
RC2G_CLASS_BALANCED_CALIB_STEPS = 12 if RUN_MODE == "smoke" else 30
RC2G_CLASS_BALANCED_CALIB_LR = 1.5e-3
RC2G_WEAK_CLASS_WEIGHT = 2.50
RC2G_MAX_CLASS_WEIGHT = 4.00
RC2G_CLASS_BALANCED_MARGIN_WEIGHT = 0.35
RC2G_SCORE_NEUTRAL_VALUE = 0.0

# RC2g tri-boundary equilibrium readout controls.
# These still use repair/policy-validation memory only.
RC2G_SELECTOR_VERSION = "RC2g_tri_boundary_class_balanced_v1"
RC2G_BOUNDARY_CLASSES = [2, 3, 4, 5]
RC2G_PROTECTED_CLASSES = [2, 4, 5]
RC2G_CLASS4_FLOOR = 0.45
RC2G_HARD_CLASS4_FLOOR = 0.05
RC2G_CLASS4_FLOOR_PENALTY = 6.00
RC2G_ZERO_SEEN_CLASS_REJECT_ENABLED = True
RC2G_ZERO_SEEN_CLASS_PENALTY = 4.00
RC2G_ZERO_CLASS_RESCUE_SCORE_MARGIN = 0.08
RC2G_BOUNDARY_MARGIN_TARGET = 0.20
RC2G_BOUNDARY_MARGIN_WEIGHT = 0.55
RC2G_CLASS4_WEIGHT = 3.50
RC2G_BOUNDARY_CLASS_WEIGHT = 2.75
RC2G_BOUNDARY_CALIB_STEPS = 18 if RUN_MODE == "smoke" else 40
RC2G_BOUNDARY_CALIB_LR = 1.2e-3

# Hydro audit/selection controls inherited from the v1.1 Hydro suite.
# Defaults are set through EXPERIMENT_PROFILE above.
RC3_HYDRO_AUDIT_ENABLED = bool(ACTIVATE_HYDRO_AUDIT)
RC3_HYDRO_REGULARIZER_ENABLED = bool(ACTIVATE_HYDRO_REGULARIZER)
RC3_HYDRO_SELECTION_ENABLED = bool(ACTIVATE_HYDRO_SELECTION)
RC3_HYDRO_REGULARIZER_MODE = HYDRO_REGULARIZER_MODE
RC3_HYDRO_POLICY_MODE = HYDRO_POLICY_MODE # "off", "audit_only", or
↳ "stability_aware"
RC3_HYDRO_SELECTION_TURBULENCE_WEIGHT = HYDRO_SELECTION_TURBULENCE_WEIGHT
RC3_HYDRO_SELECTION_LATENT_DRIFT_WEIGHT = HYDRO_SELECTION_LATENT_DRIFT_WEIGHT
RC3_HYDRO_SELECTION_ROUTE_DRIFT_WEIGHT = HYDRO_SELECTION_ROUTE_DRIFT_WEIGHT
RC3_HYDRO_REGULARIZER_COEFF = HYDRO_REGULARIZER_COEFF
RC3_HYDRO_HIGH_GAIN_IMPORTANCE_THRESHOLD = HYDRO_HIGH_GAIN_IMPORTANCE_THRESHOLD
RC3_HYDRO_BASE_VISCOSITY = 0.10
RC3_HYDRO_MAX_VISCOSITY = 2.00
RC3_HYDRO_TURBULENCE_EPS = 1e-8
RC3_HYDRO_MAX_ITEMS_PER_CLASS = VALIDATION_MAX_ITEMS_PER_CLASS
RC3_HYDRO_MECHANISTIC_TURBULENCE_WARNING = 0.20
RC3_HYDRO_MECHANISTIC_LATENT_WARNING = 0.05

```

```

RC2B_CANDIDATE_VARIANTS = [
    "proto_global_no_repair",
    "context_blend_selected_global",
    "context_temp_blend_selected_global",
    "proto_global_head_ce_kl_guard_035",
    "context_temp_blend_selected_head_guard_035",
    "context_gap_selected",
    "generic_weak_boundary_latent_calibrated",
]

# Patch missing inherited RC1b context-selection constants.
# Run this once, then rerun the failed "Run experiment" cell.

CONTEXT_TEMP_SELECTION_GRID = [0.35, 0.60, 0.80, 1.00, 1.25, 1.50]
CONTEXT_BLEND_SELECTION_GRID = [
    ("base_proto_first", 0.05, 0.75, 0.20),
    ("latent_only", 0.00, 1.00, 0.00),
    ("feature_only", 0.00, 0.00, 1.00),
    ("latent_dominant", 0.05, 0.90, 0.05),
    ("single_mid", 0.30, 0.55, 0.15),
    ("balanced_all", 0.34, 0.33, 0.33),
]

CONTEXT_SELECTION_SCORE_CONTEXT_WEIGHT = 0.20
CONTEXT_SELECTION_SCORE_CLASS2_WEIGHT = 0.10
CONTEXT_SELECTION_SCORE_MINCLASS_WEIGHT = 0.10
CONTEXT_SELECTION_SCORE_ENTROPY_PENALTY = 0.02

# Backward-compatible names used by inherited v0.14.x helper functions.
HEAD_REFRESH_EPOCHS = READOUT_TRAIN_EPOCHS
HEAD_REFRESH_STEPS = READOUT_TRAIN_STEPS
HEAD_REFRESH_LR = READOUT_LR

# Optional old helper compatibility constants.
PROBE_DISTILL_EPOCHS = 2
PROBE_DISTILL_LR = 2e-3
BIAS_CALIB_STEPS = 8
BIAS_CALIB_LR = 5e-2
PROBE_DISTILL_ALPHA_CE = 0.50
PROBE_DISTILL_ALPHA_KL = 0.50
BIAS_CALIB_REG = 1e-3

LOCAL_ADAPTER_KL_GRID = [0.05, 0.15, 0.35, 0.60]
LOCAL_ADAPTER_MARGIN_WEIGHT = HEAD_MARGIN_LAMBDA

SELECTED_LOCAL_HEAD_VARIANTS = [
    "no_repair_global_head",

```



```

    "kl_only_true_noop",
    "global_head_ce_kl_guard_035",
    "local_proto_cosine",
    "local_proto_euclidean",
    "local_trainable_proto_ce_kl_035",
    "local_diag_adapter_ce_only",
    "local_diag_adapter_ce_kl_005",
    "local_diag_adapter_ce_kl_015",
    "local_diag_adapter_ce_kl_035",
    "local_diag_adapter_ce_kl_060",
    "local_diag_adapter_ce_kl_margin_035",
    "local_adapter_validation_selected",
]

# Keep the historical variants for control rows and final candidate comparison.
SELECTED_CONTEXT_HEAD_VARIANTS = [
    "proto_global_no_repair",
    "proto_kl_only_true_noop",
    "proto_global_head_ce_kl_guard_035",
    "oracle_global_no_repair_reference",
    "oracle_global_head_ce_kl_guard_035_reference",
    "context_blend_selected_global",
    "context_temp_blend_selected_global",
    "context_blend_selected_head_guard_035",
    "context_temp_blend_selected_head_guard_035",
    "context_gap_selected",
    "generic_weak_boundary_latent_calibrated",
]

# Comparative baselines for the benchmark harness.
if RUN_MODE == "smoke":
    SELECTED_BASELINE_METHODS = [
        "finetune",
        "experience_replay",
        "balanced_replay",
        "joint_upper_bound",
    ]
else:
    SELECTED_BASELINE_METHODS = [
        "finetune",
        "ewc",
        "lwf",
        "experience_replay",
        "balanced_replay",
        "sparse_moe",
        "pnn",
        "joint_upper_bound",
    ]

```

```

]

EWC_LAMBDA = 25.0
LWF_LAMBDA = 1.0
BASELINE_REPLAY_WEIGHT = 1.0
SPARSE_MOE_ENTROPY_WEIGHT = 0.002

RUN_FINAL_PROBES = True
PROBE_CONTEXT_MODES = ["proto", "oracle"]
HOST_PROBE_MODE = "oracle"

# =====
# Biometric-style threshold diagnostics (diagnostic-only)
# =====
# These diagnostics are computed after no-leakage policy selection and
# ↪ final-test
# evaluation. They do not feed back into training, candidate construction, or
# ↪ selector decisions.
ENABLE_BIOMETRIC_STYLE_THRESHOLD_DIAGNOSTICS = True
THRESHOLD_DIAGNOSTICS_FINAL_CHECKPOINT_ONLY = True
THRESHOLD_DIAGNOSTICS_EVALUATE_CANDIDATES = True
THRESHOLD_DIAGNOSTICS_FOCUS_CLASSES = [2, 4, 5]
THRESHOLD_DIAGNOSTICS_GRID_SIZE = 501
THRESHOLD_DIAGNOSTICS_SCORE_KIND = "softmax_probability_over_seen_classes"
THRESHOLD_DIAGNOSTICS_STRICT_EXPORT = True

# =====
# Panel E - Learning-Signal Alignment audit (diagnostic-only)
# =====
# LSA compares expected update allocation across hosts with observed
# host/transform gradient norm allocation during training. It is never used
# for policy selection or final-test evaluation.
ENABLE_LEARNING_SIGNAL_ALIGNMENT_AUDIT = True
LSA_BATCH_STRIDE = 1
LSA_EXPECTED_ROUTE_WEIGHT = 0.55
LSA_EXPECTED_GAIN_WEIGHT = 0.25
LSA_EXPECTED_MEMORY_RISK_WEIGHT = 0.20
LSA_LOW_EXPECTED_QUANTILE = 0.35
LSA_HIGH_OBSERVED_QUANTILE = 0.65

# =====
# RotatedMNIST final-rotation diagnostic gate (diagnostic-only)
# =====
# This gate attributes class-5 / final-rotation weakness to route/context,
# readout boundary, or latent separability. It does not alter training,

```

```

# policy selection, Hydro, LSA, or final-test evaluation.
ENABLE_ROTATEDMNIST_FINAL_ROTATION_DIAGNOSTIC = True
ROTATED_DIAG_FINAL_TASK_ID = N_TASKS - 1
ROTATED_DIAG_CLASS5_FLOOR = 0.90
ROTATED_DIAG_MIN_TASK_FLOOR = 0.80
ROTATED_DIAG_CONTEXT_TOP1_FLOOR = 0.70
ROTATED_DIAG_CONTEXT_ENTROPY_CEILING = 1.20
ROTATED_DIAG_CONTEXT_MARGIN_FLOOR = 0.05
ROTATED_DIAG_CLASS5_TO_CLASS4_CEILING = 0.25
ROTATED_DIAG_PROBE_CLASS5_FLOOR = 0.85
ROTATED_DIAG_DEPLOYED_TO_PROBE_CLASS5_GAP = 0.12
ROTATED_DIAG_ORACLE_PROTO_CLASS5_GAP = 0.08

# =====
# RC2JG - Generic weak-boundary latent separability calibration
# =====
# Candidate-only, validation-memory selected, no final-test feedback.
ENABLE_RC2JG_GENERIC_WEAK_BOUNDARY_LATENT_CALIBRATION = str(EXPERIMENT_PROFILE).
    ↳startswith("RC2JG_")
RC2JG_VARIANT = "generic_weak_boundary_latent_calibrated"
RC2JG_ADAPTER_SCALE = 0.25
RC2JG_WEAK_CLASS_ACC_FLOOR = 0.78 if RUN_MODE == "smoke" else 0.86
RC2JG_MIN_BOUNDARY_MARGIN = 0.12 if RUN_MODE == "smoke" else 0.20
RC2JG_LATENT_SEPARATION_FLOOR = 0.60 if RUN_MODE == "smoke" else 0.75
RC2JG_CONTEXT_TOP1_FLOOR = 0.70 if RUN_MODE == "smoke" else 0.90
RC2JG_CONTEXT_ENTROPY_CEILING = 1.20 if RUN_MODE == "smoke" else 0.60
RC2JG_CONTEXT_MARGIN_FLOOR = 0.03 if RUN_MODE == "smoke" else 0.08
RC2JG_MIN_WEAK_CLASS_GAIN = 0.010
RC2JG_MIN_MINCLASS_GAIN = 0.000
RC2JG_MAX_GLOBAL_ACC_DAMAGE = 0.003 if RUN_MODE != "smoke" else 0.015
RC2JG_MIN_SCORE_GAIN = 0.000
RC2JG_KL_WEIGHT = 0.35
RC2JG_BOUNDARY_MARGIN_WEIGHT = 0.45
RC2JG_LATENT_SEPARATION_WEIGHT = 0.25
RC2JG_LATENT_ANCHOR_WEIGHT = 0.10
RC2JG_REPAIR_EPOCHS = max(1, READOUT_TRAIN_EPOCHS)
RC2JG_REPAIR_STEPS = max(8 if RUN_MODE == "smoke" else 30, READOUT_TRAIN_STEPS)
RC2JG_REPAIR_LR = READOUT_LR

COMMON_MLP = dict(
    model_family="mlp",
    strategy="none",
    balanced_replay_ce=False,
    balanced_replay_weight=0.0,
    output_distill=False,
    route=0.0,
    latent=0.0,

```

```

    output=0.0,
    context_calibrated=False,
    context_replay=False,
    temp_calibration=False,
    context_replay_scale=0.0,
    head_margin_repair=False,
    post_head_refresh=False,
    post_probe_distill=False,
    post_bias_calibration=False,
)
COMMON_MMALS = dict(
    model_family="mmals",
    route=0.5,
    latent=1.0,
    output=1.0,
    output_distill=True,
    balanced_replay_ce=True,
    balanced_replay_weight=BALANCED_REPLAY_WEIGHT,
    context_calibrated=False,
    context_replay=False,
    temp_calibration=False,
    context_replay_scale=0.0,
    batch_blend_single=PROTO_FIRST_BLEND_SINGLE,
    batch_blend_latent=PROTO_FIRST_BLEND_LATENT,
    batch_blend_feature=PROTO_FIRST_BLEND_FEATURE,
    head_margin_repair=False,
    post_head_refresh=False,
    post_probe_distill=False,
    post_bias_calibration=False,
)
COMMON_PROTO_FIRST = {
    **COMMON_MMALS,
    "strategy": "proto_first",
    "context_calibrated": True,
    "context_replay": True,
    "temp_calibration": True,
    "context_replay_scale": 1.0,
    "route": 0.5,
    "latent": 1.0,
    "balanced_replay_ce": True,
    "balanced_replay_weight": BALANCED_REPLAY_WEIGHT,
    "output_distill": True,
    "output": 1.0,
}
METHOD_CONFIGS = {
    "mlp_finetune": {**COMMON_MLP},
    "mlp_ewc": {**COMMON_MLP},

```

```

"mlp_lwf": {**COMMON_MLP},
"mlp_experience_replay": {**COMMON_MLP},
"mlp_balanced_replay": {**COMMON_MLP},
"sparse_moe": {**COMMON_MLP, "model_family": "sparse_moe"},
"pnn": {**COMMON_MLP, "model_family": "pnn"},
"mlp_joint_upper_bound": {**COMMON_MLP},
"mmals_balanced_replay": {
    **COMMON_MMALS,
    "strategy": "task_emb",
    "balanced_replay_ce": True,
    "balanced_replay_weight": BALANCED_REPLAY_WEIGHT,
    "output_distill": True,
    "output": 1.0,
},
"mmals_uniform_balanced_replay": {
    **COMMON_MMALS,
    "strategy": "uniform",
},
"mmals_proto_first_balanced_replay": {
    **COMMON_PROTO_FIRST,
},
"mmals_oracle_context_balanced_replay": {
    **COMMON_MMALS,
    "strategy": "oracle",
},
}

# =====
# RC2M-beta dynamic-boundary + ambiguity-lock selector override
# =====
# This MUST stay after inherited RC2J/RC2JG configuration, candidate-pool
# constants, Hydro constants, and RC2JG flags. It reuses the RC2JG candidate
# pool while deploying the RC2M-beta selector.
ENABLE_RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK = str(EXPERIMENT_PROFILE).
↳startswith("RC2M_BETA")
# Backward-compatible alias so older helper/audit code that checks the alpha↳
↳flag
# still sees an active boundary selector, but beta owns the final policy↳
↳decision.
ENABLE_RC2M_ALPHA_BOUNDARY_FIRST_SELECTOR = False

if ENABLE_RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK:
    POLICY_BRANCH = "RC2m_beta"

    ACTIVATE_RC2C_RISK_AWARE_SELECTION = True

```

```

ACTIVATE_HYDRO_AUDIT = True
ACTIVATE_HYDRO_REGULARIZER = False
ACTIVATE_HYDRO_SELECTION = False

HYDRO_REGULARIZER_MODE = "off"
HYDRO_POLICY_MODE = "audit_only"
RC3_HYDRO_POLICY_MODE = "audit_only"

RC2C_RISK_AWARE_SELECTOR_ENABLED = True
RC3_HYDRO_AUDIT_ENABLED = True
RC3_HYDRO_REGULARIZER_ENABLED = False
RC3_HYDRO_SELECTION_ENABLED = False
RC3_HYDRO_REGULARIZER_MODE = "off"
RC3_HYDRO_SELECTION_TURBULENCE_WEIGHT = 0.0
RC3_HYDRO_SELECTION_LATENT_DRIFT_WEIGHT = 0.0
RC3_HYDRO_SELECTION_ROUTE_DRIFT_WEIGHT = 0.0

# Keep RC2JG candidate construction available as one candidate family.
ENABLE_RC2JG_GENERIC_WEAK_BOUNDARY_LATENT_CALIBRATION = True

RC2C_METHOD = "paired_rc2m_beta_dynamic_boundary_ambiguity_lock_policy"
RC3_METHOD = RC2C_METHOD
RC2B_METHOD = RC2C_METHOD

# Dynamic, dataset-agnostic selection features. No class 2/4/5 selector_
↪ feature.
RC2M_BETA_FROZEN_WEIGHTS = {
    "macro_auc": 0.5,
    "weak_boundary_auc_dynamic": 0.5,
    "macro_eer": 0.0,
    "micro_eer": 2.0,
    "weak_boundary_eer_dynamic": 0.0,
    "worst_class_eer": 0.0,
}
RC2M_BETA_HIGH_FEATURES = ["macro_auc", "weak_boundary_auc_dynamic",
↪ "topk_boundary_auc"]
RC2M_BETA_LOW_FEATURES = ["macro_eer", "micro_eer",
↪ "weak_boundary_eer_dynamic", "worst_class_eer", "topk_boundary_eer"]
RC2M_BETA_MIN_REQUIRED_FEATURES = 2
RC2M_BETA_VERSION = "rc2m_beta_dynamic_boundary_ambiguity_lock_v1"

# Dynamic weak-boundary extraction. For MNIST-family this often yields 2/4/
↪ 5,
# but for future datasets it is detected from validation memory.
RC2M_BETA_WEAK_BOUNDARY_TOPK_ABS = 3
RC2M_BETA_WEAK_BOUNDARY_TOPK_FRACTION = 0.25
RC2M_BETA_WEAK_BOUNDARY_TOPK_MAX = 10

```

```

RC2M_BETA_MIN_CLASS_SUPPORT = 4 if RUN_MODE == "smoke" else 12

# Ambiguity lock: protect context-gap family unless challenger is clearly
↪safer.
RC2M_BETA_AMBIGUITY_LOCK_ENABLED = True
RC2M_BETA_GUARDED_CONTEXT_VARIANTS = ["context_gap_selected",
↪"context_temp_blend_selected_head_guard_035"]
RC2M_BETA_AMBIGUITY_MIN_AFTER_TASK = 2
RC2M_BETA_AMBIGUITY_SCORE_MARGIN = 0.08
RC2M_BETA_AMBIGUITY_BOUNDARY_AUC_GAIN = 0.005
RC2M_BETA_AMBIGUITY_BOUNDARY_EER_GAIN = 0.005
RC2M_BETA_NO_REGRESSION_TOL = 0.005
RC2M_BETA_HYDRO_TURBULENCE_TOL = 0.03
RC2M_BETA_HYDRO_OUTPUT_DRIFT_TOL = 0.03
RC2M_BETA_LSA_TOL = 0.02
RC2M_BETA_ENERGY_GAP_TOL = 0.05

print("EXPERIMENT_PROFILE:", EXPERIMENT_PROFILE)
print("POLICY_BRANCH:", POLICY_BRANCH)
print("ACTIVATE_RC2C_RISK_AWARE_SELECTION:", ACTIVATE_RC2C_RISK_AWARE_SELECTION)
print("ACTIVATE_HYDRO_AUDIT:", ACTIVATE_HYDRO_AUDIT)
print("ACTIVATE_HYDRO_REGULARIZER:", ACTIVATE_HYDRO_REGULARIZER, "mode=",
↪HYDRO_REGULARIZER_MODE)
print("ACTIVATE_HYDRO_SELECTION:", ACTIVATE_HYDRO_SELECTION, "mode=",
↪HYDRO_POLICY_MODE)
print("RUN_MODE:", RUN_MODE)
print("DATASET_NAME:", DATASET_NAME)
print("SEEDS:", SEEDS)
print("RC2C_METHOD:", RC2C_METHOD)
print("ENABLE_RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK:",
↪ENABLE_RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK)
if ENABLE_RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK:
    print("RC2M_BETA_VERSION:", RC2M_BETA_VERSION)
    print("RC2M_BETA_FROZEN_WEIGHTS:", RC2M_BETA_FROZEN_WEIGHTS)
    print("RC2M_BETA_HIGH_FEATURES:", RC2M_BETA_HIGH_FEATURES)
    print("RC2M_BETA_LOW_FEATURES:", RC2M_BETA_LOW_FEATURES)
    print("RC2M_BETA_DYNAMIC_BOUNDARY_TOPK:", RC2M_BETA_WEAK_BOUNDARY_TOPK_ABS,
↪RC2M_BETA_WEAK_BOUNDARY_TOPK_FRACTION, RC2M_BETA_WEAK_BOUNDARY_TOPK_MAX)
    print("RC2M_BETA_AMBIGUITY_LOCK_ENABLED:", RC2M_BETA_AMBIGUITY_LOCK_ENABLED)
print("RC3_HYDRO_POLICY_MODE:", RC3_HYDRO_POLICY_MODE)
print("RC2B_CANDIDATE_VARIANTS / RC3 candidate pool:", RC2B_CANDIDATE_VARIANTS)
print("SELECTED_BASELINE_METHODS:", SELECTED_BASELINE_METHODS)
print("v1.1-RC2M-beta selector harness: RC2JG candidate construction is
↪retained, but final deployment is selected by dynamic weak-boundary evidence
↪plus ambiguity-lock protection when
↪ENABLE_RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK=True.")

```

```

print("ROTATION_ANGLES:", ROTATION_ANGLES)
print("PERMUTATION_SEEDS:", PERMUTATION_SEEDS)

print("ENABLE_BIOMETRIC_STYLE_THRESHOLD_DIAGNOSTICS:", □
    ↳ENABLE_BIOMETRIC_STYLE_THRESHOLD_DIAGNOSTICS)
print("THRESHOLD_DIAGNOSTICS_EVALUATE_CANDIDATES:", □
    ↳THRESHOLD_DIAGNOSTICS_EVALUATE_CANDIDATES)
print("THRESHOLD_DIAGNOSTICS_FOCUS_CLASSES:", □
    ↳THRESHOLD_DIAGNOSTICS_FOCUS_CLASSES)
print("ENABLE_LEARNING_SIGNAL_ALIGNMENT_AUDIT:", □
    ↳ENABLE_LEARNING_SIGNAL_ALIGNMENT_AUDIT)
print("LSA_BATCH_STRIDE:", LSA_BATCH_STRIDE)
print("ENABLE_ROTATEDMNIST_FINAL_ROTATION_DIAGNOSTIC:", □
    ↳ENABLE_ROTATEDMNIST_FINAL_ROTATION_DIAGNOSTIC)
print("ENABLE_RC2JG_GENERIC_WEAK_BOUNDARY_LATENT_CALIBRATION:", □
    ↳ENABLE_RC2JG_GENERIC_WEAK_BOUNDARY_LATENT_CALIBRATION)

```

```

EXPERIMENT_PROFILE: RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT
POLICY_BRANCH: RC2m_beta
ACTIVATE_RC2C_RISK_AWARE_SELECTION: True
ACTIVATE_HYDRO_AUDIT: True
ACTIVATE_HYDRO_REGULARIZER: False mode= off
ACTIVATE_HYDRO_SELECTION: False mode= audit_only
RUN_MODE: robust
DATASET_NAME: FashionMNIST
SEEDS: [0, 1, 2, 3, 4]
RC2C_METHOD: paired_rc2m_beta_dynamic_boundary_ambiguity_lock_policy
ENABLE_RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK: True
RC2M_BETA_VERSION: rc2m_beta_dynamic_boundary_ambiguity_lock_v1
RC2M_BETA_FROZEN_WEIGHTS: {'macro_auc': 0.5, 'weak_boundary_auc_dynamic': 0.5,
'macro_eer': 0.0, 'micro_eer': 2.0, 'weak_boundary_eer_dynamic': 0.0,
'worst_class_eer': 0.0}
RC2M_BETA_HIGH_FEATURES: ['macro_auc', 'weak_boundary_auc_dynamic',
'topk_boundary_auc']
RC2M_BETA_LOW_FEATURES: ['macro_eer', 'micro_eer', 'weak_boundary_eer_dynamic',
'worst_class_eer', 'topk_boundary_eer']
RC2M_BETA_DYNAMIC_BOUNDARY_TOPK: 3 0.25 10
RC2M_BETA_AMBIGUITY_LOCK_ENABLED: True
RC3_HYDRO_POLICY_MODE: audit_only
RC2B_CANDIDATE_VARIANTS / RC3 candidate pool: ['proto_global_no_repair',
'context_blend_selected_global', 'context_temp_blend_selected_global',
'proto_global_head_ce_kl_guard_035',
'context_temp_blend_selected_head_guard_035', 'context_gap_selected',
'generic_weak_boundary_latent_calibrated']
SELECTED_BASELINE_METHODS: ['finetune', 'ewc', 'lwf', 'experience_replay',
'balanced_replay', 'sparse_moe', 'pnn', 'joint_upper_bound']
v1.1-RC2M-beta selector harness: RC2JG candidate construction is retained, but

```


final deployment is selected by dynamic weak-boundary evidence plus ambiguity-lock protection when `ENABLE_RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK=True`.
`ROTATION_ANGLES`: [-30, -15, 0, 15, 30]
`PERMUTATION_SEEDS`: [100, 101, 102, 103, 104]
`ENABLE_BIOMETRIC_STYLE_THRESHOLD_DIAGNOSTICS`: True
`THRESHOLD_DIAGNOSTICS_EVALUATE_CANDIDATES`: True
`THRESHOLD_DIAGNOSTICS_FOCUS_CLASSES`: [2, 4, 5]
`ENABLE_LEARNING_SIGNAL_ALIGNMENT_AUDIT`: True
`LSA_BATCH_STRIDE`: 1
`ENABLE_ROTATEDMNIST_FINAL_ROTATION_DIAGNOSTIC`: True
`ENABLE_RC2JG_GENERIC_WEAK_BOUNDARY_LATENT_CALIBRATION`: True

2.12 2a. How to activate or deactivate RC2Je / Hydro

Use `EXPERIMENT_PROFILE` in the configuration cell.

Recommended default:

`EXPERIMENT_PROFILE = "RC2JE_HARD_CLARITY_CLEAN_VETO_GUARD_HYDRO_AUDIT"`

Available RC2Je profiles:

1. `RC2B_STRICT`
Reproduces the older RC2b strict selector. Hydro disabled. RC2Je selector disabled.
2. `RC2JE_HARD_CLARITY_CLEAN_VETO_GUARD_NO_HYDRO`
RC2Je regime-aware selector enabled. Hydro disabled. Use this to isolate selector effects.
3. `RC2JE_HARD_CLARITY_CLEAN_VETO_GUARD_HYDRO_AUDIT`
Default. RC2Je regime-aware selector enabled. Hydro metrics logged as audit only.
4. `RC2JE_HARD_CLARITY_CLEAN_VETO_GUARD_HYDRO_STABILITY_ABLATION`
Experimental. RC2Je selector enabled and Hydro drift/turbulence penalty added to validation selection. Report separately.

Default scientific interpretation:

RC2Je first classifies the checkpoint as clean-context, ambiguous-context or mixed/uncertain using validation-memory diagnostics. It then applies RC2Jb late-route caution and, at the final overlap-chain checkpoint, applies a final ambiguity lock if unresolved ambiguity remains and context-gap is not catastrophically unsafe. Hydro remains an observability/audit contribution unless the Hydro stability ablation is explicitly selected.

2.13 2b. Persistence and OTEL-style traceability layer

This cell creates a persistent artifact directory. In Colab it can mount Google Drive and write results there; outside Colab it falls back to a local directory.

The trace format is OTEL-style, not a full external OTLP export by default. It writes JSONL events and span-like rows that can later be ingested by Jaeger, Grafana Tempo, Datadog, OpenSearch, or an internal STRAT-Q observability pipeline.

RC2JF Drive archive path when `PERSIST_TO_GOOGLE_DRIVE = True`:

/content/drive/MyDrive/MMALS/v1_1_rc2jf_true_guard_safe_veto_selector/<EXPERIMENT_PROFILE>/<RUN_ID>

```
[51]: # -----  
# v1.1-RC2JG Generic Weak-Boundary Latent Calibration Harness  
# -----  
# Set to False if you do not want Colab to mount Google Drive.  
PERSIST_TO_GOOGLE_DRIVE = True  
GOOGLE_DRIVE_PROJECT_DIR = f"MMALS/  
    ↪v1_1_rc2m_alpha_boundary_first_frozen_selector/{EXPERIMENT_PROFILE}"  
  
RUN_TIMESTAMP_UTC = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%SZ")  
RUN_ID =  
    ↪f"MMALS_v11_{EXPERIMENT_PROFILE}_{DATASET_NAME}_{RUN_MODE}_{RUN_TIMESTAMP_UTC}_seeds{len(SEEDS)}"  
TRACE_ID = hashlib.sha256(RUN_ID.encode("utf-8")).hexdigest()[:16]  
  
# Notebook/PDF evidence-export names. The helper cell later stores a  
    ↪run-summary PDF  
# and optionally a full notebook PDF in RESULTS_DIR/notebook_exports.  
NOTEBOOK_BASENAME = globals().get("NOTEBOOK_BASENAME",  
    ↪f"MMALS_Prototype_1_1_RC2M_alpha_Boundary_First_Frozen_Selector_Benchmark_Harness")  
NOTEBOOK_FILENAME = globals().get("NOTEBOOK_FILENAME", f"{NOTEBOOK_BASENAME}.  
    ↪ipynb")  
  
DRIVE_AVAILABLE = False  
if PERSIST_TO_GOOGLE_DRIVE:  
    try:  
        from google.colab import drive # type: ignore  
        drive.mount("/content/drive", force_remount=False)  
        DRIVE_AVAILABLE = True  
    except Exception as e:  
        print("Google Drive mount skipped; falling back to local persistence.")  
        print("Reason:", repr(e))  
  
if DRIVE_AVAILABLE:  
    PERSISTENCE_ROOT = Path("/content/drive/MyDrive") / GOOGLE_DRIVE_PROJECT_DIR  
else:  
    PERSISTENCE_ROOT = Path("./  
    ↪mmals_v11_rc2jf_true_guard_safe_veto_selector_results")  
  
RESULTS_DIR = PERSISTENCE_ROOT / RUN_ID  
TRACE_DIR = RESULTS_DIR / "traces"  
AUDIT_DIR = RESULTS_DIR / "audit"  
PLOTS_DIR = RESULTS_DIR / "plots"  
RAW_LOG_DIR = RESULTS_DIR / "raw_logs"  
NOTEBOOK_EXPORT_DIR = RESULTS_DIR / "notebook_exports"  
TRUSTED_EVIDENCE_DIR = RESULTS_DIR / "TrustedEvidence"
```

```

for d in [RESULTS_DIR, TRUSTED_EVIDENCE_DIR, TRACE_DIR, AUDIT_DIR, PLOTS_DIR,
↳RAW_LOG_DIR, NOTEBOOK_EXPORT_DIR]:
    d.mkdir(parents=True, exist_ok=True)

def ensure_persistence_dirs():
    """Re-create Drive/local artifact directories if Colab/Drive sync
↳temporarily drops them.

    This is intentionally called before every file write because Google Drive
↳mounted
    folders can disappear after remounts, manual moves, or transient sync
↳issues.
    It does not change RUN_ID or any scientific result.
    """
    for d in [RESULTS_DIR, TRUSTED_EVIDENCE_DIR, TRACE_DIR, AUDIT_DIR,
↳PLOTS_DIR, RAW_LOG_DIR, NOTEBOOK_EXPORT_DIR]:
        Path(d).mkdir(parents=True, exist_ok=True)

TRACE_JSONL_PATH = TRACE_DIR / f"otel_style_events_{RUN_MODE}.jsonl"
TRACE_SPANS_PATH = TRACE_DIR / f"otel_style_spans_{RUN_MODE}.csv"
RAW_CONSOLE_LOG_PATH = RAW_LOG_DIR / f"console_{RUN_MODE}.log"
MANIFEST_PATH = RESULTS_DIR / f"run_manifest_{RUN_MODE}.json"

TRACE_EVENTS = []
TRACE_SPANS = []
_ORIGINAL_STDOUT = sys.stdout
_ORIGINAL_STDERR = sys.stderr
_LOG_FILE_HANDLE = None
_LOGGING_ACTIVE = False

def safe_jsonable(obj):
    """Convert numpy/torch/Pandas objects to JSON-safe values without dumping
↳large tensors."""
    if obj is None:
        return None
    if isinstance(obj, (str, int, float, bool)):
        if isinstance(obj, float) and (math.isnan(obj) or math.isinf(obj)):
            return None
        return obj
    if isinstance(obj, (np.integer,)):
        return int(obj)
    if isinstance(obj, (np.floating,)):
        v = float(obj)
        return None if math.isnan(v) or math.isinf(v) else v
    if isinstance(obj, (np.ndarray,)):
        if obj.size <= 16:

```

```

        return obj.tolist()
    return {"type": "ndarray", "shape": list(obj.shape), "mean": float(np.
↪nanmean(obj)), "std": float(np.nanstd(obj))}
    if torch.is_tensor(obj):
        t = obj.detach().cpu()
        if t.numel() == 1:
            return safe_jsonable(t.item())
        if t.numel() <= 16:
            return t.tolist()
        tf = t.float()
        return {"type": "tensor", "shape": list(t.shape), "mean": float(tf.
↪mean()), "std": float(tf.std())}
    if isinstance(obj, dict):
        return {str(k): safe_jsonable(v) for k, v in obj.items()}
    if isinstance(obj, (list, tuple)):
        return [safe_jsonable(x) for x in obj]
    return str(obj)

def config_snapshot():
    keys = [
        "RUN_MODE", "DATASET_NAME", "PAIR_MODE", "N_TASKS", "N_CONTEXTS", ↪
↪"N_HOSTS", "N_CLASSES",
        "TRAIN_PER_CLASS", "TEST_PER_CLASS", "SEEDS", "EPOCHS_PER_TASK", ↪
↪"MEMORY_PER_TASK", "MEMORY_PER_CLASS",
        "LABEL_NOISE_RATE", "INPUT_NOISE_STD", "CONTEXT_NOISE_RATE", ↪
↪"LATENT_DIM", "HIDDEN_DIM",
        "READOUT_TRAIN_EPOCHS", "READOUT_TRAIN_STEPS", ↪
↪"CONTEXT_TEMP_SELECTION_GRID", "CONTEXT_BLEND_SELECTION_GRID",
        "SELECTED_CONTEXT_HEAD_VARIANTS", "SELECTED_BASELINE_METHODS", ↪
↪"RC2B_CANDIDATE_VARIANTS", "RC2B_METHOD", "RC3_METHOD", ↪
↪"RC3_HYDRO_AUDIT_ENABLED", "RC3_HYDRO_POLICY_MODE", ↪
↪"RC3_HYDRO_SELECTION_TURBULENCE_WEIGHT", ↪
↪"RC3_HYDRO_SELECTION_LATENT_DRIFT_WEIGHT", ↪
↪"RC2B_STRICT_NO_FINAL_TEST_SELECTION", "NOTEBOOK_FILENAME", ↪
↪"ROTATION_ANGLES", "PERMUTATION_SEEDS",
        "ENABLE_BIOMETRIC_STYLE_THRESHOLD_DIAGNOSTICS", ↪
↪"THRESHOLD_DIAGNOSTICS_FINAL_CHECKPOINT_ONLY",
        "THRESHOLD_DIAGNOSTICS_EVALUATE_CANDIDATES", ↪
↪"THRESHOLD_DIAGNOSTICS_FOCUS_CLASSES",
        "THRESHOLD_DIAGNOSTICS_GRID_SIZE", "THRESHOLD_DIAGNOSTICS_SCORE_KIND", ↪
↪"THRESHOLD_DIAGNOSTICS_STRICT_EXPORT",
        "ENABLE_LEARNING_SIGNAL_ALIGNMENT_AUDIT", "LSA_BATCH_STRIDE",
        "LSA_EXPECTED_ROUTE_WEIGHT", "LSA_EXPECTED_GAIN_WEIGHT", ↪
↪"LSA_EXPECTED_MEMORY_RISK_WEIGHT",
        "ENABLE_ROTATEDMNIST_FINAL_ROTATION_DIAGNOSTIC", ↪
↪"ROTATED_DIAG_FINAL_TASK_ID",

```

```

        "ROTATED_DIAG_CLASS5_FLOOR", "ROTATED_DIAG_MIN_TASK_FLOOR",
        "ROTATED_DIAG_CONTEXT_TOP1_FLOOR",␣
↪ "ROTATED_DIAG_CONTEXT_ENTROPY_CEILING",
        "ROTATED_DIAG_CONTEXT_MARGIN_FLOOR",␣
↪ "ROTATED_DIAG_CLASS5_TO_CLASS4_CEILING",
        "ROTATED_DIAG_PROBE_CLASS5_FLOOR",␣
↪ "ROTATED_DIAG_DEPLOYED_TO_PROBE_CLASS5_GAP",
        "ROTATED_DIAG_ORACLE_PROTO_CLASS5_GAP",
        "ENABLE_RC2JG_GENERIC_WEAK_BOUNDARY_LATENT_CALIBRATION",␣
↪ "RC2JG_VARIANT",
        "RC2JG_WEAK_CLASS_ACC_FLOOR", "RC2JG_MIN_BOUNDARY_MARGIN",␣
↪ "RC2JG_LATENT_SEPARATION_FLOOR",
        "RC2JG_MAX_GLOBAL_ACC_DAMAGE", "RC2JG_MIN_WEAK_CLASS_GAIN",␣
↪ "RC2JG_REPAIR_EPOCHS", "RC2JG_REPAIR_STEPS",
    ]
    snap = {}
    for k in keys:
        if k in globals():
            snap[k] = safe_jsonable(globals()[k])
    return snap

def stable_hash(obj) -> str:
    payload = json.dumps(safe_jsonable(obj), sort_keys=True,␣
↪ ensure_ascii=False).encode("utf-8")
    return hashlib.sha256(payload).hexdigest()

CONFIG_HASH = stable_hash(config_snapshot())

def log_event(event_type: str, seed=None, task_id=None, method=None,␣
↪ variant=None, phase=None, payload=None, level="INFO"):
    event = {
        "trace_id": TRACE_ID,
        "run_id": RUN_ID,
        "timestamp_utc": datetime.now(timezone.utc).isoformat(),
        "level": level,
        "event_type": event_type,
        "dataset": DATASET_NAME,
        "run_mode": RUN_MODE,
        "seed": safe_jsonable(seed),
        "task_id": safe_jsonable(task_id),
        "method": safe_jsonable(method),
        "variant": safe_jsonable(variant),
        "phase": safe_jsonable(phase),
        "config_hash": CONFIG_HASH,
        "payload": safe_jsonable(payload or {}),
    }

```

```

TRACE_EVENTS.append(event)
ensure_persistence_dirs()
with open(TRACE_JSONL_PATH, "a", encoding="utf-8") as f:
    f.write(json.dumps(event, ensure_ascii=False) + "\n")
return event

class TeeStream:
    def __init__(self, *streams):
        self.streams = streams
    def write(self, data):
        for s in self.streams:
            try:
                s.write(data)
                s.flush()
            except Exception:
                pass
    def flush(self):
        for s in self.streams:
            try:
                s.flush()
            except Exception:
                pass

def start_persistent_logging():
    global _LOG_FILE_HANDLE, _LOGGING_ACTIVE
    if _LOGGING_ACTIVE:
        return
    ensure_persistence_dirs()
    _LOG_FILE_HANDLE = open(RAW_CONSOLE_LOG_PATH, "a", buffering=1,
↪encoding="utf-8")
    sys.stdout = TeeStream(_ORIGINAL_STDOUT, _LOG_FILE_HANDLE)
    sys.stderr = TeeStream(_ORIGINAL_STDERR, _LOG_FILE_HANDLE)
    _LOGGING_ACTIVE = True
    log_event("run_logging_started", payload={"raw_console_log":
↪str(RAW_CONSOLE_LOG_PATH)})

def stop_persistent_logging():
    global _LOG_FILE_HANDLE, _LOGGING_ACTIVE
    if not _LOGGING_ACTIVE:
        return
    log_event("run_logging_stopped", payload={"raw_console_log":
↪str(RAW_CONSOLE_LOG_PATH)})
    sys.stdout = _ORIGINAL_STDOUT
    sys.stderr = _ORIGINAL_STDERR
    try:
        _LOG_FILE_HANDLE.close()
    except Exception:

```

```

        pass
    _LOG_FILE_HANDLE = None
    _LOGGING_ACTIVE = False

def file_sha256(path: Path) -> str:
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(1024 * 1024), b''):
            h.update(chunk)
    return h.hexdigest()

def write_manifest(extra=None):
    ensure_persistence_dirs()
    artifacts = []
    if RESULTS_DIR.exists():
        for p in sorted(RESULTS_DIR.rglob("*")):
            if p.is_file() and p != MANIFEST_PATH:
                try:
                    artifacts.append({
                        "path": str(p.relative_to(RESULTS_DIR)),
                        "size_bytes": int(p.stat().st_size),
                        "sha256": file_sha256(p),
                    })
                except Exception as e:
                    artifacts.append({"path": str(p), "error": repr(e)})

    manifest = {
        "run_id": RUN_ID,
        "trace_id": TRACE_ID,
        "created_utc": datetime.now(timezone.utc).isoformat(),
        "notebook_version": "MMALS_v1.
↪1_RC2M_alpha_Boundary_First_Frozen_Selector_Benchmark_Harness",
        "dataset": DATASET_NAME,
        "run_mode": RUN_MODE,
        "device": str(DEVICE),
        "python": sys.version,
        "platform": platform.platform(),
        "config_hash": CONFIG_HASH,
        "config": config_snapshot(),
        "persistence_root": str(PERSISTENCE_ROOT),
        "results_dir": str(RESULTS_DIR),
        "drive_available": DRIVE_AVAILABLE,
        "artifacts": artifacts,
        "extra": safe_jsonable(extra or {}),
    }

    ensure_persistence_dirs()
    with open(MANIFEST_PATH, "w", encoding="utf-8") as f:
        json.dump(manifest, f, indent=2, ensure_ascii=False)

```

```

    return manifest

log_event("run_initialized", payload={"results_dir": str(RESULTS_DIR),
    ↪ "drive_available": DRIVE_AVAILABLE})
print("RUN_ID:", RUN_ID)
print("TRACE_ID:", TRACE_ID)
print("RESULTS_DIR:", RESULTS_DIR)
print("TRACE_JSONL_PATH:", TRACE_JSONL_PATH)
print("RAW_CONSOLE_LOG_PATH:", RAW_CONSOLE_LOG_PATH)
print("NOTEBOOK_EXPORT_DIR:", NOTEBOOK_EXPORT_DIR)

```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

RUN_ID: MMALS_v11_RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT_FashionMNIST_robust_20260601T164013Z_seeds5

TRACE_ID: d69abf85b62186c9

RESULTS_DIR: /content/drive/MyDrive/MMALS/v1_1_rc2m_alpha_boundary_first_frozen_selector/RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT/MMALS_v11_RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT_FashionMNIST_robust_20260601T164013Z_seeds5

TRACE_JSONL_PATH: /content/drive/MyDrive/MMALS/v1_1_rc2m_alpha_boundary_first_frozen_selector/RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT/MMALS_v11_RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT_FashionMNIST_robust_20260601T164013Z_seeds5/traces/otel_style_events_robust.jsonl

RAW_CONSOLE_LOG_PATH: /content/drive/MyDrive/MMALS/v1_1_rc2m_alpha_boundary_first_frozen_selector/RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT/MMALS_v11_RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT_FashionMNIST_robust_20260601T164013Z_seeds5/raw_logs/console_robust.log

NOTEBOOK_EXPORT_DIR: /content/drive/MyDrive/MMALS/v1_1_rc2m_alpha_boundary_first_frozen_selector/RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT/MMALS_v11_RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT_FashionMNIST_robust_20260601T164013Z_seeds5/notebook_exports

2.14 3. Dataset builder: MNIST-family domain-shift overlap-chain with global labels

The benchmark remains the overlapping chain:

$(0,1) \rightarrow (1,2) \rightarrow (2,3) \rightarrow (3,4) \rightarrow (4,5)$

Labels are global class IDs, not local binary task labels.

Supported DATASET_NAME values:

- MNIST: plain MNIST overlap-chain;
- FashionMNIST: plain FashionMNIST overlap-chain;
- RotatedMNIST: MNIST overlap-chain with deterministic task rotations;
- PermutedMNIST: MNIST overlap-chain with deterministic task-specific pixel permutations.

For DATASET_NAME = "RotatedMNIST", each task receives a deterministic rotation:

Task	Classes	Rotation
0	0 vs 1	-30°
1	1 vs 2	-15°
2	2 vs 3	0°
3	3 vs 4	+15°
4	4 vs 5	+30°

For DATASET_NAME = "PermutedMNIST", each task receives a deterministic pixel permutation:

Task	Classes	Permutation seed
0	0 vs 1	100
1	1 vs 2	101
2	2 vs 3	102
3	3 vs 4	103
4	4 vs 5	104

This keeps the original class-incremental overlap-chain stress while adding explicit domain/context shift.

```
[52]: from torchvision.transforms import functional as TF
from torchvision.transforms import InterpolationMode

_PERMUTATION_CACHE = {}

def get_base_transform(name):
    # Do not flatten here. Rotated-MNIST must rotate [1, 28, 28] tensors first.
    # Permuted-MNIST flattens only after applying a deterministic task
    ↪ permutation.
    return transforms.ToTensor()

def load_base_dataset(name):
    transform = get_base_transform(name)
    if name in ["MNIST", "RotatedMNIST", "PermutedMNIST"]:
        train = datasets.MNIST(DATA_ROOT, train=True, download=True,
        ↪ transform=transform)
        test = datasets.MNIST(DATA_ROOT, train=False, download=True,
        ↪ transform=transform)
    elif name == "FashionMNIST":
        train = datasets.FashionMNIST(DATA_ROOT, train=True, download=True,
        ↪ transform=transform)
        test = datasets.FashionMNIST(DATA_ROOT, train=False, download=True,
        ↪ transform=transform)
    else:
        raise ValueError("DATASET_NAME must be MNIST, FashionMNIST,
        ↪ RotatedMNIST, or PermutedMNIST")
```

```

    return train, test

def get_labels(base_dataset):
    if hasattr(base_dataset, "targets"):
        return np.array(base_dataset.targets)
    raise ValueError("Dataset has no targets attribute")

def task_pairs(pair_mode: str):
    if pair_mode == "disjoint_pairs":
        return [(0, 1), (2, 3), (4, 5), (6, 7), (8, 9)]
    if pair_mode == "overlap_chain":
        return [(0, 1), (1, 2), (2, 3), (3, 4), (4, 5)]
    raise ValueError("Unknown PAIR_MODE")

def seen_classes_until(task_id: int):
    classes = set()
    for a, b in task_pairs(PAIR_MODE)[: task_id + 1]:
        classes.add(a)
        classes.add(b)
    return sorted(c for c in classes if c < N_CLASSES)

def task_rotation_angle(task_id: int, dataset_name: str = DATASET_NAME) -> float:
    if dataset_name == "RotatedMNIST":
        if len(ROTATION_ANGLES) != N_TASKS:
            raise ValueError(f"ROTATION_ANGLES must have length {N_TASKS}, got {len(ROTATION_ANGLES)}")
        return float(ROTATION_ANGLES[task_id])
    return 0.0

def task_permutation_seed(task_id: int, dataset_name: str = DATASET_NAME) -> Optional[int]:
    if dataset_name == "PermutedMNIST":
        if len(PERMUTATION_SEEDS) != N_TASKS:
            raise ValueError(f"PERMUTATION_SEEDS must have length {N_TASKS}, got {len(PERMUTATION_SEEDS)}")
        return int(PERMUTATION_SEEDS[task_id])
    return None

def get_task_permutation(task_id: int, input_dim: int = INPUT_DIM, dataset_name: str = DATASET_NAME):
    seed = task_permutation_seed(task_id, dataset_name)
    if seed is None:
        return None
    key = (dataset_name, task_id, int(input_dim), int(seed))
    if key not in _PERMUTATION_CACHE:
        g = torch.Generator()

```

```

        g.manual_seed(seed)
        _PERMUTATION_CACHE[key] = torch.randperm(input_dim, generator=g)
    return _PERMUTATION_CACHE[key]

def apply_task_transform(x, task_id: int, dataset_name: str):
    # x arrives as [1, 28, 28]. Return flattened [784].
    if dataset_name == "RotatedMNIST":
        x = TF.rotate(
            x,
            angle=task_rotation_angle(task_id, dataset_name),
            interpolation=InterpolationMode.BILINEAR,
            fill=0.0,
        )
        return x.view(-1).float()

    if dataset_name == "PermutedMNIST":
        x_flat = x.view(-1).float()
        perm = get_task_permutation(task_id, input_dim=x_flat.numel(),
dataset_name=dataset_name)
        return x_flat[perm].float()

    return x.view(-1).float()

def subset_pair_dataset_global(
    base_dataset,
    a,
    b,
    task_id: int,
    dataset_name: str,
    max_per_class=None,
    seed=42,
    label_noise=0.0,
    input_noise_std=0.0,
):
    rng = np.random.default_rng(seed)
    labels = get_labels(base_dataset)
    idx_a = np.where(labels == a)[0]
    idx_b = np.where(labels == b)[0]

    if max_per_class is not None:
        idx_a = rng.choice(idx_a, size=min(max_per_class, len(idx_a)),
replace=False)
        idx_b = rng.choice(idx_b, size=min(max_per_class, len(idx_b)),
replace=False)

    indices = np.concatenate([idx_a, idx_b])
    rng.shuffle(indices)

```

```

xs, ys = [], []
for idx in indices:
    x, y = base_dataset[int(idx)]
    y = int(y)
    if y >= N_CLASSES:
        continue

    x = apply_task_transform(x, task_id=task_id, dataset_name=dataset_name)

    if input_noise_std > 0:
        x = torch.clamp(x + torch.randn_like(x) * input_noise_std, 0.0, 1.0)

    xs.append(x)
    ys.append(y)

X = torch.stack(xs).float()
y = torch.tensor(ys).long()

if label_noise > 0:
    flip_mask = torch.rand(len(y)) < label_noise
    y[flip_mask & (y == a)] = b
    y[flip_mask & (y == b)] = a

return TensorDataset(X, y)

def build_stress_tasks(name=DATASET_NAME, seed=42):
    train_base, test_base = load_base_dataset(name)
    pairs = task_pairs(PAIR_MODE)
    tasks = []

    for task_id, (a, b) in enumerate(pairs):
        train_ds = subset_pair_dataset_global(
            train_base, a, b,
            task_id=task_id,
            dataset_name=name,
            max_per_class=TRAIN_PER_CLASS,
            seed=seed + task_id,
            label_noise=LABEL_NOISE_RATE,
            input_noise_std=INPUT_NOISE_STD,
        )
        test_ds = subset_pair_dataset_global(
            test_base, a, b,
            task_id=task_id,
            dataset_name=name,
            max_per_class=TEST_PER_CLASS,
            seed=seed + 100 + task_id,

```

```

        label_noise=0.0,
        input_noise_std=INPUT_NOISE_STD,
    )

    angle = task_rotation_angle(task_id, name)
    perm_seed = task_permutation_seed(task_id, name)
    task_name = f"{a} vs {b}"
    if name == "RotatedMNIST":
        task_name += f" @ {angle:+.0f}deg"
    elif name == "PermutedMNIST":
        task_name += f" @ perm_seed={perm_seed}"

    tasks.append({
        "task_id": task_id,
        "name": task_name,
        "pair": (a, b),
        "rotation_angle": angle,
        "permutation_seed": perm_seed,
        "train": train_ds,
        "test": test_ds,
    })

    return tasks

def sample_context_id(task_id: int, n_contexts: int = N_CONTEXTS, noise_rate:
    float = CONTEXT_NOISE_RATE):
    if random.random() >= noise_rate:
        return task_id
    candidates = [i for i in range(n_contexts) if i != task_id]
    return random.choice(candidates)

tasks_preview = build_stress_tasks(DATASET_NAME, seed=0)
[
    (
        t["task_id"],
        t["name"],
        t["pair"],
        t["rotation_angle"],
        t.get("permutation_seed"),
        len(t["train"]),
        len(t["test"]),
        seen_classes_until(t["task_id"]),
    )
    for t in tasks_preview
]

```

```
[52]: [(0, '0 vs 1', (0, 1), 0.0, None, 2400, 800, [0, 1]),
      (1, '1 vs 2', (1, 2), 0.0, None, 2400, 800, [0, 1, 2]),
      (2, '2 vs 3', (2, 3), 0.0, None, 2400, 800, [0, 1, 2, 3]),
      (3, '3 vs 4', (3, 4), 0.0, None, 2400, 800, [0, 1, 2, 3, 4]),
      (4, '4 vs 5', (4, 5), 0.0, None, 2400, 800, [0, 1, 2, 3, 4, 5])]
```

2.15 4. Model definitions: MLP baseline and MMALS fungal medium

MMALS keeps hosts, transformations, a context encoder, a router, and a mediated latent state **zf**.

This domain-shift notebook does not change the fungal backbone. It freezes the robustly selected RC1b candidate and tests it under rotation-induced or permutation-induced context/domain shift.

```
[53]: class MLPBaseline(nn.Module):
      def __init__(self, input_dim=INPUT_DIM, hidden_dim=HIDDEN_DIM,
        ↪latent_dim=LATENT_DIM, n_classes=N_CLASSES):
          super().__init__()
          self.encoder = nn.Sequential(
              nn.Linear(input_dim, hidden_dim),
              nn.ReLU(),
              nn.Linear(hidden_dim, latent_dim),
              nn.ReLU(),
          )
          self.global_head = nn.Linear(latent_dim, n_classes)

      def forward(self, x, return_cache=False):
          z = self.encoder(x)
          logits = self.global_head(z)
          if not return_cache:
              return logits
          q = torch.full((x.shape[0], N_CONTEXTS), 1.0 / N_CONTEXTS, device=x.
        ↪device)
          cache = {
              "zf": z,
              "routes": None,
              "transformed": None,
              "host_z": None,
              "q_context": q,
              "q_single": q,
              "context_logits": torch.zeros((x.shape[0], N_CONTEXTS), device=x.
        ↪device),
              "context_center_logits": torch.zeros((x.shape[0], N_CONTEXTS),
        ↪device=x.device),
              "context_feat": torch.zeros((x.shape[0], CONTEXT_FEAT_DIM),
        ↪device=x.device),
              "active_context_mode": "mlp_none",
          }
```

```

        return logits, cache

class MLPBlock(nn.Module):
    def __init__(self, input_dim, hidden_dim, latent_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, latent_dim),
            nn.ReLU(),
        )
    def forward(self, x):
        return self.net(x)

class ContextEncoder(nn.Module):
    def __init__(self, input_dim=INPUT_DIM, hidden_dim=128,
        ↪ feat_dim=CONTEXT_FEAT_DIM, n_contexts=N_CONTEXTS):
        super().__init__()
        self.feature_net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, feat_dim),
            nn.ReLU(),
        )
        self.logit_head = nn.Linear(feat_dim, n_contexts)
    def forward(self, x):
        feat = self.feature_net(x)
        logits = self.logit_head(feat)
        return feat, logits

class MMALSClassIncremental(nn.Module):
    def __init__(self):
        super().__init__()
        self.n_tasks = N_TASKS
        self.n_contexts = N_CONTEXTS
        self.n_hosts = N_HOSTS
        self.n_classes = N_CLASSES
        self.temperature = TEMPERATURE
        self.context_temperature = CONTEXT_TEMPERATURE
        self.hosts = nn.ModuleList([MLPBlock(INPUT_DIM, HIDDEN_DIM, LATENT_DIM)
        ↪ for _ in range(N_HOSTS)])
        self.transforms = nn.ModuleList([
            nn.Sequential(nn.Linear(LATENT_DIM, LATENT_DIM), nn.ReLU(), nn.
        ↪ Linear(LATENT_DIM, LATENT_DIM))
            for _ in range(N_HOSTS)
        ])
        self.context_encoder = ContextEncoder()

```

```

        self.context_centers = nn.Parameter(torch.randn(N_CONTEXTS,
↪CONTEXT_FEAT_DIM) * 0.05)
        self.context_embedding = nn.Embedding(N_CONTEXTS, TASK_EMB_DIM)
        self.router = nn.Sequential(
            nn.Linear(INPUT_DIM + TASK_EMB_DIM, ROUTE_HIDDEN_DIM),
            nn.ReLU(),
            nn.Linear(ROUTE_HIDDEN_DIM, N_HOSTS),
        )
        self.global_head = nn.Linear(LATENT_DIM, N_CLASSES)

    def transform_from_host_latents(self, host_z):
        transformed = [self.transforms[i](host_z[:, i, :]) for i in range(self.
↪n_hosts)]
        return torch.stack(transformed, dim=1)

    def context_probs_from_x(self, x):
        feat, logits = self.context_encoder(x)
        q = F.softmax(logits / self.context_temperature, dim=-1)
        return feat, logits, q

    def context_center_logits(self, feat):
        feat_n = F.normalize(feat, dim=-1)
        centers_n = F.normalize(self.context_centers, dim=-1)
        return (feat_n @ centers_n.T) / CENTER_TEMPERATURE

    def explicit_context_probs(self, batch_size, context_id, device):
        q = torch.zeros(batch_size, self.n_contexts, device=device)
        q[:, context_id] = 1.0
        return q

    def uniform_context_probs(self, batch_size, device):
        return torch.full((batch_size, self.n_contexts), 1.0 / self.n_contexts,
↪device=device)

    def latent_from_context_probs(self, x, q_context):
        context_emb = q_context @ self.context_embedding.weight
        host_z = torch.stack([host(x) for host in self.hosts], dim=1)
        transformed = self.transform_from_host_latents(host_z)
        route_input = torch.cat([x, context_emb], dim=1)
        route_logits = self.router(route_input)
        routes = F.softmax(route_logits / self.temperature, dim=-1)
        zf = (routes.unsqueeze(-1) * transformed).sum(dim=1)
        return zf, routes, route_logits, host_z, transformed

    def forward(self, x, task_id: int, context_mode: str = "oracle", context_id:
↪ Optional[int] = None,

```



```

        batch_context_probs: Optional[torch.Tensor] = None,
    ↪return_cache: bool = False):
        b = x.shape[0]
        device = x.device
        feat, ctx_logits, q_single = self.context_probs_from_x(x)
        if context_mode in ["oracle", "noisy"]:
            cid = task_id if context_id is None else context_id
            q_context = self.explicit_context_probs(b, cid, device)
        elif context_mode == "uniform":
            q_context = self.uniform_context_probs(b, device)
        elif context_mode == "inferred_single":
            q_context = q_single
        elif context_mode == "inferred_batch":
            q_context = batch_context_probs if batch_context_probs is not None
    ↪else q_single
        else:
            raise ValueError("context_mode must be oracle, noisy, uniform,
    ↪inferred_single, or inferred_batch")
        zf, routes, route_logits, host_z, transformed = self.
    ↪latent_from_context_probs(x, q_context)
        logits = self.global_head(zf)
        if not return_cache:
            return logits
        return logits, {
            "zf": zf,
            "routes": routes,
            "route_logits": route_logits,
            "host_z": host_z,
            "transformed": transformed,
            "context_feat": feat,
            "context_logits": ctx_logits,
            "context_center_logits": self.context_center_logits(feat),
            "q_context": q_context,
            "q_single": q_single,
        }

def build_model(config):
    if config["model_family"] == "mlp":
        return MLPBaseline().to(DEVICE)
    if config["model_family"] == "mmals":
        return MMALSClazzIncremental().to(DEVICE)
    raise ValueError(f"Unknown model_family: {config['model_family']}")

for family in ["mlp", "mmals"]:
    model_preview = build_model({"model_family": family})

```

```

    print(family, "trainable parameters:", count_parameters(model_preview),
    ↪ "total parameters:", total_parameters(model_preview), "approx MB total:",
    ↪ round(model_size_mb(model_preview, trainable_only=False), 3))
    del model_preview
clear_memory()

```

mlp trainable parameters: 217798 total parameters: 217798 approx MB total: 0.831
 mmals trainable parameters: 1337216 total parameters: 1337216 approx MB total:
 5.101

2.16 5. Context prototypes and prototype-first inference

Prototype-first context inference is preserved from v0.14.x and v1.0c. The RC1b candidate changes only validation-selected temperature/blend context posterior parameters and a guarded global read-out; it does not use explicit task IDs at inference.

```

[54]: def entropy_from_probs(q, eps=1e-8):
    return -(q * torch.log(q + eps)).sum(dim=-1)

def margin_from_probs(q):
    top2 = torch.topk(q, k=2, dim=-1).values
    return top2[:, 0] - top2[:, 1]

def brier_from_probs(q, target: int):
    y = torch.full((q.shape[0],), target, dtype=torch.long, device=q.device)
    onehot = F.one_hot(y, num_classes=q.shape[1]).float()
    return ((q - onehot) ** 2).sum(dim=-1)

def ece_from_probs(q, target: int, n_bins: int = 10):
    y = torch.full((q.shape[0],), target, dtype=torch.long, device=q.device)
    conf, pred = torch.max(q, dim=-1)
    correct = (pred == y).float()
    ece = torch.zeros((), device=q.device)
    for lo in torch.linspace(0, 1, n_bins + 1, device=q.device)[:n_bins]:
        hi = lo + 1.0 / n_bins
        mask = (conf > lo) & (conf <= hi)
        if mask.any():
            ece = ece + mask.float().mean() * torch.abs(conf[mask].mean() -
    ↪ correct[mask].mean())
    return ece

@torch.no_grad()
def gaussian_stats_from_tensor(Z):
    if Z.shape[0] < 2:
        return None
    return {"mean": Z.mean(dim=0).to(DEVICE), "var": Z.var(dim=0,
    ↪ unbiased=False).clamp_min(1e-6).to(DEVICE), "n": Z.shape[0]}

```

```

def diag_frechet_distance_from_stats(batch_values, proto_stats):
    mu = batch_values.mean(dim=0)
    var = batch_values.var(dim=0, unbiased=False).clamp_min(1e-6)
    mu_k = proto_stats["mean"].to(batch_values.device)
    var_k = proto_stats["var"].to(batch_values.device)
    return (mu - mu_k).pow(2).sum() + (torch.sqrt(var) - torch.sqrt(var_k)).
    ↪pow(2).sum()

def frechet_posterior_from_distances(distances, temperature):
    return F.softmax(-distances / temperature, dim=0)

def distance_margin_and_top(distances):
    sorted_d = torch.sort(distances).values
    margin = sorted_d[1] - sorted_d[0] if len(sorted_d) >= 2 else torch.
    ↪tensor(float("nan"), device=distances.device)
    top_context = torch.argmin(distances)
    return margin, top_context

@torch.no_grad()
def features_and_latents_for_dataset(model, dataset, context_id: int,
    ↪max_items=PROTOTYPE_MAX_ITEMS, batch_size=BATCH_SIZE):
    model.eval()
    X, _ = dataset.tensors
    if max_items is not None and len(X) > max_items:
        idx = torch.randperm(len(X))[:max_items]
        X = X[idx]
    feats, latents = [], []
    loader = make_loader(TensorDataset(X, torch.zeros(len(X), dtype=torch.
    ↪long)), batch_size=batch_size, shuffle=False)
    for xb, _ in loader:
        xb = xb.to(DEVICE)
        feat, _, _ = model.context_probs_from_x(xb)
        q_oracle = model.explicit_context_probs(xb.shape[0], context_id, xb.
        ↪device)
        zf, _, _, _, _ = model.latent_from_context_probs(xb, q_oracle)
        feats.append(feat.detach().cpu())
        latents.append(zf.detach().cpu())
    return torch.cat(feats, dim=0), torch.cat(latents, dim=0)

@torch.no_grad()
def refresh_context_prototypes(model, memory_buffers: Dict[int, TensorDataset]):
    if not isinstance(model, MMALSCClassIncremental):
        return {}
    prototypes = {}
    for context_id, mem_ds in memory_buffers.items():

```

```

        feats, latents = features_and_latents_for_dataset(model, mem_ds,
↪context_id=context_id)
        feat_stats = gaussian_stats_from_tensor(feats)
        latent_stats = gaussian_stats_from_tensor(latents)
        if feat_stats is not None and latent_stats is not None:
            prototypes[context_id] = {"feature": feat_stats, "latent":
↪latent_stats}
        return prototypes

def infer_batch_context_probs(model, x, prototypes, config=None):
    config = config or {}
    blend_single = config.get("batch_blend_single", BATCH_BLEND_SINGLE)
    blend_latent = config.get("batch_blend_latent", BATCH_BLEND_LATENT)
    blend_feature = config.get("batch_blend_feature", BATCH_BLEND_FEATURE)
    context_feat, _, q_single = model.context_probs_from_x(x)
    if not prototypes:
        return q_single, {
            "frechet_margin": torch.full((x.shape[0],), float("nan"), device=x.
↪device),
            "latent_frechet_margin": torch.full((x.shape[0],), float("nan"),
↪device=x.device),
            "feature_frechet_margin": torch.full((x.shape[0],), float("nan"),
↪device=x.device),
            "prototype_weight": torch.full((x.shape[0],), 0.0, device=x.device),
        }
    zf_prelim, _, _, _ = model.latent_from_context_probs(x, q_single)
    feature_distances = torch.full((model.n_contexts,), 1e6, device=x.device)
    latent_distances = torch.full((model.n_contexts,), 1e6, device=x.device)
    for cid, proto in prototypes.items():
        feature_distances[cid] = diag_frechet_distance_from_stats(context_feat,
↪proto["feature"])
        latent_distances[cid] = diag_frechet_distance_from_stats(zf_prelim,
↪proto["latent"])
        q_feature = frechet_posterior_from_distances(feature_distances,
↪FRECHET_TEMPERATURE)
        q_latent = frechet_posterior_from_distances(latent_distances,
↪LATENT_FRECHET_TEMPERATURE)
        proto_mass = max(blend_latent + blend_feature, 1e-8)
        q_proto = (blend_feature * q_feature + blend_latent * q_latent) / proto_mass
        q_batch = q_proto.unsqueeze(0).expand(x.shape[0], -1)
        q = blend_single * q_single + (1.0 - blend_single) * q_batch
        q = q / q.sum(dim=-1, keepdim=True).clamp_min(1e-8)
        f_margin, _ = distance_margin_and_top(feature_distances)
        l_margin, _ = distance_margin_and_top(latent_distances)
        return q, {
            "frechet_margin": l_margin.expand(x.shape[0]),

```

```

        "latent_frechet_margin": l_margin.expand(x.shape[0]),
        "feature_frechet_margin": f_margin.expand(x.shape[0]),
        "prototype_weight": torch.full((x.shape[0],), 1.0 - blend_single,
↪device=x.device),
    }

```

2.17 6. Routing helpers

This preserves uniform, oracle and proto-first routes. The robust validation harness uses the same zf extraction protocol for every cloned variant.

```

[55]: def q_uniform_like(model, x):
        return model.uniform_context_probs(x.shape[0], x.device)

def normalize_probs(q, eps=1e-8):
    return q / q.sum(dim=-1, keepdim=True).clamp_min(eps)

def logits_from_context_probs(model, x, q_context):
    zf, routes, route_logits, host_z, transformed = model.
↪latent_from_context_probs(x, q_context)
    logits = model.global_head(zf)
    cache = {
        "zf": zf,
        "routes": routes,
        "route_logits": route_logits,
        "host_z": host_z,
        "transformed": transformed,
        "q_context": q_context,
    }
    return logits, cache

def method_context_mode_and_probs(model, x, task_id, config, prototypes=None,
↪train=False, noisy_context_id=None, y=None):
    if config["model_family"] == "mlp":
        return "mlp_none", None, None, {}
    strategy = config["strategy"]
    if strategy == "oracle":
        return "oracle", None, task_id, {}
    if strategy == "noisy":
        cid = noisy_context_id if noisy_context_id is not None else task_id
        return "noisy", None, cid, {}
    if strategy == "uniform":
        return "uniform", None, None, {}
    if strategy == "inferred_single":
        return "inferred_single", None, None, {}
    if strategy in ["inferred_batch", "proto_first"]:
        if train:

```

```

        return "inferred_single", None, None, {}
        q_batch, batch_info = infer_batch_context_probs(model, x, prototypes or {},
        ↪ {}, config=config)
        return "inferred_batch", q_batch, None, batch_info
        raise ValueError(f"Unknown strategy: {strategy}")

def forward_by_method(model, x, task_id, config, prototypes=None, train=False,
        ↪ noisy_context_id=None, y=None, return_cache=True):
    if config["model_family"] == "mlp":
        logits, cache = model(x, return_cache=True)
        return (logits, cache) if return_cache else logits
    mode, batch_probs, context_id, batch_info = method_context_mode_and_probs(
        model, x, task_id, config, prototypes=prototypes, train=train,
        noisy_context_id=noisy_context_id, y=y
    )
    logits, cache = model(
        x,
        task_id,
        context_mode=mode,
        context_id=context_id,
        batch_context_probs=batch_probs,
        return_cache=True,
    )
    cache.update(batch_info)
    cache["active_context_mode"] = mode
    return (logits, cache) if return_cache else logits

def probe_config_from_mode(context_mode: str):
    if context_mode == "uniform":
        return {**COMMON_PROTO_FIRST, "strategy": "uniform"}
    if context_mode == "proto":
        return {**COMMON_PROTO_FIRST, "strategy": "proto_first"}
    if context_mode == "oracle":
        return {**COMMON_MMALS, "strategy": "oracle"}
    raise ValueError("context_mode must be uniform, proto, or oracle")

```

2.18 7. Training losses, balanced replay and functional memory

This block retains the v0.14.x continual-learning losses. v1.0-RC1b candidate-freeze validation does not change the base training objective.

```

[56]: def entropy_loss(routes, eps=1e-8):
        if routes is None:
            return torch.zeros((), device=DEVICE)
        return -(routes * torch.log(routes + eps)).sum(dim=1).mean()

def metabolic_loss(transformed, routes):

```

```

    if transformed is None or routes is None:
        return torch.zeros(), device=DEVICE)
    return (routes.unsqueeze(-1) * transformed.pow(2)).sum(dim=(1, 2)).mean()

def kl_distill_loss(logits_current, logits_teacher,
    ↪temperature=DISTILL_TEMPERATURE):
    log_p_current = F.log_softmax(logits_current / temperature, dim=-1)
    p_teacher = F.softmax(logits_teacher / temperature, dim=-1)
    return (temperature ** 2) * F.kl_div(log_p_current, p_teacher,
    ↪reduction="batchmean")

def clone_teacher(model):
    teacher = copy.deepcopy(model).to(DEVICE)
    teacher.eval()
    for p in teacher.parameters():
        p.requires_grad_(False)
    return teacher

def build_memory_subset(dataset: TensorDataset, max_items: int, seed: int):
    X, y = dataset.tensors
    rng = np.random.default_rng(seed)
    idx = rng.choice(np.arange(len(X)), size=min(max_items, len(X)),
    ↪replace=False)
    return TensorDataset(X[idx].clone(), y[idx].clone())

def sample_from_tensor_dataset(dataset: TensorDataset, batch_size: int):
    X, y = dataset.tensors
    idx = torch.randint(0, len(X), (min(batch_size, len(X)),))
    return X[idx].to(DEVICE), y[idx].to(DEVICE)

def update_class_memory(class_memory: Dict[int, Dict[str, torch.Tensor]],
    ↪dataset: TensorDataset, task_id: int, max_per_class: int, seed: int):
    X, y = dataset.tensors
    rng = np.random.default_rng(seed)
    new_memory = {k: {kk: vv.clone() for kk, vv in v.items()} for k, v in
    ↪class_memory.items()}
    for cls in sorted(torch.unique(y).tolist()):
        cls = int(cls)
        idx = torch.where(y == cls)[0]
        if len(idx) == 0:
            continue
        X_new = X[idx].clone()
        y_new = y[idx].clone()
        t_new = torch.full((len(idx),), task_id, dtype=torch.long)
        if cls in new_memory:
            X_cat = torch.cat([new_memory[cls]["X"], X_new], dim=0)
            y_cat = torch.cat([new_memory[cls]["y"], y_new], dim=0)

```

```

        t_cat = torch.cat([new_memory[cls]["task_id"], t_new], dim=0)
    else:
        X_cat, y_cat, t_cat = X_new, y_new, t_new
        if len(X_cat) > max_per_class:
            keep = rng.choice(np.arange(len(X_cat)), size=max_per_class,
↪replace=False)
            keep = torch.tensor(keep, dtype=torch.long)
            X_cat, y_cat, t_cat = X_cat[keep], y_cat[keep], t_cat[keep]
        new_memory[cls] = {"X": X_cat, "y": y_cat, "task_id": t_cat}
    return new_memory

def sample_balanced_class_memory(class_memory: Dict[int, Dict[str, torch.
↪Tensor]], batch_size: int):
    active_classes = sorted([int(c) for c, data in class_memory.items() if
↪len(data["y"]) > 0])
    if not active_classes:
        return None, None, None
    per_class = max(1, math.ceil(batch_size / len(active_classes)))
    xs, ys, ts = [], [], []
    for cls in active_classes:
        data = class_memory[cls]
        n = len(data["y"])
        k = min(per_class, n)
        idx = torch.randint(0, n, (k,))
        xs.append(data["X"][idx])
        ys.append(data["y"][idx])
        ts.append(data["task_id"][idx])
    X = torch.cat(xs, dim=0)
    y = torch.cat(ys, dim=0)
    t = torch.cat(ts, dim=0)
    perm = torch.randperm(len(y))[: min(batch_size, len(y))]
    return X[perm].to(DEVICE), y[perm].to(DEVICE), t[perm].to(DEVICE)

def balanced_replay_ce_loss(model, class_memory, config, prototypes=None,
↪batch_size=BALANCED_REPLAY_BATCH):
    zero = torch.zeros((), device=DEVICE)
    if not config.get("balanced_replay_ce", False) or not class_memory:
        return zero, {"balanced_replay_ce_raw": 0.0, "balanced_replay_classes":
↪0, "balanced_replay_items": 0}
    xb, yb, tb = sample_balanced_class_memory(class_memory,
↪batch_size=batch_size)
    if xb is None:
        return zero, {"balanced_replay_ce_raw": 0.0, "balanced_replay_classes":
↪0, "balanced_replay_items": 0}
    total_loss = zero
    n_groups = 0

```



```

for src_task_id in torch.unique(tb).detach().cpu().tolist():
    src_task_id = int(src_task_id)
    mask = (tb == src_task_id)
    if not mask.any():
        continue
    logits, _ = forward_by_method(
        model,
        xb[mask],
        src_task_id,
        config,
        prototypes=prototypes,
        train=not BALANCED_REPLAY_USE_EVAL_CONTEXT,
        y=yb[mask],
        return_cache=True,
    )
    total_loss = total_loss + F.cross_entropy(logits, yb[mask])
    n_groups += 1
raw = total_loss / max(n_groups, 1)
weight = float(config.get("balanced_replay_weight", BALANCED_REPLAY_WEIGHT))
return weight * raw, {
    "balanced_replay_ce_raw": float(raw.detach().cpu()),
    "balanced_replay_classes": len(class_memory),
    "balanced_replay_items": int(sum(len(v["y"]) for v in class_memory.
↪values()))),
}

def context_supervision_weight(config, task_id: int, epoch: int):
    if not config.get("context_calibrated", False):
        return 0.0
    total_steps = max(N_TASKS * EPOCHS_PER_TASK - 1, 1)
    step = task_id * EPOCHS_PER_TASK + epoch
    progress = min(max(step / total_steps, 0.0), 1.0)
    return CONTEXT_SUP_END + (CONTEXT_SUP_START - CONTEXT_SUP_END) * ((1.0 - ↪
↪progress) ** CONTEXT_SUP_GAMMA)

def context_calibration_loss(model, cache, task_id: int, config, epoch: int):
    zero = torch.zeros(), device=DEVICE)
    if not config.get("context_calibrated", False) or config["model_family"] != ↪
↪"mmals":
        return zero, {"ctx_weight": 0.0, "ctx_ce": 0.0}
    target = torch.full((cache["context_logits"].shape[0],), task_id, ↪
↪dtype=torch.long, device=DEVICE)
    weight = context_supervision_weight(config, task_id, epoch)
    ce = F.cross_entropy(cache["context_logits"], target, ↪
↪label_smoothing=CONTEXT_LABEL_SMOOTHING)

```

```

        center_ce = F.cross_entropy(model.
↪context_center_logits(cache["context_feat"]), target,
↪label_smoothing=CONTEXT_LABEL_SMOOTHING)
        entropy = entropy_from_probs(cache["q_single"]).mean()
        margin_penalty = F.relu(CONTEXT_MARGIN_TARGET -
↪margin_from_probs(cache["q_single"])).mean()
        loss = weight * (ce + LAMBDA_CONTEXT_CENTER * center_ce) +
↪LAMBDA_CONTEXT_ENTROPY * entropy + LAMBDA_CONTEXT_MARGIN * margin_penalty
        return loss, {"ctx_weight": float(weight), "ctx_ce": float(ce.detach().
↪cpu())}

def context_replay_loss(model, teacher, memory_buffers, config,
↪batch_size=CONTEXT_REPLAY_BATCH):
    zero = torch.zeros((), device=DEVICE)
    if config["model_family"] != "mmals" or not config.get("context_replay",
↪False) or not memory_buffers:
        return zero, {"ctx_replay_raw": 0.0, "ctx_replay_scale": config.
↪get("context_replay_scale", 0.0)}
    ce_total = zero
    center_total = zero
    kl_total = zero
    feat_total = zero
    n_terms = 0
    for old_context_id, mem_ds in memory_buffers.items():
        xb, _ = sample_from_tensor_dataset(mem_ds, batch_size=batch_size)
        feat_cur, logits_cur, _ = model.context_probs_from_x(xb)
        target = torch.full((xb.shape[0],), old_context_id, dtype=torch.long,
↪device=DEVICE)
        ce_total = ce_total + F.cross_entropy(logits_cur, target,
↪label_smoothing=CONTEXT_LABEL_SMOOTHING)
        center_total = center_total + F.cross_entropy(model.
↪context_center_logits(feat_cur), target,
↪label_smoothing=CONTEXT_LABEL_SMOOTHING)
        if teacher is not None and isinstance(teacher, MMALSClazzIncremental):
            with torch.no_grad():
                feat_t, logits_t, _ = teacher.context_probs_from_x(xb)
                kl_total = kl_total + kl_distill_loss(logits_cur, logits_t,
↪temperature=CONTEXT_DISTILL_TEMPERATURE)
                feat_total = feat_total + F.mse_loss(feat_cur, feat_t)
            n_terms += 1
    ce_loss = ce_total / max(n_terms, 1)
    center_loss = center_total / max(n_terms, 1)
    kl_loss = kl_total / max(n_terms, 1) if teacher is not None and
↪isinstance(teacher, MMALSClazzIncremental) else zero
    feat_loss = feat_total / max(n_terms, 1) if teacher is not None and
↪isinstance(teacher, MMALSClazzIncremental) else zero

```

```

        raw = LAMBDA_CONTEXT_REPLAY_CE * ce_loss + LAMBDA_CONTEXT_REPLAY_CENTER *
↪center_loss + LAMBDA_CONTEXT_TEACHER_KL * kl_loss +
↪LAMBDA_CONTEXT_FEATURE_DRIFT * feat_loss
        scale = float(config.get("context_replay_scale", 1.0))
        return scale * raw, {"ctx_replay_raw": float(raw.detach().cpu()),
↪"ctx_replay_scale": scale}

def rc3_training_hydro_importance(reference_logits, reference_routes):
    """Importance signal used only by optional RC2c Hydro training regularizer.
    ↪"""
    probs = F.softmax(reference_logits, dim=-1)
    confidence = probs.max(dim=-1).values.mean()
    if reference_routes is None:
        dominance = torch.tensor(1.0, device=reference_logits.device)
    else:
        dominance = reference_routes.max(dim=-1).values.mean()
    return torch.clamp(confidence * dominance, 0.0, 1.0)

def rc3_training_hydro_viscosity(importance):
    return torch.clamp(
        RC3_HYDRO_BASE_VISCOSITY + (RC3_HYDRO_MAX_VISCOSITY -
↪RC3_HYDRO_BASE_VISCOSITY) * importance,
        RC3_HYDRO_BASE_VISCOSITY,
        RC3_HYDRO_MAX_VISCOSITY,
    )

def old_memory_losses(model, teacher, memory_buffers, config, batch_size,
↪prototypes=None):
    zero = torch.zeros((), device=DEVICE)
    if teacher is None or not memory_buffers:
        return zero, {"route": 0.0, "latent": 0.0, "output": 0.0,
↪"hydro_viscosity": 0.0}
    route_total = zero
    latent_total = zero
    output_total = zero
    hydro_total = zero
    n_terms = 0
    for old_task_id, mem_ds in memory_buffers.items():
        xb, yb = sample_from_tensor_dataset(mem_ds, batch_size=batch_size)
        logits_cur, cache_cur = forward_by_method(model, xb, old_task_id,
↪config, prototypes=prototypes, train=False, y=yb, return_cache=True)
        with torch.no_grad():

```

```

        logits_t, cache_t = forward_by_method(teacher, xb, old_task_id,
↪config, prototypes=prototypes, train=False, y=yb, return_cache=True)
        if config["model_family"] == "mmals":
            route_total = route_total + F.mse_loss(cache_cur["routes"],
↪cache_t["routes"])
            latent_total = latent_total + F.mse_loss(cache_cur["zf"],
↪cache_t["zf"])
            if config.get("output_distill", False) or config.get("output", 0.0) > 0:
                output_total = output_total + kl_distill_loss(logits_cur, logits_t)
            if config["model_family"] == "mmals" and globals().
↪get("RC3_HYDRO_REGULARIZER_ENABLED", False):
                # Optional Hydro viscosity regularizer. It is disabled by default.
                # It reuses existing old-memory teacher comparisons and adds no
↪final-test leakage.
                importance = rc3_training_hydro_importance(logits_t.detach(),
↪cache_t.get("routes"))
                if globals().get("RC3_HYDRO_REGULARIZER_MODE", "off") ==
↪"high_gain_viscous" and float(importance.detach().cpu()) <
↪RC3_HYDRO_HIGH_GAIN_IMPORTANCE_THRESHOLD:
                    hydro_component = zero
                else:
                    nu = rc3_training_hydro_viscosity(importance)
                    # Use existing route/latent/output preservation terms as a
↪viscosity-weighted stabilizer.
                    hydro_component = nu.detach() * (
                        0.25 * F.mse_loss(cache_cur["routes"], cache_t["routes"].
↪detach())
                        + F.mse_loss(cache_cur["zf"], cache_t["zf"].detach())
                        + 0.25 * kl_distill_loss(logits_cur, logits_t.detach())
                    )
                    hydro_total = hydro_total + hydro_component
                n_terms += 1
            route_loss = route_total / max(n_terms, 1)
            latent_loss = latent_total / max(n_terms, 1)
            output_loss = output_total / max(n_terms, 1)
            hydro_loss = hydro_total / max(n_terms, 1)
            loss = config.get("route", 0.0) * route_loss + config.get("latent", 0.0) *
↪latent_loss + config.get("output", 0.0) * output_loss
            if config["model_family"] == "mmals" and globals().
↪get("RC3_HYDRO_REGULARIZER_ENABLED", False):
                loss = loss + RC3_HYDRO_REGULARIZER_COEFF * hydro_loss
            if config["model_family"] == "mlp" and config.get("output_distill", False):
                loss = output_loss
            return loss, {"route": float(route_loss.detach().cpu()), "latent":
↪float(latent_loss.detach().cpu()), "output": float(output_loss.detach().
↪cpu()), "hydro_viscosity": float(hydro_loss.detach().cpu())}

```

```

@torch.no_grad()
def calibrate_context_temperature(model, memory_buffers, config,
    ↪candidate_temps=CONTEXT_TEMP_GRID, max_items=TEMP_CALIB_MAX_ITEMS):
    if config["model_family"] != "mmals" or not config.get("temp_calibration",
    ↪False) or not memory_buffers:
        return float(getattr(model, "context_temperature", float("nan")))
    saved_temp = float(model.context_temperature)
    logits_all, targets_all = [], []
    model.eval()
    for cid, mem_ds in memory_buffers.items():
        X, _ = mem_ds.tensors
        if len(X) > max_items:
            X = X[torch.randperm(len(X))[:max_items]]
            loader = make_loader(TensorDataset(X, torch.zeros(len(X), dtype=torch.
    ↪long)), batch_size=BATCH_SIZE, shuffle=False)
            for xb, _ in loader:
                xb = xb.to(DEVICE)
                _, logits = model.context_encoder(xb)
                logits_all.append(logits.detach())
                targets_all.append(torch.full((xb.shape[0],), cid, dtype=torch.
    ↪long, device=DEVICE))
            if not logits_all:
                return saved_temp
        logits_all = torch.cat(logits_all, dim=0)
        targets_all = torch.cat(targets_all, dim=0)
        best_temp, best_loss = saved_temp, float("inf")
        for temp in candidate_temps:
            loss = F.cross_entropy(logits_all / temp, targets_all).item()
            if loss < best_loss:
                best_loss = loss
                best_temp = float(temp)
        model.context_temperature = best_temp
    return best_temp

# -----
# v0.14.8 head-repair utilities
# -----

def head_margin_repair_loss(logits, y, target_margin=HEAD_MARGIN_TARGET,
    ↪hard_classes=HARD_CLASSES, ref_class=REFERENCE_CLASS_FOR_MARGIN):
    """Penalize hard-class logits when they fall below the reference class
    ↪logit.

    This is intentionally head-focused: it does not change routes or latent
    ↪states.

```

```

"""
if logits is None or y is None:
    return torch.zeros(), device=DEVICE)
losses = []
for cls in hard_classes:
    if cls >= logits.shape[1] or ref_class >= logits.shape[1]:
        continue
    mask = (y == cls)
    if mask.any():
        margin = logits[mask, cls] - logits[mask, ref_class]
        losses.append(F.relu(target_margin - margin).mean())
if not losses:
    return torch.zeros(), device=logits.device)
return torch.stack(losses).mean()

def class_memory_to_tensor_dataset(class_memory: Dict[int, Dict[str, torch.
    ↪Tensor]]):
    if not class_memory:
        return None
    xs, ys, ts = [], [], []
    for cls, data in sorted(class_memory.items()):
        if len(data["y"]) == 0:
            continue
        xs.append(data["X"])
        ys.append(data["y"])
        ts.append(data["task_id"])
    if not xs:
        return None
    return TensorDataset(torch.cat(xs, dim=0), torch.cat(ys, dim=0), torch.
    ↪cat(ts, dim=0))

@torch.no_grad()
def extract_zf_from_batch_by_task(model, xb, tb, config, prototypes=None):
    """Extract frozen z_f for a mixed replay batch containing items from
    ↪several source tasks."""
    zf_out = torch.zeros((xb.shape[0], LATENT_DIM), device=xb.device)
    for src_task_id in torch.unique(tb).detach().cpu().tolist():
        src_task_id = int(src_task_id)
        mask = (tb == src_task_id)
        if not mask.any():
            continue
        _, cache = forward_by_method(
            model,
            xb[mask],
            src_task_id,

```

```

        config,
        prototypes=prototypes,
        train=False,
        y=None,
        return_cache=True,
    )
    zf_out[mask] = cache["zf"].detach()
    return zf_out

def sample_repair_batch(class_memory, batch_size=HEAD_REPAIR_BATCH):
    xb, yb, tb = sample_balanced_class_memory(class_memory,
    ↪batch_size=batch_size)
    return xb, yb, tb

def train_head_refresh_phase(model, class_memory, config, prototypes=None,
    ↪steps=HEAD_REFRESH_STEPS, epochs=HEAD_REFRESH_EPOCHS, lr=HEAD_REFRESH_LR):
    """Freeze the fungal backbone and update only the deployed global head
    ↪using balanced memory."""
    rows = []
    if config["model_family"] != "mmals" or not class_memory:
        return rows
    model.eval()
    opt = torch.optim.Adam(model.global_head.parameters(), lr=lr)
    for epoch in range(epochs):
        sums = {"head_refresh_loss": 0.0, "head_refresh_ce": 0.0,
    ↪"head_refresh_margin": 0.0}
        n = 0
        for _ in range(steps):
            xb, yb, tb = sample_repair_batch(class_memory)
            if xb is None:
                continue
            with torch.no_grad():
                zf = extract_zf_from_batch_by_task(model, xb, tb, config,
    ↪prototypes=prototypes)
            opt.zero_grad(set_to_none=True)
            logits = model.global_head(zf)
            ce = F.cross_entropy(logits, yb)
            margin = head_margin_repair_loss(logits, yb)
            loss = ce + HEAD_MARGIN_LAMBDA * margin
            loss.backward()
            opt.step()
            sums["head_refresh_loss"] += float(loss.detach().cpu())
            sums["head_refresh_ce"] += float(ce.detach().cpu())
            sums["head_refresh_margin"] += float(margin.detach().cpu())
            n += 1

```

```

        if n > 0:
            rows.append({k: v / n for k, v in sums.items()} | {"repair_epoch": epoch, "repair_type": "head_refresh"})
        return rows

def build_memory_feature_dataset(model, class_memory, config, prototypes=None, max_items=None):
    mem_ds = class_memory_to_tensor_dataset(class_memory)
    if mem_ds is None:
        return None
    X, y, t = mem_ds.tensors
    if max_items is not None and len(y) > max_items:
        idx = torch.randperm(len(y))[:max_items]
        X, y, t = X[idx], y[idx], t[idx]
    feats, labels = [], []
    loader = make_loader(TensorDataset(X, y, t), batch_size=BATCH_SIZE, shuffle=False)
    model.eval()
    for xb, yb, tb in loader:
        xb, yb, tb = xb.to(DEVICE), yb.to(DEVICE), tb.to(DEVICE)
        with torch.no_grad():
            zf = extract_zf_from_batch_by_task(model, xb, tb, config, prototypes=prototypes)
            feats.append(zf.detach().cpu())
            labels.append(yb.detach().cpu())
    return TensorDataset(torch.cat(feats, dim=0).float(), torch.cat(labels, dim=0).long())

def train_temporary_zf_probe(feature_ds, seed=0, epochs=PROBE_EPOCHS):
    if feature_ds is None:
        return None, []
    set_seed(seed + 81000)
    probe = LinearProbe(input_dim=LATENT_DIM).to(DEVICE)
    opt = torch.optim.Adam(probe.parameters(), lr=PROBE_LR)
    loader = make_loader(feature_ds, batch_size=BATCH_SIZE, shuffle=True)
    rows = []
    for epoch in range(epochs):
        total, n = 0.0, 0
        probe.train()
        for zf, yb in loader:
            zf, yb = zf.to(DEVICE), yb.to(DEVICE)
            opt.zero_grad(set_to_none=True)
            loss = F.cross_entropy(probe(zf), yb)
            loss.backward()
            opt.step()

```



```

        total += float(loss.detach().cpu())
        n += 1
        rows.append({"repair_epoch": epoch, "repair_type": "temporary_probe",
↪ "probe_loss": total / max(n, 1)})
        probe.eval()
        for p in probe.parameters():
            p.requires_grad_(False)
        return probe, rows

def train_probe_distilled_head_phase(model, class_memory, config,
↪ prototypes=None, seed=0, steps=HEAD_REFRESH_STEPS,
↪ epochs=PROBE_DISTILL_EPOCHS, lr=PROBE_DISTILL_LR):
    """Train a temporary linear probe on frozen z_f and distill it into the
↪ deployed global head."""
    rows = []
    if config["model_family"] != "mmals" or not class_memory:
        return rows
    feature_ds = build_memory_feature_dataset(model, class_memory, config,
↪ prototypes=prototypes)
    probe, probe_rows = train_temporary_zf_probe(feature_ds, seed=seed,
↪ epochs=max(5, PROBE_EPOCHS // 2))
    rows.extend(probe_rows)
    if probe is None:
        return rows
    model.eval()
    opt = torch.optim.Adam(model.global_head.parameters(), lr=lr)
    for epoch in range(epochs):
        sums = {"probe_distill_loss": 0.0, "probe_distill_ce": 0.0,
↪ "probe_distill_kl": 0.0, "probe_distill_margin": 0.0}
        n = 0
        for _ in range(steps):
            xb, yb, tb = sample_repair_batch(class_memory)
            if xb is None:
                continue
            with torch.no_grad():
                zf = extract_zf_from_batch_by_task(model, xb, tb, config,
↪ prototypes=prototypes)
                probe_logits = probe(zf)
                opt.zero_grad(set_to_none=True)
                logits = model.global_head(zf)
                ce = F.cross_entropy(logits, yb)
                kl = kl_distill_loss(logits, probe_logits,
↪ temperature=PROBE_DISTILL_TEMPERATURE)
                margin = head_margin_repair_loss(logits, yb)

```

```

        loss = PROBE_DISTILL_ALPHA_CE * ce + PROBE_DISTILL_ALPHA_KL * kl +
HEAD_MARGIN_LAMBDA * margin
        loss.backward()
        opt.step()
        sums["probe_distill_loss"] += float(loss.detach().cpu())
        sums["probe_distill_ce"] += float(ce.detach().cpu())
        sums["probe_distill_kl"] += float(kl.detach().cpu())
        sums["probe_distill_margin"] += float(margin.detach().cpu())
        n += 1
    if n > 0:
        rows.append({k: v / n for k, v in sums.items()} | {"repair_epoch":
epoch, "repair_type": "probe_distill"})
    return rows

def train_bias_calibration_phase(model, class_memory, config, prototypes=None,
steps=BIAS_CALIB_STEPS, lr=BIAS_CALIB_LR):
    """Learn a tiny post-hoc class-bias correction on balanced memory and fold
it into the global head bias."""
    rows = []
    if config["model_family"] != "mmals" or not class_memory:
        return rows
    delta = nn.Parameter(torch.zeros(N_CLASSES, device=DEVICE))
    opt = torch.optim.Adam([delta], lr=lr)
    for step in range(steps):
        xb, yb, tb = sample_repair_batch(class_memory)
        if xb is None:
            continue
        with torch.no_grad():
            zf = extract_zf_from_batch_by_task(model, xb, tb, config,
prototypes=prototypes)
            base_logits = model.global_head(zf).detach()
            opt.zero_grad(set_to_none=True)
            logits = base_logits + delta.unsqueeze(0)
            ce = F.cross_entropy(logits, yb)
            margin = head_margin_repair_loss(logits, yb)
            reg = BIAS_CALIB_REG * delta.pow(2).mean()
            loss = ce + HEAD_MARGIN_LAMBDA * margin + reg
            loss.backward()
            opt.step()
            rows.append({
                "repair_epoch": step,
                "repair_type": "bias_calibration",
                "bias_calib_loss": float(loss.detach().cpu()),
                "bias_calib_ce": float(ce.detach().cpu()),
                "bias_calib_margin": float(margin.detach().cpu()),
            })

```

```

with torch.no_grad():
    if model.global_head.bias is not None:
        model.global_head.bias.add_(delta.detach())
return rows

def maybe_run_head_repair_phases(model, class_memory, config, prototypes=None,
    ↪seed=0):
    rows = []
    if config["model_family"] != "mmals":
        return rows
    if config.get("post_head_refresh", False):
        rows.extend(train_head_refresh_phase(model, class_memory, config,
    ↪prototypes=prototypes))
    if config.get("post_probe_distill", False):
        rows.extend(train_probe_distilled_head_phase(model, class_memory,
    ↪config, prototypes=prototypes, seed=seed))
    if config.get("post_bias_calibration", False):
        rows.extend(train_bias_calibration_phase(model, class_memory, config,
    ↪prototypes=prototypes))
    return rows

```

2.19 8. Readout and v1.0-RC1b context-posterior utilities

The local readout classes from v1.0a/v1.0b are kept as reusable infrastructure, but the RC1b validation target is **not** a local-head branch.

The object of validation is the inferred-context posterior used to extract **zf**, plus a true CE+KL guarded global readout. Deployable variants remain task-free. Oracle variants are diagnostic references only.

```

[57]: def clone_tensor_dataset(ds: TensorDataset):
    tensors = tuple(t.clone().detach().cpu() for t in ds.tensors)
    return TensorDataset(*tensors)

def clone_memory_buffers(memory_buffers: Dict[int, TensorDataset]):
    return {int(k): clone_tensor_dataset(v) for k, v in memory_buffers.items()}

def clone_class_memory(class_memory):
    return {int(k): {kk: vv.clone().detach().cpu() for kk, vv in v.items()} for
    ↪k, v in class_memory.items()}

def clone_prototypes(prototypes):
    out = {}
    for cid, proto in prototypes.items():
        out[int(cid)] = {}
        for kind, stats in proto.items():

```

```

        out[int(cid)][kind] = {}
        for sk, sv in stats.items():
            if torch.is_tensor(sv):
                out[int(cid)][kind][sk] = sv.clone().detach().cpu()
            else:
                out[int(cid)][kind][sk] = copy.deepcopy(sv)
    return out

def cpu_state_dict(model):
    return {k: v.detach().cpu().clone() for k, v in model.state_dict().items()}

def restore_base_model_from_checkpoint(checkpoint, config):
    model = build_model(config)
    model.load_state_dict(checkpoint["model_state"])
    if hasattr(model, "context_temperature"):
        model.context_temperature = float(checkpoint.get("context_temperature",
↪CONTEXT_TEMPERATURE))
    model.to(DEVICE)
    return model

def split_class_memory_for_readout_validation(class_memory,
↪val_fraction=VALIDATION_SPLIT_FRACTION, seed=0):
    if not class_memory:
        return {}, {}
    rng = np.random.default_rng(seed)
    repair_memory, validation_memory = {}, {}
    for cls, data in class_memory.items():
        n = len(data["y"])
        if n == 0:
            continue
        idx = np.arange(n)
        rng.shuffle(idx)
        n_val = int(round(n * val_fraction))
        n_val = min(max(n_val, 1 if n >= 4 else 0), max(n - 1, 0))
        val_idx = torch.tensor(idx[:n_val], dtype=torch.long)
        train_idx = torch.tensor(idx[n_val:], dtype=torch.long)
        if len(train_idx) > 0:
            repair_memory[int(cls)] = {k: v[train_idx].clone() for k, v in data.
↪items()}
            if len(val_idx) > 0:
                validation_memory[int(cls)] = {k: v[val_idx].clone() for k, v in
↪data.items()}
    if not repair_memory:
        repair_memory = clone_class_memory(class_memory)
    if not validation_memory:
        validation_memory = clone_class_memory(class_memory)
    return repair_memory, validation_memory

```

```

@torch.no_grad()
def collect_zf_memory(model, class_memory, config, prototypes=None,
    ↪max_items=None):
    mem_ds = class_memory_to_tensor_dataset(class_memory)
    if mem_ds is None:
        return None
    X, y, t = mem_ds.tensors
    if max_items is not None and len(y) > max_items:
        idx = torch.randperm(len(y))[:max_items]
        X, y, t = X[idx], y[idx], t[idx]
    Zs, Ys, Ts, Qs = [], [], [], []
    loader = make_loader(TensorDataset(X, y, t), batch_size=BATCH_SIZE,
    ↪shuffle=False)
    model.eval()
    for xb, yb, tb in loader:
        xb, yb, tb = xb.to(DEVICE), yb.to(DEVICE), tb.to(DEVICE)
        for src_task_id in torch.unique(tb).detach().cpu().tolist():
            src_task_id = int(src_task_id)
            mask = (tb == src_task_id)
            if not mask.any():
                continue
            _, cache = forward_by_method(
                model, xb[mask], src_task_id, config,
                prototypes=prototypes, train=False, y=yb[mask],
    ↪return_cache=True
            )
            Zs.append(cache["zf"].detach().cpu())
            Ys.append(yb[mask].detach().cpu())
            Ts.append(tb[mask].detach().cpu())
            Qs.append(cache["q_context"].detach().cpu())
    if not Zs:
        return None
    return {
        "Z": torch.cat(Zs, dim=0).float(),
        "y": torch.cat(Ys, dim=0).long(),
        "task_id": torch.cat(Ts, dim=0).long(),
        "q_context": torch.cat(Qs, dim=0).float(),
    }

def compute_class_prototypes(memory_features, n_classes=N_CLASSES, eps=1e-8):
    Z = memory_features["Z"]
    y = memory_features["y"]
    mu = torch.zeros(n_classes, Z.shape[1])
    counts = torch.zeros(n_classes)
    valid = torch.zeros(n_classes, dtype=torch.bool)
    for cls in range(n_classes):

```

```

        mask = (y == cls)
        if mask.any():
            mu[cls] = Z[mask].mean(dim=0)
            counts[cls] = float(mask.sum())
            valid[cls] = True
        return mu, valid, counts

def compute_context_class_prototypes(memory_features, n_contexts=N_CONTEXTS,
    ↪n_classes=N_CLASSES):
    Z = memory_features["Z"]
    y = memory_features["y"]
    t = memory_features["task_id"]
    global_mu, global_valid, _ = compute_class_prototypes(memory_features,
    ↪n_classes=n_classes)
    mu = torch.zeros(n_contexts, n_classes, Z.shape[1])
    valid = torch.zeros(n_contexts, n_classes, dtype=torch.bool)
    for cid in range(n_contexts):
        for cls in range(n_classes):
            mask = (t == cid) & (y == cls)
            if mask.any():
                mu[cid, cls] = Z[mask].mean(dim=0)
                valid[cid, cls] = True
            elif global_valid[cls]:
                mu[cid, cls] = global_mu[cls]
                valid[cid, cls] = True
    return mu, valid, global_mu, global_valid

class GlobalLinearReadout(nn.Module):
    def __init__(self, base_head):
        super().__init__()
        self.linear = copy.deepcopy(base_head)
    def forward(self, zf, q_context=None):
        return self.linear(zf)

class PrototypeCosineReadout(nn.Module):
    def __init__(self, prototypes, valid, scale=COSINE_SCALE_INIT, bias=None):
        super().__init__()
        self.register_buffer("prototypes", F.normalize(prototypes.float(),
    ↪dim=-1))
        self.register_buffer("valid", valid.bool())
        self.scale = nn.Parameter(torch.tensor(float(scale)))
        self.bias = nn.Parameter(torch.zeros(N_CLASSES) if bias is None else
    ↪bias.float())
    def forward(self, zf, q_context=None):
        z = F.normalize(zf, dim=-1)
        logits = self.scale.clamp(1.0, 80.0) * (z @ self.prototypes.T) + self.
    ↪bias

```

```

        logits = logits.masked_fill(~self.valid.unsqueeze(0), -1e9)
        return logits

class PrototypeEuclideanReadout(nn.Module):
    def __init__(self, prototypes, valid, temperature=READOUT_TEMP, bias=None):
        super().__init__()
        self.register_buffer("prototypes", prototypes.float())
        self.register_buffer("valid", valid.bool())
        self.log_temperature = nn.Parameter(torch.log(torch.
↪tensor(float(temperature))))
        self.bias = nn.Parameter(torch.zeros(N_CLASSES) if bias is None else
↪bias.float())
    def forward(self, zf, q_context=None):
        diff = zf.unsqueeze(1) - self.prototypes.unsqueeze(0)
        dist = diff.pow(2).mean(dim=-1)
        temp = torch.exp(self.log_temperature).clamp(0.03, 5.0)
        logits = -dist / temp + self.bias
        logits = logits.masked_fill(~self.valid.unsqueeze(0), -1e9)
        return logits

class TrainablePrototypeEuclideanReadout(nn.Module):
    """Trainable prototype readout without per-dimension adapter.

    This separates the effect of learning class centers/bias/temperature from
↪the
    stronger diagonal local adapter.
    """
    def __init__(self, prototypes, valid, temperature=READOUT_TEMP):
        super().__init__()
        self.mu = nn.Parameter(prototypes.float().clone())
        self.bias = nn.Parameter(torch.zeros(N_CLASSES))
        self.log_temperature = nn.Parameter(torch.log(torch.
↪tensor(float(temperature))))
        self.register_buffer("valid", valid.bool())
    def forward(self, zf, q_context=None):
        diff = zf.unsqueeze(1) - self.mu.unsqueeze(0)
        dist = diff.pow(2).mean(dim=-1)
        temp = torch.exp(self.log_temperature).clamp(0.03, 5.0)
        logits = -dist / temp + self.bias
        logits = logits.masked_fill(~self.valid.unsqueeze(0), -1e9)
        return logits

class LocalAdapterPrototypeReadout(nn.Module):
    def __init__(self, prototypes, valid, temperature=READOUT_TEMP):
        super().__init__()
        self.mu = nn.Parameter(prototypes.float().clone())
        self.log_diag = nn.Parameter(torch.zeros_like(prototypes.float()))

```

```

        self.bias = nn.Parameter(torch.zeros(N_CLASSES))
        self.log_temperature = nn.Parameter(torch.log(torch.
↪tensor(float(temperature))))
        self.register_buffer("valid", valid.bool())
    def forward(self, zf, q_context=None):
        alpha = torch.exp(self.log_diag.clamp(-2.0, 2.0))
        diff = (zf.unsqueeze(1) - self.mu.unsqueeze(0)) * alpha.unsqueeze(0)
        dist = diff.pow(2).mean(dim=-1)
        temp = torch.exp(self.log_temperature).clamp(0.03, 5.0)
        logits = -dist / temp + self.bias
        logits = logits.masked_fill(~self.valid.unsqueeze(0), -1e9)
        return logits

class ContextPrototypeCosineReadout(nn.Module):
    def __init__(self, context_prototypes, context_valid,
↪scale=COSINE_SCALE_INIT):
        super().__init__()
        self.register_buffer("context_prototypes", F.
↪normalize(context_prototypes.float(), dim=-1))
        self.register_buffer("context_valid", context_valid.bool())
        self.scale = nn.Parameter(torch.tensor(float(scale)))
        self.bias = nn.Parameter(torch.zeros(N_CLASSES))
    def forward(self, zf, q_context=None):
        if q_context is None:
            q_context = torch.full((zf.shape[0], N_CONTEXTS), 1.0 / N_CONTEXTS,
↪device=zf.device)
            z = F.normalize(zf, dim=-1)
            # scores: batch x context x class
            scores = torch.einsum("bd,cyd->bcy", z, self.context_prototypes.to(zf.
↪device))
            scores = scores.masked_fill(~self.context_valid.to(zf.device).
↪unsqueeze(0), -1e9)
            logits = (q_context.unsqueeze(-1) * scores).sum(dim=1)
            logits = self.scale.clamp(1.0, 80.0) * logits + self.bias
            return logits

def build_readout_from_memory(model, class_memory, config, prototypes, variant,
↪seed=0):
    set_seed(seed)
    feats = collect_zf_memory(model, class_memory, config,
↪prototypes=prototypes)
    if feats is None:
        return GlobalLinearReadout(model.global_head), [{"repair_type":
↪"fallback_global_empty_memory"}]
    class_mu, class_valid, counts = compute_class_prototypes(feats)
    rows = [{

```



```

        "repair_type": "prototype_build",
        "variant": variant,
        "memory_items": int(len(feats["y"])),
        "valid_classes": int(class_valid.sum().item()),
    }]
    if variant == "local_proto_cosine":
        return PrototypeCosineReadout(class_mu, class_valid).to(DEVICE), rows
    if variant == "local_proto_euclidean":
        return PrototypeEuclideanReadout(class_mu, class_valid).to(DEVICE), rows
    if variant == "local_trainable_proto_ce_kl_035":
        return TrainablePrototypeEuclideanReadout(class_mu, class_valid).
↪to(DEVICE), rows
        if variant.startswith("local_diag_adapter") or variant ==_
↪"local_adapter_proto_ce_kl_035":
            return LocalAdapterPrototypeReadout(class_mu, class_valid).to(DEVICE),_
↪rows
        if variant == "context_local_proto_cosine":
            ctx_mu, ctx_valid, _gm, _gv = compute_context_class_prototypes(feats)
            return ContextPrototypeCosineReadout(ctx_mu, ctx_valid).to(DEVICE), rows
        raise ValueError(f"Unknown local readout variant: {variant}")

def sample_repair_batch_v10b(class_memory, batch_size=HEAD_REPAIR_BATCH):
    if not class_memory:
        return None, None, None
    return sample_balanced_class_memory(class_memory, batch_size)

def readout_ce_kl_loss(readout, zf, q_context, yb, pre_global_head=None,_
↪kl_weight=READOUT_KL_WEIGHT, use_kl=True, margin_weight=0.0):
    logits = readout(zf, q_context)
    ce = F.cross_entropy(logits, yb)
    kl = logits.sum() * 0.0
    if use_kl and pre_global_head is not None and kl_weight > 0:
        with torch.no_grad():
            pre_logits = pre_global_head(zf)
            kl = kl_distill_loss(logits, pre_logits,_
↪temperature=PROBE_DISTILL_TEMPERATURE)
            margin = head_margin_repair_loss(logits, yb) if margin_weight > 0 else_
↪logits.sum() * 0.0
            return ce + kl_weight * kl + margin_weight * margin, {"ce": ce, "kl": kl,_
↪"margin": margin}, logits

def train_readout_on_memory(readout, model, class_memory, config,_
↪prototypes=None, pre_global_head=None,
                            epochs=READOUT_TRAIN_EPOCHS,_
↪steps=READOUT_TRAIN_STEPS,

```

```

lr=READOUT_LR, kl_weight=READOUT_KL_WEIGHT,
↪use_kl=True,
margin_weight=0.0, variant="readout_train"):
    rows = []
    if not class_memory:
        return rows
    # Only train readout parameters. The fungal backbone remains frozen.
    model.eval()
    for p in model.parameters():
        p.requires_grad_(False)
    params = [p for p in readout.parameters() if p.requires_grad]
    if not params:
        return rows
    opt = torch.optim.Adam(params, lr=lr)
    for epoch in range(epochs):
        sums = {"loss": 0.0, "ce": 0.0, "kl": 0.0, "margin": 0.0}
        n = 0
        for _ in range(steps):
            xb, yb, tb = sample_repair_batch_v10b(class_memory)
            if xb is None:
                continue
            xb, yb, tb = xb.to(DEVICE), yb.to(DEVICE), tb.to(DEVICE)
            zf_list, q_list, y_list = [], [], []
            with torch.no_grad():
                for src_task_id in torch.unique(tb).detach().cpu().tolist():
                    src_task_id = int(src_task_id)
                    mask = (tb == src_task_id)
                    if not mask.any():
                        continue
                    _, cache = forward_by_method(model, xb[mask], src_task_id,
↪config, prototypes=prototypes, train=False, y=yb[mask], return_cache=True)
                    zf_list.append(cache["zf"].detach())
                    q_list.append(cache["q_context"].detach())
                    y_list.append(yb[mask].detach())
            if not zf_list:
                continue
            zf = torch.cat(zf_list, dim=0)
            q_context = torch.cat(q_list, dim=0)
            y_mix = torch.cat(y_list, dim=0)
            opt.zero_grad(set_to_none=True)
            loss, details, _ = readout_ce_kl_loss(readout, zf, q_context,
↪y_mix, pre_global_head=pre_global_head, kl_weight=kl_weight, use_kl=use_kl,
↪margin_weight=margin_weight)
            loss.backward()
            torch.nn.utils.clip_grad_norm_(params, max_norm=5.0)
            opt.step()
            sums["loss"] += float(loss.detach().cpu())

```

```

        sums["ce"] += float(details["ce"].detach().cpu())
        sums["kl"] += float(details["kl"].detach().cpu())
        sums["margin"] += float(details["margin"].detach().cpu())
        n += 1
    if n > 0:
        rows.append({
            "repair_epoch": epoch,
            "repair_type": variant,
            "readout_train_loss": sums["loss"] / n,
            "readout_train_ce": sums["ce"] / n,
            "readout_train_kl": sums["kl"] / n,
            "readout_train_margin": sums["margin"] / n,
            "kl_weight": kl_weight,
            "margin_weight": margin_weight,
        })
    return rows

@torch.no_grad()
def proxy_metrics_for_readout(model, readout, class_memory, config,
    ⇨ prototypes=None, max_items_per_class=VALIDATION_MAX_ITEMS_PER_CLASS):
    if not class_memory:
        return {"proxy_acc": np.nan, "proxy_class2": np.nan, "proxy_class5": np.
    ⇨ nan, "proxy_min_class": np.nan}
    correct_by_cls = {c: 0 for c in range(N_CLASSES)}
    total_by_cls = {c: 0 for c in range(N_CLASSES)}
    model.eval(); readout.eval()
    for cls, data in sorted(class_memory.items()):
        X, y, t = data["X"], data["y"], data["task_id"]
        if len(y) == 0:
            continue
        if len(y) > max_items_per_class:
            idx = torch.randperm(len(y))[:max_items_per_class]
            X, y, t = X[idx], y[idx], t[idx]
        loader = make_loader(TensorDataset(X, y, t), batch_size=BATCH_SIZE,
    ⇨ shuffle=False)
        for xb, yb, tb in loader:
            xb, yb, tb = xb.to(DEVICE), yb.to(DEVICE), tb.to(DEVICE)
            for src_task_id in torch.unique(tb).detach().cpu().tolist():
                src_task_id = int(src_task_id)
                mask = (tb == src_task_id)
                if not mask.any():
                    continue
                _, cache = forward_by_method(model, xb[mask], src_task_id,
    ⇨ config, prototypes=prototypes, train=False, y=yb[mask], return_cache=True)
                logits = readout(cache["zf"], cache["q_context"])
                seen = seen_classes_until(src_task_id)
                pred = mask_logits_to_seen_classes(logits, seen).argmax(dim=-1)

```

```

        yy = yb[mask]
        for c in torch.unique(yy).detach().cpu().tolist():
            c = int(c)
            m = (yy == c)
            total_by_cls[c] += int(m.sum().item())
            correct_by_cls[c] += int((pred[m] == yy[m]).sum().item())
    per_cls = {c: (correct_by_cls[c] / max(total_by_cls[c], 1) if
    ↪total_by_cls[c] > 0 else np.nan) for c in range(N_CLASSES)}
    total = sum(total_by_cls.values())
    correct = sum(correct_by_cls.values())
    valid_accs = [v for v in per_cls.values() if not np.isnan(v)]
    return {
        "proxy_acc": correct / max(total, 1),
        "proxy_class2": per_cls.get(2, np.nan),
        "proxy_class5": per_cls.get(5, np.nan),
        "proxy_min_class": float(np.min(valid_accs)) if valid_accs else np.nan,
    }

def validation_score(candidate, baseline):
    if candidate is None:
        return -1e9
    acc = candidate.get("proxy_acc", np.nan)
    c2 = candidate.get("proxy_class2", np.nan)
    minc = candidate.get("proxy_min_class", np.nan)
    base_acc = baseline.get("proxy_acc", np.nan)
    if np.isnan(acc):
        return -1e9
    damage = max(0.0, (base_acc if not np.isnan(base_acc) else acc) - acc)
    return float(
        acc
        + VALIDATION_SCORE_CLASS2_WEIGHT * (0.0 if np.isnan(c2) else c2)
        + VALIDATION_SCORE_MINCLASS_WEIGHT * (0.0 if np.isnan(minc) else minc)
        - VALIDATION_SCORE_DAMAGE_WEIGHT * damage
    )

def build_and_train_readout_variant(model, class_memory, config, prototypes,
    ↪variant, seed=0):
    """Return a readout and logs for one paired local-head variant."""
    set_seed(seed)
    repair_memory, validation_memory =
    ↪split_class_memory_for_readout_validation(class_memory, seed=seed + 100)
    pre_global_head = copy.deepcopy(model.global_head).to(DEVICE)
    pre_global_head.eval()
    for p in pre_global_head.parameters():
        p.requires_grad_(False)
    rows = []

```

```

if variant in ["no_repair_global_head", "kl_only_true_noop"]:
    readout = GlobalLinearReadout(model.global_head).to(DEVICE)
    rows.append({"repair_type": "no_op", "variant": variant})
    return readout, rows

if variant == "global_head_ce_kl_guard_035":
    readout = GlobalLinearReadout(model.global_head).to(DEVICE)
    rows.extend(train_readout_on_memory(readout, model, repair_memory,
↪config, prototypes=prototypes, pre_global_head=pre_global_head,
                                kl_weight=0.35, use_kl=True,
↪variant="global_head_ce_kl_guard_035"))
    return readout, rows

if variant in ["local_proto_cosine", "local_proto_euclidean",
↪"context_local_proto_cosine"]:
    readout, build_rows = build_readout_from_memory(model, repair_memory,
↪config, prototypes, variant, seed=seed)
    rows.extend(build_rows)
    return readout, rows

if variant == "local_adapter_proto_ce_kl_035":
    # Backward-compatible alias from v1.0a.
    readout, build_rows = build_readout_from_memory(model, repair_memory,
↪config, prototypes, "local_diag_adapter_ce_kl_035", seed=seed)
    rows.extend(build_rows)
    rows.extend(train_readout_on_memory(readout, model, repair_memory,
↪config, prototypes=prototypes, pre_global_head=pre_global_head,
                                kl_weight=0.35, use_kl=True,
↪margin_weight=0.0, variant="local_adapter_proto_ce_kl_035"))
    return readout, rows

if variant == "local_trainable_proto_ce_kl_035":
    readout, build_rows = build_readout_from_memory(model, repair_memory,
↪config, prototypes, variant, seed=seed)
    rows.extend(build_rows)
    rows.extend(train_readout_on_memory(readout, model, repair_memory,
↪config, prototypes=prototypes, pre_global_head=pre_global_head,
                                kl_weight=0.35, use_kl=True,
↪margin_weight=0.0, variant=variant))
    return readout, rows

if variant.startswith("local_diag_adapter_ce"):
    readout, build_rows = build_readout_from_memory(model, repair_memory,
↪config, prototypes, variant, seed=seed)
    rows.extend(build_rows)
    if variant == "local_diag_adapter_ce_only":

```

```

        kl_weight, use_kl, margin_weight = 0.0, False, 0.0
    elif variant == "local_diag_adapter_ce_kl_005":
        kl_weight, use_kl, margin_weight = 0.05, True, 0.0
    elif variant == "local_diag_adapter_ce_kl_015":
        kl_weight, use_kl, margin_weight = 0.15, True, 0.0
    elif variant == "local_diag_adapter_ce_kl_035":
        kl_weight, use_kl, margin_weight = 0.35, True, 0.0
    elif variant == "local_diag_adapter_ce_kl_060":
        kl_weight, use_kl, margin_weight = 0.60, True, 0.0
    elif variant == "local_diag_adapter_ce_kl_margin_035":
        kl_weight, use_kl, margin_weight = 0.35, True,
    LOCAL_ADAPTER_MARGIN_WEIGHT
    else:
        raise ValueError(f"Unknown local diagonal adapter variant:
    {variant}")
    rows.extend(train_readout_on_memory(readout, model, repair_memory,
    config, prototypes=prototypes, pre_global_head=pre_global_head,
        kl_weight=kl_weight, use_kl=use_kl,
    margin_weight=margin_weight, variant=variant))
    return readout, rows

    if variant in ["local_mycorrhizal_selected",
    "local_adapter_validation_selected"]:
        baseline_readout = GlobalLinearReadout(model.global_head).to(DEVICE)
        baseline_proxy = proxy_metrics_for_readout(model, baseline_readout,
    validation_memory, config, prototypes=prototypes)
        candidates = [
            "global_head_ce_kl_guard_035",
            "local_proto_euclidean",
            "local_trainable_proto_ce_kl_035",
            "local_diag_adapter_ce_only",
            "local_diag_adapter_ce_kl_005",
            "local_diag_adapter_ce_kl_015",
            "local_diag_adapter_ce_kl_035",
            "local_diag_adapter_ce_kl_060",
            "local_diag_adapter_ce_kl_margin_035",
        ]
        best_readout = baseline_readout
        best_name = "no_repair_global_head"
        best_proxy = baseline_proxy
        best_score = validation_score(baseline_proxy, baseline_proxy)
        rows.append({"repair_type": "selector_baseline", "candidate":
    best_name, **{f"baseline_{k}": v for k, v in baseline_proxy.items()}},
    validation_score": best_score)
        for cand in candidates:

```

```

        cand_readout, cand_rows = build_and_train_readout_variant(model,
↪repair_memory, config, prototypes, cand, seed=seed + 17)
        proxy = proxy_metrics_for_readout(model, cand_readout,
↪validation_memory, config, prototypes=prototypes)
        score = validation_score(proxy, baseline_proxy)
        rows.extend([{"r": r, "candidate": cand} for r in cand_rows])
        rows.append({
            "repair_type": "selector_candidate",
            "candidate": cand,
            "proxy_acc": proxy.get("proxy_acc", np.nan),
            "proxy_class2": proxy.get("proxy_class2", np.nan),
            "proxy_class5": proxy.get("proxy_class5", np.nan),
            "proxy_min_class": proxy.get("proxy_min_class", np.nan),
            "validation_score": score,
        })
        if score > best_score + VALIDATION_ACCEPTANCE_TOL:
            best_score = score
            best_readout = cand_readout
            best_name = cand
            best_proxy = proxy
        rows.append({
            "repair_type": "selector_selected",
            "selected_candidate": best_name,
            "selected_score": best_score,
            "selected_proxy_acc": best_proxy.get("proxy_acc", np.nan),
            "selected_proxy_class2": best_proxy.get("proxy_class2", np.nan),
            "selected_proxy_class5": best_proxy.get("proxy_class5", np.nan),
            "selected_proxy_min_class": best_proxy.get("proxy_min_class", np.
↪nan),
        })
        return best_readout, rows

    raise ValueError(f"Unknown readout variant: {variant}")

# -----
# v1.0c inferred-context / proto-to-oracle gap closure utilities
# -----

def method_name_for_context_variant(variant: str):
    return f"paired_{variant}"

def make_context_config(base_strategy="proto_first", temp=None,
↪blend_tuple=None):
    """Create an evaluation/training config for a context posterior variant.

    temp is applied to model.context_temperature outside this config.

```

```

blend_tuple is (name, single_weight, latent_weight, feature_weight).
"""
    if base_strategy == "oracle":
        cfg = copy.
    ↪deepcopy(METHOD_CONFIGS["mmals_oracle_context_balanced_replay"])
        return cfg
    cfg = copy.deepcopy(METHOD_CONFIGS["mmals_proto_first_balanced_replay"])
    cfg["strategy"] = "proto_first"
    if blend_tuple is not None:
        _name, bs, bl, bf = blend_tuple
        s = max(float(bs) + float(bl) + float(bf), 1e-8)
        cfg["batch_blend_single"] = float(bs) / s
        cfg["batch_blend_latent"] = float(bl) / s
        cfg["batch_blend_feature"] = float(bf) / s
        cfg["context_blend_name"] = str(_name)
    if temp is not None:
        cfg["selected_context_temperature"] = float(temp)
    return cfg

def set_model_context_temperature(model, temp):
    if hasattr(model, "context_temperature") and temp is not None:
        model.context_temperature = float(temp)

@torch.no_grad()
def proxy_metrics_for_context_config(model, class_memory, config,
    ↪prototypes=None, readout=None,
    ↪
    ↪max_items_per_class=VALIDATION_MAX_ITEMS_PER_CLASS):
    """Score a context-posterior config on memory validation, without final
    ↪test data.

    This combines readout accuracy on memory, context top-1 against stored
    ↪source task,
    context entropy/margin, class-2 accuracy, class-5 preservation and
    ↪min-class robustness.
    """
    if not class_memory:
        return {
            "proxy_acc": np.nan,
            "proxy_context_top1": np.nan,
            "proxy_context_entropy": np.nan,
            "proxy_context_margin": np.nan,
            "proxy_class2": np.nan,
            "proxy_class5": np.nan,

```



```

        "proxy_min_class": np.nan,
        "proxy_min_task": np.nan,
        "proxy_old_task_min": np.nan,
        "proxy_task_acc_std": np.nan,
        "proxy_score": -1e9,
    }
    if readout is None:
        readout = GlobalLinearReadout(model.global_head).to(DEVICE)
    model.eval(); readout.eval()
    correct_by_cls = {c: 0 for c in range(N_CLASSES)}
    total_by_cls = {c: 0 for c in range(N_CLASSES)}
    correct_by_task = {t: 0 for t in range(N_TASKS)}
    total_by_task = {t: 0 for t in range(N_TASKS)}
    correct_total, total = 0, 0
    top1_values, ent_values, margin_values = [], [], []
    for cls, data in sorted(class_memory.items()):
        X, y, t = data["X"], data["y"], data["task_id"]
        if len(y) == 0:
            continue
        if len(y) > max_items_per_class:
            idx = torch.randperm(len(y))[:max_items_per_class]
            X, y, t = X[idx], y[idx], t[idx]
        loader = make_loader(TensorDataset(X, y, t), batch_size=BATCH_SIZE,
↪shuffle=False)
        for xb, yb, tb in loader:
            xb, yb, tb = xb.to(DEVICE), yb.to(DEVICE), tb.to(DEVICE)
            for src_task_id in torch.unique(tb).detach().cpu().tolist():
                src_task_id = int(src_task_id)
                mask = (tb == src_task_id)
                if not mask.any():
                    continue
                _, cache = forward_by_method(
                    model, xb[mask], src_task_id, config,
                    prototypes=prototypes, train=False, y=yb[mask],
↪return_cache=True
                )
                logits = readout(cache["zf"], cache["q_context"])
                seen = seen_classes_until(src_task_id)
                pred = mask_logits_to_seen_classes(logits, seen).argmax(dim=-1)
                yy = yb[mask]
                correct = (pred == yy)
                correct_total += int(correct.sum().item())
                total += int(yy.numel())
                correct_by_task[src_task_id] += int(correct.sum().item())
                total_by_task[src_task_id] += int(yy.numel())
                q = cache["q_context"]

```

```

        top1_values.append((q.argmax(dim=-1) == src_task_id).float().
↪detach()).cpu())
        ent_values.append(entropy_from_probs(q).detach().cpu())
        margin_values.append(margin_from_probs(q).detach().cpu())
        for c in torch.unique(yy).detach().cpu().tolist():
            c = int(c)
            m = (yy == c)
            total_by_cls[c] += int(m.sum().item())
            correct_by_cls[c] += int((pred[m] == yy[m]).sum().item())
        per_cls = {c: (correct_by_cls[c] / max(total_by_cls[c], 1) if
↪total_by_cls[c] > 0 else np.nan) for c in range(N_CLASSES)}
        valid_accs = [v for v in per_cls.values() if not np.isnan(v)]
        acc = correct_total / max(total, 1)
        ctx_top1 = float(torch.cat(top1_values).mean()) if top1_values else np.nan
        ctx_entropy = float(torch.cat(ent_values).mean()) if ent_values else np.nan
        ctx_margin = float(torch.cat(margin_values).mean()) if margin_values else
↪np.nan
        min_class = float(np.min(valid_accs)) if valid_accs else np.nan
        per_task = {t: (correct_by_task[t] / max(total_by_task[t], 1) if
↪total_by_task[t] > 0 else np.nan) for t in range(N_TASKS)}
        valid_task_accs = [v for v in per_task.values() if not np.isnan(v)]
        min_task = float(np.min(valid_task_accs)) if valid_task_accs else np.nan
        active_tasks = [t for t, v in per_task.items() if not np.isnan(v)]
        max_active_task = max(active_tasks) if active_tasks else -1
        old_task_accs = [per_task[t] for t in active_tasks if t < max_active_task]
        old_task_min = float(np.min(old_task_accs)) if old_task_accs else min_task
        task_acc_std = float(np.std(valid_task_accs)) if valid_task_accs else np.nan
        c2 = per_cls.get(2, np.nan)
        c5 = per_cls.get(5, np.nan)
        score = float(
            acc
            + CONTEXT_SELECTION_SCORE_CONTEXT_WEIGHT * (0.0 if np.isnan(ctx_top1)
↪else ctx_top1)
            + CONTEXT_SELECTION_SCORE_CLASS2_WEIGHT * (0.0 if np.isnan(c2) else c2)
            + CONTEXT_SELECTION_SCORE_MINCLASS_WEIGHT * (0.0 if np.isnan(min_class)
↪else min_class)
            + 0.05 * (0.0 if np.isnan(min_task) else min_task)
            - CONTEXT_SELECTION_SCORE_ENTROPY_PENALTY * (0.0 if np.
↪isnan(ctx_entropy) else ctx_entropy)
        )
        return {
            "proxy_acc": acc,
            "proxy_context_top1": ctx_top1,
            "proxy_context_entropy": ctx_entropy,
            "proxy_context_margin": ctx_margin,
            "proxy_class2": c2,

```

```

        "proxy_class5": c5,
        "proxy_min_class": min_class,
        "proxy_min_task": min_task,
        "proxy_old_task_min": old_task_min,
        "proxy_task_acc_std": task_acc_std,
        "proxy_score": score,
    }

def candidate_context_configs(selection_mode="temp_blend"):
    """Return candidate (name, temp, blend_tuple) triples for context selection.
    ↪ """
    base_blend = ("base_proto_first", PROTO_FIRST_BLEND_SINGLE, ↪
    ↪PROTO_FIRST_BLEND_LATENT, PROTO_FIRST_BLEND_FEATURE)
    if selection_mode == "temp":
        return [(f"temp_{t}", t, base_blend) for t in ↪
    ↪CONTEXT_TEMP_SELECTION_GRID]
    if selection_mode == "blend":
        return [(f"blend_{name}", CONTEXT_TEMPERATURE, (name, bs, bl, bf)) for ↪
    ↪name, bs, bl, bf in CONTEXT_BLEND_SELECTION_GRID]
    if selection_mode == "temp_blend":
        # Cross-product is deliberately compact to keep the audit tractable.
        temps = [0.60, 0.80, 1.00, 1.25]
        blends = CONTEXT_BLEND_SELECTION_GRID
        return [(f"temp_{t}__blend_{name}", t, (name, bs, bl, bf)) for t in ↪
    ↪temps for name, bs, bl, bf in blends]
        raise ValueError("selection_mode must be temp, blend, or temp_blend")

def select_context_config_on_memory(model, class_memory, base_config, ↪
    ↪prototypes, selection_mode="temp", seed=0):
    """Select a context posterior config on held-out memory, never final test ↪
    ↪data."""
    set_seed(seed)
    _repair_memory, validation_memory = ↪
    ↪split_class_memory_for_readout_validation(class_memory, seed=seed + 303)
    saved_temp = float(getattr(model, "context_temperature", ↪
    ↪CONTEXT_TEMPERATURE))
    baseline_readout = GlobalLinearReadout(model.global_head).to(DEVICE)
    best = None
    rows = []
    for cand_name, temp, blend in candidate_context_configs(selection_mode):
        set_model_context_temperature(model, temp)
        cfg = make_context_config(base_strategy="proto_first", temp=temp, ↪
    ↪blend_tuple=blend)

```

```

    proxy = proxy_metrics_for_context_config(model, validation_memory, cfg,
↪prototypes=prototypes, readout=baseline_readout)
    row = {
        "repair_type": "context_config_candidate",
        "candidate": cand_name,
        "selection_mode": selection_mode,
        "candidate_temp": float(temp),
        "candidate_blend_name": str(blend[0]),
        "candidate_blend_single": float(cfg.get("batch_blend_single", np.
↪nan)),
        "candidate_blend_latent": float(cfg.get("batch_blend_latent", np.
↪nan)),
        "candidate_blend_feature": float(cfg.get("batch_blend_feature", np.
↪nan)),
        **proxy,
    }
    rows.append(row)
    if best is None or proxy["proxy_score"] > best["proxy_score"]:
        best = {**row, "config": cfg, "temp": float(temp), "blend": blend}
    if best is None:
        set_model_context_temperature(model, saved_temp)
        return copy.deepcopy(base_config), [{"repair_type":
↪"context_config_fallback", "selection_mode": selection_mode}]
    set_model_context_temperature(model, best["temp"])
    rows.append({
        "repair_type": "context_config_selected",
        "selection_mode": selection_mode,
        "selected_candidate": best["candidate"],
        "selected_temp": best["temp"],
        "selected_blend_name": best["candidate_blend_name"],
        "selected_blend_single": best["candidate_blend_single"],
        "selected_blend_latent": best["candidate_blend_latent"],
        "selected_blend_feature": best["candidate_blend_feature"],
        "selected_proxy_score": best["proxy_score"],
        "selected_proxy_acc": best["proxy_acc"],
        "selected_proxy_context_top1": best["proxy_context_top1"],
        "selected_proxy_class2": best["proxy_class2"],
        "selected_proxy_min_class": best["proxy_min_class"],
    })
    return best["config"], rows

```

2.20 9. Evaluation metrics and class diagnostics

The scorecard keeps the v0.14.x/v1.0 diagnostics: final average accuracy, minimum task accuracy, forgetting, class-2/class-4 ambiguity, class-5 preservation, predicted-class-4 bias, readout parameter cost, and probe gaps.

```

[58]: def mask_logits_to_seen_classes(logits, seen_classes):
    mask = torch.full_like(logits, -1e9)
    mask[:, seen_classes] = 0.0
    return logits + mask

@torch.no_grad()
def evaluate_task_with_readout(model, readout, dataset, task_id, config,
    ⇨ prototypes=None, batch_size=BATCH_SIZE, seen_classes=None):
    model.eval(); readout.eval()
    loader = make_loader(dataset, batch_size=batch_size, shuffle=False)
    correct, total = 0, 0
    context_entropies, context_margins, context_top1 = [], [], []
    context_briers, context_eces = [], []
    frechet_margins, latent_frechet_margins, prototype_weights = [], [], []
    class2_margin_24, class5_margin_54 = [], []
    class2_logit_rank, class5_logit_rank = [], []
    if seen_classes is None:
        seen_classes = list(range(N_CLASSES))
    for xb, yb in loader:
        xb, yb = xb.to(DEVICE), yb.to(DEVICE)
        _, cache = forward_by_method(model, xb, task_id, config,
    ⇨ prototypes=prototypes, train=False, y=yb, return_cache=True)
        logits = readout(cache["zf"], cache["q_context"])
        logits_eval = mask_logits_to_seen_classes(logits, seen_classes)
        pred = logits_eval.argmax(dim=-1)
        correct += (pred == yb).sum().item()
        total += yb.numel()
        q = cache["q_context"]
        context_entropies.append(entropy_from_probs(q).detach().cpu())
        context_margins.append(margin_from_probs(q).detach().cpu())
        context_top1.append((q.argmax(dim=-1) == task_id).float().detach().
    ⇨ cpu())
        context_briers.append(brier_from_probs(q, task_id).detach().cpu())
        context_eces.append(torch.tensor([float(ece_from_probs(q, task_id).
    ⇨ detach().cpu())]))
        if "frechet_margin" in cache:
            frechet_margins.append(cache["frechet_margin"].detach().cpu())
        if "latent_frechet_margin" in cache:
            latent_frechet_margins.append(cache["latent_frechet_margin"].
    ⇨ detach().cpu())
        if "prototype_weight" in cache:
            prototype_weights.append(cache["prototype_weight"].detach().cpu())
        if (yb == 2).any():
            m = (yb == 2)
            class2_margin_24.append((logits_eval[m, 2] - logits_eval[m, 4]).
    ⇨ detach().cpu())

```

```

        ranks = (logits_eval[m] > logits_eval[m, 2].unsqueeze(-1)).
↪sum(dim=-1) + 1
        class2_logit_rank.append(ranks.float().detach().cpu())
        if (yb == 5).any():
            m = (yb == 5)
            class5_margin_54.append((logits_eval[m, 5] - logits_eval[m, 4]).
↪detach().cpu())
            ranks = (logits_eval[m] > logits_eval[m, 5].unsqueeze(-1)).
↪sum(dim=-1) + 1
            class5_logit_rank.append(ranks.float().detach().cpu())
    return {
        "acc": correct / max(total, 1),
        "context_entropy": float(torch.cat(context_entropies).mean()) if
↪context_entropies else np.nan,
        "context_margin": float(torch.cat(context_margins).mean()) if
↪context_margins else np.nan,
        "context_top1_acc": float(torch.cat(context_top1).mean()) if
↪context_top1 else np.nan,
        "context_brier": float(torch.cat(context_briers).mean()) if
↪context_briers else np.nan,
        "context_ece": float(torch.cat(context_eces).mean()) if context_eces
↪else np.nan,
        "latent_frechet_margin": float(torch.cat(latent_frechet_margins).
↪mean()) if latent_frechet_margins else np.nan,
        "prototype_weight": float(torch.cat(prototype_weights).mean()) if
↪prototype_weights else 0.0,
        "class2_margin_vs4": float(torch.cat(class2_margin_24).mean()) if
↪class2_margin_24 else np.nan,
        "class5_margin_vs4": float(torch.cat(class5_margin_54).mean()) if
↪class5_margin_54 else np.nan,
        "class2_logit_rank": float(torch.cat(class2_logit_rank).mean()) if
↪class2_logit_rank else np.nan,
        "class5_logit_rank": float(torch.cat(class5_logit_rank).mean()) if
↪class5_logit_rank else np.nan,
    }

@torch.no_grad()
def evaluate_seen_tasks_with_readout(model, readout, tasks, upto_task, config,
↪prototypes=None, batch_size=BATCH_SIZE):
    seen = seen_classes_until(upto_task)
    return [
        evaluate_task_with_readout(model, readout, tasks[tid]["test"], tid,
↪config, prototypes=prototypes, batch_size=batch_size, seen_classes=seen)
        for tid in range(upto_task + 1)
    ]

```

```

def forgetting_from_history(history_df: pd.DataFrame):
    rows = []
    for (method, seed), group in history_df.groupby(["method", "seed"]):
        for after_task in sorted(group.after_task.unique()):
            sub = group[group.after_task == after_task]
            fs = []
            for task_id in sorted(sub.task_id.unique()):
                task_hist = group[(group.task_id == task_id) & (group.
↪after_task <= after_task)]
                best = task_hist.acc.max()
                current = sub[sub.task_id == task_id].acc.iloc[0]
                if task_id < after_task:
                    fs.append(best - current)
            rows.append({"method": method, "seed": seed, "after_task":
↪after_task, "avg_forgetting": float(np.mean(fs)) if fs else 0.0})
    return pd.DataFrame(rows)

@torch.no_grad()
def class_confusion_matrix_with_readout(model, readout, tasks, upto_task,
↪config, prototypes=None, batch_size=BATCH_SIZE):
    model.eval(); readout.eval()
    seen = seen_classes_until(upto_task)
    mat = torch.zeros(N_CLASSES, N_CLASSES, dtype=torch.long)
    for task_id in range(upto_task + 1):
        loader = make_loader(tasks[task_id]["test"], batch_size=batch_size,
↪shuffle=False)
        for xb, yb in loader:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            _, cache = forward_by_method(model, xb, task_id, config,
↪prototypes=prototypes, train=False, y=yb, return_cache=True)
            logits = readout(cache["zf"], cache["q_context"])
            logits_eval = mask_logits_to_seen_classes(logits, seen)
            pred = logits_eval.argmax(dim=-1).detach().cpu()
            yb_cpu = yb.detach().cpu()
            for yt, yp in zip(yb_cpu, pred):
                if int(yt) < N_CLASSES and int(yp) < N_CLASSES:
                    mat[int(yt), int(yp)] += 1

    rows = []
    for true_class in range(N_CLASSES):
        denom = mat[true_class].sum().item()
        for pred_class in range(N_CLASSES):
            rows.append({
                "true_class": true_class,
                "pred_class": pred_class,
                "count": int(mat[true_class, pred_class].item()),
                "rate": float(mat[true_class, pred_class].item() / max(denom,
↪1)),

```

```

    })
    return pd.DataFrame(rows)

def class_diagnostics_from_confusion(confusion_df: pd.DataFrame):
    if confusion_df is None or confusion_df.empty:
        return pd.DataFrame()
    rows = []
    for (method, seed), sub in confusion_df.groupby(["method", "seed"]):
        diag = {}
        for cls in range(N_CLASSES):
            m = sub[(sub.true_class == cls) & (sub.pred_class == cls)]
            diag[cls] = float(m.rate.iloc[0]) if not m.empty else np.nan
            pred4 = sub[sub.pred_class == 4].groupby("true_class")["rate"].sum()
            pred4_bias = float(pred4.reindex(range(N_CLASSES), fill_value=0.0).
↪mean())
            rows.append({
                "method": method,
                "seed": seed,
                "old_class_acc": float(np.nanmean([diag.get(0, np.nan), diag.get(1,
↪np.nan), diag.get(2, np.nan)])),
                "recent_class_acc": float(np.nanmean([diag.get(3, np.nan), diag.
↪get(4, np.nan), diag.get(5, np.nan)])),
                "class0_acc": diag.get(0, np.nan),
                "class1_acc": diag.get(1, np.nan),
                "class2_acc": diag.get(2, np.nan),
                "class3_acc": diag.get(3, np.nan),
                "class4_acc": diag.get(4, np.nan),
                "class5_acc": diag.get(5, np.nan),
                "min_class_acc": float(np.nanmin([diag.get(c, np.nan) for c in
↪range(N_CLASSES)])),
                "pred4_bias": pred4_bias,
            })
    return pd.DataFrame(rows)

```

2.21 9b. Biometric-style threshold diagnostics: ROC / DET / FAR-FRR / EER

This block adds post-selection diagnostics for MMALS policy analysis. The benchmark is multi-class, so FAR/FRR/EER are computed as **one-vs-rest** diagnostics for each class.

For class *c*: genuine samples have `true_class == c`; impostor samples have `true_class != c`; the score is the selected policy's probability assigned to class *c*. The exported curves are useful for boundary reliability analysis, especially for classes 2, 4, and 5. They are diagnostic-only and are not used for policy selection.

```

[59]: # -----
      # Biometric-style threshold diagnostics: score dumps, ROC/DET/FAR-FRR/EER.

```



```

# -----
# This is a multiclass one-vs-rest diagnostic. It is not a real biometric
# comparison protocol unless the dataset itself is built as genuine/impostor
# pairs.

@torch.no_grad()
def collect_score_dump_for_candidate(candidate, tasks, checkpoint,
    method_name=None, selected=False,
    final_checkpoint_only=True,
    max_batches=None):
    """Collect per-sample final-test scores for one selected/candidate policy.

    Selection has already happened before this function is called. This
    function is
    post-selection/post-evaluation diagnostics only and must never feed back
    into
    policy selection.
    """
    if not globals().get("ENABLE_BIOMETRIC_STYLE_THRESHOLD_DIAGNOSTICS", False):
        return pd.DataFrame()
    seed = int(checkpoint["seed"])
    after_task = int(checkpoint["after_task"])
    if final_checkpoint_only and after_task != N_TASKS - 1:
        return pd.DataFrame()

    model = candidate["model"]
    readout = candidate["readout"]
    config = candidate["config"]
    context_prototypes = candidate["context_prototypes"]
    method = method_name or candidate.get("method", "unknown_method")
    variant = candidate.get("variant", "unknown_variant")
    policy_family = candidate.get("policy_family", rc2b_policy_family(variant))
    if "rc2b_policy_family" in globals() else "unknown")
    seen = seen_classes_until(after_task)

    rows = []
    model.eval(); readout.eval()
    for task_id in range(after_task + 1):
        dataset = tasks[task_id]["test"]
        loader = make_loader(dataset, batch_size=BATCH_SIZE, shuffle=False)
        sample_offset = 0
        for batch_idx, (xb, yb) in enumerate(loader):
            if max_batches is not None and batch_idx >= max_batches:
                break
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            _, cache = forward_by_method(

```

```

        model, xb, task_id, config,
        prototypes=context_prototypes, train=False, y=yb,
↪return_cache=True
    )
    logits = readout(cache["zf"], cache.get("q_context"))
    logits_eval = mask_logits_to_seen_classes(logits, seen)
    probs = F.softmax(logits_eval, dim=-1)
    pred = logits_eval.argmax(dim=-1)
    probs_np = probs.detach().cpu().numpy()
    logits_np = logits_eval.detach().cpu().numpy()
    y_np = yb.detach().cpu().numpy()
    pred_np = pred.detach().cpu().numpy()
    ctx_entropy = entropy_from_probs(cache.get("q_context")).detach().
↪cpu().numpy() if cache.get("q_context") is not None else np.full(len(y_np),
↪np.nan)
    ctx_margin = margin_from_probs(cache.get("q_context")).detach().
↪cpu().numpy() if cache.get("q_context") is not None else np.full(len(y_np),
↪np.nan)
    ctx_top = cache.get("q_context").argmax(dim=-1).detach().cpu().
↪numpy() if cache.get("q_context") is not None else np.full(len(y_np), -1)

    batch_rows = []
    for i in range(len(y_np)):
        r = {
            "dataset": DATASET_NAME,
            "run_mode": RUN_MODE,
            "experiment_profile": EXPERIMENT_PROFILE,
            "seed": seed,
            "after_task": after_task,
            "task_id": int(task_id),
            "sample_id": int(sample_offset + i),
            "method": method,
            "variant": variant,
            "policy_family": policy_family,
            "selected_policy": bool(selected),
            "true_class": int(y_np[i]),
            "predicted_class": int(pred_np[i]),
            "correct": bool(pred_np[i] == y_np[i]),
            "context_entropy": float(ctx_entropy[i]) if np.
↪isfinite(ctx_entropy[i]) else np.nan,
            "context_margin": float(ctx_margin[i]) if np.
↪isfinite(ctx_margin[i]) else np.nan,
            "context_top": int(ctx_top[i]) if int(ctx_top[i]) >= 0 else
↪np.nan,

```

```

        "score_kind": globals().
↪get("THRESHOLD_DIAGNOSTICS_SCORE_KIND",
↪"softmax_probability_over_seen_classes"),
    }
    for c in range(N_CLASSES):
        r[f"score_class_{c}"] = float(probs_np[i, c])
        r[f"logit_class_{c}"] = float(logits_np[i, c]) if np.
↪isfinite(logits_np[i, c]) else np.nan
        batch_rows.append(r)
        rows.extend(batch_rows)
        sample_offset += len(y_np)
    return pd.DataFrame(rows)

def _auc_from_curve(fpr, tpr):
    fpr = np.asarray(fpr, dtype=float)
    tpr = np.asarray(tpr, dtype=float)
    mask = np.isfinite(fpr) & np.isfinite(tpr)
    if mask.sum() < 2:
        return np.nan
    fpr, tpr = fpr[mask], tpr[mask]
    order = np.argsort(fpr)
    return float(np.trapz(tpr[order], fpr[order]))

def compute_biometric_style_threshold_diagnostics(score_df, focus_classes=None,
↪grid_size=None):
    """Compute one-vs-rest FAR/FRR/EER curves from per-sample class scores."""
    if score_df is None or score_df.empty:
        return pd.DataFrame(), pd.DataFrame(), pd.DataFrame()
    focus_classes = list(range(N_CLASSES)) if focus_classes is None else
↪list(focus_classes)
    grid_size = int(grid_size or globals().
↪get("THRESHOLD_DIAGNOSTICS_GRID_SIZE", 501))
    thresholds = np.linspace(0.0, 1.0, grid_size)

    group_cols = [c for c in ["dataset", "run_mode", "experiment_profile",
↪"seed", "after_task", "method", "variant", "policy_family",
↪"selected_policy"] if c in score_df.columns]
    curve_rows = []
    eer_rows = []

    for group_key, g in score_df.groupby(group_cols, dropna=False):
        group = dict(zip(group_cols, group_key if isinstance(group_key, tuple)
↪else (group_key,)))
        y_true = g["true_class"].astype(int).to_numpy()

```

```

for cls in focus_classes:
    score_col = f"score_class_{cls}"
    if score_col not in g.columns:
        continue
    scores = g[score_col].astype(float).to_numpy()
    genuine = (y_true == int(cls))
    impostor = ~genuine
    n_gen = int(genuine.sum())
    n_imp = int(impostor.sum())
    if n_gen == 0 or n_imp == 0:
        continue
    fars, frrs, tprs = [], [], []
    for thr in thresholds:
        accepted = scores >= thr
        far = float((accepted & impostor).sum() / max(n_imp, 1))
        frr = float(((~accepted) & genuine).sum() / max(n_gen, 1))
        fars.append(far)
        frrs.append(frr)
        tprs.append(1.0 - frr)
        curve_rows.append({
            **group,
            "class_id": int(cls),
            "threshold": float(thr),
            "far": far,
            "frr": frr,
            "fpr": far,
            "tpr": 1.0 - frr,
            "n_genuine": n_gen,
            "n_impostor": n_imp,
        })
    fars_np = np.asarray(fars)
    frrs_np = np.asarray(frrs)
    idx = int(np.nanargmin(np.abs(fars_np - frrs_np)))
    auc = _auc_from_curve(fars_np, tprs)
    eer_rows.append({
        **group,
        "class_id": int(cls),
        "eer": float((fars_np[idx] + frrs_np[idx]) / 2.0),
        "eer_threshold": float(thresholds[idx]),
        "eer_far": float(fars_np[idx]),
        "eer_frr": float(frrs_np[idx]),
        "roc_auc": auc,
        "n_genuine": n_gen,
        "n_impostor": n_imp,
    })

```

Micro pooled one-vs-rest over all classes.

```

pooled_scores = []
pooled_labels = []
for cls in range(N_CLASSES):
    score_col = f"score_class_{cls}"
    if score_col not in g.columns:
        continue
    pooled_scores.append(g[score_col].astype(float).to_numpy())
    pooled_labels.append((y_true == int(cls)).astype(bool))
if pooled_scores:
    scores = np.concatenate(pooled_scores)
    labels = np.concatenate(pooled_labels)
    genuine = labels
    impostor = ~labels
    n_gen = int(genuine.sum())
    n_imp = int(impostor.sum())
    if n_gen > 0 and n_imp > 0:
        fars, frrs, tprs = [], [], []
        for thr in thresholds:
            accepted = scores >= thr
            far = float((accepted & impostor).sum() / max(n_imp, 1))
            frr = float(((~accepted) & genuine).sum() / max(n_gen, 1))
            fars.append(far); frrs.append(frr); tprs.append(1.0 - frr)
        idx = int(np.nanargmin(np.abs(np.asarray(fars) - np.
↪asarray(frrs))))
        eer_rows.append({
            **group,
            "class_id": "micro_all_classes",
            "eer": float((fars[idx] + frrs[idx]) / 2.0),
            "eer_threshold": float(thresholds[idx]),
            "eer_far": float(fars[idx]),
            "eer_frr": float(frrs[idx]),
            "roc_auc": _auc_from_curve(fars, tprs),
            "n_genuine": n_gen,
            "n_impostor": n_imp,
        })

curves_df = pd.DataFrame(curve_rows)
eer_by_class_df = pd.DataFrame(eer_rows)
if eer_by_class_df.empty:
    return curves_df, eer_by_class_df, pd.DataFrame()

class_only = eer_by_class_df[eer_by_class_df["class_id"].apply(lambda x:
↪isinstance(x, (int, np.integer)) or str(x).isdigit())].copy()
if not class_only.empty:
    class_only["class_id_int"] = class_only["class_id"].astype(int)
    method_summary_rows = []
    summary_group_cols = [c for c in group_cols if c in class_only.columns]

```

```

        for key, g in class_only.groupby(summary_group_cols, dropna=False):
            group = dict(zip(summary_group_cols, key if isinstance(key, tuple)
↪else (key,)))
            row = {
                **group,
                "macro_eer": float(g["eer"].mean()),
                "macro_auc": float(g["roc_auc"].mean()),
                "weak_boundary_eer_2_4_5": float(g[g["class_id_int"].isin([2,
↪4, 5]])["eer"].mean()),
                "weak_boundary_auc_2_4_5": float(g[g["class_id_int"].isin([2,
↪4, 5]])["roc_auc"].mean()),
            }
            for cls in range(N_CLASSES):
                sub = g[g["class_id_int"] == cls]
                row[f"class{cls}_eer"] = float(sub["eer"].iloc[0]) if not sub.
↪empty else np.nan
                row[f"class{cls}_auc"] = float(sub["roc_auc"].iloc[0]) if not
↪sub.empty else np.nan
                micro = eer_by_class_df[(eer_by_class_df["class_id"] ==
↪"micro_all_classes")]
                for col, val in group.items():
                    if col in micro.columns:
                        micro = micro[micro[col] == val]
                row["micro_eer"] = float(micro["eer"].iloc[0]) if not micro.empty
↪else np.nan
                row["micro_auc"] = float(micro["roc_auc"].iloc[0]) if not micro.
↪empty else np.nan
                method_summary_rows.append(row)
            eer_by_method_df = pd.DataFrame(method_summary_rows)
        else:
            eer_by_method_df = pd.DataFrame()
        return curves_df, eer_by_class_df, eer_by_method_df

def plot_biometric_style_threshold_diagnostics(curves_df, eer_by_method_df,
↪out_dir=None, focus_classes=None, key_methods=None):
    """Create compact diagnostic figures from curve/EER tables."""
    if curves_df is None or curves_df.empty:
        return []
    out_dir = Path(out_dir or (PLOTS_DIR / "threshold_diagnostics"))
    out_dir.mkdir(parents=True, exist_ok=True)
    focus_classes = list(focus_classes or globals().
↪get("THRESHOLD_DIAGNOSTICS_FOCUS_CLASSES", [2, 4, 5]))
    key_methods = key_methods or [RC2B_METHOD, "paired_context_gap_selected",
↪"paired_proto_global_no_repair", "paired_proto_global_head_ce_kl_guard_035"]
    written = []

```

```

    final_task = int(curves_df["after_task"].max()) if "after_task" in
↳curves_df.columns else None
    plot_df = curves_df.copy()
    if final_task is not None:
        plot_df = plot_df[plot_df["after_task"] == final_task]
    if "method" in plot_df.columns:
        plot_df = plot_df[plot_df["method"].isin(key_methods)] if key_methods
↳else plot_df

    # ROC-style plot for weak boundary classes.
    for cls in focus_classes:
        sub = plot_df[plot_df["class_id"].astype(str) == str(cls)]
        if sub.empty:
            continue
        fig = plt.figure(figsize=(7.5, 5.5))
        for method, g in sub.groupby("method"):
            # Average across seed/variant at each threshold.
            gg = g.groupby("threshold", as_index=False).agg(fpr=("fpr",
↳"mean"), tpr=("tpr", "mean"))
            plt.plot(gg["fpr"], gg["tpr"], label=str(method)[:40])
            plt.xlabel("FAR / FPR")
            plt.ylabel("TPR = 1 - FRR")
            plt.title(f"One-vs-rest ROC-style curve - class {cls}")
            plt.legend(fontsize=7)
            plt.grid(True, alpha=0.3)
            path = out_dir / f"roc_style_class{cls}_{RUN_MODE}.png"
            fig.savefig(path, dpi=160, bbox_inches="tight")
            plt.close(fig)
            written.append(path)

    # FAR/FRR plot.
    fig = plt.figure(figsize=(7.5, 5.5))
    for method, g in sub.groupby("method"):
        gg = g.groupby("threshold", as_index=False).agg(far=("far",
↳"mean"), frr=("frr", "mean"))
        plt.plot(gg["threshold"], gg["far"], linestyle="-",
↳label=f"{str(method)[:28]} FAR")
        plt.plot(gg["threshold"], gg["frr"], linestyle="--",
↳label=f"{str(method)[:28]} FRR")
        plt.xlabel("Acceptance threshold")
        plt.ylabel("Rate")
        plt.title(f"FAR/FRR threshold trade-off - class {cls}")
        plt.legend(fontsize=6)
        plt.grid(True, alpha=0.3)
        path = out_dir / f"far_frr_class{cls}_{RUN_MODE}.png"

```

```

fig.savefig(path, dpi=160, bbox_inches="tight")
plt.close(fig)
written.append(path)

# DET-style plot: FAR/FPR against FRR/FNR for threshold sweep.
fig = plt.figure(figsize=(7.5, 5.5))
for method, g in sub.groupby("method"):
    gg = g.groupby("threshold", as_index=False).agg(far=("far",
↪ "mean"), frr=("frr", "mean"))
    plt.plot(gg["far"], gg["frr"], label=str(method)[:40])
plt.xlabel("FAR / FPR")
plt.ylabel("FRR / FNR")
plt.title(f"DET-style FAR-vs-FRR curve - class {cls}")
plt.legend(fontsize=7)
plt.grid(True, alpha=0.3)
path = out_dir / f"det_style_class{cls}_{RUN_MODE}.png"
fig.savefig(path, dpi=160, bbox_inches="tight")
plt.close(fig)
written.append(path)

# EER by method bar plot.
if eer_by_method_df is not None and not eer_by_method_df.empty and "method"
↪ in eer_by_method_df.columns:
    sub = eer_by_method_df.copy()
    if final_task is not None and "after_task" in sub.columns:
        sub = sub[sub["after_task"] == final_task]
    if key_methods:
        sub = sub[sub["method"].isin(key_methods)]
    if not sub.empty:
        agg = sub.groupby("method", as_index=False).agg(
            macro_eer=("macro_eer", "mean"),
            weak_boundary_eer_2_4_5=("weak_boundary_eer_2_4_5", "mean"),
            micro_eer=("micro_eer", "mean"),
        ).sort_values("weak_boundary_eer_2_4_5", ascending=True)
        fig = plt.figure(figsize=(9, 5.5))
        x = np.arange(len(agg))
        width = 0.28
        plt.bar(x - width, agg["macro_eer"], width, label="macro EER")
        plt.bar(x, agg["weak_boundary_eer_2_4_5"], width,
↪ label="weak-boundary EER 2/4/5")
        plt.bar(x + width, agg["micro_eer"], width, label="micro EER")
        plt.xticks(x, [str(m)[:24] for m in agg["method"]], rotation=30,
↪ ha="right")
        plt.ylabel("EER lower is better")
        plt.title("Biometric-style one-vs-rest EER by method")
        plt.legend(fontsize=8)
        plt.grid(True, axis="y", alpha=0.3)

```



```

path = out_dir / f"eer_by_method_{RUN_MODE}.png"
fig.savefig(path, dpi=160, bbox_inches="tight")
plt.close(fig)
written.append(path)

return written

```

2.22 10. Paired same-backbone training and RC1b validation protocol

The protocol is inherited from v0.14.8e, v0.14.9, v1.0a, v1.0b, v1.0c-control-fix and the v1.0-RC1 robust harness, but the goal is now candidate freeze:

1. Train one proto-first MMALS backbone per seed.
2. Save checkpoints after every task.
3. Clone the same checkpoint.
4. Keep the fungal backbone fixed.
5. Compare no-repair, no-op, true global guard, context-only, rejected original RC1 diagnostic, RC1b context+guard candidate, and oracle references.
6. Evaluate whether the RC1b candidate remains superior or clearly robustness-preferable across several seeds.

This is not a new full-training recipe. It is a corrected freeze/validate harness for the first v1.0 benchmark candidate.

2.23 10b. Panel E — Learning-Signal Alignment audit

This diagnostic compares where MMALS **should** update against where gradients actually concentrate during training.

For each host, the notebook records:

- route weight;
- mutualistic gain proxy;
- memory-risk pressure;
- expected update importance;
- observed host/transform gradient norm;
- LSA = correlation between expected update allocation and observed update allocation.

Interpretation:

- LSA high: updates touch the modules that routes/memory/contribution suggest should adapt.
- LSA low: the model may still learn, but the update signal is poorly localized.
- LSA < 0: potentially incoherent learning signal: updates move toward modules that were not expected to carry the adaptation.

This is an audit-only metric. It does not change training, selection, Hydro, or final-test evaluation.

[60]:

```

# -----
# Panel E - Learning-Signal Alignment audit helpers.
# -----
# Diagnostic-only. These helpers are called after backpropagation and before
# the optimizer step, so they observe the actual gradient allocation that will

```

drive the update. They never feed back into training or selection.

```
def _lsa_safe_float(x, default=0.0):
    try:
        if torch.is_tensor(x):
            x = x.detach().cpu().item()
        x = float(x)
        if math.isnan(x) or math.isinf(x):
            return float(default)
        return x
    except Exception:
        return float(default)

def _lsa_normalize(v, eps=1e-12):
    v = np.asarray(v, dtype=float)
    v = np.where(np.isfinite(v), v, 0.0)
    v = np.maximum(v, 0.0)
    s = float(v.sum())
    if s <= eps:
        if len(v) == 0:
            return v
        return np.ones_like(v, dtype=float) / max(len(v), 1)
    return v / s

def _lsa_pearson(a, b, eps=1e-12):
    a = np.asarray(a, dtype=float)
    b = np.asarray(b, dtype=float)
    mask = np.isfinite(a) & np.isfinite(b)
    if mask.sum() < 2:
        return np.nan
    a = a[mask]
    b = b[mask]
    a = a - a.mean()
    b = b - b.mean()
    denom = float(np.sqrt((a * a).sum()) * np.sqrt((b * b).sum()))
    if denom <= eps:
        return np.nan
    return float((a * b).sum() / denom)

def _lsa_cosine(a, b, eps=1e-12):
    a = np.asarray(a, dtype=float)
    b = np.asarray(b, dtype=float)
    mask = np.isfinite(a) & np.isfinite(b)
    if mask.sum() < 2:
```

```

        return np.nan
    a = a[mask]
    b = b[mask]
    denom = float(np.linalg.norm(a) * np.linalg.norm(b))
    if denom <= eps:
        return np.nan
    return float(np.dot(a, b) / denom)

def _module_grad_l2_norm(module) -> float:
    total = 0.0
    try:
        for p in module.parameters():
            if p.grad is not None:
                g = p.grad.detach()
                total += float(torch.sum(g.float() * g.float()).detach().cpu())
    except Exception:
        return 0.0
    return float(math.sqrt(max(total, 0.0)))

def _memory_pressure_scalar(memory_loss, replay_loss, balanced_loss):
    # Old-memory preservation, context replay, and balanced replay are the
    # three training signals that indicate memory risk pressure. This is a
    # scalar pressure, not a final-test metric.
    vals = [_lsa_safe_float(memory_loss), _lsa_safe_float(replay_loss),
    ↪ _lsa_safe_float(balanced_loss)]
    return float(max(0.0, sum(vals)))

def learning_signal_alignment_rows_for_batch(
    model,
    cache,
    seed: int,
    task_id: int,
    epoch: int,
    batch_index: int,
    memory_loss,
    replay_loss,
    balanced_loss,
    grad_clip_norm=None,
):
    """Return per-host Learning-Signal Alignment rows for one training batch.

    Expected allocation combines:
    - route allocation: how much the router used a host;
    - mutualistic gain proxy: route-weighted cosine contribution to z_f;

```

- memory risk proxy: route allocation under old-memory pressure.

Observed allocation is the per-host gradient norm over host + transform_
modules.

Primary LSA is Pearson alignment between expected and observed allocations.
A cosine alignment is also recorded for non-negative allocation similarity.
"""

```
if not globals().get("ENABLE_LEARNING_SIGNAL_ALIGNMENT_AUDIT", False):
    return []
if not isinstance(model, MMALSClazzIncremental):
    return []
routes = cache.get("routes") if isinstance(cache, dict) else None
transformed = cache.get("transformed") if isinstance(cache, dict) else None
zf = cache.get("zf") if isinstance(cache, dict) else None
if routes is None or transformed is None or zf is None:
    return []

with torch.no_grad():
    routes_cpu = routes.detach().float().cpu()
    route_weight = routes_cpu.mean(dim=0).numpy()
    try:
        contrib_cos = F.cosine_similarity(
            transformed.detach().float(),
            zf.detach().float().unsqueeze(1),
            dim=-1,
        ).mean(dim=0).cpu().numpy()
    except Exception:
        contrib_cos = np.zeros_like(route_weight, dtype=float)

route_weight = np.asarray(route_weight, dtype=float)
contrib_cos = np.asarray(contrib_cos, dtype=float)
mutualistic_gain = route_weight * contrib_cos
positive_gain = np.maximum(mutualistic_gain, 0.0)
pressure = _memory_pressure_scalar(memory_loss, replay_loss, balanced_loss)
memory_risk = route_weight * pressure
memory_risk_allocation = _lsa_normalize(memory_risk) if pressure > 0 else
np.zeros_like(route_weight, dtype=float)

expected_raw = (
    float(globals().get("LSA_EXPECTED_ROUTE_WEIGHT", 0.55)) *
    _lsa_normalize(route_weight)
    + float(globals().get("LSA_EXPECTED_GAIN_WEIGHT", 0.25)) *
    _lsa_normalize(positive_gain)
    + float(globals().get("LSA_EXPECTED_MEMORY_RISK_WEIGHT", 0.20)) *
    memory_risk_allocation
)
expected_allocation = _lsa_normalize(expected_raw)
```

```

observed_raw = []
for h in range(model.n_hosts):
    host_norm = _module_grad_l2_norm(model.hosts[h])
    transform_norm = _module_grad_l2_norm(model.transforms[h])
    observed_raw.append(host_norm + transform_norm)
observed_raw = np.asarray(observed_raw, dtype=float)
observed_allocation = _lsa_normalize(observed_raw)

lsa_pearson = _lsa_pearson(expected_allocation, observed_allocation)
lsa_cosine = _lsa_cosine(expected_allocation, observed_allocation)

exp_low = np.nanquantile(expected_allocation, float(globals().
↳get("LSA_LOW_EXPECTED_QUANTILE", 0.35))) if len(expected_allocation) else 0.0
obs_high = np.nanquantile(observed_allocation, float(globals().
↳get("LSA_HIGH_OBSERVED_QUANTILE", 0.65))) if len(observed_allocation) else 0.
↳0
exp_high = np.nanquantile(expected_allocation, float(globals().
↳get("LSA_HIGH_OBSERVED_QUANTILE", 0.65))) if len(expected_allocation) else 0.
↳0
obs_low = np.nanquantile(observed_allocation, float(globals().
↳get("LSA_LOW_EXPECTED_QUANTILE", 0.35))) if len(observed_allocation) else 0.0
mem_high = np.nanquantile(memory_risk_allocation, float(globals().
↳get("LSA_HIGH_OBSERVED_QUANTILE", 0.65))) if len(memory_risk_allocation)
↳else 0.0

memory_risk_alloc = memory_risk_allocation
rows = []
for h in range(model.n_hosts):
    e = float(expected_allocation[h])
    o = float(observed_allocation[h])
    m = float(memory_risk_alloc[h])
    if e <= exp_low and o >= obs_high:
        verdict = "suspicious_over_update"
    elif e >= exp_high and o <= obs_low:
        verdict = "under_updated_expected_host"
    elif m >= mem_high and o >= obs_high and pressure > 0:
        verdict = "memory_risk_high_update"
    elif np.isfinite(lsa_pearson) and lsa_pearson < 0:
        verdict = "global_lsa_negative_risk"
    else:
        verdict = "OK"
    rows.append({
        "dataset": DATASET_NAME,
        "run_mode": RUN_MODE,
        "experiment_profile": EXPERIMENT_PROFILE,
        "seed": int(seed),

```

```

        "task_id": int(task_id),
        "epoch": int(epoch),
        "batch_index": int(batch_index),
        "host": f"H{h+1}",
        "host_id": int(h),
        "route_weight": float(route_weight[h]),
        "mutualistic_gain": float(mutualistic_gain[h]),
        "memory_pressure": float(pressure),
        "memory_risk": float(memory_risk[h]),
        "memory_risk_allocation": m,
        "expected_update_importance": e,
        "observed_update_norm": float(observed_raw[h]),
        "observed_update_allocation": o,
        "grad_clip_norm": _lsa_safe_float(grad_clip_norm, default=np.nan),
        "lsa_pearson": float(lsa_pearson) if np.isfinite(lsa_pearson) else
↪ np.nan,
        "lsa_cosine": float(lsa_cosine) if np.isfinite(lsa_cosine) else np.
↪ nan,
        "memory_risk_level": "high" if m >= mem_high and pressure > 0 else
↪ "low",
        "verdict": verdict,
    })
    return rows

def summarize_learning_signal_alignment(lsa_df: pd.DataFrame):
    if lsa_df is None or lsa_df.empty:
        return pd.DataFrame()
    df = lsa_df.copy()
    df["is_suspicious"] = df["verdict"].astype(str).str.
↪ contains("suspicious|negative|under_updated|risk", case=False, regex=True)
    group_cols = ["seed", "task_id", "host"]
    out = df.groupby(group_cols, as_index=False).agg(
        n_batches=("batch_index", "count"),
        mean_route_weight=("route_weight", "mean"),
        mean_mutualistic_gain=("mutualistic_gain", "mean"),
        mean_memory_risk=("memory_risk_allocation", "mean"),
        mean_expected_update_importance=("expected_update_importance", "mean"),
        mean_observed_update_allocation=("observed_update_allocation", "mean"),
        mean_lsa_pearson=("lsa_pearson", "mean"),
        mean_lsa_cosine=("lsa_cosine", "mean"),
        suspicious_rate=("is_suspicious", "mean"),
    )
    return out.sort_values(["seed", "task_id", "suspicious_rate",
↪ "mean_lsa_pearson"], ascending=[True, True, False, True])

```

```

[61]: def train_base_proto_first_checkpoints(seed: int):
    """Train one proto-first MMALS backbone and save checkpoints after every
    ↪ task."""
    set_seed(seed)
    clear_memory()
    tasks = build_stress_tasks(DATASET_NAME, seed=seed)
    config = METHOD_CONFIGS["mmals_proto_first_balanced_replay"]
    model = build_model(config)
    memory_buffers: Dict[int, TensorDataset] = {}
    class_memory: Dict[int, Dict[str, torch.Tensor]] = {}
    context_prototypes: Dict[int, Dict[str, torch.Tensor]] = {}
    teacher: Optional[nn.Module] = None
    loss_rows, checkpoints, lsa_rows = [], [], []
    started = time.time()
    for task_id, task in enumerate(tasks):
        opt = torch.optim.Adam(model.parameters(), lr=LR)
        loader = make_loader(task["train"], batch_size=BATCH_SIZE, shuffle=True)
        for epoch in range(EPOCHS_PER_TASK):
            model.train()
            sums = {"loss": 0.0, "task": 0.0, "memory": 0.0, "context": 0.0,
            ↪ "replay": 0.0, "balanced": 0.0, "fungal": 0.0}
            lsa_epoch_values, lsa_epoch_cosines, lsa_epoch_suspicious = [], [],
            ↪ []
            n_batches = 0
            for xb, yb in loader:
                xb, yb = xb.to(DEVICE), yb.to(DEVICE)
                noisy_context_id = sample_context_id(task_id)
                opt.zero_grad(set_to_none=True)
                logits, cache = forward_by_method(
                    model, xb, task_id, config,
                    prototypes=context_prototypes, train=True,
                    noisy_context_id=noisy_context_id, y=yb, return_cache=True
                )
                task_loss = F.cross_entropy(logits, yb)
                fungal_loss = LAMBDA_ENTROPY * entropy_loss(cache["routes"]) +
                ↪ LAMBDA_METABOLIC * metabolic_loss(cache["transformed"], cache["routes"])
                context_loss, _ = context_calibration_loss(model, cache,
                ↪ task_id, config, epoch)
                replay_loss, _ = context_replay_loss(model, teacher,
                ↪ memory_buffers, config, batch_size=min(CONTEXT_REPLAY_BATCH,
                ↪ MEMORY_PER_TASK))
                memory_loss, _ = old_memory_losses(model, teacher,
                ↪ memory_buffers, config, batch_size=min(BATCH_SIZE, MEMORY_PER_TASK),
                ↪ prototypes=context_prototypes)
                balanced_loss, _ = balanced_replay_ce_loss(model, class_memory,
                ↪ config, prototypes=context_prototypes, batch_size=BALANCED_REPLAY_BATCH)

```

```

        loss = task_loss + fungal_loss + context_loss + replay_loss +
memory_loss + balanced_loss
        loss.backward()
        grad_clip_norm = torch.nn.utils.clip_grad_norm_(model.
parameters(), max_norm=5.0)
        if globals().get("ENABLE_LEARNING_SIGNAL_ALIGNMENT_AUDIT",
False) and (n_batches % int(globals().get("LSA_BATCH_STRIDE", 1)) == 0):
            batch_lsa_rows = learning_signal_alignment_rows_for_batch(
                model=model,
                cache=cache,
                seed=seed,
                task_id=task_id,
                epoch=epoch,
                batch_index=n_batches,
                memory_loss=memory_loss,
                replay_loss=replay_loss,
                balanced_loss=balanced_loss,
                grad_clip_norm=grad_clip_norm,
            )
            if batch_lsa_rows:
                lsa_rows.extend(batch_lsa_rows)
                batch_lsa = batch_lsa_rows[0].get("lsa_pearson", np.nan)
                batch_lsa_cos = batch_lsa_rows[0].get("lsa_cosine", np.
nan)

                if np.isfinite(batch_lsa):
                    lsa_epoch_values.append(float(batch_lsa))
                if np.isfinite(batch_lsa_cos):
                    lsa_epoch_cosines.append(float(batch_lsa_cos))
                lsa_epoch_suspicious.append(any(str(r.get("verdict",
nan)).lower() != "ok" for r in batch_lsa_rows))
            opt.step()
            sums["loss"] += float(loss.detach().cpu())
            sums["task"] += float(task_loss.detach().cpu())
            sums["memory"] += float(memory_loss.detach().cpu())
            sums["context"] += float(context_loss.detach().cpu())
            sums["replay"] += float(replay_loss.detach().cpu())
            sums["balanced"] += float(balanced_loss.detach().cpu())
            sums["fungal"] += float(fungal_loss.detach().cpu())
            n_batches += 1
        loss_rows.append({
            "method": "base_proto_first_training",
            "seed": seed,
            "task_id": task_id,
            "epoch": epoch,
            "loss": sums["loss"] / max(n_batches, 1),
            "task_loss": sums["task"] / max(n_batches, 1),
            "memory_loss": sums["memory"] / max(n_batches, 1),

```



```

        "context_loss": sums["context"] / max(n_batches, 1),
        "context_replay_loss": sums["replay"] / max(n_batches, 1),
        "balanced_replay_loss": sums["balanced"] / max(n_batches, 1),
        "fungal_loss": sums["fungal"] / max(n_batches, 1),
        "context_temperature": float(getattr(model,
↪"context_temperature", np.nan)),
        "learning_signal_alignment_pearson": float(np.
↪nanmean(lsa_epoch_values)) if lsa_epoch_values else np.nan,
        "learning_signal_alignment_cosine": float(np.
↪nanmean(lsa_epoch_cosines)) if lsa_epoch_cosines else np.nan,
        "learning_signal_alignment_suspicious_fraction": float(np.
↪mean(lsa_epoch_suspicious)) if lsa_epoch_suspicious else np.nan,
    })
    print(
        f"[base_proto seed={seed}] task={task_id} epoch={epoch+1}/
↪{EPOCHS_PER_TASK} "
        f"loss={sums['loss']/max(n_batches,1):.4f} task={sums['task']/
↪max(n_batches,1):.4f} "
        f"memory={sums['memory']/max(n_batches,1):.4f}
↪balanced={sums['balanced']/max(n_batches,1):.4f} "
        f"lsa={float(np.nanmean(lsa_epoch_values)) if lsa_epoch_values
↪else float('nan'):.3f}"
    )
    memory_buffers[task_id] = build_memory_subset(task["train"],
↪MEMORY_PER_TASK, seed=seed + 1000 + task_id)
    class_memory = update_class_memory(class_memory, task["train"],
↪task_id, MEMORY_PER_CLASS, seed=seed + 2000 + task_id)
    context_prototypes = refresh_context_prototypes(model, memory_buffers)
    chosen_temp = calibrate_context_temperature(model, memory_buffers,
↪config)
    if config.get("temp_calibration", False):
        print(f"[base_proto seed={seed}] calibrated context temperature
↪after task {task_id}: {chosen_temp:.2f}")
    checkpoints.append({
        "seed": seed,
        "after_task": task_id,
        "model_state": cpu_state_dict(model),
        "context_temperature": float(getattr(model, "context_temperature",
↪CONTEXT_TEMPERATURE)),
        "memory_buffers": clone_memory_buffers(memory_buffers),
        "class_memory": clone_class_memory(class_memory),
        "context_prototypes": clone_prototypes(context_prototypes),
        "replay_memory_items": int(sum(len(v["y"]) for v in class_memory.
↪values()))),
    })
    teacher = clone_teacher(model)

```

```

        clear_memory()
    print(f"[base_proto seed={seed}] completed in {time.time() - started:.1f}s")
    return {
        "seed": seed,
        "tasks": tasks,
        "config": config,
        "checkpoints": checkpoints,
        "loss_df": pd.DataFrame(loss_rows),
        "lsa_df": pd.DataFrame(lsa_rows),
    }

def method_name_for_variant(variant: str):
    return f"paired_{variant}"

def apply_local_head_variant_to_checkpoint(base_run, checkpoint, variant: str):
    seed = int(checkpoint["seed"])
    after_task = int(checkpoint["after_task"])
    tasks = base_run["tasks"]
    config = copy.deepcopy(METHOD_CONFIGS["mmals_proto_first_balanced_replay"])
    method_name = method_name_for_variant(variant)
    model = restore_base_model_from_checkpoint(checkpoint, config)
    class_memory = clone_class_memory(checkpoint["class_memory"])
    context_prototypes = clone_prototypes(checkpoint["context_prototypes"])
    readout, repair_rows = build_and_train_readout_variant(
        model, class_memory, config, context_prototypes,
        variant=variant, seed=seed * 10_000 + after_task * 101
    )
    for rr in repair_rows:
        rr.update({
            "method": method_name,
            "seed": seed,
            "after_task": after_task,
            "task_id": after_task,
            "variant": variant,
            "paired_same_backbone": True,
            "local_mycorrhizal_head": not variant.startswith("global") and
↪variant not in ["no_repair_global_head", "kl_only_true_noop"],
        })
    evals = evaluate_seen_tasks_with_readout(model, readout, tasks,
↪upto_task=after_task, config=config, prototypes=context_prototypes)
    history_rows = []
    replay_memory_items = int(checkpoint.get("replay_memory_items", 0))
    readout_trainable = count_parameters(readout)
    readout_total = total_parameters(readout)
    for old_task_id, metrics in enumerate(evals):
        row = {
            "method": method_name,

```

```

        "seed": seed,
        "model_family": "mmals_local_readout",
        "after_task": after_task,
        "task_id": old_task_id,
        "task_name": tasks[old_task_id]["name"],
        "seen_classes": str(seen_classes_until(after_task)),
        "trainable_params": count_parameters(model) + readout_trainable,
        "total_params": total_parameters(model) + readout_total,
        "backbone_params": total_parameters(model),
        "readout_trainable_params": readout_trainable,
        "readout_total_params": readout_total,
        "model_size_mb_total": (total_parameters(model) + readout_total) * 4 / (1024 ** 2),
        "uses_balanced_replay": True,
        "uses_output_distill": True,
        "uses_local_mycorrhizal_head": not variant.startswith("global") and
        variant not in ["no_repair_global_head", "kl_only_true_noop"],
        "uses_global_head_refresh": variant ==
        "global_head_ce_kl_guard_035",
        "probe_row": False,
        "backbone_source": "same_proto_first_checkpoint",
        "probe_feature": "original_backbone_zf_custom_readout",
        "probe_context_mode": "proto_first",
        "replay_memory_items": replay_memory_items,
        "memory_per_class": MEMORY_PER_CLASS,
        "memory_per_task": MEMORY_PER_TASK,
        "paired_same_backbone": True,
        "readout_variant": variant,
    }
    row.update(metrics)
    history_rows.append(row)
    confusion_rows = []
    if after_task == N_TASKS - 1:
        cm_df = class_confusion_matrix_with_readout(model, readout, tasks,
        upto_task=after_task, config=config, prototypes=context_prototypes)
        cm_df["method"] = method_name
        cm_df["seed"] = seed
        cm_df["after_task"] = after_task
        cm_df["backbone_source"] = "same_proto_first_checkpoint"
        cm_df["readout_variant"] = variant
        confusion_rows = cm_df.to_dict("records")
        print(f"[{method_name}] seed={seed} after task {after_task}:
        {[round(m['acc'], 3) for m in evals]}")
    return pd.DataFrame(history_rows), pd.DataFrame(confusion_rows), pd.
    DataFrame(repair_rows)

```

```

def final_probe_artifact_from_base_run(base_run):
    final_cp = base_run["checkpoints"][-1]
    model = restore_base_model_from_checkpoint(final_cp,
METHOD_CONFIGS["mmals_proto_first_balanced_replay"])
    return {
        "method": "mmals_proto_first_balanced_replay",
        "seed": int(final_cp["seed"]),
        "model": model,
        "tasks": base_run["tasks"],
        "config": METHOD_CONFIGS["mmals_proto_first_balanced_replay"],
        "memory_buffers": clone_memory_buffers(final_cp["memory_buffers"]),
        "class_memory": clone_class_memory(final_cp["class_memory"]),
        "context_prototypes": clone_prototypes(final_cp["context_prototypes"]),
        "replay_memory_items": int(final_cp.get("replay_memory_items", 0)),
    }

# -----
# v1.0c paired context-head variant application
# -----

def context_variant_selection_mode(variant: str):
    if "temp_blend" in variant or variant == "context_gap_selected":
        return "temp_blend"
    if "blend_selected" in variant:
        return "blend"
    if "temp_selected" in variant:
        return "temp"
    return None

def is_oracle_context_reference(variant: str):
    return variant.startswith("oracle_")

def variant_uses_guarded_head(variant: str):
    """Return True only for variants that must actually train a guarded global
readout.

    v1.0c evidence exposed a control issue: the control named
    `proto_global_head_ce_kl_guard_035` did not contain the exact substring
    `head_guard_035`, so the previous helper treated it as a no-op.

    This control-fix accepts both naming conventions:
    - `head_guard_035` for context-selected guarded variants;
    - `global_head_ce_kl_guard_035` for the true global guarded control and
readout reference.
    """

```

```

return (
    "head_guard_035" in variant
    or "global_head_ce_kl_guard_035" in variant
    or variant == "context_gap_selected"
)

def build_context_variant_config_and_logs(model, class_memory,
    ↪context_prototypes, variant, seed=0):
    """Return config/logs for one deployable or reference context variant."""
    if is_oracle_context_reference(variant):
        cfg = make_context_config(base_strategy="oracle")
        return cfg, [{"repair_type": "context_reference", "variant": variant,
    ↪"uses_oracle_context_reference": True}]
    cfg = copy.deepcopy(METHOD_CONFIGS["mmals_proto_first_balanced_replay"])
    mode = context_variant_selection_mode(variant)
    if mode is None:
        return cfg, [{"repair_type": "context_no_change", "variant": variant,
    ↪"uses_oracle_context_reference": False}]
    selected_cfg, rows = select_context_config_on_memory(model, class_memory,
    ↪cfg, context_prototypes, selection_mode=mode, seed=seed)
    for r in rows:
        r.update({"variant": variant, "uses_oracle_context_reference": False})
    return selected_cfg, rows

def apply_context_head_variant_to_checkpoint(base_run, checkpoint, variant:
    ↪str):
    seed = int(checkpoint["seed"])
    after_task = int(checkpoint["after_task"])
    tasks = base_run["tasks"]
    method_name = method_name_for_context_variant(variant)

    # Start from the exact same checkpoint.
    base_cfg = METHOD_CONFIGS["mmals_proto_first_balanced_replay"]
    model = restore_base_model_from_checkpoint(checkpoint, base_cfg)
    class_memory = clone_class_memory(checkpoint["class_memory"])
    context_prototypes = clone_prototypes(checkpoint["context_prototypes"])

    # Select / set the context posterior configuration.
    config, config_rows = build_context_variant_config_and_logs(
        model, class_memory, context_prototypes, variant=variant,
        seed=seed * 10_000 + after_task * 313
    )

    # Build readout. v1.0-RC1b primarily uses global readout controls; local
    ↪adapters are not the target here.

```

```

readout = GlobalLinearReadout(model.global_head).to(DEVICE)
repair_rows = list(config_rows)

if variant_uses_guarded_head(variant):
    repair_memory, _validation_memory =
↪split_class_memory_for_readout_validation(class_memory, seed=seed * 10_000 +
↪after_task * 101)
    pre_global_head = copy.deepcopy(model.global_head).to(DEVICE)
    pre_global_head.eval()
    for p in pre_global_head.parameters():
        p.requires_grad_(False)
    repair_rows.extend(train_readout_on_memory(
        readout, model, repair_memory, config,
        prototypes=context_prototypes, pre_global_head=pre_global_head,
        kl_weight=0.35, use_kl=True, margin_weight=0.0,
        variant=f"{variant}_global_head_guard"
    ))

for rr in repair_rows:
    rr.update({
        "method": method_name,
        "seed": seed,
        "after_task": after_task,
        "task_id": after_task,
        "variant": variant,
        "paired_same_backbone": True,
        "context_gap_closure_audit": True,
        "uses_oracle_context_reference":
↪is_oracle_context_reference(variant),
        "uses_context_selection": context_variant_selection_mode(variant)
↪is not None,
        "uses_guarded_global_head": variant_uses_guarded_head(variant),
        "true_guard_control_fixed": variant ==
↪"proto_global_head_ce_kl_guard_035",
    })

evals = evaluate_seen_tasks_with_readout(
    model, readout, tasks, upto_task=after_task,
    config=config, prototypes=context_prototypes
)

history_rows = []
replay_memory_items = int(checkpoint.get("replay_memory_items", 0))
readout_trainable = count_parameters(readout)
readout_total = total_parameters(readout)
for old_task_id, metrics in enumerate(evals):
    row = {

```

```

        "method": method_name,
        "seed": seed,
        "model_family": "mmals_context_readout",
        "after_task": after_task,
        "task_id": old_task_id,
        "task_name": tasks[old_task_id]["name"],
        "seen_classes": str(seen_classes_until(after_task)),
        "trainable_params": count_parameters(model) + readout_trainable,
        "total_params": total_parameters(model) + readout_total,
        "backbone_params": total_parameters(model),
        "readout_trainable_params": readout_trainable,
        "readout_total_params": readout_total,
        "model_size_mb_total": (total_parameters(model) + readout_total) * 4 / (1024 ** 2),
        "uses_balanced_replay": True,
        "uses_output_distill": True,
        "uses_local_mycorrhizal_head": False,
        "uses_global_head_refresh": variant_uses_guarded_head(variant),
        "uses_context_selection": context_variant_selection_mode(variant)
        is not None,
        "uses_oracle_context_reference":
        is_oracle_context_reference(variant),
        "probe_row": False,
        "backbone_source": "same_proto_first_checkpoint",
        "probe_feature": "original_backbone_zf_context_variant",
        "probe_context_mode": "oracle_reference" if
        is_oracle_context_reference(variant) else "proto_first_variant",
        "replay_memory_items": replay_memory_items,
        "memory_per_class": MEMORY_PER_CLASS,
        "memory_per_task": MEMORY_PER_TASK,
        "paired_same_backbone": True,
        "readout_variant": variant,
        "context_variant": variant,
        "context_temperature_used": float(getattr(model,
        "context_temperature", np.nan)),
        "batch_blend_single_used": float(config.get("batch_blend_single",
        np.nan)),
        "batch_blend_latent_used": float(config.get("batch_blend_latent",
        np.nan)),
        "batch_blend_feature_used": float(config.get("batch_blend_feature",
        np.nan)),
    }
    row.update(metrics)
    history_rows.append(row)

confusion_rows = []

```

```

if after_task == N_TASKS - 1:
    cm_df = class_confusion_matrix_with_readout(
        model, readout, tasks, upto_task=after_task,
        config=config, prototypes=context_prototypes
    )
    cm_df["method"] = method_name
    cm_df["seed"] = seed
    cm_df["after_task"] = after_task
    cm_df["backbone_source"] = "same_proto_first_checkpoint"
    cm_df["readout_variant"] = variant
    cm_df["context_variant"] = variant
    cm_df["uses_oracle_context_reference"] = 1
    is_oracle_context_reference(variant)
    confusion_rows = cm_df.to_dict("records")

    print(f"[{method_name}] seed={seed} after task {after_task}:")
    for m in evals:
        round(m['acc'], 3)
    return pd.DataFrame(history_rows), pd.DataFrame(confusion_rows), pd.
    DataFrame(repair_rows)

```

2.24 10c. RC2c Hydro audit helpers

This block adds Hydro diagnostics to the strict no-leakage policy-selection harness.

Depending on EXPERIMENT_PROFILE, Hydro can be:

- disabled (RC2b_STRICT),
- audit-only (RC3_HYDRO_AUDIT, default),
- used as a training regularizer (RC3_HYDRO_CANDIDATE),
- used as a validation-time stability penalty (RC3_HYDRO_STABILITY_SELECTED).

The audit computes, on **policy-validation memory only**:

- route drift D_r ;
- latent drift D_z ;
- output drift D_y ;
- route speed and latent speed;
- adaptive viscosity estimate ν ;
- Reynolds-like instability;
- turbulence-like score;
- wave/context entropy.

By default these metrics are reported and exported. They do not alter selection unless the RC3_HYDRO_STABILITY_SELECTED profile is used.

```

[62]: # -----
# RC3 Hydro audit layer: validation-memory-only drift/turbulence metrics.
# -----

```



```

def rc3_hydro_context_entropy(q, eps=1e-8):
    if q is None:
        return torch.tensor(float("nan"))
    return -(q * torch.log(q + eps)).sum(dim=-1)

def rc3_hydro_adaptive_viscosity(importance, base=None, max_v=None):
    base = RC3_HYDRO_BASE_VISCOSITY if base is None else float(base)
    max_v = RC3_HYDRO_MAX_VISCOSITY if max_v is None else float(max_v)
    return torch.clamp(base + (max_v - base) * importance, base, max_v)

@torch.no_grad()
def rc3_hydro_importance(reference_logits, reference_routes):
    probs = F.softmax(reference_logits, dim=-1)
    confidence = probs.max(dim=-1).values.mean()
    if reference_routes is None:
        dominance = torch.tensor(1.0, device=reference_logits.device)
    else:
        dominance = reference_routes.max(dim=-1).values.mean()
    return torch.clamp(confidence * dominance, 0.0, 1.0)

def rc3_hydro_kl(student_logits, teacher_logits,
    ↪temperature=DISTILL_TEMPERATURE):
    log_p = F.log_softmax(student_logits / temperature, dim=-1)
    q = F.softmax(teacher_logits / temperature, dim=-1)
    return (temperature ** 2) * F.kl_div(log_p, q, reduction="batchmean")

@torch.no_grad()
def rc3_eval_candidate_cache(candidate, xb, src_task_id, yb=None):
    model = candidate["model"]
    readout = candidate["readout"]
    config = candidate["config"]
    prototypes = candidate["context_prototypes"]
    model.eval(); readout.eval()
    _, cache = forward_by_method(
        model, xb, int(src_task_id), config,
        prototypes=prototypes, train=False, y=yb, return_cache=True
    )
    logits = readout(cache["zf"], cache.get("q_context"))
    return logits, cache

def rc3_pairwise_hydro_proxy(candidate, reference_candidate, class_memory,
    ↪max_items_per_class=None):

```

"""Compute Hydro drift of candidate versus a reference policy on validation memory.

The reference is usually the no-repair/global-head candidate for the same checkpoint.

*This is a **selection-memory** audit, not a final-test audit.*
"""

```
if not RC3_HYDRO_AUDIT_ENABLED or not class_memory:
    return {}
max_items_per_class = RC3_HYDRO_MAX_ITEMS_PER_CLASS if max_items_per_class is None else int(max_items_per_class)

rows = []
for cls, data in sorted(class_memory.items()):
    X, y, t = data["X"], data["y"], data["task_id"]
    if len(y) == 0:
        continue
    if len(y) > max_items_per_class:
        # Deterministic compact sampling based on candidate and class.
        # Deterministic SHA-based sample; no Python hash().
        seed_int = int(hashlib.sha256(f"{candidate.get('variant')}:{cls}").
        encode()).hexdigest()[:16], 16) % (2**31-1)
        gen = torch.Generator().manual_seed(seed_int)
        idx = torch.randperm(len(y), generator=gen)[:max_items_per_class]
        X, y, t = X[idx], y[idx], t[idx]
        loader = make_loader(TensorDataset(X, y, t), batch_size=BATCH_SIZE, shuffle=False)
        for xb, yb, tb in loader:
            xb, yb, tb = xb.to(DEVICE), yb.to(DEVICE), tb.to(DEVICE)
            for src_task_id in torch.unique(tb).detach().cpu().tolist():
                src_task_id = int(src_task_id)
                mask = tb == src_task_id
                if not mask.any():
                    continue
                c_logits, c_cache = rc3_eval_candidate_cache(candidate,
                xb[mask], src_task_id, yb[mask])
                r_logits, r_cache =
                rc3_eval_candidate_cache(reference_candidate, xb[mask], src_task_id,
                yb[mask])

                c_routes, r_routes = c_cache.get("routes"), r_cache.
                get("routes")
                if c_routes is None or r_routes is None:
                    route_drift = torch.tensor(0.0, device=xb.device)
                else:
                    route_drift = F.mse_loss(c_routes, r_routes)
```

```

        latent_drift = F.mse_loss(c_cache["zf"], r_cache["zf"])
        output_drift = rc3_hydro_kl(c_logits, r_logits)
        importance = rc3_hydro_importance(r_logits, r_routes)
        nu = rc3_hydro_adaptive_viscosity(importance)
        route_speed = torch.sqrt(torch.clamp(route_drift, min=0.0))
        latent_speed = torch.sqrt(torch.clamp(latent_drift, min=0.0))
        reynolds_like = (route_speed + latent_speed) / (nu +
↪RC3_HYDRO_TURBULENCE_EPS)
        turbulence_score = reynolds_like * (1.0 + output_drift.detach())
        q = c_cache.get("q_context")
        wave_entropy = rc3_hydro_context_entropy(q).mean() if q is not
↪None else torch.tensor(float("nan"), device=xb.device)
        rows.append({
            "route_drift_Dr": float(route_drift.detach().cpu()),
            "latent_drift_Dz": float(latent_drift.detach().cpu()),
            "output_drift_Dy": float(output_drift.detach().cpu()),
            "teacher_importance": float(importance.detach().cpu()),
            "adaptive_viscosity_nu": float(nu.detach().cpu()),
            "route_speed": float(route_speed.detach().cpu()),
            "latent_speed": float(latent_speed.detach().cpu()),
            "reynolds_like": float(reynolds_like.detach().cpu()),
            "turbulence_score": float(turbulence_score.detach().cpu()),
            "wave_entropy": float(wave_entropy.detach().cpu()) if not
↪torch.isnan(wave_entropy) else np.nan,
        })
        if not rows:
            return {}
        df = pd.DataFrame(rows)
        out = {}
        for col in df.columns:
            out[f"hydro_{col}"] = float(df[col].mean())
        out["hydro_audit_memory_only"] = True
        out["hydro_audit_reference_variant"] = reference_candidate.get("variant",
↪"unknown")
        return out

def rc3_hydro_selection_penalty(proxy):
    """Optional Hydro penalty. Default weights are zero: audit-only mode."""
    if (not RC3_HYDRO_AUDIT_ENABLED) or (not globals().
↪get("RC3_HYDRO_SELECTION_ENABLED", False)) or RC3_HYDRO_POLICY_MODE !=
↪"stability_aware":
        return 0.0
    turb = proxy.get("hydro_turbulence_score", np.nan)
    lat = proxy.get("hydro_latent_drift_Dz", np.nan)
    route = proxy.get("hydro_route_drift_Dr", np.nan)
    penalty = 0.0

```

```

if not np.isnan(turb):
    penalty += RC3_HYDRO_SELECTION_TURBULENCE_WEIGHT * float(turb)
if not np.isnan(lat):
    penalty += RC3_HYDRO_SELECTION_LATENT_DRIFT_WEIGHT * float(lat)
if not np.isnan(route):
    penalty += RC3_HYDRO_SELECTION_ROUTE_DRIFT_WEIGHT * float(route)
return float(penalty)

```

2.25 10b. RC2c strict no-leakage policy-selection utilities

These functions implement the strict RC2c rule.

The important rule is:

Candidate policy selection uses only memory-derived repair/context/policy-validation partitions. Final test sets are evaluated only after the policy has already been selected.

For auditability, the notebook can still evaluate all candidate policies after the selection decision. Those candidate final-test metrics are diagnostic only and are not used by the selector.

```

[63]: # -----
# v1.1-RC3 Hydro-audited strict no-leakage policy-selection utilities.
# -----

def rc2b_candidate_method_name(variant: str) -> str:
    return method_name_for_context_variant(variant)

def rc2b_policy_family(variant: str) -> str:
    if variant in ["proto_global_no_repair", "proto_kl_only_true_noop"]:
        return "proto_no_repair"
    if variant in ["context_blend_selected_global", ↵
↵ "context_temp_blend_selected_global"]:
        return "context_only_global"
    if variant == "proto_global_head_ce_kl_guard_035":
        return "proto_true_global_guard"
    if variant in ["context_temp_blend_selected_head_guard_035", ↵
↵ "context_gap_selected"]:
        return "rc1b_guarded_context_gap"
    return "other"

def rc2b_policy_label(variant: str) -> str:
    labels = {
        "proto_no_repair": "proto-first no-repair / copied global head",
        "context_only_global": "validation-selected context-only global head",
        "proto_true_global_guard": "proto-first true CE/KL guarded global head",
        "rc1b_guarded_context_gap": "RC1b guarded context-gap",
    }

```

```

        "other": f"other policy: {variant}",
    }
    return labels.get(rc2b_policy_family(variant), str(variant))

def rc2b_policy_complexity_rank(variant: str) -> int:
    # Lower is simpler. Used only as a tie-breaker after validation score.
    fam = rc2b_policy_family(variant)
    if fam == "proto_no_repair":
        return 0
    if fam == "context_only_global":
        return 1
    if fam == "proto_true_global_guard":
        return 2
    if fam == "rc1b_guarded_context_gap":
        return 3
    return 9

def split_class_memory_three_way(class_memory, seed=0,
                                repair_fraction=RC2B_POLICY_REPAIR_FRACTION,
                                context_fraction=RC2B_CONTEXT_SELECTION_FRACTION):
    """Split class memory into repair, context-selection and policy-validation
    memory.

    This is the core RC2b no-leakage guard:
    - repair_memory trains guarded heads;
    - context_selection_memory selects context temp/blend when needed;
    - policy_validation_memory selects the deployable policy;
    - final test is not touched until after selection.
    """
    rng = np.random.default_rng(seed)
    repair, context_select, policy_val = {}, {}, {}
    for cls, data in class_memory.items():
        X, y, t = data["X"], data["y"], data["task_id"]
        n = len(y)
        if n == 0:
            continue
        idx = np.arange(n)
        rng.shuffle(idx)
        n_repair = max(1, int(round(n * repair_fraction))) if n >= 3 else
        max(1, n - 1)
        n_context = max(1, int(round(n * context_fraction))) if n - n_repair >=
        2 else max(0, n - n_repair - 1)
        n_repair = min(n_repair, n)
        n_context = min(n_context, max(0, n - n_repair))

```

```

repair_idx = idx[:n_repair]
context_idx = idx[n_repair:n_repair + n_context]
policy_idx = idx[n_repair + n_context:]
if len(policy_idx) == 0:
    # Make sure policy validation exists, even in tiny smoke memory.
    if len(context_idx) > 0:
        policy_idx = context_idx[-1:]
        context_idx = context_idx[:-1]
    elif len(repair_idx) > 1:
        policy_idx = repair_idx[-1:]
        repair_idx = repair_idx[:-1]
def pack(sub_idx):
    if len(sub_idx) == 0:
        return None
    tidx = torch.as_tensor(sub_idx, dtype=torch.long)
    return {"X": X[tidx].clone(), "y": y[tidx].clone(), "task_id":
↪t[tidx].clone()}
    r = pack(repair_idx)
    c = pack(context_idx)
    p = pack(policy_idx)
    if r is not None:
        repair[int(cls)] = r
    if c is not None:
        context_select[int(cls)] = c
    if p is not None:
        policy_val[int(cls)] = p
    # Fallback: if context selection is empty because smoke memory is tiny, use
↪policy validation.
    if not context_select:
        context_select = clone_class_memory(policy_val)
    return repair, context_select, policy_val

def rc2c_is_finite(x):
    try:
        return np.isfinite(float(x))
    except Exception:
        return False

def rc2c_proxy_value(proxy, key, default=np.nan):
    try:
        v = proxy.get(key, default)
        return float(v)
    except Exception:
        return default

```

```

def rc2c_floor_deficit(value, floor):
    if not rc2c_is_finite(value):
        return 0.0
    return max(0.0, float(floor) - float(value))

def rc2c_is_guarded_context_candidate(variant: str) -> bool:
    return rc2b_policy_family(variant) == "rc1b_guarded_context_gap" or variant in RC2C_GUARDED_REFERENCE_VARIANTS

def rc2c_candidate_meets_hard_floors(candidate) -> bool:
    p = candidate.get("proxy", {})
    c2 = rc2c_proxy_value(p, "proxy_class2")
    min_task = rc2c_proxy_value(p, "proxy_min_task", rc2c_proxy_value(p, "proxy_min_class"))
    min_class = rc2c_proxy_value(p, "proxy_min_class")
    ok_c2 = (not rc2c_is_finite(c2)) or c2 >= RC2C_HARD_CLASS2_FLOOR
    ok_min_task = (not rc2c_is_finite(min_task)) or min_task >= RC2C_HARD_MIN_TASK_FLOOR
    ok_min_class = (not rc2c_is_finite(min_class)) or min_class >= RC2C_HARD_MIN_CLASS_FLOOR
    return bool(ok_c2 and ok_min_task and ok_min_class)

def rc2c_risk_components(proxy, baseline_proxy, variant: str, after_task=None):
    """Return validation-memory-only score components for RC2c risk-aware policy selection."""
    c2 = rc2c_proxy_value(proxy, "proxy_class2")
    c5 = rc2c_proxy_value(proxy, "proxy_class5")
    min_class = rc2c_proxy_value(proxy, "proxy_min_class")
    min_task = rc2c_proxy_value(proxy, "proxy_min_task", min_class)
    old_task_min = rc2c_proxy_value(proxy, "proxy_old_task_min", min_task)
    task_std = rc2c_proxy_value(proxy, "proxy_task_acc_std", 0.0)
    family = rc2b_policy_family(variant)

    floor_penalty = 0.0
    floor_penalty += RC2C_CLASS2_FLOOR_PENALTY * rc2c_floor_deficit(c2, RC2C_CLASS2_FLOOR)
    floor_penalty += RC2C_MIN_CLASS_FLOOR_PENALTY * rc2c_floor_deficit(min_class, RC2C_MIN_CLASS_FLOOR)
    floor_penalty += RC2C_MIN_TASK_FLOOR_PENALTY * rc2c_floor_deficit(min_task, RC2C_MIN_TASK_FLOOR)

```

```

    floor_penalty += RC2C_OLD_TASK_FLOOR_PENALTY *␣
↪rc2c_floor_deficit(old_task_min, RC2C_OLD_TASK_FLOOR)

    dispersion_penalty = 0.0
    if rc2c_is_finite(task_std):
        dispersion_penalty = RC2C_TASK_DISPERSION_PENALTY * max(0.0, task_std)

    no_repair_soft_penalty = 0.0
    if after_task is not None and int(after_task) >=␣
↪RC2C_NO_REPAIR_VETO_AFTER_TASK and family == "proto_no_repair":
        no_repair_soft_penalty = RC2C_NO_REPAIR_SOFT_PENALTY

    guarded_bonus = RC2C_GUARDED_CONTEXT_BONUS if␣
↪rc2c_is_guarded_context_candidate(variant) else 0.0
    complexity_penalty = RC2B_COMPLEXITY_PENALTY *␣
↪rc2b_policy_complexity_rank(variant)
    class5_bonus = RC2B_VALIDATION_SCORE_CLASS5_WEIGHT * (0.0 if not␣
↪rc2c_is_finite(c5) else c5)
    hydro_penalty = rc3_hydro_selection_penalty(proxy)

    return {
        "class2_floor_penalty": RC2C_CLASS2_FLOOR_PENALTY *␣
↪rc2c_floor_deficit(c2, RC2C_CLASS2_FLOOR),
        "min_class_floor_penalty": RC2C_MIN_CLASS_FLOOR_PENALTY *␣
↪rc2c_floor_deficit(min_class, RC2C_MIN_CLASS_FLOOR),
        "min_task_floor_penalty": RC2C_MIN_TASK_FLOOR_PENALTY *␣
↪rc2c_floor_deficit(min_task, RC2C_MIN_TASK_FLOOR),
        "old_task_floor_penalty": RC2C_OLD_TASK_FLOOR_PENALTY *␣
↪rc2c_floor_deficit(old_task_min, RC2C_OLD_TASK_FLOOR),
        "floor_penalty": floor_penalty,
        "task_dispersion_penalty": dispersion_penalty,
        "no_repair_soft_penalty": no_repair_soft_penalty,
        "complexity_penalty": complexity_penalty,
        "class5_bonus": class5_bonus,
        "guarded_context_bonus": guarded_bonus,
        "hydro_selection_penalty": hydro_penalty,
        "total_risk_penalty": floor_penalty + dispersion_penalty +␣
↪no_repair_soft_penalty + complexity_penalty + hydro_penalty,
    }

def rc2b_validation_score(proxy, baseline_proxy, variant: str, after_task=None):
    """Policy-level validation score using memory validation only.

    In RC2B_STRICT this reproduces the older selector.
    In RC2c profiles this becomes a risk-aware selector:

```



```

- stronger weak-class / min-task penalties;
- trajectory-aware old-task validation-memory penalty;
- no-repair soft penalty after early tasks;
- guarded context-gap hysteresis support handled by rc2c_select_candidate.
"""

base_score = validation_score(proxy, baseline_proxy)
if base_score <= -1e8:
    return -1e9

if not RC2C_RISK_AWARE_SELECTOR_ENABLED:
    c2 = proxy.get("proxy_class2", np.nan)
    c5 = proxy.get("proxy_class5", np.nan)
    minc = proxy.get("proxy_min_class", np.nan)
    penalty = RC2B_COMPLEXITY_PENALTY * rc2b_policy_complexity_rank(variant)
    floor_penalty = 0.0
    if not np.isnan(c2) and c2 < RC2B_CLASS2_FLOOR:
        floor_penalty += RC2B_VALIDATION_SCORE_FLOOR_PENALTY *
↪(RC2B_CLASS2_FLOOR - c2)
    if not np.isnan(c5) and c5 < RC2B_CLASS5_FLOOR:
        floor_penalty += RC2B_VALIDATION_SCORE_FLOOR_PENALTY *
↪(RC2B_CLASS5_FLOOR - c5)
    if not np.isnan(minc) and minc < RC2B_MIN_CLASS_FLOOR:
        floor_penalty += RC2B_VALIDATION_SCORE_FLOOR_PENALTY *
↪(RC2B_MIN_CLASS_FLOOR - minc)
    class5_bonus = RC2B_VALIDATION_SCORE_CLASS5_WEIGHT * (0.0 if np.
↪isnan(c5) else c5)
    hydro_penalty = rc3_hydro_selection_penalty(proxy)
    return float(base_score + class5_bonus - penalty - floor_penalty -
↪hydro_penalty)

    comp = rc2c_risk_components(proxy, baseline_proxy, variant,
↪after_task=after_task)
    return float(
        base_score
        + comp["class5_bonus"]
        + comp["guarded_context_bonus"]
        - comp["total_risk_penalty"]
    )

def rc2c_select_candidate(candidates, after_task):
    """Risk-aware no-leakage selector.

    This function only consumes candidate proxy metrics computed on
↪policy-validation memory.
    It never reads final-test metrics.

```

```

"""
scored = [c for c in candidates if rc2c_is_finite(c.get("selection_score", -1e9))]
if not scored:
    selected = candidates[0]
    return selected, {
        "raw_best_variant": selected["variant"],
        "selection_reason": "fallback_no_finite_scores",
        "no_repair_veto_applied": False,
        "guarded_hysteresis_applied": False,
        "risk_floor_fallback_applied": False,
    }

sorted_by_score = sorted(
    scored,
    key=lambda c: (
        float(c.get("selection_score", -1e9)),
        -float(c["complexity_rank"]),
        float(c["proxy"].get("proxy_acc", -1e9) if not np.isnan(c["proxy"]).get("proxy_acc", np.nan)) else -1e9),
    ),
    reverse=True,
)
raw_best = sorted_by_score[0]
selected = raw_best
reason = "direct_risk_aware_score_win"
no_repair_veto = False
hysteresis = False
floor_fallback = False

guarded_candidates = [c for c in scored if rc2c_is_guarded_context_candidate(c["variant"])]
best_guarded = max(guarded_candidates, key=lambda c: float(c.get("selection_score", -1e9))) if guarded_candidates else None

# 1) No-repair veto after early tasks unless it really beats the guarded family.
if (
    RC2C_RISK_AWARE_SELECTOR_ENABLED
    and best_guarded is not None
    and int(after_task) >= RC2C_NO_REPAIR_VETO_AFTER_TASK
    and rc2b_policy_family(raw_best["variant"]) == "proto_no_repair"
):
    margin = float(raw_best.get("selection_score", -1e9)) - float(best_guarded.get("selection_score", -1e9))
    if margin < RC2C_NO_REPAIR_REQUIRED_MARGIN:

```

```

        selected = best_guarded
        reason = f"no_repair_veto_guarded_fallback_margin_{margin:+.4f}"
        no_repair_veto = True

    # 2) Hard-floor fallback: prefer a floor-safe candidate when the current
    ↪selected one is fragile.
    if RC2C_RISK_AWARE_SELECTOR_ENABLED and RC2C_FLOOR_RISK_FALLBACK_ENABLED:
        if not rc2c_candidate_meets_hard_floors(selected):
            floor_safe = [c for c in scored if
            ↪rc2c_candidate_meets_hard_floors(c)]
            if floor_safe:
                best_floor_safe = max(floor_safe, key=lambda c: float(c.
            ↪get("selection_score", -1e9)))
                if float(best_floor_safe.get("selection_score", -1e9)) >=
            ↪float(selected.get("selection_score", -1e9)) -
            ↪RC2C_GUARDED_HYSTERESIS_MARGIN:
                    selected = best_floor_safe
                    reason = "hard_floor_risk_fallback"
                    floor_fallback = True

    # 3) Guarded hysteresis: if close, prefer the historically robust guarded
    ↪context-gap family.
    if (
        RC2C_RISK_AWARE_SELECTOR_ENABLED
        and best_guarded is not None
        and not rc2c_is_guarded_context_candidate(selected["variant"])
        and float(best_guarded.get("selection_score", -1e9)) >= float(selected.
    ↪get("selection_score", -1e9)) - RC2C_GUARDED_HYSTERESIS_MARGIN
    ):
        selected = best_guarded
        reason = "guarded_context_gap_hysteresis"
        hysteresis = True

    return selected, {
        "raw_best_variant": raw_best["variant"],
        "raw_best_policy_family": raw_best["policy_family"],
        "raw_best_proxy_score": raw_best.get("selection_score", np.nan),
        "best_guarded_variant": best_guarded["variant"] if best_guarded is not
    ↪None else "",
        "best_guarded_proxy_score": best_guarded.get("selection_score", np.nan)
    ↪if best_guarded is not None else np.nan,
        "selection_reason": reason,
        "no_repair_veto_applied": no_repair_veto,
        "guarded_hysteresis_applied": hysteresis,
        "risk_floor_fallback_applied": floor_fallback,
    }

```

```

def build_rc2b_candidate_from_checkpoint(base_run, checkpoint, variant: str,
                                         repair_memory,
context_selection_memory, policy_validation_memory,
                                         seed=0):
    """Build one candidate policy and score it on policy-validation memory only.
    """
    candidate_seed = int(seed)
    cp_seed = int(checkpoint["seed"])
    after_task = int(checkpoint["after_task"])
    method_name = rc2b_candidate_method_name(variant)

    base_cfg = METHOD_CONFIGS["mmals_proto_first_balanced_replay"]
    model = restore_base_model_from_checkpoint(checkpoint, base_cfg)
    context_prototypes = clone_prototypes(checkpoint["context_prototypes"])

    config, config_rows = build_context_variant_config_and_logs(
        model, context_selection_memory, context_prototypes,
        variant=variant, seed=candidate_seed + 313,
    )

    readout = GlobalLinearReadout(model.global_head).to(DEVICE)
    repair_rows = list(config_rows)
    pre_global_head = copy.deepcopy(model.global_head).to(DEVICE)
    pre_global_head.eval()
    for p in pre_global_head.parameters():
        p.requires_grad_(False)

    if variant_uses_guarded_head(variant):
        repair_rows.extend(train_readout_on_memory(
            readout, model, repair_memory, config,
            prototypes=context_prototypes, pre_global_head=pre_global_head,
            kl_weight=0.35, use_kl=True, margin_weight=0.0,
            variant=f"{variant}_rc2b_guarded_policy_candidate",
        ))

    proxy = proxy_metrics_for_context_config(
        model, policy_validation_memory, config,
        prototypes=context_prototypes, readout=readout,
        max_items_per_class=VALIDATION_MAX_ITEMS_PER_CLASS,
    )

    for rr in repair_rows:
        rr.update({
            "method": method_name,
            "seed": cp_seed,
            "after_task": after_task,

```

```

        "task_id": after_task,
        "variant": variant,
        "repair_type": str(rr.get("repair_type", "")),
        "rc2b_strict_candidate": True,
        "rc2b_no_final_test_used_for_selection": True,
        "policy_family": rc2b_policy_family(variant),
        "uses_guarded_global_head": variant_uses_guarded_head(variant),
        "uses_context_selection": context_variant_selection_mode(variant)
    )
    if is not None,
        "uses_oracle_context_reference":
    is_oracle_context_reference(variant),
    })

    return {
        "variant": variant,
        "method": method_name,
        "policy_family": rc2b_policy_family(variant),
        "policy_label": rc2b_policy_label(variant),
        "complexity_rank": rc2b_policy_complexity_rank(variant),
        "model": model,
        "readout": readout,
        "config": config,
        "context_prototypes": context_prototypes,
        "proxy": proxy,
        "repair_rows": repair_rows,
    }

def rc2b_history_rows_from_evals(candidate, tasks, checkpoint,
    method_name=None, selected=False):
    seed = int(checkpoint["seed"])
    after_task = int(checkpoint["after_task"])
    method = method_name or candidate["method"]
    model = candidate["model"]
    readout = candidate["readout"]
    config = candidate["config"]
    context_prototypes = candidate["context_prototypes"]
    evals = evaluate_seen_tasks_with_readout(model, readout, tasks,
    upto_task=after_task, config=config, prototypes=context_prototypes)

    history_rows = []
    replay_memory_items = int(checkpoint.get("replay_memory_items", 0))
    readout_trainable = count_parameters(readout)
    readout_total = total_parameters(readout)
    for old_task_id, metrics in enumerate(evals):
        row = {
            "method": method,

```

```

        "seed": seed,
        "model_family": "mmals_rc2b_policy" if selected else
↪ "mmals_rc2b_candidate",
        "after_task": after_task,
        "task_id": old_task_id,
        "task_name": tasks[old_task_id]["name"],
        "seen_classes": str(seen_classes_until(after_task)),
        "trainable_params": count_parameters(model) + readout_trainable,
        "total_params": total_parameters(model) + readout_total,
        "backbone_params": total_parameters(model),
        "readout_trainable_params": readout_trainable,
        "readout_total_params": readout_total,
        "model_size_mb_total": (total_parameters(model) + readout_total) *
↪ 4 / (1024 ** 2),
        "uses_balanced_replay": True,
        "uses_output_distill": True,
        "uses_local_mycorrhizal_head": False,
        "uses_global_head_refresh":
↪ variant_uses_guarded_head(candidate["variant"]),
        "uses_context_selection":
↪ context_variant_selection_mode(candidate["variant"]) is not None,
        "uses_oracle_context_reference": False,
        "probe_row": False,
        "backbone_source": "same_proto_first_checkpoint",
        "probe_feature": "original_backbone_zf_rc2b_policy",
        "probe_context_mode": "proto_first_rc2b_strict",
        "replay_memory_items": replay_memory_items,
        "memory_per_class": MEMORY_PER_CLASS,
        "memory_per_task": MEMORY_PER_TASK,
        "paired_same_backbone": True,
        "readout_variant": candidate["variant"],
        "context_variant": candidate["variant"],
        "selected_candidate_method": candidate["method"] if selected else
↪ None,
        "rc2b_selected_policy": bool(selected),
        "rc2b_candidate_policy_family": candidate["policy_family"],
        "rc2b_selection_proxy_acc": candidate["proxy"].get("proxy_acc", np.
↪ nan),
        "rc2b_selection_score": candidate.get("selection_score", np.nan),
        "rc2b_no_final_test_used_for_selection": True,
        "context_temperature_used": float(getattr(model,
↪ "context_temperature", np.nan)),
        "batch_blend_single_used": float(config.get("batch_blend_single",
↪ np.nan)),
        "batch_blend_latent_used": float(config.get("batch_blend_latent",
↪ np.nan)),

```

```

        "batch_blend_feature_used": float(config.get("batch_blend_feature",
↪np.nan)),
    }
    row.update(metrics)
    history_rows.append(row)
    return pd.DataFrame(history_rows)

def rc2b_confusion_rows(candidate, tasks, checkpoint, method_name=None,
↪selected=False):
    seed = int(checkpoint["seed"])
    after_task = int(checkpoint["after_task"])
    if after_task != N_TASKS - 1:
        return pd.DataFrame()
    method = method_name or candidate["method"]
    cm_df = class_confusion_matrix_with_readout(
        candidate["model"], candidate["readout"], tasks,
        upto_task=after_task, config=candidate["config"],
↪prototypes=candidate["context_prototypes"]
    )
    cm_df["method"] = method
    cm_df["seed"] = seed
    cm_df["after_task"] = after_task
    cm_df["backbone_source"] = "same_proto_first_checkpoint"
    cm_df["readout_variant"] = candidate["variant"]
    cm_df["context_variant"] = candidate["variant"]
    cm_df["rc2b_selected_policy"] = bool(selected)
    cm_df["selected_candidate_method"] = candidate["method"] if selected else
↪None
    return cm_df

def apply_rc2b_strict_policy_to_checkpoint(base_run, checkpoint):
    """Strictly select the deployable policy before final-test evaluation."""
    seed = int(checkpoint["seed"])
    after_task = int(checkpoint["after_task"])
    tasks = base_run["tasks"]
    class_memory = clone_class_memory(checkpoint["class_memory"])

    repair_memory, context_selection_memory, policy_validation_memory =
↪split_class_memory_three_way(
        class_memory,
        seed=seed * 100_000 + after_task * 10_000 + 777,
    )

    candidates = []
    for variant in RC2B_CANDIDATE_VARIANTS:

```

```

    cand = build_rc2b_candidate_from_checkpoint(
        base_run, checkpoint, variant,
        repair_memory=repair_memory,
        context_selection_memory=context_selection_memory,
        policy_validation_memory=policy_validation_memory,
        seed=seed * 10_000 + after_task * 313 + _
    )
    candidates.append(cand)

    baseline = next((c for c in candidates if c["variant"] == _
    ↪ "proto_global_no_repair"), candidates[0])
    baseline_proxy = baseline["proxy"]

    # RC3 Hydro audit: compute candidate-vs-baseline drift/turbulence on _
    ↪ policy-validation memory only.
    # No final-test examples are used here.
    for cand in candidates:
        hydro_proxy = rc3_pairwise_hydro_proxy(cand, baseline, _
    ↪ policy_validation_memory)
        cand["hydro_proxy"] = hydro_proxy
        cand["proxy"].update(hydro_proxy)

    selection_rows = []
    for cand in candidates:
        score = rc2b_validation_score(cand["proxy"], baseline_proxy, _
    ↪ cand["variant"], after_task=after_task)
        cand["risk_components"] = rc2c_risk_components(cand["proxy"], _
    ↪ baseline_proxy, cand["variant"], after_task=after_task) if _
    ↪ RC2C_RISK_AWARE_SELECTOR_ENABLED else {}
        cand["selection_score"] = score
        p = cand["proxy"]
        selection_rows.append({
            "method": RC2B_METHOD,
            "seed": seed,
            "after_task": after_task,
            "task_id": after_task,
            "candidate_variant": cand["variant"],
            "candidate_method": cand["method"],
            "policy_family": cand["policy_family"],
            "policy_label": cand["policy_label"],
            "complexity_rank": cand["complexity_rank"],
            "proxy_acc": p.get("proxy_acc", np.nan),
            "proxy_class2": p.get("proxy_class2", np.nan),
            "proxy_class5": p.get("proxy_class5", np.nan),
            "proxy_min_class": p.get("proxy_min_class", np.nan),

```



```

        "proxy_min_task": p.get("proxy_min_task", np.nan),
        "proxy_old_task_min": p.get("proxy_old_task_min", np.nan),
        "proxy_task_acc_std": p.get("proxy_task_acc_std", np.nan),
        "proxy_score": score,
        "rc2c_risk_aware_selector_enabled": 1
    RC2C_RISK_AWARE_SELECTOR_ENABLED,
        "rc2c_class2_floor_penalty": cand.get("risk_components", {}).
    get("class2_floor_penalty", np.nan),
        "rc2c_min_class_floor_penalty": cand.get("risk_components", {}).
    get("min_class_floor_penalty", np.nan),
        "rc2c_min_task_floor_penalty": cand.get("risk_components", {}).
    get("min_task_floor_penalty", np.nan),
        "rc2c_old_task_floor_penalty": cand.get("risk_components", {}).
    get("old_task_floor_penalty", np.nan),
        "rc2c_task_dispersion_penalty": cand.get("risk_components", {}).
    get("task_dispersion_penalty", np.nan),
        "rc2c_no_repair_soft_penalty": cand.get("risk_components", {}).
    get("no_repair_soft_penalty", np.nan),
        "rc2c_guarded_context_bonus": cand.get("risk_components", {}).
    get("guarded_context_bonus", np.nan),
        "hydro_route_drift_Dr": p.get("hydro_route_drift_Dr", np.nan),
        "hydro_latent_drift_Dz": p.get("hydro_latent_drift_Dz", np.nan),
        "hydro_output_drift_Dy": p.get("hydro_output_drift_Dy", np.nan),
        "hydro_adaptive_viscosity_nu": p.get("hydro_adaptive_viscosity_nu", 1
    np.nan),
        "hydro_reynolds_like": p.get("hydro_reynolds_like", np.nan),
        "hydro_turbulence_score": p.get("hydro_turbulence_score", np.nan),
        "hydro_wave_entropy": p.get("hydro_wave_entropy", np.nan),
        "hydro_selection_penalty": rc3_hydro_selection_penalty(p),
        "hydro_audit_memory_only": p.get("hydro_audit_memory_only", False),
        "hydro_audit_reference_variant": p.
    get("hydro_audit_reference_variant", ""),
        "selection_phase": "candidate_scored_before_final_test",
        "no_final_test_used_for_selection": True,
        "repair_memory_items": int(sum(len(v["y"]) for v in repair_memory.
    values())),
        "context_selection_memory_items": int(sum(len(v["y"]) for v in 1
    context_selection_memory.values())),
        "policy_validation_memory_items": int(sum(len(v["y"]) for v in 1
    policy_validation_memory.values())),
    })

    if RC2C_RISK_AWARE_SELECTOR_ENABLED:
        selected, selection_meta = rc2c_select_candidate(candidates, 1
    after_task=after_task)
    else:

```

```

        # Deterministic tie-break: score descending, then simpler policy, then
        ↪ proxy accuracy.
        candidates_sorted = sorted(
            candidates,
            key=lambda c: (
                float(c.get("selection_score", -1e9)),
                -float(c["complexity_rank"]),
                float(c["proxy"].get("proxy_acc", -1e9) if not np.
        ↪ isnan(c["proxy"].get("proxy_acc", np.nan)) else -1e9),
            ),
            reverse=True,
        )
        best = candidates_sorted[0]
        # If multiple candidates are within the tie tolerance, choose the
        ↪ simpler policy.
        near_best = [c for c in candidates if c.get("selection_score", -1e9) >=
        ↪ best.get("selection_score", -1e9) - RC2B_SELECTION_TIE_TOL]
        selected = sorted(near_best, key=lambda c: (c["complexity_rank"],
        ↪ -float(c["proxy"].get("proxy_acc", -1e9) if not np.isnan(c["proxy"]).
        ↪ get("proxy_acc", np.nan)) else -1e9)) [0]
        selection_meta = {
            "raw_best_variant": best["variant"],
            "raw_best_policy_family": best["policy_family"],
            "raw_best_proxy_score": best.get("selection_score", np.nan),
            "best_guarded_variant": "",
            "best_guarded_proxy_score": np.nan,
            "selection_reason": "rc2b_strict_score_then_simplicity_tiebreak",
            "no_repair_veto_applied": False,
            "guarded_hysteresis_applied": False,
            "risk_floor_fallback_applied": False,
        }

        for r in selection_rows:
            r["selected_candidate_variant"] = selected["variant"]
            r["selected_candidate_method"] = selected["method"]
            r["selected_policy_family"] = selected["policy_family"]
            r["selected_policy_label"] = selected["policy_label"]
            r["selected_proxy_score"] = selected.get("selection_score", np.nan)
            r["selected_proxy_acc"] = selected["proxy"].get("proxy_acc", np.nan)
            r["is_selected_candidate"] = (r["candidate_variant"] ==
        ↪ selected["variant"])
            r.update(selection_meta)

        candidate_history, candidate_confusions, score_dump_tables = [], [], []
        if RC2B_EVALUATE_ALL_CANDIDATES_AFTER_SELECTION:
            for cand in candidates:

```

```

        h = rc2b_history_rows_from_evals(cand, tasks, checkpoint,
↪method_name=cand["method"], selected=False)
        candidate_history.append(h)
        c = rc2b_confusion_rows(cand, tasks, checkpoint,
↪method_name=cand["method"], selected=False)
        if not c.empty:
            candidate_confusions.append(c)
            if globals().get("ENABLE_BIOMETRIC_STYLE_THRESHOLD_DIAGNOSTICS",
↪False) and globals().get("THRESHOLD_DIAGNOSTICS_EVALUATE_CANDIDATES", True):
                sd = collect_score_dump_for_candidate(
                    cand, tasks, checkpoint, method_name=cand["method"],
↪selected=False,
                    final_checkpoint_only=globals().
↪get("THRESHOLD_DIAGNOSTICS_FINAL_CHECKPOINT_ONLY", True),
                )
                if not sd.empty:
                    score_dump_tables.append(sd)

        selected_history = rc2b_history_rows_from_evals(selected, tasks,
↪checkpoint, method_name=RC2B_METHOD, selected=True)
        selected_confusion = rc2b_confusion_rows(selected, tasks, checkpoint,
↪method_name=RC2B_METHOD, selected=True)
        if globals().get("ENABLE_BIOMETRIC_STYLE_THRESHOLD_DIAGNOSTICS", False):
            sd = collect_score_dump_for_candidate(
                selected, tasks, checkpoint, method_name=RC2B_METHOD, selected=True,
                final_checkpoint_only=globals().
↪get("THRESHOLD_DIAGNOSTICS_FINAL_CHECKPOINT_ONLY", True),
            )
            if not sd.empty:
                score_dump_tables.append(sd)

    repair_rows = []
    for cand in candidates:
        for rr in cand.get("repair_rows", []):
            rr = dict(rr)
            rr.update({
                "rc2b_candidate_variant": cand["variant"],
                "rc2b_selected_candidate_variant": selected["variant"],
                "rc2b_selected_candidate_method": selected["method"],
                "rc2b_is_selected_candidate": cand["variant"] ==
↪selected["variant"],
                "rc2b_no_final_test_used_for_selection": True,
            })
            repair_rows.append(rr)

    selection_df = pd.DataFrame(selection_rows)

```

```

    candidate_history_df = pd.concat(candidate_history, ignore_index=True) if
↪candidate_history else pd.DataFrame()
    candidate_confusion_df = pd.concat(candidate_confusions, ignore_index=True)
↪if candidate_confusions else pd.DataFrame()
    selected_confusion_df = selected_confusion if not selected_confusion.empty
↪else pd.DataFrame()
    repair_df = pd.DataFrame(repair_rows)
    score_dump_df = pd.concat(score_dump_tables, ignore_index=True) if
↪score_dump_tables else pd.DataFrame()

    print(
        f"[{POLICY_BRANCH} seed={seed}] after task {after_task}: selected
↪{selected['variant']} "
        f"proxy_acc={selected['proxy'].get('proxy_acc', np.nan):.4f} "
        f"score={selected.get('selection_score', np.nan):.4f}"
    )
    return selected_history, selected_confusion_df, candidate_history_df,
↪candidate_confusion_df, repair_df, selection_df, score_dump_df

```

```

[64]: # -----
# v1.1-RC2d gate-aligned selector overrides
# -----
# RC2d fixes the RC2c smoke failure modes:
# - performance gate failure;
# - class-2 robustness failure;
# - class-5 preservation failure;
# - partial selector/final-test alignment.
#
# All functions below still use validation-memory artifacts only.
# They intentionally override selected RC2c helper names so the existing
# experiment loop can remain unchanged.
# -----

RC2D_SELECTOR_VERSION = globals().get("RC2D_SELECTOR_VERSION",
↪"RC2d_gate_aligned_v1")
RC2D_GATE_ALIGNMENT_ENABLED = globals().get("RC2D_GATE_ALIGNMENT_ENABLED", True)
RC2D_AUX_PROXY_WEIGHT = globals().get("RC2D_AUX_PROXY_WEIGHT", 0.35)
RC2D_PRIMARY_PROXY_WEIGHT = globals().get("RC2D_PRIMARY_PROXY_WEIGHT", 0.65)
RC2D_PROXY_ACC_FLOOR = globals().get("RC2D_PROXY_ACC_FLOOR", 0.68)
RC2D_PROXY_ACC_FLOOR_PENALTY = globals().get("RC2D_PROXY_ACC_FLOOR_PENALTY", 3.
↪00)
RC2D_CLASS5_FLOOR = globals().get("RC2D_CLASS5_FLOOR", 0.90)
RC2D_HARD_CLASS5_FLOOR = globals().get("RC2D_HARD_CLASS5_FLOOR", 0.80)
RC2D_CLASS5_FLOOR_PENALTY = globals().get("RC2D_CLASS5_FLOOR_PENALTY", 5.00)
RC2D_CLASS2_RESCUE_MARGIN = globals().get("RC2D_CLASS2_RESCUE_MARGIN", 0.08)
RC2D_CLASS5_RESCUE_MARGIN = globals().get("RC2D_CLASS5_RESCUE_MARGIN", 0.08)

```

```

RC2D_TRUE_GUARD_RESCUE_SCORE_MARGIN = globals().
    ↪get("RC2D_TRUE_GUARD_RESCUE_SCORE_MARGIN", 0.020)
RC2D_TRUE_GUARD_CLASS2_MARGIN = globals().get("RC2D_TRUE_GUARD_CLASS2_MARGIN", 0.05)
    ↪0.05)
RC2D_TRUE_GUARD_ACC_MARGIN = globals().get("RC2D_TRUE_GUARD_ACC_MARGIN", 0.010)
RC2D_TRUE_GUARD_BONUS = globals().get("RC2D_TRUE_GUARD_BONUS", 0.003)
RC2D_CONTEXT_GUARD_BONUS = globals().get("RC2D_CONTEXT_GUARD_BONUS", 0.001)
RC2D_GUARDED_HYSTERESIS_MARGIN = globals().
    ↪get("RC2D_GUARDED_HYSTERESIS_MARGIN", 0.003)
RC2D_SELECTOR_FINAL_ALIGNMENT_TOL = globals().
    ↪get("RC2D_SELECTOR_FINAL_ALIGNMENT_TOL", 0.005)
RC2D_CLASS_BALANCE_WEIGHT = globals().get("RC2D_CLASS_BALANCE_WEIGHT", 0.35)
RC2D_MIN_TASK_WEIGHT = globals().get("RC2D_MIN_TASK_WEIGHT", 0.35)
RC2D_OLD_TASK_WEIGHT = globals().get("RC2D_OLD_TASK_WEIGHT", 0.20)
RC2D_TASK_STD_WEIGHT = globals().get("RC2D_TASK_STD_WEIGHT", 0.25)

# Use the new deployed method name while preserving RC2B/RC3 compatibility
    ↪aliases.
if globals().get("POLICY_BRANCH", "RC2d") == "RC2d":
    RC2C_METHOD = "paired_rc2d_gate_aligned_policy"
    RC2B_METHOD = RC2C_METHOD
    RC3_METHOD = RC2C_METHOD

def rc2d_finite_value(proxy, key, default=np.nan):
    try:
        v = proxy.get(key, default)
        return float(v)
    except Exception:
        return default

def rc2d_weighted_metric(primary, aux, key, default=np.nan):
    pv = rc2d_finite_value(primary, key, default)
    av = rc2d_finite_value(aux, key, np.nan) if aux is not None else np.nan
    if rc2c_is_finite(pv) and rc2c_is_finite(av):
        return RC2D_PRIMARY_PROXY_WEIGHT * pv + RC2D_AUX_PROXY_WEIGHT * av
    if rc2c_is_finite(pv):
        return pv
    if rc2c_is_finite(av):
        return av
    return default

def rc2d_merge_proxy_metrics(primary_proxy, auxiliary_proxy=None):
    """Merge primary and auxiliary validation-memory proxies.

```

The primary split remains the policy-validation split. The auxiliary split
↪ is
another memory-only view used to reduce selection noise. No final-test data
↪ is used.

```

"""
if auxiliary_proxy is None:
    merged = dict(primary_proxy)
    merged["proxy_aux_available"] = False
    return merged
merged = dict(primary_proxy)
for key in [
    "proxy_acc", "proxy_context_top1", "proxy_context_entropy",
    ↪ "proxy_context_margin",
    "proxy_class2", "proxy_class5", "proxy_min_class", "proxy_min_task",
    "proxy_old_task_min", "proxy_task_acc_std", "proxy_score",
]:
    if key in primary_proxy or key in auxiliary_proxy:
        merged[key] = rc2d_weighted_metric(primary_proxy, auxiliary_proxy,
        ↪ key)
        merged[f"primary_{key}"] = primary_proxy.get(key, np.nan)
        merged[f"aux_{key}"] = auxiliary_proxy.get(key, np.nan)
merged["proxy_aux_available"] = True
merged["proxy_aux_weight"] = RC2D_AUX_PROXY_WEIGHT
return merged

def rc2c_candidate_meets_hard_floors(candidate) -> bool:
    """RC2d hard-floor check: class 2, class 5, min task, min class and proxy
    ↪ accuracy."""
    p = candidate.get("proxy", {})
    c2 = rc2c_proxy_value(p, "proxy_class2")
    c5 = rc2c_proxy_value(p, "proxy_class5")
    min_task = rc2c_proxy_value(p, "proxy_min_task", rc2c_proxy_value(p,
    ↪ "proxy_min_class"))
    min_class = rc2c_proxy_value(p, "proxy_min_class")
    acc = rc2c_proxy_value(p, "proxy_acc")
    ok_c2 = (not rc2c_is_finite(c2)) or c2 >= RC2C_HARD_CLASS2_FLOOR
    ok_c5 = (not rc2c_is_finite(c5)) or c5 >= RC2D_HARD_CLASS5_FLOOR
    ok_min_task = (not rc2c_is_finite(min_task)) or min_task >=
    ↪ RC2C_HARD_MIN_TASK_FLOOR
    ok_min_class = (not rc2c_is_finite(min_class)) or min_class >=
    ↪ RC2C_HARD_MIN_CLASS_FLOOR
    ok_acc = (not rc2c_is_finite(acc)) or acc >= max(0.50, RC2D_PROXY_ACC_FLOOR,
    ↪ 0.05)
    return bool(ok_c2 and ok_c5 and ok_min_task and ok_min_class and ok_acc)

```

```

def rc2c_risk_components(proxy, baseline_proxy, variant: str, after_task=None):
    """RC2d risk components, still computed from validation memory only."""
    acc = rc2c_proxy_value(proxy, "proxy_acc")
    c2 = rc2c_proxy_value(proxy, "proxy_class2")
    c5 = rc2c_proxy_value(proxy, "proxy_class5")
    min_class = rc2c_proxy_value(proxy, "proxy_min_class")
    min_task = rc2c_proxy_value(proxy, "proxy_min_task", min_class)
    old_task_min = rc2c_proxy_value(proxy, "proxy_old_task_min", min_task)
    task_std = rc2c_proxy_value(proxy, "proxy_task_acc_std", 0.0)
    family = rc2b_policy_family(variant)

    class2_penalty = RC2C_CLASS2_FLOOR_PENALTY * rc2c_floor_deficit(c2,
↪RC2C_CLASS2_FLOOR)
    class5_penalty = RC2D_CLASS5_FLOOR_PENALTY * rc2c_floor_deficit(c5,
↪RC2D_CLASS5_FLOOR)
    min_class_penalty = RC2C_MIN_CLASS_FLOOR_PENALTY *
↪rc2c_floor_deficit(min_class, RC2C_MIN_CLASS_FLOOR)
    min_task_penalty = RC2C_MIN_TASK_FLOOR_PENALTY *
↪rc2c_floor_deficit(min_task, RC2C_MIN_TASK_FLOOR)
    old_task_penalty = RC2C_OLD_TASK_FLOOR_PENALTY *
↪rc2c_floor_deficit(old_task_min, RC2C_OLD_TASK_FLOOR)
    acc_penalty = RC2D_PROXY_ACC_FLOOR_PENALTY * rc2c_floor_deficit(acc,
↪RC2D_PROXY_ACC_FLOOR)
    floor_penalty = class2_penalty + class5_penalty + min_class_penalty +
↪min_task_penalty + old_task_penalty + acc_penalty

    dispersion_penalty = RC2D_TASK_STD_WEIGHT * max(0.0, task_std) if
↪rc2c_is_finite(task_std) else 0.0
    no_repair_soft_penalty = 0.0
    if after_task is not None and int(after_task) >=
↪RC2C_NO_REPAIR_VETO_AFTER_TASK and family == "proto_no_repair":
        no_repair_soft_penalty = RC2C_NO_REPAIR_SOFT_PENALTY

    guarded_bonus = RC2D_CONTEXT_GUARD_BONUS if
↪rc2c_is_guarded_context_candidate(variant) else 0.0
    true_guard_bonus = RC2D_TRUE_GUARD_BONUS if family ==
↪"proto_true_global_guard" else 0.0
    complexity_penalty = RC2B_COMPLEXITY_PENALTY *
↪rc2b_policy_complexity_rank(variant)
    class5_bonus = RC2B_VALIDATION_SCORE_CLASS5_WEIGHT * (0.0 if not
↪rc2c_is_finite(c5) else c5)
    hydro_penalty = rc3_hydro_selection_penalty(proxy)

    return {
        "proxy_acc_floor_penalty": acc_penalty,

```



```

        "class2_floor_penalty": class2_penalty,
        "class5_floor_penalty": class5_penalty,
        "min_class_floor_penalty": min_class_penalty,
        "min_task_floor_penalty": min_task_penalty,
        "old_task_floor_penalty": old_task_penalty,
        "floor_penalty": floor_penalty,
        "task_dispersion_penalty": dispersion_penalty,
        "no_repair_soft_penalty": no_repair_soft_penalty,
        "complexity_penalty": complexity_penalty,
        "class5_bonus": class5_bonus,
        "guarded_context_bonus": guarded_bonus,
        "true_guard_bonus": true_guard_bonus,
        "hydro_selection_penalty": hydro_penalty,
        "total_risk_penalty": floor_penalty + dispersion_penalty +
↪no_repair_soft_penalty + complexity_penalty + hydro_penalty,
    }

def rc2d_gate_aligned_score(proxy, baseline_proxy, variant: str,
↪after_task=None):
    """Gate-aligned validation score with explicit weak-class and
↪average-performance terms."""
    base_score = validation_score(proxy, baseline_proxy)
    if base_score <= -1e8:
        return -1e9

    acc = rc2c_proxy_value(proxy, "proxy_acc", 0.0)
    c2 = rc2c_proxy_value(proxy, "proxy_class2", 0.0)
    c5 = rc2c_proxy_value(proxy, "proxy_class5", 0.0)
    min_class = rc2c_proxy_value(proxy, "proxy_min_class", 0.0)
    min_task = rc2c_proxy_value(proxy, "proxy_min_task", min_class)
    old_task_min = rc2c_proxy_value(proxy, "proxy_old_task_min", min_task)
    task_std = rc2c_proxy_value(proxy, "proxy_task_acc_std", 0.0)

    comp = rc2c_risk_components(proxy, baseline_proxy, variant,
↪after_task=after_task)
    balance_score = (
        acc
        + RC2D_CLASS_BALANCE_WEIGHT * (0.5 * c2 + 0.5 * c5)
        + RC2D_MIN_TASK_WEIGHT * min_task
        + RC2D_OLD_TASK_WEIGHT * old_task_min
        - RC2D_TASK_STD_WEIGHT * max(0.0, task_std)
    )
    return float(
        0.50 * base_score
        + 0.50 * balance_score
        + comp["class5_bonus"]

```



```

        + comp["guarded_context_bonus"]
        + comp["true_guard_bonus"]
        - comp["total_risk_penalty"]
    )

def rc2b_validation_score(proxy, baseline_proxy, variant: str, after_task=None):
    """Override: RC2d gate-aligned selector score, with RC2b strict fallback."""
    if not RC2C_RISK_AWARE_SELECTOR_ENABLED or globals().get("POLICY_BRANCH") != "RC2b":
        base_score = validation_score(proxy, baseline_proxy)
        if base_score <= -1e8:
            return -1e9
        c2 = proxy.get("proxy_class2", np.nan)
        c5 = proxy.get("proxy_class5", np.nan)
        minc = proxy.get("proxy_min_class", np.nan)
        penalty = RC2B_COMPLEXITY_PENALTY * rc2b_policy_complexity_rank(variant)
        floor_penalty = 0.0
        if not np.isnan(c2) and c2 < RC2B_CLASS2_FLOOR:
            floor_penalty += RC2B_VALIDATION_SCORE_FLOOR_PENALTY * (RC2B_CLASS2_FLOOR - c2)
        if not np.isnan(c5) and c5 < RC2B_CLASS5_FLOOR:
            floor_penalty += RC2B_VALIDATION_SCORE_FLOOR_PENALTY * (RC2B_CLASS5_FLOOR - c5)
        if not np.isnan(minc) and minc < RC2B_MIN_CLASS_FLOOR:
            floor_penalty += RC2B_VALIDATION_SCORE_FLOOR_PENALTY * (RC2B_MIN_CLASS_FLOOR - minc)
        class5_bonus = RC2B_VALIDATION_SCORE_CLASS5_WEIGHT * (0.0 if np.isnan(c5) else c5)
        hydro_penalty = rc3_hydro_selection_penalty(proxy)
        return float(base_score + class5_bonus - penalty - floor_penalty - hydro_penalty)

    return rc2d_gate_aligned_score(proxy, baseline_proxy, variant, after_task=after_task)

def rc2d_candidate_proxy_acc(c):
    return rc2c_proxy_value(c.get("proxy", {}), "proxy_acc", -1e9)

def rc2d_candidate_class2(c):
    return rc2c_proxy_value(c.get("proxy", {}), "proxy_class2", np.nan)

def rc2d_candidate_class5(c):

```

```

    return rc2c_proxy_value(c.get("proxy", {}), "proxy_class5", np.nan)

def rc2d_not_too_far(candidate, selected, margin):
    return float(candidate.get("selection_score", -1e9)) >= float(selected.
↪get("selection_score", -1e9)) - margin

def rc2c_select_candidate(candidates, after_task):
    """Override: RC2d gate-aligned selector.

    Still uses only candidate proxies computed on validation-memory partitions.
    """

    scored = [c for c in candidates if rc2c_is_finite(c.get("selection_score",
↪-1e9))]
    if not scored:
        selected = candidates[0]
        return selected, {
            "raw_best_variant": selected["variant"],
            "selection_reason": "fallback_no_finite_scores",
            "no_repair_veto_applied": False,
            "guarded_hysteresis_applied": False,
            "risk_floor_fallback_applied": False,
            "true_guard_rescue_applied": False,
            "class2_rescue_applied": False,
            "class5_rescue_applied": False,
            "rc2d_selector_version": RC2D_SELECTOR_VERSION,
        }

    sorted_by_score = sorted(
        scored,
        key=lambda c: (
            float(c.get("selection_score", -1e9)),
            float(c.get("proxy", {}).get("proxy_acc", -1e9) if not np.isnan(c.
↪get("proxy", {}).get("proxy_acc", np.nan)) else -1e9),
            -float(c["complexity_rank"]),
        ),
        reverse=True,
    )
    raw_best = sorted_by_score[0]
    selected = raw_best
    reason = "direct_rc2d_gate_aligned_score_win"
    no_repair_veto = False
    hysteresis = False
    floor_fallback = False
    true_guard_rescue = False
    class2_rescue = False

```

```

class5_rescue = False

guarded_candidates = [c for c in scored if
↳rc2c_is_guarded_context_candidate(c["variant"])]
    best_guarded = max(guarded_candidates, key=lambda c: float(c.
↳get("selection_score", -1e9))) if guarded_candidates else None
    true_guard_candidates = [c for c in scored if
↳rc2b_policy_family(c["variant"]) == "proto_true_global_guard"]
    best_true_guard = max(true_guard_candidates, key=lambda c: float(c.
↳get("selection_score", -1e9))) if true_guard_candidates else None

    # 1) No-repair veto after early tasks, but do not force guarded if true_
↳guard is better.
    if (
        best_guarded is not None
        and int(after_task) >= RC2C_NO_REPAIR_VETO_AFTER_TASK
        and rc2b_policy_family(raw_best["variant"]) == "proto_no_repair"
    ):
        margin = float(raw_best.get("selection_score", -1e9)) -
↳float(best_guarded.get("selection_score", -1e9))
        if margin < RC2C_NO_REPAIR_REQUIRED_MARGIN:
            selected = best_guarded
            reason = f"no_repair_veto_guarded_fallback_margin_{margin:+.4f}"
            no_repair_veto = True

    # 2) True global guard rescue: avoid over-selecting context-gap if the true_
↳guard is close and better on class 2 or validation accuracy.
    if best_true_guard is not None and rc2d_not_too_far(best_true_guard,
↳selected, RC2D_TRUE_GUARD_RESCUE_SCORE_MARGIN):
        tg_c2 = rc2d_candidate_class2(best_true_guard)
        se_c2 = rc2d_candidate_class2(selected)
        tg_acc = rc2d_candidate_proxy_acc(best_true_guard)
        se_acc = rc2d_candidate_proxy_acc(selected)
        c2_gain = (rc2c_is_finite(tg_c2) and rc2c_is_finite(se_c2) and tg_c2 >=
↳se_c2 + RC2D_TRUE_GUARD_CLASS2_MARGIN)
        acc_gain = (rc2c_is_finite(tg_acc) and rc2c_is_finite(se_acc) and
↳tg_acc >= se_acc + RC2D_TRUE_GUARD_ACC_MARGIN)
        if (c2_gain or acc_gain) and
↳rc2c_candidate_meets_hard_floors(best_true_guard):
            selected = best_true_guard
            reason = "true_global_guard_rescue_for_class2_or_accuracy_alignment"
            true_guard_rescue = True

    # 3) Weak-class rescue for class 2.
    selected_c2 = rc2d_candidate_class2(selected)
    if rc2c_is_finite(selected_c2) and selected_c2 < RC2C_HARD_CLASS2_FLOOR:

```

```

        c2_candidates = [c for c in scored if
↪rc2c_is_finite(rc2d_candidate_class2(c))]
        if c2_candidates:
            best_c2 = max(c2_candidates, key=lambda c:
↪(rc2d_candidate_class2(c), float(c.get("selection_score", -1e9))))
            if rc2d_candidate_class2(best_c2) >= selected_c2 +
↪RC2D_CLASS2_RESCUE_MARGIN and rc2d_not_too_far(best_c2, selected,
↪RC2D_TRUE_GUARD_RESCUE_SCORE_MARGIN):
                selected = best_c2
                reason = "class2_hard_floor_rescue"
                class2_rescue = True

        # 4) Weak-class rescue for class 5 preservation.
        selected_c5 = rc2d_candidate_class5(selected)
        if rc2c_is_finite(selected_c5) and selected_c5 < RC2D_HARD_CLASS5_FLOOR:
            c5_candidates = [c for c in scored if
↪rc2c_is_finite(rc2d_candidate_class5(c))]
            if c5_candidates:
                best_c5 = max(c5_candidates, key=lambda c:
↪(rc2d_candidate_class5(c), float(c.get("selection_score", -1e9))))
                if rc2d_candidate_class5(best_c5) >= selected_c5 +
↪RC2D_CLASS5_RESCUE_MARGIN and rc2d_not_too_far(best_c5, selected,
↪RC2D_TRUE_GUARD_RESCUE_SCORE_MARGIN):
                    selected = best_c5
                    reason = "class5_hard_floor_rescue"
                    class5_rescue = True

        # 5) Hard-floor fallback: prefer a floor-safe candidate if close.
        if RC2C_FLOOR_RISK_FALLBACK_ENABLED and not
↪rc2c_candidate_meets_hard_floors(selected):
            floor_safe = [c for c in scored if rc2c_candidate_meets_hard_floors(c)]
            if floor_safe:
                best_floor_safe = max(floor_safe, key=lambda c: float(c.
↪get("selection_score", -1e9)))
                if rc2d_not_too_far(best_floor_safe, selected,
↪RC2D_GUARDED_HYSTERESIS_MARGIN):
                    selected = best_floor_safe
                    reason = "hard_floor_risk_fallback"
                    floor_fallback = True

        # 6) Reduced guarded hysteresis: apply only if guarded is floor-safe and
↪not weaker on class 2.
        if (
            best_guarded is not None
            and not rc2c_is_guarded_context_candidate(selected["variant"])

```

```

        and rc2d_not_too_far(best_guarded, selected,
↪RC2D_GUARDED_HYSTERESIS_MARGIN)
        and rc2c_candidate_meets_hard_floors(best_guarded)
    ):
        bg_c2 = rc2d_candidate_class2(best_guarded)
        se_c2 = rc2d_candidate_class2(selected)
        not_c2_worse = (not rc2c_is_finite(bg_c2)) or (not
↪rc2c_is_finite(se_c2)) or bg_c2 >= se_c2 - 0.03
        if not_c2_worse:
            selected = best_guarded
            reason = "reduced_guarded_context_gap_hysteresis"
            hysteresis = True

    return selected, {
        "raw_best_variant": raw_best["variant"],
        "raw_best_policy_family": raw_best["policy_family"],
        "raw_best_proxy_score": raw_best.get("selection_score", np.nan),
        "best_guarded_variant": best_guarded["variant"] if best_guarded is not
↪None else "",
        "best_guarded_proxy_score": best_guarded.get("selection_score", np.nan)
↪if best_guarded is not None else np.nan,
        "best_true_guard_variant": best_true_guard["variant"] if
↪best_true_guard is not None else "",
        "best_true_guard_proxy_score": best_true_guard.get("selection_score",
↪np.nan) if best_true_guard is not None else np.nan,
        "selection_reason": reason,
        "no_repair_veto_applied": no_repair_veto,
        "guarded_hysteresis_applied": hysteresis,
        "risk_floor_fallback_applied": floor_fallback,
        "true_guard_rescue_applied": true_guard_rescue,
        "class2_rescue_applied": class2_rescue,
        "class5_rescue_applied": class5_rescue,
        "rc2d_selector_version": RC2D_SELECTOR_VERSION,
    }

def build_rc2b_candidate_from_checkpoint(base_run, checkpoint, variant: str,
                                         repair_memory,
↪context_selection_memory, policy_validation_memory,
                                         seed=0):
    """Override: build one candidate and score it on memory-only primary +
↪auxiliary validation splits."""
    candidate_seed = int(seed)
    cp_seed = int(checkpoint["seed"])
    after_task = int(checkpoint["after_task"])
    method_name = rc2b_candidate_method_name(variant)

```

```

base_cfg = METHOD_CONFIGS["mmals_proto_first_balanced_replay"]
model = restore_base_model_from_checkpoint(checkpoint, base_cfg)
context_prototypes = clone_prototypes(checkpoint["context_prototypes"])

config, config_rows = build_context_variant_config_and_logs(
    model, context_selection_memory, context_prototypes,
    variant=variant, seed=candidate_seed + 313,
)

readout = GlobalLinearReadout(model.global_head).to(DEVICE)
repair_rows = list(config_rows)
pre_global_head = copy.deepcopy(model.global_head).to(DEVICE)
pre_global_head.eval()
for p in pre_global_head.parameters():
    p.requires_grad_(False)

if variant_uses_guarded_head(variant):
    repair_rows.extend(train_readout_on_memory(
        readout, model, repair_memory, config,
        prototypes=context_prototypes, pre_global_head=pre_global_head,
        kl_weight=0.35, use_kl=True, margin_weight=0.0,
        variant=f"{variant}_rc2d_guarded_policy_candidate",
    ))

primary_proxy = proxy_metrics_for_context_config(
    model, policy_validation_memory, config,
    prototypes=context_prototypes, readout=readout,
    max_items_per_class=VALIDATION_MAX_ITEMS_PER_CLASS,
)
auxiliary_proxy = proxy_metrics_for_context_config(
    model, context_selection_memory, config,
    prototypes=context_prototypes, readout=readout,
    max_items_per_class=VALIDATION_MAX_ITEMS_PER_CLASS,
) if RC2D_AUX_PROXY_WEIGHT > 0 else None
proxy = rc2d_merge_proxy_metrics(primary_proxy, auxiliary_proxy)

for rr in repair_rows:
    rr.update({
        "method": method_name,
        "seed": cp_seed,
        "after_task": after_task,
        "task_id": after_task,
        "variant": variant,
        "repair_type": str(rr.get("repair_type", "")),
        "rc2b_strict_candidate": True,
        "rc2d_gate_aligned_candidate": True,
    })

```

```

        "rc2b_no_final_test_used_for_selection": True,
        "policy_family": rc2b_policy_family(variant),
        "uses_guarded_global_head": variant_uses_guarded_head(variant),
        "uses_context_selection": context_variant_selection_mode(variant)
    is not None,
        "uses_oracle_context_reference":
    is_oracle_context_reference(variant),
    })

    return {
        "variant": variant,
        "method": method_name,
        "policy_family": rc2b_policy_family(variant),
        "policy_label": rc2b_policy_label(variant),
        "complexity_rank": rc2b_policy_complexity_rank(variant),
        "model": model,
        "readout": readout,
        "config": config,
        "context_prototypes": context_prototypes,
        "proxy": proxy,
        "primary_proxy": primary_proxy,
        "auxiliary_proxy": auxiliary_proxy or {},
        "repair_rows": repair_rows,
    }

print("RC2d gate-aligned selector overrides loaded:", RC2D_SELECTOR_VERSION)
print("RC2D_AUX_PROXY_WEIGHT:", RC2D_AUX_PROXY_WEIGHT)
print("RC2D_CLASS5_FLOOR:", RC2D_CLASS5_FLOOR, "RC2D_PROXY_ACC_FLOOR:",
    RC2D_PROXY_ACC_FLOOR)

```

```

RC2d gate-aligned selector overrides loaded: RC2d_gate_aligned_v1
RC2D_AUX_PROXY_WEIGHT: 0.35
RC2D_CLASS5_FLOOR: 0.9 RC2D_PROXY_ACC_FLOOR: 0.68

```

```

[65]: # -----
# v1.1-RC2g tri-boundary equilibrium selector/readout overrides
# -----
# RC2g patches the RC2D smoke failure mode without changing the backbone:
# - class-specific metrics are ignored until the class is available;
# - NaN metrics never poison the candidate score;
# - all-NaN selector fallback is replaced by a deterministic memory-only
    emergency score;
# - guarded readout candidates receive a small class-balanced calibration on
    repair memory.
# All operations remain strict: repair/context-selection/policy-validation
    memory only.
# -----

```

```

RC2G_SELECTOR_VERSION = globals().get("RC2G_SELECTOR_VERSION",
    ↪ "RC2g_nan_safe_class_balanced_v1")
RC2G_NAN_SAFE_SCORING_ENABLED = globals().get("RC2G_NAN_SAFE_SCORING_ENABLED",
    ↪ True)
RC2G_CLASS_AVAILABILITY_GATING_ENABLED = globals().
    ↪ get("RC2G_CLASS_AVAILABILITY_GATING_ENABLED", True)
RC2G_EMERGENCY_SCORE_IF_ALL_NAN = globals().
    ↪ get("RC2G_EMERGENCY_SCORE_IF_ALL_NAN", True)
RC2G_CLASS_BALANCED_READOUT_CALIBRATION_ENABLED = globals().
    ↪ get("RC2G_CLASS_BALANCED_READOUT_CALIBRATION_ENABLED", True)
RC2G_CLASS_BALANCED_CALIB_STEPS = globals().
    ↪ get("RC2G_CLASS_BALANCED_CALIB_STEPS", 12 if RUN_MODE == "smoke" else 30)
RC2G_CLASS_BALANCED_CALIB_LR = globals().get("RC2G_CLASS_BALANCED_CALIB_LR", 1.
    ↪ 5e-3)
RC2G_WEAK_CLASS_WEIGHT = globals().get("RC2G_WEAK_CLASS_WEIGHT", 2.50)
RC2G_MAX_CLASS_WEIGHT = globals().get("RC2G_MAX_CLASS_WEIGHT", 4.00)
RC2G_CLASS_BALANCED_MARGIN_WEIGHT = globals().
    ↪ get("RC2G_CLASS_BALANCED_MARGIN_WEIGHT", 0.35)
RC2G_SCORE_NEUTRAL_VALUE = globals().get("RC2G_SCORE_NEUTRAL_VALUE", 0.0)

if globals().get("POLICY_BRANCH", "RC2g") == "RC2g":
    RC2C_METHOD = "paired_rc2g_tri_boundary_equilibrium_policy"
    RC2B_METHOD = RC2C_METHOD
    RC3_METHOD = RC2C_METHOD

def rc2f_seen_classes(after_task=None):
    if after_task is None:
        return set(range(N_CLASSES))
    try:
        return set(seen_classes_until(int(after_task)))
    except Exception:
        return set(range(N_CLASSES))

def rc2f_class_applicable(class_id: int, after_task=None) -> bool:
    if not RC2G_CLASS_AVAILABILITY_GATING_ENABLED:
        return True
    return int(class_id) in rc2f_seen_classes(after_task)

def rc2f_finite(x) -> bool:
    return rc2c_is_finite(x)

```



```

def rc2f_metric(proxy, key, neutral=RC2G_SCORE_NEUTRAL_VALUE):
    value = rc2c_proxy_value(proxy, key, np.nan)
    return float(value) if rc2f_finite(value) else float(neutral)

def rc2f_class_metric(proxy, key, class_id: int, after_task=None,
    ↪neutral=RC2G_SCORE_NEUTRAL_VALUE):
    # If class is not yet part of the seen history, the metric is not
    ↪applicable.
    if not rc2f_class_applicable(class_id, after_task):
        return float(neutral), False
    value = rc2c_proxy_value(proxy, key, np.nan)
    if rc2f_finite(value):
        return float(value), True
    # Seen but absent from the small validation split: keep score finite and
    ↪audit as missing.
    return float(neutral), False

def rc2f_floor_deficit_metric(proxy, key, floor, class_id=None,
    ↪after_task=None):
    if class_id is not None and not rc2f_class_applicable(class_id, after_task):
        return 0.0
    value = rc2c_proxy_value(proxy, key, np.nan)
    if not rc2f_finite(value):
        return 0.0
    return max(0.0, float(floor) - float(value))

def rc2c_candidate_meets_hard_floors(candidate) -> bool:
    """RC2g class-aware hard-floor check.

    Class-specific floors are enforced only after the class has appeared in the
    benchmark history. Missing class metrics from tiny memory splits do not make
    the score non-finite and do not trigger accidental fallback.
    """
    p = candidate.get("proxy", {})
    after_task = candidate.get("after_task", None)
    c2 = rc2c_proxy_value(p, "proxy_class2")
    c5 = rc2c_proxy_value(p, "proxy_class5")
    min_task = rc2c_proxy_value(p, "proxy_min_task", rc2c_proxy_value(p,
    ↪"proxy_min_class"))
    min_class = rc2c_proxy_value(p, "proxy_min_class")
    acc = rc2c_proxy_value(p, "proxy_acc")
    ok_c2 = (not rc2f_class_applicable(2, after_task)) or (not rc2f_finite(c2))
    ↪or c2 >= RC2C_HARD_CLASS2_FLOOR

```

```

    ok_c5 = (not rc2f_class_applicable(5, after_task)) or (not rc2f_finite(c5))
    or c5 >= RC2D_HARD_CLASS5_FLOOR
    ok_min_task = (not rc2f_finite(min_task)) or min_task >=
    RC2C_HARD_MIN_TASK_FLOOR
    ok_min_class = (not rc2f_finite(min_class)) or min_class >=
    RC2C_HARD_MIN_CLASS_FLOOR
    ok_acc = (not rc2f_finite(acc)) or acc >= max(0.50, RC2D_PROXY_ACC_FLOOR -
    0.05)
    return bool(ok_c2 and ok_c5 and ok_min_task and ok_min_class and ok_acc)

def rc2c_risk_components(proxy, baseline_proxy, variant: str, after_task=None):
    """RC2g NaN-safe risk components, still validation-memory-only."""
    acc = rc2f_metric(proxy, "proxy_acc", neutral=0.0)
    c2, c2_available = rc2f_class_metric(proxy, "proxy_class2", 2,
    after_task=after_task, neutral=0.0)
    c5, c5_available = rc2f_class_metric(proxy, "proxy_class5", 5,
    after_task=after_task, neutral=0.0)
    min_class = rc2f_metric(proxy, "proxy_min_class", neutral=acc)
    min_task = rc2f_metric(proxy, "proxy_min_task", neutral=min_class)
    old_task_min = rc2f_metric(proxy, "proxy_old_task_min", neutral=min_task)
    task_std = rc2f_metric(proxy, "proxy_task_acc_std", neutral=0.0)
    family = rc2b_policy_family(variant)

    class2_penalty = RC2C_CLASS2_FLOOR_PENALTY *
    rc2f_floor_deficit_metric(proxy, "proxy_class2", RC2C_CLASS2_FLOOR,
    class_id=2, after_task=after_task)
    class5_penalty = RC2D_CLASS5_FLOOR_PENALTY *
    rc2f_floor_deficit_metric(proxy, "proxy_class5", RC2D_CLASS5_FLOOR,
    class_id=5, after_task=after_task)
    min_class_penalty = RC2C_MIN_CLASS_FLOOR_PENALTY *
    rc2f_floor_deficit_metric(proxy, "proxy_min_class", RC2C_MIN_CLASS_FLOOR,
    after_task=after_task)
    min_task_penalty = RC2C_MIN_TASK_FLOOR_PENALTY *
    rc2f_floor_deficit_metric(proxy, "proxy_min_task", RC2C_MIN_TASK_FLOOR,
    after_task=after_task)
    old_task_penalty = RC2C_OLD_TASK_FLOOR_PENALTY *
    rc2f_floor_deficit_metric(proxy, "proxy_old_task_min", RC2C_OLD_TASK_FLOOR,
    after_task=after_task)
    acc_penalty = RC2D_PROXY_ACC_FLOOR_PENALTY *
    rc2f_floor_deficit_metric(proxy, "proxy_acc", RC2D_PROXY_ACC_FLOOR,
    after_task=after_task)
    floor_penalty = class2_penalty + class5_penalty + min_class_penalty +
    min_task_penalty + old_task_penalty + acc_penalty

    dispersion_penalty = RC2D_TASK_STD_WEIGHT * max(0.0, task_std)

```

```

no_repair_soft_penalty = 0.0
if after_task is not None and int(after_task) >= 0
↳ RC2C_NO_REPAIR_VETO_AFTER_TASK and family == "proto_no_repair":
    no_repair_soft_penalty = RC2C_NO_REPAIR_SOFT_PENALTY

guarded_bonus = RC2D_CONTEXT_GUARD_BONUS if
↳ rc2c_is_guarded_context_candidate(variant) else 0.0
    true_guard_bonus = RC2D_TRUE_GUARD_BONUS if family ==
↳ "proto_true_global_guard" else 0.0
    complexity_penalty = RC2B_COMPLEXITY_PENALTY *
↳ rc2b_policy_complexity_rank(variant)
    class5_bonus = RC2B_VALIDATION_SCORE_CLASS5_WEIGHT * c5 if c5_available
↳ else 0.0
    hydro_penalty = rc3_hydro_selection_penalty(proxy)

return {
    "proxy_acc_floor_penalty": acc_penalty,
    "class2_floor_penalty": class2_penalty,
    "class5_floor_penalty": class5_penalty,
    "min_class_floor_penalty": min_class_penalty,
    "min_task_floor_penalty": min_task_penalty,
    "old_task_floor_penalty": old_task_penalty,
    "floor_penalty": floor_penalty,
    "task_dispersion_penalty": dispersion_penalty,
    "no_repair_soft_penalty": no_repair_soft_penalty,
    "complexity_penalty": complexity_penalty,
    "class5_bonus": class5_bonus,
    "guarded_context_bonus": guarded_bonus,
    "true_guard_bonus": true_guard_bonus,
    "hydro_selection_penalty": hydro_penalty,
    "total_risk_penalty": floor_penalty + dispersion_penalty +
↳ no_repair_soft_penalty + complexity_penalty + hydro_penalty,
    "rc2f_class2_applicable": rc2f_class_applicable(2, after_task),
    "rc2f_class5_applicable": rc2f_class_applicable(5, after_task),
    "rc2f_class2_available_in_proxy": bool(c2_available),
    "rc2f_class5_available_in_proxy": bool(c5_available),
}

def rc2f_nan_safe_validation_base(proxy, baseline_proxy):
    base = validation_score(proxy, baseline_proxy)
    if rc2f_finite(base) and base > -1e8:
        return float(base)
    # Fallback to average accuracy if the composite score is non-finite.
    acc = rc2f_metric(proxy, "proxy_acc", neutral=0.0)
    min_class = rc2f_metric(proxy, "proxy_min_class", neutral=acc)

```

```

    return 0.80 * acc + 0.20 * min_class

def rc2d_gate_aligned_score(proxy, baseline_proxy, variant: str,
    ↪after_task=None):
    """RC2g NaN-safe gate-aligned score."""
    base_score = rc2f_nan_safe_validation_base(proxy, baseline_proxy)
    acc = rc2f_metric(proxy, "proxy_acc", neutral=0.0)
    c2, _ = rc2f_class_metric(proxy, "proxy_class2", 2, after_task=after_task,
    ↪neutral=0.0)
    c5, _ = rc2f_class_metric(proxy, "proxy_class5", 5, after_task=after_task,
    ↪neutral=0.0)
    min_class = rc2f_metric(proxy, "proxy_min_class", neutral=acc)
    min_task = rc2f_metric(proxy, "proxy_min_task", neutral=min_class)
    old_task_min = rc2f_metric(proxy, "proxy_old_task_min", neutral=min_task)
    task_std = rc2f_metric(proxy, "proxy_task_acc_std", neutral=0.0)
    comp = rc2c_risk_components(proxy, baseline_proxy, variant,
    ↪after_task=after_task)

    applicable_weak_classes = []
    if rc2f_class_applicable(2, after_task):
        applicable_weak_classes.append(c2)
    if rc2f_class_applicable(5, after_task):
        applicable_weak_classes.append(c5)
    weak_class_score = float(np.mean(applicable_weak_classes)) if
    ↪applicable_weak_classes else 0.0

    balance_score = (
        acc
        + RC2D_CLASS_BALANCE_WEIGHT * weak_class_score
        + RC2D_MIN_TASK_WEIGHT * min_task
        + RC2D_OLD_TASK_WEIGHT * old_task_min
        - RC2D_TASK_STD_WEIGHT * max(0.0, task_std)
    )
    score = float(
        0.50 * base_score
        + 0.50 * balance_score
        + comp["class5_bonus"]
        + comp["guarded_context_bonus"]
        + comp["true_guard_bonus"]
        - comp["total_risk_penalty"]
    )
    return score if rc2f_finite(score) else -1e9

def rc2b_validation_score(proxy, baseline_proxy, variant: str, after_task=None):
    """Override: RC2g NaN-safe score, with RC2b strict fallback."""

```

```

    if not RC2C_RISK_AWARE_SELECTOR_ENABLED or globals().get("POLICY_BRANCH")_
    ⇨== "RC2b":
        base_score = rc2f_nan_safe_validation_base(proxy, baseline_proxy)
        c2 = rc2c_proxy_value(proxy, "proxy_class2", np.nan)
        c5 = rc2c_proxy_value(proxy, "proxy_class5", np.nan)
        minc = rc2c_proxy_value(proxy, "proxy_min_class", np.nan)
        penalty = RC2B_COMPLEXITY_PENALTY * rc2b_policy_complexity_rank(variant)
        floor_penalty = 0.0
        if rc2f_class_applicable(2, after_task) and rc2f_finite(c2) and c2 <_
    ⇨RC2B_CLASS2_FLOOR:
            floor_penalty += RC2B_VALIDATION_SCORE_FLOOR_PENALTY *_
    ⇨(RC2B_CLASS2_FLOOR - c2)
            if rc2f_class_applicable(5, after_task) and rc2f_finite(c5) and c5 <_
    ⇨RC2B_CLASS5_FLOOR:
                floor_penalty += RC2B_VALIDATION_SCORE_FLOOR_PENALTY *_
    ⇨(RC2B_CLASS5_FLOOR - c5)
                if rc2f_finite(minc) and minc < RC2B_MIN_CLASS_FLOOR:
                    floor_penalty += RC2B_VALIDATION_SCORE_FLOOR_PENALTY *_
    ⇨(RC2B_MIN_CLASS_FLOOR - minc)
                    class5_bonus = RC2B_VALIDATION_SCORE_CLASS5_WEIGHT * (c5 if_
    ⇨rc2f_class_applicable(5, after_task) and rc2f_finite(c5) else 0.0)
                    hydro_penalty = rc3_hydro_selection_penalty(proxy)
                    score = float(base_score + class5_bonus - penalty - floor_penalty -_
    ⇨hydro_penalty)
                    return score if rc2f_finite(score) else -1e9
                return rc2d_gate_aligned_score(proxy, baseline_proxy, variant,_
    ⇨after_task=after_task)

def rc2f_emergency_candidate_score(candidate, after_task=None):
    """Deterministic memory-only fallback when every candidate score is NaN."""
    p = candidate.get("proxy", {})
    acc = rc2f_metric(p, "proxy_acc", neutral=0.0)
    min_class = rc2f_metric(p, "proxy_min_class", neutral=acc)
    min_task = rc2f_metric(p, "proxy_min_task", neutral=min_class)
    c2, _ = rc2f_class_metric(p, "proxy_class2", 2, after_task=after_task,_
    ⇨neutral=0.0)
    c5, _ = rc2f_class_metric(p, "proxy_class5", 5, after_task=after_task,_
    ⇨neutral=0.0)
    weak_terms = []
    if rc2f_class_applicable(2, after_task):
        weak_terms.append(c2)
    if rc2f_class_applicable(5, after_task):
        weak_terms.append(c5)
    weak = float(np.mean(weak_terms)) if weak_terms else 0.0
    score = 0.55 * acc + 0.20 * min_task + 0.20 * min_class + 0.05 * weak

```

```

        score -= RC2B_COMPLEXITY_PENALTY * rc2b_policy_complexity_rank(candidate.
↪get("variant", ""))
        return float(score) if rc2f_finite(score) else -1e9

def rc2d_candidate_proxy_acc(c):
    return rc2f_metric(c.get("proxy", {}), "proxy_acc", neutral=-1e9)

def rc2d_candidate_class2(c):
    value, _ = rc2f_class_metric(c.get("proxy", {}), "proxy_class2", 2,
↪after_task=c.get("after_task"), neutral=np.nan)
    return value

def rc2d_candidate_class5(c):
    value, _ = rc2f_class_metric(c.get("proxy", {}), "proxy_class5", 5,
↪after_task=c.get("after_task"), neutral=np.nan)
    return value

def rc2c_select_candidate(candidates, after_task):
    """Override: RC2g tri-boundary equilibrium selector."""
    emergency_scoring_used = False
    scored = [c for c in candidates if rc2f_finite(c.get("selection_score",
↪-1e9))]
    if not scored and RC2G_EMERGENCY_SCORE_IF_ALL_NAN:
        for c in candidates:
            c["selection_score"] = rc2f_emergency_candidate_score(c,
↪after_task=after_task)
            c["rc2f_emergency_score_used"] = True
        scored = [c for c in candidates if rc2f_finite(c.get("selection_score",
↪-1e9))]
        emergency_scoring_used = True
    if not scored:
        selected = max(candidates, key=lambda c: (rc2d_candidate_proxy_acc(c),
↪-rc2b_policy_complexity_rank(c.get("variant", ""))))
    return selected, {
        "raw_best_variant": selected["variant"],
        "selection_reason": "fallback_no_finite_scores_proxy_acc_tiebreak",
        "no_repair_veto_applied": False,
        "guarded_hysteresis_applied": False,
        "risk_floor_fallback_applied": False,
        "true_guard_rescue_applied": False,
        "class2_rescue_applied": False,
        "class5_rescue_applied": False,
    }

```

```

        "rc2d_selector_version": RC2G_SELECTOR_VERSION,
        "rc2f_selector_version": RC2G_SELECTOR_VERSION,
        "rc2f_emergency_scoring_used": True,
        "rc2f_no_finite_scores_after_emergency": True,
    }

    sorted_by_score = sorted(
        scored,
        key=lambda c: (
            float(c.get("selection_score", -1e9)),
            rc2d_candidate_proxy_acc(c),
            -float(c["complexity_rank"]),
        ),
        reverse=True,
    )
    raw_best = sorted_by_score[0]
    selected = raw_best
    reason = "direct_rc2f_nan_safe_class_balanced_score_win" if not
    ↪emergency_scoring_used else "rc2g_emergency_score_win"
    no_repair_veto = False
    hysteresis = False
    floor_fallback = False
    true_guard_rescue = False
    class2_rescue = False
    class5_rescue = False

    guarded_candidates = [c for c in scored if
    ↪rc2c_is_guarded_context_candidate(c["variant"])]
    best_guarded = max(guarded_candidates, key=lambda c: float(c.
    ↪get("selection_score", -1e9))) if guarded_candidates else None
    true_guard_candidates = [c for c in scored if
    ↪rc2b_policy_family(c["variant"]) == "proto_true_global_guard"]
    best_true_guard = max(true_guard_candidates, key=lambda c: float(c.
    ↪get("selection_score", -1e9))) if true_guard_candidates else None

    if (
        best_guarded is not None
        and int(after_task) >= RC2C_NO_REPAIR_VETO_AFTER_TASK
        and rc2b_policy_family(raw_best["variant"]) == "proto_no_repair"
    ):
        margin = float(raw_best.get("selection_score", -1e9)) -
    ↪float(best_guarded.get("selection_score", -1e9))
        if margin < RC2C_NO_REPAIR_REQUIRED_MARGIN:
            selected = best_guarded
            reason = f"no_repair_veto_guarded_fallback_margin_{margin:+.4f}"
            no_repair_veto = True

```

```

    if best_true_guard is not None and rc2d_not_too_far(best_true_guard,
↪selected, RC2D_TRUE_GUARD_RESCUE_SCORE_MARGIN):
        tg_c2 = rc2d_candidate_class2(best_true_guard)
        se_c2 = rc2d_candidate_class2(selected)
        tg_acc = rc2d_candidate_proxy_acc(best_true_guard)
        se_acc = rc2d_candidate_proxy_acc(selected)
        c2_gain = (rc2f_class_applicable(2, after_task) and rc2f_finite(tg_c2))
↪and rc2f_finite(se_c2) and tg_c2 >= se_c2 + RC2D_TRUE_GUARD_CLASS2_MARGIN)
        acc_gain = (rc2f_finite(tg_acc) and rc2f_finite(se_acc) and tg_acc >=
↪se_acc + RC2D_TRUE_GUARD_ACC_MARGIN)
        if (c2_gain or acc_gain) and
↪rc2c_candidate_meets_hard_floors(best_true_guard):
            selected = best_true_guard
            reason = "true_global_guard_rescue_for_class2_or_accuracy_alignment"
            true_guard_rescue = True

    if rc2f_class_applicable(2, after_task):
        selected_c2 = rc2d_candidate_class2(selected)
        if rc2f_finite(selected_c2) and selected_c2 < RC2C_HARD_CLASS2_FLOOR:
            c2_candidates = [c for c in scored if
↪rc2f_finite(rc2d_candidate_class2(c))]
            if c2_candidates:
                best_c2 = max(c2_candidates, key=lambda c:
↪(rc2d_candidate_class2(c), float(c.get("selection_score", -1e9))))
                if rc2d_candidate_class2(best_c2) >= selected_c2 +
↪RC2D_CLASS2_RESCUE_MARGIN and rc2d_not_too_far(best_c2, selected,
↪RC2D_TRUE_GUARD_RESCUE_SCORE_MARGIN):
                    selected = best_c2
                    reason = "class2_hard_floor_rescue"
                    class2_rescue = True

    if rc2f_class_applicable(5, after_task):
        selected_c5 = rc2d_candidate_class5(selected)
        if rc2f_finite(selected_c5) and selected_c5 < RC2D_HARD_CLASS5_FLOOR:
            c5_candidates = [c for c in scored if
↪rc2f_finite(rc2d_candidate_class5(c))]
            if c5_candidates:
                best_c5 = max(c5_candidates, key=lambda c:
↪(rc2d_candidate_class5(c), float(c.get("selection_score", -1e9))))
                if rc2d_candidate_class5(best_c5) >= selected_c5 +
↪RC2D_CLASS5_RESCUE_MARGIN and rc2d_not_too_far(best_c5, selected,
↪RC2D_TRUE_GUARD_RESCUE_SCORE_MARGIN):
                    selected = best_c5
                    reason = "class5_hard_floor_rescue"
                    class5_rescue = True

```



```

    if RC2C_FLOOR_RISK_FALLBACK_ENABLED and not
→rc2c_candidate_meets_hard_floors(selected):
        floor_safe = [c for c in scored if rc2c_candidate_meets_hard_floors(c)]
        if floor_safe:
            best_floor_safe = max(floor_safe, key=lambda c: float(c.
→get("selection_score", -1e9)))
            if rc2d_not_too_far(best_floor_safe, selected,
→RC2D_GUARDED_HYSTERESIS_MARGIN):
                selected = best_floor_safe
                reason = "hard_floor_risk_fallback"
                floor_fallback = True

    if (
        best_guarded is not None
        and not rc2c_is_guarded_context_candidate(selected["variant"])
        and rc2d_not_too_far(best_guarded, selected,
→RC2D_GUARDED_HYSTERESIS_MARGIN)
        and rc2c_candidate_meets_hard_floors(best_guarded)
    ):
        bg_c2 = rc2d_candidate_class2(best_guarded)
        se_c2 = rc2d_candidate_class2(selected)
        not_c2_worse = (not rc2f_class_applicable(2, after_task)) or (not
→rc2f_finite(bg_c2)) or (not rc2f_finite(se_c2)) or bg_c2 >= se_c2 - 0.03
        if not_c2_worse:
            selected = best_guarded
            reason = "reduced_guarded_context_gap_hysteresis"
            hysteresis = True

    return selected, {
        "raw_best_variant": raw_best["variant"],
        "raw_best_policy_family": raw_best["policy_family"],
        "raw_best_proxy_score": raw_best.get("selection_score", np.nan),
        "best_guarded_variant": best_guarded["variant"] if best_guarded is not
→None else "",
        "best_guarded_proxy_score": best_guarded.get("selection_score", np.nan)
→if best_guarded is not None else np.nan,
        "best_true_guard_variant": best_true_guard["variant"] if
→best_true_guard is not None else "",
        "best_true_guard_proxy_score": best_true_guard.get("selection_score",
→np.nan) if best_true_guard is not None else np.nan,
        "selection_reason": reason,
        "no_repair_veto_applied": no_repair_veto,
        "guarded_hysteresis_applied": hysteresis,
        "risk_floor_fallback_applied": floor_fallback,
        "true_guard_rescue_applied": true_guard_rescue,

```

```

        "class2_rescue_applied": class2_rescue,
        "class5_rescue_applied": class5_rescue,
        "rc2d_selector_version": RC2G_SELECTOR_VERSION,
        "rc2f_selector_version": RC2G_SELECTOR_VERSION,
        "rc2f_emergency_scoring_used": emergency_scoring_used,
        "rc2f_no_finite_scores_after_emergency": False,
        "rc2f_class2_applicable": rc2f_class_applicable(2, after_task),
        "rc2f_class5_applicable": rc2f_class_applicable(5, after_task),
    }

def rc2f_memory_class_support(class_memory):
    support = {c: 0 for c in range(N_CLASSES)}
    for cls, data in (class_memory or {}).items():
        try:
            support[int(cls)] += int(len(data["y"]))
        except Exception:
            pass
    return support

def rc2f_class_weights_for_memory(class_memory, after_task=None):
    support = rc2f_memory_class_support(class_memory)
    weights = torch.ones(N_CLASSES, device=DEVICE)
    positive_counts = [v for v in support.values() if v > 0]
    max_count = max(positive_counts) if positive_counts else 1
    for cls, count in support.items():
        if count > 0:
            weights[int(cls)] = min(RC2G_MAX_CLASS_WEIGHT, math.sqrt(max_count /
↪ max(count, 1)))
        for weak_cls in [2, 5]:
            if support.get(weak_cls, 0) > 0 and rc2f_class_applicable(weak_cls, ↪
↪ after_task):
                weights[weak_cls] = min(RC2G_MAX_CLASS_WEIGHT, ↪
↪ max(float(weights[weak_cls]), RC2G_WEAK_CLASS_WEIGHT))
    return weights

def rc2f_readout_class_balanced_calibration(readout, model, class_memory, ↪
↪ config, prototypes=None, after_task=None, steps=None, lr=None):
    """Small repair-memory-only calibration pass for guarded readouts.

    This targets the RC2C/RC2D smoke pattern where zf probes were stronger than
    the deployed global head. It does not touch final-test data.
    """
    rows = []
    if not RC2G_CLASS_BALANCED_READOUT_CALIBRATION_ENABLED or not class_memory:

```

```

        return rows
    support = rc2f_memory_class_support(class_memory)
    active_weak = [c for c in [2, 5] if support.get(c, 0) > 0 and
↪rc2f_class_applicable(c, after_task)]
    if not active_weak:
        return rows
    params = [p for p in readout.parameters() if p.requires_grad]
    if not params:
        return rows
    steps = int(RC2G_CLASS_BALANCED_CALIB_STEPS if steps is None else steps)
    lr = float(RC2G_CLASS_BALANCED_CALIB_LR if lr is None else lr)
    weights = rc2f_class_weights_for_memory(class_memory, after_task=after_task)
    model.eval()
    for p in model.parameters():
        p.requires_grad_(False)
    readout.train()
    opt = torch.optim.Adam(params, lr=lr)
    for step in range(steps):
        xb, yb, tb = sample_repair_batch_v10b(class_memory,
↪batch_size=HEAD_REPAIR_BATCH)
        if xb is None:
            continue
        xb, yb, tb = xb.to(DEVICE), yb.to(DEVICE), tb.to(DEVICE)
        zf_list, q_list, y_list = [], [], []
        with torch.no_grad():
            for src_task_id in torch.unique(tb).detach().cpu().tolist():
                src_task_id = int(src_task_id)
                mask = (tb == src_task_id)
                if not mask.any():
                    continue
                _, cache = forward_by_method(model, xb[mask], src_task_id,
↪config, prototypes=prototypes, train=False, y=yb[mask], return_cache=True)
                zf_list.append(cache["zf"].detach())
                q_list.append(cache["q_context"].detach())
                y_list.append(yb[mask].detach())
        if not zf_list:
            continue
        zf = torch.cat(zf_list, dim=0)
        q_context = torch.cat(q_list, dim=0)
        y_mix = torch.cat(y_list, dim=0)
        opt.zero_grad(set_to_none=True)
        logits = readout(zf, q_context)
        ce = F.cross_entropy(logits, y_mix, weight=weights)
        margin = head_margin_repair_loss(logits, y_mix,
↪hard_classes=active_weak)
        loss = ce + RC2G_CLASS_BALANCED_MARGIN_WEIGHT * margin
        loss.backward()

```

```

torch.nn.utils.clip_grad_norm_(params, max_norm=5.0)
opt.step()
rows.append({
    "repair_epoch": step,
    "repair_type": "rc2f_class_balanced_readout_calibration",
    "rc2f_active_weak_classes": str(active_weak),
    "rc2f_class_balanced_loss": float(loss.detach().cpu()),
    "rc2f_class_balanced_ce": float(ce.detach().cpu()),
    "rc2f_class_balanced_margin": float(margin.detach().cpu()),
    "rc2f_class_weights": safe_jsonable(weights.detach().cpu()),
    "rc2f_selector_version": RC2G_SELECTOR_VERSION,
})
readout.eval()
return rows

def build_rc2b_candidate_from_checkpoint(base_run, checkpoint, variant: str,
                                         repair_memory,
                                         context_selection_memory, policy_validation_memory,
                                         seed=0):
    """Override: build one RC2G candidate with NaN-safe proxy and
    class-balanced guarded readout calibration."""
    candidate_seed = int(seed)
    cp_seed = int(checkpoint["seed"])
    after_task = int(checkpoint["after_task"])
    method_name = rc2b_candidate_method_name(variant)

    base_cfg = METHOD_CONFIGS["mmals_proto_first_balanced_replay"]
    model = restore_base_model_from_checkpoint(checkpoint, base_cfg)
    context_prototypes = clone_prototypes(checkpoint["context_prototypes"])

    config, config_rows = build_context_variant_config_and_logs(
        model, context_selection_memory, context_prototypes,
        variant=variant, seed=candidate_seed + 313,
    )

    readout = GlobalLinearReadout(model.global_head).to(DEVICE)
    repair_rows = list(config_rows)
    pre_global_head = copy.deepcopy(model.global_head).to(DEVICE)
    pre_global_head.eval()
    for p in pre_global_head.parameters():
        p.requires_grad_(False)

    if variant_uses_guarded_head(variant):
        repair_rows.extend(train_readout_on_memory(
            readout, model, repair_memory, config,
            prototypes=context_prototypes, pre_global_head=pre_global_head,

```

```

        kl_weight=0.35, use_kl=True, margin_weight=0.0,
        variant=f"{variant}_rc2f_guarded_policy_candidate",
    ))
    repair_rows.extend(rc2f_readout_class_balanced_calibration(
        readout, model, repair_memory, config,
        prototypes=context_prototypes, after_task=after_task,
    ))

    primary_proxy = proxy_metrics_for_context_config(
        model, policy_validation_memory, config,
        prototypes=context_prototypes, readout=readout,
        max_items_per_class=VALIDATION_MAX_ITEMS_PER_CLASS,
    )
    auxiliary_proxy = proxy_metrics_for_context_config(
        model, context_selection_memory, config,
        prototypes=context_prototypes, readout=readout,
        max_items_per_class=VALIDATION_MAX_ITEMS_PER_CLASS,
    ) if RC2D_AUX_PROXY_WEIGHT > 0 else None
    proxy = rc2d_merge_proxy_metrics(primary_proxy, auxiliary_proxy)

    for rr in repair_rows:
        rr.update({
            "method": method_name,
            "seed": cp_seed,
            "after_task": after_task,
            "task_id": after_task,
            "variant": variant,
            "repair_type": str(rr.get("repair_type", "")),
            "rc2b_strict_candidate": True,
            "rc2d_gate_aligned_candidate": True,
            "rc2f_nan_safe_class_balanced_candidate": True,
            "rc2b_no_final_test_used_for_selection": True,
            "policy_family": rc2b_policy_family(variant),
            "uses_guarded_global_head": variant_uses_guarded_head(variant),
            "uses_context_selection": context_variant_selection_mode(variant)
        })
        if is_not_None,
            "uses_oracle_context_reference": is_oracle_context_reference(variant),
            "rc2f_class2_applicable": rc2f_class_applicable(2, after_task),
            "rc2f_class5_applicable": rc2f_class_applicable(5, after_task),
        })

    return {
        "variant": variant,
        "method": method_name,
        "policy_family": rc2b_policy_family(variant),
        "policy_label": rc2b_policy_label(variant),
    }

```

```

    "complexity_rank": rc2b_policy_complexity_rank(variant),
    "model": model,
    "readout": readout,
    "config": config,
    "context_prototypes": context_prototypes,
    "proxy": proxy,
    "primary_proxy": primary_proxy,
    "auxiliary_proxy": auxiliary_proxy or {},
    "repair_rows": repair_rows,
    "after_task": after_task,
    "rc2f_selector_version": RC2G_SELECTOR_VERSION,
}

print("RC2g tri-boundary equilibrium overrides loaded:", RC2G_SELECTOR_VERSION)
print("RC2G_CLASS_BALANCED_READOUT_CALIBRATION_ENABLED:", ☐,
      ↪RC2G_CLASS_BALANCED_READOUT_CALIBRATION_ENABLED)
print("RC2G_EMERGENCY_SCORE_IF_ALL_NAN:", RC2G_EMERGENCY_SCORE_IF_ALL_NAN)

```

RC2g tri-boundary equilibrium overrides loaded:
 RC2g_tri_boundary_class_balanced_v1
 RC2G_CLASS_BALANCED_READOUT_CALIBRATION_ENABLED: True
 RC2G_EMERGENCY_SCORE_IF_ALL_NAN: True

```

[66]: # -----
# v1.1-RC2g tri-boundary equilibrium readout overrides
# -----
# RC2g addresses the RC2e smoke failure mode: class 2 recovered, but class 4
# collapsed and class 5 remained weak. The backbone and no-leakage protocol are
# unchanged. This patch modifies only validation-memory scoring and ☐,
# ↪repair-memory
# readout calibration.
#
# Boundary-safety principle:
# - class 2 must beat class 4 when the true label is 2;
# - class 3 must beat class 4 when the true label is 3;
# - class 4 must beat class 2, class 3 and class 5 when the true label is 4;
# - class 5 must beat class 4 when the true label is 5.
# -----

RC2G_SELECTOR_VERSION = globals().get("RC2G_SELECTOR_VERSION", ☐,
    ↪"RC2g_tri_boundary_class_balanced_v1")
RC2G_BOUNDARY_CLASSES = globals().get("RC2G_BOUNDARY_CLASSES", [2, 3, 4, 5])
RC2G_PROTECTED_CLASSES = globals().get("RC2G_PROTECTED_CLASSES", [2, 4, 5])
RC2G_CLASS4_FLOOR = globals().get("RC2G_CLASS4_FLOOR", 0.45)
RC2G_HARD_CLASS4_FLOOR = globals().get("RC2G_HARD_CLASS4_FLOOR", 0.05)
RC2G_CLASS4_FLOOR_PENALTY = globals().get("RC2G_CLASS4_FLOOR_PENALTY", 6.00)

```

```

RC2G_ZERO_SEEN_CLASS_REJECT_ENABLED = globals().
    ↪get("RC2G_ZERO_SEEN_CLASS_REJECT_ENABLED", True)
RC2G_ZERO_SEEN_CLASS_PENALTY = globals().get("RC2G_ZERO_SEEN_CLASS_PENALTY", 4.
    ↪00)
RC2G_ZERO_CLASS_RESCUE_SCORE_MARGIN = globals().
    ↪get("RC2G_ZERO_CLASS_RESCUE_SCORE_MARGIN", 0.08)
RC2G_BOUNDARY_MARGIN_TARGET = globals().get("RC2G_BOUNDARY_MARGIN_TARGET", 0.20)
RC2G_BOUNDARY_MARGIN_WEIGHT = globals().get("RC2G_BOUNDARY_MARGIN_WEIGHT", 0.55)
RC2G_CLASS4_WEIGHT = globals().get("RC2G_CLASS4_WEIGHT", 3.50)
RC2G_BOUNDARY_CLASS_WEIGHT = globals().get("RC2G_BOUNDARY_CLASS_WEIGHT", 2.75)
RC2G_BOUNDARY_CALIB_STEPS = globals().get("RC2G_BOUNDARY_CALIB_STEPS", 18 if
    ↪RUN_MODE == "smoke" else 40)
RC2G_BOUNDARY_CALIB_LR = globals().get("RC2G_BOUNDARY_CALIB_LR", 1.2e-3)

if globals().get("POLICY_BRANCH", "RC2g") == "RC2g":
    RC2C_METHOD = "paired_rc2g_tri_boundary_equilibrium_policy"
    RC2B_METHOD = RC2C_METHOD
    RC3_METHOD = RC2C_METHOD

def rc2f_class_key(class_id: int) -> str:
    return f"proxy_class{int(class_id)}"

def rc2f_candidate_class(candidate, class_id: int, neutral=np.nan):
    value, _ = rc2f_class_metric(candidate.get("proxy", {}),
    ↪rc2f_class_key(class_id), int(class_id), after_task=candidate.
    ↪get("after_task"), neutral=neutral)
    return value

def rc2f_candidate_zero_seen_class_count(candidate) -> int:
    try:
        return int(candidate.get("proxy", {}).
    ↪get("proxy_zero_seen_class_count", 0) or 0)
    except Exception:
        return 0

def rc2f_seen_pred_count(candidate, class_id: int) -> int:
    try:
        return int(candidate.get("proxy", {}).
    ↪get(f"proxy_pred_count_class{int(class_id)}", 0) or 0)
    except Exception:
        return 0

```

```

def rc2f_boundary_margin_loss(logits, y, target_margin=None):
    """Symmetric boundary margin for the 2/3/4/5 overlap edge.

    This replaces the one-way class2-vs-class4 pressure that caused RC2e to
    recover class 2 while sacrificing class 4. It is applied only during the
    repair-memory calibration pass for guarded readouts.
    """
    if logits is None or y is None:
        return torch.zeros(), device=DEVICE)
    target_margin = RC2G_BOUNDARY_MARGIN_TARGET if target_margin is None else
float(target_margin)
    terms = []
    constraints = [
        (2, 4),      # true 2 should outrank 4
        (3, 4),      # true 3 should outrank 4
        (4, 2),      # true 4 should outrank 2
        (4, 3),      # true 4 should outrank 3
        (4, 5),      # true 4 should outrank 5
        (5, 4),      # true 5 should outrank 4
    ]
    for true_cls, rival_cls in constraints:
        if true_cls >= logits.shape[1] or rival_cls >= logits.shape[1]:
            continue
        mask = (y == int(true_cls))
        if mask.any():
            margin = logits[mask, true_cls] - logits[mask, rival_cls]
            terms.append(F.relu(target_margin - margin).mean())
    if not terms:
        return torch.zeros(), device=logits.device)
    return torch.stack(terms).mean()

@torch.no_grad()
def proxy_metrics_for_context_config(model, class_memory, config,
prototypes=None, readout=None,
max_items_per_class=VALIDATION_MAX_ITEMS_PER_CLASS):
    """RC2g proxy metrics with boundary-class and never-predicted audits.

    Still memory-only: this is used for context/policy validation and never
reads
    final test data before policy selection.
    """
    if not class_memory:
        out = {
            "proxy_acc": np.nan,

```



```

        "proxy_context_top1": np.nan,
        "proxy_context_entropy": np.nan,
        "proxy_context_margin": np.nan,
        "proxy_min_class": np.nan,
        "proxy_min_task": np.nan,
        "proxy_old_task_min": np.nan,
        "proxy_task_acc_std": np.nan,
        "proxy_score": -1e9,
        "proxy_zero_seen_class_count": np.nan,
        "proxy_never_predicted_seen_classes": "",
    }
    for c in range(N_CLASSES):
        out[f"proxy_class{c}"] = np.nan
        out[f"proxy_pred_count_class{c}"] = 0
        out[f"proxy_pred_rate_class{c}"] = np.nan
    return out

if readout is None:
    readout = GlobalLinearReadout(model.global_head).to(DEVICE)
model.eval(); readout.eval()

correct_by_cls = {c: 0 for c in range(N_CLASSES)}
total_by_cls = {c: 0 for c in range(N_CLASSES)}
pred_by_cls = {c: 0 for c in range(N_CLASSES)}
correct_by_task = {t: 0 for t in range(N_TASKS)}
total_by_task = {t: 0 for t in range(N_TASKS)}
correct_total, total = 0, 0
top1_values, ent_values, margin_values = [], [], []

for cls, data in sorted(class_memory.items()):
    X, y, t = data["X"], data["y"], data["task_id"]
    if len(y) == 0:
        continue
    if len(y) > max_items_per_class:
        idx = torch.randperm(len(y))[:max_items_per_class]
        X, y, t = X[idx], y[idx], t[idx]
    loader = make_loader(TensorDataset(X, y, t), batch_size=BATCH_SIZE,
↪shuffle=False)
    for xb, yb, tb in loader:
        xb, yb, tb = xb.to(DEVICE), yb.to(DEVICE), tb.to(DEVICE)
        for src_task_id in torch.unique(tb).detach().cpu().tolist():
            src_task_id = int(src_task_id)
            mask = (tb == src_task_id)
            if not mask.any():
                continue
            _, cache = forward_by_method(
                model, xb[mask], src_task_id, config,

```

```

        prototypes=prototypes, train=False, y=yb[mask],
↪return_cache=True
    )
    logits = readout(cache["zf"], cache["q_context"])
    seen = seen_classes_until(src_task_id)
    pred = mask_logits_to_seen_classes(logits, seen).argmax(dim=-1)
    yy = yb[mask]
    correct = (pred == yy)
    correct_total += int(correct.sum().item())
    total += int(yy.numel())
    correct_by_task[src_task_id] += int(correct.sum().item())
    total_by_task[src_task_id] += int(yy.numel())

    q = cache["q_context"]
    top1_values.append((q.argmax(dim=-1) == src_task_id).float().
↪detach()).cpu())
    ent_values.append(entropy_from_probs(q).detach().cpu())
    margin_values.append(margin_from_probs(q).detach().cpu())

    for c in torch.unique(yy).detach().cpu().tolist():
        c = int(c)
        m = (yy == c)
        total_by_cls[c] += int(m.sum().item())
        correct_by_cls[c] += int((pred[m] == yy[m]).sum().item())
    for pcls in torch.unique(pred).detach().cpu().tolist():
        pcls = int(pcls)
        if 0 <= pcls < N_CLASSES:
            pred_by_cls[pcls] += int((pred == pcls).sum().item())

    per_cls = {c: (correct_by_cls[c] / max(total_by_cls[c], 1) if
↪total_by_cls[c] > 0 else np.nan) for c in range(N_CLASSES)}
    valid_accs = [v for v in per_cls.values() if not np.isnan(v)]
    acc = correct_total / max(total, 1)
    ctx_top1 = float(torch.cat(top1_values).mean()) if top1_values else np.nan
    ctx_entropy = float(torch.cat(ent_values).mean()) if ent_values else np.nan
    ctx_margin = float(torch.cat(margin_values).mean()) if margin_values else
↪np.nan
    min_class = float(np.min(valid_accs)) if valid_accs else np.nan
    per_task = {tid: (correct_by_task[tid] / max(total_by_task[tid], 1) if
↪total_by_task[tid] > 0 else np.nan) for tid in range(N_TASKS)}
    valid_task_accs = [v for v in per_task.values() if not np.isnan(v)]
    min_task = float(np.min(valid_task_accs)) if valid_task_accs else np.nan
    active_tasks = [tid for tid, v in per_task.items() if not np.isnan(v)]
    max_active_task = max(active_tasks) if active_tasks else -1
    old_task_accs = [per_task[tid] for tid in active_tasks if tid <
↪max_active_task]

```

```

old_task_min = float(np.min(old_task_accs)) if old_task_accs else min_task
task_acc_std = float(np.std(valid_task_accs)) if valid_task_accs else np.nan

true_seen_classes = [c for c in range(N_CLASSES) if total_by_cls[c] > 0]
never_predicted_seen = [c for c in true_seen_classes if pred_by_cls[c] == 0]
zero_seen_class_count = len(never_predicted_seen)
pred_rates = {c: (pred_by_cls[c] / max(total, 1) if total > 0 else np.nan)
↳for c in range(N_CLASSES)}

# Score remains compact; selection scoring below applies stronger gates.
c2 = per_cls.get(2, np.nan)
c4 = per_cls.get(4, np.nan)
c5 = per_cls.get(5, np.nan)
protected_terms = [x for x in [c2, c4, c5] if not np.isnan(x)]
protected_mean = float(np.mean(protected_terms)) if protected_terms else 0.0
score = float(
    acc
    + CONTEXT_SELECTION_SCORE_CONTEXT_WEIGHT * (0.0 if np.isnan(ctx_top1)
↳else ctx_top1)
    + CONTEXT_SELECTION_SCORE_MINCLASS_WEIGHT * (0.0 if np.isnan(min_class)
↳else min_class)
    + 0.08 * protected_mean
    + 0.05 * (0.0 if np.isnan(min_task) else min_task)
    - CONTEXT_SELECTION_SCORE_ENTROPY_PENALTY * (0.0 if np.
↳isnan(ctx_entropy) else ctx_entropy)
    - 0.05 * zero_seen_class_count
)

out = {
    "proxy_acc": acc,
    "proxy_context_top1": ctx_top1,
    "proxy_context_entropy": ctx_entropy,
    "proxy_context_margin": ctx_margin,
    "proxy_min_class": min_class,
    "proxy_min_task": min_task,
    "proxy_old_task_min": old_task_min,
    "proxy_task_acc_std": task_acc_std,
    "proxy_score": score,
    "proxy_zero_seen_class_count": zero_seen_class_count,
    "proxy_never_predicted_seen_classes": str(never_predicted_seen),
}
for c in range(N_CLASSES):
    out[f"proxy_class{c}"] = per_cls.get(c, np.nan)
    out[f"proxy_pred_count_class{c}"] = int(pred_by_cls[c])
    out[f"proxy_pred_rate_class{c}"] = pred_rates[c]
# Backward-compatible aliases used by older gates.
out["proxy_class2"] = out.get("proxy_class2", np.nan)

```

```

out["proxy_class4"] = out.get("proxy_class4", np.nan)
out["proxy_class5"] = out.get("proxy_class5", np.nan)
return out

def rc2f_class_weights_for_memory(class_memory, after_task=None):
    support = rc2f_memory_class_support(class_memory)
    weights = torch.ones(N_CLASSES, device=DEVICE)
    positive_counts = [v for v in support.values() if v > 0]
    max_count = max(positive_counts) if positive_counts else 1
    for cls, count in support.items():
        if count > 0:
            weights[int(cls)] = min(RC2G_MAX_CLASS_WEIGHT, math.sqrt(max_count /
↪ max(count, 1)))
    for cls in RC2G_PROTECTED_CLASSES:
        if support.get(cls, 0) > 0 and rc2f_class_applicable(cls, after_task):
            target_weight = RC2G_CLASS4_WEIGHT if int(cls) == 4 else
↪ RC2G_BOUNDARY_CLASS_WEIGHT
            weights[int(cls)] = min(RC2G_MAX_CLASS_WEIGHT,
↪ max(float(weights[int(cls)]), float(target_weight)))
    return weights

# Override the RC2g class weight helper name expected by the previous cell.
rc2f_class_weights_for_memory = rc2f_class_weights_for_memory

def rc2f_readout_class_balanced_calibration(readout, model, class_memory,
↪ config, prototypes=None, after_task=None, steps=None, lr=None):
    """Boundary-safe repair-memory-only calibration for guarded readouts."""
    rows = []
    if not RC2G_CLASS_BALANCED_READOUT_CALIBRATION_ENABLED or not class_memory:
        return rows
    support = rc2f_memory_class_support(class_memory)
    active_boundary = [c for c in RC2G_BOUNDARY_CLASSES if support.get(c, 0) >
↪ 0 and rc2f_class_applicable(c, after_task)]
    if not active_boundary:
        return rows
    params = [p for p in readout.parameters() if p.requires_grad]
    if not params:
        return rows
    steps = int(RC2G_BOUNDARY_CALIB_STEPS if steps is None else steps)
    lr = float(RC2G_BOUNDARY_CALIB_LR if lr is None else lr)
    weights = rc2f_class_weights_for_memory(class_memory, after_task=after_task)
    model.eval()
    for p in model.parameters():
        p.requires_grad_(False)

```

```

readout.train()
opt = torch.optim.Adam(params, lr=lr)
for step in range(steps):
    xb, yb, tb = sample_repair_batch_v10b(class_memory,
↪batch_size=HEAD_REPAIR_BATCH)
    if xb is None:
        continue
    xb, yb, tb = xb.to(DEVICE), yb.to(DEVICE), tb.to(DEVICE)
    zf_list, q_list, y_list = [], [], []
    with torch.no_grad():
        for src_task_id in torch.unique(tb).detach().cpu().tolist():
            src_task_id = int(src_task_id)
            mask = (tb == src_task_id)
            if not mask.any():
                continue
            _, cache = forward_by_method(model, xb[mask], src_task_id,
↪config, prototypes=prototypes, train=False, y=yb[mask], return_cache=True)
            zf_list.append(cache["zf"].detach())
            q_list.append(cache["q_context"].detach())
            y_list.append(yb[mask].detach())
    if not zf_list:
        continue
    zf = torch.cat(zf_list, dim=0)
    q_context = torch.cat(q_list, dim=0)
    y_mix = torch.cat(y_list, dim=0)
    opt.zero_grad(set_to_none=True)
    logits = readout(zf, q_context)
    ce = F.cross_entropy(logits, y_mix, weight=weights)
    boundary = rc2f_boundary_margin_loss(logits, y_mix)
    loss = ce + RC2G_BOUNDARY_MARGIN_WEIGHT * boundary
    loss.backward()
    torch.nn.utils.clip_grad_norm_(params, max_norm=5.0)
    opt.step()
    rows.append({
        "repair_epoch": step,
        "repair_type": "rc2g_tri_boundary_equilibrium_readout_calibration",
        "rc2f_active_boundary_classes": str(active_boundary),
        "rc2g_tri_boundary_loss": float(loss.detach().cpu()),
        "rc2g_tri_boundary_ce": float(ce.detach().cpu()),
        "rc2f_boundary_margin": float(boundary.detach().cpu()),
        "rc2f_class_weights": safe_jsonable(weights.detach().cpu()),
        "rc2f_selector_version": RC2G_SELECTOR_VERSION,
    })
readout.eval()
return rows

```

```

def rc2c_candidate_meets_hard_floors(candidate) -> bool:
    """RC2g class-aware hard floor check, including class-4 tri-boundaryty."""
    p = candidate.get("proxy", {})
    after_task = candidate.get("after_task", None)
    acc = rc2c_proxy_value(p, "proxy_acc")
    c2 = rc2c_proxy_value(p, "proxy_class2")
    c4 = rc2c_proxy_value(p, "proxy_class4")
    c5 = rc2c_proxy_value(p, "proxy_class5")
    min_task = rc2c_proxy_value(p, "proxy_min_task", rc2c_proxy_value(p,
↪ "proxy_min_class"))
    min_class = rc2c_proxy_value(p, "proxy_min_class")
    zero_count = int(p.get("proxy_zero_seen_class_count", 0) or 0)
    ok_c2 = (not rc2f_class_applicable(2, after_task)) or (not rc2f_finite(c2))
↪ or c2 >= RC2C_HARD_CLASS2_FLOOR
    ok_c4 = (not rc2f_class_applicable(4, after_task)) or (not rc2f_finite(c4))
↪ or c4 >= RC2G_HARD_CLASS4_FLOOR
    ok_c5 = (not rc2f_class_applicable(5, after_task)) or (not rc2f_finite(c5))
↪ or c5 >= RC2D_HARD_CLASS5_FLOOR
    ok_min_task = (not rc2f_finite(min_task)) or min_task >=
↪ RC2C_HARD_MIN_TASK_FLOOR
    ok_min_class = (not rc2f_finite(min_class)) or min_class >=
↪ RC2C_HARD_MIN_CLASS_FLOOR
    ok_acc = (not rc2f_finite(acc)) or acc >= max(0.50, RC2D_PROXY_ACC_FLOOR -
↪ 0.05)
    ok_zero = (not RC2G_ZERO_SEEN_CLASS_REJECT_ENABLED) or zero_count == 0
    return bool(ok_c2 and ok_c4 and ok_c5 and ok_min_task and ok_min_class and
↪ ok_acc and ok_zero)

def rc2c_risk_components(proxy, baseline_proxy, variant: str, after_task=None):
    """RC2g tri-boundary equilibrium risk components, still
↪ validation-memory-only."""
    acc = rc2f_metric(proxy, "proxy_acc", neutral=0.0)
    c2, c2_available = rc2f_class_metric(proxy, "proxy_class2", 2,
↪ after_task=after_task, neutral=0.0)
    c4, c4_available = rc2f_class_metric(proxy, "proxy_class4", 4,
↪ after_task=after_task, neutral=0.0)
    c5, c5_available = rc2f_class_metric(proxy, "proxy_class5", 5,
↪ after_task=after_task, neutral=0.0)
    min_class = rc2f_metric(proxy, "proxy_min_class", neutral=acc)
    min_task = rc2f_metric(proxy, "proxy_min_task", neutral=min_class)
    old_task_min = rc2f_metric(proxy, "proxy_old_task_min", neutral=min_task)
    task_std = rc2f_metric(proxy, "proxy_task_acc_std", neutral=0.0)
    family = rc2b_policy_family(variant)
    zero_count = int(proxy.get("proxy_zero_seen_class_count", 0) or 0)

```

```

class2_penalty = RC2C_CLASS2_FLOOR_PENALTY *␣
↪rc2f_floor_deficit_metric(proxy, "proxy_class2", RC2C_CLASS2_FLOOR,␣
↪class_id=2, after_task=after_task)
class4_penalty = RC2G_CLASS4_FLOOR_PENALTY *␣
↪rc2f_floor_deficit_metric(proxy, "proxy_class4", RC2G_CLASS4_FLOOR,␣
↪class_id=4, after_task=after_task)
class5_penalty = RC2D_CLASS5_FLOOR_PENALTY *␣
↪rc2f_floor_deficit_metric(proxy, "proxy_class5", RC2D_CLASS5_FLOOR,␣
↪class_id=5, after_task=after_task)
min_class_penalty = RC2C_MIN_CLASS_FLOOR_PENALTY *␣
↪rc2f_floor_deficit_metric(proxy, "proxy_min_class", RC2C_MIN_CLASS_FLOOR,␣
↪after_task=after_task)
min_task_penalty = RC2C_MIN_TASK_FLOOR_PENALTY *␣
↪rc2f_floor_deficit_metric(proxy, "proxy_min_task", RC2C_MIN_TASK_FLOOR,␣
↪after_task=after_task)
old_task_penalty = RC2C_OLD_TASK_FLOOR_PENALTY *␣
↪rc2f_floor_deficit_metric(proxy, "proxy_old_task_min", RC2C_OLD_TASK_FLOOR,␣
↪after_task=after_task)
acc_penalty = RC2D_PROXY_ACC_FLOOR_PENALTY *␣
↪rc2f_floor_deficit_metric(proxy, "proxy_acc", RC2D_PROXY_ACC_FLOOR,␣
↪after_task=after_task)
zero_penalty = RC2G_ZERO_SEEN_CLASS_PENALTY * max(0, zero_count)
floor_penalty = class2_penalty + class4_penalty + class5_penalty +␣
↪min_class_penalty + min_task_penalty + old_task_penalty + acc_penalty +␣
↪zero_penalty

dispersion_penalty = RC2D_TASK_STD_WEIGHT * max(0.0, task_std)
no_repair_soft_penalty = 0.0
if after_task is not None and int(after_task) >=␣
↪RC2C_NO_REPAIR_VETO_AFTER_TASK and family == "proto_no_repair":
    no_repair_soft_penalty = RC2C_NO_REPAIR_SOFT_PENALTY
    guarded_bonus = RC2D_CONTEXT_GUARD_BONUS if␣
↪rc2c_is_guarded_context_candidate(variant) else 0.0
    true_guard_bonus = RC2D_TRUE_GUARD_BONUS if family ==␣
↪"proto_true_global_guard" else 0.0
    complexity_penalty = RC2B_COMPLEXITY_PENALTY *␣
↪rc2b_policy_complexity_rank(variant)
    class5_bonus = RC2B_VALIDATION_SCORE_CLASS5_WEIGHT * c5 if c5_available␣
↪else 0.0
    hydro_penalty = rc3_hydro_selection_penalty(proxy)

return {
    "proxy_acc_floor_penalty": acc_penalty,
    "class2_floor_penalty": class2_penalty,
    "class4_floor_penalty": class4_penalty,
    "class5_floor_penalty": class5_penalty,

```

```

        "min_class_floor_penalty": min_class_penalty,
        "min_task_floor_penalty": min_task_penalty,
        "old_task_floor_penalty": old_task_penalty,
        "zero_seen_class_penalty": zero_penalty,
        "floor_penalty": floor_penalty,
        "task_dispersion_penalty": dispersion_penalty,
        "no_repair_soft_penalty": no_repair_soft_penalty,
        "complexity_penalty": complexity_penalty,
        "class5_bonus": class5_bonus,
        "guarded_context_bonus": guarded_bonus,
        "true_guard_bonus": true_guard_bonus,
        "hydro_selection_penalty": hydro_penalty,
        "total_risk_penalty": floor_penalty + dispersion_penalty +
↪no_repair_soft_penalty + complexity_penalty + hydro_penalty,
        "rc2f_class2_applicable": rc2f_class_applicable(2, after_task),
        "rc2f_class4_applicable": rc2f_class_applicable(4, after_task),
        "rc2f_class5_applicable": rc2f_class_applicable(5, after_task),
        "rc2f_class2_available_in_proxy": bool(c2_available),
        "rc2f_class4_available_in_proxy": bool(c4_available),
        "rc2f_class5_available_in_proxy": bool(c5_available),
        "rc2f_zero_seen_class_count": zero_count,
    }

def rc2d_gate_aligned_score(proxy, baseline_proxy, variant: str,
↪after_task=None):
    """RC2g tri-boundary equilibrium gate-aligned score."""
    base_score = rc2f_nan_safe_validation_base(proxy, baseline_proxy)
    acc = rc2f_metric(proxy, "proxy_acc", neutral=0.0)
    min_class = rc2f_metric(proxy, "proxy_min_class", neutral=acc)
    min_task = rc2f_metric(proxy, "proxy_min_task", neutral=min_class)
    old_task_min = rc2f_metric(proxy, "proxy_old_task_min", neutral=min_task)
    task_std = rc2f_metric(proxy, "proxy_task_acc_std", neutral=0.0)
    comp = rc2c_risk_components(proxy, baseline_proxy, variant,
↪after_task=after_task)

    protected = []
    for cls in RC2G_PROTECTED_CLASSES:
        if rc2f_class_applicable(cls, after_task):
            v, _ = rc2f_class_metric(proxy, rc2f_class_key(cls), cls,
↪after_task=after_task, neutral=0.0)
            protected.append(v)
    protected_score = float(np.mean(protected)) if protected else 0.0
    c4, _ = rc2f_class_metric(proxy, "proxy_class4", 4, after_task=after_task,
↪neutral=0.0)
    zero_count = int(proxy.get("proxy_zero_seen_class_count", 0) or 0)

```



```

balance_score = (
    acc
    + RC2D_CLASS_BALANCE_WEIGHT * protected_score
    + 0.15 * (c4 if rc2f_class_applicable(4, after_task) else 0.0)
    + RC2D_MIN_TASK_WEIGHT * min_task
    + RC2D_OLD_TASK_WEIGHT * old_task_min
    - RC2D_TASK_STD_WEIGHT * max(0.0, task_std)
    - 0.15 * zero_count
)
score = float(
    0.45 * base_score
    + 0.55 * balance_score
    + comp["class5_bonus"]
    + comp["guarded_context_bonus"]
    + comp["true_guard_bonus"]
    - comp["total_risk_penalty"]
)
return score if rc2f_finite(score) else -1e9

def rc2b_validation_score(proxy, baseline_proxy, variant: str, after_task=None):
    """Override: RC2g tri-boundary equilibrium score, with RC2b strict fallback.
    ↪ """
    if not RC2C_RISK_AWARE_SELECTOR_ENABLED or globals().get("POLICY_BRANCH")_
    ↪ == "RC2b":
        base_score = rc2f_nan_safe_validation_base(proxy, baseline_proxy)
        penalty = RC2B_COMPLEXITY_PENALTY * rc2b_policy_complexity_rank(variant)
        floor_penalty = 0.0
        for cls, floor in [(2, RC2B_CLASS2_FLOOR), (4, RC2G_CLASS4_FLOOR), (5, _
    ↪ RC2B_CLASS5_FLOOR)]:
            val = rc2c_proxy_value(proxy, rc2f_class_key(cls), np.nan)
            if rc2f_class_applicable(cls, after_task) and rc2f_finite(val) and _
    ↪ val < floor:
                floor_penalty += RC2B_VALIDATION_SCORE_FLOOR_PENALTY * (floor - _
    ↪ val)
            minc = rc2c_proxy_value(proxy, "proxy_min_class", np.nan)
            if rc2f_finite(minc) and minc < RC2B_MIN_CLASS_FLOOR:
                floor_penalty += RC2B_VALIDATION_SCORE_FLOOR_PENALTY * _
    ↪ (RC2B_MIN_CLASS_FLOOR - minc)
            hydro_penalty = rc3_hydro_selection_penalty(proxy)
            score = float(base_score - penalty - floor_penalty - hydro_penalty)
            return score if rc2f_finite(score) else -1e9
    return rc2d_gate_aligned_score(proxy, baseline_proxy, variant, _
    ↪ after_task=after_task)

```

```

def rc2f_emergency_candidate_score(candidate, after_task=None):
    p = candidate.get("proxy", {})
    acc = rc2f_metric(p, "proxy_acc", neutral=0.0)
    min_class = rc2f_metric(p, "proxy_min_class", neutral=acc)
    min_task = rc2f_metric(p, "proxy_min_task", neutral=min_class)
    protected = []
    for cls in RC2G_PROTECTED_CLASSES:
        if rc2f_class_applicable(cls, after_task):
            v, _ = rc2f_class_metric(p, rc2f_class_key(cls), cls,
↪after_task=after_task, neutral=0.0)
            protected.append(v)
    protected_mean = float(np.mean(protected)) if protected else 0.0
    zero_count = int(p.get("proxy_zero_seen_class_count", 0) or 0)
    score = 0.50 * acc + 0.20 * min_task + 0.15 * min_class + 0.15 *
↪protected_mean - 0.20 * zero_count
    score -= RC2B_COMPLEXITY_PENALTY * rc2b_policy_complexity_rank(candidate.
↪get("variant", ""))
    return float(score) if rc2f_finite(score) else -1e9

def rc2c_select_candidate(candidates, after_task):
    """Override: RC2g tri-boundary equilibrium selector with zero-class
↪rejection."""
    emergency_scoring_used = False
    scored = [c for c in candidates if rc2f_finite(c.get("selection_score",
↪-1e9))]
    if not scored and RC2G_EMERGENCY_SCORE_IF_ALL_NAN:
        for c in candidates:
            c["selection_score"] = rc2f_emergency_candidate_score(c,
↪after_task=after_task)
            c["rc2f_emergency_score_used"] = True
        scored = [c for c in candidates if rc2f_finite(c.get("selection_score",
↪-1e9))]
        emergency_scoring_used = True
    if not scored:
        selected = max(candidates, key=lambda c: (rc2d_candidate_proxy_acc(c),
↪rc2b_policy_complexity_rank(c.get("variant", ""))))
    return selected, {
        "raw_best_variant": selected["variant"],
        "selection_reason": "fallback_no_finite_scores_proxy_acc_tiebreak",
        "no_repair_veto_applied": False,
        "guarded_hysteresis_applied": False,
        "risk_floor_fallback_applied": False,
        "true_guard_rescue_applied": False,
        "class2_rescue_applied": False,
        "class4_rescue_applied": False,
    }

```

```

        "class5_rescue_applied": False,
        "zero_seen_class_reject_applied": False,
        "rc2f_selector_version": RC2G_SELECTOR_VERSION,
        "rc2f_emergency_scoring_used": True,
    }

    sorted_by_score = sorted(
        scored,
        key=lambda c: (
            float(c.get("selection_score", -1e9)),
            -rc2f_candidate_zero_seen_class_count(c),
            rc2d_candidate_proxy_acc(c),
            -float(c["complexity_rank"]),
        ),
        reverse=True,
    )
    raw_best = sorted_by_score[0]
    selected = raw_best
    reason = "direct_rc2g_tri_boundary_score_win" if not emergency_scoring_used
    ↪else "rc2g_emergency_score_win"
    no_repair_veto = False
    hysteresis = False
    floor_fallback = False
    true_guard_rescue = False
    class2_rescue = False
    class4_rescue = False
    class5_rescue = False
    zero_reject = False

    guarded_candidates = [c for c in scored if
    ↪rc2c_is_guarded_context_candidate(c["variant"])]
    best_guarded = max(guarded_candidates, key=lambda c: float(c.
    ↪get("selection_score", -1e9))) if guarded_candidates else None
    true_guard_candidates = [c for c in scored if
    ↪rc2b_policy_family(c["variant"]) == "proto_true_global_guard"]
    best_true_guard = max(true_guard_candidates, key=lambda c: float(c.
    ↪get("selection_score", -1e9))) if true_guard_candidates else None

    if (
        best_guarded is not None
        and int(after_task) >= RC2C_NO_REPAIR_VETO_AFTER_TASK
        and rc2b_policy_family(raw_best["variant"]) == "proto_no_repair"
    ):
        margin = float(raw_best.get("selection_score", -1e9)) -
    ↪float(best_guarded.get("selection_score", -1e9))
        if margin < RC2C_NO_REPAIR_REQUIRED_MARGIN:

```

```

        selected = best_guarded
        reason = f"no_repair_veto_guarded_fallback_margin_{margin:+.4f}"
        no_repair_veto = True

        # Hard zero-class rejection: if a candidate never predicts a seen class,
        # use the best non-zero alternative unless no such alternative exists.
        if RC2G_ZERO_SEEN_CLASS_REJECT_ENABLED and
→rc2f_candidate_zero_seen_class_count(selected) > 0:
            safe_candidates = [c for c in scored if
→rc2f_candidate_zero_seen_class_count(c) == 0]
            if safe_candidates:
                best_safe = max(safe_candidates, key=lambda c: float(c.
→get("selection_score", -1e9)))
                selected = best_safe
                reason = "zero_seen_class_reject_rescue"
                zero_reject = True

        if best_true_guard is not None and rc2d_not_too_far(best_true_guard,
→selected, RC2D_TRUE_GUARD_RESCUE_SCORE_MARGIN):
            tg_c2 = rc2f_candidate_class(best_true_guard, 2)
            se_c2 = rc2f_candidate_class(selected, 2)
            tg_c4 = rc2f_candidate_class(best_true_guard, 4)
            se_c4 = rc2f_candidate_class(selected, 4)
            tg_acc = rc2d_candidate_proxy_acc(best_true_guard)
            se_acc = rc2d_candidate_proxy_acc(selected)
            c2_gain = (rc2f_class_applicable(2, after_task) and rc2f_finite(tg_c2)
→and rc2f_finite(se_c2) and tg_c2 >= se_c2 + RC2D_TRUE_GUARD_CLASS2_MARGIN)
            c4_gain = (rc2f_class_applicable(4, after_task) and rc2f_finite(tg_c4)
→and rc2f_finite(se_c4) and tg_c4 >= se_c4 + 0.05)
            acc_gain = (rc2f_finite(tg_acc) and rc2f_finite(se_acc) and tg_acc >=
→se_acc + RC2D_TRUE_GUARD_ACC_MARGIN)
            if (c2_gain or c4_gain or acc_gain) and
→rc2c_candidate_meets_hard_floors(best_true_guard):
                selected = best_true_guard
                reason =
→"true_global_guard_rescue_for_boundary_or_accuracy_alignment"
                true_guard_rescue = True

        for cls, hard_floor, rescue_margin, flag_name in [
            (2, RC2C_HARD_CLASS2_FLOOR, RC2D_CLASS2_RESCUE_MARGIN, "class2"),
            (4, RC2G_HARD_CLASS4_FLOOR, 0.05, "class4"),
            (5, RC2D_HARD_CLASS5_FLOOR, RC2D_CLASS5_RESCUE_MARGIN, "class5"),
        ]:
            if not rc2f_class_applicable(cls, after_task):
                continue
            selected_val = rc2f_candidate_class(selected, cls)

```

```

        if rc2f_finite(selected_val) and selected_val < hard_floor:
            cls_candidates = [c for c in scored if
↪rc2f_finite(rc2f_candidate_class(c, cls))]
            if cls_candidates:
                best_cls = max(cls_candidates, key=lambda c:
↪(rc2f_candidate_class(c, cls), float(c.get("selection_score", -1e9))))
                if rc2f_candidate_class(best_cls, cls) >= selected_val +
↪rescue_margin and rc2d_not_too_far(best_cls, selected,
↪RC2D_TRUE_GUARD_RESCUE_SCORE_MARGIN + 0.03):
                    selected = best_cls
                    reason = f"{flag_name}_hard_floor_rescue"
                    if cls == 2:
                        class2_rescue = True
                    elif cls == 4:
                        class4_rescue = True
                    elif cls == 5:
                        class5_rescue = True

            if RC2C_FLOOR_RISK_FALLBACK_ENABLED and not
↪rc2c_candidate_meets_hard_floors(selected):
                floor_safe = [c for c in scored if rc2c_candidate_meets_hard_floors(c)]
                if floor_safe:
                    best_floor_safe = max(floor_safe, key=lambda c: float(c.
↪get("selection_score", -1e9)))
                    selected = best_floor_safe
                    reason = "hard_floor_risk_fallback"
                    floor_fallback = True

            if (
                best_guarded is not None
                and not rc2c_is_guarded_context_candidate(selected["variant"])
                and rc2d_not_too_far(best_guarded, selected,
↪RC2D_GUARDED_HYSTERESIS_MARGIN)
                and rc2c_candidate_meets_hard_floors(best_guarded)
            ):
                bg_c2 = rc2f_candidate_class(best_guarded, 2)
                se_c2 = rc2f_candidate_class(selected, 2)
                bg_c4 = rc2f_candidate_class(best_guarded, 4)
                se_c4 = rc2f_candidate_class(selected, 4)
                not_boundary_worse = True
                if rc2f_class_applicable(2, after_task) and rc2f_finite(bg_c2) and
↪rc2f_finite(se_c2):
                    not_boundary_worse = not_boundary_worse and bg_c2 >= se_c2 - 0.03
                    if rc2f_class_applicable(4, after_task) and rc2f_finite(bg_c4) and
↪rc2f_finite(se_c4):
                        not_boundary_worse = not_boundary_worse and bg_c4 >= se_c4 - 0.03

```

```

        if not_boundary_worse:
            selected = best_guarded
            reason = "tri_boundary_guarded_context_hysteresis"
            hysteresis = True

    return selected, {
        "raw_best_variant": raw_best["variant"],
        "raw_best_policy_family": raw_best["policy_family"],
        "raw_best_proxy_score": raw_best.get("selection_score", np.nan),
        "best_guarded_variant": best_guarded["variant"] if best_guarded is not_
↪None else "",
        "best_guarded_proxy_score": best_guarded.get("selection_score", np.nan)
↪if best_guarded is not None else np.nan,
        "best_true_guard_variant": best_true_guard["variant"] if
↪best_true_guard is not None else "",
        "best_true_guard_proxy_score": best_true_guard.get("selection_score",
↪np.nan) if best_true_guard is not None else np.nan,
        "selection_reason": reason,
        "no_repair_veto_applied": no_repair_veto,
        "guarded_hysteresis_applied": hysteresis,
        "risk_floor_fallback_applied": floor_fallback,
        "true_guard_rescue_applied": true_guard_rescue,
        "class2_rescue_applied": class2_rescue,
        "class4_rescue_applied": class4_rescue,
        "class5_rescue_applied": class5_rescue,
        "zero_seen_class_reject_applied": zero_reject,
        "rc2f_selector_version": RC2G_SELECTOR_VERSION,
        "rc2f_emergency_scoring_used": emergency_scoring_used,
        "rc2f_class2_applicable": rc2f_class_applicable(2, after_task),
        "rc2f_class4_applicable": rc2f_class_applicable(4, after_task),
        "rc2f_class5_applicable": rc2f_class_applicable(5, after_task),
    }

print("RC2g tri-boundary equilibrium readout overrides loaded:",
↪RC2G_SELECTOR_VERSION)
print("RC2G_ZERO_SEEN_CLASS_REJECT_ENABLED:",
↪RC2G_ZERO_SEEN_CLASS_REJECT_ENABLED)
print("RC2G_PROTECTED_CLASSES:", RC2G_PROTECTED_CLASSES)

```

```

RC2g tri-boundary equilibrium readout overrides loaded:
RC2g_tri_boundary_class_balanced_v1
RC2G_ZERO_SEEN_CLASS_REJECT_ENABLED: True
RC2G_PROTECTED_CLASSES: [2, 4, 5]

```

```

[67]: # -----
# v1.1-RC2g tri-boundary equilibrium overrides
# -----

```

```

# RC2G is a small patch over RC2F. It does not change the backbone, dataset,
# memory split, Hydro mode, or final-test no-leakage discipline.
#
# RC2F fixed class 4 but weakened class 2 again. RC2G therefore optimizes
# the equilibrium among classes 2, 4 and 5 rather than protecting one boundary
# class in isolation.
# -----

RC2G_SELECTOR_VERSION = "RC2g_tri_boundary_equilibrium_v1"
RC2G_TRI_CLASSES = [2, 4, 5]
RC2G_BRIDGE_CLASSES = [3]
RC2G_PROTECTED_CLASSES = [2, 4, 5]

# Smoke-friendly soft floors; robust/paper modes keep stricter pressure.
RC2G_CLASS2_FLOOR = 0.55 if RUN_MODE == "smoke" else 0.80
RC2G_CLASS4_FLOOR = 0.45 if RUN_MODE == "smoke" else 0.75
RC2G_CLASS5_FLOOR = 0.60 if RUN_MODE == "smoke" else 0.80
RC2G_TRI_MIN_FLOOR = 0.45 if RUN_MODE == "smoke" else 0.75

RC2G_CLASS2_FLOOR_PENALTY = 7.00
RC2G_CLASS4_FLOOR_PENALTY = 4.00
RC2G_CLASS5_FLOOR_PENALTY = 6.00
RC2G_TRI_MIN_FLOOR_PENALTY = 7.00
RC2G_TRI_BALANCE_PENALTY = 1.50
RC2G_TRI_MIN_SCORE_WEIGHT = 0.55
RC2G_TRI_MEAN_SCORE_WEIGHT = 0.25
RC2G_TRI_BALANCE_SCORE_WEIGHT = 0.20

# Class-4 is protected, but RC2G also prevents the selector/readout from solving
# class 4 by overpredicting it. This is validation-memory only.
RC2G_PRED4_RATE_TARGET = 0.32 if RUN_MODE == "smoke" else 0.24
RC2G_PRED4_BIAS_PENALTY = 1.25

# Calibration: class 2 and class 5 receive slightly more pressure than class 4,
# because RC2F already showed class 4 can be recovered strongly.
RC2G_CLASS2_WEIGHT = 3.40
RC2G_CLASS4_WEIGHT = 2.40
RC2G_CLASS5_WEIGHT = 3.20
RC2G_BRIDGE_CLASS3_WEIGHT = 1.70
RC2G_EQUILIBRIUM_MARGIN_TARGET = 0.18
RC2G_EQUILIBRIUM_MARGIN_WEIGHT = 0.45
RC2G_PRED4_CALIB_PENALTY = 0.25
RC2G_EQUILIBRIUM_CALIB_STEPS = 28 if RUN_MODE == "smoke" else 55
RC2G_EQUILIBRIUM_CALIB_LR = 1.1e-3

if globals().get("POLICY_BRANCH", "RC2g") == "RC2g":
    RC2C_METHOD = "paired_rc2g_tri_boundary_equilibrium_policy"

```

```

RC2B_METHOD = RC2C_METHOD
RC3_METHOD = RC2C_METHOD

def rc2g_class_value(proxy, cls: int, after_task=None, neutral=0.0):
    v, available = rc2f_class_metric(proxy, rc2f_class_key(cls), cls,
    ↪after_task=after_task, neutral=neutral)
    return float(v), bool(available)

def rc2g_tri_values(proxy, after_task=None, neutral=0.0):
    vals, available = [], []
    for cls in RC2G_TRI_CLASSES:
        if rc2f_class_applicable(cls, after_task):
            v, ok = rc2g_class_value(proxy, cls, after_task=after_task,
    ↪neutral=neutral)
            vals.append(float(v))
            available.append(bool(ok))
    return vals, available

def rc2g_tri_stats(proxy, after_task=None):
    vals, avail = rc2g_tri_values(proxy, after_task=after_task, neutral=0.0)
    if not vals:
        return {
            "tri_min": 0.0,
            "tri_mean": 0.0,
            "tri_gap": 0.0,
            "tri_available_count": 0,
            "tri_values": [],
        }
    arr = np.asarray(vals, dtype=float)
    return {
        "tri_min": float(np.min(arr)),
        "tri_mean": float(np.mean(arr)),
        "tri_gap": float(np.max(arr) - np.min(arr)),
        "tri_available_count": int(sum(avail)),
        "tri_values": vals,
    }

def rc2g_pred4_rate(proxy):
    return rc2f_metric(proxy, "proxy_pred_rate_class4", neutral=0.0)

def rc2g_pred4_bias_penalty(proxy, after_task=None):
    if not rc2f_class_applicable(4, after_task):

```



```

        return 0.0
    pred4 = rc2g_pred4_rate(proxy)
    return RC2G_PRED4_BIAS_PENALTY * max(0.0, pred4 - RC2G_PRED4_RATE_TARGET)

def rc2f_boundary_margin_loss(logits, y, target_margin=None):
    """RC2G equilibrium margin for the 2/3/4/5 boundary.

    This supersedes the RC2F boundary loss. It still protects class 4, but also
    explicitly protects class 2 against classes 1/3/4 and class 5 against class_
    ↪4.
    """
    if logits is None or y is None:
        return torch.zeros((), device=DEVICE)
    target_margin = RC2G_EQUILIBRIUM_MARGIN_TARGET if target_margin is None ↪
    ↪else float(target_margin)
    constraints = [
        (2, 1), (2, 3), (2, 4),
        (3, 2), (3, 4),
        (4, 2), (4, 3), (4, 5),
        (5, 4),
    ]
    terms = []
    for true_cls, rival_cls in constraints:
        if true_cls >= logits.shape[1] or rival_cls >= logits.shape[1]:
            continue
        mask = (y == int(true_cls))
        if mask.any():
            margin = logits[mask, true_cls] - logits[mask, rival_cls]
            terms.append(F.relu(target_margin - margin).mean())
    if not terms:
        return torch.zeros((), device=logits.device)
    return torch.stack(terms).mean()

def rc2f_class_weights_for_memory(class_memory, after_task=None):
    """RC2G class weights: tri-boundary equilibrium, not class-4 dominance."""
    support = rc2f_memory_class_support(class_memory)
    weights = torch.ones(N_CLASSES, device=DEVICE)
    positive_counts = [v for v in support.values() if v > 0]
    max_count = max(positive_counts) if positive_counts else 1
    for cls, count in support.items():
        if count > 0:
            weights[int(cls)] = min(RC2G_MAX_CLASS_WEIGHT if ↪
    ↪"RC2G_MAX_CLASS_WEIGHT" in globals() else RC2G_CLASS2_WEIGHT,
                                   math.sqrt(max_count / max(count, 1)))
    if support.get(2, 0) > 0 and rc2f_class_applicable(2, after_task):

```

```

        weights[2] = max(float(weights[2]), RC2G_CLASS2_WEIGHT)
    if support.get(3, 0) > 0 and rc2f_class_applicable(3, after_task):
        weights[3] = max(float(weights[3]), RC2G_BRIDGE_CLASS3_WEIGHT)
    if support.get(4, 0) > 0 and rc2f_class_applicable(4, after_task):
        weights[4] = max(float(weights[4]), RC2G_CLASS4_WEIGHT)
    if support.get(5, 0) > 0 and rc2f_class_applicable(5, after_task):
        weights[5] = max(float(weights[5]), RC2G_CLASS5_WEIGHT)
    max_w = max(RC2G_CLASS2_WEIGHT, RC2G_CLASS4_WEIGHT, RC2G_CLASS5_WEIGHT, 4.0)
    return weights.clamp(1.0, max_w)

def rc2f_readout_class_balanced_calibration(readout, model, class_memory,
    ↪config, prototypes=None, after_task=None, steps=None, lr=None):
    """RC2G repair-memory-only tri-boundary equilibrium calibration."""
    rows = []
    if not RC2G_CLASS_BALANCED_READOUT_CALIBRATION_ENABLED or not class_memory:
        return rows
    support = rc2f_memory_class_support(class_memory)
    active_tri = [c for c in RC2G_TRI_CLASSES if support.get(c, 0) > 0 and
    ↪rc2f_class_applicable(c, after_task)]
    if not active_tri:
        return rows
    params = [p for p in readout.parameters() if p.requires_grad]
    if not params:
        return rows
    steps = int(RC2G_EQUILIBRIUM_CALIB_STEPS if steps is None else steps)
    lr = float(RC2G_EQUILIBRIUM_CALIB_LR if lr is None else lr)
    weights = rc2f_class_weights_for_memory(class_memory, after_task=after_task)
    model.eval()
    for p in model.parameters():
        p.requires_grad_(False)
    readout.train()
    opt = torch.optim.Adam(params, lr=lr)
    for step in range(steps):
        xb, yb, tb = sample_repair_batch_v10b(class_memory,
    ↪batch_size=HEAD_REPAIR_BATCH)
        if xb is None:
            continue
        xb, yb, tb = xb.to(DEVICE), yb.to(DEVICE), tb.to(DEVICE)
        zf_list, q_list, y_list = [], [], []
        with torch.no_grad():
            for src_task_id in torch.unique(tb).detach().cpu().tolist():
                src_task_id = int(src_task_id)
                mask = (tb == src_task_id)
                if not mask.any():
                    continue

```

```

        _, cache = forward_by_method(model, xb[mask], src_task_id,
↪config, prototypes=prototypes, train=False, y=yb[mask], return_cache=True)
        zf_list.append(cache["zf"].detach())
        q_list.append(cache["q_context"].detach())
        y_list.append(yb[mask].detach())
    if not zf_list:
        continue
    zf = torch.cat(zf_list, dim=0)
    q_context = torch.cat(q_list, dim=0)
    y_mix = torch.cat(y_list, dim=0)
    opt.zero_grad(set_to_none=True)
    logits = readout(zf, q_context)
    ce = F.cross_entropy(logits, y_mix, weight=weights)
    margin = rc2f_boundary_margin_loss(logits, y_mix)
    probs = F.softmax(logits, dim=-1)
    true4_rate = (y_mix == 4).float().mean() if y_mix.numel() else torch.
↪tensor(0.0, device=logits.device)
    pred4_prob = probs[:, 4].mean() if logits.shape[1] > 4 else torch.
↪tensor(0.0, device=logits.device)
    pred4_bias = F.relu(pred4_prob - (true4_rate + 0.10)).pow(2)
    loss = ce + RC2G_EQUILIBRIUM_MARGIN_WEIGHT * margin +
↪RC2G_PRED4_CALIB_PENALTY * pred4_bias
    loss.backward()
    torch.nn.utils.clip_grad_norm_(params, max_norm=5.0)
    opt.step()
    rows.append({
        "repair_epoch": step,
        "repair_type": "rc2g_tri_boundary_equilibrium_readout_calibration",
        "rc2g_active_tri_classes": str(active_tri),
        "rc2g_equilibrium_loss": float(loss.detach().cpu()),
        "rc2g_equilibrium_ce": float(ce.detach().cpu()),
        "rc2g_equilibrium_margin": float(margin.detach().cpu()),
        "rc2g_pred4_bias_loss": float(pred4_bias.detach().cpu()),
        "rc2g_class_weights": safe_jsonable(weights.detach().cpu()),
        "rc2g_selector_version": RC2G_SELECTOR_VERSION,
    })
    readout.eval()
    return rows

def rc2c_candidate_meets_hard_floors(candidate) -> bool:
    """RC2G hard floors: allow smoke diagnostics but reject zero-class/
↪pathological policies."""
    p = candidate.get("proxy", {})
    after_task = candidate.get("after_task", None)
    acc = rc2c_proxy_value(p, "proxy_acc")
    c2 = rc2c_proxy_value(p, "proxy_class2")

```

```

c4 = rc2c_proxy_value(p, "proxy_class4")
c5 = rc2c_proxy_value(p, "proxy_class5")
min_task = rc2c_proxy_value(p, "proxy_min_task", rc2c_proxy_value(p,
↪ "proxy_min_class"))
zero_count = int(p.get("proxy_zero_seen_class_count", 0) or 0)
tri = rc2g_tri_stats(p, after_task=after_task)
ok_acc = (not rc2f_finite(acc)) or acc >= max(0.50, RC2D_PROXY_ACC_FLOOR -
↪ 0.08)
ok_c2 = (not rc2f_class_applicable(2, after_task)) or (not rc2f_finite(c2))
↪ or c2 >= max(0.20, RC2G_CLASS2_FLOOR - 0.30)
ok_c4 = (not rc2f_class_applicable(4, after_task)) or (not rc2f_finite(c4))
↪ or c4 >= max(0.10, RC2G_CLASS4_FLOOR - 0.30)
ok_c5 = (not rc2f_class_applicable(5, after_task)) or (not rc2f_finite(c5))
↪ or c5 >= max(0.20, RC2G_CLASS5_FLOOR - 0.35)
ok_min_task = (not rc2f_finite(min_task)) or min_task >= 0.25
ok_tri = tri["tri_min"] >= 0.20 if tri["tri_values"] else True
ok_zero = (not RC2G_ZERO_SEEN_CLASS_REJECT_ENABLED) or zero_count == 0
return bool(ok_acc and ok_c2 and ok_c4 and ok_c5 and ok_min_task and ok_tri
↪ and ok_zero)

def rc2c_risk_components(proxy, baseline_proxy, variant: str, after_task=None):
    """RC2G tri-boundary risk components, validation-memory-only."""
    acc = rc2f_metric(proxy, "proxy_acc", neutral=0.0)
    min_class = rc2f_metric(proxy, "proxy_min_class", neutral=acc)
    min_task = rc2f_metric(proxy, "proxy_min_task", neutral=min_class)
    old_task_min = rc2f_metric(proxy, "proxy_old_task_min", neutral=min_task)
    task_std = rc2f_metric(proxy, "proxy_task_acc_std", neutral=0.0)
    family = rc2b_policy_family(variant)
    zero_count = int(proxy.get("proxy_zero_seen_class_count", 0) or 0)
    tri = rc2g_tri_stats(proxy, after_task=after_task)

    c2_pen = RC2G_CLASS2_FLOOR_PENALTY * rc2f_floor_deficit_metric(proxy,
↪ "proxy_class2", RC2G_CLASS2_FLOOR, class_id=2, after_task=after_task)
    c4_pen = RC2G_CLASS4_FLOOR_PENALTY * rc2f_floor_deficit_metric(proxy,
↪ "proxy_class4", RC2G_CLASS4_FLOOR, class_id=4, after_task=after_task)
    c5_pen = RC2G_CLASS5_FLOOR_PENALTY * rc2f_floor_deficit_metric(proxy,
↪ "proxy_class5", RC2G_CLASS5_FLOOR, class_id=5, after_task=after_task)
    tri_min_pen = RC2G_TRI_MIN_FLOOR_PENALTY * max(0.0, RC2G_TRI_MIN_FLOOR -
↪ tri["tri_min"]) if tri["tri_values"] else 0.0
    tri_gap_pen = RC2G_TRI_BALANCE_PENALTY * tri["tri_gap"] if
↪ tri["tri_values"] else 0.0
    pred4_pen = rc2g_pred4_bias_penalty(proxy, after_task=after_task)
    min_task_pen = RC2C_MIN_TASK_FLOOR_PENALTY *
↪ rc2f_floor_deficit_metric(proxy, "proxy_min_task", RC2C_MIN_TASK_FLOOR,
↪ after_task=after_task)

```

```

    old_task_pen = RC2C_OLD_TASK_FLOOR_PENALTY *
    ↪rc2f_floor_deficit_metric(proxy, "proxy_old_task_min", RC2C_OLD_TASK_FLOOR,
    ↪after_task=after_task)
    acc_pen = RC2D_PROXY_ACC_FLOOR_PENALTY * rc2f_floor_deficit_metric(proxy,
    ↪"proxy_acc", RC2D_PROXY_ACC_FLOOR, after_task=after_task)
    zero_pen = RC2G_ZERO_SEEN_CLASS_PENALTY * max(0, zero_count)
    dispersion_pen = RC2D_TASK_STD_WEIGHT * max(0.0, task_std)
    no_repair_pen = RC2C_NO_REPAIR_SOFT_PENALTY if after_task is not None and
    ↪int(after_task) >= RC2C_NO_REPAIR_VETO_AFTER_TASK and family ==
    ↪"proto_no_repair" else 0.0
    complexity_pen = RC2B_COMPLEXITY_PENALTY *
    ↪rc2b_policy_complexity_rank(variant)
    hydro_pen = rc3_hydro_selection_penalty(proxy)
    guarded_bonus = RC2D_CONTEXT_GUARD_BONUS if
    ↪rc2c_is_guarded_context_candidate(variant) else 0.0
    true_guard_bonus = RC2D_TRUE_GUARD_BONUS if family ==
    ↪"proto_true_global_guard" else 0.0

    total_pen = c2_pen + c4_pen + c5_pen + tri_min_pen + tri_gap_pen +
    ↪pred4_pen + min_task_pen + old_task_pen + acc_pen + zero_pen +
    ↪dispersion_pen + no_repair_pen + complexity_pen + hydro_pen
    return {
        "proxy_acc_floor_penalty": acc_pen,
        "class2_floor_penalty": c2_pen,
        "class4_floor_penalty": c4_pen,
        "class5_floor_penalty": c5_pen,
        "tri_min_floor_penalty": tri_min_pen,
        "tri_balance_penalty": tri_gap_pen,
        "pred4_bias_penalty": pred4_pen,
        "zero_seen_class_penalty": zero_pen,
        "min_task_floor_penalty": min_task_pen,
        "old_task_floor_penalty": old_task_pen,
        "task_dispersion_penalty": dispersion_pen,
        "no_repair_soft_penalty": no_repair_pen,
        "complexity_penalty": complexity_pen,
        "guarded_context_bonus": guarded_bonus,
        "true_guard_bonus": true_guard_bonus,
        "hydro_selection_penalty": hydro_pen,
        "total_risk_penalty": total_pen,
        "rc2g_tri_min": tri["tri_min"],
        "rc2g_tri_mean": tri["tri_mean"],
        "rc2g_tri_gap": tri["tri_gap"],
        "rc2g_pred4_rate": rc2g_pred4_rate(proxy),
        "rc2g_selector_version": RC2G_SELECTOR_VERSION,
    }

```

```

def rc2d_gate_aligned_score(proxy, baseline_proxy, variant: str,
    ↪after_task=None):
    """RC2G tri-boundary equilibrium validation score."""
    base_score = rc2f_nan_safe_validation_base(proxy, baseline_proxy)
    acc = rc2f_metric(proxy, "proxy_acc", neutral=0.0)
    min_task = rc2f_metric(proxy, "proxy_min_task", neutral=acc)
    old_task_min = rc2f_metric(proxy, "proxy_old_task_min", neutral=min_task)
    task_std = rc2f_metric(proxy, "proxy_task_acc_std", neutral=0.0)
    tri = rc2g_tri_stats(proxy, after_task=after_task)
    comp = rc2c_risk_components(proxy, baseline_proxy, variant,
    ↪after_task=after_task)
    tri_score = (
        RC2G_TRI_MIN_SCORE_WEIGHT * tri["tri_min"]
        + RC2G_TRI_MEAN_SCORE_WEIGHT * tri["tri_mean"]
        - RC2G_TRI_BALANCE_SCORE_WEIGHT * tri["tri_gap"]
    ) if tri["tri_values"] else 0.0
    operational_score = (
        acc
        + 0.30 * min_task
        + 0.15 * old_task_min
        + tri_score
        - 0.20 * max(0.0, task_std)
        - rc2g_pred4_bias_penalty(proxy, after_task=after_task)
    )
    score = float(
        0.42 * base_score
        + 0.58 * operational_score
        + comp["guarded_context_bonus"]
        + comp["true_guard_bonus"]
        - comp["total_risk_penalty"]
    )
    return score if rc2f_finite(score) else -1e9

def rc2g_candidate_tri_min(candidate, after_task=None):
    return rc2g_tri_stats(candidate.get("proxy", {}),
    ↪after_task=after_task)["tri_min"]

def rc2g_candidate_tri_gap(candidate, after_task=None):
    return rc2g_tri_stats(candidate.get("proxy", {}),
    ↪after_task=after_task)["tri_gap"]

def rc2b_validation_score(proxy, baseline_proxy, variant: str, after_task=None):

```

```

    """Override: RC2G tri-boundary equilibrium score, with RC2b strict fallback.
    ↪"""
    if not RC2C_RISK_AWARE_SELECTOR_ENABLED or globals().get("POLICY_BRANCH")_
    ↪== "RC2b":
        base_score = rc2f_nan_safe_validation_base(proxy, baseline_proxy)
        tri = rc2g_tri_stats(proxy, after_task=after_task)
        penalty = RC2B_COMPLEXITY_PENALTY * rc2b_policy_complexity_rank(variant)
        score = base_score + 0.20 * tri["tri_min"] - 0.10 * tri["tri_gap"] -_
    ↪penalty - rc2g_pred4_bias_penalty(proxy, after_task=after_task)
        return float(score) if rc2f_finite(score) else -1e9
        return rc2d_gate_aligned_score(proxy, baseline_proxy, variant,_
    ↪after_task=after_task)

def rc2f_emergency_candidate_score(candidate, after_task=None):
    p = candidate.get("proxy", {})
    acc = rc2f_metric(p, "proxy_acc", neutral=0.0)
    min_task = rc2f_metric(p, "proxy_min_task", neutral=acc)
    tri = rc2g_tri_stats(p, after_task=after_task)
    zero_count = int(p.get("proxy_zero_seen_class_count", 0) or 0)
    score = 0.45 * acc + 0.20 * min_task + 0.30 * tri["tri_min"] - 0.15 *_
    ↪tri["tri_gap"] - 0.20 * zero_count
    score -= RC2B_COMPLEXITY_PENALTY * rc2b_policy_complexity_rank(candidate.
    ↪get("variant", ""))
    return float(score) if rc2f_finite(score) else -1e9

def rc2c_select_candidate(candidates, after_task):
    """RC2G selector: accuracy + tri-boundary equilibrium + no-leakage gates."""
    emergency_scoring_used = False
    scored = [c for c in candidates if rc2f_finite(c.get("selection_score",_
    ↪-1e9))]
    if not scored and RC2G_EMERGENCY_SCORE_IF_ALL_NAN:
        for c in candidates:
            c["selection_score"] = rc2f_emergency_candidate_score(c,_
    ↪after_task=after_task)
            c["rc2g_emergency_score_used"] = True
        scored = [c for c in candidates if rc2f_finite(c.get("selection_score",_
    ↪-1e9))]
        emergency_scoring_used = True
    if not scored:
        selected = max(candidates, key=lambda c: (rc2d_candidate_proxy_acc(c),_
    ↪rc2b_policy_complexity_rank(c.get("variant", ""))))
        return selected, {
            "raw_best_variant": selected["variant"],
            "selection_reason": "fallback_no_finite_scores_proxy_acc_tiebreak",

```

```

        "rc2g_selector_version": RC2G_SELECTOR_VERSION,
        "rc2g_emergency_scoring_used": True,
    }

    sorted_by_score = sorted(
        scored,
        key=lambda c: (
            float(c.get("selection_score", -1e9)),
            rc2g_candidate_tri_min(c, after_task),
            -rc2g_candidate_tri_gap(c, after_task),
            rc2d_candidate_proxy_acc(c),
            -rc2f_candidate_zero_seen_class_count(c),
            -float(c["complexity_rank"]),
        ),
        reverse=True,
    )
    raw_best = sorted_by_score[0]
    selected = raw_best
    reason = "direct_rc2g_tri_boundary_equilibrium_score_win" if not
    ↪emergency_scoring_used else "rc2g_emergency_score_win"
    no_repair_veto = False
    hysteresis = False
    floor_fallback = False
    true_guard_rescue = False
    class2_rescue = False
    class4_rescue = False
    class5_rescue = False
    tri_rescue = False
    zero_reject = False

    guarded_candidates = [c for c in scored if
    ↪rc2c_is_guarded_context_candidate(c["variant"])]
    best_guarded = max(guarded_candidates, key=lambda c: float(c.
    ↪get("selection_score", -1e9))) if guarded_candidates else None
    true_guard_candidates = [c for c in scored if
    ↪rc2b_policy_family(c["variant"]) == "proto_true_global_guard"]
    best_true_guard = max(true_guard_candidates, key=lambda c: float(c.
    ↪get("selection_score", -1e9))) if true_guard_candidates else None

    if best_guarded is not None and int(after_task) >=
    ↪RC2C_NO_REPAIR_VETO_AFTER_TASK and rc2b_policy_family(raw_best["variant"])
    ↪== "proto_no_repair":
        margin = float(raw_best.get("selection_score", -1e9)) -
    ↪float(best_guarded.get("selection_score", -1e9))
        if margin < RC2C_NO_REPAIR_REQUIRED_MARGIN:
            selected = best_guarded

```



```

        reason = f"no_repair_veto_guarded_fallback_margin_{margin:+.4f}"
        no_repair_veto = True

    if RC2G_ZERO_SEEN_CLASS_REJECT_ENABLED and
↳rc2f_candidate_zero_seen_class_count(selected) > 0:
        safe_candidates = [c for c in scored if
↳rc2f_candidate_zero_seen_class_count(c) == 0]
        if safe_candidates:
            selected = max(safe_candidates, key=lambda c: (float(c.
↳get("selection_score", -1e9)), rc2g_candidate_tri_min(c, after_task)))
            reason = "zero_seen_class_reject_rescue"
            zero_reject = True

    # Tri-min rescue: prefer a substantially more balanced 2/4/5 candidate if it
    # is not too far behind on validation score.
    tri_candidates = [c for c in scored if rc2g_tri_stats(c.get("proxy", {}),
↳after_task=after_task)["tri_values"]]
    if tri_candidates:
        best_tri = max(tri_candidates, key=lambda c: (rc2g_candidate_tri_min(c,
↳after_task), -rc2g_candidate_tri_gap(c, after_task), float(c.
↳get("selection_score", -1e9))))
        if (
            rc2g_candidate_tri_min(best_tri, after_task) >=
↳rc2g_candidate_tri_min(selected, after_task) + 0.08
            and rc2d_not_too_far(best_tri, selected, 0.10)
        ):
            selected = best_tri
            reason = "tri_boundary_min_rescue"
            tri_rescue = True

    # Class-specific rescue. RC2G gives class 2 and class 5 more rescue room
    # than RC2F, as long as tri-min does not collapse.
    for cls, floor, rescue_margin, flag_name in [
        (2, RC2G_CLASS2_FLOOR, 0.10, "class2"),
        (4, RC2G_CLASS4_FLOOR, 0.08, "class4"),
        (5, RC2G_CLASS5_FLOOR, 0.10, "class5"),
    ]:
        if not rc2f_class_applicable(cls, after_task):
            continue
        selected_val = rc2f_candidate_class(selected, cls)
        if rc2f_finite(selected_val) and selected_val < floor:
            cls_candidates = [c for c in scored if
↳rc2f_finite(rc2f_candidate_class(c, cls))]
            if cls_candidates:

```

```

        best_cls = max(cls_candidates, key=lambda c:
↳(rc2f_candidate_class(c, cls), rc2g_candidate_tri_min(c, after_task),
↳float(c.get("selection_score", -1e9)))

        tri_ok = rc2g_candidate_tri_min(best_cls, after_task) >=
↳rc2g_candidate_tri_min(selected, after_task) - 0.05
        not_far = rc2d_not_too_far(best_cls, selected, 0.12)
        if rc2f_candidate_class(best_cls, cls) >= selected_val +
↳rescue_margin and tri_ok and not_far:
            selected = best_cls
            reason = f"{flag_name}_equilibrium_rescue"
            class2_rescue = class2_rescue or cls == 2
            class4_rescue = class4_rescue or cls == 4
            class5_rescue = class5_rescue or cls == 5

        if best_true_guard is not None and rc2d_not_too_far(best_true_guard,
↳selected, 0.08):
            tg_tri = rc2g_candidate_tri_min(best_true_guard, after_task)
            se_tri = rc2g_candidate_tri_min(selected, after_task)
            tg_acc = rc2d_candidate_proxy_acc(best_true_guard)
            se_acc = rc2d_candidate_proxy_acc(selected)
            if (tg_tri >= se_tri + 0.06 or tg_acc >= se_acc + 0.015) and
↳rc2c_candidate_meets_hard_floors(best_true_guard):
                selected = best_true_guard
                reason = "true_global_guard_rescue_for_tri_boundary_alignment"
                true_guard_rescue = True

        if RC2C_FLOOR_RISK_FALLBACK_ENABLED and not
↳rc2c_candidate_meets_hard_floors(selected):
            floor_safe = [c for c in scored if rc2c_candidate_meets_hard_floors(c)]
            if floor_safe:
                selected = max(floor_safe, key=lambda c: (float(c.
↳get("selection_score", -1e9)), rc2g_candidate_tri_min(c, after_task)))
                reason = "tri_boundary_hard_floor_fallback"
                floor_fallback = True

        if best_guarded is not None and not
↳rc2c_is_guarded_context_candidate(selected["variant"]) and
↳rc2d_not_too_far(best_guarded, selected, RC2D_GUARDED_HYSTERESIS_MARGIN):
            bg_tri = rc2g_candidate_tri_min(best_guarded, after_task)
            se_tri = rc2g_candidate_tri_min(selected, after_task)
            if bg_tri >= se_tri - 0.03 and
↳rc2c_candidate_meets_hard_floors(best_guarded):
                selected = best_guarded
                reason = "tri_boundary_guarded_context_hysteresis"
                hysteresis = True

```

```

    selected_tri = rc2g_tri_stats(selected.get("proxy", {}),
    ↪after_task=after_task)
    raw_tri = rc2g_tri_stats(raw_best.get("proxy", {}), after_task=after_task)
    return selected, {
        "raw_best_variant": raw_best["variant"],
        "raw_best_policy_family": raw_best["policy_family"],
        "raw_best_proxy_score": raw_best.get("selection_score", np.nan),
        "best_guarded_variant": best_guarded["variant"] if best_guarded is not
    ↪None else "",
        "best_guarded_proxy_score": best_guarded.get("selection_score", np.nan),
    ↪if best_guarded is not None else np.nan,
        "best_true_guard_variant": best_true_guard["variant"] if
    ↪best_true_guard is not None else "",
        "best_true_guard_proxy_score": best_true_guard.get("selection_score",
    ↪np.nan) if best_true_guard is not None else np.nan,
        "selection_reason": reason,
        "no_repair_veto_applied": no_repair_veto,
        "guarded_hysteresis_applied": hysteresis,
        "risk_floor_fallback_applied": floor_fallback,
        "true_guard_rescue_applied": true_guard_rescue,
        "class2_rescue_applied": class2_rescue,
        "class4_rescue_applied": class4_rescue,
        "class5_rescue_applied": class5_rescue,
        "zero_seen_class_reject_applied": zero_reject,
        "rc2g_tri_rescue_applied": tri_rescue,
        "rc2g_selector_version": RC2G_SELECTOR_VERSION,
        "rc2g_emergency_scoring_used": emergency_scoring_used,
        "rc2g_selected_tri_min": selected_tri["tri_min"],
        "rc2g_selected_tri_mean": selected_tri["tri_mean"],
        "rc2g_selected_tri_gap": selected_tri["tri_gap"],
        "rc2g_raw_best_tri_min": raw_tri["tri_min"],
        "rc2g_raw_best_tri_gap": raw_tri["tri_gap"],
        "rc2g_class2_applicable": rc2f_class_applicable(2, after_task),
        "rc2g_class4_applicable": rc2f_class_applicable(4, after_task),
        "rc2g_class5_applicable": rc2f_class_applicable(5, after_task),
    }

print("RC2G tri-boundary equilibrium overrides loaded:", RC2G_SELECTOR_VERSION)
print("RC2G_TRI_CLASSES:", RC2G_TRI_CLASSES)
print("RC2G_CLASS2/4/5_FLOORS:", RC2G_CLASS2_FLOOR, RC2G_CLASS4_FLOOR,
    ↪RC2G_CLASS5_FLOOR)

```

```

RC2G tri-boundary equilibrium overrides loaded: RC2g_tri_boundary_equilibrium_v1
RC2G_TRI_CLASSES: [2, 4, 5]
RC2G_CLASS2/4/5_FLOORS: 0.8 0.75 0.8

```

```

[68]: # -----
# v1.1-RC2Je clean-context final-lock-veto selector overrides
# -----
# RC2I-MQ showed that the best deployable policy is dataset-regime dependent:
# - FashionMNIST: RC1b guarded context-gap wins.
# - MNIST / RotatedMNIST / PermutedMNIST: proto/global no-repair often wins.
#
# RC2J therefore does not use one universal score formula as the main decision.
# It first classifies the current checkpoint using no-leakage validation-memory
# diagnostics, then selects an already validated policy family.
#
# No final-test examples are used by this selector.
# -----

RC2J_SELECTOR_VERSION = "RC2jf_true_guard_safe_veto_v1"

# Validated policy families.
RC2J_PROTO_VARIANTS = ["proto_global_no_repair"]
RC2J_CONTEXT_GAP_VARIANTS = [
    "context_gap_selected",
    "context_temp_blend_selected_head_guard_035",
]
RC2J_TRUE_GUARD_VARIANTS = ["proto_global_head_ce_kl_guard_035"]
RC2J_CONTEXT_ONLY_GLOBAL_VARIANTS = [
    "context_blend_selected_global",
    "context_temp_blend_selected_global",
]
RC2J_GUARDED_FALLBACK_VARIANTS = RC2J_CONTEXT_GAP_VARIANTS +
    ↪RC2J_TRUE_GUARD_VARIANTS

# Regime thresholds inferred from RC2I-MQ policy-regime analysis.
# These are intentionally simple and auditable.
if RUN_MODE == "smoke":
    RC2J_CLEAN_CONTEXT_TOP1_FLOOR = 0.90
    RC2J_CLEAN_CONTEXT_ENTROPY_MAX = 0.35
    RC2J_CLEAN_PROTO_TRI_MIN_FLOOR = 0.45
    RC2J_AMBIGUOUS_CONTEXT_GAIN_MIN = 0.015
    RC2J_AMBIGUOUS_PROTO_TRI_WEAK_MAX = 0.70
else:
    RC2J_CLEAN_CONTEXT_TOP1_FLOOR = 0.98
    RC2J_CLEAN_CONTEXT_ENTROPY_MAX = 0.08
    RC2J_CLEAN_PROTO_TRI_MIN_FLOOR = 0.82
    RC2J_AMBIGUOUS_CONTEXT_GAIN_MIN = 0.010
    RC2J_AMBIGUOUS_PROTO_TRI_WEAK_MAX = 0.82

RC2J_AMBIGUOUS_CONTEXT_TOP1_MAX = 0.95
RC2J_AMBIGUOUS_CONTEXT_ENTROPY_MIN = 0.05

```

```

RC2J_CONTEXT_TRI_GAIN_MIN = 0.015
RC2J_HIGH_HYDRO_TURBULENCE = 0.12
RC2J_MIXED_REGIME_GUARD_MARGIN = 0.005
RC2J_REJECT_ZERO_SEEN_CLASS = True

# RC2Jb/RC2Jc patches: late ambiguity hysteresis plus final-checkpoint
↳ ambiguity lock.
# Motivation: RC2J FashionMNIST evidence was too cautious at late checkpoints;
# it labeled them mixed_or_uncertain and often fell back to proto/global or
↳ true guard.
# These thresholds are validation-memory only and intentionally conservative
↳ for clean datasets.
RC2JB_LATE_TASK_START = 3
RC2JB_LATE_PROTO_TOP1_MAX = 0.90
RC2JB_LATE_PROTO_ENTROPY_MIN = 0.10
RC2JB_LATE_GAP_SURROGATE_MIN = 0.20
RC2JB_LATE_HYDRO_TURBULENCE_MIN = 0.15
RC2JB_LATE_CONTEXT_ACC_TOL = 0.03
RC2JB_LATE_CONTEXT_SCORE_TOL = 0.06
RC2JB_LATE_CONTEXT_TRI_MIN_FLOOR = 0.70 if RUN_MODE != "smoke" else 0.35
RC2JB_LATE_REQUIRE_CONTEXT_SAFE = True

# RC2Jc patch: final-checkpoint ambiguity lock.
# Motivation: RC2Jb evidence still selected proto/global or true-guard at the
# final FashionMNIST checkpoint even when final post-selection evidence favored
# RC1b guarded context-gap. This lock is final-checkpoint-only and uses only
# validation-memory/audit signals.
RC2JC_FINAL_LOCK_ENABLED = True
RC2JC_FINAL_LOCK_CHECKPOINT = N_TASKS - 1
RC2JC_FINAL_PROTO_TOP1_MAX = 0.85
RC2JC_FINAL_PROTO_ENTROPY_MIN = 0.12
RC2JC_FINAL_GAP_SURROGATE_MIN = 0.25
RC2JC_FINAL_CONTEXT_ACC_TOL_VS_RAW = 0.04
RC2JC_FINAL_CONTEXT_ACC_TOL_VS_PROTO = 0.04
RC2JC_FINAL_CONTEXT_TRI_WARNING_FLOOR = 0.75
RC2JC_FINAL_CONTEXT_TRI_HARD_FLOOR = 0.70 if RUN_MODE != "smoke" else 0.35
RC2JC_FINAL_MIN_UNCERTAINTY_SIGNALS = 2
RC2JC_FINAL_REQUIRE_CONTEXT_SAFE = True

# RC2Je patch: hard-clarity clean-veto guard.
# Motivation: RC2Jd fixed MNIST by vetoing RC2Jc's final ambiguity lock,
# but FashionMNIST evidence showed that the veto was too permissive. RC2Je
# only vetoes when **all hard clarity gates** pass on validation memory.
RC2JE_HARD_CLARITY_VETO_ENABLED = True
RC2JE_HARD_CLARITY_FINAL_CHECKPOINT = N_TASKS - 1
RC2JE_HARD_PROTO_ACC_FLOOR = 0.92 if RUN_MODE != "smoke" else 0.78
RC2JE_HARD_PROTO_TRI_MIN_FLOOR = 0.88 if RUN_MODE != "smoke" else 0.55

```

```

RC2JE_HARD_CONTEXT_GAIN_MAX = 0.012 if RUN_MODE != "smoke" else 0.060
RC2JE_HARD_PROTO_SCORE_TOL_VS_RAW = 0.060 if RUN_MODE != "smoke" else 0.120
RC2JE_HARD_GAP_SURROGATE_MAX = 0.35 if RUN_MODE != "smoke" else 0.65
RC2JE_HARD_PROTO_TOP1_FLOOR = 0.85 if RUN_MODE != "smoke" else 0.70
RC2JE_HARD_PROTO_ENTROPY_MAX = 0.12 if RUN_MODE != "smoke" else 0.75
RC2JE_HARD_HYDRO_TURBULENCE_MAX = 0.12 if RUN_MODE != "smoke" else 0.30
RC2JE_HARD_REQUIRED_SIGNALS = 7 if RUN_MODE != "smoke" else 5
RC2JE_HARD_REQUIRE_PROTO_SAFE = True

# RC2JF patch: true-guard safe veto for domain-shift regimes such as
↳ PermutedMNIST.

# This does not add a new model; it only prevents the final ambiguity lock from
# overriding a validation-superior true global CE/KL guard when weak-boundary
# diagnostics support that choice.
RC2JF_TRUE_GUARD_SAFE_VETO_ENABLED = True
RC2JF_TRUE_GUARD_FINAL_CHECKPOINT = N_TASKS - 1
RC2JF_TRUE_GUARD_MIN_ACC_GAIN = 0.003 if RUN_MODE == "smoke" else 0.005
RC2JF_TRUE_GUARD_MIN_SCORE_GAIN = 0.001 if RUN_MODE == "smoke" else 0.003
RC2JF_TRUE_GUARD_MIN_CLASS5_GAIN = 0.010
RC2JF_TRUE_GUARD_MINCLASS_GAIN = 0.010
RC2JF_TRUE_GUARD_TRI_MIN_FLOOR = 0.55 if RUN_MODE == "smoke" else 0.65
RC2JF_TRUE_GUARD_BOUNDARY_NOT_WORSE_TOL = 0.005
RC2JF_TRUE_GUARD_RAW_BEST_REQUIRED = True

# RC2JG selector-hygiene patch: dataset-agnostic not-below-context checks.
# Motivation: FashionMNIST robust showed that a true-guard veto can be
↳ validation-attractive

# while still trailing the guarded context-gap family on the fragile boundary/
↳ min-task regime.

# These checks are validation-memory-only and do not read final-test data.
RC2JF_DATASET_AGNOSTIC_TRUE_GUARD_HYGIENE_ENABLED = True
RC2JF_CONTEXT_NOT_BELOW_TOL = 0.010 if RUN_MODE == "smoke" else 0.002
RC2JF_CLASS_NOT_BELOW_TOL = 0.012 if RUN_MODE == "smoke" else 0.003
RC2JF_MIN_TASK_NOT_BELOW_TOL = 0.012 if RUN_MODE == "smoke" else 0.003
RC2JF_HYDRO_TURBULENCE_NOT_WORSE_TOL = 0.050 if RUN_MODE == "smoke" else 0.015
RC2JF_HYDRO_OUTPUT_DRIFT_NOT_WORSE_TOL = 0.075 if RUN_MODE == "smoke" else 0.025
RC2JF_WEAK_CLASS_SENTINEL = 2 # Used only when available; otherwise falls back
↳ to weakest seen class.

def rc2j_finite(x):
    try:
        return np.isfinite(float(x))
    except Exception:
        return False

```

```

def rc2j_value(obj, key, neutral=np.nan):
    try:
        if obj is None:
            return neutral
        if isinstance(obj, dict):
            v = obj.get(key, neutral)
        else:
            v = getattr(obj, key, neutral)
        return float(v) if rc2j_finite(v) else neutral
    except Exception:
        return neutral

def rc2j_proxy(candidate):
    return candidate.get("proxy", {}) if candidate is not None else {}

def rc2j_candidate_acc(candidate):
    return rc2j_value(rc2j_proxy(candidate), "proxy_acc", neutral=-1e9)

def rc2j_score(candidate):
    return float(candidate.get("selection_score", -1e9)) if candidate is not_
↪None and rc2j_finite(candidate.get("selection_score", -1e9)) else -1e9

def rc2jf_proxy_class_ids(proxy, after_task=None):
    """Return finite class ids available in a proxy, optionally restricted to_
↪seen classes.

    Dataset-agnostic: uses proxy_class<K> keys, not a dataset name. If class 2
exists it can still be inspected as a sentinel, but the generic fallback is
the weakest seen class in validation memory.
    """
    if proxy is None:
        return []
    keys = []
    for k, v in proxy.items():
        if not str(k).startswith("proxy_class"):
            continue
        suffix = str(k)[len("proxy_class"):]
        if suffix.isdigit() and rc2j_finite(v):
            keys.append(int(suffix))
    seen_filter = None
    if after_task is not None:
        try:

```

```

        seen_filter = set(int(c) for c in
↪seen_classes_until(int(after_task)))
        except Exception:
            seen_filter = None
        if seen_filter is not None:
            keys = [c for c in keys if c in seen_filter]
        return sorted(set(keys))

def rc2jf_class_proxy(proxy, class_id, neutral=np.nan):
    return rc2j_value(proxy, f"proxy_class{int(class_id)}", neutral=neutral)

def rc2jf_weak_boundary_score(proxy, after_task=None):
    """Generic validation weak-boundary score: min available seen-class
↪accuracy."""
    direct = rc2j_value(proxy, "proxy_min_class", neutral=np.nan)
    if rc2j_finite(direct):
        return float(direct)
    vals = [rc2jf_class_proxy(proxy, c, neutral=np.nan) for c in
↪rc2jf_proxy_class_ids(proxy, after_task)]
    vals = [v for v in vals if rc2j_finite(v)]
    return float(min(vals)) if vals else np.nan

def rc2jf_anchor_weak_class_score(target_proxy, reference_proxy,
↪after_task=None):
    """Return target score on the class used as weak-class guard.

    Prefer the known class-2 sentinel when it is available in both candidates,
    because class 2 was the observed fragile boundary in FashionMNIST.
↪Otherwise
    choose the weakest finite seen class of the reference/context candidate.
    """
    sentinel = int(globals().get("RC2JF_WEAK_CLASS_SENTINEL", 2))
    if rc2j_finite(rc2jf_class_proxy(target_proxy, sentinel)) and
↪rc2j_finite(rc2jf_class_proxy(reference_proxy, sentinel)):
        return rc2jf_class_proxy(target_proxy, sentinel), sentinel
    available = []
    for c in rc2jf_proxy_class_ids(reference_proxy, after_task):
        rv = rc2jf_class_proxy(reference_proxy, c)
        tv = rc2jf_class_proxy(target_proxy, c)
        if rc2j_finite(rv) and rc2j_finite(tv):
            available.append((c, rv))
    if not available:
        return np.nan, -1

```



```

weak_c = int(sorted(available, key=lambda x: x[1])[0][0])
return rc2jf_class_proxy(target_proxy, weak_c), weak_c

def rc2jf_dataset_agnostic_not_below_context(true_guard_proxy, context_proxy,
    ↪after_task=None):
    """Validation-memory-only hygiene checks for true-guard veto.

    Higher is better for weak-boundary, weak-class, and min-task proxies.
    Lower is better for Hydro turbulence and output drift.
    Missing Hydro values are neutral so non-Hydro profiles remain comparable.
    """

    tg_weak = rc2jf_weak_boundary_score(true_guard_proxy, after_task)
    ctx_weak = rc2jf_weak_boundary_score(context_proxy, after_task)
    tg_class, anchor_class = rc2jf_anchor_weak_class_score(true_guard_proxy,
    ↪context_proxy, after_task)
    ctx_class = rc2jf_class_proxy(context_proxy, anchor_class) if
    ↪int(anchor_class) >= 0 else np.nan
    tg_min_task = rc2j_value(true_guard_proxy, "proxy_min_task", neutral=np.nan)
    ctx_min_task = rc2j_value(context_proxy, "proxy_min_task", neutral=np.nan)
    tg_turb = rc2j_value(true_guard_proxy, "hydro_turbulence_score", neutral=np.
    ↪nan)
    ctx_turb = rc2j_value(context_proxy, "hydro_turbulence_score", neutral=np.
    ↪nan)
    tg_output = rc2j_value(true_guard_proxy, "hydro_output_drift_Dy",
    ↪neutral=np.nan)
    ctx_output = rc2j_value(context_proxy, "hydro_output_drift_Dy", neutral=np.
    ↪nan)

    weak_margin = tg_weak - ctx_weak if rc2j_finite(tg_weak) and
    ↪rc2j_finite(ctx_weak) else np.nan
    class_margin = tg_class - ctx_class if rc2j_finite(tg_class) and
    ↪rc2j_finite(ctx_class) else np.nan
    min_task_margin = tg_min_task - ctx_min_task if rc2j_finite(tg_min_task)
    ↪and rc2j_finite(ctx_min_task) else np.nan
    # Negative values mean the true guard is calmer than context-gap, which is
    ↪good.
    hydro_turbulence_delta = tg_turb - ctx_turb if rc2j_finite(tg_turb) and
    ↪rc2j_finite(ctx_turb) else np.nan
    hydro_output_delta = tg_output - ctx_output if rc2j_finite(tg_output) and
    ↪rc2j_finite(ctx_output) else np.nan

    weak_boundary_ok = (not rc2j_finite(weak_margin)) or weak_margin >=
    ↪RC2JF_CONTEXT_NOT_BELOW_TOL
    weak_class_ok = (not rc2j_finite(class_margin)) or class_margin >=
    ↪RC2JF_CLASS_NOT_BELOW_TOL

```

```

    min_task_ok = (not rc2j_finite(min_task_margin)) or min_task_margin >=
↪RC2JF_MIN_TASK_NOT_BELOW_TOL
    hydro_turbulence_ok = (not rc2j_finite(hydro_turbulence_delta)) or
↪hydro_turbulence_delta <= RC2JF_HYDRO_TURBULENCE_NOT_WORSE_TOL
    hydro_output_ok = (not rc2j_finite(hydro_output_delta)) or
↪hydro_output_delta <= RC2JF_HYDRO_OUTPUT_DRIFT_NOT_WORSE_TOL
    passed = bool(weak_boundary_ok and weak_class_ok and min_task_ok and
↪hydro_turbulence_ok and hydro_output_ok)
    return passed, {
        "dataset_agnostic_context_not_below": passed,
        "weak_anchor_class": int(anchor_class),
        "weak_boundary_margin_vs_context": weak_margin,
        "weak_class_margin_vs_context": class_margin,
        "min_task_margin_vs_context": min_task_margin,
        "hydro_turbulence_delta_vs_context": hydro_turbulence_delta,
        "hydro_output_delta_vs_context": hydro_output_delta,
        "weak_boundary_not_below_context": bool(weak_boundary_ok),
        "weak_class_not_below_context": bool(weak_class_ok),
        "min_task_not_below_context": bool(min_task_ok),
        "hydro_turbulence_not_worse_than_context": bool(hydro_turbulence_ok),
        "hydro_output_not_worse_than_context": bool(hydro_output_ok),
        "true_guard_weak_boundary_score": tg_weak,
        "context_weak_boundary_score": ctx_weak,
        "true_guard_anchor_class_score": tg_class,
        "context_anchor_class_score": ctx_class,
        "true_guard_min_task": tg_min_task,
        "context_min_task": ctx_min_task,
        "true_guard_hydro_turbulence": tg_turb,
        "context_hydro_turbulence": ctx_turb,
        "true_guard_hydro_output_drift": tg_output,
        "context_hydro_output_drift": ctx_output,
    }
}

```

```

def rc2j_tri_stats_from_proxy(proxy, after_task):
    if "rc2g_tri_stats" in globals():
        try:
            return rc2g_tri_stats(proxy, after_task=after_task)
        except Exception:
            pass
    vals = []
    for cls, key in [(2, "proxy_class2"), (4, "proxy_class4"), (5,
↪"proxy_class5")]:
        try:
            applicable = cls in seen_classes_until(int(after_task))
        except Exception:
            applicable = True

```

```

        if applicable:
            v = rc2j_value(proxy, key, neutral=np.nan)
            if rc2j_finite(v):
                vals.append(v)
        if not vals:
            return {"tri_min": np.nan, "tri_mean": np.nan, "tri_gap": np.nan}
        return {"tri_min": float(np.min(vals)), "tri_mean": float(np.mean(vals)),
        ↪ "tri_gap": float(np.max(vals) - np.min(vals))}

def rc2j_tri_stats(candidate, after_task):
    return rc2j_tri_stats_from_proxy(rc2j_proxy(candidate),
    ↪ after_task=after_task)

def rc2j_variant(candidate):
    return str(candidate.get("variant", "")) if candidate is not None else ""

def rc2j_family(candidate):
    return rc2b_policy_family(rc2j_variant(candidate)) if candidate is not None
    ↪ else "unknown"

def rc2j_find(candidates, variants):
    variants = set(variants)
    found = [c for c in candidates if rc2j_variant(c) in variants]
    return found

def rc2j_best(candidates, default=None):
    if not candidates:
        return default
    return max(
        candidates,
        key=lambda c: (
            rc2j_score(c),
            rc2j_candidate_acc(c),
            -float(c.get("complexity_rank",
    ↪ rc2b_policy_complexity_rank(rc2j_variant(c))))),
        ),
    )

def rc2j_zero_seen_count(candidate):
    try:

```

```

        return int(rc2j_proxy(candidate).get("proxy_zero_seen_class_count", 0))
    or 0)
    except Exception:
        return 0

def rc2j_safe(candidate):
    if candidate is None:
        return False
    if RC2J_REJECT_ZERO_SEEN_CLASS and rc2j_zero_seen_count(candidate) > 0:
        return False
    if not rc2j_finite(rc2j_candidate_acc(candidate)):
        return False
    return True

def rc2j_regime_metrics(candidates, after_task):
    proto = rc2j_best(rc2j_find(candidates, RC2J_PROTO_VARIANTS))
    context = rc2j_best(rc2j_find(candidates, RC2J_CONTEXT_GAP_VARIANTS))
    true_guard = rc2j_best(rc2j_find(candidates, RC2J_TRUE_GUARD_VARIANTS))
    guarded = rc2j_best(rc2j_find(candidates, RC2J_GUARDED_FALLBACK_VARIANTS))
    context_only = rc2j_best(rc2j_find(candidates,
    RC2J_CONTEXT_ONLY_GLOBAL_VARIANTS))
    raw_best = rc2j_best(candidates)
    raw_best_acc = rc2j_candidate_acc(raw_best)

    proto_proxy = rc2j_proxy(proto)
    context_proxy = rc2j_proxy(context)
    guarded_proxy = rc2j_proxy(guarded)

    proto_acc = rc2j_candidate_acc(proto)
    context_acc = rc2j_candidate_acc(context)
    guarded_acc = rc2j_candidate_acc(guarded)
    true_guard_acc = rc2j_candidate_acc(true_guard)
    context_only_acc = rc2j_candidate_acc(context_only)

    proto_tri = rc2j_tri_stats(proto, after_task)
    context_tri = rc2j_tri_stats(context, after_task)
    guarded_tri = rc2j_tri_stats(guarded, after_task)

    proto_top1 = rc2j_value(proto_proxy, "proxy_context_top1",
    neutral=rc2j_value(proto_proxy, "proxy_context_top1_acc", neutral=np.nan))
    context_top1 = rc2j_value(context_proxy, "proxy_context_top1",
    neutral=rc2j_value(context_proxy, "proxy_context_top1_acc", neutral=np.nan))
    proto_entropy = rc2j_value(proto_proxy, "proxy_context_entropy", neutral=np.
    nan)

```

```

    context_entropy = rc2j_value(context_proxy, "proxy_context_entropy",
↪neutral=np.nan)
    proto_margin = rc2j_value(proto_proxy, "proxy_context_margin", neutral=np.
↪nan)
    context_margin = rc2j_value(context_proxy, "proxy_context_margin",
↪neutral=np.nan)

    context_gain_vs_proto = context_acc - proto_acc if rc2j_finite(context_acc)
↪and rc2j_finite(proto_acc) else np.nan
    guarded_gain_vs_proto = guarded_acc - proto_acc if rc2j_finite(guarded_acc)
↪and rc2j_finite(proto_acc) else np.nan
    context_tri_gain_vs_proto = context_tri.get("tri_min", np.nan) - proto_tri.
↪get("tri_min", np.nan) if rc2j_finite(context_tri.get("tri_min", np.nan))
↪and rc2j_finite(proto_tri.get("tri_min", np.nan)) else np.nan

    hydro_context_turbulence = rc2j_value(context_proxy,
↪"hydro_turbulence_score", neutral=np.nan)
    hydro_proto_turbulence = rc2j_value(proto_proxy, "hydro_turbulence_score",
↪neutral=np.nan)
    hydro_guarded_turbulence = rc2j_value(guarded_proxy,
↪"hydro_turbulence_score", neutral=np.nan)

    # This is a no-leakage surrogate for the proto+oracle gap. The true oracle
↪probe is
    # computed after training for analysis, not for selection. Here we use
↪validation-memory
    # uncertainty and the validation-memory gain of context-gap over proto as a
↪proxy.
    gap_surrogate = 0.0
    if rc2j_finite(proto_top1):
        gap_surrogate += max(0.0, 1.0 - proto_top1)
    if rc2j_finite(proto_entropy):
        gap_surrogate += min(1.0, proto_entropy)
    if rc2j_finite(context_gain_vs_proto):
        gap_surrogate += max(0.0, context_gain_vs_proto)

    return {
        "proto": proto,
        "context": context,
        "true_guard": true_guard,
        "guarded": guarded,
        "context_only": context_only,
        "raw_best": raw_best,
        "raw_best_acc": raw_best_acc,
        "proto_acc": proto_acc,
        "context_acc": context_acc,

```

```

    "guarded_acc": guarded_acc,
    "true_guard_acc": true_guard_acc,
    "context_only_acc": context_only_acc,
    "context_gain_vs_proto": context_gain_vs_proto,
    "guarded_gain_vs_proto": guarded_gain_vs_proto,
    "context_tri_gain_vs_proto": context_tri_gain_vs_proto,
    "proto_top1": proto_top1,
    "context_top1": context_top1,
    "proto_entropy": proto_entropy,
    "context_entropy": context_entropy,
    "proto_margin": proto_margin,
    "context_margin": context_margin,
    "proto_tri_min": proto_tri.get("tri_min", np.nan),
    "context_tri_min": context_tri.get("tri_min", np.nan),
    "guarded_tri_min": guarded_tri.get("tri_min", np.nan),
    "proto_tri_gap": proto_tri.get("tri_gap", np.nan),
    "context_tri_gap": context_tri.get("tri_gap", np.nan),
    "hydro_context_turbulence": hydro_context_turbulence,
    "hydro_proto_turbulence": hydro_proto_turbulence,
    "hydro_guarded_turbulence": hydro_guarded_turbulence,
    "gap_surrogate": gap_surrogate,
}

```

```

def rc2j_classify_regime(metrics, after_task):
    proto_top1 = metrics["proto_top1"]
    proto_entropy = metrics["proto_entropy"]
    proto_tri_min = metrics["proto_tri_min"]
    context_gain = metrics["context_gain_vs_proto"]
    context_tri_gain = metrics["context_tri_gain_vs_proto"]
    hydro_context = metrics["hydro_context_turbulence"]

    clean_conditions = []
    clean_conditions.append(rc2j_finite(proto_top1) and proto_top1 >=
↪RC2J_CLEAN_CONTEXT_TOP1_FLOOR)
    clean_conditions.append((not rc2j_finite(proto_entropy)) or proto_entropy
↪<= RC2J_CLEAN_CONTEXT_ENTROPY_MAX)
    clean_conditions.append((not rc2j_finite(proto_tri_min)) or proto_tri_min
↪>= RC2J_CLEAN_PROTO_TRI_MIN_FLOOR)
    clean_conditions.append((not rc2j_finite(context_gain)) or context_gain <=
↪RC2J_AMBIGUOUS_CONTEXT_GAIN_MIN)

    ambiguous_signals = []
    ambiguous_signals.append(rc2j_finite(context_gain) and context_gain >=
↪RC2J_AMBIGUOUS_CONTEXT_GAIN_MIN)
    ambiguous_signals.append(rc2j_finite(context_tri_gain) and context_tri_gain
↪>= RC2J_CONTEXT_TRI_GAIN_MIN)

```

```

        ambiguous_signals.append(rc2j_finite(proto_top1) and proto_top1 <=
↪RC2J_AMBIGUOUS_CONTEXT_TOP1_MAX)
        ambiguous_signals.append(rc2j_finite(proto_entropy) and proto_entropy >=
↪RC2J_AMBIGUOUS_CONTEXT_ENTROPY_MIN)
        ambiguous_signals.append(rc2j_finite(proto_tri_min) and proto_tri_min <=
↪RC2J_AMBIGUOUS_PROTO_TRI_WEAK_MAX)
        ambiguous_signals.append(rc2j_finite(hydro_context) and hydro_context >=
↪RC2J_HIGH_HYDRO_TURBULENCE)

    n_ambiguous = int(sum(bool(x) for x in ambiguous_signals))
    n_clean = int(sum(bool(x) for x in clean_conditions))

    if n_ambiguous >= 2 and bool(ambiguous_signals[0]):
        return "ambiguous_context",
↪f"context_gain_positive_and_{n_ambiguous}_ambiguous_signals"
    if n_clean >= 3 and not bool(ambiguous_signals[0]):
        return "clean_context", f"{n_clean}_clean_signals_and_no_context_gain"
    return "mixed_or_uncertain", f"clean_signals={n_clean};
↪ambiguous_signals={n_ambiguous}"

def rc2jb_late_ambiguity_trigger(metrics, after_task, context_candidate=None,
↪raw_best=None):
    """Return whether late ambiguity hysteresis should force RC1b context-gap.

This uses only validation-memory/audit signals already computed before
↪final-test
evaluation. It is designed to repair the RC2J FashionMNIST evidence failure:
late checkpoints were uncertain, but the selector fell back to proto/global
↪or true guard.
    """
    try:
        after_task = int(after_task)
    except Exception:
        after_task = -1
    if after_task < RC2JB_LATE_TASK_START:
        return False, {
            "applied": False,
            "reason":
↪f"after_task={after_task}<late_start={RC2JB_LATE_TASK_START}",
            "late_uncertainty_signals": 0,
            "context_score_margin_vs_raw_best": np.nan,
            "context_acc_margin_vs_proto": np.nan,
        }

```

```

    if context_candidate is None or (RC2JB_LATE_REQUIRE_CONTEXT_SAFE and not
↳rc2j_safe(context_candidate)):
        return False, {
            "applied": False,
            "reason": "no_safe_context_gap_candidate",
            "late_uncertainty_signals": 0,
            "context_score_margin_vs_raw_best": np.nan,
            "context_acc_margin_vs_proto": np.nan,
        }

    proto_top1 = metrics.get("proto_top1", np.nan)
    proto_entropy = metrics.get("proto_entropy", np.nan)
    gap_surrogate = metrics.get("gap_surrogate", np.nan)
    hydro_context = metrics.get("hydro_context_turbulence", np.nan)
    context_gain = metrics.get("context_gain_vs_proto", np.nan)
    context_tri_min = metrics.get("context_tri_min", np.nan)

    uncertainty_signals = []
    uncertainty_signals.append(rc2j_finite(proto_top1) and proto_top1 <=
↳RC2JB_LATE_PROTO_TOP1_MAX)
    uncertainty_signals.append(rc2j_finite(proto_entropy) and proto_entropy >=
↳RC2JB_LATE_PROTO_ENTROPY_MIN)
    uncertainty_signals.append(rc2j_finite(gap_surrogate) and gap_surrogate >=
↳RC2JB_LATE_GAP_SURROGATE_MIN)
    uncertainty_signals.append(rc2j_finite(hydro_context) and hydro_context >=
↳RC2JB_LATE_HYDRO_TURBULENCE_MIN)
    late_uncertainty_signals = int(sum(bool(x) for x in uncertainty_signals))

    context_acc_margin_vs_proto = context_gain if rc2j_finite(context_gain)
↳else np.nan
    if raw_best is None:
        context_score_margin_vs_raw_best = np.nan
    else:
        context_score_margin_vs_raw_best = rc2j_score(context_candidate) -
↳rc2j_score(raw_best)

    context_not_too_bad_vs_proto = (not
↳rc2j_finite(context_acc_margin_vs_proto)) or context_acc_margin_vs_proto >=
↳RC2JB_LATE_CONTEXT_ACC_TOL
    context_not_too_bad_vs_raw = (not
↳rc2j_finite(context_score_margin_vs_raw_best)) or
↳context_score_margin_vs_raw_best >= -RC2JB_LATE_CONTEXT_SCORE_TOL
    context_boundary_ok = (not rc2j_finite(context_tri_min)) or context_tri_min
↳>= RC2JB_LATE_CONTEXT_TRI_MIN_FLOOR

    apply = bool(

```



```

        late_uncertainty_signals >= 2
        and context_not_too_bad_vs_proto
        and context_not_too_bad_vs_raw
        and context_boundary_ok
    )
    reasons = []
    reasons.append(f"late_uncertainty_signals={late_uncertainty_signals}")
    reasons.append(f"proto_top1={proto_top1}")
    reasons.append(f"proto_entropy={proto_entropy}")
    reasons.append(f"gap_surrogate={gap_surrogate}")
    reasons.append(f"hydro_context={hydro_context}")
    reasons.append(f"context_acc_margin_vs_proto={context_acc_margin_vs_proto}")
    reasons.append(f"context_score_margin_vs_raw_best={context_score_margin_vs_raw_best}")
    reasons.append(f"context_tri_min={context_tri_min}")
    return apply, {
        "applied": apply,
        "reason": "; ".join(reasons),
        "late_uncertainty_signals": late_uncertainty_signals,
        "context_score_margin_vs_raw_best": context_score_margin_vs_raw_best,
        "context_acc_margin_vs_proto": context_acc_margin_vs_proto,
    }

def rc2jc_final_checkpoint_ambiguity_lock(metrics, after_task,
    context_candidate=None, raw_best=None, proto_candidate=None):
    """Return whether RC2Jc should lock the final checkpoint to RC1b
    context-gap.

    This is deliberately narrower than RC2Jb late hysteresis:
    - it applies only at the final overlap-chain checkpoint;
    - it uses only validation-memory / audit signals;
    - it does not require the context-gap validation score to be close, because
      RC2Jb evidence showed proxy-score pessimism against context-gap;
    - it blocks the lock only when context-gap is catastrophically worse on
      validation-memory accuracy or tri-boundary robustness.
    """
    try:
        after_task = int(after_task)
    except Exception:
        after_task = -1
    if not RC2JC_FINAL_LOCK_ENABLED or after_task != 0:
        return False, {
            "applied": False,

```

```

        "reason": f"after_task={after_task}";
    ↪final_checkpoint={RC2JC_FINAL_LOCK_CHECKPOINT};
    ↪enabled={RC2JC_FINAL_LOCK_ENABLED}",
        "final_uncertainty_signals": 0,
        "context_acc_margin_vs_raw_best": np.nan,
        "context_acc_margin_vs_proto": np.nan,
        "context_tri_warning": False,
        "context_tri_hard_fail": False,
    }

    if context_candidate is None or (RC2JC_FINAL_REQUIRE_CONTEXT_SAFE and not
    ↪rc2j_safe(context_candidate)):
        return False, {
            "applied": False,
            "reason": "no_safe_context_gap_candidate",
            "final_uncertainty_signals": 0,
            "context_acc_margin_vs_raw_best": np.nan,
            "context_acc_margin_vs_proto": np.nan,
            "context_tri_warning": False,
            "context_tri_hard_fail": False,
        }

    proto_top1 = metrics.get("proto_top1", np.nan)
    proto_entropy = metrics.get("proto_entropy", np.nan)
    gap_surrogate = metrics.get("gap_surrogate", np.nan)
    hydro_context = metrics.get("hydro_context_turbulence", np.nan)
    context_gain = metrics.get("context_gain_vs_proto", np.nan)
    context_tri_gain = metrics.get("context_tri_gain_vs_proto", np.nan)
    context_tri_min = metrics.get("context_tri_min", np.nan)
    proto_tri_min = metrics.get("proto_tri_min", np.nan)

    context_acc = rc2j_candidate_acc(context_candidate)
    raw_best_acc = rc2j_candidate_acc(raw_best) if raw_best is not None else
    ↪metrics.get("raw_best_acc", np.nan)
    proto_acc = rc2j_candidate_acc(proto_candidate) if proto_candidate is not
    ↪None else metrics.get("proto_acc", np.nan)

    context_acc_margin_vs_raw_best = context_acc - raw_best_acc if
    ↪rc2j_finite(context_acc) and rc2j_finite(raw_best_acc) else np.nan
    context_acc_margin_vs_proto = context_acc - proto_acc if
    ↪rc2j_finite(context_acc) and rc2j_finite(proto_acc) else np.nan

    uncertainty_signals = []
    uncertainty_signals.append(rc2j_finite(proto_top1) and proto_top1 <=
    ↪RC2JC_FINAL_PROTO_TOP1_MAX)

```

```

    uncertainty_signals.append(rc2j_finite(proto_entropy) and proto_entropy >=
↳RC2JC_FINAL_PROTO_ENTROPY_MIN)
    uncertainty_signals.append(rc2j_finite(gap_surrogate) and gap_surrogate >=
↳RC2JC_FINAL_GAP_SURROGATE_MIN)
    uncertainty_signals.append(rc2j_finite(hydro_context) and hydro_context >=
↳RC2J_HIGH_HYDRO_TURBULENCE)
    uncertainty_signals.append(rc2j_finite(context_gain) and context_gain >=
↳RC2JC_FINAL_CONTEXT_ACC_TOL_VS_PROTO)
    uncertainty_signals.append(rc2j_finite(context_tri_gain) and
↳context_tri_gain >= -0.05)
    uncertainty_signals.append(rc2j_finite(proto_tri_min) and proto_tri_min <=
↳RC2J_AMBIGUOUS_PROTO_TRI_WEAK_MAX)
    final_uncertainty_signals = int(sum(bool(x) for x in uncertainty_signals))

    context_not_catastrophic_vs_raw = (not
↳rc2j_finite(context_acc_margin_vs_raw_best)) or
↳context_acc_margin_vs_raw_best >= -RC2JC_FINAL_CONTEXT_ACC_TOL_VS_RAW
    context_not_catastrophic_vs_proto = (not
↳rc2j_finite(context_acc_margin_vs_proto)) or context_acc_margin_vs_proto >=
↳RC2JC_FINAL_CONTEXT_ACC_TOL_VS_PROTO
    context_tri_warning = bool(rc2j_finite(context_tri_min) and context_tri_min
↳< RC2JC_FINAL_CONTEXT_TRI_WARNING_FLOOR)
    context_tri_hard_fail = bool(rc2j_finite(context_tri_min) and
↳context_tri_min < RC2JC_FINAL_CONTEXT_TRI_HARD_FLOOR)

    apply = bool(
        final_uncertainty_signals >= RC2JC_FINAL_MIN_UNCERTAINTY_SIGNALS
        and context_not_catastrophic_vs_raw
        and context_not_catastrophic_vs_proto
        and not context_tri_hard_fail
    )

    reasons = []
    reasons.append(f"final_uncertainty_signals={final_uncertainty_signals}")
    reasons.append(f"proto_top1={proto_top1}")
    reasons.append(f"proto_entropy={proto_entropy}")
    reasons.append(f"gap_surrogate={gap_surrogate}")
    reasons.append(f"hydro_context={hydro_context}")
    reasons.
↳append(f"context_acc_margin_vs_raw_best={context_acc_margin_vs_raw_best}")
    reasons.append(f"context_acc_margin_vs_proto={context_acc_margin_vs_proto}")
    reasons.append(f"context_tri_min={context_tri_min}")
    reasons.append(f"context_tri_warning={context_tri_warning}")
    reasons.append(f"context_tri_hard_fail={context_tri_hard_fail}")

    return apply, {

```

```

        "applied": apply,
        "reason": "; ".join(reasons),
        "final_uncertainty_signals": final_uncertainty_signals,
        "context_acc_margin_vs_raw_best": context_acc_margin_vs_raw_best,
        "context_acc_margin_vs_proto": context_acc_margin_vs_proto,
        "context_tri_warning": context_tri_warning,
        "context_tri_hard_fail": context_tri_hard_fail,
    }

def rc2je_hard_clarity_clean_veto_guard(metrics, after_task,
    proto_candidate=None, context_candidate=None, raw_best=None):
    """Return whether RC2Je should veto RC2Jc's final ambiguity lock.

    RC2Je is deliberately stricter than RC2Jd. The veto is final-checkpoint-only
    and no-leakage: it uses only validation-memory proxy scores, context
    posterior
    diagnostics and Hydro audit metrics that are already available before
    final-test
    evaluation. It requires hard clarity rather than a loose count of soft
    signals.
    """
    try:
        after_task = int(after_task)
    except Exception:
        after_task = -1

    empty_info = {
        "applied": False,
        "reason": "not_evaluated",
        "hard_clarity_signal_count": 0,
        "proto_acc_margin_vs_context": np.nan,
        "proto_score_margin_vs_raw_best": np.nan,
        "proto_tri_healthy": False,
        "context_gain_veto_ok": False,
        "proto_top1_hard_clear": False,
        "proto_entropy_hard_clear": False,
        "gap_surrogate_hard_clear": False,
        "hydro_context_hard_clear": False,
        "hard_clarity_all_gates_pass": False,
    }

    if not RC2JE_HARD_CLARITY_VETO_ENABLED or after_task !=
    RC2JE_HARD_CLARITY_FINAL_CHECKPOINT:
        info = dict(empty_info)

```

```

        info["reason"] = f"after_task={after_task}";
    ↪final_checkpoint={RC2JE_HARD_CLARITY_FINAL_CHECKPOINT};
    ↪enabled={RC2JE_HARD_CLARITY_VETO_ENABLED}"
        return False, info

    if proto_candidate is None or (RC2JE_HARD_REQUIRE_PROTO_SAFE and not
    ↪rc2j_safe(proto_candidate)):
        info = dict(empty_info)
        info["reason"] = "no_safe_proto_candidate"
        return False, info

    proto_acc = rc2j_candidate_acc(proto_candidate)
    context_acc = rc2j_candidate_acc(context_candidate) if context_candidate is
    ↪not None else metrics.get("context_acc", np.nan)
    raw_best_score = rc2j_score(raw_best) if raw_best is not None else -1e9
    proto_score = rc2j_score(proto_candidate)

    proto_acc_margin_vs_context = proto_acc - context_acc if
    ↪rc2j_finite(proto_acc) and rc2j_finite(context_acc) else np.nan
    proto_score_margin_vs_raw_best = proto_score - raw_best_score if
    ↪rc2j_finite(proto_score) and rc2j_finite(raw_best_score) else np.nan

    proto_tri_min = metrics.get("proto_tri_min", np.nan)
    context_gain = metrics.get("context_gain_vs_proto", np.nan)
    gap_surrogate = metrics.get("gap_surrogate", np.nan)
    proto_top1 = metrics.get("proto_top1", np.nan)
    proto_entropy = metrics.get("proto_entropy", np.nan)
    hydro_context = metrics.get("hydro_context_turbulence", np.nan)

    proto_acc_strong = rc2j_finite(proto_acc) and proto_acc >=
    ↪RC2JE_HARD_PROTO_ACC_FLOOR
    proto_tri_healthy = rc2j_finite(proto_tri_min) and proto_tri_min >=
    ↪RC2JE_HARD_PROTO_TRI_MIN_FLOOR
    context_gain_veto_ok = rc2j_finite(context_gain) and context_gain <=
    ↪RC2JE_HARD_CONTEXT_GAIN_MAX
    proto_not_far_from_raw_best = (not
    ↪rc2j_finite(proto_score_margin_vs_raw_best)) or
    ↪proto_score_margin_vs_raw_best >= -RC2JE_HARD_PROTO_SCORE_TOL_VS_RAW
    proto_top1_hard_clear = rc2j_finite(proto_top1) and proto_top1 >=
    ↪RC2JE_HARD_PROTO_TOP1_FLOOR
    proto_entropy_hard_clear = rc2j_finite(proto_entropy) and proto_entropy <=
    ↪RC2JE_HARD_PROTO_ENTROPY_MAX
    gap_surrogate_hard_clear = rc2j_finite(gap_surrogate) and gap_surrogate <=
    ↪RC2JE_HARD_GAP_SURROGATE_MAX

    # Hydro can be unavailable in no-Hydro profiles; when missing, do not block
    ↪the selector.

```

```

hydro_context_hard_clear = (not rc2j_finite(hydro_context)) or
↳hydro_context <= RC2JE_HARD_HYDRO_TURBULENCE_MAX

hard_clarity_gates = [
    proto_acc_strong,
    proto_tri_healthy,
    context_gain_veto_ok,
    proto_not_far_from_raw_best,
    proto_top1_hard_clear,
    proto_entropy_hard_clear,
    gap_surrogate_hard_clear,
    hydro_context_hard_clear,
]
hard_clarity_signal_count = int(sum(bool(x) for x in hard_clarity_gates))
hard_clarity_all_gates_pass = bool(
    hard_clarity_signal_count >= RC2JE_HARD_REQUIRED_SIGNALS
    and proto_acc_strong
    and proto_tri_healthy
    and context_gain_veto_ok
    and proto_top1_hard_clear
    and proto_entropy_hard_clear
    and gap_surrogate_hard_clear
    and hydro_context_hard_clear
)

apply = bool(hard_clarity_all_gates_pass)

reasons = []
reasons.append(f"hard_clarity_signal_count={hard_clarity_signal_count}")
reasons.append(f"proto_acc={proto_acc}")
reasons.append(f"context_acc={context_acc}")
reasons.append(f"proto_acc_margin_vs_context={proto_acc_margin_vs_context}")
reasons.
↳append(f"proto_score_margin_vs_raw_best={proto_score_margin_vs_raw_best}")
reasons.append(f"proto_tri_min={proto_tri_min}")
reasons.append(f"context_gain={context_gain}")
reasons.append(f"gap_surrogate={gap_surrogate}")
reasons.append(f"proto_top1={proto_top1}")
reasons.append(f"proto_entropy={proto_entropy}")
reasons.append(f"hydro_context={hydro_context}")
reasons.append(f"proto_acc_strong={proto_acc_strong}")
reasons.append(f"proto_tri_healthy={proto_tri_healthy}")
reasons.append(f"context_gain_veto_ok={context_gain_veto_ok}")
reasons.append(f"proto_not_far_from_raw_best={proto_not_far_from_raw_best}")
reasons.append(f"proto_top1_hard_clear={proto_top1_hard_clear}")
reasons.append(f"proto_entropy_hard_clear={proto_entropy_hard_clear}")
reasons.append(f"gap_surrogate_hard_clear={gap_surrogate_hard_clear}")

```

```

reasons.append(f"hydro_context_hard_clear={hydro_context_hard_clear}")
reasons.append(f"hard_clarity_all_gates_pass={hard_clarity_all_gates_pass}")

return apply, {
    "applied": apply,
    "reason": "; ".join(reasons),
    "clean_signal_count": hard_clarity_signal_count,
    "hard_clarity_signal_count": hard_clarity_signal_count,
    "proto_acc_margin_vs_context": proto_acc_margin_vs_context,
    "proto_score_margin_vs_raw_best": proto_score_margin_vs_raw_best,
    "proto_tri_healthy": bool(proto_tri_healthy),
    "context_gain_veto_ok": bool(context_gain_veto_ok),
    "proto_top1_hard_clear": bool(proto_top1_hard_clear),
    "proto_entropy_hard_clear": bool(proto_entropy_hard_clear),
    "gap_surrogate_hard_clear": bool(gap_surrogate_hard_clear),
    "hydro_context_hard_clear": bool(hydro_context_hard_clear),
    "hard_clarity_all_gates_pass": bool(hard_clarity_all_gates_pass),
}

def rc2jf_true_guard_safe_veto(metrics, after_task, true_guard_candidate=None,
    ↪context_candidate=None, raw_best=None):
    """Return whether RC2JF should veto the final context-gap ambiguity lock.

    This selector-only patch targets clean/domain-shift regimes where the true
    global CE/KL guard is validation-superior to the guarded context-gap policy.
    It uses only policy-validation memory proxies and audit metrics. It never
    ↪uses
    final-test results.
    """
    try:
        after_task = int(after_task)
    except Exception:
        after_task = -1

    empty_info = {
        "applied": False,
        "reason": "not_evaluated",
        "true_guard_acc_margin_vs_context": np.nan,
        "true_guard_score_margin_vs_context": np.nan,
        "true_guard_class5_margin_vs_context": np.nan,
        "true_guard_minclass_margin_vs_context": np.nan,
        "true_guard_tri_min": np.nan,
        "true_guard_boundary_improved": False,
        "true_guard_raw_best": False,
        "true_guard_score_or_acc_superior": False,
        "true_guard_boundary_not_worse": False,
    }

```

```

        "true_guard_tri_ok": False,
        "dataset_agnostic_context_not_below": False,
        "weak_anchor_class": -1,
        "weak_boundary_margin_vs_context": np.nan,
        "weak_class_margin_vs_context": np.nan,
        "min_task_margin_vs_context": np.nan,
        "hydro_turbulence_delta_vs_context": np.nan,
        "hydro_output_delta_vs_context": np.nan,
        "weak_boundary_not_below_context": False,
        "weak_class_not_below_context": False,
        "min_task_not_below_context": False,
        "hydro_turbulence_not_worse_than_context": False,
        "hydro_output_not_worse_than_context": False,
    }

    if (not RC2JF_TRUE_GUARD_SAFE_VETO_ENABLED) or after_task !=␣
↪RC2JF_TRUE_GUARD_FINAL_CHECKPOINT:
        info = dict(empty_info)
        info["reason"] = f"after_task={after_task};␣
↪final_checkpoint={RC2JF_TRUE_GUARD_FINAL_CHECKPOINT};␣
↪enabled={RC2JF_TRUE_GUARD_SAFE_VETO_ENABLED}"
        return False, info

    if true_guard_candidate is None or not rc2j_safe(true_guard_candidate):
        info = dict(empty_info)
        info["reason"] = "no_safe_true_guard_candidate"
        return False, info

    true_guard_proxy = rc2j_proxy(true_guard_candidate)
    context_proxy = rc2j_proxy(context_candidate)

    tg_acc = rc2j_candidate_acc(true_guard_candidate)
    ctx_acc = rc2j_candidate_acc(context_candidate) if context_candidate is not␣
↪None else np.nan
    tg_score = rc2j_score(true_guard_candidate)
    ctx_score = rc2j_score(context_candidate) if context_candidate is not None␣
↪else np.nan

    tg_class5 = rc2j_value(true_guard_proxy, "proxy_class5", neutral=np.nan)
    ctx_class5 = rc2j_value(context_proxy, "proxy_class5", neutral=np.nan)
    tg_minclass = rc2j_value(true_guard_proxy, "proxy_min_class", neutral=np.
↪nan)
    ctx_minclass = rc2j_value(context_proxy, "proxy_min_class", neutral=np.nan)
    tg_tri_min = rc2j_tri_stats(true_guard_candidate, after_task).
↪get("tri_min", np.nan)

```



```

    acc_margin = tg_acc - ctx_acc if rc2j_finite(tg_acc) and
↪rc2j_finite(ctx_acc) else np.nan
    score_margin = tg_score - ctx_score if rc2j_finite(tg_score) and
↪rc2j_finite(ctx_score) else np.nan
    class5_margin = tg_class5 - ctx_class5 if rc2j_finite(tg_class5) and
↪rc2j_finite(ctx_class5) else np.nan
    minclass_margin = tg_minclass - ctx_minclass if rc2j_finite(tg_minclass)
↪and rc2j_finite(ctx_minclass) else np.nan

    raw_best_variant = rc2j_variant(raw_best)
    true_guard_raw_best = raw_best_variant in RC2J_TRUE_GUARD_VARIANTS
    score_or_acc_superior = (
        (rc2j_finite(acc_margin) and acc_margin >=
↪RC2JF_TRUE_GUARD_MIN_ACC_GAIN)
        or (rc2j_finite(score_margin) and score_margin >=
↪RC2JF_TRUE_GUARD_MIN_SCORE_GAIN)
    )
    boundary_improved = (
        (rc2j_finite(class5_margin) and class5_margin >=
↪RC2JF_TRUE_GUARD_MIN_CLASS5_GAIN)
        or (rc2j_finite(minclass_margin) and minclass_margin >=
↪RC2JF_TRUE_GUARD_MINCLASS_GAIN)
    )
    boundary_not_worse = (
        ((not rc2j_finite(class5_margin)) or class5_margin >=
↪RC2JF_TRUE_GUARD_BOUNDARY_NOT_WORSE_TOL)
        and ((not rc2j_finite(minclass_margin)) or minclass_margin >=
↪RC2JF_TRUE_GUARD_BOUNDARY_NOT_WORSE_TOL)
    )
    tri_ok = (not rc2j_finite(tg_tri_min)) or tg_tri_min >=
↪RC2JF_TRUE_GUARD_TRI_MIN_FLOOR

    hygiene_passed, hygiene_info = rc2jf_dataset_agnostic_not_below_context(
        true_guard_proxy, context_proxy, after_task=after_task
    )
    hygiene_required = bool(globals().
↪get("RC2JF_DATASET_AGNOSTIC_TRUE_GUARD_HYGIENE_ENABLED", True))

    apply = bool(
        (true_guard_raw_best or not RC2JF_TRUE_GUARD_RAW_BEST_REQUIRED)
        and score_or_acc_superior
        and boundary_not_worse
        and (boundary_improved or (rc2j_finite(score_margin) and score_margin
↪>= 0.05))
        and tri_ok
        and ((not hygiene_required) or hygiene_passed)

```

```

)

reasons = []
reasons.append(f"true_guard_raw_best={true_guard_raw_best}")
reasons.append(f"raw_best_variant={raw_best_variant}")
reasons.append(f"true_guard_acc={tg_acc}")
reasons.append(f"context_acc={ctx_acc}")
reasons.append(f"true_guard_acc_margin_vs_context={acc_margin}")
reasons.append(f"true_guard_score_margin_vs_context={score_margin}")
reasons.append(f"true_guard_class5_margin_vs_context={class5_margin}")
reasons.append(f"true_guard_minclass_margin_vs_context={minclass_margin}")
reasons.append(f"true_guard_tri_min={tg_tri_min}")
reasons.append(f"score_or_acc_superior={score_or_acc_superior}")
reasons.append(f"boundary_improved={boundary_improved}")
reasons.append(f"boundary_not_worse={boundary_not_worse}")
reasons.append(f"tri_ok={tri_ok}")
reasons.append(f"dataset_agnostic_context_not_below={hygiene_passed}")
reasons.append(f"weak_anchor_class={hygiene_info.get('weak_anchor_class')}")
reasons.append(f"weak_boundary_margin_vs_context={hygiene_info.
↳get('weak_boundary_margin_vs_context')}")
    reasons.append(f"weak_class_margin_vs_context={hygiene_info.
↳get('weak_class_margin_vs_context')}")
    reasons.append(f"min_task_margin_vs_context={hygiene_info.
↳get('min_task_margin_vs_context')}")
    reasons.append(f"hydro_turbulence_delta_vs_context={hygiene_info.
↳get('hydro_turbulence_delta_vs_context')}")
    reasons.append(f"hydro_output_delta_vs_context={hygiene_info.
↳get('hydro_output_delta_vs_context')}")
    reasons.append(f"hygiene_required={hygiene_required}")
    reasons.append(f"applied={apply}")

return apply, {
    "applied": apply,
    "reason": "; ".join(reasons),
    "true_guard_acc_margin_vs_context": acc_margin,
    "true_guard_score_margin_vs_context": score_margin,
    "true_guard_class5_margin_vs_context": class5_margin,
    "true_guard_minclass_margin_vs_context": minclass_margin,
    "true_guard_tri_min": tg_tri_min,
    "true_guard_boundary_improved": bool(boundary_improved),
    "true_guard_raw_best": bool(true_guard_raw_best),
    "true_guard_score_or_acc_superior": bool(score_or_acc_superior),
    "true_guard_boundary_not_worse": bool(boundary_not_worse),
    "true_guard_tri_ok": bool(tri_ok),
    **hygiene_info,
}

```

```

def rc2j_validation_score(proxy, baseline_proxy, variant: str, after_task=None):
    """Score candidates for audit ordering; final decision is made by
    ↪rc2j_select_candidate()."""
    try:
        base_score = rc2d_gate_aligned_score(proxy, baseline_proxy, variant,
        ↪after_task=after_task)
    except Exception:
        try:
            base_score = rc2f_nan_safe_validation_base(proxy, baseline_proxy)
        except Exception:
            base_score = validation_score(proxy, baseline_proxy)
    tri = rc2j_tri_stats_from_proxy(proxy, after_task)
    family = rc2b_policy_family(variant)
    score = float(base_score)
    score += 0.05 * (0.0 if not rc2j_finite(tri.get("tri_min", np.nan)) else
    ↪tri["tri_min"])
    score -= 0.02 * (0.0 if not rc2j_finite(tri.get("tri_gap", np.nan)) else
    ↪tri["tri_gap"])
    if variant in RC2J_CONTEXT_GAP_VARIANTS:
        score += 0.001
    if family == "context_only_global":
        score -= 0.005 # still audited; not the main deployable family in RC2J.
    if rc2j_finite(score):
        return float(score)
    return -1e9

# Override previous scoring only for RC2J. Other compatibility profiles keep
↪existing logic.
def rc2b_validation_score(proxy, baseline_proxy, variant: str, after_task=None):
    if globals().get("POLICY_BRANCH") == "RC2j":
        return rc2j_validation_score(proxy, baseline_proxy, variant,
        ↪after_task=after_task)
    if globals().get("POLICY_BRANCH") == "RC2i":
        try:
            return rc2d_gate_aligned_score(proxy, baseline_proxy, variant,
        ↪after_task=after_task)
        except Exception:
            return validation_score(proxy, baseline_proxy)
    if globals().get("POLICY_BRANCH") == "RC2h":
        try:
            return rc2h_validation_score(proxy, baseline_proxy, variant,
        ↪after_task=after_task)
        except Exception:
            return validation_score(proxy, baseline_proxy)

```

```

    if globals().get("POLICY_BRANCH") == "RC2b" or not_
↪RC2C_RISK_AWARE_SELECTOR_ENABLED:
        try:
            return rc2f_nan_safe_validation_base(proxy, baseline_proxy)
        except Exception:
            return validation_score(proxy, baseline_proxy)
    try:
        return rc2d_gate_aligned_score(proxy, baseline_proxy, variant,
↪after_task=after_task)
    except Exception:
        return validation_score(proxy, baseline_proxy)

def rc2c_select_candidate(candidates, after_task):
    """RC2J selector: classify regime, then choose the corresponding validated_
↪policy family."""
    if globals().get("POLICY_BRANCH") != "RC2j":
        scored = [c for c in candidates if rc2j_finite(c.get("selection_score",
↪-1e9))]
        selected = rc2j_best(scored or candidates)
        return selected, {
            "selection_reason": "compatibility_best_score_fallback",
            "raw_best_variant": rc2j_variant(selected),
            "raw_best_policy_family": rc2j_family(selected),
            "raw_best_proxy_score": rc2j_score(selected),
            "best_guarded_variant": "",
            "best_guarded_proxy_score": np.nan,
            "no_repair_veto_applied": False,
            "guarded_hysteresis_applied": False,
            "risk_floor_fallback_applied": False,
        }

    scored = [c for c in candidates if rc2j_finite(c.get("selection_score",
↪-1e9))]
    emergency_scoring_used = False
    if not scored and globals().get("RC2G_EMERGENCY_SCORE_IF_ALL_NAN", True):
        for c in candidates:
            try:
                c["selection_score"] = rc2f_emergency_candidate_score(c,
↪after_task=after_task)
            except Exception:
                c["selection_score"] = rc2j_candidate_acc(c)
                c["rc2j_emergency_score_used"] = True
        scored = [c for c in candidates if rc2j_finite(c.get("selection_score",
↪-1e9))]
    emergency_scoring_used = True

```

```

if not scored:
    scored = candidates

metrics = rc2j_regime_metrics(scored, after_task)
regime_label, regime_reason = rc2j_classify_regime(metrics, after_task)
raw_best = metrics["raw_best"] or rc2j_best(scored)
proto = metrics["proto"]
context = metrics["context"]
guarded = metrics["guarded"] or metrics["true_guard"] or context
true_guard = metrics["true_guard"]

selected = None
selection_reason = ""
fallback_used = False
final_lock_applied, final_lock_info = rc2jc_final_checkpoint_ambiguity_lock(
    metrics, after_task=after_task, context_candidate=context,
↪raw_best=raw_best, proto_candidate=proto
)
late_ambiguity_applied, late_ambiguity_info = rc2jb_late_ambiguity_trigger(
    metrics, after_task=after_task, context_candidate=context,
↪raw_best=raw_best
)
clean_veto_applied, clean_veto_info = rc2je_hard_clarity_clean_veto_guard(
    metrics, after_task=after_task, proto_candidate=proto,
↪context_candidate=context, raw_best=raw_best
)
true_guard_veto_applied, true_guard_veto_info = rc2jf_true_guard_safe_veto(
    metrics, after_task=after_task, true_guard_candidate=true_guard,
↪context_candidate=context, raw_best=raw_best
)

if clean_veto_applied and rc2j_safe(proto):
    selected = proto
    regime_label = "final_lock_vetoed_hard_clear_context"
    regime_reason = f"rc2je_clean_context_veto: {clean_veto_info.
↪get('reason', '')}"
    selection_reason =
↪"rc2je_hard_clarity_clean_veto_guard_select_proto_global_no_repair"
    elif true_guard_veto_applied and rc2j_safe(true_guard):
        selected = true_guard
        regime_label = "final_lock_vetoed_true_guard_safe"
        regime_reason = f"rc2jf_true_guard_safe_veto: {true_guard_veto_info.
↪get('reason', '')}"
        selection_reason =
↪"rc2jf_true_guard_safe_veto_select_proto_global_head_ce_kl_guard_035"
        elif final_lock_applied and rc2j_safe(context):

```

```

        selected = context
        regime_label = "final_locked_ambiguous_context"
        regime_reason = f"rc2jc_final_lock: {final_lock_info.get('reason', '')}"
        selection_reason = ""
    ↪ "rc2jc_final_checkpoint_ambiguity_lock_select_guarded_context_gap"
        elif late_ambiguity_applied and rc2j_safe(context):
            selected = context
            # Keep the original regime label for audit, but expose the late_
    ↪ override explicitly.
            if regime_label != "ambiguous_context":
                regime_label = "late_ambiguous_context"
                regime_reason = f"rc2jb_late_hysteresis: {late_ambiguity_info.
    ↪ get('reason', '')}"
                selection_reason = ""
    ↪ "rc2jb_late_ambiguity_hysteresis_select_guarded_context_gap"
            elif regime_label == "clean_context" and rc2j_safe(proto):
                selected = proto
                selection_reason = "rc2j_clean_context_select_proto_global_no_repair"
            elif regime_label == "ambiguous_context" and rc2j_safe(context):
                selected = context
                selection_reason = "rc2j_ambiguous_context_select_guarded_context_gap"
            else:
                # Mixed regime: prefer the best guarded anchor if it is close to_
    ↪ raw-best; otherwise
                # keep raw-best only if it is safe and not a fragile context-only_
    ↪ candidate.
                fallback_used = True
                if guarded is not None and rc2j_safe(guarded):
                    if raw_best is None or rc2j_score(guarded) >= rc2j_score(raw_best)_
    ↪ RC2J_MIXED_REGIME_GUARD_MARGIN:
                    selected = guarded
                    selection_reason = ""
    ↪ "rc2j_mixed_regime_conservative_guarded_fallback"
                    if selected is None and raw_best is not None and rc2j_safe(raw_best)_
    ↪ and rc2b_policy_family(rc2j_variant(raw_best)) != "context_only_global":
                        selected = raw_best
                        selection_reason = "rc2j_mixed_regime_safe_raw_best"
                    if selected is None and rc2j_safe(true_guard):
                        selected = true_guard
                        selection_reason = "rc2j_mixed_regime_true_guard_fallback"
                    if selected is None:
                        selected = rc2j_best(scored)
                        selection_reason = "rc2j_emergency_best_available"

best_guarded = guarded or context or true_guard
return selected, {

```

```

    "raw_best_variant": rc2j_variant(raw_best),
    "raw_best_policy_family": rc2j_family(raw_best),
    "raw_best_proxy_score": rc2j_score(raw_best),
    "best_guarded_variant": rc2j_variant(best_guarded),
    "best_guarded_proxy_score": rc2j_score(best_guarded),
    "selection_reason": selection_reason,
    "no_repair_veto_applied": False,
    "guarded_hysteresis_applied": bool(regime_label in
↪["ambiguous_context", "mixed_or_uncertain"]),
    "risk_floor_fallback_applied": bool(fallback_used),
    "true_guard_rescue_applied": bool(rc2j_variant(selected) in
↪RC2J_TRUE_GUARD_VARIANTS),
    "class2_rescue_applied": False,
    "class4_rescue_applied": False,
    "class5_rescue_applied": False,
    "tri_rescue_applied": bool(regime_label == "ambiguous_context" and
↪rc2j_finite(metrics.get("context_tri_gain_vs_proto", np.nan)) and
↪metrics["context_tri_gain_vs_proto"] > 0),
    "zero_seen_class_reject_applied": False,
    "rc2j_selector_version": RC2J_SELECTOR_VERSION,
    "rc2j_regime_label": regime_label,
    "rc2j_regime_reason": regime_reason,
    "rc2j_selected_policy_family": rc2j_family(selected),
    "rc2j_selected_variant": rc2j_variant(selected),
    "rc2j_proto_variant": rc2j_variant(proto),
    "rc2j_context_variant": rc2j_variant(context),
    "rc2j_true_guard_variant": rc2j_variant(true_guard),
    "rc2j_raw_best_variant": rc2j_variant(raw_best),
    "rc2j_proto_acc": metrics.get("proto_acc", np.nan),
    "rc2j_context_acc": metrics.get("context_acc", np.nan),
    "rc2j_guarded_acc": metrics.get("guarded_acc", np.nan),
    "rc2j_context_gain_vs_proto": metrics.get("context_gain_vs_proto", np.
↪nan),
    "rc2j_guarded_gain_vs_proto": metrics.get("guarded_gain_vs_proto", np.
↪nan),
    "rc2j_context_tri_gain_vs_proto": metrics.
↪get("context_tri_gain_vs_proto", np.nan),
    "rc2j_proto_context_top1": metrics.get("proto_top1", np.nan),
    "rc2j_context_top1": metrics.get("context_top1", np.nan),
    "rc2j_proto_context_entropy": metrics.get("proto_entropy", np.nan),
    "rc2j_context_entropy": metrics.get("context_entropy", np.nan),
    "rc2j_proto_context_margin": metrics.get("proto_margin", np.nan),
    "rc2j_context_margin": metrics.get("context_margin", np.nan),
    "rc2j_proto_tri_min": metrics.get("proto_tri_min", np.nan),
    "rc2j_context_tri_min": metrics.get("context_tri_min", np.nan),
    "rc2j_guarded_tri_min": metrics.get("guarded_tri_min", np.nan),

```



```

        "rc2j_proto_tri_gap": metrics.get("proto_tri_gap", np.nan),
        "rc2j_context_tri_gap": metrics.get("context_tri_gap", np.nan),
        "rc2j_hydro_context_turbulence": metrics.
↪get("hydro_context_turbulence", np.nan),
        "rc2j_hydro_proto_turbulence": metrics.get("hydro_proto_turbulence", np.
↪nan),
        "rc2j_hydro_guarded_turbulence": metrics.
↪get("hydro_guarded_turbulence", np.nan),
        "rc2j_gap_surrogate": metrics.get("gap_surrogate", np.nan),
        "rc2j_emergency_scoring_used": bool(emergency_scoring_used),
        "rc2jb_late_ambiguity_hysteresis_applied": bool(late_ambiguity_info.
↪get("applied", False)),
        "rc2jb_late_ambiguity_reason": late_ambiguity_info.get("reason", ""),
        "rc2jb_late_uncertainty_signals": late_ambiguity_info.
↪get("late_uncertainty_signals", 0),
        "rc2jb_context_score_margin_vs_raw_best": late_ambiguity_info.
↪get("context_score_margin_vs_raw_best", np.nan),
        "rc2jb_context_acc_margin_vs_proto": late_ambiguity_info.
↪get("context_acc_margin_vs_proto", np.nan),
        "rc2jc_final_ambiguity_lock_applied": bool(final_lock_info.
↪get("applied", False)),
        "rc2jc_final_ambiguity_reason": final_lock_info.get("reason", ""),
        "rc2jc_final_uncertainty_signals": final_lock_info.
↪get("final_uncertainty_signals", 0),
        "rc2jc_context_acc_margin_vs_raw_best": final_lock_info.
↪get("context_acc_margin_vs_raw_best", np.nan),
        "rc2jc_context_acc_margin_vs_proto": final_lock_info.
↪get("context_acc_margin_vs_proto", np.nan),
        "rc2jc_context_tri_warning": bool(final_lock_info.
↪get("context_tri_warning", False)),
        "rc2jc_context_tri_hard_fail": bool(final_lock_info.
↪get("context_tri_hard_fail", False)),
        "rc2je_hard_clarity_clean_veto_guard_applied": bool(clean_veto_info.
↪get("applied", False)),
        "rc2je_clean_context_veto_reason": clean_veto_info.get("reason", ""),
        "rc2je_clean_signal_count": clean_veto_info.get("clean_signal_count",
↪0),
        "rc2je_proto_acc_margin_vs_context": clean_veto_info.
↪get("proto_acc_margin_vs_context", np.nan),
        "rc2je_proto_score_margin_vs_raw_best": clean_veto_info.
↪get("proto_score_margin_vs_raw_best", np.nan),
        "rc2je_proto_tri_healthy": bool(clean_veto_info.
↪get("proto_tri_healthy", False)),
        "rc2je_context_gain_veto_ok": bool(clean_veto_info.
↪get("context_gain_veto_ok", False)),

```



```

        "rc2je_hard_clarity_clean_veto_applied": bool(clean_veto_info.
↪get("applied", False)),
        "rc2je_hard_clarity_veto_reason": clean_veto_info.get("reason", ""),
        "rc2je_hard_clarity_signal_count": clean_veto_info.
↪get("hard_clarity_signal_count", clean_veto_info.get("clean_signal_count", 0)),
        "rc2je_proto_top1_hard_clear": bool(clean_veto_info.
↪get("proto_top1_hard_clear", False)),
        "rc2je_proto_entropy_hard_clear": bool(clean_veto_info.
↪get("proto_entropy_hard_clear", False)),
        "rc2je_gap_surrogate_hard_clear": bool(clean_veto_info.
↪get("gap_surrogate_hard_clear", False)),
        "rc2je_hydro_context_hard_clear": bool(clean_veto_info.
↪get("hydro_context_hard_clear", False)),
        "rc2je_hard_clarity_all_gates_pass": bool(clean_veto_info.
↪get("hard_clarity_all_gates_pass", False)),
        "rc2jf_true_guard_safe_veto_applied": bool(true_guard_veto_info.
↪get("applied", False)),
        "rc2jf_true_guard_safe_veto_reason": true_guard_veto_info.get("reason", ""),
        "rc2jf_true_guard_acc_margin_vs_context": true_guard_veto_info.
↪get("true_guard_acc_margin_vs_context", np.nan),
        "rc2jf_true_guard_score_margin_vs_context": true_guard_veto_info.
↪get("true_guard_score_margin_vs_context", np.nan),
        "rc2jf_true_guard_class5_margin_vs_context": true_guard_veto_info.
↪get("true_guard_class5_margin_vs_context", np.nan),
        "rc2jf_true_guard_minclass_margin_vs_context": true_guard_veto_info.
↪get("true_guard_minclass_margin_vs_context", np.nan),
        "rc2jf_true_guard_tri_min": true_guard_veto_info.
↪get("true_guard_tri_min", np.nan),
        "rc2jf_true_guard_boundary_improved": bool(true_guard_veto_info.
↪get("true_guard_boundary_improved", False)),
        "rc2jf_true_guard_raw_best": bool(true_guard_veto_info.
↪get("true_guard_raw_best", False)),
        "rc2jf_true_guard_score_or_acc_superior": bool(true_guard_veto_info.
↪get("true_guard_score_or_acc_superior", False)),
        "rc2jf_true_guard_boundary_not_worse": bool(true_guard_veto_info.
↪get("true_guard_boundary_not_worse", False)),
        "rc2jf_true_guard_tri_ok": bool(true_guard_veto_info.
↪get("true_guard_tri_ok", False)),
        "rc2jf_dataset_agnostic_context_not_below": bool(true_guard_veto_info.
↪get("dataset_agnostic_context_not_below", False)),
        "rc2jf_weak_anchor_class": true_guard_veto_info.
↪get("weak_anchor_class", -1),

```

```

        "rc2jf_weak_boundary_margin_vs_context": true_guard_veto_info.
↪get("weak_boundary_margin_vs_context", np.nan),
        "rc2jf_weak_class_margin_vs_context": true_guard_veto_info.
↪get("weak_class_margin_vs_context", np.nan),
        "rc2jf_min_task_margin_vs_context": true_guard_veto_info.
↪get("min_task_margin_vs_context", np.nan),
        "rc2jf_hydro_turbulence_delta_vs_context": true_guard_veto_info.
↪get("hydro_turbulence_delta_vs_context", np.nan),
        "rc2jf_hydro_output_delta_vs_context": true_guard_veto_info.
↪get("hydro_output_delta_vs_context", np.nan),
        "rc2jf_weak_boundary_not_below_context": bool(true_guard_veto_info.
↪get("weak_boundary_not_below_context", False)),
        "rc2jf_weak_class_not_below_context": bool(true_guard_veto_info.
↪get("weak_class_not_below_context", False)),
        "rc2jf_min_task_not_below_context": bool(true_guard_veto_info.
↪get("min_task_not_below_context", False)),
        "rc2jf_hydro_turbulence_not_worse_than_context":
↪bool(true_guard_veto_info.get("hydro_turbulence_not_worse_than_context",
↪False)),
        "rc2jf_hydro_output_not_worse_than_context": bool(true_guard_veto_info.
↪get("hydro_output_not_worse_than_context", False)),
        # RC2I dual-anchor compatibility audit columns.
        # RC2JF does not use the RC2I override, but keeping these neutral fields
        # preserves longitudinal audit-schema continuity across RC2I -> RC2J ->
↪RC2JE -> RC2JF.
        "rc2i_safe_anchor_variant": "not_active_rc2jf",
        "rc2i_override_rejected_reason": "not_active_rc2jf",
        "rc2i_context_only_override_rejected": False,
    }

print("RC2JF true-guard safe-veto selector installed:", RC2J_SELECTOR_VERSION)
print("RC2J clean thresholds:", RC2J_CLEAN_CONTEXT_TOP1_FLOOR,
↪RC2J_CLEAN_CONTEXT_ENTROPY_MAX, RC2J_CLEAN_PROTO_TRI_MIN_FLOOR)
print("RC2J ambiguous thresholds:", RC2J_AMBIGUOUS_CONTEXT_GAIN_MIN,
↪RC2J_CONTEXT_TRI_GAIN_MIN, RC2J_HIGH_HYDRO_TURBULENCE)
print("RC2Jc final lock / RC2Jb late thresholds:", {
    "final_lock_checkpoint": RC2JC_FINAL_LOCK_CHECKPOINT,
    "final_proto_top1_max": RC2JC_FINAL_PROTO_TOP1_MAX,
    "final_proto_entropy_min": RC2JC_FINAL_PROTO_ENTROPY_MIN,
    "final_gap_surrogate_min": RC2JC_FINAL_GAP_SURROGATE_MIN,
    "final_context_acc_tol_vs_raw": RC2JC_FINAL_CONTEXT_ACC_TOL_VS_RAW,
    "final_context_tri_warning_floor": RC2JC_FINAL_CONTEXT_TRI_WARNING_FLOOR,
    "final_context_tri_hard_floor": RC2JC_FINAL_CONTEXT_TRI_HARD_FLOOR,
    "late_task_start": RC2JB_LATE_TASK_START,
    "proto_top1_max": RC2JB_LATE_PROTO_TOP1_MAX,
    "proto_entropy_min": RC2JB_LATE_PROTO_ENTROPY_MIN,

```

```

    "gap_surrogate_min": RC2JB_LATE_GAP_SURROGATE_MIN,
    "context_acc_tol": RC2JB_LATE_CONTEXT_ACC_TOL,
    "context_score_tol": RC2JB_LATE_CONTEXT_SCORE_TOL,
    "context_tri_min_floor": RC2JB_LATE_CONTEXT_TRI_MIN_FLOOR,
})
print("RC2Je hard-clarity clean-veto guard thresholds:", {
    "hard_clarity_veto_enabled": RC2JE_HARD_CLARITY_VETO_ENABLED,
    "hard_clarity_final_checkpoint": RC2JE_HARD_CLARITY_FINAL_CHECKPOINT,
    "hard_proto_acc_floor": RC2JE_HARD_PROTO_ACC_FLOOR,
    "hard_proto_tri_min_floor": RC2JE_HARD_PROTO_TRI_MIN_FLOOR,
    "hard_context_gain_max": RC2JE_HARD_CONTEXT_GAIN_MAX,
    "hard_proto_score_tol_vs_raw": RC2JE_HARD_PROTO_SCORE_TOL_VS_RAW,
    "hard_gap_surrogate_max": RC2JE_HARD_GAP_SURROGATE_MAX,
    "hard_proto_top1_floor": RC2JE_HARD_PROTO_TOP1_FLOOR,
    "hard_proto_entropy_max": RC2JE_HARD_PROTO_ENTROPY_MAX,
    "hard_hydro_turbulence_max": RC2JE_HARD_HYDRO_TURBULENCE_MAX,
    "hard_required_signals": RC2JE_HARD_REQUIRED_SIGNALS,
})
print("RC2JF policy families:", {"clean": RC2J_PROTO_VARIANTS, "ambiguous": RC2J_CONTEXT_GAP_VARIANTS, "true_guard": RC2J_TRUE_GUARD_VARIANTS, "fallback": RC2J_GUARDED_FALLBACK_VARIANTS})
print("RC2JF/RC2JG true-guard safe-veto thresholds:", {"enabled": RC2JF_TRUE_GUARD_SAFE_VETO_ENABLED, "final_checkpoint": RC2JF_TRUE_GUARD_FINAL_CHECKPOINT, "min_acc_gain": RC2JF_TRUE_GUARD_MIN_ACC_GAIN, "min_score_gain": RC2JF_TRUE_GUARD_MIN_SCORE_GAIN, "min_class5_gain": RC2JF_TRUE_GUARD_MIN_CLASS5_GAIN, "minclass_gain": RC2JF_TRUE_GUARD_MINCLASS_GAIN, "tri_min_floor": RC2JF_TRUE_GUARD_TRI_MIN_FLOOR, "raw_best_required": RC2JF_TRUE_GUARD_RAW_BEST_REQUIRED, "dataset_agnostic_hygiene_enabled": RC2JF_DATASET_AGNOSTIC_TRUE_GUARD_HYGIENE_ENABLED, "weak_boundary_not_below_tol": RC2JF_CONTEXT_NOT_BELOW_TOL, "weak_class_not_below_tol": RC2JF_CLASS_NOT_BELOW_TOL, "min_task_not_below_tol": RC2JF_MIN_TASK_NOT_BELOW_TOL, "hydro_turbulence_not_worse_tol": RC2JF_HYDRO_TURBULENCE_NOT_WORSE_TOL, "hydro_output_not_worse_tol": RC2JF_HYDRO_OUTPUT_DRIFT_NOT_WORSE_TOL})

```

RC2JF true-guard safe-veto selector installed: RC2jf_true_guard_safe_veto_v1

RC2J clean thresholds: 0.98 0.08 0.82

RC2J ambiguous thresholds: 0.01 0.015 0.12

RC2Jc final lock / RC2Jb late thresholds: {'final_lock_checkpoint': 4, 'final_proto_top1_max': 0.85, 'final_proto_entropy_min': 0.12, 'final_gap_surrogate_min': 0.25, 'final_context_acc_tol_vs_raw': 0.04, 'final_context_tri_warning_floor': 0.75, 'final_context_tri_hard_floor': 0.7, 'late_task_start': 3, 'proto_top1_max': 0.9, 'proto_entropy_min': 0.1, 'gap_surrogate_min': 0.2, 'context_acc_tol': 0.03, 'context_score_tol': 0.06, 'context_tri_min_floor': 0.7}

```

RC2Je hard-clarity clean-veto guard thresholds: {'hard_clarity_veto_enabled':
True, 'hard_clarity_final_checkpoint': 4, 'hard_proto_acc_floor': 0.92,
'hard_proto_tri_min_floor': 0.88, 'hard_context_gain_max': 0.012,
'hard_proto_score_tol_vs_raw': 0.06, 'hard_gap_surrogate_max': 0.35,
'hard_proto_top1_floor': 0.85, 'hard_proto_entropy_max': 0.12,
'hard_hydro_turbulence_max': 0.12, 'hard_required_signals': 7}
RC2JF policy families: {'clean': ['proto_global_no_repair'], 'ambiguous':
['context_gap_selected', 'context_temp_blend_selected_head_guard_035'],
'true_guard': ['proto_global_head_ce_kl_guard_035'], 'fallback':
['context_gap_selected', 'context_temp_blend_selected_head_guard_035',
'proto_global_head_ce_kl_guard_035']}
RC2JF/RC2JG true-guard safe-veto thresholds: {'enabled': True,
'final_checkpoint': 4, 'min_acc_gain': 0.005, 'min_score_gain': 0.003,
'min_class5_gain': 0.01, 'minclass_gain': 0.01, 'tri_min_floor': 0.65,
'raw_best_required': True, 'dataset_agnostic_hygiene_enabled': True,
'weak_boundary_not_below_tol': 0.002, 'weak_class_not_below_tol': 0.003,
'min_task_not_below_tol': 0.003, 'hydro_turbulence_not_worse_tol': 0.015,
'hydro_output_not_worse_tol': 0.025}

```

2.26 RC2M-beta selector patch

The following cell installs the RC2M-beta selector into the inherited RC2JH/RC2JG/RC2M-alpha v3 harness.

It does two things before final-test evaluation:

1. Attaches candidate-level, validation-memory boundary metrics to every candidate.
2. Selects using dynamic weak-boundary evidence, then applies an ambiguity lock that protects the guarded context-gap family when validation-memory signals still indicate a foggy/ambiguous regime.

No final-test data is used for candidate construction, dynamic boundary discovery, ambiguity detection, or selector decision.

2.26.1 RC2M-beta note — from fixed sentinel classes to dynamic boundaries

RC2M-alpha used the current MNIST-family sentinel features `weak_boundary_eer_2_4_5`, `weak_boundary_auc_2_4_5`, and `class2_eer`.

RC2M-beta replaces them with dynamic features computed on policy-validation memory:

- `weak_boundary_classes_dynamic`
- `weak_boundary_eer_dynamic`
- `weak_boundary_auc_dynamic`
- `worst_class_eer`
- `worst_class_auc`
- `topk_boundary_eer`
- `topk_boundary_auc`

This makes the selector structurally ready for future datasets where class IDs 2/4/5 have no special meaning.

```

[69]: # -----
# v1.1-RC2JG - Generic weak-boundary latent separability calibration
# -----
# Candidate-only, no-leakage patch.
# The candidate detects weak class boundaries on repair/policy-validation,
↳memory only.
# It never uses final-test labels, RotatedMNIST task identity, class-5,
↳identity, or final-rotation diagnostics.
# -----

RC2JG_SELECTOR_VERSION = "rc2jg_generic_weak_boundary_latent_calibration_v1"
RC2JG_VARIANTS = [globals().get("RC2JG_VARIANT",
↳"generic_weak_boundary_latent_calibrated")]

_rc2jg_prev_policy_family = rc2b_policy_family
_rc2jg_prev_policy_label = rc2b_policy_label
_rc2jg_prev_policy_complexity_rank = rc2b_policy_complexity_rank
_rc2jg_prev_context_variant_selection_mode = context_variant_selection_mode
_rc2jg_prev_rc2c_select_candidate = rc2c_select_candidate

def rc2b_policy_family(variant: str) -> str:
    if str(variant) in RC2JG_VARIANTS:
        return "generic_latent_boundary_calibration"
    return _rc2jg_prev_policy_family(variant)

def rc2b_policy_label(variant: str) -> str:
    if str(variant) in RC2JG_VARIANTS:
        return "RC2JG generic weak-boundary latent calibration"
    return _rc2jg_prev_policy_label(variant)

def rc2b_policy_complexity_rank(variant: str) -> int:
    if str(variant) in RC2JG_VARIANTS:
        return 4
    return _rc2jg_prev_policy_complexity_rank(variant)

def context_variant_selection_mode(variant: str):
    if str(variant) in RC2JG_VARIANTS:
        return "temp_blend"
    return _rc2jg_prev_context_variant_selection_mode(variant)

class WeakBoundaryLatentCalibrationReadout(nn.Module):
    """Small residual latent adapter + copied global head, initialized as no-op.
    ↳"""
    def __init__(self, base_head, latent_dim=LATENT_DIM, adapter_scale=None):
        super().__init__()
        self.linear = copy.deepcopy(base_head)
        self.adapter = nn.Linear(latent_dim, latent_dim, bias=False)

```

```

        nn.init.zeros_(self.adapter.weight)
        scale = float(adapter_scale if adapter_scale is not None else globals().
↪get("RC2JG_ADAPTER_SCALE", 0.25))
        self.adapter_scale = nn.Parameter(torch.tensor(scale))
    def transform_zf(self, zf):
        return zf + self.adapter_scale.clamp(0.0, 1.0) * torch.tanh(self.
↪adapter(zf))
    def forward(self, zf, q_context=None):
        return self.linear(self.transform_zf(zf))

@torch.no_grad()
def rc2jg_boundary_diagnostic_on_memory(model, readout, class_memory, config,
↪prototypes=None,
                                ↪
↪max_items_per_class=VALIDATION_MAX_ITEMS_PER_CLASS):
    empty = {"rc2jg_memory_items": 0, "rc2jg_seen_classes": "[]",
↪"rc2jg_weak_class": -1,
            "rc2jg_competing_class": -1, "rc2jg_weak_class_acc": np.nan,
            "rc2jg_competing_class_acc": np.nan, "rc2jg_min_class_acc": np.nan,
            "rc2jg_global_acc": np.nan, "rc2jg_boundary_confusion_rate": np.
↪nan,
            "rc2jg_boundary_logit_margin": np.nan, "rc2jg_latent_separation":
↪np.nan,
            "rc2jg_context_top1": np.nan, "rc2jg_context_entropy": np.nan,
            "rc2jg_context_margin": np.nan, "rc2jg_context_healthy": False,
            "rc2jg_latent_weak_boundary_detected": False,
↪"rc2jg_activation_candidate": False}
    if not class_memory:
        return dict(empty)
    correct_by_cls = {c: 0 for c in range(N_CLASSES)}
    total_by_cls = {c: 0 for c in range(N_CLASSES)}
    pred_counts = {c: {k: 0 for k in range(N_CLASSES)} for c in
↪range(N_CLASSES)}
    margin_values = {c: [] for c in range(N_CLASSES)}
    z_by_cls = {c: [] for c in range(N_CLASSES)}
    ctx_top_values, ctx_entropy_values, ctx_margin_values = [], [], []
    correct_total, total = 0, 0
    model.eval(); readout.eval()
    for cls, data in sorted(class_memory.items()):
        X, y, t = data["X"], data["y"], data["task_id"]
        if len(y) == 0:
            continue
        if len(y) > max_items_per_class:
            idx = torch.randperm(len(y))[:max_items_per_class]
            X, y, t = X[idx], y[idx], t[idx]

```

```

        loader = make_loader(TensorDataset(X, y, t), batch_size=BATCH_SIZE,
↪shuffle=False)
        for xb, yb, tb in loader:
            xb, yb, tb = xb.to(DEVICE), yb.to(DEVICE), tb.to(DEVICE)
            for src_task_id in torch.unique(tb).detach().cpu().tolist():
                src_task_id = int(src_task_id)
                mask = (tb == src_task_id)
                if not mask.any():
                    continue
                _, cache = forward_by_method(model, xb[mask], src_task_id,
↪config,
                                                    prototypes=prototypes,
↪train=False, y=yb[mask], return_cache=True)
                zf = cache["zf"]
                logits = readout(zf, cache.get("q_context"))
                logits_eval = mask_logits_to_seen_classes(logits,
↪seen_classes_until(src_task_id))
                pred = logits_eval.argmax(dim=-1)
                yy = yb[mask]
                q = cache.get("q_context")
                if q is not None:
                    ctx_top_values.append((q.argmax(dim=-1) == src_task_id).
↪float().detach().cpu())
                    ctx_entropy_values.append(entropy_from_probs(q).detach().
↪cpu())
                    ctx_margin_values.append(margin_from_probs(q).detach().
↪cpu())
                correct = (pred == yy)
                correct_total += int(correct.sum().item()); total += int(yy.
↪numel())
                for c in torch.unique(yy).detach().cpu().tolist():
                    c = int(c); m = (yy == c)
                    total_by_cls[c] += int(m.sum().item())
                    correct_by_cls[c] += int((pred[m] == yy[m]).sum().item())
                    for pcls in pred[m].detach().cpu().tolist():
                        pred_counts[c][int(pcls)] = pred_counts[c].
↪get(int(pcls), 0) + 1
                    z_by_cls[c].append(zf[m].detach().cpu())
                    sample_logits = logits_eval[m].detach()
                    if sample_logits.numel() > 0:
                        true_logits = sample_logits[:, c]
                        competitor_logits = sample_logits.clone();
↪competitor_logits[:, c] = -1e9
                        strongest_other = competitor_logits.max(dim=-1).values
                        margin_values[c].extend((true_logits - strongest_other).
↪detach().cpu().tolist())

```

```

per_cls = {c: (correct_by_cls[c] / max(total_by_cls[c], 1) if
↳total_by_cls[c] > 0 else np.nan) for c in range(N_CLASSES)}
seen_classes = [c for c in range(N_CLASSES) if total_by_cls[c] > 0]
if not seen_classes:
    return dict(empty)
weak_class = int(min(seen_classes, key=lambda c: per_cls.get(c, np.inf) if
↳np.isfinite(per_cls.get(c, np.nan)) else np.inf))
wrong_counts = {k: v for k, v in pred_counts[weak_class].items() if k !=
↳weak_class and v > 0}
if wrong_counts:
    competing_class = int(max(wrong_counts, key=wrong_counts.get))
    boundary_confusion_rate = float(wrong_counts[competing_class] /
↳max(total_by_cls[weak_class], 1))
else:
    competing_class = -1; boundary_confusion_rate = 0.0
    if z_by_cls[weak_class]:
        zw = torch.cat(z_by_cls[weak_class], dim=0).float(); mw = zw.
↳mean(dim=0)
        best_dist = float("inf")
        for c in seen_classes:
            if c == weak_class or not z_by_cls[c]:
                continue
            zc = torch.cat(z_by_cls[c], dim=0).float()
            dist = float(torch.norm(mw - zc.mean(dim=0)).item())
            if dist < best_dist:
                best_dist = dist; competing_class = int(c)
    if competing_class < 0 and len(seen_classes) >= 2:
        competing_class = int([c for c in seen_classes if c != weak_class][0])
    latent_sep = np.nan
    if competing_class >= 0 and z_by_cls.get(weak_class) and z_by_cls.
↳get(competing_class):
        zw = torch.cat(z_by_cls[weak_class], dim=0).float(); zc = torch.
↳cat(z_by_cls[competing_class], dim=0).float()
        mw, mc = zw.mean(dim=0), zc.mean(dim=0)
        pooled = 0.5 * (zw.var(dim=0, unbiased=False).mean() + zc.var(dim=0,
↳unbiased=False).mean())
        latent_sep = float(torch.norm(mw - mc).item() / math.sqrt(float(pooled.
↳item()) + 1e-8))
    ctx_top = float(torch.cat(ctx_top_values).mean()) if ctx_top_values else np.
↳nan
    ctx_entropy = float(torch.cat(ctx_entropy_values).mean()) if
↳ctx_entropy_values else np.nan
    ctx_margin = float(torch.cat(ctx_margin_values).mean()) if
↳ctx_margin_values else np.nan
    min_class_acc = float(np.nanmin([per_cls[c] for c in seen_classes]))
    weak_acc = float(per_cls.get(weak_class, np.nan))

```



```

        competing_acc = float(per_cls.get(competing_class, np.nan)) if
↪competing_class >= 0 else np.nan
        boundary_margin = float(np.nanmean(margin_values[weak_class])) if
↪margin_values[weak_class] else np.nan
        context_healthy = bool((not np.isfinite(ctx_top) or ctx_top >= globals().
↪get("RC2JG_CONTEXT_TOP1_FLOOR", 0.70))
                                and (not np.isfinite(ctx_entropy) or ctx_entropy <=
↪globals().get("RC2JG_CONTEXT_ENTROPY_CEILING", 1.20))
                                and (not np.isfinite(ctx_margin) or ctx_margin >=
↪globals().get("RC2JG_CONTEXT_MARGIN_FLOOR", 0.03)))
        latent_weak = bool((np.isfinite(weak_acc) and weak_acc < globals().
↪get("RC2JG_WEAK_CLASS_ACC_FLOOR", 0.86))
                                or (np.isfinite(boundary_margin) and boundary_margin <
↪globals().get("RC2JG_MIN_BOUNDARY_MARGIN", 0.20))
                                or (np.isfinite(latent_sep) and latent_sep < globals().
↪get("RC2JG_LATENT_SEPARATION_FLOOR", 0.75)))
        return {"rc2jg_memory_items": int(total), "rc2jg_seen_classes":
↪str(seen_classes),
                "rc2jg_weak_class": int(weak_class), "rc2jg_competing_class":
↪int(competing_class),
                "rc2jg_weak_class_acc": weak_acc, "rc2jg_competing_class_acc":
↪competing_acc,
                "rc2jg_min_class_acc": min_class_acc, "rc2jg_global_acc":
↪float(correct_total / max(total, 1)),
                "rc2jg_boundary_confusion_rate": boundary_confusion_rate,
↪"rc2jg_boundary_logit_margin": boundary_margin,
                "rc2jg_latent_separation": latent_sep, "rc2jg_context_top1":
↪ctx_top,
                "rc2jg_context_entropy": ctx_entropy, "rc2jg_context_margin":
↪ctx_margin,
                "rc2jg_context_healthy": context_healthy,
↪"rc2jg_latent_weak_boundary_detected": latent_weak,
                "rc2jg_activation_candidate": bool(context_healthy and latent_weak
↪and competing_class >= 0)}

def rc2jg_boundary_margin_loss(logits, y, weak_pair, target_margin=None):
    target_margin = float(target_margin if target_margin is not None else
↪globals().get("RC2JG_MIN_BOUNDARY_MARGIN", 0.20))
    if weak_pair is None:
        return logits.sum() * 0.0
    a, b = int(weak_pair[0]), int(weak_pair[1]); losses = []
    if 0 <= a < logits.shape[1] and 0 <= b < logits.shape[1]:
        ma = (y == a); mb = (y == b)
        if ma.any(): losses.append(F.relu(target_margin - (logits[ma, a] -
↪logits[ma, b])).mean())

```

```

        if mb.any(): losses.append(F.relu(target_margin - (logits[mb, b] -
↳logits[mb, a])).mean())
        return torch.stack(losses).mean() if losses else logits.sum() * 0.0

def rc2jg_latent_separation_loss(z_cal, y, weak_pair, target_sep=None):
    target_sep = float(target_sep if target_sep is not None else globals().
↳get("RC2JG_LATENT_SEPARATION_FLOOR", 0.75))
    if weak_pair is None:
        return z_cal.sum() * 0.0
    a, b = int(weak_pair[0]), int(weak_pair[1]); ma, mb = (y == a), (y == b)
    if not ma.any() or not mb.any():
        return z_cal.sum() * 0.0
    ca = F.normalize(z_cal[ma].mean(dim=0, keepdim=True), dim=-1)
    cb = F.normalize(z_cal[mb].mean(dim=0, keepdim=True), dim=-1)
    return F.relu(target_sep - (1.0 - (ca * cb).sum(dim=-1))).mean()

def train_rc2jg_latent_calibration(readout, model, repair_memory, config,
↳prototypes=None,
                                weak_pair=None, pre_global_head=None,
↳epochs=None, steps=None, lr=None):
    rows = []
    if not repair_memory or weak_pair is None:
        return rows
    epochs = int(epochs if epochs is not None else globals().
↳get("RC2JG_REPAIR_EPOCHS", READOUT_TRAIN_EPOCHS))
    steps = int(steps if steps is not None else globals().
↳get("RC2JG_REPAIR_STEPS", READOUT_TRAIN_STEPS))
    lr = float(lr if lr is not None else globals().get("RC2JG_REPAIR_LR",
↳READOUT_LR))
    model.eval()
    for p in model.parameters(): p.requires_grad_(False)
    params = [p for p in readout.parameters() if p.requires_grad]
    if not params: return rows
    opt = torch.optim.Adam(params, lr=lr)
    for epoch in range(epochs):
        sums = {"loss":0.0, "ce":0.0, "kl":0.0, "boundary":0.0, "sep":0.0, "anchor":
↳0.0}; n=0
        for _ in range(steps):
            xb, yb, tb = sample_repair_batch(repair_memory,
↳batch_size=HEAD_REPAIR_BATCH)
            if xb is None: continue
            xb, yb, tb = xb.to(DEVICE), yb.to(DEVICE), tb.to(DEVICE)
            zf_list, y_list = [], []
            with torch.no_grad():
                for src_task_id in torch.unique(tb).detach().cpu().tolist():
                    src_task_id = int(src_task_id); mask = (tb == src_task_id)

```

```

        if not mask.any(): continue
        _, cache = forward_by_method(model, xb[mask], src_task_id,
    ↪config,
                                                prototypes=prototypes,
    ↪train=False, y=yb[mask], return_cache=True)
        zf_list.append(cache["zf"].detach()); y_list.
    ↪append(yb[mask].detach())
        if not zf_list: continue
        zf = torch.cat(zf_list, dim=0); y_mix = torch.cat(y_list, dim=0)
        opt.zero_grad(set_to_none=True)
        z_cal = readout.transform_zf(zf); logits = readout.linear(z_cal)
        ce = F.cross_entropy(logits, y_mix); kl = logits.sum() * 0.0
        if pre_global_head is not None and globals().get("RC2JG_KL_WEIGHT",
    ↪0.0) > 0:
            with torch.no_grad(): pre_logits = pre_global_head(zf)
            kl = kl_distill_loss(logits, pre_logits,
    ↪temperature=PROBE_DISTILL_TEMPERATURE)
            boundary = rc2jg_boundary_margin_loss(logits, y_mix, weak_pair)
            sep = rc2jg_latent_separation_loss(z_cal, y_mix, weak_pair)
            anchor = F.mse_loss(z_cal, zf)
            loss = ce + float(globals().get("RC2JG_KL_WEIGHT",0.35))*kl +
    ↪float(globals().get("RC2JG_BOUNDARY_MARGIN_WEIGHT",0.45))*boundary +
    ↪float(globals().get("RC2JG_LATENT_SEPARATION_WEIGHT",0.25))*sep +
    ↪float(globals().get("RC2JG_LATENT_ANCHOR_WEIGHT",0.10))*anchor
            loss.backward(); torch.nn.utils.clip_grad_norm_(params, max_norm=5.
    ↪0); opt.step()
            sums["loss"] += float(loss.detach().cpu()); sums["ce"] += float(ce.
    ↪detach().cpu()); sums["kl"] += float(kl.detach().cpu()); sums["boundary"] +=
    ↪float(boundary.detach().cpu()); sums["sep"] += float(sep.detach().cpu());
    ↪sums["anchor"] += float(anchor.detach().cpu()); n += 1
            if n > 0:
                rows.append({"repair_type":"rc2jg_latent_boundary_calibration",
    ↪"repair_epoch":epoch,
                                "weak_class":int(weak_pair[0]), "competing_class":
    ↪int(weak_pair[1]),
                                "rc2jg_loss":sums["loss"]/n, "rc2jg_ce":sums["ce"]/n,
                                "rc2jg_kl":sums["kl"]/n, "rc2jg_boundary_margin_loss":
    ↪sums["boundary"]/n,
                                "rc2jg_latent_separation_loss":sums["sep"]/n,
    ↪"rc2jg_anchor_loss":sums["anchor"]/n})
            return rows

def build_rc2b_candidate_from_checkpoint(base_run, checkpoint, variant: str,
                                         repair_memory,
    ↪context_selection_memory, policy_validation_memory,
                                         seed=0):

```

```

candidate_seed = int(seed); cp_seed = int(checkpoint["seed"]); after_task =
↪int(checkpoint["after_task"])
method_name = rc2b_candidate_method_name(variant)
base_cfg = METHOD_CONFIGS["mmals_proto_first_balanced_replay"]
model = restore_base_model_from_checkpoint(checkpoint, base_cfg)
context_prototypes = clone_prototypes(checkpoint["context_prototypes"])
config, config_rows = build_context_variant_config_and_logs(model,
↪context_selection_memory, context_prototypes, variant=variant,
↪seed=candidate_seed + 313)
readout = GlobalLinearReadout(model.global_head).to(DEVICE); repair_rows =
↪list(config_rows)
pre_global_head = copy.deepcopy(model.global_head).to(DEVICE);
↪pre_global_head.eval()
for p in pre_global_head.parameters(): p.requires_grad_(False)
rc2jg_pre, rc2jg_activated = {}, False
if variant in RC2JG_VARIANTS and globals().
↪get("ENABLE_RC2JG_GENERIC_WEAK_BOUNDARY_LATENT_CALIBRATION", False):
rc2jg_pre = rc2jg_boundary_diagnostic_on_memory(model, readout,
↪policy_validation_memory, config, prototypes=context_prototypes)
readout = WeakBoundaryLatentCalibrationReadout(model.global_head).
↪to(DEVICE)
if bool(rc2jg_pre.get("rc2jg_activation_candidate", False)):
weak_pair = (int(rc2jg_pre.get("rc2jg_weak_class", -1)),
↪int(rc2jg_pre.get("rc2jg_competing_class", -1)))
repair_rows.extend(train_rc2jg_latent_calibration(readout, model,
↪repair_memory, config, prototypes=context_prototypes, weak_pair=weak_pair,
↪pre_global_head=pre_global_head))
rc2jg_activated = True
elif variant_uses_guarded_head(variant):
repair_rows.extend(train_readout_on_memory(readout, model,
↪repair_memory, config, prototypes=context_prototypes,
↪pre_global_head=pre_global_head, kl_weight=0.35, use_kl=True,
↪margin_weight=0.0, variant=f"{variant}_rc2d_guarded_policy_candidate"))
primary_proxy = proxy_metrics_for_context_config(model,
↪policy_validation_memory, config, prototypes=context_prototypes,
↪readout=readout, max_items_per_class=VALIDATION_MAX_ITEMS_PER_CLASS)
auxiliary_proxy = proxy_metrics_for_context_config(model,
↪context_selection_memory, config, prototypes=context_prototypes,
↪readout=readout, max_items_per_class=VALIDATION_MAX_ITEMS_PER_CLASS) if
↪RC2D_AUX_PROXY_WEIGHT > 0 else None
proxy = rc2d_merge_proxy_metrics(primary_proxy, auxiliary_proxy)
if variant in RC2JG_VARIANTS:
rc2jg_post = rc2jg_boundary_diagnostic_on_memory(model, readout,
↪policy_validation_memory, config, prototypes=context_prototypes)
weak_gain = rc2jg_post.get("rc2jg_weak_class_acc", np.nan) - rc2jg_pre.
↪get("rc2jg_weak_class_acc", np.nan)

```

```

        min_gain = rc2jg_post.get("rc2jg_min_class_acc", np.nan) - rc2jg_pre.
        ↪get("rc2jg_min_class_acc", np.nan)
        global_damage = rc2jg_pre.get("rc2jg_global_acc", np.nan) - rc2jg_post.
        ↪get("rc2jg_global_acc", np.nan)
        candidate_safe = bool(rc2jg_activated and bool(rc2jg_pre.
        ↪get("rc2jg_context_healthy", False)) and (not np.isfinite(weak_gain) or
        ↪weak_gain >= globals().get("RC2JG_MIN_WEAK_CLASS_GAIN", 0.010)) and (not np.
        ↪isfinite(min_gain) or min_gain >= globals().get("RC2JG_MIN_MINCLASS_GAIN", 0.
        ↪0)) and (not np.isfinite(global_damage) or global_damage <= globals().
        ↪get("RC2JG_MAX_GLOBAL_ACC_DAMAGE", 0.003)))
        proxy.update({"rc2jg_enabled": bool(globals().
        ↪get("ENABLE_RC2JG_GENERIC_WEAK_BOUNDARY_LATENT_CALIBRATION", False)),
        ↪"rc2jg_variant": variant, "rc2jg_candidate_activated":
        ↪bool(rc2jg_activated), "rc2jg_candidate_validation_safe":
        ↪bool(candidate_safe), "rc2jg_pre_weak_class": rc2jg_pre.
        ↪get("rc2jg_weak_class", -1), "rc2jg_pre_competing_class": rc2jg_pre.
        ↪get("rc2jg_competing_class", -1), "rc2jg_pre_weak_class_acc": rc2jg_pre.
        ↪get("rc2jg_weak_class_acc", np.nan), "rc2jg_pre_min_class_acc": rc2jg_pre.
        ↪get("rc2jg_min_class_acc", np.nan), "rc2jg_pre_global_acc": rc2jg_pre.
        ↪get("rc2jg_global_acc", np.nan), "rc2jg_context_healthy": bool(rc2jg_pre.
        ↪get("rc2jg_context_healthy", False)), "rc2jg_latent_weak_boundary_detected":
        ↪bool(rc2jg_pre.get("rc2jg_latent_weak_boundary_detected", False)),
        ↪"rc2jg_post_weak_class_acc": rc2jg_post.get("rc2jg_weak_class_acc", np.nan),
        ↪"rc2jg_post_min_class_acc": rc2jg_post.get("rc2jg_min_class_acc", np.nan),
        ↪"rc2jg_post_global_acc": rc2jg_post.get("rc2jg_global_acc", np.nan),
        ↪"rc2jg_weak_class_gain": weak_gain, "rc2jg_min_class_gain": min_gain,
        ↪"rc2jg_global_acc_damage": global_damage})
        for rr in repair_rows:
            rr.update({"method": method_name, "seed": cp_seed, "after_task":
            ↪after_task, "task_id": after_task, "variant": variant, "repair_type": str(rr.
            ↪get("repair_type", "")), "rc2b_strict_candidate": True,
            ↪"rc2d_gate_aligned_candidate": True, "rc2jg_generic_weak_boundary_candidate":
            ↪bool(variant in RC2JG_VARIANTS), "rc2b_no_final_test_used_for_selection":
            ↪True, "policy_family": rc2b_policy_family(variant),
            ↪"uses_guarded_global_head": variant_uses_guarded_head(variant),
            ↪"uses_context_selection": context_variant_selection_mode(variant) is not
            ↪None, "uses_oracle_context_reference": is_oracle_context_reference(variant),
            ↪"rc2jg_candidate_activated": bool(rc2jg_activated), "rc2jg_weak_class":
            ↪rc2jg_pre.get("rc2jg_weak_class", -1), "rc2jg_competing_class": rc2jg_pre.
            ↪get("rc2jg_competing_class", -1)})
            return {"variant": variant, "method": method_name, "policy_family":
            ↪rc2b_policy_family(variant), "policy_label": rc2b_policy_label(variant),
            ↪"complexity_rank": rc2b_policy_complexity_rank(variant), "model": model,
            ↪"readout": readout, "config": config, "context_prototypes":
            ↪context_prototypes, "proxy": proxy, "primary_proxy": primary_proxy,
            ↪"auxiliary_proxy": auxiliary_proxy or {}, "repair_rows": repair_rows}

```

```

def rc2jg_candidate_safe_for_override(candidate, selected_base=None):
    if candidate is None or rc2j_variant(candidate) not in RC2JG_VARIANTS:
        ↪return False, "no_rc2jg_candidate"
    p = candidate.get("proxy", {}) or {}
    if not bool(p.get("rc2jg_candidate_validation_safe", False)): return False,
    ↪"candidate_validation_safe_false"
    if not rc2j_safe(candidate): return False, "rc2j_safe_false"
    if selected_base is not None:
        score_margin = rc2j_score(candidate) - rc2j_score(selected_base);
    ↪acc_margin = rc2j_candidate_acc(candidate) -
    ↪rc2j_candidate_acc(selected_base)
        if rc2j_finite(score_margin) and score_margin < -globals().
    ↪get("RC2JG_MIN_SCORE_GAIN", 0.0): return False,
    ↪f"score_margin_too_low={score_margin}"
        if rc2j_finite(acc_margin) and acc_margin < -globals().
    ↪get("RC2JG_MAX_GLOBAL_ACC_DAMAGE", 0.003): return False,
    ↪f"acc_margin_too_low={acc_margin}"
        return True, "rc2jg_validation_memory_safe"

def rc2c_select_candidate(candidates, after_task):
    base_candidates = [c for c in candidates if rc2j_variant(c) not in
    ↪RC2JG_VARIANTS] or candidates
    selected_base, meta = _rc2jg_prev_rc2c_select_candidate(base_candidates,
    ↪after_task)
    if not globals().
    ↪get("ENABLE_RC2JG_GENERIC_WEAK_BOUNDARY_LATENT_CALIBRATION", False):
        meta.update({"rc2jg_selector_version": RC2JG_SELECTOR_VERSION,
    ↪f"rc2jg_override_considered": False, "rc2jg_override_applied": False,
    ↪f"rc2jg_override_reason": "disabled"})
        return selected_base, meta
    rc2jg = rc2j_best([c for c in candidates if rc2j_variant(c) in
    ↪RC2JG_VARIANTS])
    safe, reason = rc2jg_candidate_safe_for_override(rc2jg,
    ↪selected_base=selected_base); applied = bool(safe)
    selected = rc2jg if applied else selected_base
    if applied:
        meta = dict(meta); meta.update({"selection_reason":
    ↪f"rc2jg_generic_weak_boundary_latent_calibration_validation_safe_override",
    ↪f"rc2j_regime_label": "generic_weak_boundary_latent_calibration",
    ↪f"rc2j_regime_reason": f"RC2JG selected by validation-memory gates: {reason}",
    ↪f"rc2j_selected_policy_family": rc2b_policy_family(rc2j_variant(rc2jg)),
    ↪f"rc2j_selected_variant": rc2j_variant(rc2jg)})
    p = rc2jg.get("proxy", {}) if rc2jg is not None else {}

```



```

    meta.update({"rc2jg_selector_version": RC2JG_SELECTOR_VERSION,
        ↪ "rc2jg_override_considered": rc2jg is not None, "rc2jg_override_applied":
        ↪ applied, "rc2jg_override_reason": reason, "rc2jg_candidate_variant":
        ↪ rc2j_variant(rc2jg), "rc2jg_candidate_score": rc2j_score(rc2jg) if rc2jg is
        ↪ not None else np.nan, "rc2jg_candidate_acc": rc2j_candidate_acc(rc2jg) if
        ↪ rc2jg is not None else np.nan, "rc2jg_selected_base_variant":
        ↪ rc2j_variant(selected_base), "rc2jg_selected_base_score":
        ↪ rc2j_score(selected_base) if selected_base is not None else np.nan,
        ↪ "rc2jg_selected_base_acc": rc2j_candidate_acc(selected_base) if
        ↪ selected_base is not None else np.nan, "rc2jg_pre_weak_class": p.
        ↪ get("rc2jg_pre_weak_class", -1), "rc2jg_pre_competing_class": p.
        ↪ get("rc2jg_pre_competing_class", -1), "rc2jg_pre_weak_class_acc": p.
        ↪ get("rc2jg_pre_weak_class_acc", np.nan), "rc2jg_post_weak_class_acc": p.
        ↪ get("rc2jg_post_weak_class_acc", np.nan), "rc2jg_weak_class_gain": p.
        ↪ get("rc2jg_weak_class_gain", np.nan), "rc2jg_min_class_gain": p.
        ↪ get("rc2jg_min_class_gain", np.nan), "rc2jg_global_acc_damage": p.
        ↪ get("rc2jg_global_acc_damage", np.nan), "rc2jg_context_healthy": bool(p.
        ↪ get("rc2jg_context_healthy", False)), "rc2jg_latent_weak_boundary_detected":
        ↪ bool(p.get("rc2jg_latent_weak_boundary_detected", False)),
        ↪ "rc2jg_candidate_validation_safe": bool(p.
        ↪ get("rc2jg_candidate_validation_safe", False)))})
    return selected, meta

print("RC2JG generic weak-boundary latent calibration installed:",
    ↪ RC2JG_SELECTOR_VERSION)
print("RC2JG variant active in candidate pool:", globals().get("RC2JG_VARIANT",
    ↪ "generic_weak_boundary_latent_calibrated") in globals().
    ↪ get("RC2B_CANDIDATE_VARIANTS", []))

```

RC2JG generic weak-boundary latent calibration installed:
rc2jg_generic_weak_boundary_latent_calibration_v1
RC2JG variant active in candidate pool: True

```

[70]: # -----
# v1.1-RC2M-beta - Dynamic boundary selector with ambiguity lock
# -----
# Selector-only, no-leakage patch.
#
# This cell MUST run after the RC2JG weak-boundary latent-calibration cell.
# It wraps candidate construction so every candidate gets validation-memory
# dynamic boundary metrics, then overrides candidate selection only when
# EXPERIMENT_PROFILE starts with RC2M_BETA.
#
# No new backbone, no new training objective, no Hydro regularizer, and no
# final-test data in candidate construction/selection.
# -----

```

```

RC2M_BETA_SELECTOR_VERSION = globals().get(
    "RC2M_BETA_VERSION",
    "rc2m_beta_dynamic_boundary_ambiguity_lock_v1",
)
RC2M_BETA_FROZEN_WEIGHTS = globals().get("RC2M_BETA_FROZEN_WEIGHTS", {
    "macro_auc": 0.5,
    "weak_boundary_auc_dynamic": 0.5,
    "macro_eer": 0.0,
    "micro_eer": 2.0,
    "weak_boundary_eer_dynamic": 0.0,
    "worst_class_eer": 0.0,
})
RC2M_BETA_HIGH_FEATURES = globals().get("RC2M_BETA_HIGH_FEATURES", [
    ↪["macro_auc", "weak_boundary_auc_dynamic", "topk_boundary_auc"])
RC2M_BETA_LOW_FEATURES = globals().get("RC2M_BETA_LOW_FEATURES", ["macro_eer", ↪
    ↪"micro_eer", "weak_boundary_eer_dynamic", "worst_class_eer", ↪
    ↪"topk_boundary_eer"])
RC2M_BETA_MIN_REQUIRED_FEATURES = int(globals().
    ↪get("RC2M_BETA_MIN_REQUIRED_FEATURES", 2))

_rc2m_beta_prev_build_rc2b_candidate_from_checkpoint = ↪
    ↪build_rc2b_candidate_from_checkpoint
_rc2m_beta_prev_rc2c_select_candidate = rc2c_select_candidate

def _rc2m_beta_finite(x):
    try:
        x = float(x)
        return np.isfinite(x)
    except Exception:
        return False

def _rc2m_beta_auc_binary(y_true, scores):
    y = np.asarray(y_true).astype(int)
    s = np.asarray(scores).astype(float)
    mask = np.isfinite(s)
    y, s = y[mask], s[mask]
    n_pos = int((y == 1).sum())
    n_neg = int((y == 0).sum())
    if n_pos == 0 or n_neg == 0:
        return np.nan
    order = np.argsort(s)
    ranks = np.empty_like(order, dtype=float)
    ranks[order] = np.arange(1, len(s) + 1, dtype=float)
    sorted_s = s[order]
    start = 0

```



```

while start < len(s):
    end = start + 1
    while end < len(s) and sorted_s[end] == sorted_s[start]:
        end += 1
    avg_rank = (start + 1 + end) / 2.0
    ranks[order[start:end]] = avg_rank
    start = end
sum_pos = ranks[y == 1].sum()
return float((sum_pos - n_pos * (n_pos + 1) / 2.0) / max(n_pos * n_neg, 1))

def _rc2m_beta_eer_binary(y_true, scores):
    y = np.asarray(y_true).astype(int)
    s = np.asarray(scores).astype(float)
    mask = np.isfinite(s)
    y, s = y[mask], s[mask]
    n_pos = int((y == 1).sum())
    n_neg = int((y == 0).sum())
    if n_pos == 0 or n_neg == 0:
        return np.nan
    thresholds = np.unique(s)
    if len(thresholds) > 512:
        thresholds = np.quantile(thresholds, np.linspace(0, 1, 512))
    best = (1.0, np.nan, np.nan)
    for thr in thresholds:
        pred_pos = s >= thr
        fpr = float((pred_pos & (y == 0)).sum() / max(n_neg, 1))
        fnr = float(((~pred_pos) & (y == 1)).sum() / max(n_pos, 1))
        diff = abs(fpr - fnr)
        if diff < best[0]:
            best = (diff, fpr, fnr)
    return float((best[1] + best[2]) / 2.0) if np.isfinite(best[1]) and np.
    isfinite(best[2]) else np.nan

def _rc2m_beta_pick_dynamic_weak_classes(class_aucs, class_eers, class_support):
    min_support = int(globals().get("RC2M_BETA_MIN_CLASS_SUPPORT", 4))
    valid_classes = []
    for c in sorted(set(class_aucs) | set(class_eers)):
        if int(class_support.get(c, 0)) < min_support:
            continue
        auc = class_aucs.get(c, np.nan)
        eer = class_eers.get(c, np.nan)
        if np.isfinite(auc) or np.isfinite(eer):
            valid_classes.append(int(c))
    if not valid_classes:
        return []

```

```

k_abs = int(globals().get("RC2M_BETA_WEAK_BOUNDARY_TOPK_ABS", 3))
k_frac = float(globals().get("RC2M_BETA_WEAK_BOUNDARY_TOPK_FRACTION", 0.25))
k_max = int(globals().get("RC2M_BETA_WEAK_BOUNDARY_TOPK_MAX", 10))
k = max(1, min(k_max, max(k_abs, int(math.ceil(len(valid_classes) *
↪k_frac)))))
# Higher weakness score = worse boundary. Prefer high EER and low AUC.
weakness = []
for c in valid_classes:
    eer = class_eers.get(c, np.nan)
    auc = class_aucs.get(c, np.nan)
    eer_part = float(eer) if np.isfinite(eer) else 0.0
    auc_part = (1.0 - float(auc)) if np.isfinite(auc) else 0.0
    weakness.append((eer_part + auc_part, c))
weakness.sort(reverse=True)
return [c for _, c in weakness[:min(k, len(weakness))]]

@torch.no_grad()
def rc2m_beta_dynamic_boundary_metrics_on_memory(model, readout, class_memory,
↪config, prototypes=None,
max_items_per_class=None):
    """Compute dataset-agnostic dynamic boundary diagnostics on validation
↪memory only."""
    max_items_per_class = max_items_per_class or globals().
↪get("VALIDATION_MAX_ITEMS_PER_CLASS", 256)
    if not class_memory:
        return {
            "rc2m_beta_boundary_metrics_available": False,
            "rc2m_beta_boundary_items": 0,
            "rc2m_beta_boundary_error": "empty_policy_validation_memory",
        }

    rows = []
    support_by_class = {}
    model.eval(); readout.eval()
    for cls, data in sorted(class_memory.items()):
        X, y, t = data["X"], data["y"], data["task_id"]
        if len(y) == 0:
            continue
        if len(y) > max_items_per_class:
            idx = torch.randperm(len(y))[:max_items_per_class]
            X, y, t = X[idx], y[idx], t[idx]
        loader = make_loader(TensorDataset(X, y, t), batch_size=BATCH_SIZE,
↪shuffle=False)
        for xb, yb, tb in loader:
            xb, yb, tb = xb.to(DEVICE), yb.to(DEVICE), tb.to(DEVICE)
            for src_task_id in torch.unique(tb).detach().cpu().tolist():

```

```

        src_task_id = int(src_task_id)
        mask = (tb == src_task_id)
        if not mask.any():
            continue
        _, cache = forward_by_method(
            model, xb[mask], src_task_id, config,
            prototypes=prototypes, train=False, y=yb[mask],
↪return_cache=True,
        )
        logits = readout(cache["zf"], cache.get("q_context"))
        seen = seen_classes_until(src_task_id)
        logits_eval = mask_logits_to_seen_classes(logits, seen)
        probs = torch.softmax(logits_eval, dim=-1).detach().cpu().
↪numpy()

        yy = yb[mask].detach().cpu().numpy().astype(int)
        for j, true_c in enumerate(yy):
            true_c = int(true_c)
            support_by_class[true_c] = support_by_class.get(true_c, 0)
↪+ 1

            row = {"y": true_c, "task_id": int(src_task_id)}
            for c in seen:
                row[f"score_class_{int(c)}"] = float(probs[j, int(c)])
            rows.append(row)

        if not rows:
            return {"rc2m_beta_boundary_metrics_available": False,
↪"rc2m_beta_boundary_items": 0}

    df = pd.DataFrame(rows)
    present_classes = sorted(int(c) for c in df["y"].dropna().unique().tolist())
    class_aucs, class_eers = {}, {}
    for c in present_classes:
        score_col = f"score_class_{c}"
        if score_col not in df.columns:
            continue
        y_bin = (df["y"].astype(int).values == int(c)).astype(int)
        scores = pd.to_numeric(df[score_col], errors="coerce").values
        class_aucs[int(c)] = _rc2m_beta_auc_binary(y_bin, scores)
        class_eers[int(c)] = _rc2m_beta_eer_binary(y_bin, scores)

    weak_classes = _rc2m_beta_pick_dynamic_weak_classes(class_aucs, class_eers,
↪support_by_class)
    auc_values = [v for v in class_aucs.values() if np.isfinite(v)]
    eer_values = [v for v in class_eers.values() if np.isfinite(v)]
    weak_aucs = [class_aucs[c] for c in weak_classes if c in class_aucs and np.
↪isfinite(class_aucs[c])]

```

```

    weak_eers = [class_eers[c] for c in weak_classes if c in class_eers and np.
↳isfinite(class_eers[c])]

    micro_y, micro_s = [], []
    for c in present_classes:
        score_col = f"score_class_{c}"
        if score_col not in df.columns:
            continue
        micro_y.extend((df["y"].astype(int).values == int(c)).astype(int).
↳tolist())
        micro_s.extend(pd.to_numeric(df[score_col], errors="coerce").values.
↳tolist())

    macro_auc = float(np.nanmean(auc_values)) if auc_values else np.nan
    micro_auc = _rc2m_beta_auc_binary(micro_y, micro_s) if micro_y else np.nan
    macro_eer = float(np.nanmean(eer_values)) if eer_values else np.nan
    micro_eer = _rc2m_beta_eer_binary(micro_y, micro_s) if micro_y else np.nan
    weak_auc = float(np.nanmean(weak_aucs)) if weak_aucs else np.nan
    weak_eer = float(np.nanmean(weak_eers)) if weak_eers else np.nan
    worst_class_auc = float(np.nanmin(auc_values)) if auc_values else np.nan
    worst_class_eer = float(np.nanmax(eer_values)) if eer_values else np.nan

    usable = [macro_auc, weak_auc, macro_eer, micro_eer, weak_eer,
↳worst_class_eer, worst_class_auc]
    n_usable = int(sum(np.isfinite(v) for v in usable))
    return {
        "rc2m_beta_boundary_metrics_available": bool(n_usable > 0),
        "rc2m_beta_boundary_feature_count": n_usable,
        "rc2m_beta_boundary_items": int(len(df)),
        "macro_auc": macro_auc,
        "micro_auc": micro_auc,
        "macro_eer": macro_eer,
        "micro_eer": micro_eer,
        "weak_boundary_classes_dynamic": json.dumps([int(c) for c in
↳weak_classes]),
        "weak_boundary_auc_dynamic": weak_auc,
        "weak_boundary_eer_dynamic": weak_eer,
        "worst_class_auc": worst_class_auc,
        "worst_class_eer": worst_class_eer,
        "topk_boundary_auc": weak_auc,
        "topk_boundary_eer": weak_eer,
        "rc2m_beta_classes_with_boundary_metrics": json.dumps(present_classes),
        "rc2m_beta_class_support_json": json.dumps({int(k): int(v) for k, v in
↳support_by_class.items()}), sort_keys=True),
        "rc2m_beta_no_final_test_used_for_boundary_metrics": True,
    }

```

```

def build_rc2b_candidate_from_checkpoint(base_run, checkpoint, variant: str,
                                         repair_memory,
⇨ context_selection_memory, policy_validation_memory,
                                         seed=0):
    cand = _rc2m_beta_prev_build_rc2b_candidate_from_checkpoint(
        base_run, checkpoint, variant,
        repair_memory=repair_memory,
        context_selection_memory=context_selection_memory,
        policy_validation_memory=policy_validation_memory,
        seed=seed,
    )
    if globals().get("ENABLE_RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK", False):
        try:
            boundary_memory_source = "policy_validation_memory"
            boundary_memory = policy_validation_memory
            if not boundary_memory:
                boundary_memory_source = "context_selection_memory_fallback"
                boundary_memory = context_selection_memory
            if not boundary_memory:
                boundary_memory_source = "repair_memory_fallback"
                boundary_memory = repair_memory
            boundary = rc2m_beta_dynamic_boundary_metrics_on_memory(
                cand["model"], cand["readout"], boundary_memory,
                cand["config"], prototypes=cand.get("context_prototypes"),
            )
            boundary["rc2m_beta_boundary_memory_source"] =
⇨ boundary_memory_source
        except Exception as e:
            boundary = {
                "rc2m_beta_boundary_metrics_available": False,
                "rc2m_beta_boundary_error": repr(e),
                "rc2m_beta_boundary_items": 0,
            }
            cand["proxy"].update(boundary)
            cand["rc2m_beta_boundary_proxy"] = boundary
            for rr in cand.get("repair_rows", []):
                rr.update({
                    "rc2m_beta_selector_candidate": True,
                    "rc2m_beta_boundary_metrics_available": boundary.
⇨ get("rc2m_beta_boundary_metrics_available", False),
                    "rc2m_beta_boundary_items": boundary.
⇨ get("rc2m_beta_boundary_items", 0),
                    "rc2m_beta_no_final_test_used_for_boundary_metrics": True,
                    "rc2m_beta_boundary_memory_source": boundary.
⇨ get("rc2m_beta_boundary_memory_source", "policy_validation_memory"),

```

```

        "weak_boundary_classes_dynamic": boundary.
    ↪get("weak_boundary_classes_dynamic", "[]"),
        })
    return cand

def _rc2m_beta_normalized_candidate_scores(candidates, high_feats, low_feats, ↪
    ↪weights):
    feature_values = {}
    used = []
    for feat in list(high_feats) + list(low_feats):
        vals = np.asarray([c.get("proxy", {}).get(feat, np.nan) for c in ↪
    ↪candidates], dtype=float)
        finite = np.isfinite(vals)
        if finite.sum() == 0:
            continue
        median = float(np.nanmedian(vals[finite]))
        vals = np.where(finite, vals, median)
        rng = float(np.nanmax(vals) - np.nanmin(vals))
        if rng < 1e-12:
            norm = np.zeros_like(vals, dtype=float)
        elif feat in low_feats:
            norm = 1.0 - (vals - np.nanmin(vals)) / rng
        else:
            norm = (vals - np.nanmin(vals)) / rng
        feature_values[feat] = norm
        used.append(feat)
    if not used:
        return None, []
    score = np.zeros(len(candidates), dtype=float)
    for feat in used:
        score += float(weights.get(feat, 1.0)) * feature_values[feat]
    return score, used

def _rc2m_beta_policy_value(c, key, default=np.nan):
    return c.get("proxy", {}).get(key, c.get(key, default))

def _rc2m_beta_best_guarded_candidate(candidates):
    guarded = set(globals().get("RC2M_BETA_GUARDED_CONTEXT_VARIANTS", ↪
    ↪["context_gap_selected", "context_temp_blend_selected_head_guard_035"]))
    matches = [c for c in candidates if c.get("variant", "") in guarded]
    if not matches:
        return None
    return max(matches, key=lambda c: float(c.get("selection_score", -1e9)) if ↪
    ↪_rc2m_beta_finite(c.get("selection_score", np.nan)) else -1e9)

```

```

def _rc2m_beta_detect_ambiguity(candidates, inherited_selected, inherited_meta,
    after_task):
    if after_task < int(globals().get("RC2M_BETA_AMBIGUITY_MIN_AFTER_TASK", 2)):
        return False, {"ambiguity_reason": "too_early"}
    guarded = set(globals().get("RC2M_BETA_GUARDED_CONTEXT_VARIANTS",
    ["context_gap_selected", "context_temp_blend_selected_head_guard_035"]))
    reasons = []
    if inherited_selected is not None and inherited_selected.get("variant", "")
    in guarded:
        reasons.append("inherited_selector_prefers_guarded_context_gap")
        # If guarded candidate has strong enough validation score and weak-boundary
        evidence,
        # treat the regime as ambiguous/protective. This is validation-memory only.
        best_guarded = _rc2m_beta_best_guarded_candidate(candidates)
        if best_guarded is not None:
            g_proxy = best_guarded.get("proxy", {})
            g_score = best_guarded.get("selection_score", np.nan)
            raw_best_score = max([float(c.get("selection_score", -1e9)) for c in
    candidates if _rc2m_beta_finite(c.get("selection_score", np.nan))] or [-1e9])
            if _rc2m_beta_finite(g_score) and float(g_score) >= raw_best_score - 0.
    03:
                reasons.append("guarded_context_gap_within_validation_tolerance")
                if _rc2m_beta_finite(g_proxy.get("proxy_context_entropy", np.nan)) and
    float(g_proxy.get("proxy_context_entropy")) > 0.60:
                    reasons.append("validation_context_entropy_not_low")
                    if _rc2m_beta_finite(g_proxy.get("proxy_context_margin", np.nan)) and
    float(g_proxy.get("proxy_context_margin")) < 0.15:
                        reasons.append("validation_context_margin_low")
                        if _rc2m_beta_finite(g_proxy.get("weak_boundary_eer_dynamic", np.nan))
    and float(g_proxy.get("weak_boundary_eer_dynamic")) > 0.05:
                            reasons.append("dynamic_weak_boundary_eer_nontrivial")
                            return bool(reasons), {"ambiguity_reason": ";\n".join(reasons) if reasons
    else "no_ambiguity_signal"}

def _rc2m_beta_not_worse(challenger, guarded, key, tol, higher_is_better=True):
    a = _rc2m_beta_policy_value(challenger, key, np.nan)
    b = _rc2m_beta_policy_value(guarded, key, np.nan)
    if not (_rc2m_beta_finite(a) and _rc2m_beta_finite(b)):
        return True
    if higher_is_better:
        return float(a) >= float(b) - float(tol)
    return float(a) <= float(b) + float(tol)

```

```

def _rc2m_beta_ambiguity_override_allowed(challenger, guarded, beta_score_map):
    if guarded is None or challenger is None:
        return True, "no_guarded_candidate"
    if challenger.get("variant", "") == guarded.get("variant", ""):
        return True, "challenger_is_guarded"
    score_margin = float(beta_score_map.get(challenger.get("variant", ""), 0.
↪0)) - float(beta_score_map.get(guarded.get("variant", ""), 0.0))
    min_margin = float(globals().get("RC2M_BETA_AMBIGUITY_SCORE_MARGIN", 0.08))
    checks = []
    checks.append((score_margin >= min_margin, f"score_margin={score_margin:.
↪4f}>={min_margin:.4f}"))
    tol = float(globals().get("RC2M_BETA_NO_REGRESSION_TOL", 0.005))
    for key in ["proxy_min_task", "proxy_old_task_min", "proxy_min_class",
↪"proxy_acc", "macro_auc"]:
        checks.append((_rc2m_beta_not_worse(challenger, guarded, key, tol,
↪higher_is_better=True), f"no_regression_{key}"))
        checks.append((_rc2m_beta_not_worse(challenger, guarded,
↪"weak_boundary_auc_dynamic", -float(globals().
↪get("RC2M_BETA_AMBIGUITY_BOUNDARY_AUC_GAIN", 0.005)),
↪higher_is_better=True), "boundary_auc_gain"))
        checks.append((_rc2m_beta_not_worse(challenger, guarded,
↪"weak_boundary_eer_dynamic", -float(globals().
↪get("RC2M_BETA_AMBIGUITY_BOUNDARY_EER_GAIN", 0.005)),
↪higher_is_better=False), "boundary_eer_gain"))
        # Audit features are veto/watchpoint only when available.
        checks.append((_rc2m_beta_not_worse(challenger, guarded,
↪"hydro_turbulence_score", float(globals().
↪get("RC2M_BETA_HYDRO_TURBULENCE_TOL", 0.03)), higher_is_better=False),
↪"hydro_turbulence_safe"))
        checks.append((_rc2m_beta_not_worse(challenger, guarded,
↪"hydro_output_drift_Dy", float(globals().
↪get("RC2M_BETA_HYDRO_OUTPUT_DRIFT_TOL", 0.03)), higher_is_better=False),
↪"hydro_output_drift_safe"))
        checks.append((_rc2m_beta_not_worse(challenger, guarded, "lsa_cosine_mean",
↪float(globals().get("RC2M_BETA_LSA_TOL", 0.02)), higher_is_better=True),
↪"lsa_safe"))
        checks.append((_rc2m_beta_not_worse(challenger, guarded,
↪"mean_normalized_gap", float(globals().get("RC2M_BETA_ENERGY_GAP_TOL", 0.
↪05)), higher_is_better=False), "energy_gap_safe"))
    passed = [name for ok, name in checks if ok]
    failed = [name for ok, name in checks if not ok]
    return (len(failed) == 0), f"passed={passed};failed={failed}"

def rc2c_select_candidate(candidates, after_task):

```



```

    if not globals().get("ENABLE_RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK",
↪False):
        return _rc2m_beta_prev_rc2c_select_candidate(candidates, after_task)

    # First compute inherited selector decision as a no-leakage reference
↪signal.
    inherited_selected, inherited_meta =
↪_rc2m_beta_prev_rc2c_select_candidate(candidates, after_task)

    score, used_features = _rc2m_beta_normalized_candidate_scores(
        candidates,
        globals().get("RC2M_BETA_HIGH_FEATURES", RC2M_BETA_HIGH_FEATURES),
        globals().get("RC2M_BETA_LOW_FEATURES", RC2M_BETA_LOW_FEATURES),
        globals().get("RC2M_BETA_FROZEN_WEIGHTS", RC2M_BETA_FROZEN_WEIGHTS),
    )
    if score is None or len(used_features) < int(globals().
↪get("RC2M_BETA_MIN_REQUIRED_FEATURES", 2)):
        meta = dict(inherited_meta)
        meta.update({
            "rc2m_beta_selector_version": RC2M_BETA_SELECTOR_VERSION,
            "rc2m_beta_enabled": True,
            "rc2m_beta_abstained": True,
            "rc2m_beta_abstain_reason":
↪f"insufficient_dynamic_boundary_features:{0 if score is None else
↪len(used_features)}",
            "rc2m_beta_fallback_selector_used": True,
            "rc2m_beta_used_features": json.dumps(used_features),
            "rc2m_beta_no_final_test_used_for_selection": True,
        })
        return inherited_selected, meta

    for c, s in zip(candidates, score):
        c["rc2m_beta_score"] = float(s)
    best_idx = int(np.argmax(score))
    boundary_best = candidates[best_idx]
    beta_score_map = {c.get("variant", f"idx_{i}"): float(score[i]) for i, c in
↪enumerate(candidates)}
    best_guarded = _rc2m_beta_best_guarded_candidate(candidates)
    ambiguity_detected, ambiguity_meta =
↪_rc2m_beta_detect_ambiguity(candidates, inherited_selected, inherited_meta,
↪after_task)

    selected = boundary_best
    lock_applied = False
    override_allowed = True
    override_reason = "not_evaluated"

```

```

    if globals().get("RC2M_BETA_AMBIGUITY_LOCK_ENABLED", True) and
↪ambiguity_detected and best_guarded is not None:
        override_allowed, override_reason =
↪rc2m_beta_ambiguity_override_allowed(boundary_best, best_guarded,
↪beta_score_map)
        if not override_allowed:
            selected = best_guarded
            lock_applied = True

    raw_best = max(candidates, key=lambda c: float(c.get("selection_score",
↪-1e9)) if _rc2m_beta_finite(c.get("selection_score", np.nan)) else -1e9)
    meta = {
        "selection_reason": "rc2m_beta_dynamic_boundary_ambiguity_lock" if
↪lock_applied else "rc2m_beta_dynamic_boundary_score_win",
        "raw_best_variant": raw_best.get("variant", ""),
        "raw_best_policy_family": raw_best.get("policy_family", ""),
        "raw_best_proxy_score": raw_best.get("selection_score", np.nan),
        "best_guarded_variant": best_guarded.get("variant", "") if best_guarded
↪is not None else "",
        "best_guarded_proxy_score": best_guarded.get("selection_score", np.nan)
↪if best_guarded is not None else np.nan,
        "no_repair_veto_applied": False,
        "guarded_hysteresis_applied": bool(lock_applied),
        "risk_floor_fallback_applied": False,
        "rc2m_beta_selector_version": RC2M_BETA_SELECTOR_VERSION,
        "rc2m_beta_enabled": True,
        "rc2m_beta_abstained": False,
        "rc2m_beta_fallback_selector_used": False,
        "rc2m_beta_boundary_best_variant": boundary_best.get("variant", ""),
        "rc2m_beta_selected_variant": selected.get("variant", ""),
        "rc2m_beta_selected_policy_family": selected.get("policy_family", ""),
        "rc2m_beta_selected_score": float(beta_score_map.get(selected.
↪get("variant", ""), np.nan)),
        "rc2m_beta_boundary_best_score": float(score[best_idx]),
        "rc2m_beta_used_features": json.dumps(used_features),
        "rc2m_beta_weights": json.dumps(globals().
↪get("RC2M_BETA_FROZEN_WEIGHTS", RC2M_BETA_FROZEN_WEIGHTS), sort_keys=True),
        "rc2m_beta_candidate_scores_json": json.dumps(beta_score_map,
↪sort_keys=True),
        "rc2m_beta_candidate_feature_counts_json": json.dumps({c.get("variant",
↪f"idx_{i}"): int(c.get("proxy", {}).get("rc2m_beta_boundary_feature_count",
↪0) or 0) for i, c in enumerate(candidates)}, sort_keys=True),
        "rc2m_beta_candidate_boundary_sources_json": json.dumps({c.
↪get("variant", f"idx_{i}"): str(c.get("proxy", {})).
↪get("rc2m_beta_boundary_memory_source", "unknown")) for i, c in
↪enumerate(candidates)}, sort_keys=True),

```

```

        "rc2m_beta_candidate_dynamic_weak_classes_json": json.dumps({c.
↪get("variant", f"idx_{i}"): str(c.get("proxy", {})).
↪get("weak_boundary_classes_dynamic", "[]")) for i, c in
↪enumerate(candidates)}, sort_keys=True),
        "rc2m_beta_ambiguity_detected": bool(ambiguity_detected),
        "rc2m_beta_ambiguity_reason": ambiguity_meta.get("ambiguity_reason",
↪""),
        "rc2m_beta_ambiguity_lock_applied": bool(lock_applied),
        "rc2m_beta_ambiguity_override_allowed": bool(override_allowed),
        "rc2m_beta_ambiguity_override_reason": override_reason,
        "rc2m_beta_no_final_test_used_for_selection": True,
    }
    return selected, meta

print("RC2M-beta dynamic-boundary ambiguity-lock selector installed:",
↪RC2M_BETA_SELECTOR_VERSION)
print("RC2M-beta enabled:", globals().
↪get("ENABLE_RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK", False))
print("RC2M-beta weights:", globals().get("RC2M_BETA_FROZEN_WEIGHTS",
↪RC2M_BETA_FROZEN_WEIGHTS))
print("RC2M-beta high features:", globals().get("RC2M_BETA_HIGH_FEATURES",
↪RC2M_BETA_HIGH_FEATURES))
print("RC2M-beta low features:", globals().get("RC2M_BETA_LOW_FEATURES",
↪RC2M_BETA_LOW_FEATURES))

```

```

RC2M-beta dynamic-boundary ambiguity-lock selector installed:
rc2m_beta_dynamic_boundary_ambiguity_lock_v1
RC2M-beta enabled: True
RC2M-beta weights: {'macro_auc': 0.5, 'weak_boundary_auc_dynamic': 0.5,
'macro_eer': 0.0, 'micro_eer': 2.0, 'weak_boundary_eer_dynamic': 0.0,
'worst_class_eer': 0.0}
RC2M-beta high features: ['macro_auc', 'weak_boundary_auc_dynamic',
'topk_boundary_auc']
RC2M-beta low features: ['macro_eer', 'micro_eer', 'weak_boundary_eer_dynamic',
'worst_class_eer', 'topk_boundary_eer']

```

2.27 11. Final frozen representation probes

The probes remain the diagnostic ceiling. If the deployed candidate stays close to the proto-context zf probe while the oracle-context probe remains higher, the remaining frontier is context inference / context memory rather than readout micro-repair.

```

[71]: class LinearProbe(nn.Module):
        def __init__(self, input_dim, n_classes=N_CLASSES):
            super().__init__()
            self.linear = nn.Linear(input_dim, n_classes)
        def forward(self, x):

```

```

        return self.linear(x)

def freeze_model(model):
    model.eval()
    for p in model.parameters():
        p.requires_grad_(False)
    return model

@torch.no_grad()
def extract_mmals_features(model, dataset, task_id, context_mode, prototypes,
    ↪feature_kind="zf", max_items=None, batch_size=BATCH_SIZE):
    model.eval()
    config = probe_config_from_mode(context_mode)
    X, y = dataset.tensors
    if max_items is not None and len(X) > max_items:
        idx = torch.randperm(len(X))[:max_items]
        X = X[idx]
        y = y[idx]
    feats, labels, tasks = [], [], []
    loader = make_loader(TensorDataset(X, y), batch_size=batch_size,
    ↪shuffle=False)
    for xb, yb in loader:
        xb, yb = xb.to(DEVICE), yb.to(DEVICE)
        logits, cache = forward_by_method(model, xb, task_id, config,
    ↪prototypes=prototypes, train=False, y=yb, return_cache=True)
        if feature_kind == "zf":
            f = cache["zf"]
        elif feature_kind == "context_feat":
            f = cache["context_feat"]
        elif feature_kind.startswith("host"):
            hid = int(feature_kind.replace("host", ""))
            f = cache["host_z"][:, hid, :]
        elif feature_kind.startswith("transformed"):
            hid = int(feature_kind.replace("transformed", ""))
            f = cache["transformed"][:, hid, :]
        elif feature_kind == "routes":
            f = cache["routes"]
        else:
            raise ValueError(f"Unknown feature_kind: {feature_kind}")
        feats.append(f.detach().cpu())
        labels.append(yb.detach().cpu())
        tasks.append(torch.full((yb.shape[0],), task_id, dtype=torch.long))
    return torch.cat(feats, dim=0).float(), torch.cat(labels, dim=0).long(),
    ↪torch.cat(tasks, dim=0).long()

def build_probe_dataset(model, tasks, prototypes, context_mode="proto",
    ↪feature_kind="zf", train=True, max_per_task=PROBE_MAX_PER_TASK):

```

```

Xs, ys, ts = [], [], []
key = "train" if train else "test"
for task_id, task in enumerate(tasks):
    X, y, t = extract_mmals_features(
        model, task[key], task_id, context_mode=context_mode,
        prototypes=prototypes, feature_kind=feature_kind,
        max_items=max_per_task if train else None,
    )
    Xs.append(X)
    ys.append(y)
    ts.append(t)
    return TensorDataset(torch.cat(Xs, dim=0), torch.cat(ys, dim=0), torch.
↪cat(ts, dim=0))

def train_linear_probe(train_ds, input_dim, seed=0, epochs=PROBE_EPOCHS):
    set_seed(seed + 70000)
    probe = LinearProbe(input_dim=input_dim).to(DEVICE)
    opt = torch.optim.Adam(probe.parameters(), lr=PROBE_LR)
    loader = make_loader(train_ds, batch_size=BATCH_SIZE, shuffle=True)
    rows = []
    for epoch in range(epochs):
        probe.train()
        total_loss, n_batches = 0.0, 0
        for xb, yb, _tb in loader:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            opt.zero_grad(set_to_none=True)
            logits = probe(xb)
            loss = F.cross_entropy(logits, yb)
            loss.backward()
            opt.step()
            total_loss += float(loss.detach().cpu())
            n_batches += 1
        rows.append({"epoch": epoch, "probe_loss": total_loss / max(n_batches, 1)})
↪1)})
    return probe, pd.DataFrame(rows)

@torch.no_grad()
def evaluate_probe(probe, test_ds, method_name, seed, backbone_source,
↪feature_kind, context_mode):
    probe.eval()
    X, y, task_ids = test_ds.tensors
    final_task = N_TASKS - 1
    seen = seen_classes_until(final_task)
    rows = []
    mat = torch.zeros(N_CLASSES, N_CLASSES, dtype=torch.long)
    for task_id in range(N_TASKS):
        mask = (task_ids == task_id)

```

```

if not mask.any():
    continue
ds = TensorDataset(X[mask], y[mask])
loader = make_loader(ds, batch_size=BATCH_SIZE, shuffle=False)
correct, total = 0, 0
class2_margin_24, class5_margin_54 = [], []
class2_logit_rank, class5_logit_rank = [], []
for xb, yb in loader:
    xb, yb = xb.to(DEVICE), yb.to(DEVICE)
    logits = mask_logits_to_seen_classes(probe(xb), seen)
    pred = logits.argmax(dim=-1)
    correct += (pred == yb).sum().item()
    total += yb.numel()
    if (yb == 2).any():
        m = (yb == 2)
        class2_margin_24.append((logits[m, 2] - logits[m, 4]).detach().
↪cpu())

        ranks = (logits[m] > logits[m, 2].unsqueeze(-1)).sum(dim=-1) + 1
        class2_logit_rank.append(ranks.float().detach().cpu())
    if (yb == 5).any():
        m = (yb == 5)
        class5_margin_54.append((logits[m, 5] - logits[m, 4]).detach().
↪cpu())

        ranks = (logits[m] > logits[m, 5].unsqueeze(-1)).sum(dim=-1) + 1
        class5_logit_rank.append(ranks.float().detach().cpu())
    for yt, yp in zip(yb.detach().cpu(), pred.detach().cpu()):
        mat[int(yt), int(yp)] += 1
rows.append({
    "method": method_name,
    "seed": seed,
    "model_family": "probe",
    "after_task": final_task,
    "task_id": task_id,
    "task_name": f"probe_task_{task_id}",
    "seen_classes": str(seen),
    "trainable_params": count_parameters(probe),
    "total_params": total_parameters(probe),
    "model_size_mb_trainable": model_size_mb(probe,
↪trainable_only=True),
    "model_size_mb_total": model_size_mb(probe, trainable_only=False),
    "uses_balanced_replay": False,
    "uses_output_distill": False,
    "probe_row": True,
    "backbone_source": backbone_source,
    "probe_feature": feature_kind,
    "probe_context_mode": context_mode,
    "replay_memory_items": 0,

```

```

        "memory_per_class": MEMORY_PER_CLASS,
        "memory_per_task": MEMORY_PER_TASK,
        "acc": correct / max(total, 1),
        "context_entropy": np.nan,
        "context_margin": np.nan,
        "context_top1_acc": np.nan,
        "context_brier": np.nan,
        "context_ece": np.nan,
        "latent_frechet_margin": np.nan,
        "prototype_weight": np.nan,
        "class2_margin_vs4": float(torch.cat(class2_margin_24).mean()) if_
↪class2_margin_24 else np.nan,
        "class5_margin_vs4": float(torch.cat(class5_margin_54).mean()) if_
↪class5_margin_54 else np.nan,
        "class2_logit_rank": float(torch.cat(class2_logit_rank).mean()) if_
↪class2_logit_rank else np.nan,
        "class5_logit_rank": float(torch.cat(class5_logit_rank).mean()) if_
↪class5_logit_rank else np.nan,
    })
    cm_rows = []
    for true_class in range(N_CLASSES):
        denom = mat[true_class].sum().item()
        for pred_class in range(N_CLASSES):
            cm_rows.append({
                "method": method_name,
                "seed": seed,
                "after_task": final_task,
                "backbone_source": backbone_source,
                "true_class": true_class,
                "pred_class": pred_class,
                "count": int(mat[true_class, pred_class].item()),
                "rate": float(mat[true_class, pred_class].item() / max(denom,
↪1)),
            })
    return pd.DataFrame(rows), pd.DataFrame(cm_rows)

def short_backbone_name(method: str) -> str:
    mapping = {
        "mmals_uniform_balanced_replay": "uniform",
        "mmals_proto_first_balanced_replay": "proto",
        "mmals_oracle_context_balanced_replay": "oracle",
    }
    return mapping.get(method, method.replace("mmals_", "").
↪replace("_balanced_replay", ""))

def run_representation_audit(artifact: Dict[str, Any]):

```

```

backbone_source = artifact["method"]
seed = artifact["seed"]
model = freeze_model(artifact["model"])
tasks = artifact["tasks"]
prototypes = artifact["context_prototypes"]
all_h, all_c, probe_loss_rows = [], [], []
# zf probes under uniform/proto/oracle contexts.
for mode in PROBE_CONTEXT_MODES:
    train_ds = build_probe_dataset(model, tasks, prototypes,
↪context_mode=mode, feature_kind="zf", train=True)
    test_ds = build_probe_dataset(model, tasks, prototypes,
↪context_mode=mode, feature_kind="zf", train=False)
    input_dim = train_ds.tensors[0].shape[1]
    probe, loss_df = train_linear_probe(train_ds, input_dim=input_dim,
↪seed=seed)
    method_name = f"probe_{short_backbone_name(backbone_source)}_zf_{mode}"
    loss_df["method"] = method_name
    loss_df["seed"] = seed
    loss_df["backbone_source"] = backbone_source
    loss_df["probe_feature"] = "zf"
    loss_df["probe_context_mode"] = mode
    h, c = evaluate_probe(probe, test_ds, method_name, seed,
↪backbone_source, "zf", mode)
    all_h.append(h)
    all_c.append(c)
    probe_loss_rows.append(loss_df)
    print(f"[{method_name} seed={seed}] probe final task accs: {[round(x,
↪3) for x in h.acc.tolist()]}")
    # Host/transformed-host probes, limited to transformed host latents for
↪compactness.
    for hid in range(N_HOSTS):
        feature_kind = f"transformed{hid}"
        mode = HOST_PROBE_MODE
        train_ds = build_probe_dataset(model, tasks, prototypes,
↪context_mode=mode, feature_kind=feature_kind, train=True)
        test_ds = build_probe_dataset(model, tasks, prototypes,
↪context_mode=mode, feature_kind=feature_kind, train=False)
        input_dim = train_ds.tensors[0].shape[1]
        probe, loss_df = train_linear_probe(train_ds, input_dim=input_dim,
↪seed=seed + hid + 10)
        method_name =
↪f"probe_{short_backbone_name(backbone_source)}_{feature_kind}_{mode}"
        loss_df["method"] = method_name
        loss_df["seed"] = seed
        loss_df["backbone_source"] = backbone_source
        loss_df["probe_feature"] = feature_kind

```



```

        loss_df["probe_context_mode"] = mode
        h, c = evaluate_probe(probe, test_ds, method_name, seed,
↪backbone_source, feature_kind, mode)
        all_h.append(h)
        all_c.append(c)
        probe_loss_rows.append(loss_df)
        print(f"[{method_name} seed={seed}] host probe accs: {[round(x, 3) for
↪x in h.acc.tolist()]}")
        return pd.concat(all_h, ignore_index=True), pd.concat(all_c,
↪ignore_index=True), pd.concat(probe_loss_rows, ignore_index=True)

```

2.28 12. Run paired v1.0-RC1b validation audit

This cell performs:

one base MMALS backbone → cloned controls and RC1b candidate variants

The primary controls are:

- paired_proto_global_no_repair: proto-first context, copied global head.
- paired_proto_kl_only_true_noop: deliberately identical no-op. It should match no-repair.
- paired_proto_global_head_ce_kl_guard_035: true corrected global CE+KL guarded-head control.

The deployable context rows are:

- paired_context_blend_selected_global: context blend selection only.
- paired_context_temp_blend_selected_global: context temp+blend selection only.
- paired_context_blend_selected_head_guard_035: rejected original RC1 diagnostic.
- paired_context_temp_blend_selected_head_guard_035: RC1b equivalent diagnostic.
- paired_context_gap_selected: **primary v1.0-RC1b candidate row**.

Oracle rows are references only. They measure the ceiling when the task/context is known; they are not deployable task-free methods.

2.28.1 12.1 Control sanity audit

Before running the expensive experiment, this cell verifies that guarded-head activation is explicit and correct: true global guard and context+guard must activate guard; no-repair, no-op and context-only must not.

```

[72]: # Control-fix sanity audit: verify that guarded-head activation is explicit and
↪correct.
guard_map = {variant: variant_uses_guarded_head(variant) for variant in
↪SELECTED_CONTEXT_HEAD_VARIANTS}
print("Guard activation map:")
for k, v in guard_map.items():
    print(f" - {k}: {v}")

```

```

assert guard_map["proto_global_head_ce_kl_guard_035"] is True, "Control-fix_
↳failed: true global guard is inactive."
assert guard_map["proto_global_no_repair"] is False, "No-repair must not_
↳activate guard."
assert guard_map["proto_kl_only_true_noop"] is False, "No-op must not activate_
↳guard."
assert guard_map["context_temp_blend_selected_head_guard_035"] is True,
↳"Context+guard candidate must activate guard."
assert guard_map["context_temp_blend_selected_global"] is False, "Context-only_
↳candidate must not activate guard."
print("Control-fix sanity checks passed.")

```

Guard activation map:

- proto_global_no_repair: False
- proto_kl_only_true_noop: False
- proto_global_head_ce_kl_guard_035: True
- oracle_global_no_repair_reference: False
- oracle_global_head_ce_kl_guard_035_reference: True
- context_blend_selected_global: False
- context_temp_blend_selected_global: False
- context_blend_selected_head_guard_035: True
- context_temp_blend_selected_head_guard_035: True
- context_gap_selected: True
- generic_weak_boundary_latent_calibrated: False

Control-fix sanity checks passed.

2.29 12.2 Comparative baseline suite

This section adds standard continual-learning baselines to the same overlapping class-incremental stress protocol.

The MMALS paired audit remains unchanged. Baselines are trained sequentially on the same tasks and evaluated with the same seen-class mask. The joint upper bound is trained on all tasks jointly and is diagnostic, not deployable in a continual-learning setting.

Diagnostic patch. Standard baseline confusion matrices now export normalized `rate` values, matching the MMALS confusion tables. The interpretation section also separates deployable baselines, deployable MMALS rows, oracle references, probes, and the joint upper bound.

```

[73]: class SparseMoEClassifier(nn.Module):
    """Small sparse mixture-of-experts baseline with a learned soft gate."""
    def __init__(self, input_dim=INPUT_DIM, hidden_dim=HIDDEN_DIM,
↳latent_dim=LATENT_DIM, n_experts=N_HOSTS, n_classes=N_CLASSES):
        super().__init__()
        self.experts = nn.ModuleList([MLPBlock(input_dim, hidden_dim,
↳latent_dim) for _ in range(n_experts)])
        self.gate = nn.Sequential(
            nn.Linear(input_dim, ROUTE_HIDDEN_DIM),

```

```

        nn.ReLU(),
        nn.Linear(ROUTE_HIDDEN_DIM, n_experts),
    )
    self.head = nn.Linear(latent_dim, n_classes)

    def forward(self, x, return_gate=False):
        weights = F.softmax(self.gate(x) / TEMPERATURE, dim=-1)
        z_stack = torch.stack([expert(x) for expert in self.experts], dim=1)
        z = (weights.unsqueeze(-1) * z_stack).sum(dim=1)
        logits = self.head(z)
        if return_gate:
            return logits, weights
        return logits

def set_requires_grad(module_or_param, requires_grad: bool):
    """
    Enable or disable gradient updates for a module or a single parameter.
    Used by the PNN baseline to freeze old columns and train only the current_
    ↪column.
    """
    if isinstance(module_or_param, nn.Module):
        for p in module_or_param.parameters():
            p.requires_grad_(requires_grad)
    elif isinstance(module_or_param, torch.nn.Parameter):
        module_or_param.requires_grad_(requires_grad)
    else:
        raise TypeError(f"Unsupported type for set_requires_grad: ↪
    ↪{type(module_or_param)}")

class PNNSimpleClassifier(nn.Module):
    """Fixed-column PNN-style modular baseline.

    One column is activated for each task; old columns are frozen after their_
    ↪task.
    The global head reads the concatenation of all columns. This is a compact_
    ↪Colab-safe
    approximation of a progressive neural network control.
    """
    def __init__(self, input_dim=INPUT_DIM, hidden_dim=HIDDEN_DIM, ↪
    ↪latent_dim=LATENT_DIM, n_tasks=N_TASKS, n_classes=N_CLASSES):
        super().__init__()
        self.columns = nn.ModuleList([MLPBlock(input_dim, hidden_dim, ↪
    ↪latent_dim) for _ in range(n_tasks)])
        self.head = nn.Linear(latent_dim * n_tasks, n_classes)

```

```

def set_trainable_column(self, task_id: int):
    for i, col in enumerate(self.columns):
        set_requires_grad(col, i == task_id)
    set_requires_grad(self.head, True)

def forward(self, x):
    zs = [col(x) for col in self.columns]
    return self.head(torch.cat(zs, dim=-1))

def baseline_forward(model, xb, method=None):
    if isinstance(model, SparseMoEClassifier):
        logits, gate = model(xb, return_gate=True)
        return logits, gate
    return model(xb), None

@torch.no_grad()
def evaluate_standard_model(model, tasks, upto_task, method, seed,
    ↪model_family, replay_memory_items=0):
    model.eval()
    rows = []
    seen = seen_classes_until(upto_task)
    trainable = count_parameters(model)
    total = total_parameters(model)
    for task_id in range(upto_task + 1):
        loader = make_loader(tasks[task_id]["test"], batch_size=BATCH_SIZE,
    ↪shuffle=False)
        correct, total_items = 0, 0
        gate_entropies = []
        class2_margin_24, class5_margin_54 = [], []
        class2_rank, class5_rank = [], []
        for xb, yb in loader:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            logits, gate = baseline_forward(model, xb, method=method)
            logits_eval = mask_logits_to_seen_classes(logits, seen)
            pred = logits_eval.argmax(dim=-1)
            correct += (pred == yb).sum().item()
            total_items += yb.numel()
            if gate is not None:
                gate_entropies.append(entropy_from_probs(gate).detach().cpu())
            if (yb == 2).any():
                m = (yb == 2)
                class2_margin_24.append((logits_eval[m, 2] - logits_eval[m, 4]).
    ↪detach().cpu())
                class2_rank.append(((logits_eval[m] > logits_eval[m, 2].
    ↪unsqueeze(-1)).sum(dim=-1) + 1).float().detach().cpu())
            if (yb == 5).any():
                m = (yb == 5)

```

```

        class5_margin_54.append((logits_eval[m, 5] - logits_eval[m, 4]).
↪detach().cpu())
        class5_rank.append(((logits_eval[m] > logits_eval[m, 5].
↪unsqueeze(-1)).sum(dim=-1) + 1).float().detach().cpu())
    rows.append({
        "method": method,
        "seed": seed,
        "model_family": model_family,
        "after_task": upto_task,
        "task_id": task_id,
        "task_name": tasks[task_id]["name"],
        "seen_classes": str(seen),
        "acc": correct / max(total_items, 1),
        "context_entropy": float(torch.cat(gate_entropies).mean()) if_
↪gate_entropies else np.nan,
        "context_margin": np.nan,
        "context_top1_acc": np.nan,
        "context_brier": np.nan,
        "context_ece": np.nan,
        "latent_frechet_margin": np.nan,
        "prototype_weight": 0.0,
        "class2_margin_vs4": float(torch.cat(class2_margin_24).mean()) if_
↪class2_margin_24 else np.nan,
        "class5_margin_vs4": float(torch.cat(class5_margin_54).mean()) if_
↪class5_margin_54 else np.nan,
        "class2_logit_rank": float(torch.cat(class2_rank).mean()) if_
↪class2_rank else np.nan,
        "class5_logit_rank": float(torch.cat(class5_rank).mean()) if_
↪class5_rank else np.nan,
        "trainable_params": trainable,
        "total_params": total,
        "backbone_params": total,
        "readout_trainable_params": np.nan,
        "readout_total_params": np.nan,
        "model_size_mb_total": total * 4 / (1024 ** 2),
        "uses_balanced_replay": method == "baseline_balanced_replay",
        "uses_output_distill": method == "baseline_lwf",
        "uses_local_mycorrhizal_head": False,
        "uses_global_head_refresh": False,
        "uses_context_selection": False,
        "uses_oracle_context_reference": False,
        "probe_row": False,
        "backbone_source": "baseline_sequential_or_joint",
        "probe_feature": "baseline_logits",
        "probe_context_mode": "none",
        "replay_memory_items": replay_memory_items,

```

```

        "memory_per_class": MEMORY_PER_CLASS,
        "memory_per_task": MEMORY_PER_TASK,
        "paired_same_backbone": False,
        "readout_variant": "baseline",
        "context_variant": "none",
    })
    return pd.DataFrame(rows)

@torch.no_grad()
def class_confusion_matrix_standard(model, tasks, upto_task, method, seed):
    """Class-confusion table for standard baselines, normalized like the MMALS_
    ↪confusion table.

    Diagnostic patch:
    the original benchmark-harness smoke run wrote raw counts but did not write_
    ↪the
    per-true-class `rate` column. `class_diagnostics_from_confusion` expects_
    ↪this
    normalized field; without it, baseline class diagnostics become NaN/flat and
    cannot be compared fairly to MMALS rows.
    """
    model.eval()
    seen = seen_classes_until(upto_task)
    cm = torch.zeros((N_CLASSES, N_CLASSES), dtype=torch.long)
    for task_id in range(upto_task + 1):
        loader = make_loader(tasks[task_id]["test"], batch_size=BATCH_SIZE,
        ↪shuffle=False)
        for xb, yb in loader:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            logits, _ = baseline_forward(model, xb, method=method)
            pred = mask_logits_to_seen_classes(logits, seen).argmax(dim=-1)
            for t, p in zip(yb.detach().cpu(), pred.detach().cpu()):
                cm[int(t), int(p)] += 1

    rows = []
    for true_cls in range(N_CLASSES):
        row_total = int(cm[true_cls].sum().item())
        for pred_cls in range(N_CLASSES):
            count = int(cm[true_cls, pred_cls].item())
            rows.append({
                "true_class": true_cls,
                "pred_class": pred_cls,
                "count": count,
                "rate": float(count / max(row_total, 1)),
                "method": method,
                "seed": seed,
                "after_task": upto_task,
            })

```

```

        "backbone_source": "baseline_sequential_or_joint",
        "readout_variant": "baseline",
        "context_variant": "none",
        "uses_oracle_context_reference": False,
    })
    return pd.DataFrame(rows)

def ewc_compute_fisher(model, dataset, max_items=512):
    model.eval()
    params = {n: p.detach().clone() for n, p in model.named_parameters() if p.
    ↪requires_grad}
    fisher = {n: torch.zeros_like(p, device=DEVICE) for n, p in model.
    ↪named_parameters() if p.requires_grad}
    X, y = dataset.tensors
    if len(X) > max_items:
        idx = torch.randperm(len(X))[:max_items]
        dataset = TensorDataset(X[idx], y[idx])
    loader = make_loader(dataset, batch_size=BATCH_SIZE, shuffle=False)
    n_batches = 0
    for xb, yb in loader:
        xb, yb = xb.to(DEVICE), yb.to(DEVICE)
        model.zero_grad(set_to_none=True)
        logits, _ = baseline_forward(model, xb)
        loss = F.cross_entropy(logits, yb)
        loss.backward()
        for name, p in model.named_parameters():
            if p.grad is not None and name in fisher:
                fisher[name] += p.grad.detach().pow(2)
        n_batches += 1
    for name in fisher:
        fisher[name] = fisher[name] / max(n_batches, 1)
    return {"params": params, "fisher": fisher}

def ewc_penalty(model, ewc_snapshots):
    if not ewc_snapshots:
        return torch.zeros((), device=DEVICE)
    penalty = torch.zeros((), device=DEVICE)
    current = dict(model.named_parameters())
    for snap in ewc_snapshots:
        for name, old_param in snap["params"].items():
            if name in current:
                penalty = penalty + (snap["fisher"][name] * (current[name] -
    ↪old_param.to(DEVICE)).pow(2)).sum()
    return penalty

def train_sequential_baseline(method: str, seed: int):
    set_seed(seed)

```

```

clear_memory()
tasks = build_stress_tasks(DATASET_NAME, seed=seed)
if method == "baseline_sparse_moe":
    model = SparseMoEClassifier().to(DEVICE)
    model_family = "sparse_moe"
elif method == "baseline_pnn":
    model = PNNSimpleClassifier().to(DEVICE)
    model_family = "pnn_fixed_columns"
else:
    model = MLPBaseline().to(DEVICE)
    model_family = "mlp_baseline"
class_memory: Dict[int, Dict[str, torch.Tensor]] = {}
memory_buffers: Dict[int, TensorDataset] = {}
teacher = None
ewc_snapshots = []
history_rows, loss_rows, confusion_rows = [], [], []
for task_id, task in enumerate(tasks):
    if method == "baseline_pnn":
        model.set_trainable_column(task_id)
        opt = torch.optim.Adam([p for p in model.parameters() if p.
↪requires_grad], lr=LR)
        loader = make_loader(task["train"], batch_size=BATCH_SIZE, shuffle=True)
        for epoch in range(EPOCHS_PER_TASK):
            model.train()
            sums = {"loss": 0.0, "task": 0.0, "replay": 0.0, "distill": 0.0,
↪"ewc": 0.0, "sparse": 0.0}
            n_batches = 0
            for xb, yb in loader:
                xb, yb = xb.to(DEVICE), yb.to(DEVICE)
                opt.zero_grad(set_to_none=True)
                logits, gate = baseline_forward(model, xb, method=method)
                task_loss = F.cross_entropy(logits, yb)
                replay_loss = torch.zeros(), device=DEVICE)
                if method in ["baseline_experience_replay",
↪"baseline_balanced_replay"] and class_memory:
                    xb_r, yb_r, _tb =
↪sample_balanced_class_memory(class_memory, batch_size=BALANCED_REPLAY_BATCH)
                    if xb_r is not None:
                        logits_r, _ = baseline_forward(model, xb_r,
↪method=method)
                        replay_loss = F.cross_entropy(logits_r, yb_r) *
↪BASELINE_REPLAY_WEIGHT
                        distill_loss = torch.zeros(), device=DEVICE)
                    if method == "baseline_lwf" and teacher is not None:
                        with torch.no_grad():

```



```

        teacher_logits, _ = baseline_forward(teacher, xb,
↪method=method)
        distill_loss = LWF_LAMBDA * kl_distill_loss(logits,
↪teacher_logits)
        ewc_loss = torch.zeros(), device=DEVICE)
        if method == "baseline_ewc" and ewc_snapshots:
            ewc_loss = EWC_LAMBDA * ewc_penalty(model, ewc_snapshots)
        sparse_loss = torch.zeros(), device=DEVICE)
        if method == "baseline_sparse_moe" and gate is not None:
            sparse_loss = SPARSE_MOE_ENTROPY_WEIGHT *
↪entropy_from_probs(gate).mean()
        loss = task_loss + replay_loss + distill_loss + ewc_loss +
↪sparse_loss
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)
        opt.step()
        sums["loss"] += float(loss.detach().cpu())
        sums["task"] += float(task_loss.detach().cpu())
        sums["replay"] += float(replay_loss.detach().cpu())
        sums["distill"] += float(distill_loss.detach().cpu())
        sums["ewc"] += float(ewc_loss.detach().cpu())
        sums["sparse"] += float(sparse_loss.detach().cpu())
        n_batches += 1
        loss_rows.append({
            "method": method,
            "seed": seed,
            "task_id": task_id,
            "epoch": epoch,
            "loss": sums["loss"] / max(n_batches, 1),
            "task_loss": sums["task"] / max(n_batches, 1),
            "memory_loss": sums["distill"] / max(n_batches, 1),
            "context_loss": 0.0,
            "context_replay_loss": 0.0,
            "balanced_replay_loss": sums["replay"] / max(n_batches, 1),
            "fungal_loss": sums["sparse"] / max(n_batches, 1),
            "ewc_loss": sums["ewc"] / max(n_batches, 1),
        })
        eval_rows = evaluate_standard_model(model, tasks, task_id, method,
↪seed, model_family, replay_memory_items=int(sum(len(v["y"]) for v in
↪class_memory.values()))
        history_rows.append(eval_rows)
        print(f"[{method}] seed={seed}] after task {task_id}: {[round(a, 3) for
↪a in eval_rows['acc'].tolist()]}")
        class_memory = update_class_memory(class_memory, task["train"],
↪task_id, MEMORY_PER_CLASS, seed=seed + 3000 + task_id)

```

```

        memory_buffers[task_id] = build_memory_subset(task["train"],
MEMORY_PER_TASK, seed=seed + 4000 + task_id)
        if method == "baseline_lwf":
            teacher = clone_teacher(model)
        if method == "baseline_ewc":
            ewc_snapshots.append(ewc_compute_fisher(model, task["train"],
max_items=min(512, len(task["train"]))))
        clear_memory()
        confusion_rows.append(class_confusion_matrix_standard(model, tasks, N_TASKS -
1, method, seed))
        return pd.concat(history_rows, ignore_index=True), pd.DataFrame(loss_rows),
pd.concat(confusion_rows, ignore_index=True)

def train_joint_upper_bound_baseline(seed: int):
    set_seed(seed)
    clear_memory()
    tasks = build_stress_tasks(DATASET_NAME, seed=seed)
    model = MLPBaseline().to(DEVICE)
    X_train = torch.cat([t["train"].tensors[0] for t in tasks], dim=0)
    y_train = torch.cat([t["train"].tensors[1] for t in tasks], dim=0)
    joint_ds = TensorDataset(X_train, y_train)
    opt = torch.optim.Adam(model.parameters(), lr=LR)
    # Same total training budget as one pass over all sequential tasks.
    for epoch in range(EPOCHS_PER_TASK * N_TASKS):
        model.train()
        for xb, yb in make_loader(joint_ds, batch_size=BATCH_SIZE,
shuffle=True):
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            opt.zero_grad(set_to_none=True)
            logits, _ = baseline_forward(model, xb)
            loss = F.cross_entropy(logits, yb)
            loss.backward()
            opt.step()

        method = "baseline_joint_upper_bound"
        h = evaluate_standard_model(model, tasks, N_TASKS - 1, method, seed,
"joint_upper_bound", replay_memory_items=len(joint_ds))
        # Fill a final-only loss row; forgetting is not meaningful for joint upper
bound.
        l = pd.DataFrame([{"method": method, "seed": seed, "task_id": N_TASKS - 1,
"epoch": EPOCHS_PER_TASK * N_TASKS, "loss": np.nan, "task_loss": np.nan}])
        c = class_confusion_matrix_standard(model, tasks, N_TASKS - 1, method, seed)
        print(f"[{method} seed={seed}] final joint eval: {[round(a, 3) for a in
h['acc'].tolist()}]")
        return h, l, c

def run_benchmark_baselines_for_seed(seed: int):

```

```

method_map = {
    "finetune": "baseline_finetune",
    "ewc": "baseline_ewc",
    "lwf": "baseline_lwf",
    "experience_replay": "baseline_experience_replay",
    "balanced_replay": "baseline_balanced_replay",
    "sparse_moe": "baseline_sparse_moe",
    "pnn": "baseline_pnn",
    "joint_upper_bound": "baseline_joint_upper_bound",
}
histories, losses, confusions = [], [], []
for short_name in SELECTED_BASELINE_METHODS:
    method = method_map[short_name]
    if method == "baseline_joint_upper_bound":
        h, l, c = train_joint_upper_bound_baseline(seed)
    else:
        h, l, c = train_sequential_baseline(method, seed)
    histories.append(h)
    losses.append(l)
    confusions.append(c)
return histories, losses, confusions

```

2.30 RC2jh Energy Context Probe Audit

Audit-only energy landscape probe for context recovery. This cell group must run before the main experiment loop.

```

[74]: # -----
# RC2jh Energy Context Probe - audit-only controls
# -----
# This probe does not alter MMALS training, policy selection, Hydro, LSA, or
↳ final-test gates.
# It trains a small auxiliary energy model on replay/validation memory only,
↳ then freezes it and
# optimizes the continuous context vector c while z0 and m0 are fixed.
ACTIVATE_ENERGY_CONTEXT_PROBE = True
ENERGY_PROBE_AUDIT_ONLY = True
ENERGY_PROBE_VERSION = "RC2jh_energy_context_probe_audit_v1"

ENERGY_PROBE_LAMBDA_MEMORY = 1.0
ENERGY_PROBE_LAMBDA_CONTEXT = 1.0
ENERGY_PROBE_SIGMA_C = 0.25
ENERGY_PROBE_LR_CONTEXT = 1e-2
ENERGY_PROBE_STEPS = 200 if RUN_MODE == "smoke" else 500
ENERGY_PROBE_N_RESTARTS = 4 if RUN_MODE == "smoke" else 8
ENERGY_PROBE_MAX_ITEMS = 96 if RUN_MODE == "smoke" else 256
ENERGY_PROBE_THETA_EPOCHS = 80 if RUN_MODE == "smoke" else 180

```

```

ENERGY_PROBE_THETA_LR = 2e-3
ENERGY_PROBE_PLOT_GRID = 31
ENERGY_PROBE_ORACLE_GAP_TARGET = 0.029
ENERGY_PROBE_MIN_REL_ENERGY_DROP = 0.05
ENERGY_PROBE_PLATEAU_GRAD_NORM = 1e-5
ENERGY_PROBE_DIVERGENCE_ENERGY_MULT = 5.0

print("Energy Context Probe:", ENERGY_PROBE_VERSION)
print("ACTIVATE_ENERGY_CONTEXT_PROBE:", ACTIVATE_ENERGY_CONTEXT_PROBE)
print("ENERGY_PROBE_AUDIT_ONLY:", ENERGY_PROBE_AUDIT_ONLY)

```

```

Energy Context Probe: RC2jh_energy_context_probe_audit_v1
ACTIVATE_ENERGY_CONTEXT_PROBE: True
ENERGY_PROBE_AUDIT_ONLY: True

```

```

[75]: # -----
# RC2jh Energy Context Probe - implementation
# -----

import json
from pathlib import Path

class EnergyProbeTheta(nn.Module):
    """Small auxiliary energy model for audit-only context recovery.

    
$$E(z, m, c) = ||z - D(m, c)||^2 + \lambda_m ||m - M(c)||^2 + \lambda_c * (1/(2 \sigma_c^2)) ||c - \mu(z)||^2$$


    """
    def __init__(self, z_dim, m_dim, c_dim, hidden=128):
        super().__init__()
        self.z_dim = int(z_dim)
        self.m_dim = int(m_dim)
        self.c_dim = int(c_dim)
        self.decoder = nn.Sequential(
            nn.Linear(m_dim + c_dim, hidden), nn.LayerNorm(hidden), nn.GELU(),
            nn.Linear(hidden, hidden), nn.GELU(), nn.Linear(hidden, z_dim),
        )
        self.memory_predictor = nn.Sequential(
            nn.Linear(c_dim, hidden), nn.LayerNorm(hidden), nn.GELU(),
            nn.Linear(hidden, m_dim),
        )
        self.context_encoder = nn.Sequential(
            nn.Linear(z_dim, hidden), nn.LayerNorm(hidden), nn.GELU(),
            nn.Linear(hidden, c_dim),
        )

    def forward_terms(self, z, m, c, lambda_memory=None, lambda_context=None,
        ↪sigma_c=None):

```

```

        if lambda_memory is None:
            lambda_memory = ENERGY_PROBE_LAMBDA_MEMORY
        if lambda_context is None:
            lambda_context = ENERGY_PROBE_LAMBDA_CONTEXT
        if sigma_c is None:
            sigma_c = ENERGY_PROBE_SIGMA_C
        z_hat = self.decoder(torch.cat([m, c], dim=-1))
        m_hat = self.memory_predictor(c)
        c_mu = self.context_encoder(z)
        reconstruction_term = F.mse_loss(z_hat, z, reduction="none").
↪mean(dim=-1)
        memory_term = F.mse_loss(m_hat, m, reduction="none").mean(dim=-1)
        context_nll_term = (1.0 / (2.0 * float(sigma_c) ** 2)) * F.mse_loss(c,
↪c_mu, reduction="none").mean(dim=-1)
        total_energy = reconstruction_term + float(lambda_memory) * memory_term
↪+ float(lambda_context) * context_nll_term
        return {
            "reconstruction_term": reconstruction_term,
            "memory_term": memory_term,
            "context_nll_term": context_nll_term,
            "total_energy": total_energy,
            "z_hat": z_hat,
            "m_hat": m_hat,
            "c_mu": c_mu,
        }

    def energy(self, z, m, c, **kwargs):
        return self.forward_terms(z, m, c, **kwargs)["total_energy"]

def _ep_to_tensor(x, dtype=torch.float32):
    if isinstance(x, torch.Tensor):
        return x.detach().float().cpu()
    return torch.tensor(x, dtype=dtype).detach().cpu()

def _ep_extract_proto_vector(proto):
    """Best-effort extraction from RC2 context prototype dictionaries."""
    if isinstance(proto, torch.Tensor):
        return _ep_to_tensor(proto).flatten()
    if isinstance(proto, np.ndarray):
        return _ep_to_tensor(proto).flatten()
    if isinstance(proto, dict):
        for outer in ["latent", "feature", "context", "center", "mu", "mean"]:
            if outer in proto:
                value = proto[outer]
                if isinstance(value, dict):

```

```

        for key in ["mean", "mu", "center", "avg", "prototype"]:
            if key in value:
                return _ep_to_tensor(value[key]).flatten()
        return _ep_extract_proto_vector(value)
    return None

def stack_context_prototypes(context_prototypes, n_contexts=N_CONTEXTS,
    ↪ fallback_dim=CONTEXT_FEAT_DIM):
    vecs = []
    if isinstance(context_prototypes, dict) and context_prototypes:
        for cid in sorted(context_prototypes.keys()):
            v = _ep_extract_proto_vector(context_prototypes[cid])
            if v is not None and v.numel() > 0:
                vecs.append(v)
    if not vecs:
        gen = torch.Generator().manual_seed(12345)
        return torch.randn(int(n_contexts), int(fallback_dim), generator=gen).
    ↪ float()
    dim = min(int(v.numel()) for v in vecs)
    bank = torch.stack([v[:dim].float() for v in vecs], dim=0)
    if bank.shape[0] < n_contexts:
        pad = bank.mean(dim=0, keepdim=True).repeat(n_contexts - bank.shape[0],
    ↪ 1)
        bank = torch.cat([bank, pad], dim=0)
    return bank[:n_contexts].float()

def _ep_dataset_tensors(ds):
    if isinstance(ds, TensorDataset):
        tensors = ds.tensors
        if len(tensors) >= 2:
            return tensors[0].detach().cpu(), tensors[1].detach().cpu()
    if isinstance(ds, dict):
        x = ds.get("x", ds.get("X", None))
        y = ds.get("y", ds.get("Y", None))
        if x is not None and y is not None:
            return _ep_to_tensor(x), torch.as_tensor(y).long().detach().cpu()
    return None, None

def _ep_sample_memory_dataset(ds, max_items, seed=0):
    X, y = _ep_dataset_tensors(ds)
    if X is None:
        return None, None
    n = int(len(y))
    if n == 0:

```

```

        return None, None
    k = min(int(max_items), n)
    gen = torch.Generator().manual_seed(int(seed))
    idx = torch.randperm(n, generator=gen)[:k]
    return X[idx].float(), y[idx].long()

@torch.no_grad()
def _ep_forward_zf(model, x, task_id, config, prototypes):
    """Extract mediated latent zf with the same forward protocol as the RC2_
    ↪ harness when available."""
    x = x.to(DEVICE)
    model.eval()
    # Preferred RC2h/RC2jg path.
    if "forward_by_method" in globals():
        try:
            _, cache = forward_by_method(model, x, int(task_id), config,
    ↪ prototypes=prototypes, train=False, return_cache=True)
            if isinstance(cache, dict) and "zf" in cache:
                return cache["zf"].detach().cpu(), cache.get("q_context", None)
            except Exception:
                pass
        # v0.13/CE path.
        try:
            _, cache = model(x, int(task_id), context_id=int(task_id),
    ↪ return_cache=True)
            if isinstance(cache, dict) and "zf" in cache:
                return cache["zf"].detach().cpu(), cache.get("q_context", None)
            except Exception:
                pass
        # v0.7/v0.9 path.
        try:
            out = model.forward_with_details(x, int(task_id))
            zf = out[-1]
            return zf.detach().cpu(), None
        except Exception as e:
            raise RuntimeError(f"Unable to extract zf for energy probe: {type(e)}.
    ↪ __name__: {e}")

def _ep_context_for_task(context_bank, task_id):
    idx = int(task_id) % int(context_bank.shape[0])
    return context_bank[idx].float()

def build_energy_probe_dataset_from_artifact(artifact, max_items=None):
    if max_items is None:

```

```

        max_items = ENERGY_PROBE_MAX_ITEMS
    model = artifact["model"]
    config = artifact.get("config", {})
    prototypes = artifact.get("context_prototypes", {})
    memory_buffers = artifact.get("memory_buffers", {})
    context_bank = stack_context_prototypes(prototypes, n_contexts=N_CONTEXTS)
    rows = []
    z_parts, m_parts, c_parts, tid_parts = [], [], [], []
    for task_id, mem_ds in sorted(memory_buffers.items()):
        X, _ = _ep_sample_memory_dataset(mem_ds, max_items=max_items,
        ↪seed=10_000 + int(task_id))
        if X is None:
            continue
        zf, _ = _ep_forward_zf(model, X, int(task_id), config, prototypes)
        if zf is None or zf.numel() == 0:
            continue
        m0 = zf.mean(dim=0, keepdim=True).repeat(zf.shape[0], 1)
        c0 = _ep_context_for_task(context_bank, int(task_id)).view(1, -1).
        ↪repeat(zf.shape[0], 1)
        z_parts.append(zf.float())
        m_parts.append(m0.float())
        c_parts.append(c0.float())
        tid_parts.append(torch.full((zf.shape[0],), int(task_id), dtype=torch.
        ↪long))
        if not z_parts:
            return None
    data = {
        "Z": torch.cat(z_parts, dim=0).float(),
        "M": torch.cat(m_parts, dim=0).float(),
        "C": torch.cat(c_parts, dim=0).float(),
        "task_id": torch.cat(tid_parts, dim=0).long(),
        "context_bank": context_bank.float(),
    }
    return data

def train_energy_probe_theta(probe_data, seed=0):
    set_seed(int(seed))
    Z, M, C = probe_data["Z"].to(DEVICE), probe_data["M"].to(DEVICE),
    ↪probe_data["C"].to(DEVICE)
    model = EnergyProbeTheta(Z.shape[1], M.shape[1], C.shape[1]).to(DEVICE)
    opt = torch.optim.Adam(model.parameters(), lr=ENERGY_PROBE_THETA_LR)
    n = Z.shape[0]
    batch = min(BATCH_SIZE, n)
    loss_rows = []
    for epoch in range(int(ENERGY_PROBE_THETA_EPOCHS)):
        perm = torch.randperm(n, device=DEVICE)

```



```

        totals = []
        for start in range(0, n, batch):
            idx = perm[start:start + batch]
            terms = model.forward_terms(Z[idx], M[idx], C[idx])
            loss = terms["total_energy"].mean()
            opt.zero_grad(set_to_none=True)
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), 5.0)
            opt.step()
            totals.append(float(loss.detach().cpu()))
        if epoch in [0, 1, 2, int(ENERGY_PROBE_THETA_EPOCHS) - 1] or epoch % 25 == 0:
            loss_rows.append({"epoch": epoch, "energy_theta_loss": float(np.
            ↪mean(totals))})
        return model, pd.DataFrame(loss_rows)

def diagnose_energy_probe(trace_df, final_nearest_hit):
    if trace_df is None or trace_df.empty:
        return "DIVERGENCE_OR_UNSTABLE"
    if not np.isfinite(trace_df["energy"].astype(float)).all():
        return "DIVERGENCE_OR_UNSTABLE"
    e0 = float(trace_df["energy"].iloc[0])
    e1 = float(trace_df["energy"].iloc[-1])
    g1 = float(trace_df["grad_norm"].iloc[-1])
    d0 = float(trace_df["oracle_gap"].iloc[0])
    d1 = float(trace_df["oracle_gap"].iloc[-1])
    denom = max(abs(e0), 1e-8)
    rel_drop = (e0 - e1) / denom
    if e1 > ENERGY_PROBE_DIVERGENCE_ENERGY_MULT * max(e0, 1e-8):
        return "DIVERGENCE_OR_UNSTABLE"
    if abs(rel_drop) < 0.01 and g1 < ENERGY_PROBE_PLATEAU_GRAD_NORM and d1 >= 0.
    ↪95 * d0:
        return "PLATEAU_FLAT_LANDSCAPE"
    if final_nearest_hit and d1 < d0 and rel_drop >= ENERGY_PROBE_MIN_REL_ENERGY_DROP:
        ↪ENERGY_PROBE_MIN_REL_ENERGY_DROP:
            return "SUCCESS_INFORMATIVE_ENERGY"
    if (not final_nearest_hit) and rel_drop >= ENERGY_PROBE_MIN_REL_ENERGY_DROP
    ↪and g1 < 5e-4:
        return "LOCAL_MINIMUM_WRONG_CONTEXT"
    if d1 < 0.85 * d0:
        return "AMBIGUOUS_PARTIAL_SIGNAL"
    return "PLATEAU_FLAT_LANDSCAPE"

def optimize_context_probe(theta, z0, m0, c_oracle, context_bank, sample_id=0,
    ↪seed=0):

```

```

theta.eval()
z0 = z0.view(1, -1).to(DEVICE).detach()
m0 = m0.view(1, -1).to(DEVICE).detach()
c_oracle = c_oracle.view(1, -1).to(DEVICE).detach()
context_bank = context_bank.to(DEVICE).float()
best = None
all_traces = []
for restart in range(int(ENERGY_PROBE_N_RESTARTS)):
    gen = torch.Generator(device=DEVICE).manual_seed(int(seed) + 1009 *
↪restart + int(sample_id))
    c = (torch.randn(c_oracle.shape, generator=gen, device=DEVICE) * 0.5 +
↪c_oracle.mean()).detach().clone().requires_grad_(True)
    opt = torch.optim.Adam([c], lr=ENERGY_PROBE_LR_CONTEXT)
    rows = []
    for step in range(int(ENERGY_PROBE_STEPS)):
        terms = theta.forward_terms(z0, m0, c)
        energy = terms["total_energy"].mean()
        opt.zero_grad(set_to_none=True)
        energy.backward()
        grad_norm = float(c.grad.detach().norm().cpu()) if c.grad is not
↪None else 0.0
        opt.step()
        with torch.no_grad():
            gap = float(torch.norm(c - c_oracle).detach().cpu())
            cosine = float(F.cosine_similarity(c, c_oracle, dim=-1).
↪detach().cpu().item())
            dist = torch.norm(context_bank - c.detach().view(1, -1), dim=-1)
            nearest = int(torch.argmax(dist).detach().cpu())
            rows.append({
                "sample_id": int(sample_id), "restart": int(restart), "step":
↪int(step),
                "energy": float(energy.detach().cpu()),
                "reconstruction_term": float(terms["reconstruction_term"].
↪mean().detach().cpu()),
                "memory_term": float(terms["memory_term"].mean().detach().
↪cpu()),
                "context_nll_term": float(terms["context_nll_term"].mean().
↪detach().cpu()),
                "grad_norm": grad_norm,
                "oracle_gap": gap,
                "oracle_cosine": cosine,
                "nearest_context_id": nearest,
            })
    trace = pd.DataFrame(rows)
    all_traces.append(trace)

```

```

        final_energy = float(trace.energy.iloc[-1]) if not trace.empty else
↪float("inf")
        if best is None or final_energy < best["final_energy"]:
            best = {"final_energy": final_energy, "trace": trace, "c_star": c.
↪detach().cpu()}
        best_trace = best["trace"]
        final_nearest = int(best_trace.nearest_context_id.iloc[-1])
        oracle_id = int(torch.argmax(torch.norm(context_bank.detach().cpu() -
↪c_oracle.detach().cpu(), dim=-1)))
        hit = bool(final_nearest == oracle_id)
        diagnosis = diagnose_energy_probe(best_trace, hit)
        summary = {
            "sample_id": int(sample_id),
            "oracle_context_id": oracle_id,
            "final_nearest_context_id": final_nearest,
            "oracle_hit": hit,
            "initial_context_gap": float(best_trace.oracle_gap.iloc[0]),
            "final_context_gap": float(best_trace.oracle_gap.iloc[-1]),
            "oracle_gap_target": float(ENERGY_PROBE_ORACLE_GAP_TARGET),
            "gap_ratio_vs_target": float(best_trace.oracle_gap.iloc[-1]) /
↪max(float(ENERGY_PROBE_ORACLE_GAP_TARGET), 1e-12),
            "energy_initial": float(best_trace.energy.iloc[0]),
            "energy_final": float(best_trace.energy.iloc[-1]),
            "energy_drop": float(best_trace.energy.iloc[0] - best_trace.energy.
↪iloc[-1]),
            "diagnosis": diagnosis,
        }
        trace_all = pd.concat(all_traces, ignore_index=True)
        return summary, best_trace, trace_all

def plot_energy_probe_trace(trace_df, out_png, title="Energy Context Probe"):
    if trace_df is None or trace_df.empty:
        return None
    plt.figure(figsize=(8, 4))
    plt.plot(trace_df["step"], trace_df["energy"], label="energy")
    plt.plot(trace_df["step"], trace_df["oracle_gap"], label="oracle_gap")
    plt.plot(trace_df["step"], trace_df["grad_norm"], label="grad_norm")
    plt.yscale("log")
    plt.xlabel("Optimization step")
    plt.title(title)
    plt.legend()
    plt.tight_layout()
    plt.savefig(out_png, dpi=160)
    plt.close()
    return str(out_png)

```

```

def run_energy_context_probe_from_artifact(artifact):
    """Main entry point called after
    ↪ final_probe_artifact_from_base_run(base_run).

    Uses replay/memory artifacts only. It does not read final test data and
    ↪ does not change the trained model.
    """
    seed = int(artifact.get("seed", 0))
    probe_dir = RESULTS_DIR / f"energy_context_probe_seed{seed}_{RUN_MODE}"
    probe_dir.mkdir(parents=True, exist_ok=True)
    probe_data = build_energy_probe_dataset_from_artifact(artifact,
    ↪ max_items=ENERGY_PROBE_MAX_ITEMS)
    if probe_data is None:
        summary_df = pd.DataFrame([{
            "seed": seed, "energy_probe_version": ENERGY_PROBE_VERSION,
            "diagnosis": "NO_MEMORY_AVAILABLE", "oracle_hit_rate": np.nan,
        }])
        return pd.DataFrame(), summary_df, pd.DataFrame()
    theta, theta_loss_df = train_energy_probe_theta(probe_data, seed=seed)
    theta_loss_df["seed"] = seed
    theta_loss_df.to_csv(probe_dir / "energy_theta_train_loss.csv", index=False)
    Z, M, C, T = probe_data["Z"], probe_data["M"], probe_data["C"],
    ↪ probe_data["task_id"]
    context_bank = probe_data["context_bank"]
    n = Z.shape[0]
    # Sample a small but balanced-ish set of probe points.
    sample_indices = []
    for tid in sorted(torch.unique(T).tolist()):
        idx = torch.where(T == int(tid))[0]
        if len(idx) > 0:
            sample_indices.extend(idx[:min(4 if RUN_MODE == "smoke" else 8,
    ↪ len(idx))].tolist())
    sample_indices = sample_indices[:min(len(sample_indices), 16 if RUN_MODE ==
    ↪ "smoke" else 40)]
    summary_rows, best_traces, all_traces = [], [], []
    for j, idx in enumerate(sample_indices):
        s, best_trace, trace_all = optimize_context_probe(
            theta, Z[idx], M[idx], C[idx], context_bank, sample_id=j, seed=seed
    ↪ * 1000 + j
        )
        s.update({"seed": seed, "task_id": int(T[idx]), "energy_probe_version":
    ↪ ENERGY_PROBE_VERSION})
        summary_rows.append(s)
        best_trace["seed"] = seed
        best_trace["task_id"] = int(T[idx])

```

```

        best_trace["trace_kind"] = "best_restart"
        trace_all["seed"] = seed
        trace_all["task_id"] = int(T[idx])
        best_traces.append(best_trace)
        all_traces.append(trace_all)
        sample_df = pd.DataFrame(summary_rows)
        trace_df = pd.concat(all_traces, ignore_index=True) if all_traces else pd.
↪DataFrame()
        best_trace_df = pd.concat(best_traces, ignore_index=True) if best_traces
↪else pd.DataFrame()
        if not sample_df.empty:
            agg = {
                "seed": seed,
                "energy_probe_version": ENERGY_PROBE_VERSION,
                "run_mode": RUN_MODE,
                "dataset_name": DATASET_NAME,
                "n_probe_samples": int(len(sample_df)),
                "oracle_hit_rate": float(sample_df["oracle_hit"].mean()),
                "median_final_context_gap": float(sample_df["final_context_gap"].
↪median()),
                "mean_final_context_gap": float(sample_df["final_context_gap"].
↪mean()),
                "median_gap_ratio_vs_target":
↪float(sample_df["gap_ratio_vs_target"].median()),
                "mean_energy_drop": float(sample_df["energy_drop"].mean()),
                "plateau_rate": float((sample_df["diagnosis"] ==
↪"PLATEAU_FLAT_LANDSCAPE").mean()),
                "wrong_context_rate": float((sample_df["diagnosis"] ==
↪"LOCAL_MINIMUM_WRONG_CONTEXT").mean()),
                "divergence_rate": float((sample_df["diagnosis"] ==
↪"DIVERGENCE_OR_UNSTABLE").mean()),
                "success_rate": float((sample_df["diagnosis"] ==
↪"SUCCESS_INFORMATIVE_ENERGY").mean()),
            }
        else:
            agg = {"seed": seed, "energy_probe_version": ENERGY_PROBE_VERSION,
↪"n_probe_samples": 0}
            agg_df = pd.DataFrame([agg])
            sample_df.to_csv(probe_dir / "energy_probe_sample_summary.csv", index=False)
            trace_df.to_csv(probe_dir / "energy_probe_all_restarts_trace.csv",
↪index=False)
            best_trace_df.to_csv(probe_dir / "energy_probe_best_trace.csv", index=False)
            agg_df.to_csv(probe_dir / "energy_probe_seed_aggregate.csv", index=False)
            with open(probe_dir / "energy_probe_summary.json", "w") as f:
                json.dump({"aggregate": agg, "samples": summary_rows[:5]}, f, indent=2)
            if not best_trace_df.empty:

```

```

        plot_energy_probe_trace(
            best_trace_df[best_trace_df.sample_id == int(best_trace_df.
↪sample_id.iloc[0])],
            probe_dir / "energy_probe_trace_first_sample.png",
            title=f"Energy Context Probe seed={seed}"
        )
    print("Energy Context Probe aggregate:")
    display(agg_df)
    return agg_df, sample_df, trace_df

```

```

[ ]: start_persistent_logging()
log_event("experiment_started", payload={
    "rc2c_candidate_variants": RC2B_CANDIDATE_VARIANTS,
    "baseline_methods": SELECTED_BASELINE_METHODS,
    "strict_no_final_test_selection": RC2B_STRICT_NO_FINAL_TEST_SELECTION,
    "experiment_profile": EXPERIMENT_PROFILE,
    "policy_branch": POLICY_BRANCH,
    "hydro_audit_enabled": RC3_HYDRO_AUDIT_ENABLED,
    "hydro_regularizer_enabled": RC3_HYDRO_REGULARIZER_ENABLED,
    "hydro_regularizer_mode": RC3_HYDRO_REGULARIZER_MODE,
    "hydro_selection_enabled": RC3_HYDRO_SELECTION_ENABLED,
    "hydro_policy_mode": RC3_HYDRO_POLICY_MODE,
    "learning_signal_alignment_enabled": ENABLE_LEARNING_SIGNAL_ALIGNMENT_AUDIT,
})
experiment_t0 = time.time()

all_history, all_losses, all_confusions, all_probe_losses, all_repair_logs, ↵
↪all_selection_logs, all_score_dumps, all_lsa_logs = [], [], [], [], [], [], ↵
↪[], []
all_energy_probe_agg, all_energy_probe_summary, all_energy_probe_trace = [], ↵
↪[], []

for seed in SEEDS:
    seed_t0 = time.time()
    log_event("seed_started", seed=seed, ↵
↪phase="rc2c_risk_aware_base_proto_and_policy_selection")
    base_run = train_base_proto_first_checkpoints(seed)
    all_losses.append(base_run["loss_df"])
    if isinstance(base_run.get("lsa_df", None), pd.DataFrame) and not ↵
↪base_run["lsa_df"].empty:
        all_lsa_logs.append(base_run["lsa_df"])

    for checkpoint in base_run["checkpoints"]:
        selected_h, selected_c, candidate_h, candidate_c, rlog, slog, ↵
↪score_dump = apply_rc2b_strict_policy_to_checkpoint(base_run, checkpoint)
        all_history.append(selected_h)

```

```

        if not candidate_h.empty:
            all_history.append(candidate_h)
        if not selected_c.empty:
            all_confusions.append(selected_c)
        if not candidate_c.empty:
            all_confusions.append(candidate_c)
        if not rlog.empty:
            all_repair_logs.append(rlog)
        if not slog.empty:
            all_selection_logs.append(slog)
        if not score_dump.empty:
            all_score_dumps.append(score_dump)
        log_event(
            "rc2b_policy_selected",
            seed=int(checkpoint["seed"]),
            task_id=int(checkpoint["after_task"]),
            method=RC2B_METHOD,
            phase="policy_selection_before_final_test",
            payload=slog[slog.get("is_selected_candidate", False) == True].
↪to_dict("records")[:1] if not slog.empty else {},
        )

    if RUN_FINAL_PROBES:
        artifact = final_probe_artifact_from_base_run(base_run)
        ph, pc, pl = run_representation_audit(artifact)
        all_history.append(ph)
        all_confusions.append(pc)
        all_probe_losses.append(pl)
        del artifact
        # RC2jh audit-only Energy Context Probe.
        # Uses replay/memory artifacts only; does not affect training or selector
↪decisions.
        if ACTIVATE_ENERGY_CONTEXT_PROBE:
            try:
                log_event("energy_context_probe_started", seed=seed,
↪phase="energy_context_probe_audit")
            except Exception:
                pass
            artifact_ep = final_probe_artifact_from_base_run(base_run)
            ep_agg, ep_summary, ep_trace =
↪run_energy_context_probe_from_artifact(artifact_ep)
            if ep_agg is not None and not ep_agg.empty:
                all_energy_probe_agg.append(ep_agg)
            if ep_summary is not None and not ep_summary.empty:
                all_energy_probe_summary.append(ep_summary)
            if ep_trace is not None and not ep_trace.empty:
                all_energy_probe_trace.append(ep_trace)

```

```

        try:
            log_event("energy_context_probe_completed", seed=seed,
↪phase="energy_context_probe_audit", payload=ep_agg.to_dict("records")[:1] if
↪ep_agg is not None and not ep_agg.empty else {})
            except Exception:
                pass
        del artifact_ep

    del base_run
    clear_memory()

    # Comparative baseline suite on the same seed and task construction.
    base_histories, base_losses, base_confusions =
↪run_benchmark_baselines_for_seed(seed)
    all_history.extend(base_histories)
    all_losses.extend(base_losses)
    all_confusions.extend(base_confusions)
    clear_memory()
    log_event("seed_completed", seed=seed,
↪phase="rc2c_risk_aware_base_proto_and_policy_selection",
↪payload={"duration_sec": time.time() - seed_t0})

history_df = pd.concat(all_history, ignore_index=True) if all_history else pd.
↪DataFrame()
loss_df = pd.concat(all_losses, ignore_index=True) if all_losses else pd.
↪DataFrame()
repair_log_df = pd.concat(all_repair_logs, ignore_index=True) if
↪all_repair_logs else pd.DataFrame()
selection_log_df = pd.concat(all_selection_logs, ignore_index=True) if
↪all_selection_logs else pd.DataFrame()

# RC2JF audit-continuity patch: restore neutral RC2I dual-anchor columns
# before CSV/ZIP export, even if older/cached rows did not contain them.
def ensure_rc2i_dual_anchor_audit_columns(df: pd.DataFrame) -> pd.DataFrame:
    if df is None or df.empty:
        return df
    defaults = {
        "rc2i_safe_anchor_variant": "not_active_rc2jf",
        "rc2i_override_rejected_reason": "not_active_rc2jf",
        "rc2i_context_only_override_rejected": False,
    }
    for col, default in defaults.items():
        if col not in df.columns:
            df[col] = default
        else:
            df[col] = df[col].fillna(default)

```



```

    df["rc2i_context_only_override_rejected"] =
    ↪df["rc2i_context_only_override_rejected"].astype(bool)
    return df

selection_log_df = ensure_rc2i_dual_anchor_audit_columns(selection_log_df)
confusion_df = pd.concat(all_confusions, ignore_index=True) if all_confusions
    ↪else pd.DataFrame()
probe_loss_df = pd.concat(all_probe_losses, ignore_index=True) if
    ↪all_probe_losses else pd.DataFrame()
score_dump_df = pd.concat(all_score_dumps, ignore_index=True) if
    ↪all_score_dumps else pd.DataFrame()
learning_signal_alignment_df = pd.concat(all_lsa_logs, ignore_index=True) if
    ↪all_lsa_logs else pd.DataFrame()
learning_signal_alignment_summary_df =
    ↪summarize_learning_signal_alignment(learning_signal_alignment_df) if not
    ↪learning_signal_alignment_df.empty else pd.DataFrame()

deployed_history = history_df[(history_df.probe_row == False)] if not
    ↪history_df.empty and "probe_row" in history_df.columns else history_df
forgetting_df = forgetting_from_history(deployed_history) if not
    ↪deployed_history.empty else pd.DataFrame()
class_diag_df = class_diagnostics_from_confusion(confusion_df) if not
    ↪confusion_df.empty else pd.DataFrame()

history_path = RESULTS_DIR / f"history_{RUN_MODE}.csv"
loss_path = RESULTS_DIR / f"losses_{RUN_MODE}.csv"
repair_log_path = RESULTS_DIR / f"context_gap_repair_logs_{RUN_MODE}.csv"
selection_log_path = RESULTS_DIR / f"rc2b_policy_selection_trace_{RUN_MODE}.csv"
probe_loss_path = RESULTS_DIR / f"probe_losses_{RUN_MODE}.csv"
forgetting_path = RESULTS_DIR / f"forgetting_{RUN_MODE}.csv"
confusion_path = RESULTS_DIR / f"class_confusion_{RUN_MODE}.csv"
class_diag_path = RESULTS_DIR / f"class_diagnostics_{RUN_MODE}.csv"
score_dump_path = RESULTS_DIR / f"biometric_style_score_dump_{RUN_MODE}.csv"
learning_signal_alignment_path = RESULTS_DIR /
    ↪f"learning_signal_alignment_panel_e_{RUN_MODE}.csv"
learning_signal_alignment_summary_path = RESULTS_DIR /
    ↪f"learning_signal_alignment_panel_e_summary_{RUN_MODE}.csv"
threshold_curve_dir = RESULTS_DIR / "threshold_curves"
threshold_plot_dir = PLOTS_DIR / "threshold_diagnostics"
threshold_curve_dir.mkdir(parents=True, exist_ok=True)
threshold_plot_dir.mkdir(parents=True, exist_ok=True)
threshold_curves_path = threshold_curve_dir /
    ↪f"biometric_style_far_frr_roc_det_curves_{RUN_MODE}.csv"
eer_by_class_path = threshold_curve_dir /
    ↪f"biometric_style_eer_by_class_{RUN_MODE}.csv"

```

```

eer_by_method_path = threshold_curve_dir / ␣
␣f"biometric_style_eer_by_method_{RUN_MODE}.csv"

history_df.to_csv(history_path, index=False)
loss_df.to_csv(loss_path, index=False)
repair_log_df.to_csv(repair_log_path, index=False)
selection_log_df.to_csv(selection_log_path, index=False)
probe_loss_df.to_csv(probe_loss_path, index=False)
forgetting_df.to_csv(forgetting_path, index=False)
confusion_df.to_csv(confusion_path, index=False)
class_diag_df.to_csv(class_diag_path, index=False)
if not learning_signal_alignment_df.empty:
    learning_signal_alignment_df.to_csv(learning_signal_alignment_path, ␣
␣index=False)
    learning_signal_alignment_summary_df.
␣to_csv(learning_signal_alignment_summary_path, index=False)

# Biometric-style threshold diagnostics from post-selection final-test scores.
threshold_curves_df, eer_by_class_df, eer_by_method_df = pd.DataFrame(), pd.
␣DataFrame(), pd.DataFrame()
threshold_plot_paths = []

if globals().get("ENABLE_BIOMETRIC_STYLE_THRESHOLD_DIAGNOSTICS", False):
    if score_dump_df.empty:
        msg = (
            "Biometric-style threshold diagnostics are enabled, but ␣
␣score_dump_df is empty. "
            "This usually means ENABLE_BIOMETRIC_STYLE_THRESHOLD_DIAGNOSTICS ␣
␣was not set before policy evaluation, "
            "or no final checkpoint was evaluated. Re-run the notebook from the ␣
␣configuration cell."
        )
        if globals().get("THRESHOLD_DIAGNOSTICS_STRICT_EXPORT", True):
            raise RuntimeError(msg)
        print("WARNING:", msg)
    else:
        score_dump_df.to_csv(score_dump_path, index=False)
        threshold_curves_df, eer_by_class_df, eer_by_method_df = ␣
␣compute_biometric_style_threshold_diagnostics(
            score_dump_df,
            focus_classes=list(range(N_CLASSES)),
            grid_size=globals().get("THRESHOLD_DIAGNOSTICS_GRID_SIZE", 501),
        )
        threshold_curves_df.to_csv(threshold_curves_path, index=False)
        eer_by_class_df.to_csv(eer_by_class_path, index=False)
        eer_by_method_df.to_csv(eer_by_method_path, index=False)

```

```

        threshold_plot_paths = plot_biometric_style_threshold_diagnostics(
            threshold_curves_df, eer_by_method_df,
            out_dir=threshold_plot_dir,
            focus_classes=globals().get("THRESHOLD_DIAGNOSTICS_FOCUS_CLASSES",
    ↪[2, 4, 5]),
        )
        if threshold_curves_df.empty or eer_by_class_df.empty or
    ↪eer_by_method_df.empty:
            msg = "Threshold diagnostics ran, but at least one generated table
    ↪is empty. Check score columns and class coverage."
            if globals().get("THRESHOLD_DIAGNOSTICS_STRICT_EXPORT", True):
                raise RuntimeError(msg)
            print("WARNING:", msg)
            print("Biometric-style threshold diagnostics saved:")
            for p in [score_dump_path, threshold_curves_path, eer_by_class_path,
    ↪eer_by_method_path] + list(threshold_plot_paths):
                print(" -", p)
    else:
        print("Biometric-style threshold diagnostics disabled; no score dump / EER
    ↪artifacts generated.")

    print("Saved:")
    _saved_core_paths = [history_path, loss_path, repair_log_path,
    ↪selection_log_path, probe_loss_path, forgetting_path, confusion_path,
    ↪class_diag_path]
    if not learning_signal_alignment_df.empty:
        _saved_core_paths += [learning_signal_alignment_path,
    ↪learning_signal_alignment_summary_path]
    if not score_dump_df.empty:
        _saved_core_paths += [score_dump_path, threshold_curves_path,
    ↪eer_by_class_path, eer_by_method_path]
    for p in _saved_core_paths:
        print(" -", p)
    if not selection_log_df.empty:
        print("RC2b selected policy counts:")
        display(selection_log_df[selection_log_df["is_selected_candidate"] == True].
    ↪groupby(["selected_candidate_variant", "selected_policy_family"],
    ↪as_index=False).size())
    log_event("experiment_core_results_saved", phase="export",
    ↪payload={"duration_sec": time.time() - experiment_t0, "files": [str(p) for p
    ↪in _saved_core_paths]})
    # Keep logging active for interpretation/audit cells; it will be stopped in the
    ↪final packaging cell.

    if "eer_by_method_df" in globals() and not eer_by_method_df.empty:

```

```

print("Biometric-style EER summary by method, lower is better:")
display(eer_by_method_df.sort_values(["selected_policy", ↵
↵ "weak_boundary_eer_2_4_5"], ascending=[False, True]).head(20))
print("Threshold diagnostic plots:")
for p in threshold_plot_paths:
    print(" -", p)

```

```

[base_proto seed=0] task=0 epoch=1/8 loss=1.9982 task=0.9697 memory=0.0000
balanced=0.0000 lsa=0.991
[base_proto seed=0] task=0 epoch=2/8 loss=0.6132 task=0.3132 memory=0.0000
balanced=0.0000 lsa=1.000
[base_proto seed=0] task=0 epoch=3/8 loss=0.5025 task=0.2561 memory=0.0000
balanced=0.0000 lsa=1.000
[base_proto seed=0] task=0 epoch=4/8 loss=0.4533 task=0.2236 memory=0.0000
balanced=0.0000 lsa=1.000
[base_proto seed=0] task=0 epoch=5/8 loss=0.4301 task=0.2097 memory=0.0000
balanced=0.0000 lsa=1.000
[base_proto seed=0] task=0 epoch=6/8 loss=0.4155 task=0.2039 memory=0.0000
balanced=0.0000 lsa=1.000
[base_proto seed=0] task=0 epoch=7/8 loss=0.4028 task=0.2001 memory=0.0000
balanced=0.0000 lsa=1.000
[base_proto seed=0] task=0 epoch=8/8 loss=0.3904 task=0.1956 memory=0.0000
balanced=0.0000 lsa=1.000
[base_proto seed=0] calibrated context temperature after task 0: 0.25
[base_proto seed=0] task=1 epoch=1/8 loss=5.9971 task=1.6252 memory=0.6382
balanced=0.2744 lsa=1.000
[base_proto seed=0] task=1 epoch=2/8 loss=2.9186 task=0.5433 memory=0.2923
balanced=0.2405 lsa=1.000
[base_proto seed=0] task=1 epoch=3/8 loss=2.2425 task=0.3952 memory=0.1251
balanced=0.2533 lsa=1.000
[base_proto seed=0] task=1 epoch=4/8 loss=2.0118 task=0.3610 memory=0.0787
balanced=0.2713 lsa=1.000
[base_proto seed=0] task=1 epoch=5/8 loss=1.8401 task=0.3361 memory=0.0641
balanced=0.2382 lsa=1.000
[base_proto seed=0] task=1 epoch=6/8 loss=1.7256 task=0.3140 memory=0.0538
balanced=0.2066 lsa=1.000
[base_proto seed=0] task=1 epoch=7/8 loss=1.7098 task=0.2994 memory=0.0507
balanced=0.2413 lsa=1.000
[base_proto seed=0] task=1 epoch=8/8 loss=1.6227 task=0.2966 memory=0.0541
balanced=0.2014 lsa=0.999
[base_proto seed=0] calibrated context temperature after task 1: 0.80
[base_proto seed=0] task=2 epoch=1/8 loss=5.3338 task=1.6420 memory=0.4796
balanced=0.3281 lsa=0.999
[base_proto seed=0] task=2 epoch=2/8 loss=3.1753 task=0.6899 memory=0.3581
balanced=0.3122 lsa=0.982
[base_proto seed=0] task=2 epoch=3/8 loss=2.4895 task=0.4932 memory=0.1913
balanced=0.2910 lsa=0.979

```

```

[base_proto seed=0] task=2 epoch=4/8 loss=2.0783 task=0.3543 memory=0.1259
balanced=0.2568 lsa=0.978
[base_proto seed=0] task=2 epoch=5/8 loss=1.8385 task=0.3098 memory=0.0788
balanced=0.2311 lsa=0.968
[base_proto seed=0] task=2 epoch=6/8 loss=1.6558 task=0.2775 memory=0.0535
balanced=0.1757 lsa=0.958
[base_proto seed=0] task=2 epoch=7/8 loss=1.6173 task=0.2648 memory=0.0523
balanced=0.1784 lsa=0.902
[base_proto seed=0] task=2 epoch=8/8 loss=1.5575 task=0.2679 memory=0.0538
balanced=0.1815 lsa=0.897
[base_proto seed=0] calibrated context temperature after task 2: 0.60
[base_proto seed=0] task=3 epoch=1/8 loss=6.0587 task=2.1681 memory=1.0736
balanced=0.3271 lsa=0.969
[base_proto seed=0] task=3 epoch=2/8 loss=3.6216 task=1.0233 memory=0.7329
balanced=0.2342 lsa=0.952
[base_proto seed=0] task=3 epoch=3/8 loss=3.0602 task=0.7107 memory=0.6900
balanced=0.2503 lsa=0.972
[base_proto seed=0] task=3 epoch=4/8 loss=2.8869 task=0.5717 memory=0.7975
balanced=0.2071 lsa=0.969
[base_proto seed=0] task=3 epoch=5/8 loss=2.4093 task=0.5095 memory=0.4575
balanced=0.1914 lsa=0.970
[base_proto seed=0] task=3 epoch=6/8 loss=2.2695 task=0.4645 memory=0.4097
balanced=0.1946 lsa=0.974
[base_proto seed=0] task=3 epoch=7/8 loss=2.1135 task=0.4516 memory=0.3104
balanced=0.1933 lsa=0.967
[base_proto seed=0] task=3 epoch=8/8 loss=1.9416 task=0.4018 memory=0.2489
balanced=0.1720 lsa=0.956
[base_proto seed=0] calibrated context temperature after task 3: 0.60
[base_proto seed=0] task=4 epoch=1/8 loss=3.9757 task=1.3421 memory=0.3556
balanced=0.5376 lsa=0.970
[base_proto seed=0] task=4 epoch=2/8 loss=2.3971 task=0.3832 memory=0.1746
balanced=0.4670 lsa=0.884
[base_proto seed=0] task=4 epoch=3/8 loss=2.4329 task=0.3768 memory=0.3868
balanced=0.4598 lsa=0.848
[base_proto seed=0] task=4 epoch=4/8 loss=2.1288 task=0.3248 memory=0.2532
balanced=0.4398 lsa=0.843
[base_proto seed=0] task=4 epoch=5/8 loss=1.9261 task=0.3061 memory=0.1862
balanced=0.4003 lsa=0.842
[base_proto seed=0] task=4 epoch=6/8 loss=1.8328 task=0.2914 memory=0.2039
balanced=0.3756 lsa=0.841
[base_proto seed=0] task=4 epoch=7/8 loss=1.6287 task=0.2769 memory=0.0716
balanced=0.3605 lsa=0.839
[base_proto seed=0] task=4 epoch=8/8 loss=1.6770 task=0.2687 memory=0.1635
balanced=0.3570 lsa=0.859
[base_proto seed=0] calibrated context temperature after task 4: 0.60
[base_proto seed=0] completed in 199.4s
[RC2m_beta seed=0] after task 0: selected proto_global_no_repair
proxy_acc=0.9043 score=1.2109

```

```

[RC2m_beta seed=0] after task 1: selected
generic_weak_boundary_latent_calibrated proxy_acc=0.9234 score=1.7550
[RC2m_beta seed=0] after task 2: selected
context_temp_blend_selected_head_guard_035 proxy_acc=0.9316 score=1.7178
[RC2m_beta seed=0] after task 3: selected
context_temp_blend_selected_head_guard_035 proxy_acc=0.8781 score=1.2787
[RC2m_beta seed=0] after task 4: selected proto_global_head_ce_kl_guard_035
proxy_acc=0.9352 score=1.4778
[probe_proto__zf_proto seed=0] probe final task accs: [0.935, 0.931, 0.86,
0.833, 0.932]
[probe_proto__zf_oracle seed=0] probe final task accs: [0.96, 0.896, 0.95,
0.899, 0.919]
[probe_proto__transformed0_oracle seed=0] host probe accs: [0.767, 0.591, 0.315,
0.554, 0.796]
[probe_proto__transformed1_oracle seed=0] host probe accs: [0.73, 0.78, 0.757,
0.691, 0.814]
[probe_proto__transformed2_oracle seed=0] host probe accs: [0.564, 0.864, 0.715,
0.268, 0.481]
[probe_proto__transformed3_oracle seed=0] host probe accs: [0.9, 0.866, 0.799,
0.859, 0.887]
[probe_proto__transformed4_oracle seed=0] host probe accs: [0.787, 0.771, 0.344,
0.246, 0.748]
Energy Context Probe aggregate:

      seed          energy_probe_version run_mode  dataset_name  \
0      0 RC2jh_energy_context_probe_audit_v1  robust FashionMNIST

      n_probe_samples  oracle_hit_rate  median_final_context_gap  \
0              40              0.95              1.0872

      mean_final_context_gap  median_gap_ratio_vs_target  mean_energy_drop  \
0              1.637415              37.489649              8.278513

      plateau_rate  wrong_context_rate  divergence_rate  success_rate
0              0.025              0.0              0.0              0.95

[baseline_finetune seed=0] after task 0: [0.981]
[baseline_finetune seed=0] after task 1: [0.486, 0.99]
[baseline_finetune seed=0] after task 2: [0.0, 0.487, 0.961]
[baseline_finetune seed=0] after task 3: [0.0, 0.0, 0.477, 0.941]
[baseline_finetune seed=0] after task 4: [0.0, 0.0, 0.0, 0.5, 1.0]
[baseline_ewc seed=0] after task 0: [0.981]
[baseline_ewc seed=0] after task 1: [0.489, 0.989]
[baseline_ewc seed=0] after task 2: [0.0, 0.487, 0.964]
[baseline_ewc seed=0] after task 3: [0.0, 0.0, 0.485, 0.943]
[baseline_ewc seed=0] after task 4: [0.0, 0.0, 0.0, 0.5, 1.0]
[baseline_lwf seed=0] after task 0: [0.981]
[baseline_lwf seed=0] after task 1: [0.96, 0.515]
[baseline_lwf seed=0] after task 2: [0.907, 0.974, 0.502]

```

[baseline_lwf seed=0] after task 3: [0.253, 0.699, 0.92, 0.555]
 [baseline_lwf seed=0] after task 4: [0.007, 0.005, 0.388, 0.876, 0.985]
 [baseline_experience_replay seed=0] after task 0: [0.981]
 [baseline_experience_replay seed=0] after task 1: [0.96, 0.971]
 [baseline_experience_replay seed=0] after task 2: [0.886, 0.959, 0.92]
 [baseline_experience_replay seed=0] after task 3: [0.874, 0.825, 0.809, 0.892]
 [baseline_experience_replay seed=0] after task 4: [0.9, 0.756, 0.686, 0.856, 0.968]
 [baseline_balanced_replay seed=0] after task 0: [0.981]
 [baseline_balanced_replay seed=0] after task 1: [0.96, 0.971]
 [baseline_balanced_replay seed=0] after task 2: [0.886, 0.959, 0.92]
 [baseline_balanced_replay seed=0] after task 3: [0.874, 0.825, 0.809, 0.892]
 [baseline_balanced_replay seed=0] after task 4: [0.9, 0.756, 0.686, 0.856, 0.968]
 [baseline_sparse_moe seed=0] after task 0: [0.978]
 [baseline_sparse_moe seed=0] after task 1: [0.485, 0.99]
 [baseline_sparse_moe seed=0] after task 2: [0.0, 0.487, 0.964]
 [baseline_sparse_moe seed=0] after task 3: [0.0, 0.0, 0.474, 0.949]
 [baseline_sparse_moe seed=0] after task 4: [0.0, 0.0, 0.0, 0.5, 0.999]
 [baseline_pnn seed=0] after task 0: [0.988]
 [baseline_pnn seed=0] after task 1: [0.481, 0.984]
 [baseline_pnn seed=0] after task 2: [0.0, 0.486, 0.96]
 [baseline_pnn seed=0] after task 3: [0.0, 0.0, 0.492, 0.924]
 [baseline_pnn seed=0] after task 4: [0.0, 0.0, 0.0, 0.496, 0.998]
 [baseline_joint_upper_bound seed=0] final joint eval: [0.924, 0.836, 0.795, 0.891, 0.915]
 [base_proto seed=1] task=0 epoch=1/8 loss=2.2393 task=0.9040 memory=0.0000
 balanced=0.0000 lsa=0.929
 [base_proto seed=1] task=0 epoch=2/8 loss=0.6105 task=0.2822 memory=0.0000
 balanced=0.0000 lsa=0.825
 [base_proto seed=1] task=0 epoch=3/8 loss=0.4889 task=0.2299 memory=0.0000
 balanced=0.0000 lsa=0.823
 [base_proto seed=1] task=0 epoch=4/8 loss=0.4348 task=0.1996 memory=0.0000
 balanced=0.0000 lsa=0.873
 [base_proto seed=1] task=0 epoch=5/8 loss=0.4193 task=0.1959 memory=0.0000
 balanced=0.0000 lsa=0.862
 [base_proto seed=1] task=0 epoch=6/8 loss=0.3992 task=0.1845 memory=0.0000
 balanced=0.0000 lsa=0.807
 [base_proto seed=1] task=0 epoch=7/8 loss=0.3903 task=0.1841 memory=0.0000
 balanced=0.0000 lsa=0.844
 [base_proto seed=1] task=0 epoch=8/8 loss=0.3758 task=0.1762 memory=0.0000
 balanced=0.0000 lsa=0.809
 [base_proto seed=1] calibrated context temperature after task 0: 0.25
 [base_proto seed=1] task=1 epoch=1/8 loss=6.4131 task=1.4879 memory=0.5950
 balanced=0.2143 lsa=0.856
 [base_proto seed=1] task=1 epoch=2/8 loss=3.4552 task=0.5878 memory=0.2650
 balanced=0.2296 lsa=0.877
 [base_proto seed=1] task=1 epoch=3/8 loss=2.7261 task=0.4016 memory=0.1468

balanced=0.1950 lsa=0.922
 [base_proto seed=1] task=1 epoch=4/8 loss=2.3917 task=0.3514 memory=0.0924
 balanced=0.1760 lsa=0.832
 [base_proto seed=1] task=1 epoch=5/8 loss=2.1689 task=0.3279 memory=0.0589
 balanced=0.1633 lsa=0.850
 [base_proto seed=1] task=1 epoch=6/8 loss=2.0627 task=0.3088 memory=0.0486
 balanced=0.1656 lsa=0.832
 [base_proto seed=1] task=1 epoch=7/8 loss=1.9546 task=0.2907 memory=0.0390
 balanced=0.1494 lsa=0.879
 [base_proto seed=1] task=1 epoch=8/8 loss=1.9031 task=0.2793 memory=0.0369
 balanced=0.1377 lsa=0.884
 [base_proto seed=1] calibrated context temperature after task 1: 0.80
 [base_proto seed=1] task=2 epoch=1/8 loss=5.4834 task=1.4050 memory=0.4411
 balanced=0.2619 lsa=0.912
 [base_proto seed=1] task=2 epoch=2/8 loss=3.2329 task=0.5008 memory=0.2680
 balanced=0.2641 lsa=0.843
 [base_proto seed=1] task=2 epoch=3/8 loss=2.5083 task=0.3895 memory=0.1401
 balanced=0.2197 lsa=0.766
 [base_proto seed=1] task=2 epoch=4/8 loss=2.2068 task=0.3494 memory=0.1153
 balanced=0.2033 lsa=0.821
 [base_proto seed=1] task=2 epoch=5/8 loss=2.0123 task=0.3221 memory=0.0985
 balanced=0.1881 lsa=0.784
 [base_proto seed=1] task=2 epoch=6/8 loss=1.8730 task=0.3002 memory=0.0895
 balanced=0.1724 lsa=0.807
 [base_proto seed=1] task=2 epoch=7/8 loss=1.7574 task=0.2895 memory=0.0819
 balanced=0.1617 lsa=0.871
 [base_proto seed=1] task=2 epoch=8/8 loss=1.6584 task=0.2742 memory=0.0742
 balanced=0.1426 lsa=0.676
 [base_proto seed=1] calibrated context temperature after task 2: 0.60
 [base_proto seed=1] task=3 epoch=1/8 loss=5.8071 task=2.0950 memory=0.8113
 balanced=0.2983 lsa=0.968
 [base_proto seed=1] task=3 epoch=2/8 loss=3.8773 task=1.0068 memory=0.7247
 balanced=0.2882 lsa=0.964
 [base_proto seed=1] task=3 epoch=3/8 loss=3.1464 task=0.7732 memory=0.5837
 balanced=0.2866 lsa=0.892
 [base_proto seed=1] task=3 epoch=4/8 loss=2.6342 task=0.6241 memory=0.3976
 balanced=0.2547 lsa=0.902
 [base_proto seed=1] task=3 epoch=5/8 loss=2.2963 task=0.5022 memory=0.3089
 balanced=0.2198 lsa=0.982
 [base_proto seed=1] task=3 epoch=6/8 loss=2.0697 task=0.4341 memory=0.2369
 balanced=0.1916 lsa=0.967
 [base_proto seed=1] task=3 epoch=7/8 loss=1.9027 task=0.3826 memory=0.2012
 balanced=0.1604 lsa=0.978
 [base_proto seed=1] task=3 epoch=8/8 loss=1.7993 task=0.3524 memory=0.1573
 balanced=0.1778 lsa=0.946
 [base_proto seed=1] calibrated context temperature after task 3: 0.60
 [base_proto seed=1] task=4 epoch=1/8 loss=5.2031 task=1.9321 memory=0.7822
 balanced=0.4425 lsa=0.942


```

[base_proto seed=1] task=4 epoch=2/8 loss=3.0526 task=0.8186 memory=0.2713
balanced=0.3212 lsa=0.922
[base_proto seed=1] task=4 epoch=3/8 loss=2.2470 task=0.3418 memory=0.1905
balanced=0.2888 lsa=0.977
[base_proto seed=1] task=4 epoch=4/8 loss=2.0519 task=0.3002 memory=0.1344
balanced=0.3078 lsa=0.970
[base_proto seed=1] task=4 epoch=5/8 loss=1.8827 task=0.2602 memory=0.1494
balanced=0.2663 lsa=0.969
[base_proto seed=1] task=4 epoch=6/8 loss=1.8056 task=0.2458 memory=0.1664
balanced=0.2639 lsa=0.971
[base_proto seed=1] task=4 epoch=7/8 loss=1.8055 task=0.2276 memory=0.1856
balanced=0.3248 lsa=0.940
[base_proto seed=1] task=4 epoch=8/8 loss=1.5281 task=0.2165 memory=0.1127
balanced=0.1789 lsa=0.767
[base_proto seed=1] calibrated context temperature after task 4: 0.60
[base_proto seed=1] completed in 209.6s
[RC2m_beta seed=1] after task 0: selected proto_global_head_ce_kl_guard_035
proxy_acc=0.9402 score=1.2619
[RC2m_beta seed=1] after task 1: selected
generic_weak_boundary_latent_calibrated proxy_acc=0.9245 score=1.7664
[RC2m_beta seed=1] after task 2: selected
context_temp_blend_selected_head_guard_035 proxy_acc=0.9238 score=1.6794
[RC2m_beta seed=1] after task 3: selected
context_temp_blend_selected_head_guard_035 proxy_acc=0.9333 score=1.7031
[RC2m_beta seed=1] after task 4: selected context_blend_selected_global
proxy_acc=0.9538 score=1.6735
[probe_proto__zf_proto seed=1] probe final task accs: [0.976, 0.929, 0.874,
0.885, 0.909]
[probe_proto__zf_oracle seed=1] probe final task accs: [0.974, 0.958, 0.948,
0.935, 0.943]
[probe_proto__transformed0_oracle seed=1] host probe accs: [0.623, 0.632, 0.879,
0.444, 0.5]
[probe_proto__transformed1_oracle seed=1] host probe accs: [0.779, 0.886, 0.419,
0.191, 0.656]
[probe_proto__transformed2_oracle seed=1] host probe accs: [0.766, 0.64, 0.451,
0.674, 0.899]
[probe_proto__transformed3_oracle seed=1] host probe accs: [0.83, 0.838, 0.814,
0.844, 0.912]
[probe_proto__transformed4_oracle seed=1] host probe accs: [0.782, 0.601, 0.152,
0.207, 0.703]
Energy Context Probe aggregate:
      seed          energy_probe_version run_mode  dataset_name  \
0      1  RC2jh_energy_context_probe_audit_v1  robust  FashionMNIST

      n_probe_samples  oracle_hit_rate  median_final_context_gap  \
0              40              0.9              0.844832

```

	mean_final_context_gap	median_gap_ratio_vs_target	mean_energy_drop	\
0	1.724728	29.132148	8.436279	

	plateau_rate	wrong_context_rate	divergence_rate	success_rate
0	0.025	0.0	0.0	0.9

[baseline_finetune seed=1] after task 0: [0.98]
 [baseline_finetune seed=1] after task 1: [0.487, 0.985]
 [baseline_finetune seed=1] after task 2: [0.0, 0.477, 0.961]
 [baseline_finetune seed=1] after task 3: [0.0, 0.0, 0.464, 0.943]
 [baseline_finetune seed=1] after task 4: [0.0, 0.0, 0.0, 0.5, 0.999]
 [baseline_ewc seed=1] after task 0: [0.98]
 [baseline_ewc seed=1] after task 1: [0.492, 0.978]
 [baseline_ewc seed=1] after task 2: [0.0, 0.484, 0.956]
 [baseline_ewc seed=1] after task 3: [0.0, 0.0, 0.464, 0.948]
 [baseline_ewc seed=1] after task 4: [0.0, 0.0, 0.0, 0.5, 0.999]
 [baseline_lwf seed=1] after task 0: [0.98]
 [baseline_lwf seed=1] after task 1: [0.97, 0.505]
 [baseline_lwf seed=1] after task 2: [0.925, 0.954, 0.52]
 [baseline_lwf seed=1] after task 3: [0.245, 0.62, 0.9, 0.614]
 [baseline_lwf seed=1] after task 4: [0.0, 0.0, 0.315, 0.814, 0.985]
 [baseline_experience_replay seed=1] after task 0: [0.98]
 [baseline_experience_replay seed=1] after task 1: [0.945, 0.966]
 [baseline_experience_replay seed=1] after task 2: [0.894, 0.927, 0.925]
 [baseline_experience_replay seed=1] after task 3: [0.87, 0.777, 0.802, 0.891]
 [baseline_experience_replay seed=1] after task 4: [0.922, 0.765, 0.635, 0.816, 0.97]
 [baseline_balanced_replay seed=1] after task 0: [0.98]
 [baseline_balanced_replay seed=1] after task 1: [0.945, 0.966]
 [baseline_balanced_replay seed=1] after task 2: [0.894, 0.927, 0.925]
 [baseline_balanced_replay seed=1] after task 3: [0.87, 0.777, 0.802, 0.891]
 [baseline_balanced_replay seed=1] after task 4: [0.922, 0.765, 0.635, 0.816, 0.97]
 [baseline_sparse_moe seed=1] after task 0: [0.975]
 [baseline_sparse_moe seed=1] after task 1: [0.492, 0.985]
 [baseline_sparse_moe seed=1] after task 2: [0.0, 0.486, 0.956]
 [baseline_sparse_moe seed=1] after task 3: [0.0, 0.0, 0.46, 0.944]
 [baseline_sparse_moe seed=1] after task 4: [0.0, 0.0, 0.0, 0.5, 1.0]
 [baseline_pnn seed=1] after task 0: [0.976]
 [baseline_pnn seed=1] after task 1: [0.487, 0.988]
 [baseline_pnn seed=1] after task 2: [0.0, 0.486, 0.956]
 [baseline_pnn seed=1] after task 3: [0.0, 0.0, 0.469, 0.948]
 [baseline_pnn seed=1] after task 4: [0.0, 0.0, 0.0, 0.5, 1.0]
 [baseline_joint_upper_bound seed=1] final joint eval: [0.911, 0.904, 0.875, 0.869, 0.846]
 [base_proto seed=2] task=0 epoch=1/8 loss=2.0397 task=0.9444 memory=0.0000
 balanced=0.0000 lsa=0.996
 [base_proto seed=2] task=0 epoch=2/8 loss=0.6053 task=0.3010 memory=0.0000

```

balanced=0.0000 lsa=1.000
[base_proto seed=2] task=0 epoch=3/8 loss=0.4877 task=0.2358 memory=0.0000
balanced=0.0000 lsa=1.000
[base_proto seed=2] task=0 epoch=4/8 loss=0.4408 task=0.2075 memory=0.0000
balanced=0.0000 lsa=1.000
[base_proto seed=2] task=0 epoch=5/8 loss=0.4283 task=0.2043 memory=0.0000
balanced=0.0000 lsa=1.000
[base_proto seed=2] task=0 epoch=6/8 loss=0.4158 task=0.2018 memory=0.0000
balanced=0.0000 lsa=1.000
[base_proto seed=2] task=0 epoch=7/8 loss=0.3971 task=0.1905 memory=0.0000
balanced=0.0000 lsa=1.000
[base_proto seed=2] task=0 epoch=8/8 loss=0.3865 task=0.1881 memory=0.0000
balanced=0.0000 lsa=1.000
[base_proto seed=2] calibrated context temperature after task 0: 0.25
[base_proto seed=2] task=1 epoch=1/8 loss=6.5111 task=1.5935 memory=0.6193
balanced=0.2447 lsa=0.997
[base_proto seed=2] task=1 epoch=2/8 loss=3.2524 task=0.4515 memory=0.2993
balanced=0.2131 lsa=1.000
[base_proto seed=2] task=1 epoch=3/8 loss=2.3255 task=0.3479 memory=0.1581
balanced=0.2302 lsa=1.000
[base_proto seed=2] task=1 epoch=4/8 loss=1.9621 task=0.3313 memory=0.1143
balanced=0.2331 lsa=1.000
[base_proto seed=2] task=1 epoch=5/8 loss=1.8228 task=0.3098 memory=0.0754
balanced=0.2259 lsa=1.000
[base_proto seed=2] task=1 epoch=6/8 loss=1.7436 task=0.2865 memory=0.0674
balanced=0.2095 lsa=1.000
[base_proto seed=2] task=1 epoch=7/8 loss=1.6431 task=0.2666 memory=0.0568
balanced=0.2044 lsa=1.000
[base_proto seed=2] task=1 epoch=8/8 loss=1.5738 task=0.2519 memory=0.0540
balanced=0.1921 lsa=1.000
[base_proto seed=2] calibrated context temperature after task 1: 0.80
[base_proto seed=2] task=2 epoch=1/8 loss=5.1642 task=1.5283 memory=0.3634
balanced=0.2751 lsa=1.000
[base_proto seed=2] task=2 epoch=2/8 loss=3.2974 task=0.7620 memory=0.3398
balanced=0.2502 lsa=0.996
[base_proto seed=2] task=2 epoch=3/8 loss=2.6253 task=0.4739 memory=0.2383
balanced=0.2683 lsa=0.993
[base_proto seed=2] task=2 epoch=4/8 loss=2.2398 task=0.3885 memory=0.1647
balanced=0.2367 lsa=0.983
[base_proto seed=2] task=2 epoch=5/8 loss=1.9636 task=0.3088 memory=0.1118
balanced=0.2171 lsa=0.968
[base_proto seed=2] task=2 epoch=6/8 loss=1.7815 task=0.2729 memory=0.0816
balanced=0.1952 lsa=0.905
[base_proto seed=2] task=2 epoch=7/8 loss=1.6984 task=0.2527 memory=0.0660
balanced=0.1896 lsa=0.922
[base_proto seed=2] task=2 epoch=8/8 loss=1.6223 task=0.2455 memory=0.0554
balanced=0.1871 lsa=0.955
[base_proto seed=2] calibrated context temperature after task 2: 0.80

```

```

[base_proto seed=2] task=3 epoch=1/8 loss=6.5100 task=2.3757 memory=1.1634
balanced=0.3128 lsa=0.947
[base_proto seed=2] task=3 epoch=2/8 loss=3.9558 task=1.1287 memory=0.8516
balanced=0.2534 lsa=0.964
[base_proto seed=2] task=3 epoch=3/8 loss=3.0964 task=0.7906 memory=0.5680
balanced=0.2549 lsa=0.944
[base_proto seed=2] task=3 epoch=4/8 loss=3.0956 task=0.6363 memory=0.8558
balanced=0.2349 lsa=0.975
[base_proto seed=2] task=3 epoch=5/8 loss=2.6420 task=0.5894 memory=0.5195
balanced=0.2347 lsa=0.971
[base_proto seed=2] task=3 epoch=6/8 loss=2.3708 task=0.5143 memory=0.3817
balanced=0.2194 lsa=0.975
[base_proto seed=2] task=3 epoch=7/8 loss=2.2239 task=0.4737 memory=0.3398
balanced=0.2033 lsa=0.960
[base_proto seed=2] task=3 epoch=8/8 loss=2.0686 task=0.4045 memory=0.3093
balanced=0.1774 lsa=0.968
[base_proto seed=2] calibrated context temperature after task 3: 0.60
[base_proto seed=2] task=4 epoch=1/8 loss=5.7209 task=1.4460 memory=1.6991
balanced=0.6096 lsa=0.960
[base_proto seed=2] task=4 epoch=2/8 loss=3.1962 task=0.4697 memory=0.7557
balanced=0.4623 lsa=0.958
[base_proto seed=2] task=4 epoch=3/8 loss=2.6306 task=0.4175 memory=0.5424
balanced=0.3565 lsa=0.926
[base_proto seed=2] task=4 epoch=4/8 loss=2.7051 task=0.3660 memory=0.7190
balanced=0.3633 lsa=0.916
[base_proto seed=2] task=4 epoch=5/8 loss=2.2111 task=0.3472 memory=0.4570
balanced=0.2739 lsa=0.887
[base_proto seed=2] task=4 epoch=6/8 loss=1.9362 task=0.3130 memory=0.2635
balanced=0.2935 lsa=0.943
[base_proto seed=2] task=4 epoch=7/8 loss=1.8509 task=0.2997 memory=0.2677
balanced=0.2827 lsa=0.935
[base_proto seed=2] task=4 epoch=8/8 loss=1.9189 task=0.2815 memory=0.3253
balanced=0.3328 lsa=0.916
[base_proto seed=2] calibrated context temperature after task 4: 1.00
[base_proto seed=2] completed in 216.2s
[RC2m_beta seed=2] after task 0: selected proto_global_no_repair
proxy_acc=0.9574 score=1.2842
[RC2m_beta seed=2] after task 1: selected
context_temp_blend_selected_head_guard_035 proxy_acc=0.9349 score=1.7627
[RC2m_beta seed=2] after task 2: selected
context_temp_blend_selected_head_guard_035 proxy_acc=0.9283 score=1.7289
[RC2m_beta seed=2] after task 3: selected proto_global_head_ce_kl_guard_035
proxy_acc=0.9113 score=1.6406
[RC2m_beta seed=2] after task 4: selected proto_global_head_ce_kl_guard_035
proxy_acc=0.9441 score=1.6010
[probe_proto_zf_proto seed=2] probe final task accs: [0.959, 0.953, 0.884,
0.92, 0.876]
[probe_proto_zf_oracle seed=2] probe final task accs: [0.97, 0.955, 0.966,

```

```

0.93, 0.931]
[probe_proto__transformed0_oracle seed=2] host probe accs: [0.807, 0.83, 0.733,
0.398, 0.546]
[probe_proto__transformed1_oracle seed=2] host probe accs: [0.713, 0.599, 0.65,
0.82, 0.799]
[probe_proto__transformed2_oracle seed=2] host probe accs: [0.886, 0.876, 0.828,
0.818, 0.906]
[probe_proto__transformed3_oracle seed=2] host probe accs: [0.718, 0.69, 0.575,
0.634, 0.836]
[probe_proto__transformed4_oracle seed=2] host probe accs: [0.705, 0.824, 0.936,
0.465, 0.496]
Energy Context Probe aggregate:

      seed          energy_probe_version run_mode  dataset_name  \
0      2  RC2jh_energy_context_probe_audit_v1  robust  FashionMNIST

      n_probe_samples  oracle_hit_rate  median_final_context_gap  \
0              40              0.975              1.091405

      mean_final_context_gap  median_gap_ratio_vs_target  mean_energy_drop  \
0              1.720546              37.63464              10.269692

      plateau_rate  wrong_context_rate  divergence_rate  success_rate
0              0.025              0.0              0.0              0.95

[baseline_finetune seed=2] after task 0: [0.98]
[baseline_finetune seed=2] after task 1: [0.497, 0.983]
[baseline_finetune seed=2] after task 2: [0.0, 0.492, 0.97]
[baseline_finetune seed=2] after task 3: [0.0, 0.0, 0.468, 0.931]
[baseline_finetune seed=2] after task 4: [0.0, 0.0, 0.0, 0.5, 0.999]
[baseline_ewc seed=2] after task 0: [0.98]
[baseline_ewc seed=2] after task 1: [0.495, 0.984]
[baseline_ewc seed=2] after task 2: [0.0, 0.489, 0.974]
[baseline_ewc seed=2] after task 3: [0.0, 0.0, 0.477, 0.946]
[baseline_ewc seed=2] after task 4: [0.0, 0.0, 0.0, 0.5, 0.998]
[baseline_lwf seed=2] after task 0: [0.98]
[baseline_lwf seed=2] after task 1: [0.963, 0.516]
[baseline_lwf seed=2] after task 2: [0.89, 0.974, 0.521]
[baseline_lwf seed=2] after task 3: [0.31, 0.64, 0.916, 0.721]
[baseline_lwf seed=2] after task 4: [0.018, 0.0, 0.326, 0.826, 0.989]
[baseline_experience_replay seed=2] after task 0: [0.98]
[baseline_experience_replay seed=2] after task 1: [0.951, 0.978]
[baseline_experience_replay seed=2] after task 2: [0.891, 0.956, 0.939]
[baseline_experience_replay seed=2] after task 3: [0.895, 0.831, 0.839, 0.853]
[baseline_experience_replay seed=2] after task 4: [0.894, 0.748, 0.726, 0.902,
0.97]
[baseline_balanced_replay seed=2] after task 0: [0.98]
[baseline_balanced_replay seed=2] after task 1: [0.951, 0.978]
[baseline_balanced_replay seed=2] after task 2: [0.891, 0.956, 0.939]

```

[baseline_balanced_replay seed=2] after task 3: [0.895, 0.831, 0.839, 0.853]
 [baseline_balanced_replay seed=2] after task 4: [0.894, 0.748, 0.726, 0.902, 0.97]
 [baseline_sparse_moe seed=2] after task 0: [0.984]
 [baseline_sparse_moe seed=2] after task 1: [0.495, 0.983]
 [baseline_sparse_moe seed=2] after task 2: [0.0, 0.482, 0.968]
 [baseline_sparse_moe seed=2] after task 3: [0.0, 0.0, 0.479, 0.943]
 [baseline_sparse_moe seed=2] after task 4: [0.0, 0.0, 0.0, 0.5, 1.0]
 [baseline_pnn seed=2] after task 0: [0.98]
 [baseline_pnn seed=2] after task 1: [0.497, 0.984]
 [baseline_pnn seed=2] after task 2: [0.0, 0.491, 0.969]
 [baseline_pnn seed=2] after task 3: [0.0, 0.0, 0.499, 0.885]
 [baseline_pnn seed=2] after task 4: [0.0, 0.0, 0.0, 0.5, 0.991]
 [baseline_joint_upper_bound seed=2] final joint eval: [0.94, 0.899, 0.851, 0.818, 0.879]
 [base_proto seed=3] task=0 epoch=1/8 loss=1.7168 task=0.9251 memory=0.0000
 balanced=0.0000 lsa=0.926
 [base_proto seed=3] task=0 epoch=2/8 loss=0.6411 task=0.3351 memory=0.0000
 balanced=0.0000 lsa=0.883
 [base_proto seed=3] task=0 epoch=3/8 loss=0.5219 task=0.2716 memory=0.0000
 balanced=0.0000 lsa=0.847
 [base_proto seed=3] task=0 epoch=4/8 loss=0.4611 task=0.2288 memory=0.0000
 balanced=0.0000 lsa=0.701
 [base_proto seed=3] task=0 epoch=5/8 loss=0.4355 task=0.2164 memory=0.0000
 balanced=0.0000 lsa=0.731
 [base_proto seed=3] task=0 epoch=6/8 loss=0.4179 task=0.2084 memory=0.0000
 balanced=0.0000 lsa=0.695
 [base_proto seed=3] task=0 epoch=7/8 loss=0.4105 task=0.2095 memory=0.0000
 balanced=0.0000 lsa=0.712
 [base_proto seed=3] task=0 epoch=8/8 loss=0.3897 task=0.1976 memory=0.0000
 balanced=0.0000 lsa=0.716
 [base_proto seed=3] calibrated context temperature after task 0: 0.25
 [base_proto seed=3] task=1 epoch=1/8 loss=5.3004 task=1.2692 memory=0.4542
 balanced=0.2134 lsa=0.854
 [base_proto seed=3] task=1 epoch=2/8 loss=2.6863 task=0.3874 memory=0.1556
 balanced=0.2098 lsa=0.889
 [base_proto seed=3] task=1 epoch=3/8 loss=2.1135 task=0.3177 memory=0.0605
 balanced=0.1991 lsa=0.900
 [base_proto seed=3] task=1 epoch=4/8 loss=1.8606 task=0.2899 memory=0.0362
 balanced=0.1978 lsa=0.891
 [base_proto seed=3] task=1 epoch=5/8 loss=1.7236 task=0.2645 memory=0.0296
 balanced=0.1838 lsa=0.865
 [base_proto seed=3] task=1 epoch=6/8 loss=1.6693 task=0.2527 memory=0.0252
 balanced=0.1789 lsa=0.905
 [base_proto seed=3] task=1 epoch=7/8 loss=1.5905 task=0.2379 memory=0.0258
 balanced=0.1617 lsa=0.916
 [base_proto seed=3] task=1 epoch=8/8 loss=1.5140 task=0.2267 memory=0.0236
 balanced=0.1555 lsa=0.907

```

[base_proto seed=3] calibrated context temperature after task 1: 0.80
[base_proto seed=3] task=2 epoch=1/8 loss=4.9913 task=1.3832 memory=0.5394
balanced=0.2704 lsa=0.955
[base_proto seed=3] task=2 epoch=2/8 loss=3.0276 task=0.6518 memory=0.3594
balanced=0.2442 lsa=0.917
[base_proto seed=3] task=2 epoch=3/8 loss=2.4574 task=0.4861 memory=0.2192
balanced=0.2257 lsa=0.944
[base_proto seed=3] task=2 epoch=4/8 loss=2.1622 task=0.4218 memory=0.1605
balanced=0.2065 lsa=0.941
[base_proto seed=3] task=2 epoch=5/8 loss=1.9097 task=0.3673 memory=0.1336
balanced=0.1953 lsa=0.935
[base_proto seed=3] task=2 epoch=6/8 loss=1.7711 task=0.3317 memory=0.1056
balanced=0.1798 lsa=0.797
[base_proto seed=3] task=2 epoch=7/8 loss=1.6853 task=0.3177 memory=0.1044
balanced=0.1736 lsa=0.901
[base_proto seed=3] task=2 epoch=8/8 loss=1.5633 task=0.2836 memory=0.0952
balanced=0.1399 lsa=0.802
[base_proto seed=3] calibrated context temperature after task 2: 0.60
[base_proto seed=3] task=3 epoch=1/8 loss=5.3704 task=1.9456 memory=0.6752
balanced=0.2698 lsa=0.955
[base_proto seed=3] task=3 epoch=2/8 loss=3.6252 task=0.9292 memory=0.7020
balanced=0.2487 lsa=0.979
[base_proto seed=3] task=3 epoch=3/8 loss=2.9037 task=0.7257 memory=0.4809
balanced=0.2040 lsa=0.989
[base_proto seed=3] task=3 epoch=4/8 loss=2.4155 task=0.5305 memory=0.3106
balanced=0.2035 lsa=0.978
[base_proto seed=3] task=3 epoch=5/8 loss=2.0381 task=0.4086 memory=0.2001
balanced=0.1564 lsa=0.909
[base_proto seed=3] task=3 epoch=6/8 loss=1.8397 task=0.3290 memory=0.1510
balanced=0.1543 lsa=0.865
[base_proto seed=3] task=3 epoch=7/8 loss=1.6912 task=0.3004 memory=0.1194
balanced=0.1209 lsa=0.862
[base_proto seed=3] task=3 epoch=8/8 loss=1.6148 task=0.2871 memory=0.1007
balanced=0.1212 lsa=0.863
[base_proto seed=3] calibrated context temperature after task 3: 0.60
[base_proto seed=3] task=4 epoch=1/8 loss=4.9632 task=1.6479 memory=0.9139
balanced=0.5925 lsa=0.932
[base_proto seed=3] task=4 epoch=2/8 loss=2.7452 task=0.5903 memory=0.2955
balanced=0.4873 lsa=0.958
[base_proto seed=3] task=4 epoch=3/8 loss=2.2233 task=0.3175 memory=0.2431
balanced=0.4652 lsa=0.976
[base_proto seed=3] task=4 epoch=4/8 loss=2.0204 task=0.2791 memory=0.1994
balanced=0.4631 lsa=0.983
[base_proto seed=3] task=4 epoch=5/8 loss=1.8123 task=0.2578 memory=0.1488
balanced=0.4124 lsa=0.977
[base_proto seed=3] task=4 epoch=6/8 loss=1.7498 task=0.2532 memory=0.1560
balanced=0.3997 lsa=0.970
[base_proto seed=3] task=4 epoch=7/8 loss=1.6558 task=0.2373 memory=0.1494

```

```

balanced=0.3795 lsa=0.986
[base_proto seed=3] task=4 epoch=8/8 loss=1.6848 task=0.2364 memory=0.1825
balanced=0.3962 lsa=0.973
[base_proto seed=3] calibrated context temperature after task 4: 0.60
[base_proto seed=3] completed in 187.5s
[RC2m_beta seed=3] after task 0: selected proto_global_no_repair
proxy_acc=0.9406 score=1.2602
[RC2m_beta seed=3] after task 1: selected proto_global_head_ce_kl_guard_035
proxy_acc=0.9318 score=1.7885
[RC2m_beta seed=3] after task 2: selected
context_temp_blend_selected_head_guard_035 proxy_acc=0.9391 score=1.7392
[RC2m_beta seed=3] after task 3: selected
context_temp_blend_selected_head_guard_035 proxy_acc=0.9309 score=1.6980
[RC2m_beta seed=3] after task 4: selected
context_temp_blend_selected_head_guard_035 proxy_acc=0.9357 score=1.3725
[probe_proto__zf_proto seed=3] probe final task accs: [0.974, 0.974, 0.833,
0.885, 0.976]
[probe_proto__zf_oracle seed=3] probe final task accs: [0.975, 0.98, 0.848,
0.875, 0.98]
[probe_proto__transformed0_oracle seed=3] host probe accs: [0.754, 0.912, 0.879,
0.441, 0.484]
[probe_proto__transformed1_oracle seed=3] host probe accs: [0.855, 0.854, 0.828,
0.841, 0.902]
[probe_proto__transformed2_oracle seed=3] host probe accs: [0.7, 0.752, 0.701,
0.396, 0.579]
[probe_proto__transformed3_oracle seed=3] host probe accs: [0.787, 0.823, 0.85,
0.66, 0.708]
[probe_proto__transformed4_oracle seed=3] host probe accs: [0.757, 0.705, 0.631,
0.68, 0.806]
Energy Context Probe aggregate:
      seed          energy_probe_version run_mode  dataset_name \
0      3  RC2jh_energy_context_probe_audit_v1  robust  FashionMNIST

      n_probe_samples  oracle_hit_rate  median_final_context_gap \
0              40              0.975              0.404737

      mean_final_context_gap  median_gap_ratio_vs_target  mean_energy_drop \
0              0.509302              13.956445              8.421558

      plateau_rate  wrong_context_rate  divergence_rate  success_rate
0              0.0              0.0              0.0              0.975

[baseline_finetune seed=3] after task 0: [0.973]
[baseline_finetune seed=3] after task 1: [0.496, 0.984]
[baseline_finetune seed=3] after task 2: [0.0, 0.486, 0.964]
[baseline_finetune seed=3] after task 3: [0.0, 0.0, 0.477, 0.931]
[baseline_finetune seed=3] after task 4: [0.0, 0.0, 0.0, 0.5, 0.998]
[baseline_ewc seed=3] after task 0: [0.973]

```


[baseline_ewc seed=3] after task 1: [0.494, 0.986]
 [baseline_ewc seed=3] after task 2: [0.0, 0.486, 0.965]
 [baseline_ewc seed=3] after task 3: [0.0, 0.0, 0.482, 0.936]
 [baseline_ewc seed=3] after task 4: [0.0, 0.0, 0.0, 0.5, 0.999]
 [baseline_lwf seed=3] after task 0: [0.973]
 [baseline_lwf seed=3] after task 1: [0.984, 0.5]
 [baseline_lwf seed=3] after task 2: [0.938, 0.929, 0.468]
 [baseline_lwf seed=3] after task 3: [0.312, 0.362, 0.86, 0.748]
 [baseline_lwf seed=3] after task 4: [0.172, 0.0, 0.385, 0.877, 0.98]
 [baseline_experience_replay seed=3] after task 0: [0.973]
 [baseline_experience_replay seed=3] after task 1: [0.955, 0.958]
 [baseline_experience_replay seed=3] after task 2: [0.891, 0.949, 0.936]
 [baseline_experience_replay seed=3] after task 3: [0.877, 0.781, 0.775, 0.876]
 [baseline_experience_replay seed=3] after task 4: [0.877, 0.821, 0.764, 0.844, 0.946]
 [baseline_balanced_replay seed=3] after task 0: [0.973]
 [baseline_balanced_replay seed=3] after task 1: [0.955, 0.958]
 [baseline_balanced_replay seed=3] after task 2: [0.891, 0.949, 0.936]
 [baseline_balanced_replay seed=3] after task 3: [0.877, 0.781, 0.775, 0.876]
 [baseline_balanced_replay seed=3] after task 4: [0.877, 0.821, 0.764, 0.844, 0.946]
 [baseline_sparse_moe seed=3] after task 0: [0.991]
 [baseline_sparse_moe seed=3] after task 1: [0.494, 0.983]
 [baseline_sparse_moe seed=3] after task 2: [0.0, 0.477, 0.968]
 [baseline_sparse_moe seed=3] after task 3: [0.0, 0.0, 0.49, 0.943]
 [baseline_sparse_moe seed=3] after task 4: [0.0, 0.0, 0.0, 0.5, 0.998]
 [baseline_pnn seed=3] after task 0: [0.973]
 [baseline_pnn seed=3] after task 1: [0.496, 0.983]
 [baseline_pnn seed=3] after task 2: [0.0, 0.486, 0.963]
 [baseline_pnn seed=3] after task 3: [0.0, 0.0, 0.454, 0.924]
 [baseline_pnn seed=3] after task 4: [0.0, 0.0, 0.0, 0.5, 0.999]
 [baseline_joint_upper_bound seed=3] final joint eval: [0.939, 0.892, 0.84, 0.805, 0.881]
 [base_proto seed=4] task=0 epoch=1/8 loss=1.8377 task=0.8274 memory=0.0000
 balanced=0.0000 lsa=0.931
 [base_proto seed=4] task=0 epoch=2/8 loss=0.5534 task=0.2754 memory=0.0000
 balanced=0.0000 lsa=0.845
 [base_proto seed=4] task=0 epoch=3/8 loss=0.4730 task=0.2277 memory=0.0000
 balanced=0.0000 lsa=0.892
 [base_proto seed=4] task=0 epoch=4/8 loss=0.4159 task=0.1908 memory=0.0000
 balanced=0.0000 lsa=0.843
 [base_proto seed=4] task=0 epoch=5/8 loss=0.3981 task=0.1821 memory=0.0000
 balanced=0.0000 lsa=0.789
 [base_proto seed=4] task=0 epoch=6/8 loss=0.3867 task=0.1790 memory=0.0000
 balanced=0.0000 lsa=0.817
 [base_proto seed=4] task=0 epoch=7/8 loss=0.3819 task=0.1819 memory=0.0000
 balanced=0.0000 lsa=0.825
 [base_proto seed=4] task=0 epoch=8/8 loss=0.3773 task=0.1844 memory=0.0000

balanced=0.0000 lsa=0.831
 [base_proto seed=4] calibrated context temperature after task 0: 0.25
 [base_proto seed=4] task=1 epoch=1/8 loss=5.4183 task=1.4637 memory=0.4375
 balanced=0.2131 lsa=0.920
 [base_proto seed=4] task=1 epoch=2/8 loss=2.9962 task=0.6040 memory=0.1977
 balanced=0.1871 lsa=0.824
 [base_proto seed=4] task=1 epoch=3/8 loss=2.3804 task=0.3889 memory=0.1252
 balanced=0.1923 lsa=0.773
 [base_proto seed=4] task=1 epoch=4/8 loss=2.0680 task=0.3454 memory=0.0844
 balanced=0.1764 lsa=0.809
 [base_proto seed=4] task=1 epoch=5/8 loss=1.9159 task=0.3204 memory=0.0601
 balanced=0.1639 lsa=0.786
 [base_proto seed=4] task=1 epoch=6/8 loss=1.8193 task=0.2982 memory=0.0518
 balanced=0.1547 lsa=0.816
 [base_proto seed=4] task=1 epoch=7/8 loss=1.7361 task=0.2844 memory=0.0498
 balanced=0.1421 lsa=0.865
 [base_proto seed=4] task=1 epoch=8/8 loss=1.7106 task=0.2808 memory=0.0596
 balanced=0.1391 lsa=0.909
 [base_proto seed=4] calibrated context temperature after task 1: 0.60
 [base_proto seed=4] task=2 epoch=1/8 loss=5.1953 task=1.4983 memory=0.3977
 balanced=0.2978 lsa=0.919
 [base_proto seed=4] task=2 epoch=2/8 loss=3.2786 task=0.8136 memory=0.2887
 balanced=0.3217 lsa=0.936
 [base_proto seed=4] task=2 epoch=3/8 loss=2.5224 task=0.4726 memory=0.1686
 balanced=0.2891 lsa=0.920
 [base_proto seed=4] task=2 epoch=4/8 loss=2.1573 task=0.3806 memory=0.1073
 balanced=0.2611 lsa=0.876
 [base_proto seed=4] task=2 epoch=5/8 loss=1.8892 task=0.3272 memory=0.0793
 balanced=0.2052 lsa=0.882
 [base_proto seed=4] task=2 epoch=6/8 loss=1.7000 task=0.2999 memory=0.0679
 balanced=0.1787 lsa=0.890
 [base_proto seed=4] task=2 epoch=7/8 loss=1.6482 task=0.2822 memory=0.0776
 balanced=0.1789 lsa=0.612
 [base_proto seed=4] task=2 epoch=8/8 loss=1.6045 task=0.2745 memory=0.0799
 balanced=0.1763 lsa=0.622
 [base_proto seed=4] calibrated context temperature after task 2: 0.80
 [base_proto seed=4] task=3 epoch=1/8 loss=5.7505 task=2.1065 memory=0.8068
 balanced=0.3040 lsa=0.910
 [base_proto seed=4] task=3 epoch=2/8 loss=3.2543 task=0.7429 memory=0.4585
 balanced=0.2736 lsa=0.917
 [base_proto seed=4] task=3 epoch=3/8 loss=2.4797 task=0.4893 memory=0.2020
 balanced=0.2673 lsa=0.916
 [base_proto seed=4] task=3 epoch=4/8 loss=2.1952 task=0.3860 memory=0.1390
 balanced=0.2542 lsa=0.883
 [base_proto seed=4] task=3 epoch=5/8 loss=2.0103 task=0.3400 memory=0.1117
 balanced=0.2364 lsa=0.857
 [base_proto seed=4] task=3 epoch=6/8 loss=1.8794 task=0.3111 memory=0.0938
 balanced=0.2124 lsa=0.832

```

[base_proto seed=4] task=3 epoch=7/8 loss=1.8278 task=0.2907 memory=0.0898
balanced=0.2223 lsa=0.811
[base_proto seed=4] task=3 epoch=8/8 loss=1.7703 task=0.2844 memory=0.0819
balanced=0.2199 lsa=0.867
[base_proto seed=4] calibrated context temperature after task 3: 0.80
[base_proto seed=4] task=4 epoch=1/8 loss=8.0603 task=1.9888 memory=2.9831
balanced=1.1856 lsa=0.935
[base_proto seed=4] task=4 epoch=2/8 loss=3.5411 task=0.6399 memory=0.7924
balanced=0.5625 lsa=0.801
[base_proto seed=4] task=4 epoch=3/8 loss=3.4593 task=0.3135 memory=1.0801
balanced=0.7081 lsa=0.859
[base_proto seed=4] task=4 epoch=4/8 loss=2.2786 task=0.2834 memory=0.2962
balanced=0.4461 lsa=0.648
[base_proto seed=4] task=4 epoch=5/8 loss=2.5325 task=0.2338 memory=0.5335
balanced=0.5983 lsa=0.776
[base_proto seed=4] task=4 epoch=6/8 loss=1.8937 task=0.2221 memory=0.1835
balanced=0.4162 lsa=0.754
[base_proto seed=4] task=4 epoch=7/8 loss=2.0464 task=0.2054 memory=0.3986
balanced=0.4461 lsa=0.561
[base_proto seed=4] task=4 epoch=8/8 loss=2.1608 task=0.2207 memory=0.5008
balanced=0.4855 lsa=0.578
[base_proto seed=4] calibrated context temperature after task 4: 0.80
[base_proto seed=4] completed in 270.4s
[RC2m_beta seed=4] after task 0: selected proto_global_head_ce_kl_guard_035
proxy_acc=0.9281 score=1.2450
[RC2m_beta seed=4] after task 1: selected
generic_weak_boundary_latent_calibrated proxy_acc=0.9271 score=1.7753
[RC2m_beta seed=4] after task 2: selected
context_temp_blend_selected_head_guard_035 proxy_acc=0.9352 score=1.7081
[RC2m_beta seed=4] after task 3: selected
generic_weak_boundary_latent_calibrated proxy_acc=0.9486 score=1.6806
[RC2m_beta seed=4] after task 4: selected
context_temp_blend_selected_head_guard_035 proxy_acc=0.9266 score=1.3539
[probe_proto__zf_proto seed=4] probe final task accs: [0.948, 0.949, 0.759,
0.818, 0.887]
[probe_proto__zf_oracle seed=4] probe final task accs: [0.958, 0.92, 0.943,
0.907, 0.964]
[probe_proto__transformed0_oracle seed=4] host probe accs: [0.84, 0.759, 0.666,
0.782, 0.846]
[probe_proto__transformed1_oracle seed=4] host probe accs: [0.738, 0.671, 0.316,
0.445, 0.739]
[probe_proto__transformed2_oracle seed=4] host probe accs: [0.574, 0.456, 0.608,
0.81, 0.828]
[probe_proto__transformed3_oracle seed=4] host probe accs: [0.844, 0.839, 0.825,
0.579, 0.681]
[probe_proto__transformed4_oracle seed=4] host probe accs: [0.704, 0.762, 0.736,
0.263, 0.496]
Energy Context Probe aggregate:

```

seed	energy_probe_version	run_mode	dataset_name	\
0	4	RC2jh_energy_context_probe_audit_v1	robust	FashionMNIST

n_probe_samples	oracle_hit_rate	median_final_context_gap	\
0	40	0.9	1.211664

mean_final_context_gap	median_gap_ratio_vs_target	mean_energy_drop	\
0	1.612966	41.781534	5.478084

plateau_rate	wrong_context_rate	divergence_rate	success_rate
0	0.0	0.075	0.0


```

[baseline_finetune seed=4] after task 0: [0.98]
[baseline_finetune seed=4] after task 1: [0.495, 0.989]
[baseline_finetune seed=4] after task 2: [0.0, 0.491, 0.964]
[baseline_finetune seed=4] after task 3: [0.0, 0.0, 0.476, 0.934]
[baseline_finetune seed=4] after task 4: [0.0, 0.0, 0.0, 0.5, 0.998]
[baseline_ewc seed=4] after task 0: [0.98]
[baseline_ewc seed=4] after task 1: [0.492, 0.986]
[baseline_ewc seed=4] after task 2: [0.0, 0.492, 0.958]
[baseline_ewc seed=4] after task 3: [0.0, 0.0, 0.489, 0.94]
[baseline_ewc seed=4] after task 4: [0.0, 0.0, 0.0, 0.5, 0.999]
[baseline_lwf seed=4] after task 0: [0.98]
[baseline_lwf seed=4] after task 1: [0.983, 0.499]
[baseline_lwf seed=4] after task 2: [0.926, 0.96, 0.482]
[baseline_lwf seed=4] after task 3: [0.044, 0.357, 0.836, 0.741]
[baseline_lwf seed=4] after task 4: [0.001, 0.003, 0.367, 0.866, 0.985]
[baseline_experience_replay seed=4] after task 0: [0.98]
[baseline_experience_replay seed=4] after task 1: [0.948, 0.976]
[baseline_experience_replay seed=4] after task 2: [0.885, 0.966, 0.932]
[baseline_experience_replay seed=4] after task 3: [0.836, 0.801, 0.825, 0.884]
[baseline_experience_replay seed=4] after task 4: [0.914, 0.777, 0.681, 0.851, 0.984]
[baseline_balanced_replay seed=4] after task 0: [0.98]
[baseline_balanced_replay seed=4] after task 1: [0.948, 0.976]
[baseline_balanced_replay seed=4] after task 2: [0.885, 0.966, 0.932]
[baseline_balanced_replay seed=4] after task 3: [0.836, 0.801, 0.825, 0.884]
[baseline_balanced_replay seed=4] after task 4: [0.914, 0.777, 0.681, 0.851, 0.984]
[baseline_sparse_moe seed=4] after task 0: [0.989]
[baseline_sparse_moe seed=4] after task 1: [0.495, 0.994]
[baseline_sparse_moe seed=4] after task 2: [0.0, 0.486, 0.966]
[baseline_sparse_moe seed=4] after task 3: [0.0, 0.0, 0.48, 0.945]
[baseline_sparse_moe seed=4] after task 4: [0.0, 0.0, 0.0, 0.5, 0.999]
[baseline_pnn seed=4] after task 0: [0.986]
[baseline_pnn seed=4] after task 1: [0.492, 0.989]
[baseline_pnn seed=4] after task 2: [0.0, 0.49, 0.961]
[baseline_pnn seed=4] after task 3: [0.0, 0.0, 0.484, 0.94]

```

```
[baseline_pnn seed=4] after task 4: [0.0, 0.0, 0.0, 0.499, 0.998]
[baseline_joint_upper_bound seed=4] final joint eval: [0.956, 0.896, 0.829,
0.83, 0.894]
```

```
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
```

```
    return float(np.trapz(tpr[order], fpr[order]))
```

```
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
```

```
    return float(np.trapz(tpr[order], fpr[order]))
```

```
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
```

```
    return float(np.trapz(tpr[order], fpr[order]))
```

```
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
```

```
    return float(np.trapz(tpr[order], fpr[order]))
```

```
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
```

```
    return float(np.trapz(tpr[order], fpr[order]))
```

```
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
```

```
    return float(np.trapz(tpr[order], fpr[order]))
```

```
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
```

```
    return float(np.trapz(tpr[order], fpr[order]))
```

```
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
```

```
    return float(np.trapz(tpr[order], fpr[order]))
```

```
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
```

```
    return float(np.trapz(tpr[order], fpr[order]))
```

```
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
```

```
    return float(np.trapz(tpr[order], fpr[order]))
```

```
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
```

```
    return float(np.trapz(tpr[order], fpr[order]))
```

```

/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))

```

```

/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))

```

```

/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))

```



```

/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))

```

```

/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))

```

```

/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))

```

```

/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))

```

```

/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))

```

```

/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))

```

```

/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))

```

```

/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))

```



```

/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))

```

```

/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))

```

```

/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))

```

```

/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))

```

```

/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))

```

```

/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))

```

```

/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))

```

```

/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))

```



```

/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))

```

```

/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))

```

```

/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))

```

```

/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))
/tmp/ipykernel_1079/3836257314.py:97: DeprecationWarning: `trapz` is deprecated.
Use `trapezoid` instead, or one of the numerical integration functions in
`scipy.integrate`.
    return float(np.trapz(tpr[order], fpr[order]))

```

Biometric-style threshold diagnostics saved:

```

- /content/drive/MyDrive/MMALS/v1_1_rc2m_alpha_boundary_first_frozen_selector/R
C2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT/MMALS_v11_RC2M_BETA_DYNAMIC
_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT_FashionMNIST_robust_20260601T164013Z_seeds5
/biometric_style_score_dump_robust.csv
- /content/drive/MyDrive/MMALS/v1_1_rc2m_alpha_boundary_first_frozen_selector/R
C2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT/MMALS_v11_RC2M_BETA_DYNAMIC
_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT_FashionMNIST_robust_20260601T164013Z_seeds5
/threshold_curves/biometric_style_far_frr_roc_det_curves_robust.csv
- /content/drive/MyDrive/MMALS/v1_1_rc2m_alpha_boundary_first_frozen_selector/R
C2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT/MMALS_v11_RC2M_BETA_DYNAMIC
_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT_FashionMNIST_robust_20260601T164013Z_seeds5
/threshold_curves/biometric_style_eer_by_class_robust.csv
- /content/drive/MyDrive/MMALS/v1_1_rc2m_alpha_boundary_first_frozen_selector/R
C2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT/MMALS_v11_RC2M_BETA_DYNAMIC
_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT_FashionMNIST_robust_20260601T164013Z_seeds5
/threshold_curves/biometric_style_eer_by_method_robust.csv
- /content/drive/MyDrive/MMALS/v1_1_rc2m_alpha_boundary_first_frozen_selector/R
C2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT/MMALS_v11_RC2M_BETA_DYNAMIC
_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT_FashionMNIST_robust_20260601T164013Z_seeds5
/plots/threshold_diagnostics/roc_style_class2_robust.png
- /content/drive/MyDrive/MMALS/v1_1_rc2m_alpha_boundary_first_frozen_selector/R
C2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT/MMALS_v11_RC2M_BETA_DYNAMIC
_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT_FashionMNIST_robust_20260601T164013Z_seeds5
/plots/threshold_diagnostics/far_frr_class2_robust.png
- /content/drive/MyDrive/MMALS/v1_1_rc2m_alpha_boundary_first_frozen_selector/R
C2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT/MMALS_v11_RC2M_BETA_DYNAMIC

```



```

C2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT/MMALS_v11_RC2M_BETA_DYNAMIC
_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT_FashionMNIST_robust_20260601T164013Z_seeds5
/probe_losses_robust.csv
- /content/drive/MyDrive/MMALS/v1_1_rc2m_alpha_boundary_first_frozen_selector/R
C2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT/MMALS_v11_RC2M_BETA_DYNAMIC
_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT_FashionMNIST_robust_20260601T164013Z_seeds5
/forgetting_robust.csv
- /content/drive/MyDrive/MMALS/v1_1_rc2m_alpha_boundary_first_frozen_selector/R
C2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT/MMALS_v11_RC2M_BETA_DYNAMIC
_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT_FashionMNIST_robust_20260601T164013Z_seeds5
/class_confusion_robust.csv
- /content/drive/MyDrive/MMALS/v1_1_rc2m_alpha_boundary_first_frozen_selector/R
C2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT/MMALS_v11_RC2M_BETA_DYNAMIC
_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT_FashionMNIST_robust_20260601T164013Z_seeds5
/class_diagnostics_robust.csv
- /content/drive/MyDrive/MMALS/v1_1_rc2m_alpha_boundary_first_frozen_selector/R
C2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT/MMALS_v11_RC2M_BETA_DYNAMIC
_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT_FashionMNIST_robust_20260601T164013Z_seeds5
/learning_signal_alignment_panel_e_robust.csv
- /content/drive/MyDrive/MMALS/v1_1_rc2m_alpha_boundary_first_frozen_selector/R
C2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT/MMALS_v11_RC2M_BETA_DYNAMIC
_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT_FashionMNIST_robust_20260601T164013Z_seeds5
/learning_signal_alignment_panel_e_summary_robust.csv
- /content/drive/MyDrive/MMALS/v1_1_rc2m_alpha_boundary_first_frozen_selector/R
C2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT/MMALS_v11_RC2M_BETA_DYNAMIC
_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT_FashionMNIST_robust_20260601T164013Z_seeds5
/biometric_style_score_dump_robust.csv
- /content/drive/MyDrive/MMALS/v1_1_rc2m_alpha_boundary_first_frozen_selector/R
C2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT/MMALS_v11_RC2M_BETA_DYNAMIC
_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT_FashionMNIST_robust_20260601T164013Z_seeds5
/threshold_curves/biometric_style_far_frr_roc_det_curves_robust.csv
- /content/drive/MyDrive/MMALS/v1_1_rc2m_alpha_boundary_first_frozen_selector/R
C2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT/MMALS_v11_RC2M_BETA_DYNAMIC
_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT_FashionMNIST_robust_20260601T164013Z_seeds5
/threshold_curves/biometric_style_eer_by_class_robust.csv
- /content/drive/MyDrive/MMALS/v1_1_rc2m_alpha_boundary_first_frozen_selector/R
C2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT/MMALS_v11_RC2M_BETA_DYNAMIC
_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT_FashionMNIST_robust_20260601T164013Z_seeds5
/threshold_curves/biometric_style_eer_by_method_robust.csv
RC2b selected policy counts:

```

```

                selected_candidate_variant \
0                context_blend_selected_global
1 context_temp_blend_selected_head_guard_035
2    generic_weak_boundary_latent_calibrated
3        proto_global_head_ce_kl_guard_035
4                proto_global_no_repair

```

	selected_policy_family	size
0	context_only_global	1
1	rc1b_guarded_context_gap	11
2	generic_latent_boundary_calibration	4
3	proto_true_global_guard	6
4	proto_no_repair	3

Biometric-style EER summary by method, lower is better:

	dataset	run_mode	experiment_profile \
31	FashionMNIST	robust	RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDR...
39	FashionMNIST	robust	RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDR...
23	FashionMNIST	robust	RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDR...
7	FashionMNIST	robust	RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDR...
15	FashionMNIST	robust	RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDR...
17	FashionMNIST	robust	RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDR...
19	FashionMNIST	robust	RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDR...
16	FashionMNIST	robust	RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDR...
18	FashionMNIST	robust	RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDR...
28	FashionMNIST	robust	RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDR...
12	FashionMNIST	robust	RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDR...
25	FashionMNIST	robust	RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDR...
27	FashionMNIST	robust	RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDR...
29	FashionMNIST	robust	RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDR...
33	FashionMNIST	robust	RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDR...
35	FashionMNIST	robust	RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDR...
21	FashionMNIST	robust	RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDR...
20	FashionMNIST	robust	RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDR...
22	FashionMNIST	robust	RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDR...
1	FashionMNIST	robust	RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDR...

	seed	after_task	method \
31	3	4	paired_rc2m_beta_dynamic_boundary_ambiguity_lo...
39	4	4	paired_rc2m_beta_dynamic_boundary_ambiguity_lo...
23	2	4	paired_rc2m_beta_dynamic_boundary_ambiguity_lo...
7	0	4	paired_rc2m_beta_dynamic_boundary_ambiguity_lo...
15	1	4	paired_rc2m_beta_dynamic_boundary_ambiguity_lo...
17	2	4	paired_context_gap_selected
19	2	4	paired_context_temp_blend_selected_head_guard_035
16	2	4	paired_context_blend_selected_global
18	2	4	paired_context_temp_blend_selected_global
28	3	4	paired_generic_weak_boundary_latent_calibrated
12	1	4	paired_generic_weak_boundary_latent_calibrated
25	3	4	paired_context_gap_selected
27	3	4	paired_context_temp_blend_selected_head_guard_035
29	3	4	paired_proto_global_head_ce_kl_guard_035
33	4	4	paired_context_gap_selected
35	4	4	paired_context_temp_blend_selected_head_guard_035
21	2	4	paired_proto_global_head_ce_kl_guard_035

20	2	4	paired_generic_weak_boundary_latent_calibrated
22	2	4	paired_proto_global_no_repair
1	0	4	paired_context_gap_selected

			variant \
31			context_temp_blend_selected_head_guard_035
39			context_temp_blend_selected_head_guard_035
23			proto_global_head_ce_kl_guard_035
7			proto_global_head_ce_kl_guard_035
15			context_blend_selected_global
17			context_gap_selected
19			context_temp_blend_selected_head_guard_035
16			context_blend_selected_global
18			context_temp_blend_selected_global
28			generic_weak_boundary_latent_calibrated
12			generic_weak_boundary_latent_calibrated
25			context_gap_selected
27			context_temp_blend_selected_head_guard_035
29			proto_global_head_ce_kl_guard_035
33			context_gap_selected
35			context_temp_blend_selected_head_guard_035
21			proto_global_head_ce_kl_guard_035
20			generic_weak_boundary_latent_calibrated
22			proto_global_no_repair
1			context_gap_selected

		policy_family	selected_policy	macro_eer	...	\
31		rc1b_guarded_context_gap	True	0.036238	...	
39		rc1b_guarded_context_gap	True	0.046045	...	
23		proto_true_global_guard	True	0.047520	...	
7		proto_true_global_guard	True	0.054010	...	
15		context_only_global	True	0.066328	...	
17		rc1b_guarded_context_gap	False	0.033354	...	
19		rc1b_guarded_context_gap	False	0.033354	...	
16		context_only_global	False	0.035391	...	
18		context_only_global	False	0.035391	...	
28		generic_latent_boundary_calibration	False	0.034711	...	
12		generic_latent_boundary_calibration	False	0.041557	...	
25		rc1b_guarded_context_gap	False	0.036238	...	
27		rc1b_guarded_context_gap	False	0.036238	...	
29		proto_true_global_guard	False	0.036921	...	
33		rc1b_guarded_context_gap	False	0.046045	...	
35		rc1b_guarded_context_gap	False	0.046045	...	
21		proto_true_global_guard	False	0.047520	...	
20		generic_latent_boundary_calibration	False	0.046557	...	
22		proto_no_repair	False	0.046895	...	
1		rc1b_guarded_context_gap	False	0.052784	...	

	class2_eer	class2_auc	class3_eer	class3_auc	class4_eer	class4_auc	\
31	0.078594	0.972929	0.030156	0.995221	0.068125	0.978939	
39	0.078437	0.973779	0.062656	0.977293	0.071250	0.979635	
23	0.078750	0.975553	0.039688	0.992038	0.078750	0.970931	
7	0.110156	0.958024	0.046094	0.988772	0.087500	0.964747	
15	0.156250	0.913861	0.059375	0.990571	0.133594	0.928306	
17	0.048594	0.991413	0.037031	0.992381	0.040469	0.988819	
19	0.048594	0.991413	0.037031	0.992381	0.040469	0.988819	
16	0.042500	0.992910	0.042188	0.990008	0.056250	0.980356	
18	0.042500	0.992910	0.042188	0.990008	0.056250	0.980356	
28	0.064844	0.976298	0.036094	0.994520	0.064688	0.981261	
12	0.063438	0.984944	0.055156	0.986852	0.081719	0.979016	
25	0.078594	0.972929	0.030156	0.995221	0.068125	0.978939	
27	0.078594	0.972929	0.030156	0.995221	0.068125	0.978939	
29	0.077656	0.974476	0.033906	0.994603	0.069688	0.979392	
33	0.078437	0.973779	0.062656	0.977293	0.071250	0.979635	
35	0.078437	0.973779	0.062656	0.977293	0.071250	0.979635	
21	0.078750	0.975553	0.039688	0.992038	0.078750	0.970931	
20	0.073594	0.977225	0.045469	0.990463	0.086250	0.964096	
22	0.073594	0.977286	0.046562	0.990475	0.087188	0.964056	
1	0.103750	0.959249	0.040156	0.991159	0.081406	0.970008	

	class5_eer	class5_auc	micro_eer	micro_auc
31	0.000417	0.999855	0.041450	0.991709
39	0.000278	0.999851	0.056450	0.986338
23	0.000556	0.999893	0.051125	0.987471
7	0.002500	0.999668	0.058500	0.983569
15	0.000000	0.999999	0.084075	0.970467
17	0.000000	0.999997	0.032150	0.994347
19	0.000000	0.999997	0.032150	0.994347
16	0.000000	1.000000	0.039775	0.991331
18	0.000000	1.000000	0.039775	0.991331
28	0.000417	0.999847	0.037100	0.992267
12	0.000278	0.999991	0.050600	0.988793
25	0.000417	0.999855	0.041450	0.991709
27	0.000417	0.999855	0.041450	0.991709
29	0.000417	0.999866	0.038550	0.992110
33	0.000278	0.999851	0.056450	0.986338
35	0.000278	0.999851	0.056450	0.986338
21	0.000556	0.999893	0.051125	0.987471
20	0.000417	0.999852	0.047500	0.988012
22	0.000417	0.999852	0.047675	0.988095
1	0.002500	0.999650	0.056350	0.983591

[20 rows x 27 columns]

Threshold diagnostic plots:

- /content/drive/MyDrive/MMALS/v1_1_rc2m_alpha_boundary_first_frozen_selector/R

```

C2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT/MMALS_v11_RC2M_BETA_DYNAMIC
_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT_FashionMNIST_robust_20260601T164013Z_seeds5
/plots/threshold_diagnostics/roc_style_class2_robust.png
- /content/drive/MyDrive/MMALS/v1_1_rc2m_alpha_boundary_first_frozen_selector/R
C2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT/MMALS_v11_RC2M_BETA_DYNAMIC
_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT_FashionMNIST_robust_20260601T164013Z_seeds5
/plots/threshold_diagnostics/far_frr_class2_robust.png
- /content/drive/MyDrive/MMALS/v1_1_rc2m_alpha_boundary_first_frozen_selector/R
C2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT/MMALS_v11_RC2M_BETA_DYNAMIC
_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT_FashionMNIST_robust_20260601T164013Z_seeds5
/plots/threshold_diagnostics/det_style_class2_robust.png
- /content/drive/MyDrive/MMALS/v1_1_rc2m_alpha_boundary_first_frozen_selector/R
C2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT/MMALS_v11_RC2M_BETA_DYNAMIC
_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT_FashionMNIST_robust_20260601T164013Z_seeds5
/plots/threshold_diagnostics/roc_style_class4_robust.png
- /content/drive/MyDrive/MMALS/v1_1_rc2m_alpha_boundary_first_frozen_selector/R
C2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT/MMALS_v11_RC2M_BETA_DYNAMIC
_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT_FashionMNIST_robust_20260601T164013Z_seeds5
/plots/threshold_diagnostics/far_frr_class4_robust.png
- /content/drive/MyDrive/MMALS/v1_1_rc2m_alpha_boundary_first_frozen_selector/R
C2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT/MMALS_v11_RC2M_BETA_DYNAMIC
_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT_FashionMNIST_robust_20260601T164013Z_seeds5
/plots/threshold_diagnostics/det_style_class4_robust.png
- /content/drive/MyDrive/MMALS/v1_1_rc2m_alpha_boundary_first_frozen_selector/R
C2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT/MMALS_v11_RC2M_BETA_DYNAMIC
_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT_FashionMNIST_robust_20260601T164013Z_seeds5
/plots/threshold_diagnostics/roc_style_class5_robust.png
- /content/drive/MyDrive/MMALS/v1_1_rc2m_alpha_boundary_first_frozen_selector/R
C2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT/MMALS_v11_RC2M_BETA_DYNAMIC
_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT_FashionMNIST_robust_20260601T164013Z_seeds5
/plots/threshold_diagnostics/far_frr_class5_robust.png
- /content/drive/MyDrive/MMALS/v1_1_rc2m_alpha_boundary_first_frozen_selector/R
C2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT/MMALS_v11_RC2M_BETA_DYNAMIC
_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT_FashionMNIST_robust_20260601T164013Z_seeds5
/plots/threshold_diagnostics/det_style_class5_robust.png
- /content/drive/MyDrive/MMALS/v1_1_rc2m_alpha_boundary_first_frozen_selector/R
C2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT/MMALS_v11_RC2M_BETA_DYNAMIC
_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT_FashionMNIST_robust_20260601T164013Z_seeds5
/plots/threshold_diagnostics/eer_by_method_robust.png

```

2.31 9c. Biometric-style threshold diagnostic plots

This section displays the generated diagnostic plots directly inside the executed notebook/PDF.

The plots are created **after** no-leakage policy selection and final-test evaluation. They are not used by the selector.

```
[ ]: # =====
# 9c. Display biometric-style ROC / DET / FAR-FRR plots
# =====
from IPython.display import display, Image, Markdown
from pathlib import Path

threshold_plot_dir = PLOTS_DIR / "threshold_diagnostics"
display(Markdown("### 9c. Biometric-style ROC / DET / FAR-FRR plots"))
display(Markdown(
    "These plots are one-vs-rest diagnostics for the final checkpoint. "
    "They are generated after policy selection and are not used for selection."
))

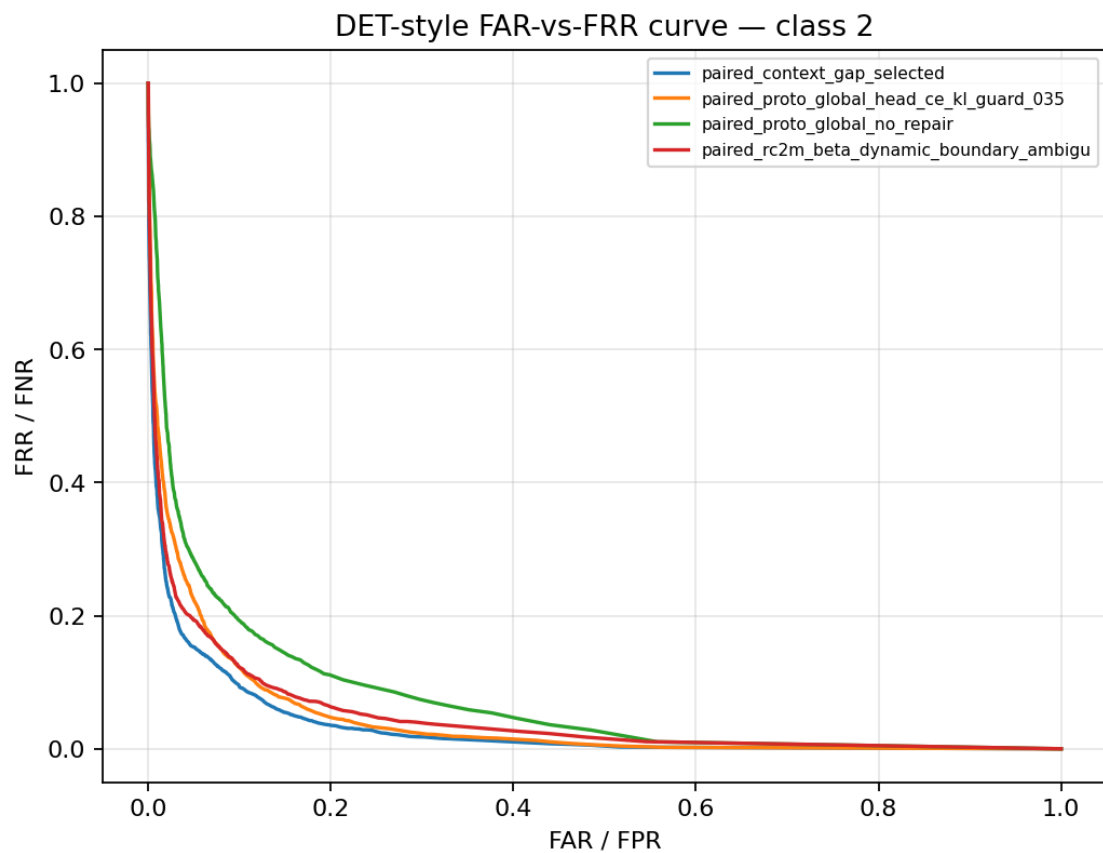
plot_paths = sorted(threshold_plot_dir.glob(f"*_{RUN_MODE}.png"))
if not plot_paths:
    raise FileNotFoundError(
        f"No threshold diagnostic plots found in {threshold_plot_dir}. "
        "Check that score_dump_df is non-empty and that_
        ↪plot_biometric_style_threshold_diagnostics(...) executed."
    )

for p in plot_paths:
    display(Markdown(f"#### {p.name}"))
    display(Image(filename=str(p)))
```

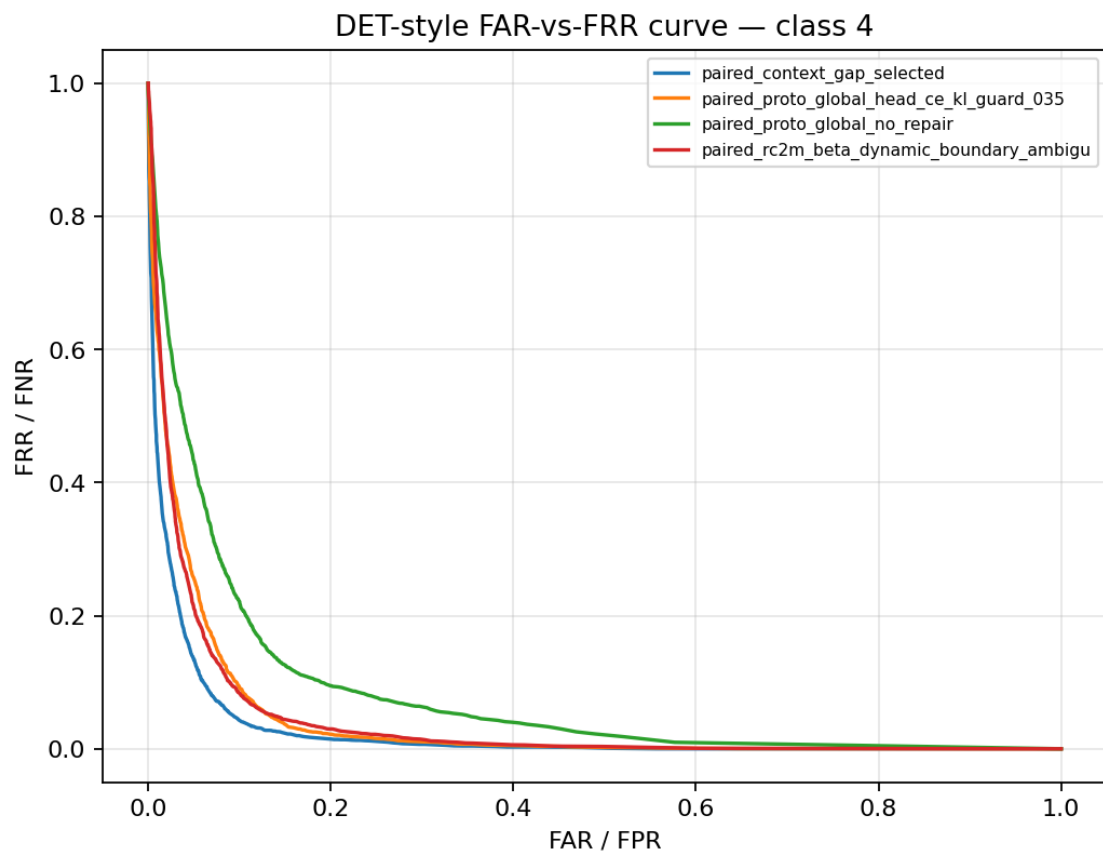
2.31.1 9c. Biometric-style ROC / DET / FAR-FRR plots

These plots are one-vs-rest diagnostics for the final checkpoint. They are generated after policy selection and are not used for selection.

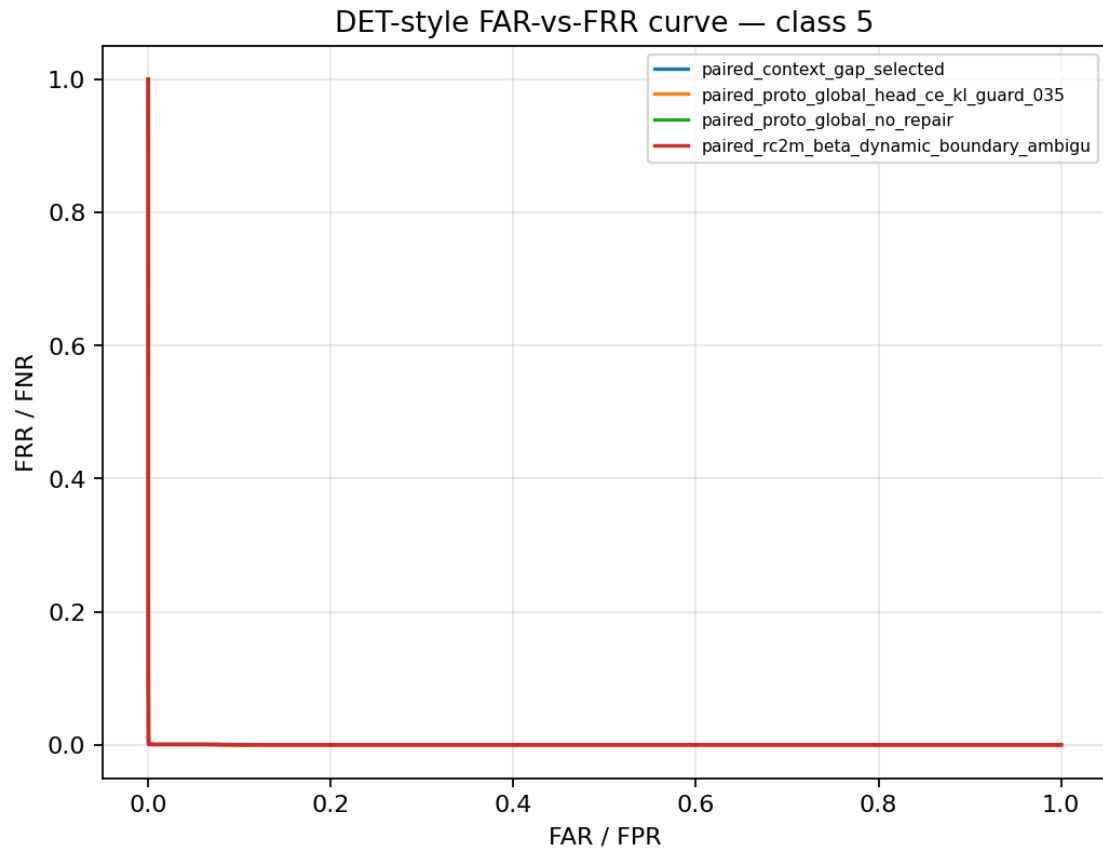
det_style_class2_robust.png



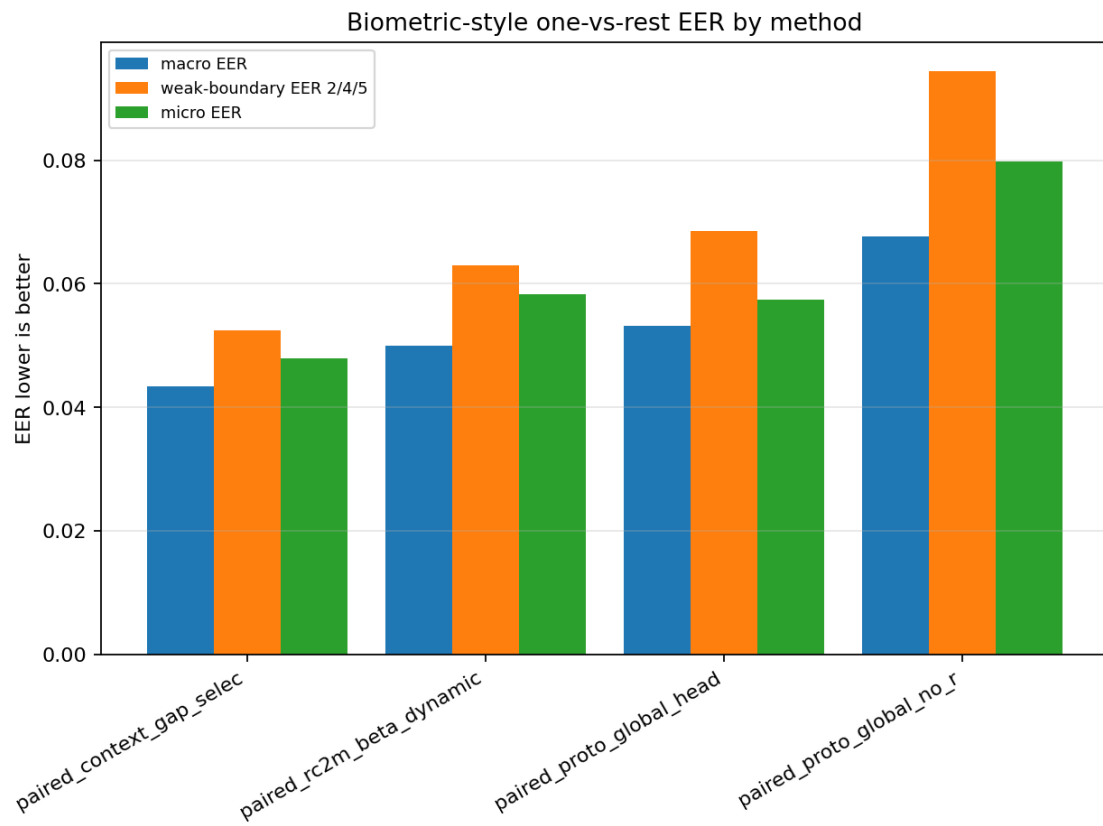
det_style_class4_robust.png



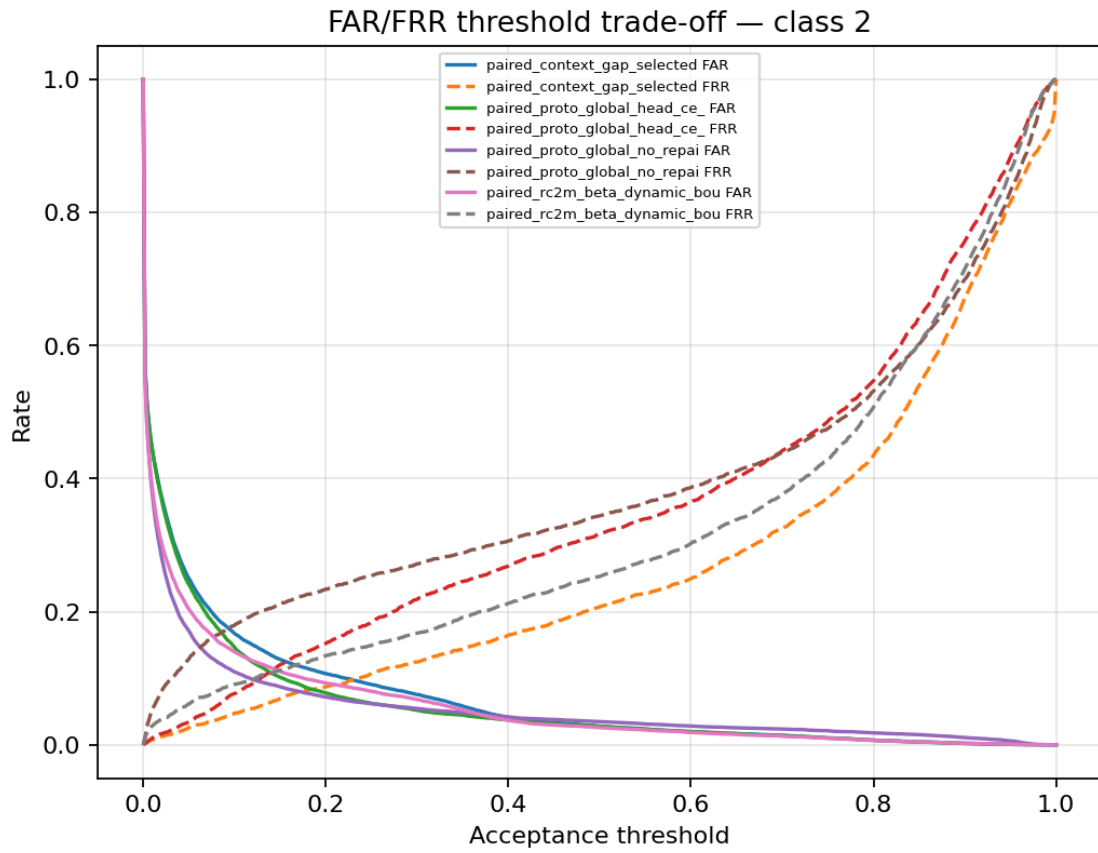
det_style_class5_robust.png



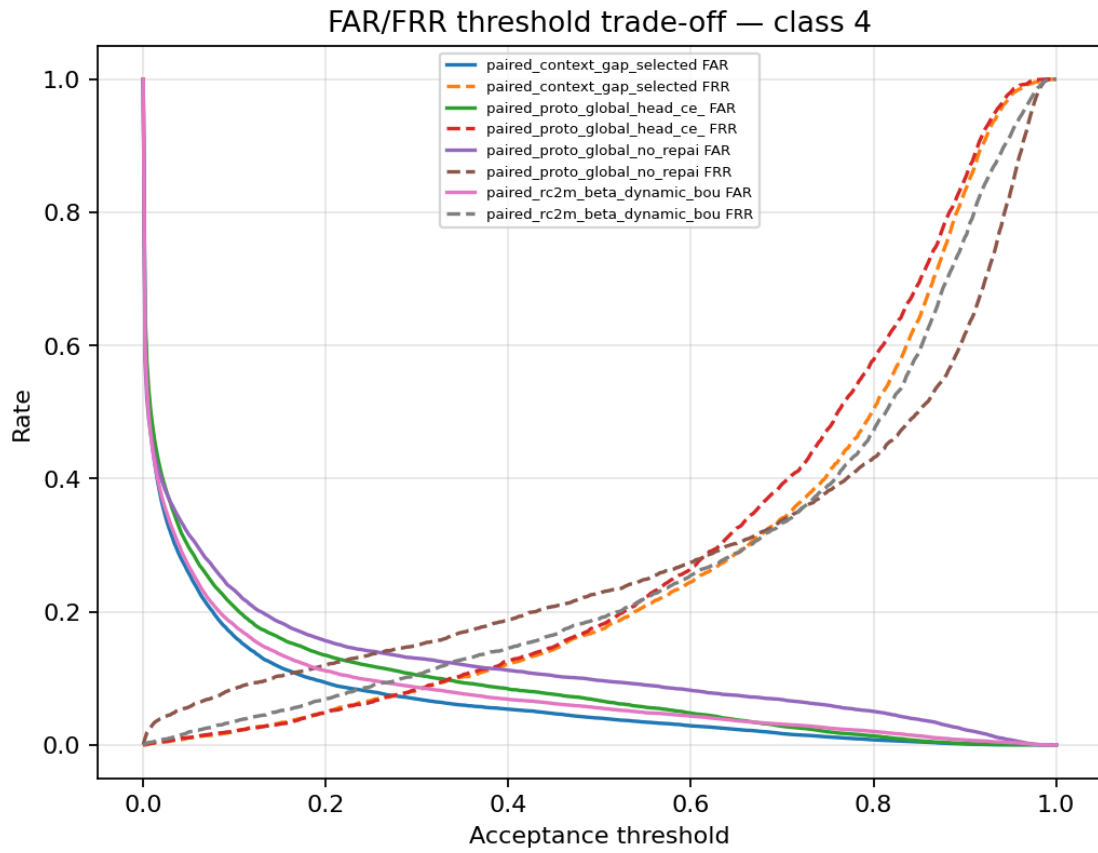
eer_by_method_robust.png



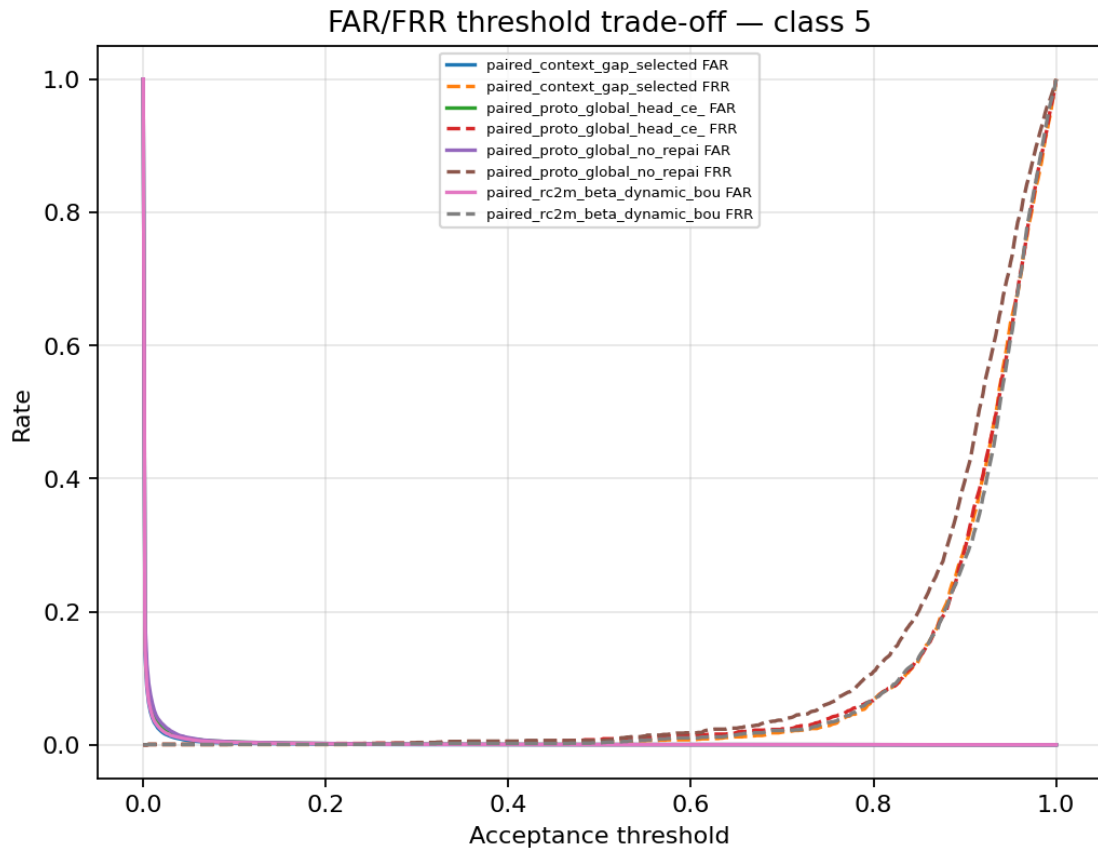
far_frr_class2_robust.png



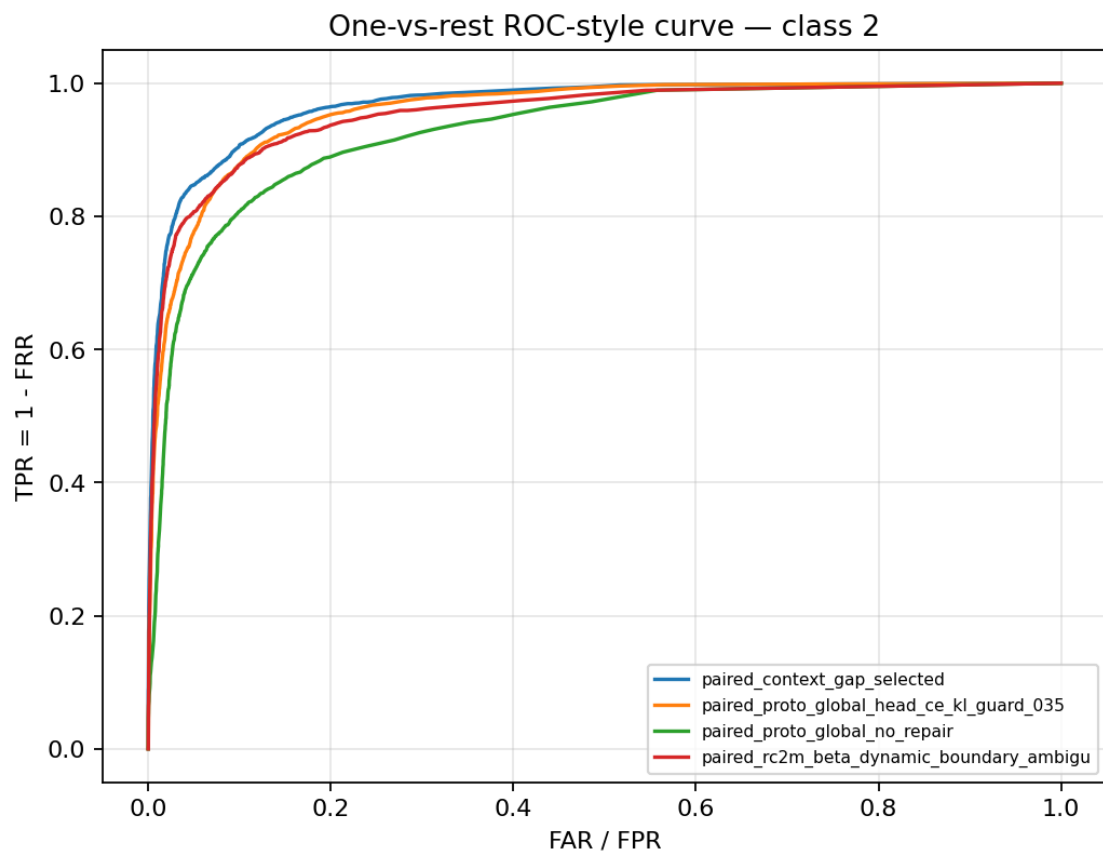
far_frr_class4_robust.png



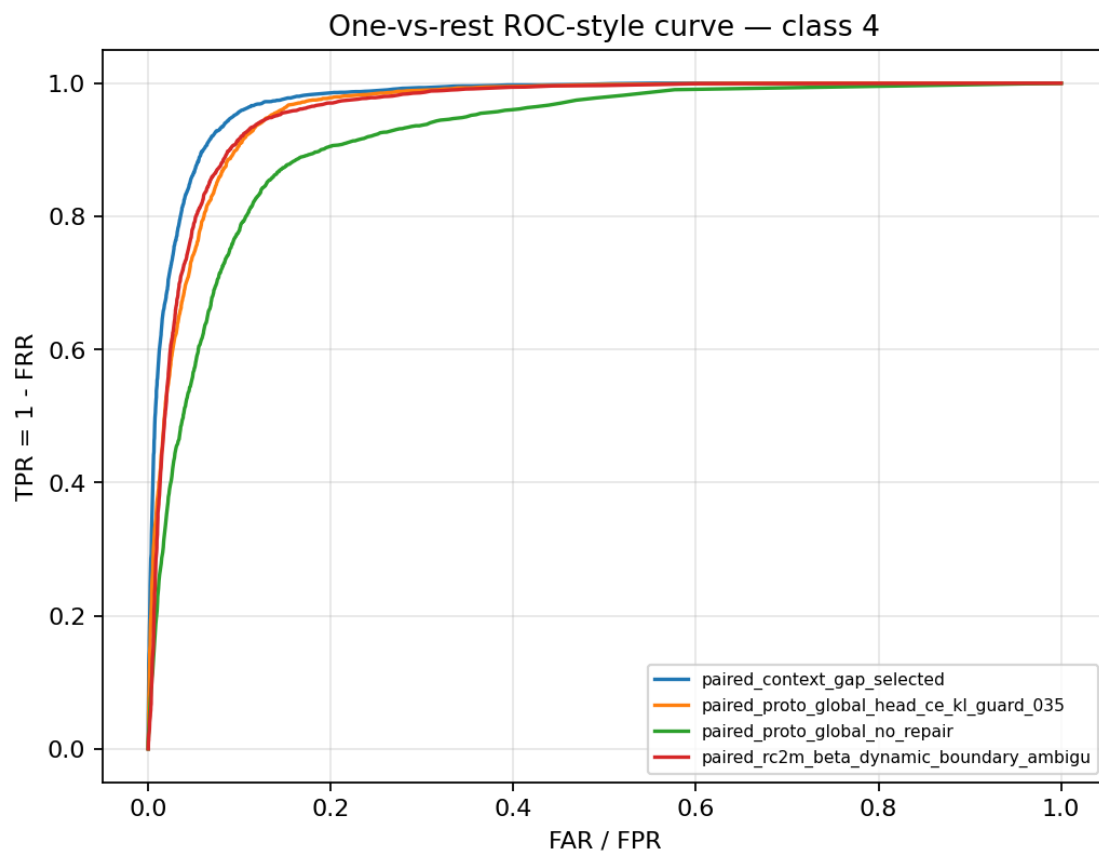
far_frr_class5_robust.png



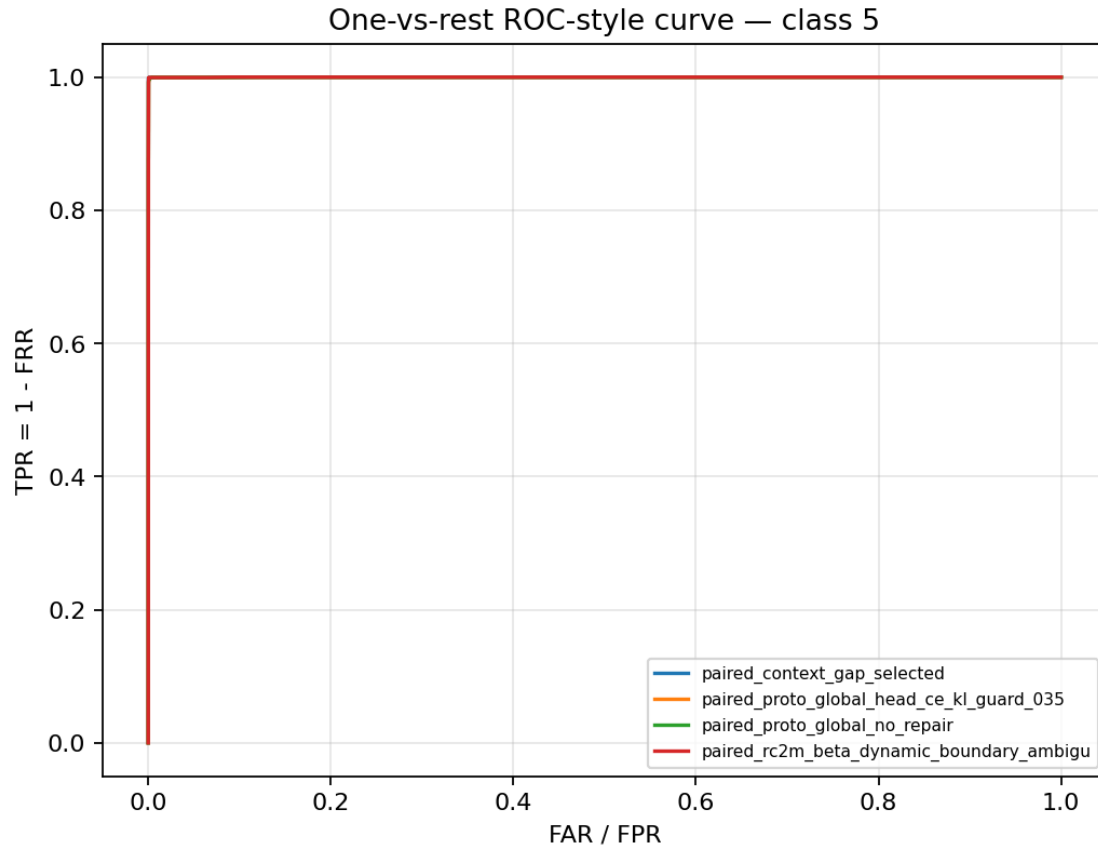
roc_style_class2_robust.png



roc_style_class4_robust.png



roc_style_class5_robust.png



2.32 9d. Biometric-style EER summary table

This table summarizes macro EER, micro EER, weak-boundary EER for classes 2/4/5, and class-level EER/AUC.

Lower EER is better. Higher AUC is better. This remains a multiclass one-vs-rest diagnostic, not a real biometric matcher protocol.

```
[ ]: # =====
# 9d. Display biometric-style EER summary tables
# =====
from IPython.display import display, Markdown

if "eer_by_method_df" not in globals() or not isinstance(eer_by_method_df, pd.
↳DataFrame) or eer_by_method_df.empty:
    raise RuntimeError("No EER summary table was generated. Check score_dump_df_
↳and threshold diagnostics execution.")

cols = [
    "selected_policy",
```

```

    "method",
    "variant",
    "policy_family",
    "macro_eer",
    "micro_eer",
    "weak_boundary_eer_2_4_5",
    "class2_eer",
    "class4_eer",
    "class5_eer",
    "macro_auc",
    "weak_boundary_auc_2_4_5",
]
cols = [c for c in cols if c in eer_by_method_df.columns]

summary_view = (
    eer_by_method_df[cols]
    .sort_values(["selected_policy", "weak_boundary_eer_2_4_5"],
    ↪ascending=[False, True])
    .reset_index(drop=True)
)

display(Markdown("### 9d. EER by method / selected policy"))
display(summary_view)

if "eer_by_class_df" in globals() and isinstance(eer_by_class_df, pd.DataFrame)
    ↪and not eer_by_class_df.empty:
    class_cols = [c for c in [
        "selected_policy", "method", "variant", "class_id", "eer",
    ↪"eer_threshold", "eer_far", "eer_frr", "roc_auc", "n_genuine", "n_impostor"
    ] if c in eer_by_class_df.columns]
    class_view = eer_by_class_df[class_cols].copy()
    # Focus the rendered table on classes 2/4/5 and micro; full CSV remains
    ↪exported.
    class_view = class_view[class_view["class_id"].astype(str).isin(["2", "4",
    ↪"5", "micro_all_classes"])]
    class_view = class_view.sort_values(["selected_policy", "method",
    ↪"class_id"], ascending=[False, True, True]).reset_index(drop=True)
    display(Markdown("### Class-level EER focus: classes 2 / 4 / 5 + micro"))
    display(class_view)

```

2.32.1 9d. EER by method / selected policy

	selected_policy	method \
0	True	paired_rc2m_beta_dynamic_boundary_ambiguity_lo...
1	True	paired_rc2m_beta_dynamic_boundary_ambiguity_lo...
2	True	paired_rc2m_beta_dynamic_boundary_ambiguity_lo...
3	True	paired_rc2m_beta_dynamic_boundary_ambiguity_lo...

```

4      True  paired_rc2m_beta_dynamic_boundary_ambiguity_lo...
5      False paired_context_gap_selected
6      False paired_context_temp_blend_selected_head_guard_035
7      False paired_context_blend_selected_global
8      False paired_context_temp_blend_selected_global
9      False paired_generic_weak_boundary_latent_calibrated
10     False paired_generic_weak_boundary_latent_calibrated
11     False paired_context_gap_selected
12     False paired_context_temp_blend_selected_head_guard_035
13     False paired_proto_global_head_ce_kl_guard_035
14     False paired_context_gap_selected
15     False paired_context_temp_blend_selected_head_guard_035
16     False paired_proto_global_head_ce_kl_guard_035
17     False paired_generic_weak_boundary_latent_calibrated
18     False paired_proto_global_no_repair
19     False paired_context_gap_selected
20     False paired_context_temp_blend_selected_head_guard_035
21     False paired_context_temp_blend_selected_global
22     False paired_generic_weak_boundary_latent_calibrated
23     False paired_proto_global_head_ce_kl_guard_035
24     False paired_proto_global_head_ce_kl_guard_035
25     False paired_context_blend_selected_global
26     False paired_context_temp_blend_selected_global
27     False paired_proto_global_no_repair
28     False paired_context_gap_selected
29     False paired_context_temp_blend_selected_head_guard_035
30     False paired_proto_global_no_repair
31     False paired_context_blend_selected_global
32     False paired_context_temp_blend_selected_global
33     False paired_proto_global_no_repair
34     False paired_context_blend_selected_global
35     False paired_context_temp_blend_selected_global
36     False paired_proto_global_head_ce_kl_guard_035
37     False paired_context_blend_selected_global
38     False paired_generic_weak_boundary_latent_calibrated
39     False paired_proto_global_no_repair

```

```

                                variant \
0  context_temp_blend_selected_head_guard_035
1  context_temp_blend_selected_head_guard_035
2      proto_global_head_ce_kl_guard_035
3      proto_global_head_ce_kl_guard_035
4      context_blend_selected_global
5      context_gap_selected
6  context_temp_blend_selected_head_guard_035
7      context_blend_selected_global
8      context_temp_blend_selected_global
9      generic_weak_boundary_latent_calibrated

```

```

10     generic_weak_boundary_latent_calibrated
11         context_gap_selected
12 context_temp_blend_selected_head_guard_035
13     proto_global_head_ce_kl_guard_035
14         context_gap_selected
15 context_temp_blend_selected_head_guard_035
16     proto_global_head_ce_kl_guard_035
17     generic_weak_boundary_latent_calibrated
18         proto_global_no_repair
19             context_gap_selected
20 context_temp_blend_selected_head_guard_035
21     context_temp_blend_selected_global
22     generic_weak_boundary_latent_calibrated
23         proto_global_head_ce_kl_guard_035
24         proto_global_head_ce_kl_guard_035
25             context_blend_selected_global
26     context_temp_blend_selected_global
27         proto_global_no_repair
28             context_gap_selected
29 context_temp_blend_selected_head_guard_035
30     proto_global_no_repair
31         context_blend_selected_global
32     context_temp_blend_selected_global
33         proto_global_no_repair
34         context_blend_selected_global
35     context_temp_blend_selected_global
36     proto_global_head_ce_kl_guard_035
37         context_blend_selected_global
38     generic_weak_boundary_latent_calibrated
39         proto_global_no_repair

```

	policy_family	macro_eer	micro_eer \
0	rc1b_guarded_context_gap	0.036238	0.041450
1	rc1b_guarded_context_gap	0.046045	0.056450
2	proto_true_global_guard	0.047520	0.051125
3	proto_true_global_guard	0.054010	0.058500
4	context_only_global	0.066328	0.084075
5	rc1b_guarded_context_gap	0.033354	0.032150
6	rc1b_guarded_context_gap	0.033354	0.032150
7	context_only_global	0.035391	0.039775
8	context_only_global	0.035391	0.039775
9	generic_latent_boundary_calibration	0.034711	0.037100
10	generic_latent_boundary_calibration	0.041557	0.050600
11	rc1b_guarded_context_gap	0.036238	0.041450
12	rc1b_guarded_context_gap	0.036238	0.041450
13	proto_true_global_guard	0.036921	0.038550
14	rc1b_guarded_context_gap	0.046045	0.056450
15	rc1b_guarded_context_gap	0.046045	0.056450

16	proto_true_global_guard	0.047520	0.051125
17	generic_latent_boundary_calibration	0.046557	0.047500
18	proto_no_repair	0.046895	0.047675
19	rc1b_guarded_context_gap	0.052784	0.056350
20	rc1b_guarded_context_gap	0.052784	0.056350
21	context_only_global	0.055460	0.064950
22	generic_latent_boundary_calibration	0.052321	0.055425
23	proto_true_global_guard	0.045365	0.052600
24	proto_true_global_guard	0.054010	0.058500
25	context_only_global	0.057477	0.064625
26	context_only_global	0.057477	0.064625
27	proto_no_repair	0.056510	0.062825
28	rc1b_guarded_context_gap	0.048281	0.052975
29	rc1b_guarded_context_gap	0.048281	0.052975
30	proto_no_repair	0.049447	0.055450
31	context_only_global	0.051733	0.057925
32	context_only_global	0.051733	0.057925
33	proto_no_repair	0.060833	0.079125
34	context_only_global	0.066328	0.084075
35	context_only_global	0.066328	0.084075
36	proto_true_global_guard	0.081976	0.086450
37	context_only_global	0.080868	0.095475
38	generic_latent_boundary_calibration	0.086418	0.096800
39	proto_no_repair	0.124737	0.154150

	weak_boundary_eer_2_4_5	class2_eer	class4_eer	class5_eer	macro_auc	\
0	0.049045	0.078594	0.068125	0.000417	0.990090	
1	0.049988	0.078437	0.071250	0.000278	0.985293	
2	0.052685	0.078750	0.078750	0.000556	0.986637	
3	0.066719	0.110156	0.087500	0.002500	0.981859	
4	0.096615	0.156250	0.133594	0.000000	0.969616	
5	0.029688	0.048594	0.040469	0.000000	0.993411	
6	0.029688	0.048594	0.040469	0.000000	0.993411	
7	0.032917	0.042500	0.056250	0.000000	0.991930	
8	0.032917	0.042500	0.056250	0.000000	0.991930	
9	0.043316	0.064844	0.064688	0.000417	0.990826	
10	0.048478	0.063438	0.081719	0.000278	0.989276	
11	0.049045	0.078594	0.068125	0.000417	0.990090	
12	0.049045	0.078594	0.068125	0.000417	0.990090	
13	0.049253	0.077656	0.069688	0.000417	0.990366	
14	0.049988	0.078437	0.071250	0.000278	0.985293	
15	0.049988	0.078437	0.071250	0.000278	0.985293	
16	0.052685	0.078750	0.078750	0.000556	0.986637	
17	0.053420	0.073594	0.086250	0.000417	0.986208	
18	0.053733	0.073594	0.087188	0.000417	0.986237	
19	0.062552	0.103750	0.081406	0.002500	0.982568	
20	0.062552	0.103750	0.081406	0.002500	0.982568	
21	0.063472	0.084844	0.105156	0.000417	0.977573	

22	0.065052	0.105937	0.086719	0.002500	0.982789
23	0.065677	0.108750	0.088281	0.000000	0.983640
24	0.066719	0.110156	0.087500	0.002500	0.981859
25	0.066823	0.125156	0.072813	0.002500	0.981087
26	0.066823	0.125156	0.072813	0.002500	0.981087
27	0.069479	0.106250	0.099688	0.002500	0.980965
28	0.071146	0.119844	0.093594	0.000000	0.981711
29	0.071146	0.119844	0.093594	0.000000	0.981711
30	0.072587	0.113750	0.103594	0.000417	0.984284
31	0.077216	0.118594	0.112500	0.000556	0.982890
32	0.077216	0.118594	0.112500	0.000556	0.982890
33	0.087656	0.142656	0.120312	0.000000	0.973301
34	0.096615	0.156250	0.133594	0.000000	0.969616
35	0.096615	0.156250	0.133594	0.000000	0.969607
36	0.108281	0.166563	0.155781	0.002500	0.961914
37	0.116806	0.095469	0.254531	0.000417	0.957848
38	0.117118	0.203750	0.147187	0.000417	0.958004
39	0.188640	0.266719	0.296563	0.002639	0.920060

weak_boundary_auc_2_4_5	
0	0.983908
1	0.984422
2	0.982126
3	0.974146
4	0.947389
5	0.993409
6	0.993409
7	0.991089
8	0.991089
9	0.985802
10	0.987984
11	0.983908
12	0.983908
13	0.984578
14	0.984422
15	0.984422
16	0.982126
17	0.980391
18	0.980398
19	0.976303
20	0.976303
21	0.972174
22	0.975372
23	0.973133
24	0.974146
25	0.973739
26	0.973739
27	0.972586

28	0.969283
29	0.969283
30	0.972868
31	0.970084
32	0.970084
33	0.954424
34	0.947389
35	0.947372
36	0.943587
37	0.931745
38	0.936361
39	0.864613

2.32.2 Class-level EER focus: classes 2 / 4 / 5 + micro

	selected_policy	method \
0	True	paired_rc2m_beta_dynamic_boundary_ambiguity_lo...
1	True	paired_rc2m_beta_dynamic_boundary_ambiguity_lo...
2	True	paired_rc2m_beta_dynamic_boundary_ambiguity_lo...
3	True	paired_rc2m_beta_dynamic_boundary_ambiguity_lo...
4	True	paired_rc2m_beta_dynamic_boundary_ambiguity_lo...
..	...	...
155	False	paired_proto_global_no_repair
156	False	paired_proto_global_no_repair
157	False	paired_proto_global_no_repair
158	False	paired_proto_global_no_repair
159	False	paired_proto_global_no_repair

	variant	class_id	eer \
0	proto_global_head_ce_kl_guard_035	2	0.110156
1	context_blend_selected_global	2	0.156250
2	proto_global_head_ce_kl_guard_035	2	0.078750
3	context_temp_blend_selected_head_guard_035	2	0.078594
4	context_temp_blend_selected_head_guard_035	2	0.078437
..	...	...	...
155	proto_global_no_repair	micro_all_classes	0.062825
156	proto_global_no_repair	micro_all_classes	0.079125
157	proto_global_no_repair	micro_all_classes	0.047675
158	proto_global_no_repair	micro_all_classes	0.055450
159	proto_global_no_repair	micro_all_classes	0.154150

	eer_threshold	eer_far	eer_frr	roc_auc	n_genuine	n_impostor
0	0.138	0.110312	0.11000	0.958024	800	3200
1	0.036	0.155000	0.15750	0.913861	800	3200
2	0.104	0.078750	0.07875	0.975553	800	3200
3	0.214	0.078437	0.07875	0.972929	800	3200
4	0.392	0.079375	0.07750	0.973779	800	3200
..	...	...	...	...	...	...

155	0.158	0.062900	0.06275	0.981552	4000	20000
156	0.142	0.079250	0.07900	0.973809	4000	20000
157	0.190	0.047850	0.04750	0.988095	4000	20000
158	0.160	0.055650	0.05525	0.987079	4000	20000
159	0.082	0.153800	0.15450	0.918655	4000	20000

[160 rows x 11 columns]

2.33 13. Scorecards and diagnostic tables

The central comparison is paired delta versus paired_proto_global_no_repair and versus the corrected paired_proto_global_head_ce_kl_guard_035, not independent model-vs-model training.

```
[ ]: def final_scorecard(history_df, forgetting_df, confusion_df=None):
    final_task = int(history_df.after_task.max())
    final_hist = history_df[history_df.after_task == final_task]
    final_forgetting = forgetting_df[forgetting_df.after_task == final_task] if
    ↪not forgetting_df.empty else pd.DataFrame(columns=["method", "seed",
    ↪"avg_forgetting"])
    # Ensure optional columns exist.
    for col in ["backbone_params", "readout_trainable_params",
    ↪"readout_total_params"]:
        if col not in final_hist.columns:
            final_hist[col] = np.nan
    acc_summary = final_hist.groupby(["method", "seed"], as_index=False).agg(
        model_family=("model_family", "first"),
        final_avg_acc=("acc", "mean"),
        min_task_acc=("acc", "min"),
        context_entropy_mean=("context_entropy", "mean"),
        context_margin_mean=("context_margin", "mean"),
        context_top1_acc_mean=("context_top1_acc", "mean"),
        latent_frechet_margin_mean=("latent_frechet_margin", "mean"),
        prototype_weight_mean=("prototype_weight", "mean"),
        class2_margin_vs4_mean=("class2_margin_vs4", "mean"),
        class5_margin_vs4_mean=("class5_margin_vs4", "mean"),
        class2_logit_rank_mean=("class2_logit_rank", "mean"),
        class5_logit_rank_mean=("class5_logit_rank", "mean"),
        trainable_params=("trainable_params", "max"),
        total_params=("total_params", "max"),
        backbone_params=("backbone_params", "max"),
        readout_trainable_params=("readout_trainable_params", "max"),
        readout_total_params=("readout_total_params", "max"),
        model_size_mb_total=("model_size_mb_total", "max"),
        probe_row=("probe_row", "max"),
        backbone_source=("backbone_source", "first"),
        probe_feature=("probe_feature", "first"),
        probe_context_mode=("probe_context_mode", "first"),
```

```

)
mid = final_hist[final_hist.task_id.isin([2, 3])].groupby(["method",
↪ "seed"], as_index=False).agg(
    middle_avg_acc=("acc", "mean"), middle_min_acc=("acc", "min")
)
merged = acc_summary.merge(mid, on=["method", "seed"], how="left")
merged = merged.merge(final_forgetting[["method", "seed",
↪ "avg_forgetting"]], on=["method", "seed"], how="left")
merged.loc[merged["probe_row"].astype(bool), "avg_forgetting"] = np.nan
class_diag = class_diagnostics_from_confusion(confusion_df) if confusion_df
↪ is not None and not confusion_df.empty else pd.DataFrame()
if not class_diag.empty:
    merged = merged.merge(class_diag, on=["method", "seed"], how="left")
else:
    for col in ["old_class_acc", "recent_class_acc", "class0_acc",
↪ "class1_acc", "class2_acc", "class3_acc", "class4_acc", "class5_acc",
↪ "min_class_acc", "pred4_bias"]:
        merged[col] = np.nan
summary = merged.groupby("method", as_index=False).agg(
    model_family=("model_family", "first"),
    final_avg_acc_mean=("final_avg_acc", "mean"),
    final_avg_acc_std=("final_avg_acc", "std"),
    min_task_acc_mean=("min_task_acc", "mean"),
    avg_forgetting_mean=("avg_forgetting", "mean"),
    middle_avg_acc_mean=("middle_avg_acc", "mean"),
    middle_min_acc_mean=("middle_min_acc", "mean"),
    old_class_acc_mean=("old_class_acc", "mean"),
    recent_class_acc_mean=("recent_class_acc", "mean"),
    class0_acc_mean=("class0_acc", "mean"),
    class1_acc_mean=("class1_acc", "mean"),
    class2_acc_mean=("class2_acc", "mean"),
    class3_acc_mean=("class3_acc", "mean"),
    class4_acc_mean=("class4_acc", "mean"),
    class5_acc_mean=("class5_acc", "mean"),
    min_class_acc_mean=("min_class_acc", "mean"),
    pred4_bias_mean=("pred4_bias", "mean"),
    class2_margin_vs4_mean=("class2_margin_vs4_mean", "mean"),
    class5_margin_vs4_mean=("class5_margin_vs4_mean", "mean"),
    class2_logit_rank_mean=("class2_logit_rank_mean", "mean"),
    class5_logit_rank_mean=("class5_logit_rank_mean", "mean"),
    trainable_params_mean=("trainable_params", "mean"),
    total_params_mean=("total_params", "mean"),
    backbone_params_mean=("backbone_params", "mean"),
    readout_total_params_mean=("readout_total_params", "mean"),
    model_size_mb_total_mean=("model_size_mb_total", "mean"),
    probe_row=("probe_row", "max"),
    backbone_source=("backbone_source", "first"),

```

```

        probe_feature=("probe_feature", "first"),
        probe_context_mode=("probe_context_mode", "first"),
    )
    return summary.sort_values("final_avg_acc_mean", ascending=False), merged

scorecard_df, per_seed_df = final_scorecard(history_df, forgetting_df,
    ↪confusion_df)
scorecard_path = RESULTS_DIR / f"scorecard_{RUN_MODE}.csv"
per_seed_path = RESULTS_DIR / f"per_seed_scorecard_{RUN_MODE}.csv"
scorecard_df.to_csv(scorecard_path, index=False)
per_seed_df.to_csv(per_seed_path, index=False)
print("Saved scorecards:")
print(" -", scorecard_path)
print(" -", per_seed_path)
display(scorecard_df)

# -----
# Diagnostic patch: separate deployable methods from diagnostic ceilings/probes.
# -----
def method_role(method: str, probe_row: bool = False):
    m = str(method)
    if bool(probe_row):
        return "diagnostic_probe"
    if "joint_upper_bound" in m:
        return "diagnostic_upper_bound"
    if "oracle" in m:
        return "diagnostic_oracle_reference"
    if m.startswith("paired_"):
        return "deployable_mmals_paired"
    if m.startswith("baseline_"):
        return "deployable_continual_baseline"
    return "other"

scorecard_df["method_role"] = [
    method_role(row.method, row.probe_row)
    for row in scorecard_df.itertuples(index=False)
]
per_seed_df["method_role"] = [
    method_role(row.method, row.probe_row)
    for row in per_seed_df.itertuples(index=False)
]

scorecard_path = RESULTS_DIR / f"scorecard_{RUN_MODE}.csv"
per_seed_path = RESULTS_DIR / f"per_seed_scorecard_{RUN_MODE}.csv"
scorecard_df.to_csv(scorecard_path, index=False)
per_seed_df.to_csv(per_seed_path, index=False)

```

```
compact_cols = [
    "method_role", "method", "model_family", "final_avg_acc_mean",
    "min_task_acc_mean", "avg_forgetting_mean", "class2_acc_mean",
    "class5_acc_mean", "pred4_bias_mean", "total_params_mean",
    "probe_row"
]
compact_cols = [c for c in compact_cols if c in scorecard_df.columns]
benchmark_groups_df = scorecard_df[compact_cols].sort_values(
    ["method_role", "final_avg_acc_mean"], ascending=[True, False]
)
benchmark_groups_path = RESULTS_DIR / f"benchmark_method_groups_{RUN_MODE}.csv"
benchmark_groups_df.to_csv(benchmark_groups_path, index=False)
print("Saved benchmark method groups:", benchmark_groups_path)
display(benchmark_groups_df)
```

	method	model_family \
21	probe_proto_zf_oracle	probe
22	probe_proto_zf_proto	probe
9	paired_context_gap_selected	mmals_rc2b_candidate
11	paired_context_temp_blend_selected_head_guard_035	mmals_rc2b_candidate
15	paired_rc2m_beta_dynamic_boundary_ambiguity_lo...	mmals_rc2b_policy
12	paired_generic_weak_boundary_latent_calibrated	mmals_rc2b_candidate
4	baseline_joint_upper_bound	joint_upper_bound
13	paired_proto_global_head_ce_kl_guard_035	mmals_rc2b_candidate
10	paired_context_temp_blend_selected_global	mmals_rc2b_candidate
8	paired_context_blend_selected_global	mmals_rc2b_candidate
14	paired_proto_global_no_repair	mmals_rc2b_candidate
0	baseline_balanced_replay	mlp_baseline
2	baseline_experience_replay	mlp_baseline
19	probe_proto_transformed3_oracle	probe
17	probe_proto_transformed1_oracle	probe
18	probe_proto_transformed2_oracle	probe
16	probe_proto_transformed0_oracle	probe
20	probe_proto_transformed4_oracle	probe
5	baseline_lwf	mlp_baseline
7	baseline_sparse_moe	sparse_moe
1	baseline_ewc	mlp_baseline
3	baseline_finetune	mlp_baseline
6	baseline_pnn	pnn_fixed_columns

	final_avg_acc_mean	final_avg_acc_std	min_task_acc_mean \
21	0.93925	0.011639	0.90325
22	0.90625	0.021984	0.83475
9	0.90100	0.024358	0.81825
11	0.90100	0.024358	0.81825
15	0.88560	0.025278	0.77000
12	0.88035	0.046148	0.75675

4	0.87660	0.004585	0.81850
13	0.87490	0.044259	0.72700
10	0.87120	0.030284	0.74150
8	0.86365	0.035399	0.69325
14	0.84185	0.079184	0.65475
0	0.83900	0.011727	0.69850
2	0.83900	0.011727	0.69850
19	0.78385	0.071010	0.68525
17	0.69890	0.116703	0.52500
18	0.68155	0.108792	0.47775
16	0.67115	0.070136	0.45275
20	0.61240	0.090431	0.35150
5	0.44685	0.023186	0.00125
7	0.29980	0.000209	0.00000
1	0.29975	0.000177	0.00000
3	0.29970	0.000209	0.00000
6	0.29920	0.000716	0.00000

	avg_forgetting_mean	middle_avg_acc_mean	middle_min_acc_mean	\
21	NaN	0.920000	0.90375	
22	NaN	0.854875	0.83625	
9	0.055625	0.876250	0.83200	
11	0.055625	0.876250	0.83200	
15	0.069312	0.849750	0.78350	
12	0.073313	0.828750	0.79200	
4	0.000000	0.840250	0.82300	
13	0.085812	0.811500	0.72700	
10	0.076250	0.820250	0.74150	
8	0.070187	0.824000	0.73400	
14	0.100875	0.759000	0.65475	
0	0.135187	0.776250	0.69850	
2	0.135187	0.776250	0.69850	
19	NaN	0.743750	0.68525	
17	NaN	0.595875	0.53525	
18	NaN	0.626750	0.50800	
16	NaN	0.609000	0.45275	
20	NaN	0.466125	0.35150	
5	0.607125	0.604125	0.35625	
7	0.844688	0.250000	0.00000	
1	0.842250	0.250000	0.00000	
3	0.841187	0.250000	0.00000	
6	0.838125	0.249500	0.00000	

	old_class_acc_mean	recent_class_acc_mean	...	class5_logit_rank_mean	\
21	0.949083	0.940167	...	1.0255	
22	0.921917	0.913583	...	1.0135	
9	0.910417	0.917833	...	1.0030	
11	0.910417	0.917833	...	1.0030	

15	0.893000	0.909250	...	1.0035
12	0.898000	0.894083	...	1.0035
4	0.896583	0.873917	...	1.0660
13	0.874333	0.908917	...	1.0095
10	0.902083	0.875750	...	1.0030
8	0.910333	0.856083	...	1.0030
14	0.863500	0.865417	...	1.0075
0	0.806833	0.903167	...	1.0060
2	0.806833	0.903167	...	1.0060
19	0.793417	0.799833	...	1.0585
17	0.719167	0.705417	...	1.1740
18	0.671833	0.722583	...	1.0475
16	0.738500	0.652667	...	1.1920
20	0.725500	0.576667	...	1.0370
5	0.026667	0.892750	...	1.0465
7	0.000000	0.666083	...	1.0015
1	0.000000	0.665917	...	1.0020
3	0.000000	0.665750	...	1.0025
6	0.000000	0.664583	...	1.0045

	trainable_params_mean	total_params_mean	backbone_params_mean	\
21	390.0	390.0	NaN	
22	390.0	390.0	NaN	
9	390.0	1337606.0	1337216.0	
11	390.0	1337606.0	1337216.0	
15	267833.2	1337606.0	1337216.0	
12	1074259.8	1341703.0	1337216.0	
4	217798.0	217798.0	217798.0	
13	390.0	1337606.0	1337216.0	
10	1337606.0	1337606.0	1337216.0	
8	1337606.0	1337606.0	1337216.0	
14	1337606.0	1337606.0	1337216.0	
0	217798.0	217798.0	217798.0	
2	217798.0	217798.0	217798.0	
19	390.0	390.0	NaN	
17	390.0	390.0	NaN	
18	390.0	390.0	NaN	
16	390.0	390.0	NaN	
20	390.0	390.0	NaN	
5	217798.0	217798.0	217798.0	
7	1188555.0	1188555.0	1188555.0	
1	217798.0	217798.0	217798.0	
3	217798.0	217798.0	217798.0	
6	219334.0	1088966.0	1088966.0	

	readout_total_params_mean	model_size_mb_total_mean	probe_row	\
21	NaN	0.001488	True	
22	NaN	0.001488	True	

9	390.0	5.102562	False
11	390.0	5.102562	False
15	390.0	5.102562	False
12	4487.0	5.118191	False
4	NaN	0.830833	False
13	390.0	5.102562	False
10	390.0	5.102562	False
8	390.0	5.102562	False
14	390.0	5.102562	False
0	NaN	0.830833	False
2	NaN	0.830833	False
19	NaN	0.001488	True
17	NaN	0.001488	True
18	NaN	0.001488	True
16	NaN	0.001488	True
20	NaN	0.001488	True
5	NaN	0.830833	False
7	NaN	4.533978	False
1	NaN	0.830833	False
3	NaN	0.830833	False
6	NaN	4.154076	False

	backbone_source	probe_feature \
21	mmals_proto_first_balanced_replay	zf
22	mmals_proto_first_balanced_replay	zf
9	same_proto_first_checkpoint	original_backbone_zf_rc2b_policy
11	same_proto_first_checkpoint	original_backbone_zf_rc2b_policy
15	same_proto_first_checkpoint	original_backbone_zf_rc2b_policy
12	same_proto_first_checkpoint	original_backbone_zf_rc2b_policy
4	baseline_sequential_or_joint	baseline_logits
13	same_proto_first_checkpoint	original_backbone_zf_rc2b_policy
10	same_proto_first_checkpoint	original_backbone_zf_rc2b_policy
8	same_proto_first_checkpoint	original_backbone_zf_rc2b_policy
14	same_proto_first_checkpoint	original_backbone_zf_rc2b_policy
0	baseline_sequential_or_joint	baseline_logits
2	baseline_sequential_or_joint	baseline_logits
19	mmals_proto_first_balanced_replay	transformed3
17	mmals_proto_first_balanced_replay	transformed1
18	mmals_proto_first_balanced_replay	transformed2
16	mmals_proto_first_balanced_replay	transformed0
20	mmals_proto_first_balanced_replay	transformed4
5	baseline_sequential_or_joint	baseline_logits
7	baseline_sequential_or_joint	baseline_logits
1	baseline_sequential_or_joint	baseline_logits
3	baseline_sequential_or_joint	baseline_logits
6	baseline_sequential_or_joint	baseline_logits

probe_context_mode

21	oracle
22	proto
9	proto_first_rc2b_strict
11	proto_first_rc2b_strict
15	proto_first_rc2b_strict
12	proto_first_rc2b_strict
4	none
13	proto_first_rc2b_strict
10	proto_first_rc2b_strict
8	proto_first_rc2b_strict
14	proto_first_rc2b_strict
0	none
2	none
19	oracle
17	oracle
18	oracle
16	oracle
20	oracle
5	none
7	none
1	none
3	none
6	none

[23 rows x 31 columns]

	method_role \
0	deployable_continual_baseline
2	deployable_continual_baseline
5	deployable_continual_baseline
7	deployable_continual_baseline
1	deployable_continual_baseline
3	deployable_continual_baseline
6	deployable_continual_baseline
9	deployable_mmals_paired
11	deployable_mmals_paired
15	deployable_mmals_paired
12	deployable_mmals_paired
13	deployable_mmals_paired
10	deployable_mmals_paired
8	deployable_mmals_paired
14	deployable_mmals_paired
21	diagnostic_probe
22	diagnostic_probe
19	diagnostic_probe
17	diagnostic_probe
18	diagnostic_probe
16	diagnostic_probe

20	diagnostic_probe		
4	diagnostic_upper_bound		

	method	model_family \
0	baseline_balanced_replay	mlp_baseline
2	baseline_experience_replay	mlp_baseline
5	baseline_lwf	mlp_baseline
7	baseline_sparse_moe	sparse_moe
1	baseline_ewc	mlp_baseline
3	baseline_finetune	mlp_baseline
6	baseline_pnn	pnn_fixed_columns
9	paired_context_gap_selected	mmals_rc2b_candidate
11	paired_context_temp_blend_selected_head_guard_035	mmals_rc2b_candidate
15	paired_rc2m_beta_dynamic_boundary_ambiguity_lo...	mmals_rc2b_policy
12	paired_generic_weak_boundary_latent_calibrated	mmals_rc2b_candidate
13	paired_proto_global_head_ce_kl_guard_035	mmals_rc2b_candidate
10	paired_context_temp_blend_selected_global	mmals_rc2b_candidate
8	paired_context_blend_selected_global	mmals_rc2b_candidate
14	paired_proto_global_no_repair	mmals_rc2b_candidate
21	probe_proto__zf_oracle	probe
22	probe_proto__zf_proto	probe
19	probe_proto__transformed3_oracle	probe
17	probe_proto__transformed1_oracle	probe
18	probe_proto__transformed2_oracle	probe
16	probe_proto__transformed0_oracle	probe
20	probe_proto__transformed4_oracle	probe
4	baseline_joint_upper_bound	joint_upper_bound

	final_avg_acc_mean	min_task_acc_mean	avg_forgetting_mean \
0	0.83900	0.69850	0.135187
2	0.83900	0.69850	0.135187
5	0.44685	0.00125	0.607125
7	0.29980	0.00000	0.844688
1	0.29975	0.00000	0.842250
3	0.29970	0.00000	0.841187
6	0.29920	0.00000	0.838125
9	0.90100	0.81825	0.055625
11	0.90100	0.81825	0.055625
15	0.88560	0.77000	0.069312
12	0.88035	0.75675	0.073313
13	0.87490	0.72700	0.085812
10	0.87120	0.74150	0.076250
8	0.86365	0.69325	0.070187
14	0.84185	0.65475	0.100875
21	0.93925	0.90325	NaN
22	0.90625	0.83475	NaN
19	0.78385	0.68525	NaN
17	0.69890	0.52500	NaN

18	0.68155	0.47775	NaN
16	0.67115	0.45275	NaN
20	0.61240	0.35150	NaN
4	0.87660	0.81850	0.000000

	class2_acc_mean	class5_acc_mean	pred4_bias_mean	total_params_mean \
0	0.61875	0.9970	0.240417	217798.0
2	0.61875	0.9970	0.240417	217798.0
5	0.00100	0.9715	0.473083	217798.0
7	0.00000	0.9985	0.830750	1188555.0
1	0.00000	0.9980	0.830042	217798.0
3	0.00000	0.9975	0.830917	217798.0
6	0.00000	0.9955	0.824125	1088966.0
9	0.82350	0.9985	0.176375	1337606.0
11	0.82350	0.9985	0.176375	1337606.0
15	0.77350	0.9975	0.184333	1337606.0
12	0.79125	0.9975	0.192875	1341703.0
13	0.71900	0.9925	0.196917	1337606.0
10	0.80475	0.9980	0.185833	1337606.0
8	0.82150	0.9980	0.171125	1337606.0
14	0.68250	0.9940	0.206958	1337606.0
21	0.91000	0.9775	0.171625	390.0
22	0.84125	0.9895	0.168125	390.0
19	0.75100	0.9720	0.156250	390.0
17	0.62925	0.9135	0.177500	390.0
18	0.62125	0.9710	0.135083	390.0
16	0.70275	0.9155	0.116417	390.0
20	0.67900	0.9805	0.093500	390.0
4	0.82300	0.9425	0.179000	217798.0

	probe_row
0	False
2	False
5	False
7	False
1	False
3	False
6	False
9	False
11	False
15	False
12	False
13	False
10	False
8	False
14	False
21	True
22	True

```

19         True
17         True
18         True
16         True
20         True
4         False

```

2.34 14. Plots

Quick visual debugging plots for the paired RC1b candidate-freeze audit. These are not publication figures.

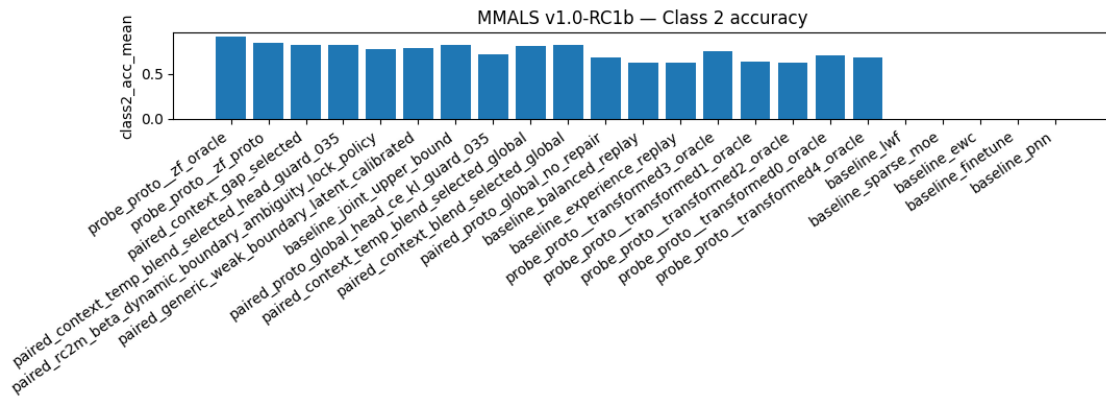
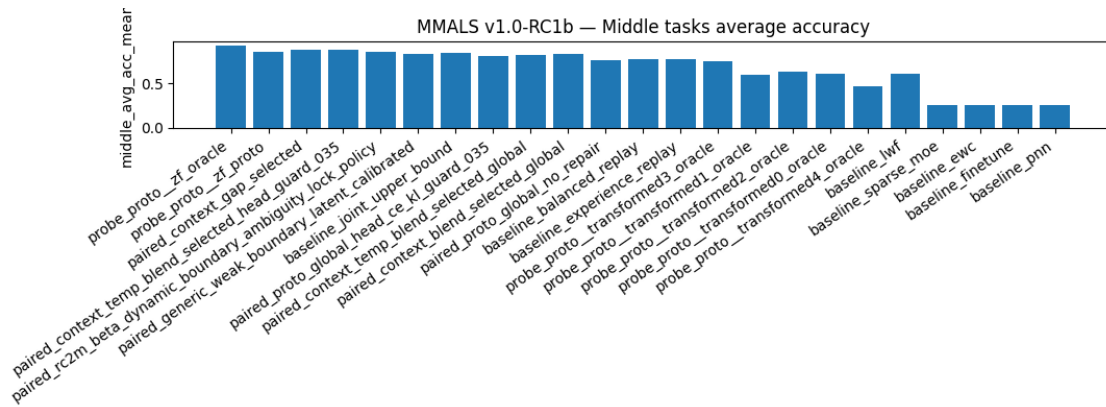
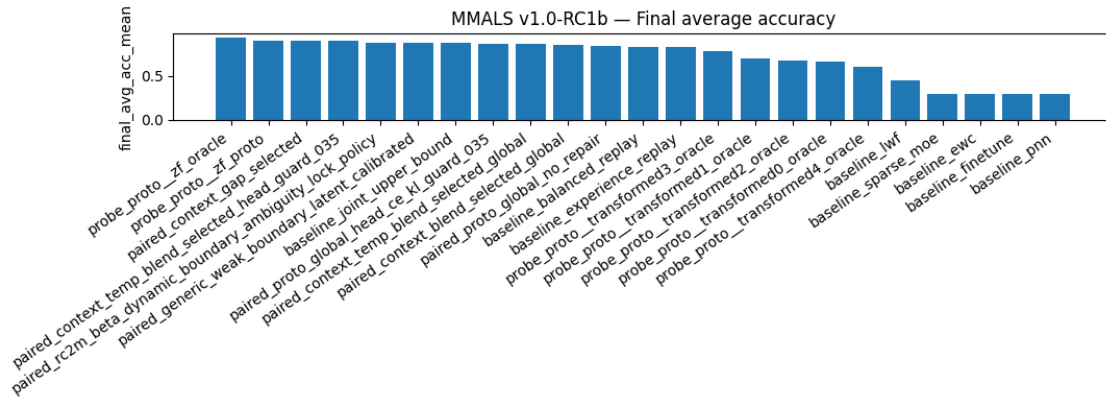
```

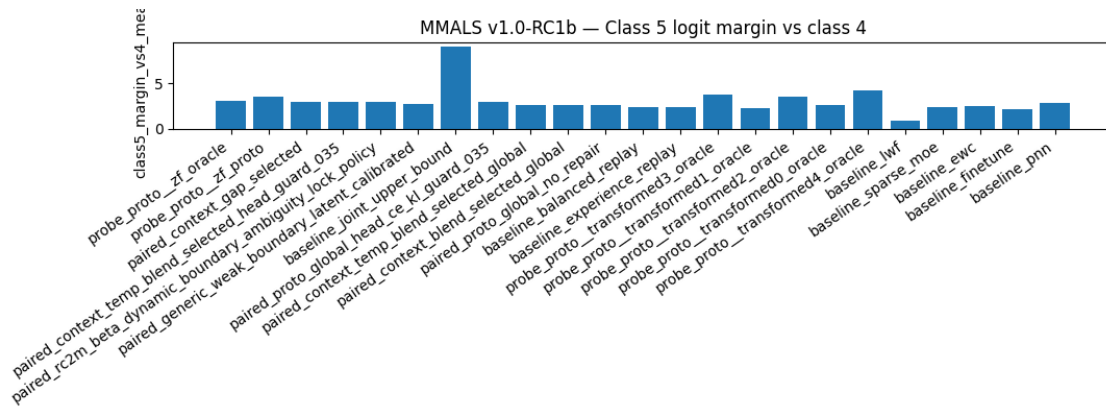
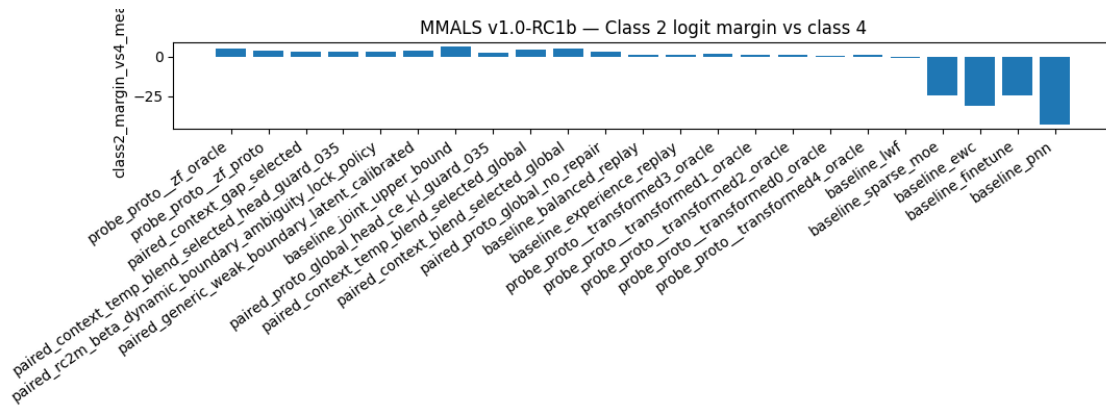
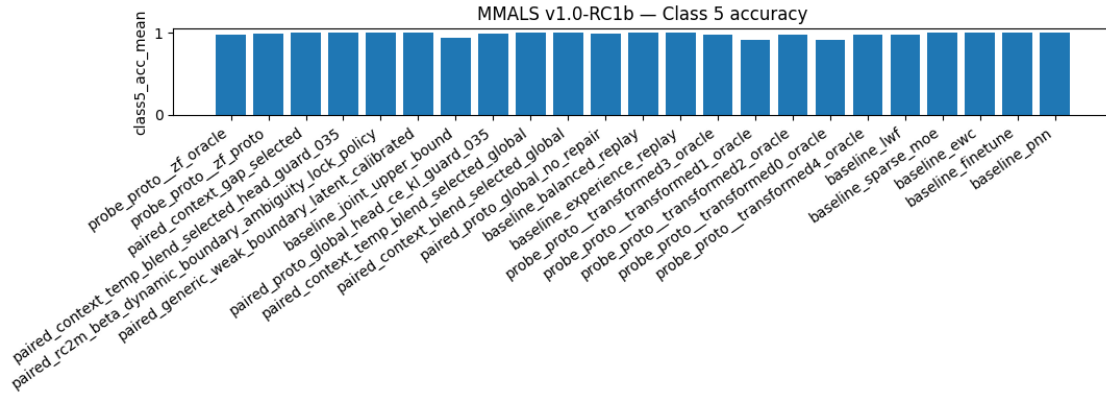
[ ]: if not scorecard_df.empty:
    for metric, title in [
        ("final_avg_acc_mean", "Final average accuracy"),
        ("middle_avg_acc_mean", "Middle tasks average accuracy"),
        ("class2_acc_mean", "Class 2 accuracy"),
        ("class5_acc_mean", "Class 5 accuracy"),
        ("class2_margin_vs4_mean", "Class 2 logit margin vs class 4"),
        ("class5_margin_vs4_mean", "Class 5 logit margin vs class 4"),
        ("pred4_bias_mean", "Predicted class-4 bias"),
        ("readout_total_params_mean", "Readout parameters"),
        ("total_params_mean", "Total parameters"),
    ]:
        if metric not in scorecard_df.columns:
            continue
        plt.figure(figsize=(11, 4))
        plt.bar(scorecard_df["method"], scorecard_df[metric])
        plt.xticks(rotation=35, ha="right")
        plt.ylabel(metric)
        plt.title(f"MMALS v1.0-RC1b - {title}")
        plt.tight_layout()
        plt.show()

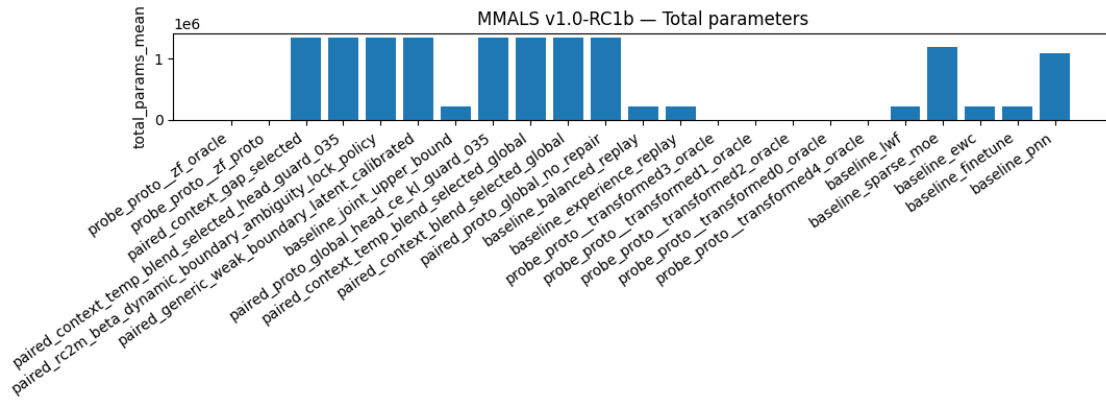
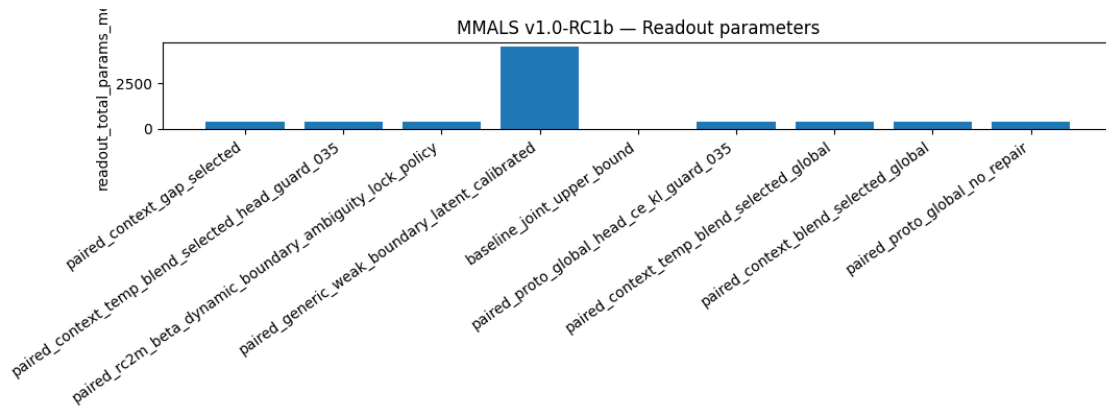
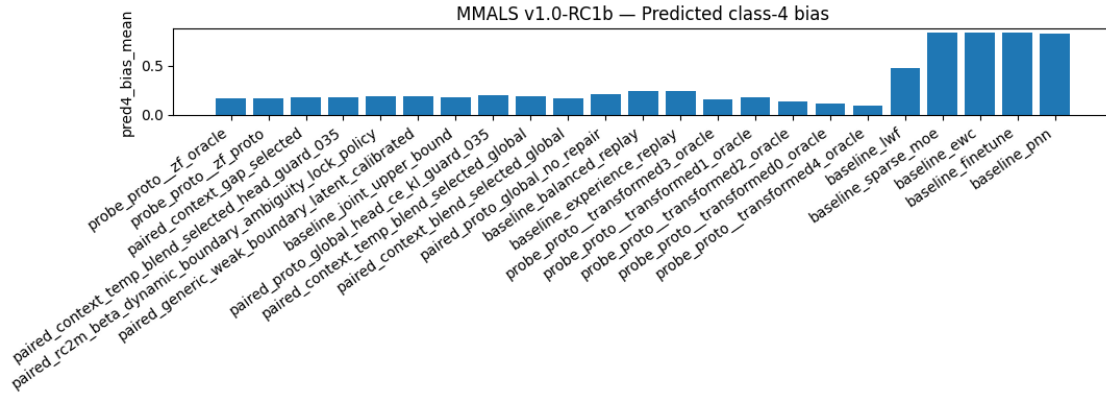
if 'confusion_df' in globals() and not confusion_df.empty:
    for method in confusion_df.method.unique():
        cm = confusion_df[confusion_df.method == method].pivot_table(
            index="true_class", columns="pred_class", values="rate",
            fill_value=0.0
        ).reindex(index=range(N_CLASSES), columns=range(N_CLASSES),
            fill_value=0.0)
        plt.figure(figsize=(5, 4))
        plt.imshow(cm.values, aspect="auto")
        plt.xticks(range(N_CLASSES), range(N_CLASSES))
        plt.yticks(range(N_CLASSES), range(N_CLASSES))
        plt.xlabel("Predicted class")
        plt.ylabel("True class")
        plt.title(f"Class confusion - {method}")

```

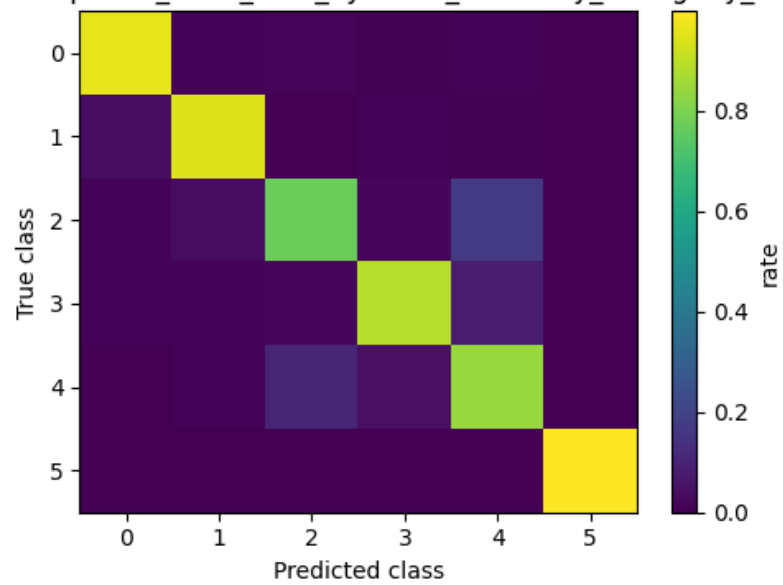
```
plt.colorbar(label="rate")
plt.tight_layout()
plt.show()
```



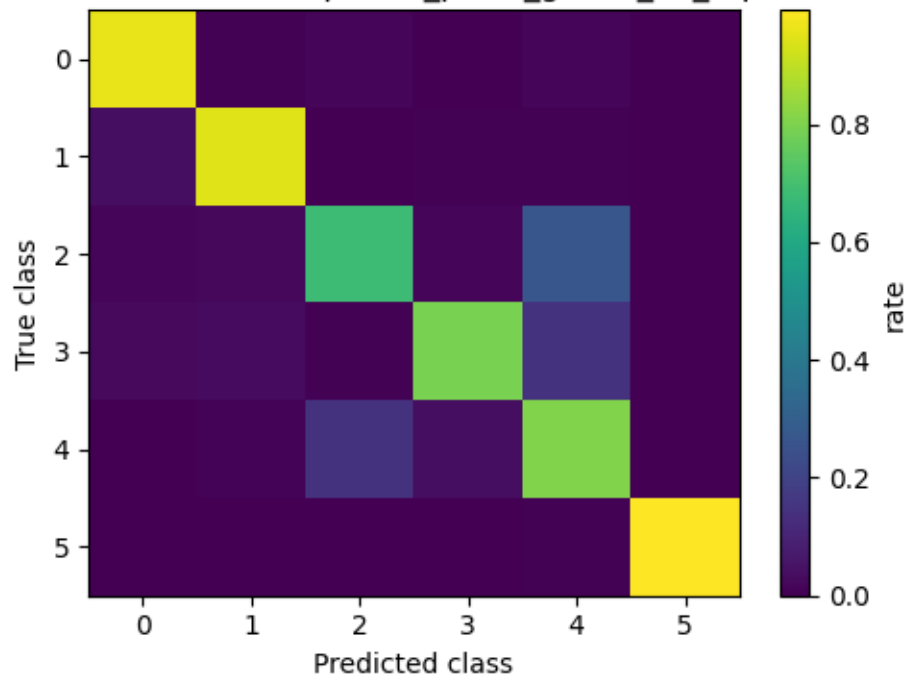




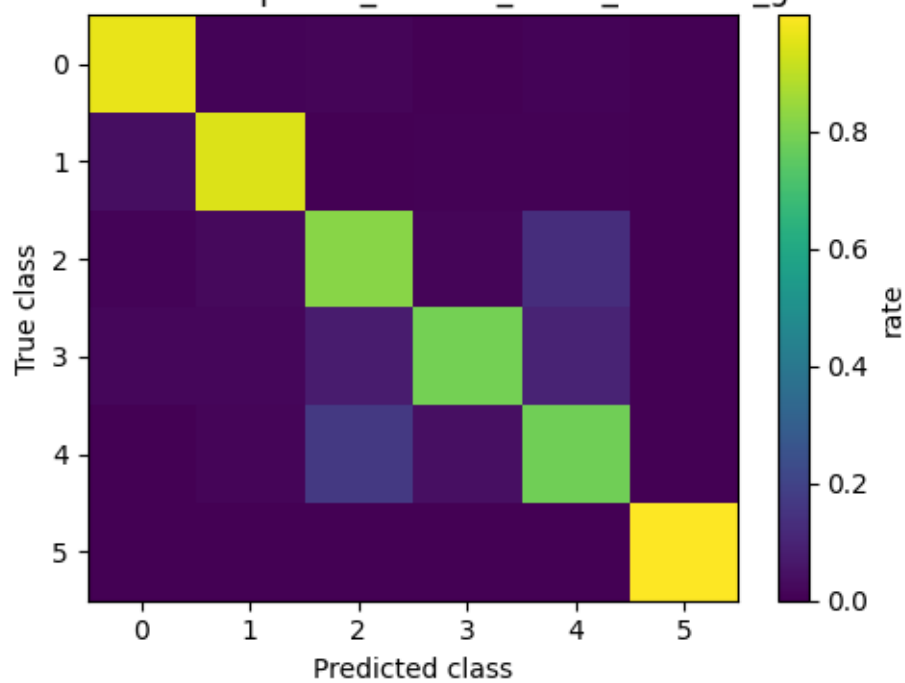
Class confusion — paired_rc2m_beta_dynamic_boundary_ambiguity_lock_policy



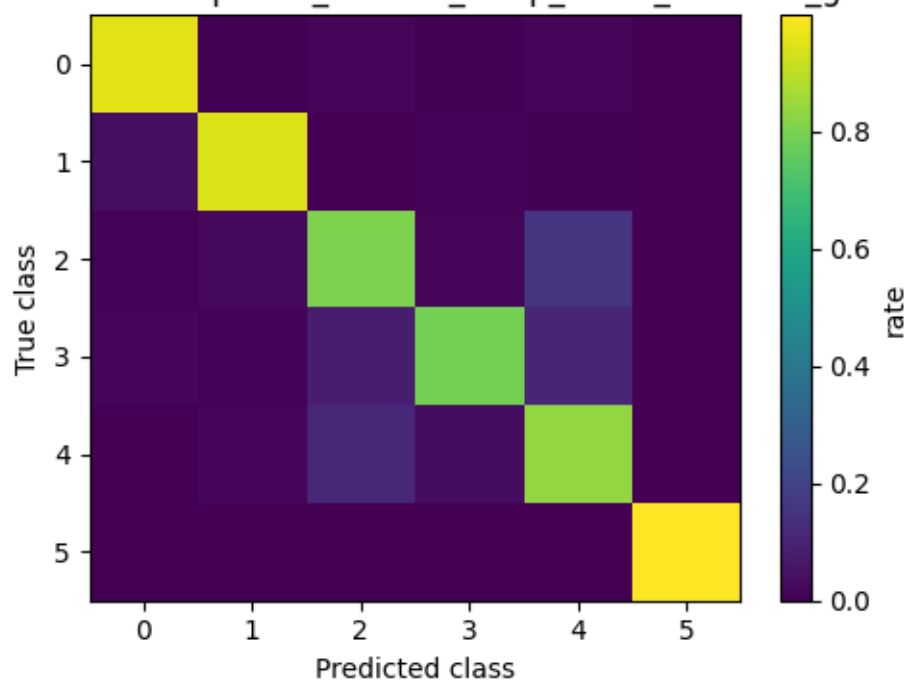
Class confusion — paired_proto_global_no_repair



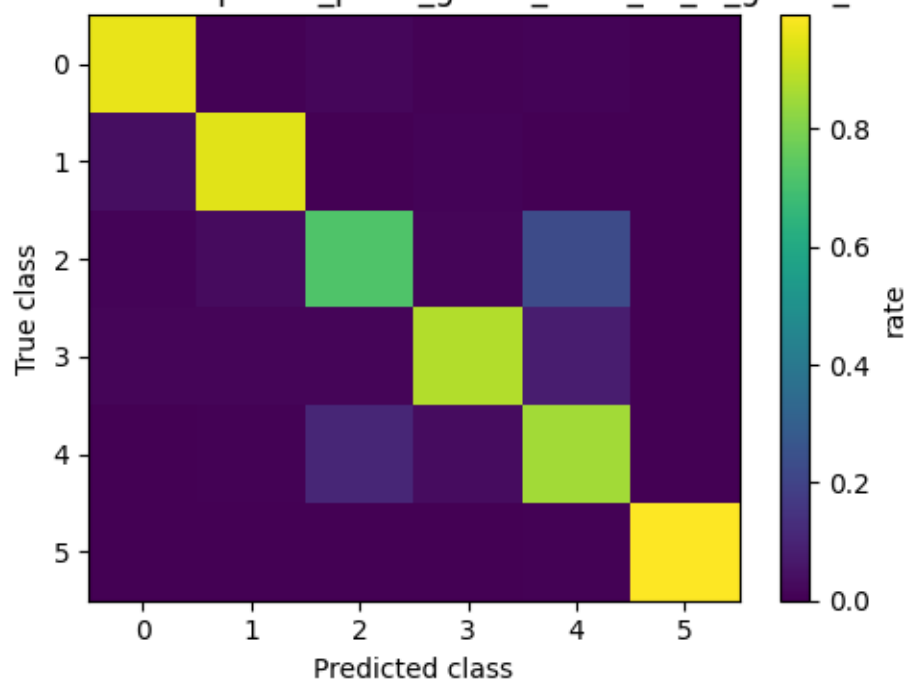
Class confusion — paired_context_blend_selected_global



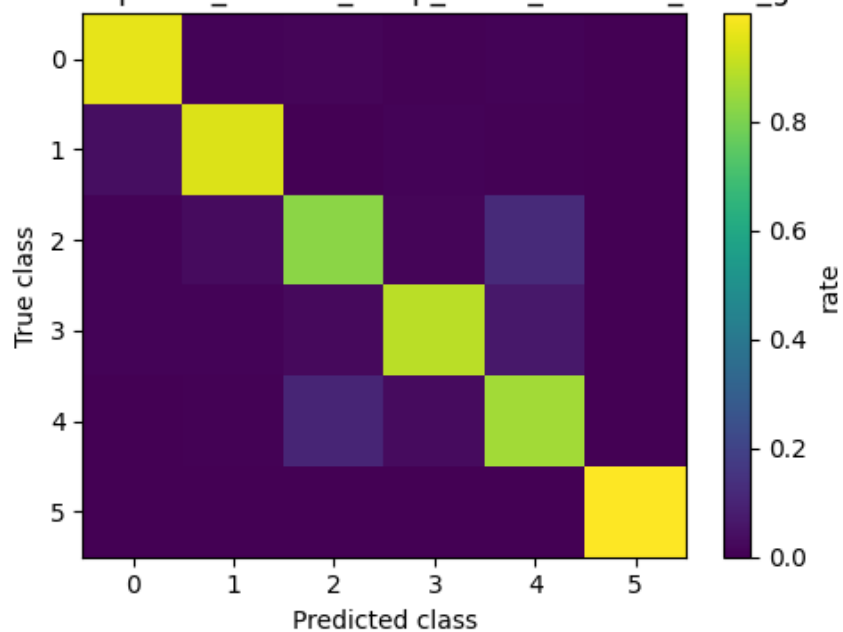
Class confusion — paired_context_temp_blend_selected_global



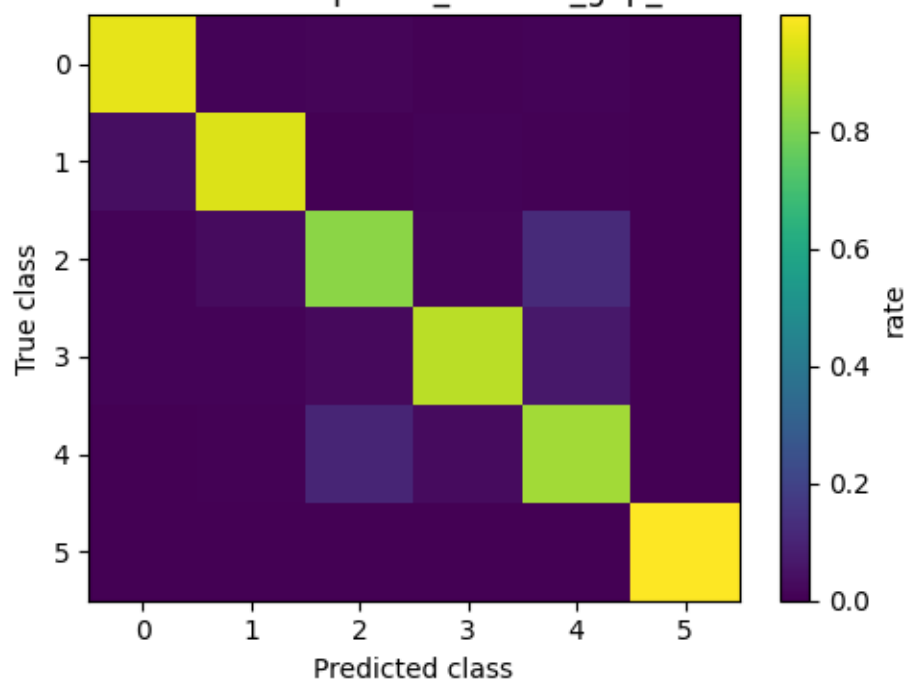
Class confusion — paired_proto_global_head_ce_kl_guard_035



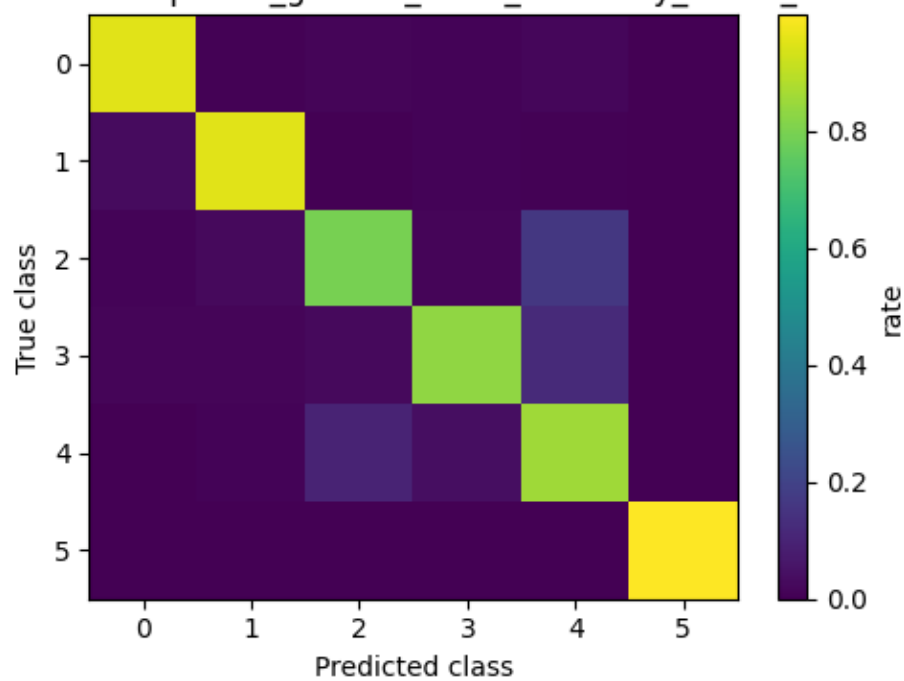
Class confusion — paired_context_temp_blend_selected_head_guard_035

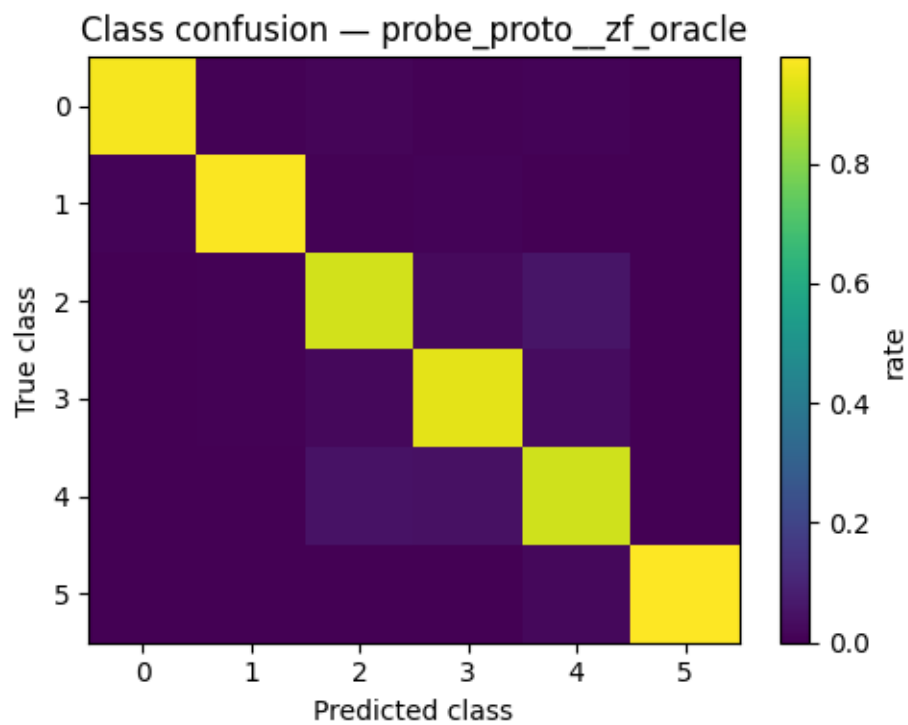
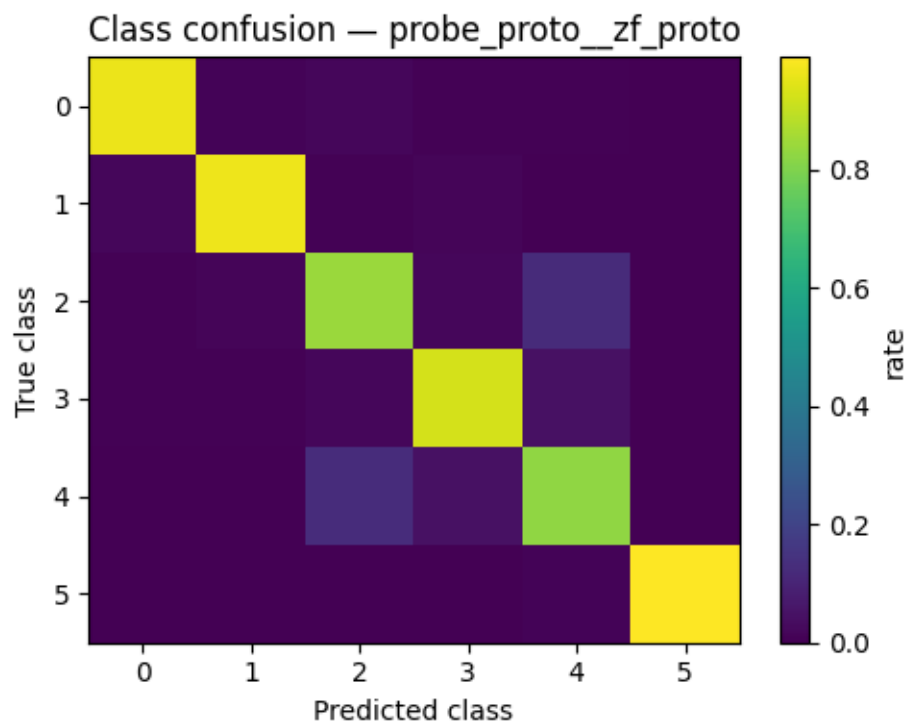


Class confusion — paired_context_gap_selected

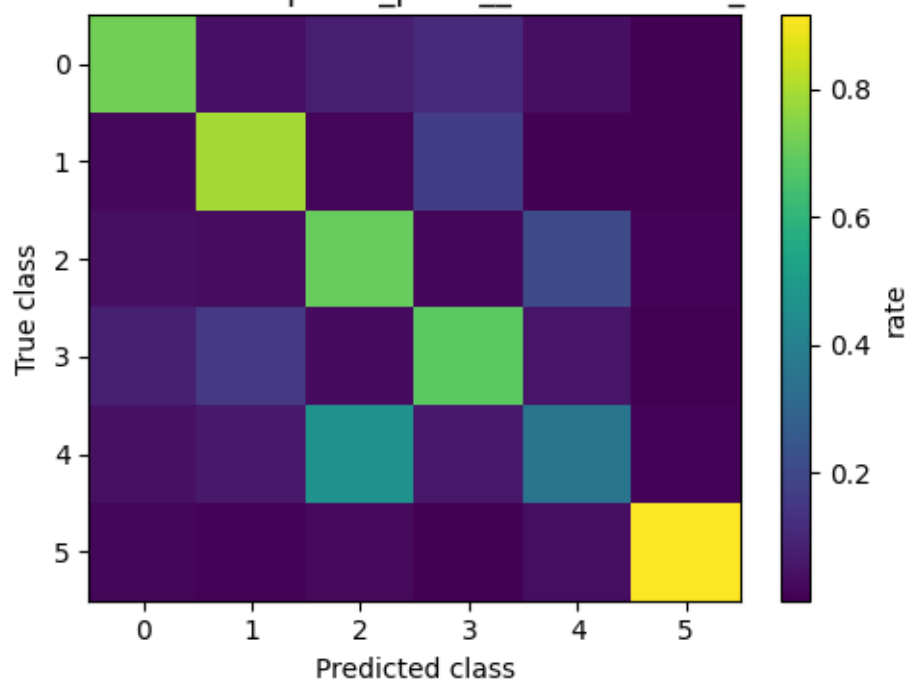


Class confusion — paired_generic_weak_boundary_latent_calibrated

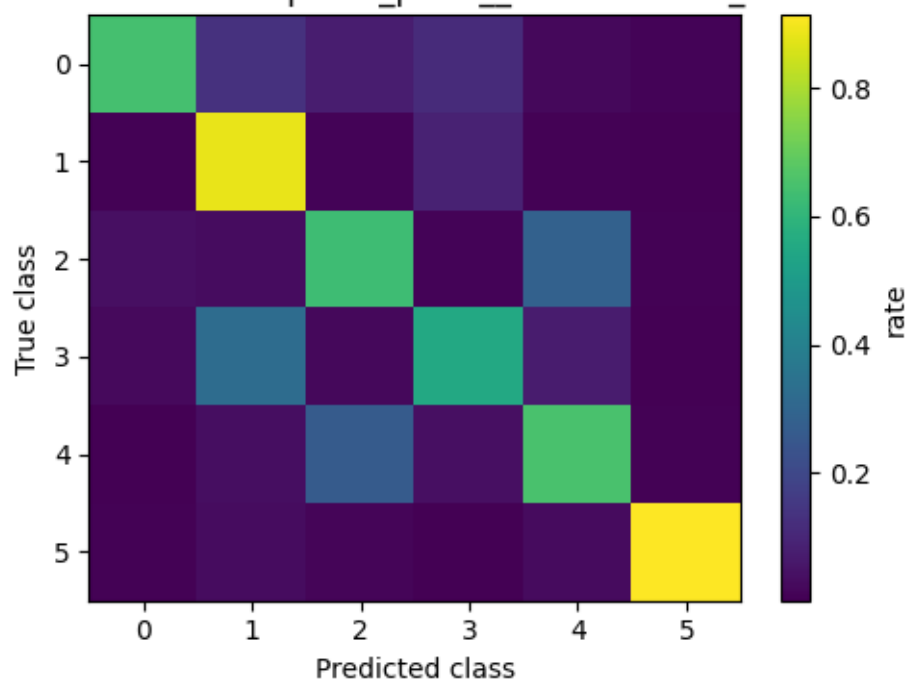




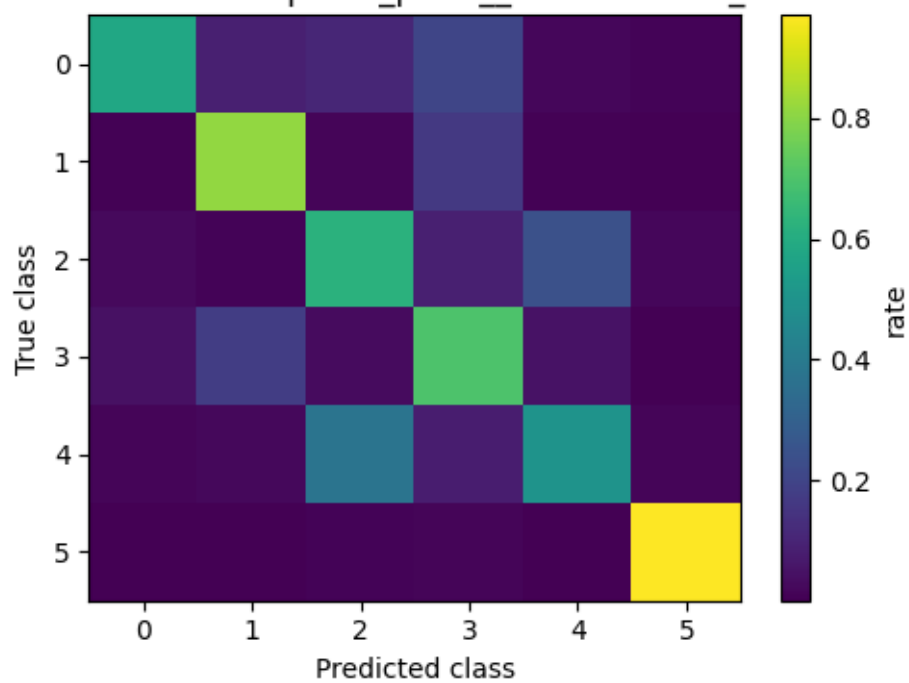
Class confusion — probe_proto_transformed0_oracle



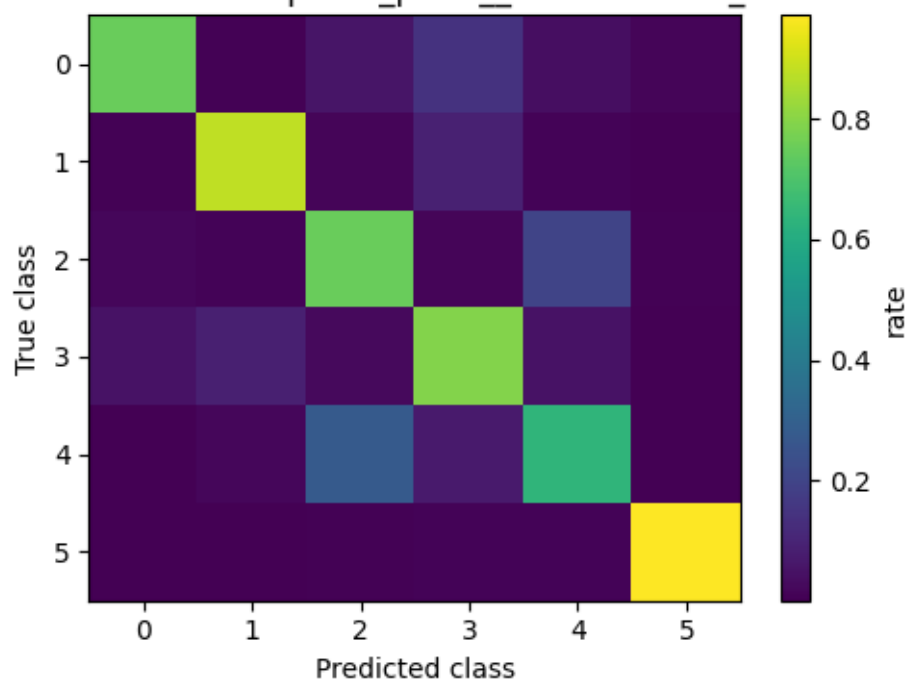
Class confusion — probe_proto_transformed1_oracle



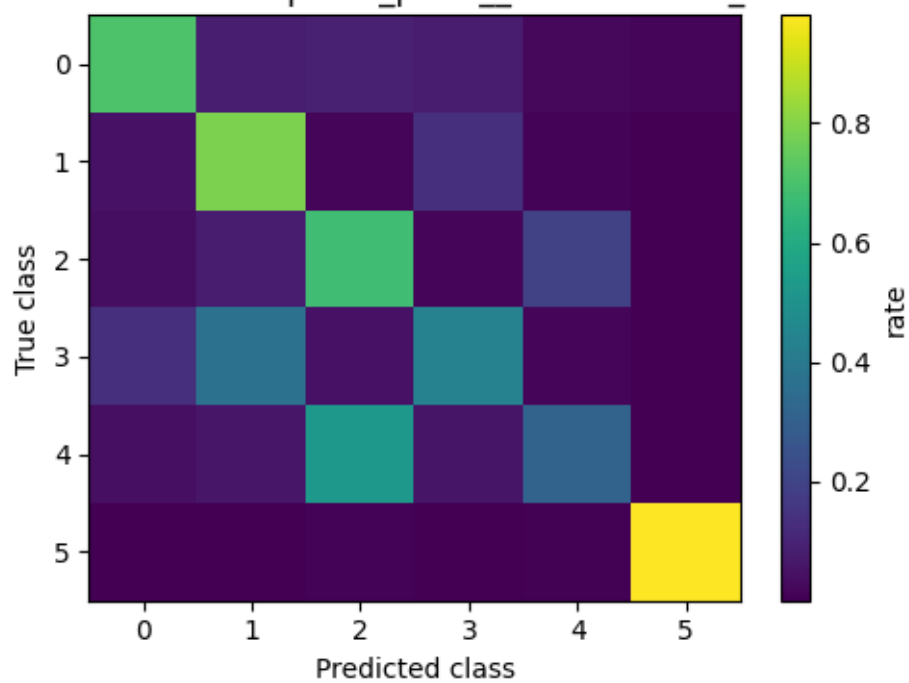
Class confusion — probe_proto_transformed2_oracle



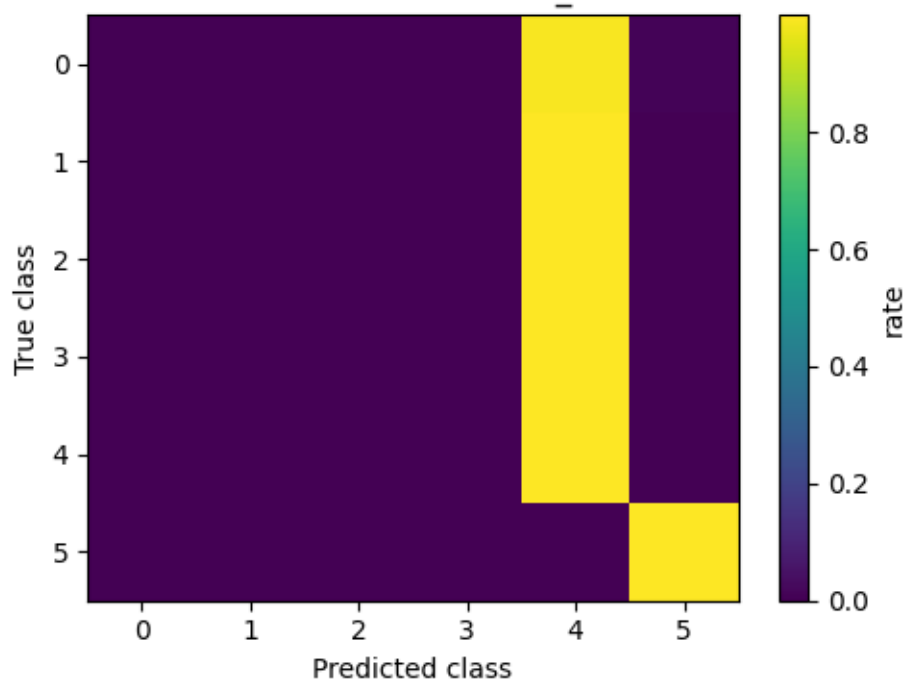
Class confusion — probe_proto_transformed3_oracle

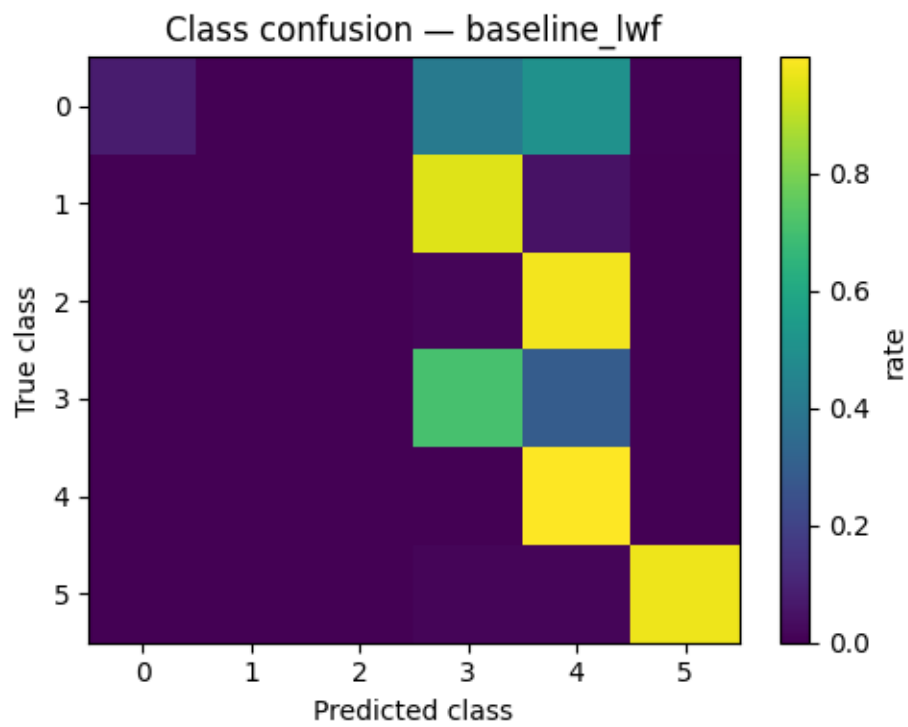
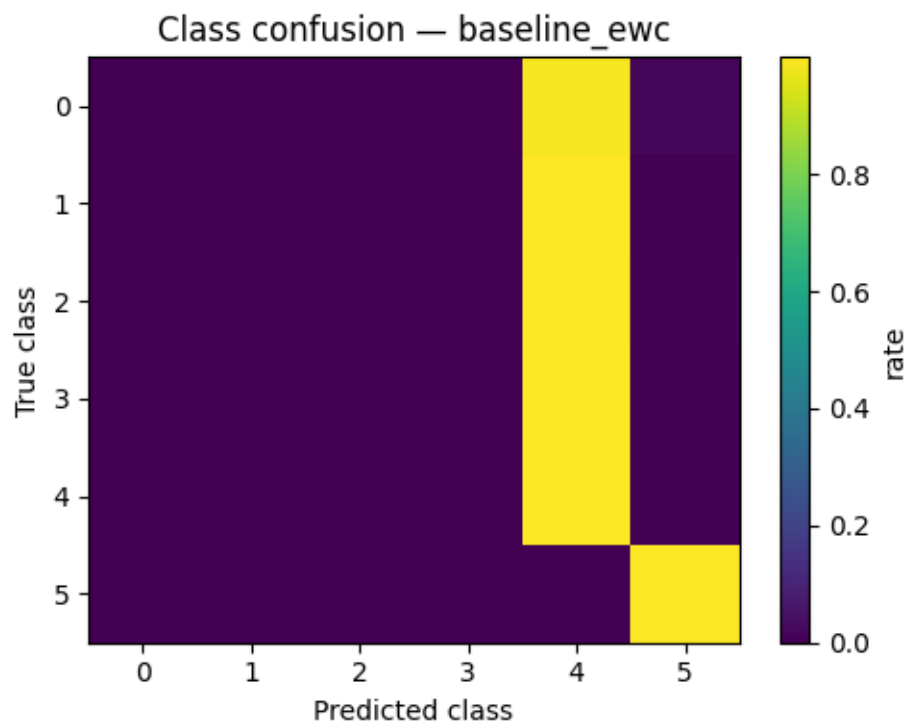


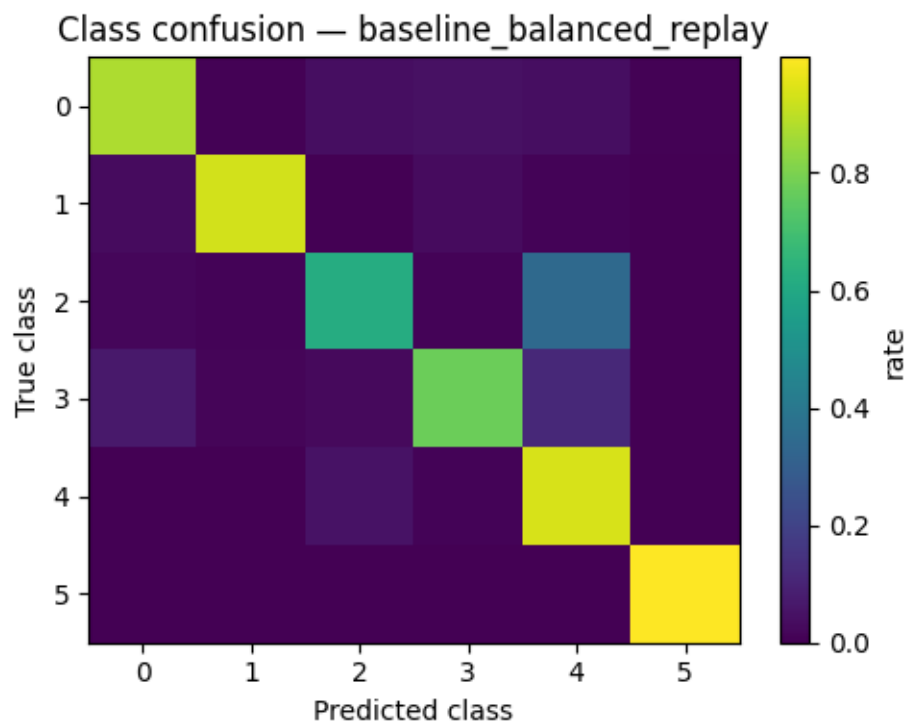
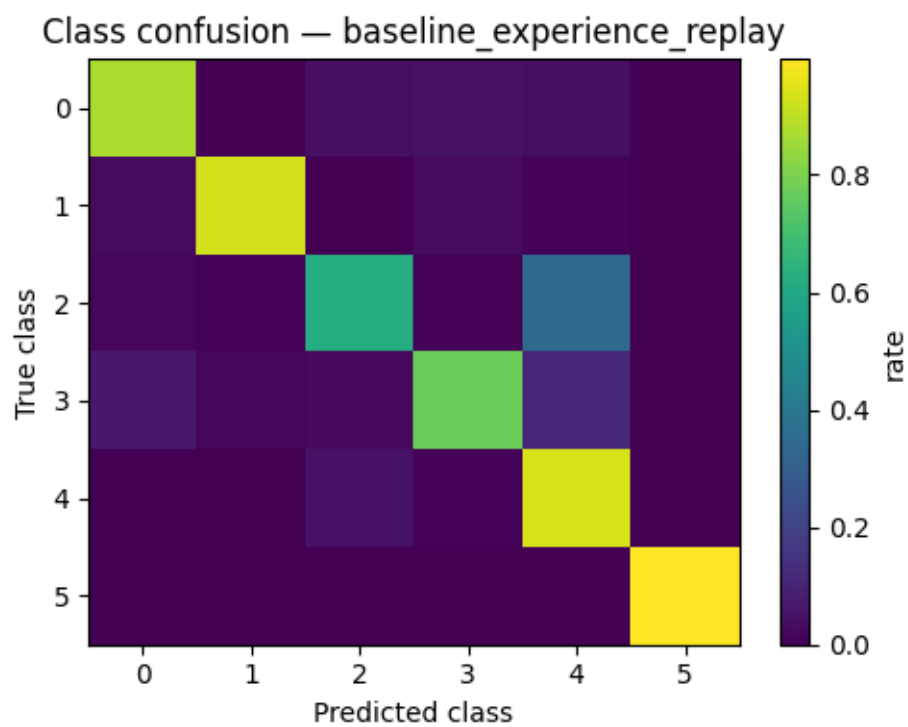
Class confusion — probe_proto_transformed4_oracle

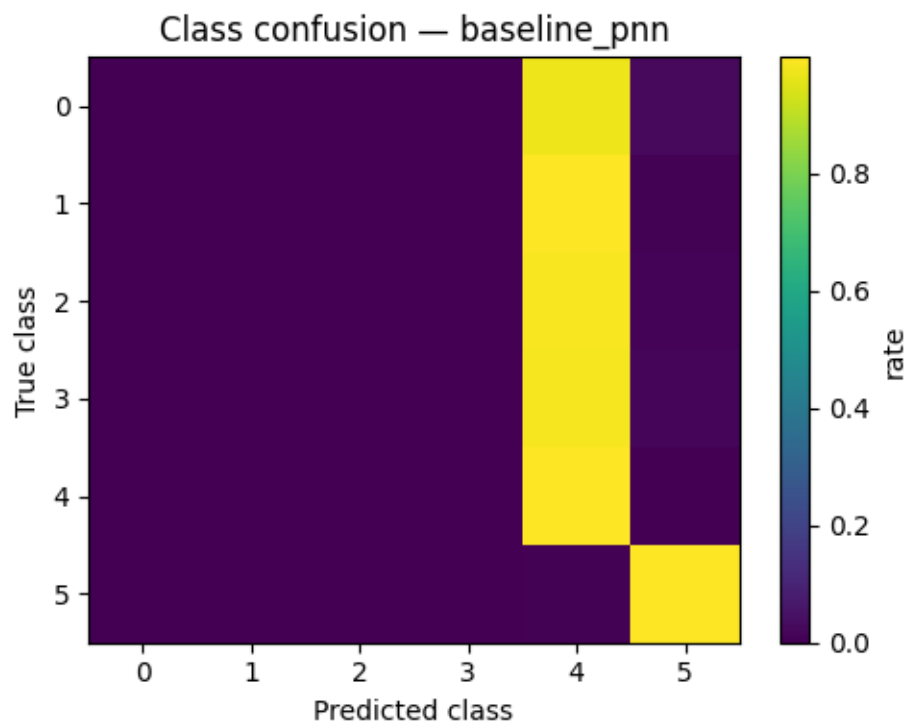
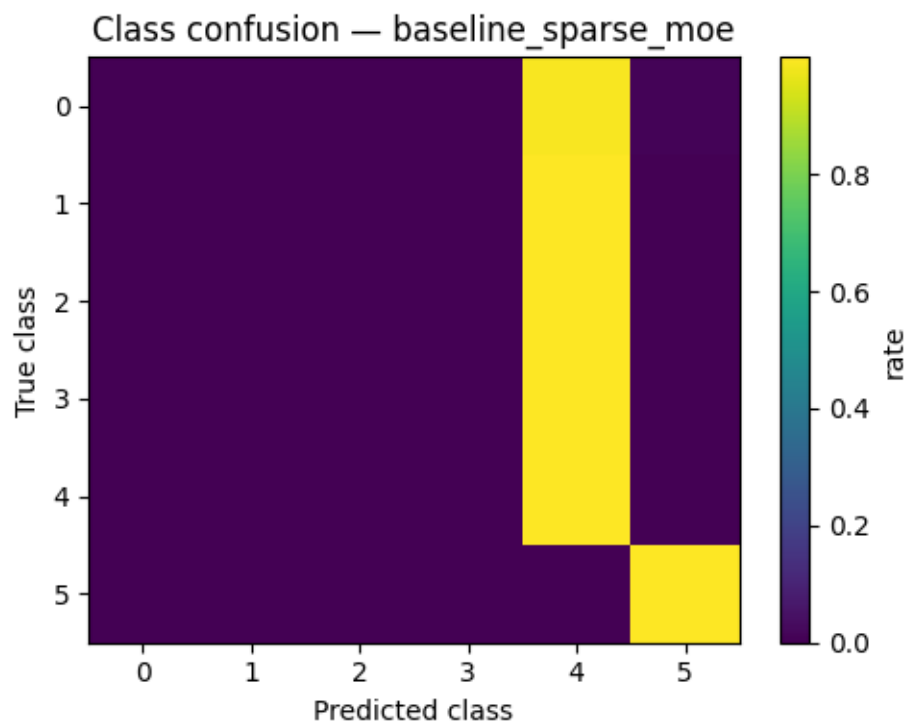


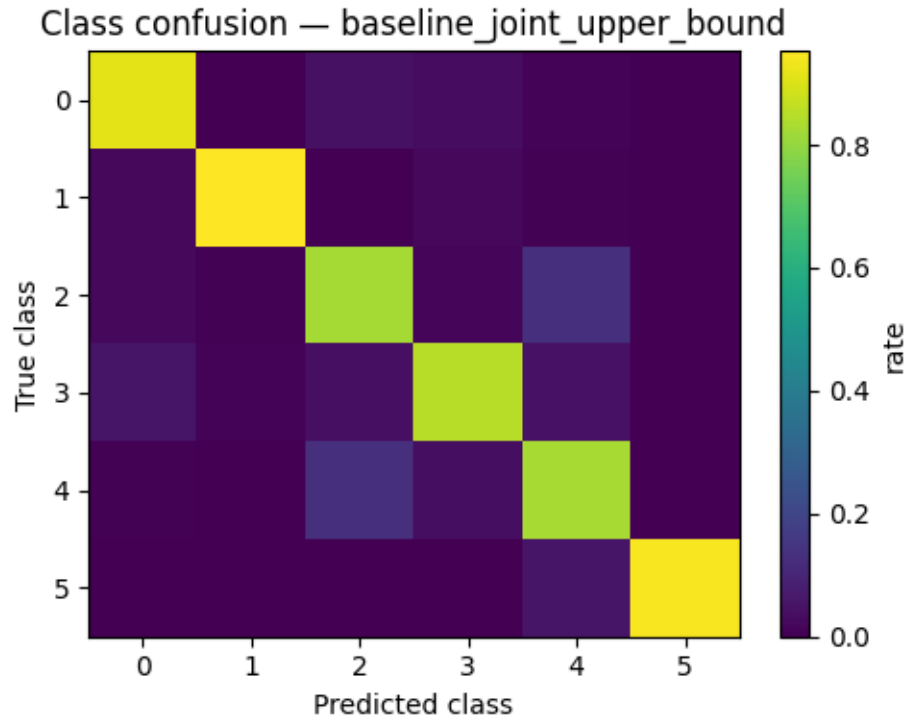
Class confusion — baseline_finetune











2.35 15. Automatic interpretation and RC1b validation gate

This interpretation is deliberately conservative. It separates deployable inferred-context variants from oracle references, compares the RC1b candidate against the true global guard, computes probe gaps, writes a pass/fail validation-gate table, and keeps the original RC1 row only as a rejected diagnostic.

```
[ ]: def get_row(scorecard_df, method):
    sub = scorecard_df[scorecard_df.method == method]
    return sub.iloc[0] if not sub.empty else None

def metric(row, name):
    if row is None or name not in row:
        return np.nan
    return float(row[name])

def method_role(method: str, probe_row: bool = False):
    """Classify methods so diagnostics are not mistaken for deployable models.
    ↪ """
    m = str(method)
    if bool(probe_row):
        return "diagnostic_probe"
    if "joint_upper_bound" in m:
```

```

        return "diagnostic_upper_bound"
    if "oracle" in m:
        return "diagnostic_oracle_reference"
    if m.startswith("paired_"):
        return "deployable_mmals_paired"
    if m.startswith("baseline_"):
        return "deployable_continual_baseline"
    return "other"

def non_diagnostic_scorecard(scorecard_df):
    if scorecard_df.empty:
        return scorecard_df
    if "method_role" not in scorecard_df.columns:
        tmp = scorecard_df.copy()
        tmp["method_role"] = [method_role(r.method, r.probe_row) for r in tmp.
↪itertuples(index=False)]
    else:
        tmp = scorecard_df
        return tmp[tmp["method_role"].isin(["deployable_mmals_paired",
↪"deployable_continual_baseline"])]

def paired_context_delta_table(scorecard_df):
    base = get_row(scorecard_df, "paired_proto_global_no_repair")
    global_guard = get_row(scorecard_df,
↪"paired_proto_global_head_ce_kl_guard_035")
    rows = []
    for variant in SELECTED_CONTEXT_HEAD_VARIANTS:
        method = method_name_for_context_variant(variant)
        row = get_row(scorecard_df, method)
        if row is None or base is None:
            continue
        rows.append({
            "method": method,
            "is_oracle_reference": "oracle" in method,
            "delta_acc_vs_proto_no_repair": metric(row, "final_avg_acc_mean") -
↪metric(base, "final_avg_acc_mean"),
            "delta_acc_vs_proto_global_guard": metric(row,
↪"final_avg_acc_mean") - metric(global_guard, "final_avg_acc_mean") if
↪global_guard is not None else np.nan,
            "delta_min_task_vs_no_repair": metric(row, "min_task_acc_mean") -
↪metric(base, "min_task_acc_mean"),
            "delta_forgetting_vs_no_repair": metric(base,
↪"avg_forgetting_mean") - metric(row, "avg_forgetting_mean"),
            "delta_class2_vs_no_repair": metric(row, "class2_acc_mean") -
↪metric(base, "class2_acc_mean"),

```

```

        "delta_class5_vs_no_repair": metric(row, "class5_acc_mean") -
        ↪metric(base, "class5_acc_mean"),
        "context_temperature": metric(row, "context_entropy_mean"),
        "readout_params": metric(row, "readout_total_params_mean"),
    })
    return pd.DataFrame(rows).sort_values("delta_acc_vs_proto_no_repair",
    ↪ascending=False)

delta_df = paired_context_delta_table(scorecard_df)
delta_path = RESULTS_DIR / f"paired_context_deltas_{RUN_MODE}.csv"
delta_df.to_csv(delta_path, index=False)
print("Saved paired context deltas:", delta_path)
display(delta_df)

base = get_row(scorecard_df, "paired_proto_global_no_repair")
noop = get_row(scorecard_df, "paired_proto_kl_only_true_noop")
global_guard = get_row(scorecard_df, "paired_proto_global_head_ce_kl_guard_035")
oracle_ref = get_row(scorecard_df, "paired_oracle_global_no_repair_reference")
oracle_guard = get_row(scorecard_df,
    ↪"paired_oracle_global_head_ce_kl_guard_035_reference")

non_probe = scorecard_df[scorecard_df.probe_row == False] if not scorecard_df.
    ↪empty else pd.DataFrame()
deployable = non_diagnostic_scorecard(scorecard_df)
deployable_mmals = deployable[deployable.method.str.startswith("paired_",
    ↪na=False)] if not deployable.empty else pd.DataFrame()
deployable_baselines = deployable[deployable.method.str.startswith("baseline_",
    ↪na=False)] if not deployable.empty else pd.DataFrame()
context_selected = deployable_mmals[deployable_mmals.method.str.
    ↪contains("context_", case=False, na=False)] if not deployable_mmals.empty
    ↪else pd.DataFrame()

best_deployable = deployable.sort_values("final_avg_acc_mean", ascending=False).
    ↪iloc[0] if not deployable.empty else None
best_deployable_mmals = deployable_mmals.sort_values("final_avg_acc_mean",
    ↪ascending=False).iloc[0] if not deployable_mmals.empty else None
best_continual_baseline = deployable_baselines.
    ↪sort_values("final_avg_acc_mean", ascending=False).iloc[0] if not
    ↪deployable_baselines.empty else None
best_context = context_selected.sort_values("final_avg_acc_mean",
    ↪ascending=False).iloc[0] if not context_selected.empty else None

print("\n=== MMALS v1.0-RC1b automatic interpretation ===")
if base is not None:
    print(f"paired_proto_global_no_repair final_avg_acc = {metric(base,
    ↪'final_avg_acc_mean'):.4f}")

```

```

if noop is not None and base is not None:
    noop_delta = metric(noop, "final_avg_acc_mean") - metric(base,
↪ "final_avg_acc_mean")
    print(f"proto_kl_only_true_noop delta vs proto_no_repair = {noop_delta:+.
↪ 6f}")
    if abs(noop_delta) < 1e-6:
        print("No-op control is clean: proto_kl_only_true_noop matches
↪ proto_global_no_repair.")
    else:
        print("Warning: no-op differs from no_repair. Check deterministic
↪ evaluation or audit implementation.")
if global_guard is not None:
    print(f"TRUE proto global CE+KL guard: {global_guard.method}
↪ acc={global_guard.final_avg_acc_mean:.4f}")
    if base is not None:
        true_guard_delta = global_guard.final_avg_acc_mean - metric(base,
↪ "final_avg_acc_mean")
        print(f"TRUE guard delta vs no-repair: {true_guard_delta:+.4f}")
        if abs(true_guard_delta) < 1e-6:
            print("Warning: TRUE guard still equals no-repair. Inspect the
↪ guarded-head training logs.")
        else:
            print("Control-fix signal: TRUE guard differs from no-repair.")
if best_context is not None:
    print(f"Best deployable inferred-context variant: {best_context.method}
↪ acc={best_context.final_avg_acc_mean:.4f}")
    if base is not None:
        print(f"Delta vs proto no-repair: {best_context.final_avg_acc_mean -
↪ metric(base, 'final_avg_acc_mean'):+.4f}")
    if global_guard is not None:
        print(f"Delta vs TRUE proto global guarded head: {best_context.
↪ final_avg_acc_mean - global_guard.final_avg_acc_mean:+.4f}")
        print(f"Class-2 delta vs no-repair: {best_context.class2_acc_mean -
↪ metric(base, 'class2_acc_mean'):+.4f}")
        print(f"Min-task delta vs no-repair: {best_context.min_task_acc_mean -
↪ metric(base, 'min_task_acc_mean'):+.4f}")
        print(f"Forgetting reduction vs no-repair: {metric(base,
↪ 'avg_forgetting_mean') - best_context.avg_forgetting_mean:+.4f}")
if best_deployable is not None:
    print(f"Best deployable method overall, excluding probes/oracle/joint upper
↪ bound: {best_deployable.method} acc={best_deployable.final_avg_acc_mean:.
↪ 4f}")
if best_deployable_mmals is not None:
    print(f"Best deployable MMALS paired row: {best_deployable_mmals.method}
↪ acc={best_deployable_mmals.final_avg_acc_mean:.4f}")
if best_continual_baseline is not None:

```



```

    print(f"Best deployable continual-learning baseline:␣
↪{best_continual_baseline.method} acc={best_continual_baseline.
↪final_avg_acc_mean:.4f}")
if oracle_ref is not None:
    print(f"Oracle context reference: {oracle_ref.method} acc={oracle_ref.
↪final_avg_acc_mean:.4f}")
if oracle_guard is not None:
    print(f"Oracle context + guarded head reference: {oracle_guard.method}␣
↪acc={oracle_guard.final_avg_acc_mean:.4f}")

probe_rows = scorecard_df[(scorecard_df.probe_row == True) & (scorecard_df.
↪probe_feature == "zf")] if not scorecard_df.empty else pd.DataFrame()
if not probe_rows.empty and best_deployable is not None:
    proto_probe = probe_rows[probe_rows.probe_context_mode == "proto"].
↪sort_values("final_avg_acc_mean", ascending=False).iloc[0]
    oracle_probe = probe_rows[probe_rows.probe_context_mode == "oracle"].
↪sort_values("final_avg_acc_mean", ascending=False).iloc[0]
    head_gap = proto_probe.final_avg_acc_mean - best_deployable.
↪final_avg_acc_mean
    context_gap = oracle_probe.final_avg_acc_mean - proto_probe.
↪final_avg_acc_mean
    total_gap = oracle_probe.final_avg_acc_mean - best_deployable.
↪final_avg_acc_mean
    print(f"Best proto-context z_f probe: {proto_probe.method} acc={proto_probe.
↪final_avg_acc_mean:.4f}")
    print(f"Best oracle-context z_f probe: {oracle_probe.method}␣
↪acc={oracle_probe.final_avg_acc_mean:.4f}")
    print(f"Remaining deployed-to-proto-probe head gap: {head_gap:+.4f}")
    print(f"Remaining proto-to-oracle context gap: {context_gap:+.4f}")
    print(f"Total deployed-to-oracle-probe gap: {total_gap:+.4f}")

# Explicit context-effect isolation: context-only vs context+guard.
context_only = get_row(scorecard_df,␣
↪"paired_context_temp_blend_selected_global")
context_guard = get_row(scorecard_df,␣
↪"paired_context_temp_blend_selected_head_guard_035")
if context_only is not None and context_guard is not None:
    print("\nContext effect isolation:")
    print(f"Context-only temp+blend: {context_only.final_avg_acc_mean:.4f}")
    print(f"Context+guard temp+blend: {context_guard.final_avg_acc_mean:.4f}")
    print(f"Added value of guarded readout after context selection:␣
↪{context_guard.final_avg_acc_mean - context_only.final_avg_acc_mean:+.4f}")
    if global_guard is not None:
        print(f"Context+guard vs TRUE global guard: {context_guard.
↪final_avg_acc_mean - global_guard.final_avg_acc_mean:+.4f}")

```

```

if RUN_MODE == "smoke":
    print("\nSmoke-mode note:")
    print("- Smoke is an execution and diagnostics check only.")
    print("- Do not freeze or reject RC1b from smoke-mode benchmark numbers.")
    print("- Evidence/robust/benchmark modes are required before scientific_
↳ranking versus baselines.")

print("\nDecision rule:")
print("- If deployable context-selected variants beat the TRUE proto global_
↳guard and reduce total gap, v1.0-RC1b supports freezing temp/blend_
↳context-gap selected guarded MMALS.")
print("- If they only match the TRUE global guard, context selection is_
↳diagnostic but not yet a sufficient architecture change.")
print("- If oracle reference remains much higher, the next branch should target_
↳learned context posterior / context memory, not more readout-only variants.")
print("- If proto and oracle probes converge, remaining work returns to readout/_
↳probe-gap closure.")
print("- Do not claim oracle rows as deployable task-free MMALS.")

# Compact benchmark separation table for interpretation.
if "method_role" in scorecard_df.columns:
    role_summary = scorecard_df.groupby("method_role", as_index=False).agg(
        best_acc=("final_avg_acc_mean", "max"),
        n_methods=("method", "count"),
    ).sort_values("best_acc", ascending=False)
    role_summary_path = RESULTS_DIR / f"benchmark_role_summary_{RUN_MODE}.csv"
    role_summary.to_csv(role_summary_path, index=False)
    print("\nSaved benchmark role summary:", role_summary_path)
    display(role_summary)

# -----
# v1.0-RC1b robust validation gate
# -----
PRIMARY_RC1 = "paired_context_gap_selected"
SECONDARY_RC1 = "paired_context_temp_blend_selected_head_guard_035"
REJECTED_ORIGINAL_RC1 = "paired_context_blend_selected_head_guard_035"
TRUE_GUARD = "paired_proto_global_head_ce_kl_guard_035"
BASELINE = "paired_proto_global_no_repair"
NOOP = "paired_proto_kl_only_true_noop"

def row_or_nan(method):
    return get_row(scorecard_df, method)

rc1 = row_or_nan(PRIMARY_RC1)
rc1_secondary = row_or_nan(SECONDARY_RC1)

```

```

true_guard = row_or_nan(TRUE_GUARD)
base = row_or_nan(BASELINE)
noop = row_or_nan(NOOP)

# Per-seed win-rate against the corrected true global guard.
per_seed_wins = np.nan
per_seed_ties_or_wins = np.nan
if rc1 is not None and true_guard is not None and not per_seed_df.empty:
    p = per_seed_df[per_seed_df.method == PRIMARY_RC1][["seed",
↪ "final_avg_acc", "min_task_acc", "avg_forgetting", "class2_acc",
↪ "class5_acc"]]
    g = per_seed_df[per_seed_df.method == TRUE_GUARD][["seed", "final_avg_acc",
↪ "min_task_acc", "avg_forgetting", "class2_acc", "class5_acc"]]
    pg = p.merge(g, on="seed", suffixes=("_rc1", "_guard"))
    if not pg.empty:
        pg["acc_delta_rc1_vs_guard"] = pg["final_avg_acc_rc1"] -
↪ pg["final_avg_acc_guard"]
        pg["min_task_delta_rc1_vs_guard"] = pg["min_task_acc_rc1"] -
↪ pg["min_task_acc_guard"]
        pg["forgetting_reduction_rc1_vs_guard"] = pg["avg_forgetting_guard"] -
↪ pg["avg_forgetting_rc1"]
        pg["class2_delta_rc1_vs_guard"] = pg["class2_acc_rc1"] -
↪ pg["class2_acc_guard"]
        per_seed_wins = float((pg["acc_delta_rc1_vs_guard"] > 0.0).mean())
        per_seed_ties_or_wins = float((pg["acc_delta_rc1_vs_guard"] >= -0.0025).
↪ mean())
        per_seed_cmp_path = RESULTS_DIR /
↪ f"rc1b_per_seed_vs_true_guard_{RUN_MODE}.csv"
        pg.to_csv(per_seed_cmp_path, index=False)
        print("Saved RC1b per-seed comparison:", per_seed_cmp_path)
        display(pg)

# Probe gaps.
probe_rows = scorecard_df[(scorecard_df.probe_row == True) & (scorecard_df.
↪ probe_feature == "zf")] if not scorecard_df.empty else pd.DataFrame()
proto_probe = None
oracle_probe = None
if not probe_rows.empty:
    proto_probe = probe_rows[probe_rows.probe_context_mode == "proto"].
↪ sort_values("final_avg_acc_mean", ascending=False).iloc[0]
    oracle_probe = probe_rows[probe_rows.probe_context_mode == "oracle"].
↪ sort_values("final_avg_acc_mean", ascending=False).iloc[0]

checks = []

def add_check(name, value, threshold_text, passed, details=""):

```

```

checks.append({
    "check": name,
    "value": value,
    "threshold": threshold_text,
    "passed": bool(passed),
    "details": details,
})

if base is not None and noop is not None:
    noop_delta = metric(noop, "final_avg_acc_mean") - metric(base,
↪ "final_avg_acc_mean")
    add_check("no_op_equals_no_repair", noop_delta, "abs(delta) <= 1e-6",
↪ abs(noop_delta) <= 1e-6,
        "No-op must be an evaluation no-op, not a hidden training variant.
↪ ")

if base is not None and true_guard is not None:
    guard_delta = metric(true_guard, "final_avg_acc_mean") - metric(base,
↪ "final_avg_acc_mean")
    add_check("true_guard_active", guard_delta, "delta != 0 and preferably > +0.
↪ 005", abs(guard_delta) > 1e-6 and guard_delta > 0.005,
        "Corrected global CE+KL guard should not behave like no-repair.")

if rc1 is not None and base is not None:
    rc1_delta_base = metric(rc1, "final_avg_acc_mean") - metric(base,
↪ "final_avg_acc_mean")
    add_check("rc1_beats_no_repair_accuracy", rc1_delta_base, ">= +0.020",
↪ rc1_delta_base >= 0.020,
        "Primary RC1b candidate must materially beat no-repair.")
    rc1_min_delta_base = metric(rc1, "min_task_acc_mean") - metric(base,
↪ "min_task_acc_mean")
    add_check("rc1_improves_min_task_vs_no_repair", rc1_min_delta_base, ">= +0.
↪ 050", rc1_min_delta_base >= 0.050,
        "RC1b candidate must improve weakest-task robustness.")
    rc1_forget_red_base = metric(base, "avg_forgetting_mean") - metric(rc1,
↪ "avg_forgetting_mean")
    add_check("rc1_reduces_forgetting_vs_no_repair", rc1_forget_red_base, ">=
↪ +0.020", rc1_forget_red_base >= 0.020,
        "RC1b candidate should reduce forgetting, not only improve the
↪ last task.")

if rc1 is not None and true_guard is not None:
    rc1_delta_guard = metric(rc1, "final_avg_acc_mean") - metric(true_guard,
↪ "final_avg_acc_mean")
    rc1_min_delta_guard = metric(rc1, "min_task_acc_mean") - metric(true_guard,
↪ "min_task_acc_mean")

```

```

    rc1_forget_red_guard = metric(true_guard, "avg_forgetting_mean") -
    ↪metric(rc1, "avg_forgetting_mean")
    rc1_class2_delta_guard = metric(rc1, "class2_acc_mean") -
    ↪metric(true_guard, "class2_acc_mean")
    rc1_class5_delta_guard = metric(rc1, "class5_acc_mean") -
    ↪metric(true_guard, "class5_acc_mean")
    robustness_preferable = (rc1_min_delta_guard >= 0.010 and
    ↪rc1_forget_red_guard >= 0.005 and rc1_class2_delta_guard >= -0.005)
    add_check("rc1_vs_true_guard_accuracy_or_robustness", rc1_delta_guard,
              "accuracy >= +0.005 OR robustness-preferable", rc1_delta_guard >=
    ↪0.005 or robustness_preferable,
              f"min_task_delta={rc1_min_delta_guard:+.4f},
    ↪forgetting_reduction={rc1_forget_red_guard:+.4f},
    ↪class2_delta={rc1_class2_delta_guard:+.4f}")
    add_check("rc1_preserves_class5_vs_true_guard", rc1_class5_delta_guard, ">=
    ↪-0.010", rc1_class5_delta_guard >= -0.010,
              "Class 5 preservation must not be sacrificed.")

if not np.isnan(per_seed_ties_or_wins):
    add_check("rc1_seed_stability_vs_true_guard", per_seed_ties_or_wins, ">= 0.
    ↪60 ties-or-wins", per_seed_ties_or_wins >= 0.60,
              "RC1b candidate should not be a one-seed artifact.")

if proto_probe is not None and oracle_probe is not None and rc1 is not None:
    head_gap = proto_probe.final_avg_acc_mean - metric(rc1,
    ↪"final_avg_acc_mean")
    context_gap = oracle_probe.final_avg_acc_mean - proto_probe.
    ↪final_avg_acc_mean
    add_check("probe_gap_interpretable", head_gap, "head_gap <= 0.025
    ↪preferred", head_gap <= 0.025,
              f"proto_to_oracle_context_gap={context_gap:+.4f}; larger context
    ↪gap suggests context frontier.")

rejected_original = row_or_nan(REJECTED_ORIGINAL_RC1)
secondary = row_or_nan(SECONDARY_RC1)

if rc1 is not None and rejected_original is not None:
    rc1b_delta_original = metric(rc1, "final_avg_acc_mean") -
    ↪metric(rejected_original, "final_avg_acc_mean")
    add_check("rc1b_beats_rejected_original_rc1", rc1b_delta_original, ">= +0.
    ↪005 preferred", rc1b_delta_original >= 0.005,
              "RC1b should improve over the original blend-only guarded row
    ↪that failed robust freeze.")

validation_gate_df = pd.DataFrame(checks)

```

```

validation_gate_path = RESULTS_DIR / f"rc1b_validation_gate_{RUN_MODE}.csv"
validation_gate_df.to_csv(validation_gate_path, index=False)
print("\n=== v1.0-RC1b validation gate ===")
print("Saved validation gate:", validation_gate_path)
display(validation_gate_df)

if not validation_gate_df.empty:
    passed_count = int(validation_gate_df["passed"].sum())
    total_count = int(len(validation_gate_df))
    print(f"Validation checks passed: {passed_count}/{total_count}")
    if RUN_MODE == "smoke":
        print("Decision: smoke passed only if the harness executed and
↳diagnostics are coherent. Do not freeze/reject RC1b from smoke.")
        elif passed_count == total_count:
            print("Decision: RC1b passes this validation mode. Next step: benchmark
↳campaign, not a new architecture branch.")
            elif passed_count >= max(1, total_count - 1):
                print("Decision: RC1b is nearly validated. Inspect the failed check
↳before freezing.")
            else:
                print("Decision: do not freeze RC1b yet. Diagnose failed checks before
↳benchmark campaign.")

```

	method	is_oracle_reference \
4	paired_context_temp_blend_selected_head_guard_035	False
5	paired_context_gap_selected	False
6	paired_generic_weak_boundary_latent_calibrated	False
1	paired_proto_global_head_ce_kl_guard_035	False
3	paired_context_temp_blend_selected_global	False
2	paired_context_blend_selected_global	False
0	paired_proto_global_no_repair	False

	delta_acc_vs_proto_no_repair	delta_acc_vs_proto_global_guard \
4	0.05915	0.02610
5	0.05915	0.02610
6	0.03850	0.00545
1	0.03305	0.00000
3	0.02935	-0.00370
2	0.02180	-0.01125
0	0.00000	-0.03305

	delta_min_task_vs_no_repair	delta_forgetting_vs_no_repair \
4	0.16350	0.045250
5	0.16350	0.045250
6	0.10200	0.027562
1	0.07225	0.015062
3	0.08675	0.024625

2	0.03850	0.030687
0	0.00000	0.000000

	delta_class2_vs_no_repair	delta_class5_vs_no_repair	context_temperature \
4	0.14100	0.0045	NaN
5	0.14100	0.0045	NaN
6	0.10875	0.0035	NaN
1	0.03650	-0.0015	NaN
3	0.12225	0.0040	NaN
2	0.13900	0.0040	NaN
0	0.00000	0.0000	NaN

	readout_params
4	390.0
5	390.0
6	4487.0
1	390.0
3	390.0
2	390.0
0	390.0

	method_role	best_acc	n_methods
2	diagnostic_probe	0.93925	7
1	deployable_mmals_paired	0.90100	8
3	diagnostic_upper_bound	0.87660	1
0	deployable_continual_baseline	0.83900	7

	seed	final_avg_acc_rc1	min_task_acc_rc1	avg_forgetting_rc1 \
0	0	0.88525	0.84125	0.076875
1	1	0.87675	0.74625	0.077187
2	2	0.93900	0.91375	0.029688
3	3	0.90875	0.79375	0.068125
4	4	0.89525	0.79625	0.026250

	class2_acc_rc1	class5_acc_rc1	final_avg_acc_guard	min_task_acc_guard \
0	0.79625	0.9975	0.88375	0.84125
1	0.68250	0.9975	0.88350	0.74500
2	0.92375	1.0000	0.89700	0.76375
3	0.79875	0.9975	0.91175	0.79125
4	0.91625	1.0000	0.79850	0.49375

	avg_forgetting_guard	class2_acc_guard	class5_acc_guard \
0	0.076875	0.79375	0.9975
1	0.078438	0.67250	0.9950
2	0.077500	0.75875	0.9975
3	0.060625	0.80250	0.9975
4	0.135625	0.56750	0.9750

	acc_delta_rc1_vs_guard	min_task_delta_rc1_vs_guard \
--	------------------------	-------------------------------

0	0.00150	0.00000
1	-0.00675	0.00125
2	0.04200	0.15000
3	-0.00300	0.00250
4	0.09675	0.30250

	forgetting_reduction_rc1_vs_guard	class2_delta_rc1_vs_guard
0	0.000000	0.00250
1	0.001250	0.01000
2	0.047813	0.16500
3	-0.007500	-0.00375
4	0.109375	0.34875

	check	value	\
0	true_guard_active	0.03305	
1	rc1_beats_no_repair_accuracy	0.05915	
2	rc1_improves_min_task_vs_no_repair	0.16350	
3	rc1_reduces_forgetting_vs_no_repair	0.04525	
4	rc1_vs_true_guard_accuracy_or_robustness	0.02610	
5	rc1_preserves_class5_vs_true_guard	0.00600	
6	rc1_seed_stability_vs_true_guard	0.60000	
7	probe_gap_interpretable	0.00525	

	threshold	passed	\
0	delta != 0 and preferably > +0.005	True	
1	>= +0.020	True	
2	>= +0.050	True	
3	>= +0.020	True	
4	accuracy >= +0.005 OR robustness-preferable	True	
5	>= -0.010	True	
6	>= 0.60 ties-or-wins	True	
7	head_gap <= 0.025 preferred	True	

	details
0	Corrected global CE+KL guard should not behave...
1	Primary RC1b candidate must materially beat no...
2	RC1b candidate must improve weakest-task robus...
3	RC1b candidate should reduce forgetting, not o...
4	min_task_delta=+0.0912, forgetting_reduction=+...
5	Class 5 preservation must not be sacrificed.
6	RC1b candidate should not be a one-seed artifact.
7	proto_to_oracle_context_gap=+0.0330; larger co...

2.36 15.2 Benchmark leaderboard and comparative gate

This cell compares the frozen RC1b candidate against the strongest non-oracle baseline rows. The decision rule is stricter than the internal paired audit:

- RC1b should beat fine-tuning, EWC, LwF and sparse MoE on average accuracy and robust-

ness.

- Replay/PNN may remain strong controls; RC1b is benchmark-interesting if it is competitive while preserving the MMALS interpretability/probe-gap story.
- Joint upper bound is diagnostic, not a deployable continual-learning baseline.

```
[ ]: # Benchmark-level leaderboard: exclude probes and oracle references.
leaderboard_df = scorecard_df.copy()
if not leaderboard_df.empty:
    leaderboard_df["is_probe"] = leaderboard_df["probe_row"].astype(bool)
    leaderboard_df["is_oracle_reference"] = leaderboard_df["method"].str.
    ↪contains("oracle", case=False, na=False)
    benchmark_leaderboard = leaderboard_df[(~leaderboard_df["is_probe"]) &
    ↪(~leaderboard_df["is_oracle_reference"])]].copy()
    benchmark_leaderboard = benchmark_leaderboard.
    ↪sort_values("final_avg_acc_mean", ascending=False)
    bench_path = RESULTS_DIR / f"benchmark_leaderboard_{RUN_MODE}.csv"
    benchmark_leaderboard.to_csv(bench_path, index=False)
    print("Saved benchmark leaderboard:", bench_path)
    display_cols = [
        "method", "final_avg_acc_mean", "final_avg_acc_std",
    ↪"min_task_acc_mean", "avg_forgetting_mean",
        "class2_acc_mean", "class5_acc_mean", "pred4_bias_mean",
    ↪"total_params_mean", "replay_memory_items_mean"
    ]
    display(benchmark_leaderboard[[c for c in display_cols if c in
    ↪benchmark_leaderboard.columns]])

    rc1b = get_row(scorecard_df, "paired_context_gap_selected")
    rc1b_equiv = get_row(scorecard_df,
    ↪"paired_context_temp_blend_selected_head_guard_035")
    true_guard = get_row(scorecard_df,
    ↪"paired_proto_global_head_ce_kl_guard_035")
    joint = get_row(scorecard_df, "baseline_joint_upper_bound")
    deployable_competitors = benchmark_leaderboard[~benchmark_leaderboard.
    ↪method.isin([
        "paired_context_gap_selected",
        "paired_context_temp_blend_selected_head_guard_035",
        "baseline_joint_upper_bound",
    ])]
    best_competitor = deployable_competitors.iloc[0] if not
    ↪deployable_competitors.empty else None

    print("\n=== Benchmark comparative interpretation ===")
    if rc1b is not None:
        print(f"RC1b primary candidate: {rc1b.method} acc={rc1b.
    ↪final_avg_acc_mean:.4f}, min_task={rc1b.min_task_acc_mean:.4f},
    ↪forgetting={rc1b.avg_forgetting_mean:.4f}")
```

```

    if rc1b_equiv is not None:
        print(f"RC1b equivalent diagnostic: {rc1b_equiv.method} acc={rc1b_equiv.
↪final_avg_acc_mean:.4f}")
    if true_guard is not None and rc1b is not None:
        print(f"Delta RC1b vs true global guard: {rc1b.final_avg_acc_mean -
↪true_guard.final_avg_acc_mean:+.4f} acc, {rc1b.min_task_acc_mean -
↪true_guard.min_task_acc_mean:+.4f} min-task")
    if best_competitor is not None and rc1b is not None:
        print(f"Best non-oracle, non-joint competitor: {best_competitor.method}
↪acc={best_competitor.final_avg_acc_mean:.4f}")
        print(f"Delta RC1b vs best competitor: {rc1b.final_avg_acc_mean -
↪best_competitor.final_avg_acc_mean:+.4f}")
    if joint is not None and rc1b is not None:
        print(f"Joint upper bound: {joint.final_avg_acc_mean:.4f}; RC1b gap to
↪joint={joint.final_avg_acc_mean - rc1b.final_avg_acc_mean:+.4f}")

    benchmark_gate_rows = []
    if rc1b is not None:
        def add_gate(name, passed, detail):
            benchmark_gate_rows.append({"check": name, "passed": bool(passed),
↪"detail": detail})
        add_gate("rc1b_above_true_guard", true_guard is not None and rc1b.
↪final_avg_acc_mean >= true_guard.final_avg_acc_mean - 0.002,
            f"RC1b={rc1b.final_avg_acc_mean:.4f}; true_guard={true_guard.
↪final_avg_acc_mean if true_guard is not None else np.nan:.4f}")
        add_gate("rc1b_above_finetune", get_row(scorecard_df,
↪"baseline_finetune") is not None and rc1b.final_avg_acc_mean >
↪get_row(scorecard_df, "baseline_finetune").final_avg_acc_mean,
            "RC1b should beat naive fine-tuning.")
        add_gate("rc1b_min_task_strong", rc1b.min_task_acc_mean >= 0.80 if not
↪np.isnan(rc1b.min_task_acc_mean) else False,
            f"min_task={rc1b.min_task_acc_mean:.4f}")
        add_gate("rc1b_forgetting_controlled", rc1b.avg_forgetting_mean <= 0.08
↪if not np.isnan(rc1b.avg_forgetting_mean) else False,
            f"forgetting={rc1b.avg_forgetting_mean:.4f}")
        add_gate("rc1b_class2_not_collapsed", rc1b.class2_acc_mean >= 0.80 if
↪not np.isnan(rc1b.class2_acc_mean) else False,
            f"class2={rc1b.class2_acc_mean:.4f}")
        add_gate("rc1b_class5_preserved", rc1b.class5_acc_mean >= 0.95 if not
↪np.isnan(rc1b.class5_acc_mean) else False,
            f"class5={rc1b.class5_acc_mean:.4f}")
    if best_competitor is not None:
        add_gate("rc1b_competitive_with_best_baseline", rc1b.
↪final_avg_acc_mean >= best_competitor.final_avg_acc_mean - 0.010,
            f"RC1b={rc1b.final_avg_acc_mean:.4f};
↪best_competitor={best_competitor.final_avg_acc_mean:.4f}")

```

```

benchmark_gate_df = pd.DataFrame(benchmark_gate_rows)
gate_path = RESULTS_DIR / f"benchmark_gate_{RUN_MODE}.csv"
benchmark_gate_df.to_csv(gate_path, index=False)
print("Saved benchmark gate:", gate_path)
display(benchmark_gate_df)
else:
    print("Scorecard is empty; run the experiment cells first.")

```

	method	final_avg_acc_mean \
9	paired_context_gap_selected	0.90100
11	paired_context_temp_blend_selected_head_guard_035	0.90100
15	paired_rc2m_beta_dynamic_boundary_ambiguity_lo...	0.88560
12	paired_generic_weak_boundary_latent_calibrated	0.88035
4	baseline_joint_upper_bound	0.87660
13	paired_proto_global_head_ce_kl_guard_035	0.87490
10	paired_context_temp_blend_selected_global	0.87120
8	paired_context_blend_selected_global	0.86365
14	paired_proto_global_no_repair	0.84185
0	baseline_balanced_replay	0.83900
2	baseline_experience_replay	0.83900
5	baseline_lwf	0.44685
7	baseline_sparse_moe	0.29980
1	baseline_ewc	0.29975
3	baseline_finetune	0.29970
6	baseline_pnn	0.29920

	final_avg_acc_std	min_task_acc_mean	avg_forgetting_mean \
9	0.024358	0.81825	0.055625
11	0.024358	0.81825	0.055625
15	0.025278	0.77000	0.069312
12	0.046148	0.75675	0.073313
4	0.004585	0.81850	0.000000
13	0.044259	0.72700	0.085812
10	0.030284	0.74150	0.076250
8	0.035399	0.69325	0.070187
14	0.079184	0.65475	0.100875
0	0.011727	0.69850	0.135187
2	0.011727	0.69850	0.135187
5	0.023186	0.00125	0.607125
7	0.000209	0.00000	0.844688
1	0.000177	0.00000	0.842250
3	0.000209	0.00000	0.841187
6	0.000716	0.00000	0.838125

	class2_acc_mean	class5_acc_mean	pred4_bias_mean	total_params_mean
9	0.82350	0.9985	0.176375	1337606.0
11	0.82350	0.9985	0.176375	1337606.0

15	0.77350	0.9975	0.184333	1337606.0
12	0.79125	0.9975	0.192875	1341703.0
4	0.82300	0.9425	0.179000	217798.0
13	0.71900	0.9925	0.196917	1337606.0
10	0.80475	0.9980	0.185833	1337606.0
8	0.82150	0.9980	0.171125	1337606.0
14	0.68250	0.9940	0.206958	1337606.0
0	0.61875	0.9970	0.240417	217798.0
2	0.61875	0.9970	0.240417	217798.0
5	0.00100	0.9715	0.473083	217798.0
7	0.00000	0.9985	0.830750	1188555.0
1	0.00000	0.9980	0.830042	217798.0
3	0.00000	0.9975	0.830917	217798.0
6	0.00000	0.9955	0.824125	1088966.0

	check	passed \
0	rc1b_above_true_guard	True
1	rc1b_above_finetune	True
2	rc1b_min_task_strong	True
3	rc1b_forgetting_controlled	True
4	rc1b_class2_not_collapsed	True
5	rc1b_class5_preserved	True
6	rc1b_competitive_with_best_baseline	True

	detail
0	RC1b=0.9010; true_guard=0.8749
1	RC1b should beat naive fine-tuning.
2	min_task=0.8182
3	forgetting=0.0556
4	class2=0.8235
5	class5=0.9985
6	RC1b=0.9010; best_competitor=0.8856

2.37 16. Observability audit package and weak-seed diagnosis

This cell converts the benchmark results into auditable artifacts. It does not change scores. It produces seed-level, context-level, head-level, memory-level, and OTEL-style trace tables so that weak runs can be inspected instead of treated as black-box noise.

```
[ ]: # -----
# Observability post-processing: seed, context, head, memory, and trace audits.
# -----
def _first_available(df, cols):
    return [c for c in cols if c in df.columns]

def build_seed_diagnostics(per_seed_df, class_diag_df):
    if per_seed_df.empty:
        return pd.DataFrame()
```

```

    keep = _first_available(per_seed_df, [
        "method", "seed", "method_role", "model_family", "final_avg_acc",
        ↪ "min_task_acc",
        "avg_forgetting", "class2_acc", "class5_acc", "pred4_bias",
        ↪ "total_params", "replay_memory_items"
    ])
    out = per_seed_df[keep].copy()
    out["is_rc1b_primary"] = out["method"].eq("paired_context_gap_selected")
    out["is_true_guard"] = out["method"].
    ↪ eq("paired_proto_global_head_ce_kl_guard_035")
    return out.sort_values(["seed", "final_avg_acc"], ascending=[True, False])

def build_rc1b_vs_guard_audit(per_seed_df):
    if per_seed_df.empty:
        return pd.DataFrame()
    rc = per_seed_df[per_seed_df.method == "paired_context_gap_selected"].copy()
    gd = per_seed_df[per_seed_df.method ==
    ↪ "paired_proto_global_head_ce_kl_guard_035"].copy()
    if rc.empty or gd.empty:
        return pd.DataFrame()
    cols = ["seed", "final_avg_acc", "min_task_acc", "avg_forgetting",
    ↪ "class2_acc", "class5_acc", "pred4_bias"]
    rc = rc[[c for c in cols if c in rc.columns]].rename(columns={c:
    ↪ f"rc1b_{c}" for c in cols if c != "seed"})
    gd = gd[[c for c in cols if c in gd.columns]].rename(columns={c:
    ↪ f"guard_{c}" for c in cols if c != "seed"})
    out = rc.merge(gd, on="seed", how="outer")
    for m in ["final_avg_acc", "min_task_acc", "class2_acc", "class5_acc"]:
        if f"rc1b_{m}" in out and f"guard_{m}" in out:
            out[f"delta_{m}_rc1b_minus_guard"] = out[f"rc1b_{m}"] -
    ↪ out[f"guard_{m}"]
        if "rc1b_avg_forgetting" in out and "guard_avg_forgetting" in out:
            out["delta_forgetting_reduction_rc1b_vs_guard"] =
    ↪ out["guard_avg_forgetting"] - out["rc1b_avg_forgetting"]
        out["weak_seed_flag"] = False
        if "delta_final_avg_acc_rc1b_minus_guard" in out:
            out["weak_seed_flag"] = out["weak_seed_flag"] |
    ↪ (out["delta_final_avg_acc_rc1b_minus_guard"] < 0)
        if "rc1b_class2_acc" in out:
            out["weak_seed_flag"] = out["weak_seed_flag"] | (out["rc1b_class2_acc"]
    ↪ < 0.80)
        if "rc1b_min_task_acc" in out:
            out["weak_seed_flag"] = out["weak_seed_flag"] |
    ↪ (out["rc1b_min_task_acc"] < 0.80)
    return out.sort_values("seed")

```

```

def build_context_selection_audit(repair_log_df):
    if repair_log_df.empty:
        return pd.DataFrame()
    mask = repair_log_df.get("repair_type", pd.Series(dtype=str)).astype(str).
    ↪str.contains("context_config", na=False)
    cols = _first_available(repair_log_df, [
        "method", "seed", "after_task", "task_id", "variant", "repair_type",
    ↪"selection_mode",
        "candidate", "selected_candidate", "candidate_temp", "selected_temp",
    ↪"candidate_blend_name",
        "selected_blend_name", "candidate_blend_single",
    ↪"candidate_blend_latent", "candidate_blend_feature",
        "selected_blend_single", "selected_blend_latent",
    ↪"selected_blend_feature", "proxy_acc",
        "proxy_context_top1", "proxy_context_entropy", "proxy_context_margin",
    ↪"proxy_class2",
        "proxy_class5", "proxy_min_class", "proxy_score",
    ↪"selected_proxy_score", "selected_proxy_acc",
        "selected_proxy_context_top1", "selected_proxy_class2",
    ↪"selected_proxy_min_class"
    ])
    return repair_log_df.loc[mask, cols].copy()

def build_head_repair_audit(repair_log_df):
    if repair_log_df.empty:
        return pd.DataFrame()
    rt = repair_log_df.get("repair_type", pd.Series(dtype=str)).astype(str)
    mask = rt.str.contains("guard|head|readout|ce_kl", case=False, na=False) |
    ↪repair_log_df.get("uses_guarded_global_head", pd.Series(False,
    ↪index=repair_log_df.index)).fillna(False).astype(bool)
    cols = _first_available(repair_log_df, [
        "method", "seed", "after_task", "task_id", "variant", "repair_type",
    ↪"readout_train_loss",
        "readout_train_ce", "readout_train_kl", "readout_train_margin",
    ↪"kl_weight", "margin_weight",
        "uses_guarded_global_head", "true_guard_control_fixed",
    ↪"paired_same_backbone"
    ])
    return repair_log_df.loc[mask, cols].copy()

def build_memory_audit(history_df):
    if history_df.empty:
        return pd.DataFrame()
    final_task = int(history_df.after_task.max()) if "after_task" in history_df
    ↪else N_TASKS - 1

```

```

    sub = history_df[(history_df.after_task == final_task) & (history_df.
↪task_id == final_task)].copy()
    cols = _first_available(sub, [
        "method", "seed", "model_family", "replay_memory_items",
↪"memory_per_class", "memory_per_task",
        "total_params", "readout_total_params", "model_size_mb_total",
↪"uses_balanced_replay",
        "uses_context_selection", "uses_global_head_refresh",
↪"context_temperature_used",
        "batch_blend_single_used", "batch_blend_latent_used",
↪"batch_blend_feature_used"
    ])
    return sub[cols].sort_values(["seed", "method"]).copy()

def build_probe_gap_audit(scorecard_df):
    if scorecard_df.empty:
        return pd.DataFrame()
    def row(method):
        s = scorecard_df[scorecard_df.method == method]
        return s.iloc[0] if not s.empty else None
    rc = row("paired_context_gap_selected")
    proto = row("probe_proto__zf_proto")
    oracle = row("probe_proto__zf_oracle")
    rows = []
    if rc is not None and proto is not None:
        rows.append({
            "gap_name": "deployed_to_proto_zf_probe",
            "from_method": "paired_context_gap_selected",
            "to_method": "probe_proto__zf_proto",
            "gap": float(proto.final_avg_acc_mean - rc.final_avg_acc_mean),
        })
    if proto is not None and oracle is not None:
        rows.append({
            "gap_name": "proto_to_oracle_context_gap",
            "from_method": "probe_proto__zf_proto",
            "to_method": "probe_proto__zf_oracle",
            "gap": float(oracle.final_avg_acc_mean - proto.final_avg_acc_mean),
        })
    if rc is not None and oracle is not None:
        rows.append({
            "gap_name": "deployed_to_oracle_zf_probe",
            "from_method": "paired_context_gap_selected",
            "to_method": "probe_proto__zf_oracle",
            "gap": float(oracle.final_avg_acc_mean - rc.final_avg_acc_mean),
        })
    return pd.DataFrame(rows)

```

```

seed_diag_df = build_seed_diagnostics(per_seed_df, class_diag_df)
rc1b_guard_audit_df = build_rc1b_vs_guard_audit(per_seed_df)
context_selection_audit_df = build_context_selection_audit(repair_log_df)
head_repair_audit_df = build_head_repair_audit(repair_log_df)
memory_audit_df = build_memory_audit(history_df)
probe_gap_audit_df = build_probe_gap_audit(scorecard_df)

exports = {
    "seed_diagnostics": seed_diag_df,
    "rc1b_vs_true_guard_seed_audit": rc1b_guard_audit_df,
    "context_selection_trace": context_selection_audit_df,
    "head_repair_trace": head_repair_audit_df,
    "memory_audit": memory_audit_df,
    "probe_gap_audit": probe_gap_audit_df,
}
for name, df in exports.items():
    path = AUDIT_DIR / f"{name}_{RUN_MODE}.csv"
    df.to_csv(path, index=False)
    print("Saved audit artifact:", path)

# Create OTEL-style span rows: one span per seed/method final summary and per_
# weak-seed diagnosis.
span_rows = []
for _, r in seed_diag_df.iterrows():
    span_rows.append({
        "trace_id": TRACE_ID,
        "span_id": hashlib.sha256(f"{RUN_ID}:{r.get('seed')}:{r.get('method')}").
        encode().hexdigest()[:16],
        "parent_span_id": TRACE_ID,
        "span_name": "method_seed_final_score",
        "timestamp_utc": datetime.now(timezone.utc).isoformat(),
        "seed": safe_jsonable(r.get("seed")),
        "method": safe_jsonable(r.get("method")),
        "final_avg_acc": safe_jsonable(r.get("final_avg_acc")),
        "min_task_acc": safe_jsonable(r.get("min_task_acc")),
        "avg_forgetting": safe_jsonable(r.get("avg_forgetting")),
        "class2_acc": safe_jsonable(r.get("class2_acc")),
        "class5_acc": safe_jsonable(r.get("class5_acc")),
        "pred4_bias": safe_jsonable(r.get("pred4_bias")),
    })
TRACE_SPANS.extend(span_rows)
span_df = pd.DataFrame(TRACE_SPANS)
span_df.to_csv(TRACE_SPANS_PATH, index=False)
print("Saved OTEL-style spans:", TRACE_SPANS_PATH)

# Add compact JSONL events for key audit rows.
for _, r in rc1b_guard_audit_df.iterrows():

```



```

    log_event("rc1b_vs_true_guard_seed_audit", seed=safe_jsonable(r.
    ↪get("seed")), method="paired_context_gap_selected", phase="post_run_audit",
    ↪payload=r.to_dict(), level="WARN" if bool(r.get("weak_seed_flag")) else
    ↪"INFO")
for _, r in probe_gap_audit_df.iterrows():
    log_event("probe_gap_audit", method=safe_jsonable(r.get("from_method")),
    ↪phase="post_run_audit", payload=r.to_dict())

weak_rows = rc1b_guard_audit_df[rc1b_guard_audit_df.get("weak_seed_flag",
    ↪False) == True] if not rc1b_guard_audit_df.empty else pd.DataFrame()
weak_seed_report = {
    "n_weak_seeds": int(len(weak_rows)),
    "weak_seeds": safe_jsonable(weak_rows.to_dict("records")) if not weak_rows.
    ↪empty else [],
    "diagnostic_interpretation": "Weak seeds are candidates for localized
    ↪inspection: check context_selection_trace, head_repair_trace, memory_audit,
    ↪class_diagnostics, and probe_gap_audit.",
}
weak_seed_report_path = AUDIT_DIR / f"weak_seed_report_{RUN_MODE}.json"
with open(weak_seed_report_path, "w", encoding="utf-8") as f:
    json.dump(weak_seed_report, f, indent=2, ensure_ascii=False)
print("Saved weak seed report:", weak_seed_report_path)
log_event("observability_audit_completed", phase="post_run_audit",
    ↪payload={"audit_files": [str(AUDIT_DIR / f"{k}_{RUN_MODE}.csv") for k in
    ↪exports] + [str(weak_seed_report_path)]})

# Write/update manifest after audit outputs are available.
manifest = write_manifest(extra={
    "audit_exports": list(exports.keys()),
    "weak_seed_report": str(weak_seed_report_path),
    "trace_jsonl": str	TRACE_JSONL_PATH),
    "trace_spans": str	TRACE_SPANS_PATH),
})
print("Saved manifest:", MANIFEST_PATH)

print("\n=== Weak seed audit preview ===")
display(rc1b_guard_audit_df)
print("\n=== Probe-gap audit ===")
display(probe_gap_audit_df)

```

	seed	rc1b_final_avg_acc	rc1b_min_task_acc	rc1b_avg_forgetting	\
0	0	0.88525	0.84125	0.076875	
1	1	0.87675	0.74625	0.077187	
2	2	0.93900	0.91375	0.029688	
3	3	0.90875	0.79375	0.068125	
4	4	0.89525	0.79625	0.026250	

	rc1b_class2_acc	rc1b_class5_acc	rc1b_pred4_bias	guard_final_avg_acc	\
0	0.79625	0.9975	0.183958	0.88375	
1	0.68250	0.9975	0.182500	0.88350	
2	0.92375	1.0000	0.178958	0.89700	
3	0.79875	0.9975	0.189583	0.91175	
4	0.91625	1.0000	0.146875	0.79850	

	guard_min_task_acc	guard_avg_forgetting	guard_class2_acc	\
0	0.84125	0.076875	0.79375	
1	0.74500	0.078438	0.67250	
2	0.76375	0.077500	0.75875	
3	0.79125	0.060625	0.80250	
4	0.49375	0.135625	0.56750	

	guard_class5_acc	guard_pred4_bias	delta_final_avg_acc_rc1b_minus_guard	\
0	0.9975	0.187708	0.00150	
1	0.9950	0.188958	-0.00675	
2	0.9975	0.207708	0.04200	
3	0.9975	0.190000	-0.00300	
4	0.9750	0.210208	0.09675	

	delta_min_task_acc_rc1b_minus_guard	delta_class2_acc_rc1b_minus_guard	\
0	0.00000	0.00250	
1	0.00125	0.01000	
2	0.15000	0.16500	
3	0.00250	-0.00375	
4	0.30250	0.34875	

	delta_class5_acc_rc1b_minus_guard	\
0	0.0000	
1	0.0025	
2	0.0025	
3	0.0000	
4	0.0250	

	delta_forgetting_reduction_rc1b_vs_guard	weak_seed_flag
0	0.000000	True
1	0.001250	True
2	0.047813	False
3	-0.007500	True
4	0.109375	True

	gap_name	from_method	\
0	deployed_to_proto_zf_probe	paired_context_gap_selected	
1	proto_to_oracle_context_gap	probe_proto__zf_proto	
2	deployed_to_oracle_zf_probe	paired_context_gap_selected	

	to_method	gap
--	-----------	-----

```

0 probe_proto__zf_proto 0.00525
1 probe_proto__zf_oracle 0.03300
2 probe_proto__zf_oracle 0.03825

```

2.38 16c. RC2d / Hydro policy-selection audit

This table summarizes the Hydro diagnostics that were computed before final-test evaluation, using only the policy-validation memory split. It is the main Hydro audit layer beside RC2d.

```

[ ]: # -----
# RC2d Hydro policy-selection audit summary.
# -----
if "selection_log_df" in globals() and not selection_log_df.empty:
    hydro_cols = [
        "hydro_route_drift_Dr", "hydro_latent_drift_Dz",
        ↪ "hydro_output_drift_Dy",
        "hydro_adaptive_viscosity_nu", "hydro_reynolds_like",
        ↪ "hydro_turbulence_score",
        "hydro_wave_entropy", "hydro_selection_penalty",
    ]
    available_hydro_cols = [c for c in hydro_cols if c in selection_log_df.
        ↪ columns]
    group_cols = ["candidate_variant", "candidate_method", "policy_family",
        ↪ "is_selected_candidate"]
    agg_spec = {c: (c, "mean") for c in available_hydro_cols}
    agg_spec.update({
        "n_rows": ("candidate_variant", "size"),
        "mean_proxy_acc": ("proxy_acc", "mean"),
        "mean_proxy_score": ("proxy_score", "mean"),
    })
    rc3_hydro_policy_audit_df = selection_log_df.groupby(group_cols,
        ↪ as_index=False).agg(**agg_spec)
    rc3_hydro_policy_audit_df["hydro_warning"] = False
    if "hydro_turbulence_score" in rc3_hydro_policy_audit_df:
        rc3_hydro_policy_audit_df["hydro_warning"] =
        ↪ rc3_hydro_policy_audit_df["hydro_warning"] |
        ↪ (rc3_hydro_policy_audit_df["hydro_turbulence_score"] >
        ↪ RC3_HYDRO_MECHANISTIC_TURBULENCE_WARNING)
    if "hydro_latent_drift_Dz" in rc3_hydro_policy_audit_df:
        rc3_hydro_policy_audit_df["hydro_warning"] =
        ↪ rc3_hydro_policy_audit_df["hydro_warning"] |
        ↪ (rc3_hydro_policy_audit_df["hydro_latent_drift_Dz"] >
        ↪ RC3_HYDRO_MECHANISTIC_LATENT_WARNING)
    hydro_policy_audit_path = AUDIT_DIR /
        ↪ f"rc2d_hydro_policy_selection_audit_{RUN_MODE}.csv"
    rc3_hydro_policy_audit_df.to_csv(hydro_policy_audit_path, index=False)
    print("Saved RC2d Hydro policy-selection audit:", hydro_policy_audit_path)

```

```

        display(rc3_hydro_policy_audit_df.sort_values(["is_selected_candidate",
↪ "mean_proxy_score"], ascending=[False, False]))
        log_event("rc2d_hydro_policy_audit_completed", phase="post_run_audit",
↪ payload={"audit_file": str(hydro_policy_audit_path), "hydro_policy_mode":
↪ RC3_HYDRO_POLICY_MODE})
    else:
        rc3_hydro_policy_audit_df = pd.DataFrame()
        print("No selection_log_df found; RC3 Hydro audit summary not generated.")

```

	candidate_variant \	candidate_method \	policy_family is_selected_candidate \
7	generic_weak_boundary_latent_calibrated	paired_generic_weak_boundary_latent_calibrated	generic_latent_boundary_calibration True
1	context_blend_selected_global	paired_context_blend_selected_global	context_only_global True
5	context_temp_blend_selected_head_guard_035	paired_context_temp_blend_selected_head_guard_035	rc1b_guarded_context_gap True
9	proto_global_head_ce_kl_guard_035	paired_proto_global_head_ce_kl_guard_035	proto_true_global_guard True
11	proto_global_no_repair	paired_proto_global_no_repair	proto_no_repair True
2	context_gap_selected	paired_context_gap_selected	rc1b_guarded_context_gap False
4	context_temp_blend_selected_head_guard_035	paired_context_temp_blend_selected_head_guard_035	rc1b_guarded_context_gap False
0	context_blend_selected_global	paired_context_blend_selected_global	context_only_global False
8	proto_global_head_ce_kl_guard_035	paired_proto_global_head_ce_kl_guard_035	proto_true_global_guard False
3	context_temp_blend_selected_global	paired_context_temp_blend_selected_global	
6	generic_weak_boundary_latent_calibrated	paired_generic_weak_boundary_latent_calibrated	
10	proto_global_no_repair	paired_proto_global_no_repair	

3	context_only_global	False
6	generic_latent_boundary_calibration	False
10	proto_no_repair	False

	hydro_route_drift_Dr	hydro_latent_drift_Dz	hydro_output_drift_Dy \
7	0.000358	0.015023	3.267691e-02
1	0.000043	0.001047	1.630749e-03
5	0.006230	0.058987	2.810021e-01
9	0.000000	0.000000	4.449983e-02
11	0.000000	0.000000	2.380148e-09
2	0.003789	0.045780	1.724297e-01
4	0.001871	0.035403	8.712274e-02
0	0.006101	0.083674	1.975053e-01
8	0.000000	0.000000	7.015290e-02
3	0.003972	0.067142	1.433305e-01
6	0.001658	0.029728	1.260542e-01
10	0.000000	0.000000	2.871945e-09

	hydro_adaptive_viscosity_nu	hydro_reynolds_like	hydro_turbulence_score \
7	1.775052	0.040674	0.043126
1	1.666807	0.017556	0.017640
5	1.589410	0.081315	0.296806
9	1.561166	0.000000	0.000000
11	1.803719	0.000000	0.000000
2	1.641147	0.071893	0.182375
4	1.681798	0.064489	0.092465
0	1.640078	0.091919	0.288956
8	1.666404	0.000000	0.000000
3	1.641147	0.083442	0.212464
6	1.615642	0.060681	0.143060
10	1.618978	0.000000	0.000000

	hydro_wave_entropy	hydro_selection_penalty	n_rows	mean_proxy_acc \
7	1.536280e-01	0.0	4	0.930898
1	2.184864e-12	0.0	1	0.953776
5	1.273934e-02	0.0	11	0.927044
9	1.944946e-01	0.0	6	0.931780
11	1.375333e-05	0.0	3	0.934115
2	4.369675e-02	0.0	25	0.928361
4	6.802043e-02	0.0	14	0.929395
0	4.774660e-02	0.0	24	0.924632
8	1.881447e-01	0.0	19	0.926937
3	1.205328e-01	0.0	25	0.924917
6	1.106262e-01	0.0	21	0.926253
10	2.155307e-01	0.0	22	0.919638

	mean_proxy_score	hydro_warning
7	1.744300	False

1	1.673456	False
5	1.612941	True
9	1.502497	False
11	1.251725	False
2	1.560436	False
4	1.519182	False
0	1.497268	True
8	1.483549	False
3	1.478928	True
6	1.457924	False
10	1.078464	False

2.39 16d. Panneau E — Learning-Signal Alignment

This panel audits whether MMALS updates the hosts that the route/memory/contribution signals suggest should receive the learning signal.

It compares:

Expected update allocation vs Observed update allocation

The main metric is:

LSA = alignment(expected_update_importance, observed_update_norm)

High LSA means the update signal touches the right modules. Low or negative LSA means the model may still learn, but the learning signal may be circulating through the wrong hosts.

```
[ ]: # -----
# Panel E display: Learning-Signal Alignment.
# -----
from IPython.display import display, Markdown, Image

lsa_plot_paths = []
if "learning_signal_alignment_df" not in globals() or not
↳ isinstance(learning_signal_alignment_df, pd.DataFrame) or
↳ learning_signal_alignment_df.empty:
    display(Markdown("### Panel E - Learning-Signal Alignment\nNo LSA rows were
↳ generated. Check `ENABLE_LEARNING_SIGNAL_ALIGNMENT_AUDIT`."))
else:
    display(Markdown("### Panel E - Learning-Signal Alignment summary"))
    if "learning_signal_alignment_summary_df" not in globals() or
↳ learning_signal_alignment_summary_df.empty:
        learning_signal_alignment_summary_df =
↳ summarize_learning_signal_alignment(learning_signal_alignment_df)
        cols = [
            "seed", "task_id", "host", "n_batches", "mean_route_weight",
↳ "mean_mutualistic_gain",
            "mean_memory_risk", "mean_expected_update_importance",
↳ "mean_observed_update_allocation",
```

```

        "mean_lsa_pearson", "mean_lsa_cosine", "suspicious_rate",
    ]
    cols = [c for c in cols if c in learning_signal_alignment_summary_df.
↪columns]
    display(learning_signal_alignment_summary_df[cols].head(30))

    display(Markdown("### Panel E - most suspicious host-update rows"))
    suspicious =
↪learning_signal_alignment_df[learning_signal_alignment_df["verdict"].
↪astype(str).str.lower() != "ok"].copy()
    if suspicious.empty:
        display(Markdown("No suspicious LSA host-update rows detected."))
    else:
        keep = [
            "seed", "task_id", "epoch", "batch_index", "host", "route_weight",
↪"mutualistic_gain",
            "memory_risk_level", "expected_update_importance",
↪"observed_update_allocation",
            "lsa_pearson", "lsa_cosine", "verdict",
        ]
        keep = [c for c in keep if c in suspicious.columns]
        display(suspicious[keep].sort_values(["lsa_pearson",
↪"observed_update_allocation"], ascending=[True, False]).head(30))

    try:
        plot_df = learning_signal_alignment_df.groupby(["task_id", "epoch"],
↪as_index=False).agg(
            lsa_pearson=("lsa_pearson", "mean"),
            lsa_cosine=("lsa_cosine", "mean"),
        )
        fig = plt.figure(figsize=(9, 5))
        for task_id, g in plot_df.groupby("task_id"):
            plt.plot(g["epoch"], g["lsa_pearson"], marker="o", label=f"task_
↪{task_id}")
            plt.axhline(0.0, linestyle="--", linewidth=1)
            plt.xlabel("Epoch")
            plt.ylabel("LSA Pearson alignment")
            plt.title("Panel E - Learning-Signal Alignment by task/epoch")
            plt.legend(fontsize=8)
            plt.grid(True, alpha=0.3)
            lsa_plot_dir = PLOTS_DIR / "learning_signal_alignment"
            lsa_plot_dir.mkdir(parents=True, exist_ok=True)
            lsa_plot_path = lsa_plot_dir / f"panel_e_lsa_by_task_epoch_{RUN_MODE}.
↪png"
            fig.savefig(lsa_plot_path, dpi=160, bbox_inches="tight")
            plt.close(fig)

```

```

lsa_plot_paths.append(lsa_plot_path)
display(Markdown(f"Panel E plot saved: `{lsa_plot_path}`"))
display(Image(filename=str(lsa_plot_path)))
except Exception as e:
    display(Markdown(f"Panel E plot generation skipped: `{repr(e)}`"))

```

2.39.1 Panel E — Learning-Signal Alignment summary

	seed	task_id	host	n_batches	mean_route_weight	mean_mutualistic_gain \
4	0	0	H5	152	0.003341	0.001199
0	0	0	H1	152	0.002432	0.000919
1	0	0	H2	152	0.003020	0.001033
2	0	0	H3	152	0.004166	0.000616
3	0	0	H4	152	0.987041	0.985376
8	0	1	H4	152	0.999062	0.998836
6	0	1	H2	152	0.000494	0.000117
7	0	1	H3	152	0.000278	0.000091
5	0	1	H1	152	0.000109	0.000007
9	0	1	H5	152	0.000057	0.000011
13	0	2	H4	152	0.787511	0.487841
11	0	2	H2	152	0.205014	0.128800
10	0	2	H1	152	0.003329	-0.000526
12	0	2	H3	152	0.001099	0.000266
14	0	2	H5	152	0.003047	0.000952
18	0	3	H4	152	0.681271	0.302470
16	0	3	H2	152	0.290320	0.155184
15	0	3	H1	152	0.004305	-0.001197
17	0	3	H3	152	0.005335	0.002844
19	0	3	H5	152	0.018768	0.009045
23	0	4	H4	152	0.586971	0.232690
21	0	4	H2	152	0.330956	0.112756
20	0	4	H1	152	0.001763	-0.000271
22	0	4	H3	152	0.005801	0.003186
24	0	4	H5	152	0.074509	0.047806
27	1	0	H3	152	0.492320	0.383333
28	1	0	H4	152	0.488274	0.381012
25	1	0	H1	152	0.007429	0.003138
26	1	0	H2	152	0.006776	0.002780
29	1	0	H5	152	0.005200	0.001730

	mean_memory_risk	mean_expected_update_importance \
4	0.000000	0.002993
0	0.000000	0.002275
1	0.000000	0.002759
2	0.000000	0.003256
3	0.000000	0.988716
8	0.999062	0.999239

6	0.000494	0.000400
7	0.000278	0.000231
5	0.000109	0.000085
9	0.000057	0.000045
13	0.787511	0.773040
11	0.205014	0.220723
10	0.003329	0.002498
12	0.001099	0.000954
14	0.003047	0.002785
18	0.681271	0.670137
16	0.290320	0.302333
15	0.004305	0.003230
17	0.005335	0.005490
19	0.018768	0.018811
23	0.586971	0.579071
21	0.330956	0.324862
20	0.001763	0.001322
22	0.005801	0.006447
24	0.074509	0.088299
27	0.000000	0.493272
28	0.000000	0.489492
25	0.000000	0.006519
26	0.000000	0.006170
29	0.000000	0.004548

	mean_observed_update_allocation	mean_lsa_pearson	mean_lsa_cosine \
4	0.002701	0.998893	0.999951
0	0.001989	0.998893	0.999951
1	0.002906	0.998893	0.999951
2	0.003653	0.998893	0.999951
3	0.988750	0.998893	0.999951
8	0.990235	0.999891	0.999869
6	0.004639	0.999891	0.999869
7	0.002435	0.999891	0.999869
5	0.002041	0.999891	0.999869
9	0.000651	0.999891	0.999869
13	0.756197	0.957909	0.970114
11	0.213877	0.957909	0.970114
10	0.022436	0.957909	0.970114
12	0.000471	0.957909	0.970114
14	0.007019	0.957909	0.970114
18	0.700335	0.966187	0.976341
16	0.252855	0.966187	0.976341
15	0.026013	0.966187	0.976341
17	0.004045	0.966187	0.976341
19	0.016752	0.966187	0.976341
23	0.727575	0.865816	0.912478
21	0.111184	0.865816	0.912478

20	0.029065	0.865816	0.912478
22	0.033437	0.865816	0.912478
24	0.098739	0.865816	0.912478
27	0.660047	0.846358	0.895171
28	0.304295	0.846358	0.895171
25	0.020509	0.846358	0.895171
26	0.008808	0.846358	0.895171
29	0.006340	0.846358	0.895171

	suspicious_rate
4	0.006579
0	0.000000
1	0.000000
2	0.000000
3	0.000000
8	1.000000
6	0.611842
7	0.256579
5	0.138158
9	0.000000
13	1.000000
11	0.993421
10	0.000000
12	0.000000
14	0.000000
18	1.000000
16	0.986842
15	0.013158
17	0.000000
19	0.000000
23	1.000000
21	0.605263
20	0.026316
22	0.000000
24	0.000000
27	0.006579
28	0.006579
25	0.000000
26	0.000000
29	0.000000

2.39.2 Panel E — most suspicious host-update rows

	seed	task_id	epoch	batch_index	host	route_weight	mutualistic_gain	\
12135	3	0	7	14	H1	0.041123	0.025238	
12138	3	0	7	14	H4	0.404916	0.263460	
12136	3	0	7	14	H2	0.531043	0.460410	
12139	3	0	7	14	H5	0.003857	0.001838	

12137	3	0	7	14	H3	0.019060	0.015371
17453	4	2	7	13	H4	0.987424	0.986728
17353	4	2	6	12	H4	0.980472	0.977599
17398	4	2	7	2	H4	0.980919	0.976925
17413	4	2	7	5	H4	0.981848	0.979864
5853	1	2	5	11	H4	0.968032	0.959556
17368	4	2	6	15	H4	0.975733	0.971968
18893	4	4	6	16	H4	0.314346	-0.062278
18892	4	4	6	16	H3	0.399429	0.283683
7597	1	4	7	18	H3	0.032520	0.013054
7598	1	4	7	18	H4	0.822389	0.549710
18585	4	4	3	12	H1	0.464573	0.244218
18587	4	4	3	12	H3	0.262316	0.154403
6057	1	2	7	14	H3	0.008093	0.000006
6058	1	2	7	14	H4	0.966682	0.957041
18870	4	4	6	12	H1	0.429529	0.231891
18872	4	4	6	12	H3	0.305293	0.186037
18605	4	4	3	16	H1	0.468999	0.247471
18607	4	4	3	16	H3	0.264117	0.164092
18363	4	4	1	5	H4	0.286196	-0.025112
18598	4	4	3	14	H4	0.266362	-0.046746
18595	4	4	3	14	H1	0.441951	0.226107
18597	4	4	3	14	H3	0.254691	0.150774
17953	4	3	4	18	H4	0.371798	0.115504
7507	1	4	7	0	H3	0.015733	0.004390
7508	1	4	7	0	H4	0.856412	0.523191

	memory_risk_level	expected_update_importance \
12135	low	0.038564
12138	low	0.385817
12136	low	0.552845
12139	low	0.003401
12137	low	0.019372
17453	high	0.990426
17353	high	0.985214
17398	high	0.985560
17413	high	0.985999
5853	high	0.975320
17368	high	0.981597
18893	high	0.235760
18892	high	0.483203
7597	low	0.029753
7598	high	0.842651
18585	high	0.497561
18587	high	0.291023
6057	low	0.006071
6058	high	0.973856
18870	high	0.456564

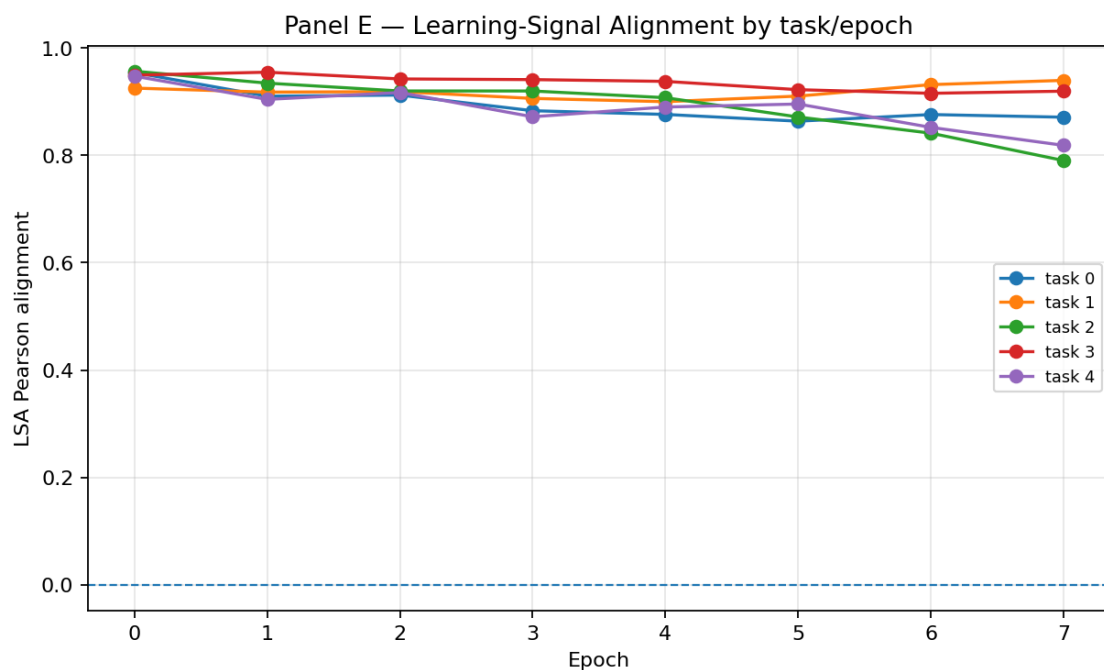
18872	high	0.336807
18605	high	0.498692
18607	high	0.295521
18363	high	0.214647
18598	high	0.199771
18595	high	0.477627
18597	low	0.288484
17953	high	0.338668
7507	low	0.013731
7508	high	0.872475

	observed_update_allocation	lsa_pearson	lsa_cosine	\
12135	0.436103	-0.090437	0.516596	
12138	0.247686	-0.090437	0.516596	
12136	0.130518	-0.090437	0.516596	
12139	0.097018	-0.090437	0.516596	
12137	0.088675	-0.090437	0.516596	
17453	0.211825	0.020729	0.275066	
17353	0.216406	0.027283	0.279662	
17398	0.215815	0.027330	0.281582	
17413	0.222091	0.041340	0.297742	
5853	0.232454	0.070590	0.339013	
17368	0.247913	0.099489	0.358156	
18893	0.461124	0.130250	0.657134	
18892	0.081038	0.130250	0.657134	
7597	0.434649	0.146920	0.510083	
7598	0.238757	0.146920	0.510083	
18585	0.227210	0.152464	0.589913	
18587	0.033380	0.152464	0.589913	
6057	0.535377	0.170461	0.435986	
6058	0.265200	0.170461	0.435986	
18870	0.262666	0.206271	0.602598	
18872	0.041905	0.206271	0.602598	
18605	0.259613	0.240700	0.645646	
18607	0.057538	0.240700	0.645646	
18363	0.603413	0.246886	0.615169	
18598	0.328978	0.253276	0.739595	
18595	0.277274	0.253276	0.739595	
18597	0.077423	0.253276	0.739595	
17953	0.531761	0.261363	0.612193	
7507	0.439742	0.278874	0.554254	
7508	0.289433	0.278874	0.554254	

	verdict
12135	global_lsa_negative_risk
12138	global_lsa_negative_risk
12136	global_lsa_negative_risk
12139	global_lsa_negative_risk

12137	global_lsa_negative_risk
17453	memory_risk_high_update
17353	memory_risk_high_update
17398	memory_risk_high_update
17413	memory_risk_high_update
5853	memory_risk_high_update
17368	memory_risk_high_update
18893	memory_risk_high_update
18892	under_updated_expected_host
7597	suspicious_over_update
7598	memory_risk_high_update
18585	memory_risk_high_update
18587	under_updated_expected_host
6057	suspicious_over_update
6058	memory_risk_high_update
18870	memory_risk_high_update
18872	under_updated_expected_host
18605	memory_risk_high_update
18607	under_updated_expected_host
18363	memory_risk_high_update
18598	memory_risk_high_update
18595	memory_risk_high_update
18597	under_updated_expected_host
17953	memory_risk_high_update
7507	suspicious_over_update
7508	memory_risk_high_update

Panel E plot saved: /content/drive/MyDrive/MMALS/v1_1_rc2m_alpha_boundary_first_frozen_selector/RC



2.40 16e. RotatedMNIST final-rotation diagnostic gate

This diagnostic is specific to `DATASET_NAME = "RotatedMNIST"`. It focuses on the final overlap-chain task (4,5) @ +30deg and answers:

- Is class 5 / final rotation weak?
- If yes, is the most likely source route/context inference, readout boundary calibration, or latent separability?

The gate is diagnostic-only. It does not change training, policy selection, Hydro, Panel E / LSA, candidate construction, or final-test evaluation.

```
[ ]: # -----  
# RotatedMNIST-specific final-rotation weakness diagnostic gate.  
# -----  
# Diagnostic-only. This runs after final-test evaluation and after policy  
# selection, so it must never feed back into selection or training.  
from IPython.display import display, Markdown, Image  
  
def _rot_safe_float(x, default=np.nan):  
    try:  
        if x is None:  
            return default  
        if hasattr(x, "item"):  
            x = x.item()  
        x = float(x)  
        return x if np.isfinite(x) else default  
    except Exception:  
        return default  
  
def _rot_metric(row, col, default=np.nan):  
    if row is None:  
        return default  
    try:  
        if col not in row:  
            return default  
        return _rot_safe_float(row[col], default=default)  
    except Exception:  
        return default  
  
def _rot_scorecard_row(scorecard, method):  
    if scorecard is None or scorecard.empty or "method" not in scorecard:  
        return None
```

```

sub = scorecard[scorecard["method"].astype(str) == str(method)]
return sub.iloc[0] if not sub.empty else None

def _rot_confusion_rate(confusion, method, true_class, pred_class):
    if confusion is None or confusion.empty:
        return np.nan
    needed = {"method", "true_class", "pred_class", "rate"}
    if not needed.issubset(set(confusion.columns)):
        return np.nan
    sub = confusion[
        (confusion["method"].astype(str) == str(method))
        & (confusion["true_class"].astype(int) == int(true_class))
        & (confusion["pred_class"].astype(int) == int(pred_class))
    ]
    return float(sub["rate"].mean()) if not sub.empty else np.nan

def _rot_probe_row(scorecard, method):
    return _rot_scorecard_row(scorecard, method)

def build_rotatedmnist_final_rotation_diagnostic(
    scorecard_df,
    per_seed_df,
    history_df,
    confusion_df,
    score_dump_df=None,
    lsa_summary_df=None,
    primary_method=None,
):
    if not bool(globals().get("ENABLE_ROTATEDMNIST_FINAL_ROTATION_DIAGNOSTIC",
↪False)):
        return pd.DataFrame(), pd.DataFrame()
    if str(globals().get("DATASET_NAME", "")) != "RotatedMNIST":
        return pd.DataFrame(), pd.DataFrame()

    primary_method = primary_method or globals().get("RC2B_METHOD",
↪"paired_rc2jf_true_guard_safe_veto_policy")
    final_task = int(globals().get("ROTATED_DIAG_FINAL_TASK_ID", globals().
↪get("N_TASKS", 5) - 1))

    selected_row = _rot_scorecard_row(scorecard_df, primary_method)
    proto_probe_row = _rot_probe_row(scorecard_df, "probe_proto_zf_proto")
    oracle_probe_row = _rot_probe_row(scorecard_df, "probe_proto_zf_oracle")
    true_guard_row = _rot_scorecard_row(scorecard_df,
↪"paired_proto_global_head_ce_kl_guard_035")

```

```

    context_gap_row = _rot_scorecard_row(scorecard_df,
↪"paired_context_gap_selected")
    no_repair_row = _rot_scorecard_row(scorecard_df,
↪"paired_proto_global_no_repair")

    # Final-rotation task metrics for the selected deployed policy.
    final_hist = pd.DataFrame()
    if history_df is not None and not history_df.empty:
        final_hist = history_df[
            (history_df.get("method", pd.Series(dtype=str)).astype(str) ==
↪str(primary_method))
            & (history_df.get("after_task", pd.Series(dtype=int)).astype(int)
↪== final_task)
            & (history_df.get("task_id", pd.Series(dtype=int)).astype(int) ==
↪final_task)
        ].copy()

    final_task_acc = float(final_hist["acc"].mean()) if not final_hist.empty
↪and "acc" in final_hist else np.nan
    final_context_top1 = float(final_hist["context_top1_acc"].mean()) if not
↪final_hist.empty and "context_top1_acc" in final_hist else np.nan
    final_context_entropy = float(final_hist["context_entropy"].mean()) if not
↪final_hist.empty and "context_entropy" in final_hist else np.nan
    final_context_margin = float(final_hist["context_margin"].mean()) if not
↪final_hist.empty and "context_margin" in final_hist else np.nan
    final_latent_margin = float(final_hist["latent_frechet_margin"].mean()) if
↪not final_hist.empty and "latent_frechet_margin" in final_hist else np.nan
    final_class5_margin_vs4 = float(final_hist["class5_margin_vs4"].mean()) if
↪not final_hist.empty and "class5_margin_vs4" in final_hist else
↪_rot_metric(selected_row, "class5_margin_vs4_mean")
    final_class5_logit_rank = float(final_hist["class5_logit_rank"].mean()) if
↪not final_hist.empty and "class5_logit_rank" in final_hist else
↪_rot_metric(selected_row, "class5_logit_rank_mean")

    class5_acc = _rot_metric(selected_row, "class5_acc_mean")
    class4_acc = _rot_metric(selected_row, "class4_acc_mean")
    min_task = _rot_metric(selected_row, "min_task_acc_mean")
    final_avg = _rot_metric(selected_row, "final_avg_acc_mean")
    forgetting = _rot_metric(selected_row, "avg_forgetting_mean")
    class5_to4 = _rot_confusion_rate(confusion_df, primary_method, 5, 4)
    class5_self = _rot_confusion_rate(confusion_df, primary_method, 5, 5)

    # Score-dump final-task class-5 margin signals, when available.
    score_class5_mean = np.nan
    score_class4_mean_on_true5 = np.nan
    probb_margin_5_vs4 = np.nan

```



```

logit_margin_5_vs4 = np.nan
if score_dump_df is not None and isinstance(score_dump_df, pd.DataFrame)
↳and not score_dump_df.empty:
    sd = score_dump_df.copy()
    if "method" in sd and "task_id" in sd and "true_class" in sd:
        sd = sd[(sd["method"].astype(str) == str(primary_method)) &
↳(sd["task_id"].astype(int) == final_task) & (sd["true_class"].astype(int) ==
↳5)]

    if not sd.empty:
        if "score_class_5" in sd:
            score_class5_mean = float(sd["score_class_5"].mean())
        if "score_class_4" in sd:
            score_class4_mean_on_true5 = float(sd["score_class_4"].
↳mean())

        if "score_class_5" in sd and "score_class_4" in sd:
            prob_margin_5_vs4 = float((sd["score_class_5"] -
↳sd["score_class_4"])).mean())
        if "logit_class_5" in sd and "logit_class_4" in sd:
            logit_margin_5_vs4 = float((sd["logit_class_5"] -
↳sd["logit_class_4"])).mean())

    proto_probe_class5 = _rot_metric(proto_probe_row, "class5_acc_mean")
    oracle_probe_class5 = _rot_metric(oracle_probe_row, "class5_acc_mean")
    best_probe_class5 = np.nanmax([proto_probe_class5, oracle_probe_class5]) if
↳any(np.isfinite([proto_probe_class5, oracle_probe_class5])) else np.nan
    proto_probe_final = _rot_metric(proto_probe_row, "final_avg_acc_mean")
    oracle_probe_final = _rot_metric(oracle_probe_row, "final_avg_acc_mean")

    # Candidate comparison helps identify whether the selected selector choice
↳or the deployed readout family is the issue.
    true_guard_class5 = _rot_metric(true_guard_row, "class5_acc_mean")
    context_gap_class5 = _rot_metric(context_gap_row, "class5_acc_mean")
    no_repair_class5 = _rot_metric(no_repair_row, "class5_acc_mean")

    # Panel E / LSA supporting signal for final task.
    final_lsa_pearson = np.nan
    final_lsa_cosine = np.nan
    final_lsa_suspicious_rate = np.nan
    if lsa_summary_df is not None and isinstance(lsa_summary_df, pd.DataFrame)
↳and not lsa_summary_df.empty:
        if "task_id" in lsa_summary_df:
            lsa_final = lsa_summary_df[lsa_summary_df["task_id"].astype(int) ==
↳final_task]
            if not lsa_final.empty:
                final_lsa_pearson = float(lsa_final["mean_lsa_pearson"].mean())
↳if "mean_lsa_pearson" in lsa_final else np.nan

```

```

        final_lsa_cosine = float(lsa_final["mean_lsa_cosine"].mean())
    if "mean_lsa_cosine" in lsa_final else np.nan
        final_lsa_suspicious_rate = float(lsa_final["suspicious_rate"].
mean()) if "suspicious_rate" in lsa_final else np.nan

    class5_floor = float(globals().get("ROTATED_DIAG_CLASS5_FLOOR", 0.90))
    min_task_floor = float(globals().get("ROTATED_DIAG_MIN_TASK_FLOOR", 0.80))
    context_top1_floor = float(globals().get("ROTATED_DIAG_CONTEXT_TOP1_FLOOR",
0.70))
    entropy_ceiling = float(globals().
get("ROTATED_DIAG_CONTEXT_ENTROPY_CEILING", 1.20))
    context_margin_floor = float(globals().
get("ROTATED_DIAG_CONTEXT_MARGIN_FLOOR", 0.05))
    class5_to4_ceiling = float(globals().
get("ROTATED_DIAG_CLASS5_TO_CLASS4_CEILING", 0.25))
    probe_floor = float(globals().get("ROTATED_DIAG_PROBE_CLASS5_FLOOR", 0.85))
    deployed_to_probe_gap_threshold = float(globals().
get("ROTATED_DIAG_DEPLOYED_TO_PROBE_CLASS5_GAP", 0.12))
    oracle_proto_gap_threshold = float(globals().
get("ROTATED_DIAG_ORACLE_PROTO_CLASS5_GAP", 0.08))

    class5_weak = np.isfinite(class5_acc) and class5_acc < class5_floor
    min_task_weak = np.isfinite(min_task) and min_task < min_task_floor
    final_rotation_weak = bool(class5_weak or min_task_weak)

    route_context_issue = bool(
        (np.isfinite(final_context_top1) and final_context_top1 <
context_top1_floor)
        or (np.isfinite(final_context_entropy) and final_context_entropy >
entropy_ceiling)
        or (np.isfinite(final_context_margin) and final_context_margin <
context_margin_floor)
        or (
            np.isfinite(oracle_probe_class5) and np.isfinite(proto_probe_class5)
            and (oracle_probe_class5 - proto_probe_class5) >
oracle_proto_gap_threshold
        )
    )

    readout_boundary_issue = bool(
        (np.isfinite(class5_to4) and class5_to4 > class5_to4_ceiling)
        or (np.isfinite(final_class5_margin_vs4) and final_class5_margin_vs4 <
0.0)
        or (np.isfinite(logit_margin_5_vs4) and logit_margin_5_vs4 < 0.0)
        or (
            np.isfinite(best_probe_class5) and np.isfinite(class5_acc)

```

```

        and (best_probe_class5 - class5_acc) >
↪deployed_to_probe_gap_threshold
    )
)

latent_separability_issue = bool(
    np.isfinite(best_probe_class5) and best_probe_class5 < probe_floor
)

if not final_rotation_weak:
    primary_diagnosis = "no_final_rotation_weakness"
elif latent_separability_issue:
    primary_diagnosis = "latent_separability_limit"
elif route_context_issue and readout_boundary_issue:
    primary_diagnosis = "mixed_route_context_and_readout_boundary"
elif route_context_issue:
    primary_diagnosis = "route_context_inference_issue"
elif readout_boundary_issue:
    primary_diagnosis = "readout_boundary_4_vs_5_issue"
else:
    primary_diagnosis = "unclassified_capacity_or_training_scale_issue"

route_context_risk_score = float(np.nanmean([
    max(0.0, (context_top1_floor - final_context_top1) /
↪max(context_top1_floor, 1e-8)) if np.isfinite(final_context_top1) else np.
↪nan,
    max(0.0, (final_context_entropy - entropy_ceiling) /
↪max(entropy_ceiling, 1e-8)) if np.isfinite(final_context_entropy) else np.
↪nan,
    max(0.0, (context_margin_floor - final_context_margin) /
↪max(context_margin_floor, 1e-8)) if np.isfinite(final_context_margin) else
↪np.nan,
    max(0.0, (oracle_probe_class5 - proto_probe_class5) /
↪max(oracle_proto_gap_threshold, 1e-8)) if np.isfinite(oracle_probe_class5)
↪and np.isfinite(proto_probe_class5) else np.nan,
]))
if not np.isfinite(route_context_risk_score):
    route_context_risk_score = 0.0

readout_boundary_risk_score = float(np.nanmean([
    max(0.0, class5_to4 / max(class5_to4_ceiling, 1e-8)) if np.
↪isfinite(class5_to4) else np.nan,
    max(0.0, -final_class5_margin_vs4) if np.
↪isfinite(final_class5_margin_vs4) else np.nan,

```

```

        max(0.0, best_probe_class5 - class5_acc) /
↪max(deployed_to_probe_gap_threshold, 1e-8) if np.isfinite(best_probe_class5)
↪and np.isfinite(class5_acc) else np.nan,
    ]))
    if not np.isfinite(readout_boundary_risk_score):
        readout_boundary_risk_score = 0.0

    latent_separability_risk_score = float(max(0.0, (probe_floor -
↪best_probe_class5) / max(probe_floor, 1e-8))) if np.
↪isfinite(best_probe_class5) else 0.0

    selected_variant = None
    selected_reason = None
    if "selected_rows" in globals() and isinstance(globals().
↪get("selected_rows"), pd.DataFrame) and not globals().get("selected_rows").
↪empty:
        sr = globals().get("selected_rows")
        sr_final = sr[sr.get("after_task", pd.Series(dtype=int)).astype(int) ==
↪final_task]
        if not sr_final.empty:
            selected_variant = ",".join(sorted(sr_final.
↪get("selected_candidate_variant", pd.Series(dtype=str)).dropna().astype(str).
↪unique()))
            if "selection_reason" in sr_final:
                selected_reason = ",".join(sorted(sr_final["selection_reason"].
↪dropna().astype(str).unique()))

    diag_row = {
        "dataset": DATASET_NAME,
        "run_mode": RUN_MODE,
        "experiment_profile": EXPERIMENT_PROFILE,
        "primary_method": primary_method,
        "final_task_id": final_task,
        "final_rotation_angle": ROTATION_ANGLES[final_task] if final_task <
↪len(ROTATION_ANGLES) else np.nan,
        "selected_candidate_variant_final": selected_variant,
        "selected_reason_final": selected_reason,
        "final_avg_acc": final_avg,
        "min_task_acc": min_task,
        "final_rotation_task_acc": final_task_acc,
        "avg_forgetting": forgetting,
        "class4_acc": class4_acc,
        "class5_acc": class5_acc,
        "class5_floor": class5_floor,
        "class5_weak": bool(class5_weak),
        "min_task_floor": min_task_floor,
    }

```

```

    "min_task_weak": bool(min_task_weak),
    "final_rotation_weak": bool(final_rotation_weak),
    "class5_to_class4_rate": class5_to4,
    "class5_self_rate": class5_self,
    "class5_margin_vs4_history": final_class5_margin_vs4,
    "class5_logit_rank_history": final_class5_logit_rank,
    "score_class5_mean_on_true5": score_class5_mean,
    "score_class4_mean_on_true5": score_class4_mean_on_true5,
    "prob_margin_5_vs4_on_true5": prob_margin_5_vs4,
    "logit_margin_5_vs4_on_true5": logit_margin_5_vs4,
    "final_context_top1_acc": final_context_top1,
    "final_context_entropy": final_context_entropy,
    "final_context_margin": final_context_margin,
    "final_latent_frechet_margin": final_latent_margin,
    "probe_proto_class5_acc": proto_probe_class5,
    "probe_oracle_class5_acc": oracle_probe_class5,
    "best_probe_class5_acc": best_probe_class5,
    "probe_proto_final_avg_acc": proto_probe_final,
    "probe_oracle_final_avg_acc": oracle_probe_final,
    "candidate_true_guard_class5_acc": true_guard_class5,
    "candidate_context_gap_class5_acc": context_gap_class5,
    "candidate_no_repair_class5_acc": no_repair_class5,
    "final_lsa_pearson": final_lsa_pearson,
    "final_lsa_cosine": final_lsa_cosine,
    "final_lsa_suspicious_rate": final_lsa_suspicious_rate,
    "route_context_issue": bool(route_context_issue),
    "readout_boundary_issue": bool(readout_boundary_issue),
    "latent_separability_issue": bool(latent_separability_issue),
    "route_context_risk_score": route_context_risk_score,
    "readout_boundary_risk_score": readout_boundary_risk_score,
    "latent_separability_risk_score": latent_separability_risk_score,
    "primary_diagnosis": primary_diagnosis,
    "diagnostic_only_no_selection_feedback": True,
}

diagnostic_df = pd.DataFrame([diag_row])

gates = []
def add_gate(name, passed, detail):
    gates.append({"check": name, "passed": bool(passed), "detail":
↪str(detail)})

add_gate(
    "rotated_final_rotation_diagnostic_complete",
    True,
    f"primary_diagnosis={primary_diagnosis}; final_task={final_task};
↪angle={diag_row['final_rotation_angle']}",

```

```

)
add_gate(
    "rotated_class5_floor",
    (not np.isfinite(class5_acc)) or class5_acc >= class5_floor,
    f"class5={class5_acc:.4f} target>={class5_floor:.2f}" if np.
↪isfinite(class5_acc) else "class5=nan",
)
add_gate(
    "rotated_min_task_floor",
    (not np.isfinite(min_task)) or min_task >= min_task_floor,
    f"min_task={min_task:.4f} target>={min_task_floor:.2f}" if np.
↪isfinite(min_task) else "min_task=nan",
)
add_gate(
    "route_context_issue_not_primary",
    not route_context_issue,
    f"route_context_issue={route_context_issue}; top1={final_context_top1:.
↪4f}; entropy={final_context_entropy:.4f}; margin={final_context_margin:.4f}"
↪if np.isfinite(final_context_top1) else
↪f"route_context_issue={route_context_issue}",
)
add_gate(
    "readout_boundary_issue_not_primary",
    not readout_boundary_issue,
    f"readout_boundary_issue={readout_boundary_issue};
↪class5_to4={class5_to4:.4f}; margin5v4={final_class5_margin_vs4:.4f};
↪best_probe_gap={(best_probe_class5 - class5_acc) if np.
↪isfinite(best_probe_class5) and np.isfinite(class5_acc) else np.nan:.4f}",
)
add_gate(
    "latent_separability_issue_not_primary",
    not latent_separability_issue,
    f"latent_separability_issue={latent_separability_issue};
↪best_probe_class5={best_probe_class5:.4f}; target>={probe_floor:.2f}" if np.
↪isfinite(best_probe_class5) else
↪f"latent_separability_issue={latent_separability_issue};
↪best_probe_class5=nan",
)
add_gate(
    "diagnostic_only_no_selection_feedback_declared",
    bool(diag_row["diagnostic_only_no_selection_feedback"]),
    "Rotated final-rotation diagnosis is post-selection / post-evaluation
↪only.",
)

gate_df = pd.DataFrame(gates)

```

```

    return diagnostic_df, gate_df

rotated_final_rotation_diagnostic_df, rotated_final_rotation_gate_df =
    ↪ build_rotatedmnist_final_rotation_diagnostic(
        scorecard_df=scorecard_df,
        per_seed_df=per_seed_df,
        history_df=history_df,
        confusion_df=confusion_df,
        score_dump_df=score_dump_df if "score_dump_df" in globals() else pd.
    ↪ DataFrame(),
        lsa_summary_df=learning_signal_alignment_summary_df if
    ↪ "learning_signal_alignment_summary_df" in globals() else pd.DataFrame(),
        primary_method=RC2B_METHOD,
    )

if rotated_final_rotation_diagnostic_df is not None and not
    ↪ rotated_final_rotation_diagnostic_df.empty:
    rotated_diag_path = RESULTS_DIR /
    ↪ f"rotatedmnist_final_rotation_diagnostic_summary_{RUN_MODE}.csv"
    rotated_gate_path = RESULTS_DIR /
    ↪ f"rotatedmnist_final_rotation_gate_{RUN_MODE}.csv"
    rotated_final_rotation_diagnostic_df.to_csv(rotated_diag_path, index=False)
    rotated_final_rotation_gate_df.to_csv(rotated_gate_path, index=False)
    print("Saved RotatedMNIST final-rotation diagnostic:", rotated_diag_path)
    print("Saved RotatedMNIST final-rotation gate:", rotated_gate_path)
    display(Markdown("### RotatedMNIST final-rotation diagnostic"))
    display(rotated_final_rotation_diagnostic_df)
    display(Markdown("### RotatedMNIST final-rotation gate"))
    display(rotated_final_rotation_gate_df)

    # Compact risk plot. Higher bar means stronger evidence for that failure
    ↪ mode.
    try:
        plot_dir = PLOTS_DIR / "rotatedmnist_final_rotation_diagnostic"
        plot_dir.mkdir(parents=True, exist_ok=True)
        r = rotated_final_rotation_diagnostic_df.iloc[0]
        labels = ["route/context", "readout 4 5", "latent separability"]
        values = [
            _rot_safe_float(r.get("route_context_risk_score", 0.0), 0.0),
            _rot_safe_float(r.get("readout_boundary_risk_score", 0.0), 0.0),
            _rot_safe_float(r.get("latent_separability_risk_score", 0.0), 0.0),
        ]
        fig = plt.figure(figsize=(8, 4.8))
        plt.bar(labels, values)
        plt.ylabel("Diagnostic risk score")

```

```

plt.title(f"RotatedMNIST final-rotation weakness attribution -{
↳{RUN_MODE}")
plt.xticks(rotation=15, ha="right")
plt.grid(True, axis="y", alpha=0.3)
plot_path = plot_dir /
↳f"rotatedmnist_final_rotation_diagnostic_{RUN_MODE}.png"
fig.savefig(plot_path, dpi=160, bbox_inches="tight")
plt.close(fig)
print("Saved RotatedMNIST final-rotation diagnostic plot:", plot_path)
display(Image(filename=str(plot_path)))
except Exception as e:
    print("RotatedMNIST final-rotation diagnostic plot skipped:", repr(e))
else:
    rotated_final_rotation_diagnostic_df = pd.DataFrame()
    rotated_final_rotation_gate_df = pd.DataFrame()
    display(Markdown("### RotatedMNIST final-rotation diagnostic\nSkipped:↳
↳DATASET_NAME is not RotatedMNIST or diagnostic disabled."))

```

2.40.1 RotatedMNIST final-rotation diagnostic

Skipped: DATASET_NAME is not RotatedMNIST or diagnostic disabled.

2.41 16. Export results

This cell zips all CSV outputs for upload and external analysis. The ZIP includes history, losses, repair logs, probe losses, forgetting, confusion, scorecards, paired deltas, per-seed RC1b comparisons, and the validation-gate table.

The ZIP also includes the **Panel E Learning-Signal Alignment** tables and plot when `ENABLE_LEARNING_SIGNAL_ALIGNMENT_AUDIT=True`.

2.42 16b. Notebook/PDF evidence archival

This cell stores notebook-level evidence inside the same Google Drive results directory.

It always creates a compact **run-summary PDF** from the tables already produced by the notebook.

It also tries to copy the source `.ipynb` and any already-generated full notebook PDF from Google Drive into `notebook_exports/`.

Optional full notebook PDF export is supported, but disabled by default because LaTeX/Pandoc installation can be slow in Colab.

```

[ ]: # -----
# Notebook/PDF evidence archival helper.
# -----
# By default this cell creates a compact run-summary PDF that is always stored
# in RESULTS_DIR/notebook_exports and included in the final ZIP package.
#

```



```

# If you want to export the full notebook to PDF from inside Colab, set both:
#   INSTALL_FULL_PDF_EXPORT_DEPS = True
#   TRY_FULL_NOTEBOOK_PDF_EXPORT = True
# The dependency installation can take several minutes.
# -----
from matplotlib.backends.backend_pdf import PdfPages
import subprocess
import re

INSTALL_FULL_PDF_EXPORT_DEPS = False
TRY_FULL_NOTEBOOK_PDF_EXPORT = False
COPY_EXISTING_NOTEBOOK_PDF_IF_FOUND = True

NOTEBOOK_EXPORT_DIR.mkdir(parents=True, exist_ok=True)

# Drive-safe filenames. The older notebook used:
#   f"{NOTEBOOK_BASENAME}_{DATASET_NAME}_{RUN_MODE}_{RUN_ID}_summary.pdf"
# which can exceed Google Drive / Matplotlib PDF filename limits and trigger
# OSError: [Errno 22] Invalid argument.
def safe_filename(s, max_len=80):
    s = re.sub(r"[^A-Za-z0-9_.-]+", "_", str(s))
    s = s.strip("._-") or "artifact"
    return s[:max_len].strip("._-") or "artifact"

EXPECTED_FULL_NOTEBOOK_PDF = f"{safe_filename(NOTEBOOK_BASENAME, 48)}_{DATASET_NAME}_{RUN_MODE}.pdf"
RUN_SUMMARY_PDF_PATH = NOTEBOOK_EXPORT_DIR / "run_summary.pdf"

def find_file_by_name(filename: str, max_hits: int = 5):
    roots = []
    for p in [Path("/content/drive/MyDrive"), Path("/content"), Path(".")]:
        if p.exists():
            roots.append(p)
    hits = []
    for root in roots:
        try:
            hits.extend(list(root.rglob(filename))[:max_hits])
        except Exception:
            pass
    # Prefer files outside RESULTS_DIR to avoid copying the same file onto
    # itself.
    unique = []
    seen = set()
    for h in hits:
        try:
            hp = h.resolve()
            if str(hp) not in seen:

```

```

        seen.add(str(hp))
        unique.append(h)
    except Exception:
        pass
    return unique[:max_hits]

def copy_if_exists(src: Path, dst: Path):
    try:
        if src.exists():
            dst.parent.mkdir(parents=True, exist_ok=True)
            if src.resolve() != dst.resolve():
                shutil.copy2(src, dst)
            return dst
    except Exception as e:
        print(f"Copy skipped for {src}: {repr(e)}")
    return None

def text_page(pdf, title, lines, fontsize=10):
    fig = plt.figure(figsize=(8.27, 11.69)) # A4 portrait
    fig.text(0.06, 0.96, title, fontsize=15, weight="bold", va="top")
    y = 0.91
    for line in lines:
        if y < 0.06:
            pdf.savefig(fig, bbox_inches="tight")
            plt.close(fig)
            fig = plt.figure(figsize=(8.27, 11.69))
            y = 0.96
        fig.text(0.06, y, str(line), fontsize=fontsize, va="top",
↪family="monospace")
        y -= 0.026
    pdf.savefig(fig, bbox_inches="tight")
    plt.close(fig)

def image_page(pdf, image_path, title=None):
    fig = plt.figure(figsize=(11.69, 8.27)) # A4 landscape
    title = title or Path(image_path).name
    fig.text(0.04, 0.96, str(title), fontsize=14, weight="bold", va="top")
    try:
        img = plt.imread(str(image_path))
        ax = fig.add_axes([0.04, 0.05, 0.92, 0.86])
        ax.imshow(img)
        ax.axis("off")
    except Exception as e:
        fig.text(0.06, 0.80, f"Could not render image {image_path}: {repr(e)}",
↪fontsize=10)
    pdf.savefig(fig, bbox_inches="tight")
    plt.close(fig)

```

```

def write_run_summary_pdf():
    lines = [
        f"Notebook: {NOTEBOOK_FILENAME}",
        f"Run ID: {RUN_ID}",
        f"Dataset: {DATASET_NAME}",
        f"Pair mode: {PAIR_MODE}",
        f"Rotation angles: {ROTATION_ANGLES if DATASET_NAME == 'RotatedMNIST'
↪else 'n/a'}",
        f"Permutation seeds: {PERMUTATION_SEEDS if DATASET_NAME ==
↪'PermutedMNIST' else 'n/a'}",
        f"RC2b candidates: {RC2B_CANDIDATE_VARIANTS}",
        f"Run mode: {RUN_MODE}",
        f"Seeds: {SEEDS}",
        f"Results dir: {RESULTS_DIR}",
        f"Trace dir: {TRACE_DIR}",
        f"Audit dir: {AUDIT_DIR}",
        f"Config hash: {CONFIG_HASH}",
        "",
        "Strict selected candidate: paired_rc2b_validation_selected_policy",
        "Historical fixed candidate: paired_context_gap_selected",
        "Architecture: proto-first MMALS + validation-memory-selected
↪deployable policy; no final-test leakage for policy selection",
        "",
        "This PDF is a compact evidence summary generated by the notebook.",
        "For the full executed notebook PDF, use the optional nbconvert export
↪or Colab browser PDF export.",
        "",
        f"Biometric-style diagnostics enabled: {globals().
↪get('ENABLE_BIOMETRIC_STYLE_THRESHOLD_DIAGNOSTICS', False)}",
        f"Threshold diagnostics score kind: {globals().
↪get('THRESHOLD_DIAGNOSTICS_SCORE_KIND', 'n/a')}",
    ]

    top_methods = []
    try:
        if "scorecard_df" in globals() and isinstance(scorecard_df, pd.
↪DataFrame) and not scorecard_df.empty:
            sort_col = "final_avg_acc_mean" if "final_avg_acc_mean" in
↪scorecard_df.columns else None
            cols = [
                "method_role", "method", "final_avg_acc_mean",
↪"min_task_acc_mean",
                "avg_forgetting_mean", "class2_acc_mean", "class5_acc_mean"
            ]

```

```

        cols = [c for c in cols if c in scorecard_df.columns]
        if sort_col:
            top = scorecard_df[cols].sort_values(sort_col, ascending=False).
↪head(15)
        else:
            top = scorecard_df[cols].head(15)
            top_methods = top.to_string(index=False).splitlines()
    except Exception as e:
        top_methods = [f"Could not render scorecard table: {repr(e)}"]

    eer_lines = []
    try:
        if "eer_by_method_df" in globals() and isinstance(eer_by_method_df, pd.
↪DataFrame) and not eer_by_method_df.empty:
            cols = [c for c in ["selected_policy", "method", "variant",
↪"macro_eer", "weak_boundary_eer_2_4_5", "micro_eer", "macro_auc"] if c in
↪eer_by_method_df.columns]
            eer_top = eer_by_method_df[cols].sort_values(["selected_policy",
↪"weak_boundary_eer_2_4_5"], ascending=[False, True]).head(20)
            eer_lines = eer_top.to_string(index=False).splitlines()
        except Exception as e:
            eer_lines = [f"Could not render EER diagnostics: {repr(e)}"]

    lsa_lines = []
    try:
        if "learning_signal_alignment_summary_df" in globals() and
↪isinstance(learning_signal_alignment_summary_df, pd.DataFrame) and not
↪learning_signal_alignment_summary_df.empty:
            cols = [c for c in ["seed", "task_id", "host", "mean_lsa_pearson",
↪"mean_lsa_cosine", "suspicious_rate", "mean_expected_update_importance",
↪"mean_observed_update_allocation"] if c in
↪learning_signal_alignment_summary_df.columns]
            lsa_top = learning_signal_alignment_summary_df[cols].
↪sort_values(["mean_lsa_pearson", "suspicious_rate"], ascending=[True,
↪False]).head(30)
            lsa_lines = lsa_top.to_string(index=False).splitlines()
        except Exception as e:
            lsa_lines = [f"Could not render Learning-Signal Alignment summary:
↪{repr(e)}"]

    rotated_diag_lines = []
    try:
        if "rotated_final_rotation_diagnostic_df" in globals() and
↪isinstance(rotated_final_rotation_diagnostic_df, pd.DataFrame) and not
↪rotated_final_rotation_diagnostic_df.empty:

```

```

        cols = [c for c in ["primary_method", "final_rotation_angle",
↪ "class5_acc", "min_task_acc", "final_rotation_task_acc",
↪ "class5_to_class4_rate", "final_context_top1_acc", "final_context_entropy",
↪ "probe_proto_class5_acc", "probe_oracle_class5_acc", "route_context_issue",
↪ "readout_boundary_issue", "latent_separability_issue", "primary_diagnosis"]]
↪ if c in rotated_final_rotation_diagnostic_df.columns]
        rotated_diag_lines += rotated_final_rotation_diagnostic_df[cols].
to_string(index=False).splitlines()
        if "rotated_final_rotation_gate_df" in globals() and
↪ isinstance(rotated_final_rotation_gate_df, pd.DataFrame) and not
↪ rotated_final_rotation_gate_df.empty:
            rotated_diag_lines += ["", "RotatedMNIST final-rotation gate:"]
            rotated_diag_lines += rotated_final_rotation_gate_df.
to_string(index=False).splitlines()
        except Exception as e:
            rotated_diag_lines = [f"Could not render RotatedMNIST final-rotation
↪ diagnostic: {repr(e)}"]

    gate_lines = []
    try:
        if "validation_gate_df" in globals() and isinstance(validation_gate_df,
↪ pd.DataFrame) and not validation_gate_df.empty:
            gate_lines += ["", "RC1b validation gate:"]
            gate_lines += validation_gate_df.to_string(index=False).splitlines()
        if "benchmark_gate_df" in globals() and isinstance(benchmark_gate_df,
↪ pd.DataFrame) and not benchmark_gate_df.empty:
            gate_lines += ["", "Benchmark gate:"]
            gate_lines += benchmark_gate_df.to_string(index=False).splitlines()
        except Exception as e:
            gate_lines += [f"Could not render gate tables: {repr(e)}"]

    artifact_lines = []
    try:
        artifact_lines = [str(p.relative_to(RESULTS_DIR)) for p in
↪ sorted(RESULTS_DIR.rglob("*")) if p.is_file()]
        except Exception:
            artifact_lines = []

    with PdfPages(RUN_SUMMARY_PDF_PATH) as pdf:
        text_page(pdf, "MMALS v1.1-RC2Je EER Diagnostics - Run Summary", lines)
        text_page(pdf, "Top scorecard rows", top_methods if top_methods else
↪ ["Scorecard not available yet."])
        text_page(pdf, "Biometric-style EER summary", eer_lines if eer_lines
↪ else ["EER diagnostics not available yet."])
        text_page(pdf, "Panel E - Learning-Signal Alignment summary", lsa_lines
↪ if lsa_lines else ["Learning-Signal Alignment not available yet."])

```

```

    text_page(pdf, "RotatedMNIST final-rotation diagnostic",
    ↪rotated_diag_lines if rotated_diag_lines else ["RotatedMNIST final-rotation
    ↪diagnostic not available yet."])
    # Embed generated diagnostic plots directly in the compact PDF.
    try:
        # RotatedMNIST final-rotation diagnostic plots
        rotated_plot_dir_local = PLOTS_DIR /
    ↪"rotatedmnist_final_rotation_diagnostic"
        rotated_plot_files = sorted(rotated_plot_dir_local.
    ↪glob(f"*_{RUN_MODE}.png"))
        for p in rotated_plot_files:
            image_page(pdf, p, title=f"RotatedMNIST final-rotation
    ↪diagnostic plot: {p.name}")

        # Panel E plots
        lsa_plot_dir_local = PLOTS_DIR / "learning_signal_alignment"
        lsa_plot_files = sorted(lsa_plot_dir_local.glob(f"*_{RUN_MODE}.
    ↪png"))
        for p in lsa_plot_files:
            image_page(pdf, p, title=f"Panel E Learning-Signal Alignment
    ↪plot: {p.name}")

        threshold_plot_dir_local = PLOTS_DIR / "threshold_diagnostics"
        plot_files = sorted(threshold_plot_dir_local.glob(f"*_{RUN_MODE}.
    ↪png"))
        if plot_files:
            for p in plot_files:
                image_page(pdf, p, title=f"9c threshold diagnostic plot: {p.
    ↪name}")
        else:
            text_page(pdf, "9c threshold diagnostic plots", [f"No plots
    ↪found in {threshold_plot_dir_local}"])
            except Exception as e:
                text_page(pdf, "9c threshold diagnostic plots", [f"Could not embed
    ↪plots: {repr(e)}"])
            text_page(pdf, "Validation / benchmark gates", gate_lines if gate_lines
    ↪else ["Gate tables not available yet."])
            text_page(pdf, "Evidence artifacts stored in this run folder",
    ↪artifact_lines if artifact_lines else ["No artifacts listed yet."],
    ↪fontsize=8)

    return RUN_SUMMARY_PDF_PATH

# Always create a compact summary PDF inside the evidence folder.
try:
    summary_pdf_path = write_run_summary_pdf()

```

```

    print("Run-summary PDF stored at:", summary_pdf_path)
except Exception as e:
    print("WARNING: run-summary PDF failed, continuing run:", repr(e))
    summary_pdf_path = None

# Try to archive the source notebook if it is saved in Drive or /content.
archived_notebook_path = None
notebook_hits = find_file_by_name(NOTEBOOK_FILENAME)
if notebook_hits:
    archived_notebook_path = copy_if_exists(notebook_hits[0],
    ↪NOTEBOOK_EXPORT_DIR / NOTEBOOK_FILENAME)
    print("Notebook source archived:", archived_notebook_path)
else:
    print("Notebook source not found automatically. Save the notebook in Google,
    ↪Drive with this exact filename if you want it archived:")
    print(" -", NOTEBOOK_FILENAME)

# Optionally copy an existing full notebook PDF into the evidence folder.
archived_full_pdf_path = None
if COPY_EXISTING_NOTEBOOK_PDF_IF_FOUND:
    candidate_pdf_names = [
        EXPECTED_FULL_NOTEBOOK_PDF,
        f"{NOTEBOOK_BASENAME}.pdf",
        f"{NOTEBOOK_BASENAME}_{DATASET_NAME}_{RUN_MODE}.pdf",
    ]
    for pdf_name in candidate_pdf_names:
        hits = find_file_by_name(pdf_name)
        if hits:
            archived_full_pdf_path = copy_if_exists(hits[0],
            ↪NOTEBOOK_EXPORT_DIR / pdf_name)
            print("Existing full notebook PDF archived:",
            ↪archived_full_pdf_path)
            break
    if archived_full_pdf_path is None:
        print("No existing full notebook PDF found automatically. Expected one,
        ↪of:")
        for pdf_name in candidate_pdf_names:
            print(" -", pdf_name)

# Optional: install dependencies and try full notebook PDF export from inside,
    ↪Colab.
nbconvert_status = {"attempted": False, "success": False, "message": "not,
    ↪attempted"}
if INSTALL_FULL_PDF_EXPORT_DEPS:
    print("Installing full PDF export dependencies. This can take several,
    ↪minutes.")

```

```

        subprocess.run("apt-get update -qq && apt-get install -y texlive_
↳texlive-xetex texlive-latex-extra pandoc", shell=True, check=False)
        subprocess.run([sys.executable, "-m", "pip", "install", "-q", "nbformat"],_
↳check=False)

if TRY_FULL_NOTEBOOK_PDF_EXPORT:
    nbconvert_status["attempted"] = True
    source_for_nbconvert = archived_notebook_path
    if source_for_nbconvert is None and notebook_hits:
        source_for_nbconvert = notebook_hits[0]
    if source_for_nbconvert is None:
        nbconvert_status["message"] = "No notebook source found for nbconvert."
    else:
        try:
            cmd = [
                "jupyter", "nbconvert",
                "--to", "pdf",
                "--output-dir", str(NOTEBOOK_EXPORT_DIR),
                "--PDFExporter.latex_command=['xelatex', '{filename}'],_
↳'-interaction=nonstopmode']",
                str(source_for_nbconvert),
            ]
            print("Running:", " ".join(cmd))
            proc = subprocess.run(cmd, text=True, capture_output=True,_
↳check=False)
            nbconvert_status["returncode"] = proc.returncode
            nbconvert_status["stdout_tail"] = proc.stdout[-2000:]
            nbconvert_status["stderr_tail"] = proc.stderr[-2000:]
            nbconvert_status["success"] = proc.returncode == 0
            nbconvert_status["message"] = "nbconvert completed" if proc.
↳returncode == 0 else "nbconvert failed"
            except Exception as e:
                nbconvert_status["message"] = repr(e)

with open(NOTEBOOK_EXPORT_DIR / "notebook_pdf_export_status.json", "w",_
↳encoding="utf-8") as f:
    json.dump({
        "notebook_filename": NOTEBOOK_FILENAME,
        "expected_full_notebook_pdf": EXPECTED_FULL_NOTEBOOK_PDF,
        "summary_pdf_path": str(summary_pdf_path),
        "archived_notebook_path": str(archived_notebook_path) if_
↳archived_notebook_path else None,
        "archived_full_pdf_path": str(archived_full_pdf_path) if_
↳archived_full_pdf_path else None,
        "nbconvert_status": nbconvert_status,
    }, f, indent=2)

```



```

log_event("notebook_pdf_evidence_archived", phase="export", payload={
    "summary_pdf_path": str(summary_pdf_path),
    "archived_notebook_path": str(archived_notebook_path) if
↳ archived_notebook_path else None,
    "archived_full_pdf_path": str(archived_full_pdf_path) if
↳ archived_full_pdf_path else None,
    "nbconvert_status": nbconvert_status,
})
write_manifest(extra={"notebook_pdf_evidence": str(NOTEBOOK_EXPORT_DIR)})

```

```

[ ]: {'run_id': 'MMALS_v11_RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT_Fash
ionMNIST_robust_20260601T164013Z_seeds5',
      'trace_id': 'd69abf85b62186c9',
      'created_utc': '2026-06-01T18:12:31.511420+00:00',
      'notebook_version':
'MMALS_v1.1_RC2M_alpha_Boundary_First_Frozen_Selector_Benchmark_Harness',
      'dataset': 'FashionMNIST',
      'run_mode': 'robust',
      'device': 'cpu',
      'python': '3.12.13 (main, Mar  4 2026, 09:23:07) [GCC 11.4.0]',
      'platform': 'Linux-6.6.122+-x86_64-with-glibc2.35',
      'config_hash':
'dc3e4ee21552acc3748ba9c77157552384f88dc1af1f7a6019d32a0beef62d07',
      'config': {'RUN_MODE': 'robust',
                  'DATASET_NAME': 'FashionMNIST',
                  'PAIR_MODE': 'overlap_chain',
                  'N_TASKS': 5,
                  'N_CONTEXTS': 5,
                  'N_HOSTS': 5,
                  'N_CLASSES': 6,
                  'TRAIN_PER_CLASS': 1200,
                  'TEST_PER_CLASS': 400,
                  'SEEDS': [0, 1, 2, 3, 4],
                  'EPOCHS_PER_TASK': 8,
                  'MEMORY_PER_TASK': 256,
                  'MEMORY_PER_CLASS': 256,
                  'LABEL_NOISE_RATE': 0.1,
                  'INPUT_NOISE_STD': 0.05,
                  'CONTEXT_NOISE_RATE': 0.2,
                  'LATENT_DIM': 64,
                  'HIDDEN_DIM': 256,
                  'READOUT_TRAIN_EPOCHS': 5,
                  'READOUT_TRAIN_STEPS': 50,
                  'CONTEXT_TEMP_SELECTION_GRID': [0.35, 0.6, 0.8, 1.0, 1.25, 1.5],
                  'CONTEXT_BLEND_SELECTION_GRID': [['base_proto_first', 0.05, 0.75, 0.2],
                                                    ['latent_only', 0.0, 1.0, 0.0]],

```

```

['feature_only', 0.0, 0.0, 1.0],
['latent_dominant', 0.05, 0.9, 0.05],
['single_mid', 0.3, 0.55, 0.15],
['balanced_all', 0.34, 0.33, 0.33]],
'SELECTED_CONTEXT_HEAD_VARIANTS': ['proto_global_no_repair',
'proto_kl_only_true_noop',
'proto_global_head_ce_kl_guard_035',
'oracle_global_no_repair_reference',
'oracle_global_head_ce_kl_guard_035_reference',
'context_blend_selected_global',
'context_temp_blend_selected_global',
'context_blend_selected_head_guard_035',
'context_temp_blend_selected_head_guard_035',
'context_gap_selected',
'generic_weak_boundary_latent_calibrated'],
'SELECTED_BASELINE_METHODS': ['finetune',
'ewc',
'lfw',
'experience_replay',
'balanced_replay',
'sparse_moe',
'pnn',
'joint_upper_bound'],
'RC2B_CANDIDATE_VARIANTS': ['proto_global_no_repair',
'context_blend_selected_global',
'context_temp_blend_selected_global',
'proto_global_head_ce_kl_guard_035',
'context_temp_blend_selected_head_guard_035',
'context_gap_selected',
'generic_weak_boundary_latent_calibrated'],
'RC2B_METHOD': 'paired_rc2m_beta_dynamic_boundary_ambiguity_lock_policy',
'RC3_METHOD': 'paired_rc2m_beta_dynamic_boundary_ambiguity_lock_policy',
'RC3_HYDRO_AUDIT_ENABLED': True,
'RC3_HYDRO_POLICY_MODE': 'audit_only',
'RC3_HYDRO_SELECTION_TURBULENCE_WEIGHT': 0.0,
'RC3_HYDRO_SELECTION_LATENT_DRIFT_WEIGHT': 0.0,
'RC2B_STRICT_NO_FINAL_TEST_SELECTION': True,
'NOTEBOOK_FILENAME': 'MMALS_Prototype_1_1_RC2jh_Generic_Weak_Boundary_Latent_C
alibration_RotatedMNIST_Robust_LSA_PANEL_E_STORY_UPDATED_Energy_Context_Probe.ip
ynb',
'ROTATION_ANGLES': [-30, -15, 0, 15, 30],
'PERMUTATION_SEEDS': [100, 101, 102, 103, 104],
'ENABLE_BIOMETRIC_STYLE_THRESHOLD_DIAGNOSTICS': True,
'THRESHOLD_DIAGNOSTICS_FINAL_CHECKPOINT_ONLY': True,
'THRESHOLD_DIAGNOSTICS_EVALUATE_CANDIDATES': True,
'THRESHOLD_DIAGNOSTICS_FOCUS_CLASSES': [2, 4, 5],
'THRESHOLD_DIAGNOSTICS_GRID_SIZE': 501,

```

```

'THRESHOLD_DIAGNOSTICS_SCORE_KIND': 'softmax_probability_over_seen_classes',
'THRESHOLD_DIAGNOSTICS_STRICT_EXPORT': True,
'ENABLE_LEARNING_SIGNAL_ALIGNMENT_AUDIT': True,
'LSA_BATCH_STRIDE': 1,
'LSA_EXPECTED_ROUTE_WEIGHT': 0.55,
'LSA_EXPECTED_GAIN_WEIGHT': 0.25,
'LSA_EXPECTED_MEMORY_RISK_WEIGHT': 0.2,
'ENABLE_ROTATEDMNIST_FINAL_ROTATION_DIAGNOSTIC': True,
'ROTATED_DIAG_FINAL_TASK_ID': 4,
'ROTATED_DIAG_CLASS5_FLOOR': 0.9,
'ROTATED_DIAG_MIN_TASK_FLOOR': 0.8,
'ROTATED_DIAG_CONTEXT_TOP1_FLOOR': 0.7,
'ROTATED_DIAG_CONTEXT_ENTROPY_CEILING': 1.2,
'ROTATED_DIAG_CONTEXT_MARGIN_FLOOR': 0.05,
'ROTATED_DIAG_CLASS5_TO_CLASS4_CEILING': 0.25,
'ROTATED_DIAG_PROBE_CLASS5_FLOOR': 0.85,
'ROTATED_DIAG_DEPLOYED_TO_PROBE_CLASS5_GAP': 0.12,
'ROTATED_DIAG_ORACLE_PROTO_CLASS5_GAP': 0.08,
'ENABLE_RC2JG_GENERIC_WEAK_BOUNDARY_LATENT_CALIBRATION': True,
'RC2JG_VARIANT': 'generic_weak_boundary_latent_calibrated',
'RC2JG_WEAK_CLASS_ACC_FLOOR': 0.86,
'RC2JG_MIN_BOUNDARY_MARGIN': 0.2,
'RC2JG_LATENT_SEPARATION_FLOOR': 0.75,
'RC2JG_MAX_GLOBAL_ACC_DAMAGE': 0.003,
'RC2JG_MIN_WEAK_CLASS_GAIN': 0.01,
'RC2JG_REPAIR_EPOCHS': 5,
'RC2JG_REPAIR_STEPS': 50},
'persistence_root': '/content/drive/MyDrive/MMALS/v1_1_rc2m_alpha_boundary_firs
t_frozen_selector/RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT',
'results_dir': '/content/drive/MyDrive/MMALS/v1_1_rc2m_alpha_boundary_first_fro
zen_selector/RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT/MMALS_v11_RC2
M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT_FashionMNIST_robust_20260601T
164013Z_seeds5',
'drive_available': True,
'artifacts': [{'path': 'audit/context_selection_trace_robust.csv',
'size_bytes': 736131,
'sha256':
'6635d4613975435ca9d153c0672eea2901efd8998bc3da872475a74b46ef6931'},
{'path': 'audit/head_repair_trace_robust.csv',
'size_bytes': 226089,
'sha256':
'9c963e7b1b44b12158b0b8eda53f67b5c4b1e173cc4dce6423b1a603afd7eef3'},
{'path': 'audit/memory_audit_robust.csv',
'size_bytes': 12856,
'sha256':
'ba99396d27b83bb6b28a504306aafbc23ac7bbb56d44be5b18049e56a2257810'},
{'path': 'audit/probe_gap_audit_robust.csv',

```

```

    'size_bytes': 326,
    'sha256':
'3b571919092d6cdfd55805772efe2bcd5431197182f17735fc61dd4e4afb0083'},
    {'path': 'audit/rc1b_vs_true_guard_seed_audit_robust.csv',
     'size_bytes': 1559,
     'sha256':
'e75cff1daec6b93f03bc84571d6ffe62584514c9c86a29fe01e0dad333129713'},
    {'path': 'audit/rc2d_hydro_policy_selection_audit_robust.csv',
     'size_bytes': 3558,
     'sha256':
'34d7e081c91b5a421dff0daf11949ce0bdfc36c9e76ee84975bec399cb4258d7'},
    {'path': 'audit/seed_diagnostics_robust.csv',
     'size_bytes': 17055,
     'sha256':
'601549c2e5b349bd728526e77b0788132d379d801584aae4db0ef182b7d9698e'},
    {'path': 'audit/weak_seed_report_robust.json',
     'size_bytes': 3599,
     'sha256':
'7c554e0a8fd99beb591cbb9037cc85663ee0f99e90981098d90eeb2ddb4ee228'},
    {'path': 'benchmark_gate_robust.csv',
     'size_bytes': 394,
     'sha256':
'4895a31631117d2bd0cd38b821143de7b3d8649d46fa2d3f148044a5b0d78214'},
    {'path': 'benchmark_leaderboard_robust.csv',
     'size_bytes': 8105,
     'sha256':
'6888b3728b009348e812c4d41548b4ea95feb99c5e898c740a916074d325a609'},
    {'path': 'benchmark_method_groups_robust.csv',
     'size_bytes': 3554,
     'sha256':
'e101a96f348d0257bda359a15bd60e802056b7031f69baca4c13d1a945834d0f'},
    {'path': 'benchmark_role_summary_robust.csv',
     'size_bytes': 173,
     'sha256':
'2b456cccb1ab28149d8ce921ba85ba477dbe216a6c2d1322658872d6064f8872'},
    {'path': 'biometric_style_score_dump_robust.csv',
     'size_bytes': 80703416,
     'sha256':
'a652c61ab667ece91597ff74cb842d9c1322a715bfc6db8b1c5500bac42bcc10'},
    {'path': 'class_confusion_robust.csv',
     'size_bytes': 470583,
     'sha256':
'3c27f524bca6c7e1c597bf90357988994bcdf96f9a340a1e0e2ee3ba369794f1'},
    {'path': 'class_diagnostics_robust.csv',
     'size_bytes': 14422,
     'sha256':
'c13ab1e191083b83c74ac762aa98d08852b9375f36a54c9c29e974a3e49e7502'},

```

```

    {'path': 'context_gap_repair_logs_robust.csv',
     'size_bytes': 2068937,
     'sha256':
'df5dbe4be8dd47610a53a34d0add7fd15ed2f42c2ed8facdc77ce308653718f1'},
    {'path':
'energy_context_probe_seed0_robust/energy_probe_all_restarts_trace.csv',
     'size_bytes': 24068580,
     'sha256':
'9b4c0fa07bee4f9a6e813cd8a7d2db169fce7a2a766ea3912833c172c8191ba2'},
    {'path': 'energy_context_probe_seed0_robust/energy_probe_best_trace.csv',
     'size_bytes': 3281622,
     'sha256':
'b0a79fb16026b6be7fed93e94b05500e53ba97f7dbe080a49fbdfa806eaf6e89'},
    {'path': 'energy_context_probe_seed0_robust/energy_probe_sample_summary.csv',
     'size_bytes': 8053,
     'sha256':
'4f5224d3837a6ca15897f1e165e2ddf7af6fda8ada005c59725bd8f0e2c73304'},
    {'path': 'energy_context_probe_seed0_robust/energy_probe_seed_aggregate.csv',
     'size_bytes': 392,
     'sha256':
'd7c901374bbd5c02526e189218cbfa63a6c9176e6236ac1b58f6d69562e9c233'},
    {'path': 'energy_context_probe_seed0_robust/energy_probe_summary.json',
     'size_bytes': 3483,
     'sha256':
'16949bb21f56564566309eee0c4deea68a4c070dd4097dbb53bd66793e35fa42'},
    {'path':
'energy_context_probe_seed0_robust/energy_probe_trace_first_sample.png',
     'size_bytes': 63447,
     'sha256':
'7521ea7b915aa6fcb2b2a512685b381f664a965f1cdf5cd5f9a48e18c4e169ca'},
    {'path': 'energy_context_probe_seed0_robust/energy_theta_train_loss.csv',
     'size_bytes': 292,
     'sha256':
'2d579fb4dd03528fb74225399d174cebec2201f767fb40638c2aa1eee3fd3499'},
    {'path':
'energy_context_probe_seed1_robust/energy_probe_all_restarts_trace.csv',
     'size_bytes': 24001159,
     'sha256':
'fe731acc64b7d5a75ad24326b68af5848aaec7070cf4633e95de6c5bc437cffd'},
    {'path': 'energy_context_probe_seed1_robust/energy_probe_best_trace.csv',
     'size_bytes': 3271964,
     'sha256':
'989b0bd7592f182fd746f29158f839c19ea2dbf3560f2e5cfd56ab385281fd36'},
    {'path': 'energy_context_probe_seed1_robust/energy_probe_sample_summary.csv',
     'size_bytes': 8053,
     'sha256':
'202a766c662430b0601ecd32f26364d71866f2d8ce8a06259e41361ef44029d1'},

```

```

    {'path': 'energy_context_probe_seed1_robust/energy_probe_seed_aggregate.csv',
      'size_bytes': 391,
      'sha256':
'418d309c6eb7c08a97113a1c4b43c62939b02b6708edf1d0031e32fb64bdba50'},
    {'path': 'energy_context_probe_seed1_robust/energy_probe_summary.json',
      'size_bytes': 3479,
      'sha256':
'893c8ace9517dac855588e66d34107adf365afeab1d007e28422976e892371de'},
    {'path':
'energy_context_probe_seed1_robust/energy_probe_trace_first_sample.png',
      'size_bytes': 62998,
      'sha256':
'26b032adfc866ece42f75959d101e25f0299b964a39b108621b7ee135f0a5607'},
    {'path': 'energy_context_probe_seed1_robust/energy_theta_train_loss.csv',
      'size_bytes': 292,
      'sha256':
'd062c14a24b57e6eac446034a2f1a2b6562e0a27917260be340029c50e140c25'},
    {'path':
'energy_context_probe_seed2_robust/energy_probe_all_restarts_trace.csv',
      'size_bytes': 23998313,
      'sha256':
'65a2d70efd9237458951d38baf6118550f76c06ec589582b8bd7b462101407c5'},
    {'path': 'energy_context_probe_seed2_robust/energy_probe_best_trace.csv',
      'size_bytes': 3270919,
      'sha256':
'668b7f2793d4b760794ddece37df5f53d7288aadd13aac93380b88e7e1d0fd61'},
    {'path': 'energy_context_probe_seed2_robust/energy_probe_sample_summary.csv',
      'size_bytes': 8053,
      'sha256':
'bfb9bd1c76225c7556198466865de3e801d9d1ed5df3c25efe5eb0976f242cc1'},
    {'path': 'energy_context_probe_seed2_robust/energy_probe_seed_aggregate.csv',
      'size_bytes': 394,
      'sha256':
'96086fae5ad463b1a1535436bd530c2c560e40474157f91a43942b333c331829'},
    {'path': 'energy_context_probe_seed2_robust/energy_probe_summary.json',
      'size_bytes': 3482,
      'sha256':
'980e3b60531ad742dd3ac9bba21416654fcd536d3640cd231a601342f5eed70a'},
    {'path':
'energy_context_probe_seed2_robust/energy_probe_trace_first_sample.png',
      'size_bytes': 61883,
      'sha256':
'1bf73fb4bb27bd28232f0aa9e1fad1232888ca526d5481fe61afc23f76667df9'},
    {'path': 'energy_context_probe_seed2_robust/energy_theta_train_loss.csv',
      'size_bytes': 288,
      'sha256':
'74467d1cd26e93ab3e52b320c91d60a7277e60b29f05ce45eec3ebc97be43bf9'},

```

```

    {'path':
'energy_context_probe_seed3_robust/energy_probe_all_restarts_trace.csv',
  'size_bytes': 24126729,
  'sha256':
'a94f961f0f5539364c1e2040a8518bd7548560ead91161194ea61a6e2048530d'},
    {'path': 'energy_context_probe_seed3_robust/energy_probe_best_trace.csv',
  'size_bytes': 3289722,
  'sha256':
'541e3737950eafab797ed1501e6887ff9ae5d3aa677f444e055375ecc2201155'},
    {'path': 'energy_context_probe_seed3_robust/energy_probe_sample_summary.csv',
  'size_bytes': 8074,
  'sha256':
'd83b7d8d9684987503b33551eebd6f6ac066e5ba1af89c120ff77f39758289b7'},
    {'path': 'energy_context_probe_seed3_robust/energy_probe_seed_aggregate.csv',
  'size_bytes': 393,
  'sha256':
'433236b7e69bbe67a38db4ecb337c84228ee0805a0997476535d74798e1e18fa'},
    {'path': 'energy_context_probe_seed3_robust/energy_probe_summary.json',
  'size_bytes': 3475,
  'sha256':
'c87ca84343281946c448ed7b0278c1e436679a635642cea5c03b5be3fd0bc6e4'},
    {'path':
'energy_context_probe_seed3_robust/energy_probe_trace_first_sample.png',
  'size_bytes': 64402,
  'sha256':
'ec1c7ac12717cdede5886336a6415a34e6f37272a5d1839e232881222dc2d89a'},
    {'path': 'energy_context_probe_seed3_robust/energy_theta_train_loss.csv',
  'size_bytes': 290,
  'sha256':
'c6b4fd848bf9920c49dc88a373625040190937033bf8e9138e6e1136afd450d8'},
    {'path':
'energy_context_probe_seed4_robust/energy_probe_all_restarts_trace.csv',
  'size_bytes': 24153553,
  'sha256':
'eb698c6551a54e3ad581ec34b39fdc84ed9b6e9defdd21f3cfe3e958ae5ffb41'},
    {'path': 'energy_context_probe_seed4_robust/energy_probe_best_trace.csv',
  'size_bytes': 3293511,
  'sha256':
'57b4c24df65959614dbd5378ed1daf4e1b2b63710bf40d0ad074211ca688d692'},
    {'path': 'energy_context_probe_seed4_robust/energy_probe_sample_summary.csv',
  'size_bytes': 8042,
  'sha256':
'2fb0e18e497b698f4fae8a5891f3257b4068f72f8a6cfb4521f7887b9c915239'},
    {'path': 'energy_context_probe_seed4_robust/energy_probe_seed_aggregate.csv',
  'size_bytes': 390,
  'sha256':
'9b345f48e95fad6356d9f51bc0f826014c67faded8e01b7157f89cc2f991197d'},

```

```

    {'path': 'energy_context_probe_seed4_robust/energy_probe_summary.json',
      'size_bytes': 3474,
      'sha256':
'b4a5d75d8df04a94d971e1080b18936af5f6c614722cef5cd816d53d88453056'},
    {'path':
'energy_context_probe_seed4_robust/energy_probe_trace_first_sample.png',
      'size_bytes': 56907,
      'sha256':
'918ffa27ea319e08178d315bd9d670209e75ecef68670177e2e1eef33d7d91d9'},
    {'path': 'energy_context_probe_seed4_robust/energy_theta_train_loss.csv',
      'size_bytes': 294,
      'sha256':
'418594f4309cb6b950544bda13721dee5eb84a40290c28529fa22174c5e8b5f6'},
    {'path': 'forgetting_robust.csv',
      'size_bytes': 18625,
      'sha256':
'6aa74d54758dd4b4753cee4db54747e0e9390be99079b617e2932740918cb1cd'},
    {'path': 'history_robust.csv',
      'size_bytes': 536657,
      'sha256':
'c2bac91641323fc4e4f22a0d4acdfccff1d99e02f82b6d770ecbc33139ecda0b'},
    {'path': 'learning_signal_alignment_panel_e_robust.csv',
      'size_bytes': 5894024,
      'sha256':
'5390210052619cd26243808765be70ff89c7ae38ed37b6db6da97670106fee8b'},
    {'path': 'learning_signal_alignment_panel_e_summary_robust.csv',
      'size_bytes': 20218,
      'sha256':
'df4e8d1cbcd393ee9b393e390d3ef71ae5cc96fc481e0ac4bb8d896895a300de'},
    {'path': 'losses_robust.csv',
      'size_bytes': 182594,
      'sha256':
'5a2b8ad4593b4b78460c94a071c0f6aeae630e31f9943eb886170407958b499c'},
    {'path': 'notebook_exports/MMALS_Prototype_1_1_RC2jh_Generic_Weak_Boundary_Lat
ent_Calibration_RotatedMNIST_Robust_LSA_PANEL_E_STORY_UPDATED_Energy_Context_Pro
be.ipynb',
      'size_bytes': 4291157,
      'sha256':
'e314ed9d0f28ca10c6a784f48d1a67035d544141240e52a4c772f5b3d2ca5c9a'},
    {'path': 'notebook_exports/notebook_pdf_export_status.json',
      'size_bytes': 1137,
      'sha256':
'9f88c32b2723a144d4b54324c431b001aefbb1c34413ab6e00c8d64cf3a164ab'},
    {'path': 'notebook_exports/run_summary.pdf',
      'size_bytes': 1086546,
      'sha256':
'd76589ffb399bc66ad09b432bfcddffd106dd0465cb0b9b07a23bb959db6bf64'},

```



```

    {'path': 'paired_context_deltas_robust.csv',
     'size_bytes': 1345,
     'sha256':
'b82f3272b9652c96670b2d777935a4a00051b02c3fb230c40a14b1077a4cc251'},
    {'path': 'per_seed_scorecard_robust.csv',
     'size_bytes': 48763,
     'sha256':
'9d10da5c9adcbc27670a53b65bc52db03d528e2eb46164284004426b4340e58c'},
    {'path':
'plots/learning_signal_alignment/panel_e_lsa_by_task_epoch_robust.png',
     'size_bytes': 66391,
     'sha256':
'0a67b4bb2b7ad7c7cf5b168a808b8784f50ef106c5a6b7996974ea02293e0b86'},
    {'path': 'plots/threshold_diagnostics/det_style_class2_robust.png',
     'size_bytes': 78099,
     'sha256':
'c6ffd474437e4d2df0d0f91b6e4601c09d2ccfcdcabbbd3d68820a5de2c6eabc'},
    {'path': 'plots/threshold_diagnostics/det_style_class4_robust.png',
     'size_bytes': 77994,
     'sha256':
'318bb33c0cf5d4344e351e52f2be370695baecf043967227e3751ab48c823946'},
    {'path': 'plots/threshold_diagnostics/det_style_class5_robust.png',
     'size_bytes': 47893,
     'sha256':
'253d5c1925f5386c074ff950127c0a0ba2ee30d1629e3e0b41e7550149a12b82'},
    {'path': 'plots/threshold_diagnostics/eer_by_method_robust.png',
     'size_bytes': 76000,
     'sha256':
'c54e71e53b8f6e72294b70603691efcc904d4d746cd786f92669f54e819af725'},
    {'path': 'plots/threshold_diagnostics/far_frr_class2_robust.png',
     'size_bytes': 138456,
     'sha256':
'a616ef2c4b285e9d3be51814d24e7d13e8d104424930408f73b8ea1bbbfd23f8'},
    {'path': 'plots/threshold_diagnostics/far_frr_class4_robust.png',
     'size_bytes': 142465,
     'sha256':
'302de6264a3f2665eabb69b95104e49f6c8d19fee9292228f7cf764a6dd8d4ec'},
    {'path': 'plots/threshold_diagnostics/far_frr_class5_robust.png',
     'size_bytes': 92460,
     'sha256':
'97984a4efaff4f785186c430fc4708918e1f29c3ea9510d3bc733cc994343e22'},
    {'path': 'plots/threshold_diagnostics/roc_style_class2_robust.png',
     'size_bytes': 76556,
     'sha256':
'6082407f6761690b979fb0fda3229b51fd9a25ca46ede4f5fa40e16cd9aa0d16'},
    {'path': 'plots/threshold_diagnostics/roc_style_class4_robust.png',
     'size_bytes': 76776,

```

```

    'sha256':
'38a17d2dcddfa57e39940ab074164f0d0140e025358580f9b321abab7750dc89'},
  {'path': 'plots/threshold_diagnostics/roc_style_class5_robust.png',
   'size_bytes': 48582,
   'sha256':
'e5f12ad476ef0c990be808364213da2110b21d9572fbedd138790f07c1ca3e36'},
  {'path': 'probe_losses_robust.csv',
   'size_bytes': 91599,
   'sha256':
'bff1d4ec0f9677f5bc513239cbeebdc8c26439b0e6c0b662c88063a33b9350de'},
  {'path': 'raw_logs/console_robust.log',
   'size_bytes': 122537,
   'sha256':
'bfe61a0ac601e179402eb4aa6aa825e75fbb6fec46b33621d16ff33f2fa77cf'},
  {'path': 'rc1b_per_seed_vs_true_guard_robust.csv',
   'size_bytes': 1161,
   'sha256':
'ef4eb46d99aca0e373ee6481ead08f60efb39400fd50fa10879d5552e49e6823'},
  {'path': 'rc1b_validation_gate_robust.csv',
   'size_bytes': 1146,
   'sha256':
'710b0a5c71e09b36c96b233b25cc1822a9f4e41b199a90e2084a117e691c637b'},
  {'path': 'rc2b_policy_selection_trace_robust.csv',
   'size_bytes': 561372,
   'sha256':
'e5996c1a2868efe215478442d76db385c5c9dfa564972f34b4821b7b8a99be25'},
  {'path': 'scorecard_robust.csv',
   'size_bytes': 10712,
   'sha256':
'9f4c4c2374ee60fa86aea2bd688255682122685011b8a489873e993e30261f16'},
  {'path': 'threshold_curves/biometric_style_eer_by_class_robust.csv',
   'size_bytes': 70041,
   'sha256':
'0ccd89b67013805120c6cf8e53ca7c9624ca9d8d2d11a982d5753889adb239a2'},
  {'path': 'threshold_curves/biometric_style_eer_by_method_robust.csv',
   'size_bytes': 18865,
   'sha256':
'01a729f5f6331efecc51f0bdaa16377bec78aea5b0162b08208c4b69a932cc95'},
  {'path': 'threshold_curves/biometric_style_far_frr_roc_det_curves_robust.csv',
   'size_bytes': 28703536,
   'sha256':
'9b80cbe0ba7ea2170cd9d7eeb5f661c48a2445c30b6170f4274b4a55b3bd6937'},
  {'path': 'traces/otel_style_events_robust.jsonl',
   'size_bytes': 180944,
   'sha256':
'c61bccf8b761b8592ec91edc15f7d2000b85c1eebf9364327d6fcadff8373371'},
  {'path': 'traces/otel_style_spans_robust.csv',

```

```

'size_bytes': 23028,
'sha256':
'cee061ab5a86d8deb2d72c9c5b29897632b5b7873253054ec3a8fa95bfef2fef'}]],
'extra': {'notebook_pdf_evidence': '/content/drive/MyDrive/MMALS/v1_1_rc2m_alpha_boundary_first_frozen_selector/RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT/MMALS_v11_RC2M_BETA_DYNAMIC_BOUNDARY_AMBIGUITY_LOCK_HYDRO_AUDIT_FashionMNIST_robust_20260601T164013Z_seeds5/notebook_exports'}}

```

2.43 15.3 RC2d gate-aligned no-leakage policy-selection gate

This gate evaluates the selected policy row `paired_rc2d_gate_aligned_policy`.

It checks both scientific performance and the strict process claim: policy selection must be performed from validation memory before final-test evaluation.

```

[ ]: # -----
# v1.1-RC2Je clean-context final-lock-veto strict policy-selection gate.
# -----
RC2C_PRIMARY = RC2B_METHOD
RC3_PRIMARY = RC2C_PRIMARY
RC2B_PRIMARY = RC3_PRIMARY
RC1B_PRIMARY = "paired_context_gap_selected"
TRUE_GUARD_PRIMARY = "paired_proto_global_head_ce_kl_guard_035"

def _score_row(method):
    return get_row(scorecard_df, method)

rc2b_row = _score_row(RC2B_PRIMARY)
rc1b_row = _score_row(RC1B_PRIMARY)
balanced_replay_row = _score_row("baseline_balanced_replay")
experience_replay_row = _score_row("baseline_experience_replay")

replay_candidates = [r for r in [balanced_replay_row, experience_replay_row] if
    ↪r is not None]
best_replay_row = max(replay_candidates, key=lambda r: metric(r,
    ↪"final_avg_acc_mean")) if replay_candidates else None

non_diag = non_diagnostic_scorecard(scorecard_df)
mmals_deployable = non_diag[non_diag.method.str.startswith("paired_",
    ↪na=False)] if not non_diag.empty else pd.DataFrame()
# Best deployable MMALS row, but exclude oracle references and probes by
    ↪construction.
best_mmals_row = mmals_deployable.sort_values("final_avg_acc_mean",
    ↪ascending=False).iloc[0] if not mmals_deployable.empty else None

selection_expected_rows = len(SEEDS) * N_TASKS * len(RC2B_CANDIDATE_VARIANTS)

```

```

selected_rows = selection_log_df[selection_log_df.get("is_selected_candidate",
↳pd.Series(dtype=bool)).astype(bool)] if "selection_log_df" in globals() and
↳not selection_log_df.empty else pd.DataFrame()
selected_per_checkpoint = selected_rows.groupby(["seed", "after_task"]).size().
↳reset_index(name="n_selected") if not selected_rows.empty else pd.DataFrame()

rc2b_gate_rows = []
def add_rc2b_gate(name, passed, detail):
    rc2b_gate_rows.append({"check": name, "passed": bool(passed), "detail":
↳str(detail)})

add_rc2b_gate(
    "policy_level_validation_proxy_available_all_candidates",
    (not selection_log_df.empty) and len(selection_log_df) >=
↳selection_expected_rows and selection_log_df["proxy_acc"].notna().all(),
    f"selection_rows={len(selection_log_df) if 'selection_log_df' in globals()
↳else 0}; expected>={selection_expected_rows}",
)
add_rc2b_gate(
    "exactly_one_policy_selected_per_seed_task",
    (not selected_per_checkpoint.empty) and
↳bool((selected_per_checkpoint["n_selected"] == 1).all()) and
↳len(selected_per_checkpoint) == len(SEEDS) * N_TASKS,
    f"selected_checkpoints={len(selected_per_checkpoint)}; expected={len(SEEDS)
↳* N_TASKS}",
)
add_rc2b_gate(
    "no_final_test_used_for_selection_declared",
    (not selection_log_df.empty) and
↳bool(selection_log_df["no_final_test_used_for_selection"].astype(bool).
↳all()),
    "selection trace marks no_final_test_used_for_selection=True for every
↳candidate row.",
)
if rc2b_row is not None and rc1b_row is not None:
    add_rc2b_gate(
        "rc2b_at_least_fixed_rc1b_with_tolerance",
        metric(rc2b_row, "final_avg_acc_mean") >= metric(rc1b_row,
↳"final_avg_acc_mean") - RC2B_SELECTION_TIE_TOL,
        f"RC2b={metric(rc2b_row, 'final_avg_acc_mean'):.4f};
↳RC1b={metric(rc1b_row, 'final_avg_acc_mean'):.4f}; delta={metric(rc2b_row,
↳'final_avg_acc_mean') - metric(rc1b_row, 'final_avg_acc_mean'):.4f}",
    )
if rc2b_row is not None and best_replay_row is not None:
    add_rc2b_gate(
        "rc2b_beats_best_replay_margin",

```

```

        metric(rc2b_row, "final_avg_acc_mean") >= metric(best_replay_row,
↪ "final_avg_acc_mean") + 0.005,
        f"RC2b={metric(rc2b_row, 'final_avg_acc_mean'):.4f}";
↪ best_replay={metric(best_replay_row, 'final_avg_acc_mean'):.4f}",
    )
if rc2b_row is not None and best_mmals_row is not None:
    add_rc2b_gate(
        "rc2b_within_tolerance_of_best_deployable_mmals",
        metric(rc2b_row, "final_avg_acc_mean") >= metric(best_mmals_row,
↪ "final_avg_acc_mean") - max(0.005, RC2B_SELECTION_TIE_TOL),
        f"RC2b={metric(rc2b_row, 'final_avg_acc_mean'):.4f}";
↪ best_mmals={metric(best_mmals_row, 'final_avg_acc_mean'):.4f}";
↪ best={best_mmals_row.method}",
    )
if rc2b_row is not None:
    add_rc2b_gate("rc2b_min_task_floor", metric(rc2b_row, "min_task_acc_mean")
↪ >= 0.80, f"min_task={metric(rc2b_row, 'min_task_acc_mean'):.4f}")
    add_rc2b_gate("rc2b_class2_floor", metric(rc2b_row, "class2_acc_mean") >= 0.
↪ 80, f"class2={metric(rc2b_row, 'class2_acc_mean'):.4f}")
    add_rc2b_gate("rc2b_class4_boundary_floor", metric(rc2b_row,
↪ "class4_acc_mean") >= 0.20, f"class4={metric(rc2b_row, 'class4_acc_mean'):.
↪ 4f}")
    add_rc2b_gate("rc2b_class5_floor", metric(rc2b_row, "class5_acc_mean") >= 0.
↪ 90, f"class5={metric(rc2b_row, 'class5_acc_mean'):.4f}")
    add_rc2b_gate("rc2b_forgetting_controlled", metric(rc2b_row,
↪ "avg_forgetting_mean") <= 0.08, f"forgetting={metric(rc2b_row,
↪ 'avg_forgetting_mean'):.4f}")

# RC2c-specific risk-aware selector audit gates.
if RC2C_RISK_AWARE_SELECTOR_ENABLED:
    add_rc2b_gate(
        "rc2c_risk_aware_selector_enabled",
        True,
        "RC2c risk-aware validation selector is enabled for this profile.",
    )
    if "selection_log_df" in globals() and not selection_log_df.empty:
        required_rc2c_cols = [
            "proxy_min_task",
            "proxy_old_task_min",
            "proxy_task_acc_std",
            "selection_reason",
            "raw_best_variant",
            "best_guarded_variant",
        ]

```

```

        present_rc2c_cols = [c for c in required_rc2c_cols if c in
↪selection_log_df.columns]
        add_rc2b_gate(
            "rc2c_selection_audit_columns_present",
            len(present_rc2c_cols) == len(required_rc2c_cols),
            f"present={present_rc2c_cols}; required={required_rc2c_cols}",
        )
        if "selection_reason" in selection_log_df.columns:
            reason_counts = selection_log_df.loc[selection_log_df.
↪get("is_selected_candidate", False).astype(bool), "selection_reason"].
↪value_counts().to_dict()
            add_rc2b_gate(
                "rc2c_selection_reason_recorded",
                len(reason_counts) > 0,
                f"selected_reason_counts={reason_counts}",
            )

# RC2c / Hydro audit gates.
if RC3_HYDRO_AUDIT_ENABLED:
    hydro_cols_required = ["hydro_latent_drift_Dz", "hydro_turbulence_score",
↪"hydro_audit_memory_only"]
    hydro_cols_present = [c for c in hydro_cols_required if ("selection_log_df"
↪in globals() and not selection_log_df.empty and c in selection_log_df.
↪columns)]
    add_rc2b_gate(
        "rc3b_hydro_audit_metrics_present",
        len(hydro_cols_present) == len(hydro_cols_required),
        f"present={hydro_cols_present}; required={hydro_cols_required}",
    )
    if "selection_log_df" in globals() and not selection_log_df.empty and
↪"hydro_audit_memory_only" in selection_log_df.columns:
        add_rc2b_gate(
            "rc3b_hydro_audit_uses_validation_memory_only",
            bool(selection_log_df["hydro_audit_memory_only"].fillna(False).
↪astype(bool).any()),
            "Hydro audit rows are marked memory-only; final test is not used
↪for selection/audit metrics.",
        )
        if "selection_log_df" in globals() and not selection_log_df.empty and
↪"hydro_turbulence_score" in selection_log_df.columns:
            selected_hydro = selected_rows if "selected_rows" in globals() and not
↪selected_rows.empty else pd.DataFrame()
            selected_turbulence = float(selected_hydro["hydro_turbulence_score"].
↪mean()) if not selected_hydro.empty and "hydro_turbulence_score" in
↪selected_hydro else np.nan
            add_rc2b_gate(

```

```

        "rc3b_selected_policy_hydro_turbulence_reported",
        not np.isnan(selected_turbulence),
        f"selected_policy_mean_hydro_turbulence={selected_turbulence}",
    )
else:
    add_rc2b_gate(
        "rc3b_hydro_audit_disabled_by_profile",
        True,
        f"EXPERIMENT_PROFILE={EXPERIMENT_PROFILE}; Hydro audit intentionally_
↳disabled.",
    )

# RC2I-specific anchor/selector/final-alignment gates.
if rc2b_row is not None and best_mmals_row is not None:
    add_rc2b_gate(
        "rc2i_selector_final_alignment_to_best_deployable",
        metric(rc2b_row, "final_avg_acc_mean") >= metric(best_mmals_row,
↳"final_avg_acc_mean") - RC2D_SELECTOR_FINAL_ALIGNMENT_TOL,
        f"RC2I={metric(rc2b_row, 'final_avg_acc_mean'):.4f};
↳best_deployable={metric(best_mmals_row, 'final_avg_acc_mean'):.4f};
↳tol={RC2D_SELECTOR_FINAL_ALIGNMENT_TOL}",
    )
if rc2b_row is not None:
    add_rc2b_gate(
        "rc2i_class2_robustness_floor",
        metric(rc2b_row, "class2_acc_mean") >= 0.80,
        f"class2={metric(rc2b_row, 'class2_acc_mean'):.4f}; target>=0.80",
    )
    add_rc2b_gate(
        "rc2i_class5_preservation_floor",
        metric(rc2b_row, "class5_acc_mean") >= 0.90,
        f"class5={metric(rc2b_row, 'class5_acc_mean'):.4f}; target>=0.90",
    )

    tri_vals = [metric(rc2b_row, "class2_acc_mean"), metric(rc2b_row,
↳"class4_acc_mean"), metric(rc2b_row, "class5_acc_mean")]
    tri_vals = [v for v in tri_vals if not np.isnan(v)]
    tri_min = float(np.min(tri_vals)) if tri_vals else np.nan
    tri_gap = float(np.max(tri_vals) - np.min(tri_vals)) if tri_vals else np.nan
    tri_target = 0.45 if RUN_MODE == "smoke" else 0.75
    add_rc2b_gate(
        "rc2i_tri_boundary_min_floor",
        (not np.isnan(tri_min)) and tri_min >= tri_target,
        f"tri_min={tri_min:.4f}; tri_gap={tri_gap:.4f}; target>={tri_target:.
↳2f}; classes=(2,4,5)",
    )

```

```

    add_rc2b_gate(
        "rc2i_tri_boundary_balance_gap",
        (not np.isnan(tri_gap)) and tri_gap <= (0.55 if RUN_MODE == "smoke"
↪else 0.30),
        f"tri_gap={tri_gap:.4f}; target<= {(0.55 if RUN_MODE == 'smoke' else 0.
↪30):.2f}",
    )
if not selection_log_df.empty:
    add_rc2b_gate(
        "rc2i_selection_reason_audit_present",
        "selection_reason" in selection_log_df.columns and
↪selection_log_df["selection_reason"].notna().any(),
        "selection_reason is present in policy-selection trace.",
    )

    rc2i_cols = ["rc2i_safe_anchor_variant", "rc2i_override_rejected_reason",
↪"rc2i_context_only_override_rejected"]
    rc2i_present = [c for c in rc2i_cols if c in selection_log_df.columns]
    add_rc2b_gate(
        "rc2i_dual_anchor_audit_columns_present",
        len(rc2i_present) == len(rc2i_cols),
        f"present={rc2i_present}; required={rc2i_cols}",
    )
    if "rc2i_context_only_override_rejected" in selection_log_df.columns:
        add_rc2b_gate(
            "rc2i_context_only_overrides_rejected_when_applicable",
            True,
            "Context-only global policies are audited; RC2I rejects them as
↪deployable overrides by default.",
        )

# RC2J/RC2Je-specific regime-aware selector audit gates.
if globals().get("POLICY_BRANCH") == "RC2j":
    add_rc2b_gate(
        "rc2je_hard_clarity_clean_veto_guard_selector_enabled",
        True,
        "RC2Je regime-aware validation selector with hard-clarity clean-veto
↪guard is enabled for this profile.",
    )
    if "selection_log_df" in globals() and not selection_log_df.empty:
        required_rc2j_cols = [
            "rc2j_regime_label",
            "rc2j_regime_reason",
            "rc2j_context_gain_vs_proto",

```



```

        "rc2j_proto_context_top1",
        "rc2j_proto_context_entropy",
        "rc2j_proto_tri_min",
        "rc2j_context_tri_min",
        "rc2j_selected_policy_family",
    ]
    present_rc2j_cols = [col for col in required_rc2j_cols if col in
↪selection_log_df.columns]
    add_rc2b_gate(
        "rc2j_regime_audit_columns_present",
        len(present_rc2j_cols) == len(required_rc2j_cols),
        f"present={present_rc2j_cols}; required={required_rc2j_cols}",
    )
    if "rc2j_regime_label" in selection_log_df.columns:
        selected_policy_rows = selection_log_df[selection_log_df.
↪get("is_selected_candidate", False).astype(bool)]
        regime_counts = selected_policy_rows["rc2j_regime_label"].
↪value_counts().to_dict() if not selected_policy_rows.empty else {}
        add_rc2b_gate(
            "rc2j_regime_decision_recorded",
            len(regime_counts) > 0,
            f"selected_regime_counts={regime_counts}",
        )
    if "rc2j_selected_variant" in selection_log_df.columns:
        selected_policy_rows = selection_log_df[selection_log_df.
↪get("is_selected_candidate", False).astype(bool)]
        variant_counts = selected_policy_rows["rc2j_selected_variant"].
↪value_counts().to_dict() if not selected_policy_rows.empty else {}
        add_rc2b_gate(
            "rc2j_policy_family_decision_recorded",
            len(variant_counts) > 0,
            f"selected_variant_counts={variant_counts}",
        )
    rc2jb_cols = [
        "rc2jb_late_ambiguity_hysteresis_applied",
        "rc2jb_late_ambiguity_reason",
        "rc2jb_late_uncertainty_signals",
        "rc2jb_context_score_margin_vs_raw_best",
        "rc2jb_context_acc_margin_vs_proto",
    ]
    present_rc2jb_cols = [col for col in rc2jb_cols if col in
↪selection_log_df.columns]
    add_rc2b_gate(
        "rc2jb_late_ambiguity_hysteresis_audit_columns_present",
        len(present_rc2jb_cols) == len(rc2jb_cols),
        f"present={present_rc2jb_cols}; required={rc2jb_cols}",
    )

```

```

)
rc2jc_cols = [
    "rc2jc_final_ambiguity_lock_applied",
    "rc2jc_final_ambiguity_reason",
    "rc2jc_final_uncertainty_signals",
    "rc2jc_context_acc_margin_vs_raw_best",
    "rc2jc_context_acc_margin_vs_proto",
    "rc2jc_context_tri_warning",
    "rc2jc_context_tri_hard_fail",
]
present_rc2jc_cols = [col for col in rc2jc_cols if col in ↪
selection_log_df.columns]
add_rc2b_gate(
    "rc2jc_final_checkpoint_ambiguity_lock_audit_columns_present",
    len(present_rc2jc_cols) == len(rc2jc_cols),
    f"present={present_rc2jc_cols}; required={rc2jc_cols}",
)
rc2je_cols = [
    "rc2je_hard_clarity_clean_veto_guard_applied",
    "rc2je_clean_context_veto_reason",
    "rc2je_clean_signal_count",
    "rc2je_proto_acc_margin_vs_context",
    "rc2je_proto_score_margin_vs_raw_best",
    "rc2je_proto_tri_healthy",
    "rc2je_context_gain_veto_ok",
    "rc2je_hard_clarity_clean_veto_applied",
    "rc2je_hard_clarity_veto_reason",
    "rc2je_hard_clarity_signal_count",
    "rc2je_proto_top1_hard_clear",
    "rc2je_proto_entropy_hard_clear",
    "rc2je_gap_surrogate_hard_clear",
    "rc2je_hydro_context_hard_clear",
]
present_rc2je_cols = [col for col in rc2je_cols if col in ↪
selection_log_df.columns]
add_rc2b_gate(
    "rc2je_hard_clarity_clean_veto_guard_audit_columns_present",
    len(present_rc2je_cols) == len(rc2je_cols),
    f"present={present_rc2je_cols}; required={rc2je_cols}",
)

rc2jf_cols = [
    "rc2jf_true_guard_safe_veto_applied",
    "rc2jf_true_guard_safe_veto_reason",
    "rc2jf_true_guard_acc_margin_vs_context",
    "rc2jf_true_guard_score_margin_vs_context",
    "rc2jf_true_guard_class5_margin_vs_context",

```

```

        "rc2jf_true_guard_minclass_margin_vs_context",
        "rc2jf_true_guard_tri_min",
        "rc2jf_true_guard_boundary_improved",
        "rc2jf_true_guard_raw_best",
        "rc2jf_true_guard_score_or_acc_superior",
        "rc2jf_true_guard_boundary_not_worse",
        "rc2jf_true_guard_tri_ok",
        "rc2jf_dataset_agnostic_context_not_below",
        "rc2jf_weak_anchor_class",
        "rc2jf_weak_boundary_margin_vs_context",
        "rc2jf_weak_class_margin_vs_context",
        "rc2jf_min_task_margin_vs_context",
        "rc2jf_hydro_turbulence_delta_vs_context",
        "rc2jf_hydro_output_delta_vs_context",
        "rc2jf_weak_boundary_not_below_context",
        "rc2jf_weak_class_not_below_context",
        "rc2jf_min_task_not_below_context",
        "rc2jf_hydro_turbulence_not_worse_than_context",
        "rc2jf_hydro_output_not_worse_than_context",
    ]
    present_rc2jf_cols = [col for col in rc2jf_cols if col in ↵
↵selection_log_df.columns]
    add_rc2b_gate(
        "rc2jf_true_guard_safe_veto_audit_columns_present",
        len(present_rc2jf_cols) == len(rc2jf_cols),
        f"present={present_rc2jf_cols}; required={rc2jf_cols}",
    )

    rc2jg_cols = [
        "rc2jg_selector_version", "rc2jg_override_considered", ↵
↵"rc2jg_override_applied",
        "rc2jg_override_reason", "rc2jg_candidate_variant", ↵
↵"rc2jg_candidate_score",
        "rc2jg_candidate_acc", "rc2jg_selected_base_variant", ↵
↵"rc2jg_pre_weak_class",
        "rc2jg_pre_competing_class", "rc2jg_pre_weak_class_acc", ↵
↵"rc2jg_post_weak_class_acc",
        "rc2jg_weak_class_gain", "rc2jg_min_class_gain", ↵
↵"rc2jg_global_acc_damage",
        "rc2jg_context_healthy", "rc2jg_latent_weak_boundary_detected",
        "rc2jg_candidate_validation_safe",
    ]
    present_rc2jg_cols = [col for col in rc2jg_cols if col in ↵
↵selection_log_df.columns]

```

```

        add_rc2b_gate(
            ↵
            ↵"rc2jg_generic_weak_boundary_latent_calibration_audit_columns_present",
                (not globals().
            ↵get("ENABLE_RC2JG_GENERIC_WEAK_BOUNDARY_LATENT_CALIBRATION", False)) or ↵
            ↵len(present_rc2jg_cols) == len(rc2jg_cols),
                f"present={present_rc2jg_cols}; required={rc2jg_cols}",
            )

rc2b_gate_df = pd.DataFrame(rc2b_gate_rows)
rc2b_gate_path = RESULTS_DIR / ↵
    ↵f"rc2jg_generic_weak_boundary_latent_calibration_policy_gate_{RUN_MODE}.csv"
rc2b_gate_df.to_csv(rc2b_gate_path, index=False)
print("Saved RC2JG generic weak-boundary latent calibration policy gate:", ↵
    ↵rc2b_gate_path)
display(rc2b_gate_df)

RC2D_GATE_ALIGNED_STRICT_PASSED = bool(rc2b_gate_df["passed"].all()) if not ↵
    ↵rc2b_gate_df.empty else False
RC2B_STRICT_PASSED = RC2D_GATE_ALIGNED_STRICT_PASSED
print("RC3B_CONFIGURABLE_STRICT_PASSED:", RC2D_GATE_ALIGNED_STRICT_PASSED)

if not selected_rows.empty:
    selected_policy_summary_df = selected_rows.groupby([
        "selected_candidate_variant", "selected_candidate_method", ↵
        ↵"selected_policy_family", "selected_policy_label"
    ], as_index=False).agg(
        n_selected=("is_selected_candidate", "sum"),
        mean_selected_proxy_acc=("selected_proxy_acc", "mean"),
        mean_selected_proxy_score=("selected_proxy_score", "mean"),
    ).sort_values("n_selected", ascending=False)
    selected_policy_summary_path = RESULTS_DIR / ↵
    ↵f"rc2b_selected_policy_summary_{RUN_MODE}.csv"
    selected_policy_summary_df.to_csv(selected_policy_summary_path, index=False)
    print("Saved selected policy summary:", selected_policy_summary_path)
    display(selected_policy_summary_df)
else:
    selected_policy_summary_df = pd.DataFrame()

```

	check	passed \
0	policy_level_validation_proxy_available_all_ca...	True
1	exactly_one_policy_selected_per_seed_task	True
2	no_final_test_used_for_selection_declared	True
3	rc2b_at_least_fixed_rc1b_with_tolerance	False
4	rc2b_beats_best_replay_margin	True
5	rc2b_within_tolerance_of_best_deployable_mmals	False
6	rc2b_min_task_floor	False

7	rc2b_class2_floor	False
8	rc2b_class4_boundary_floor	True
9	rc2b_class5_floor	True
10	rc2b_forgetting_controlled	True
11	rc2c_risk_aware_selector_enabled	True
12	rc2c_selection_audit_columns_present	True
13	rc2c_selection_reason_recorded	True
14	rc3b_hydro_audit_metrics_present	True
15	rc3b_hydro_audit_uses_validation_memory_only	True
16	rc3b_selected_policy_hydro_turbulence_reported	True
17	rc2i_selector_final_alignment_to_best_deployable	False
18	rc2i_class2_robustness_floor	False
19	rc2i_class5_preservation_floor	True
20	rc2i_tri_boundary_min_floor	True
21	rc2i_tri_boundary_balance_gap	True
22	rc2i_selection_reason_audit_present	True
23	rc2i_dual_anchor_audit_columns_present	True
24	rc2i_context_only_overrides_rejected_when_appl...	True

```

                                detail
0      selection_rows=175; expected>=175
1      selected_checkpoints=25; expected=25
2  selection trace marks no_final_test_used_for_s...
3      RC2b=0.8856; RC1b=0.9010; delta=-0.0154
4      RC2b=0.8856; best_replay=0.8390
5  RC2b=0.8856; best_mmals=0.9010; best=paired_co...
6      min_task=0.7700
7      class2=0.7735
8      class4=0.8438
9      class5=0.9975
10     forgetting=0.0693
11  RC2c risk-aware validation selector is enabled...
12  present=['proxy_min_task', 'proxy_old_task_min...
13  selected_reason_counts={'rc2m_beta_dynamic_bou...
14  present=['hydro_latent_drift_Dz', 'hydro_turbu...
15  Hydro audit rows are marked memory-only; final...
16  selected_policy_mean_hydro_turbulence=0.138200...
17      RC2I=0.8856; best_deployable=0.9010; tol=0.005
18      class2=0.7735; target>=0.80
19      class5=0.9975; target>=0.90
20  tri_min=0.7735; tri_gap=0.2240; target>=0.75; ...
21      tri_gap=0.2240; target<= 0.30
22  selection_reason is present in policy-selectio...
23  present=['rc2i_safe_anchor_variant', 'rc2i_ove...
24  Context-only global policies are audited; RC2I...

```

```

                                selected_candidate_variant \
1  context_temp_blend_selected_head_guard_035

```

```

3         proto_global_head_ce_kl_guard_035
2     generic_weak_boundary_latent_calibrated
4         proto_global_no_repair
0         context_blend_selected_global

        selected_candidate_method \
1 paired_context_temp_blend_selected_head_guard_035
3     paired_proto_global_head_ce_kl_guard_035
2     paired_generic_weak_boundary_latent_calibrated
4         paired_proto_global_no_repair
0         paired_context_blend_selected_global

        selected_policy_family \
1         rc1b_guarded_context_gap
3         proto_true_global_guard
2 generic_latent_boundary_calibration
4         proto_no_repair
0         context_only_global

        selected_policy_label  n_selected \
1         RC1b guarded context-gap          11
3     proto-first true CE/KL guarded global head      6
2 RC2JG generic weak-boundary latent calibration      4
4     proto-first no-repair / copied global head      3
0     validation-selected context-only global head    1

mean_selected_proxy_acc  mean_selected_proxy_score
1         0.927044          1.612941
3         0.931780          1.502497
2         0.930898          1.744300
4         0.934115          1.251725
0         0.953776          1.673456

```

2.44 RC2jh Energy Context Probe — exports

```

[ ]: # -----
# RC2jh Energy Context Probe - final export and evidence summary
# -----
if ACTIVATE_ENERGY_CONTEXT_PROBE:
    energy_probe_agg_df = pd.concat(all_energy_probe_agg, ignore_index=True) if
    ↪ "all_energy_probe_agg" in globals() and all_energy_probe_agg else pd.
    ↪ DataFrame()
    energy_probe_summary_df = pd.concat(all_energy_probe_summary,
    ↪ ignore_index=True) if "all_energy_probe_summary" in globals() and
    ↪ all_energy_probe_summary else pd.DataFrame()

```

```

energy_probe_trace_df = pd.concat(all_energy_probe_trace,
↳ ignore_index=True) if "all_energy_probe_trace" in globals() and
↳ all_energy_probe_trace else pd.DataFrame()

energy_probe_agg_path = RESULTS_DIR / f"energy_context_probe_agg_{RUN_MODE}."
↳ csv"

energy_probe_summary_path = RESULTS_DIR /
↳ f"energy_context_probe_samples_{RUN_MODE}.csv"
energy_probe_trace_path = RESULTS_DIR /
↳ f"energy_context_probe_trace_{RUN_MODE}.csv"
energy_probe_json_path = RESULTS_DIR /
↳ f"energy_context_probe_summary_{RUN_MODE}.json"

if not energy_probe_agg_df.empty:
    energy_probe_agg_df.to_csv(energy_probe_agg_path, index=False)
if not energy_probe_summary_df.empty:
    energy_probe_summary_df.to_csv(energy_probe_summary_path, index=False)
if not energy_probe_trace_df.empty:
    energy_probe_trace_df.to_csv(energy_probe_trace_path, index=False)

payload = {
    "energy_probe_version": ENERGY_PROBE_VERSION,
    "audit_only": bool(ENERGY_PROBE_AUDIT_ONLY),
    "run_mode": RUN_MODE,
    "dataset_name": DATASET_NAME,
    "lambda_memory": float(ENERGY_PROBE_LAMBDA_MEMORY),
    "lambda_context": float(ENERGY_PROBE_LAMBDA_CONTEXT),
    "sigma_c": float(ENERGY_PROBE_SIGMA_C),
    "oracle_gap_target": float(ENERGY_PROBE_ORACLE_GAP_TARGET),
    "aggregate_rows": energy_probe_agg_df.to_dict("records") if not
↳ energy_probe_agg_df.empty else [],
    "interpretation": (
        "Audit-only probe. If oracle_hit_rate is high and final context
↳ gaps decrease, the z/m landscape contains recoverable context signal. "
        "If plateau/wrong-context/divergence dominates, the energy
↳ landscape documents why inferred context remains difficult. "
        "The 0.029 target is comparable only if the context embedding scale
↳ matches the reference run."
    ),
}
with open(energy_probe_json_path, "w") as f:
    json.dump(payload, f, indent=2)

print("Energy Context Probe exports:")
print("-", energy_probe_agg_path)
print("-", energy_probe_summary_path)

```

```

    print("-", energy_probe_trace_path)
    print("-", energy_probe_json_path)
else:
    print("Energy Context Probe disabled.")

```

```

[ ]: # Finalize persistent logs, manifest, and ZIP package.
log_event("packaging_started", phase="export", payload={"results_dir":
    ↪str(RESULTS_DIR)})
write_manifest(extra={"packaging_started": True})
stop_persistent_logging()

output_base = RESULTS_DIR.parent / f"{RESULTS_DIR.name}_package"
zip_path = Path(shutil.make_archive(str(output_base), "zip", RESULTS_DIR))
zip_sha = file_sha256(zip_path)

package_sha_path = zip_path.with_name(zip_path.stem + "_sha256.txt")
package_sha_path.write_text(f"{zip_sha} {zip_path.name}\n", encoding="utf-8")

release_manifest_path = zip_path.with_name(zip_path.stem + "_release_manifest.
    ↪json")
release_manifest = {
    "run_id": RUN_ID,
    "trace_id": TRACE_ID,
    "dataset": DATASET_NAME,
    "experiment_profile": EXPERIMENT_PROFILE,
    "policy_branch": POLICY_BRANCH,
    "activate_energy_context_probe": bool(globals().
    ↪get("ACTIVATE_ENERGY_CONTEXT_PROBE", False)),
    "energy_probe_audit_only": bool(globals().get("ENERGY_PROBE_AUDIT_ONLY",
    ↪True)),
    "energy_probe_version": globals().get("ENERGY_PROBE_VERSION", None),
    "activate_hydro_audit": ACTIVATE_HYDRO_AUDIT,
    "activate_hydro_regularizer": ACTIVATE_HYDRO_REGULARIZER,
    "activate_hydro_selection": ACTIVATE_HYDRO_SELECTION,
    "hydro_regularizer_mode": HYDRO_REGULARIZER_MODE,
    "run_mode": RUN_MODE,
    "pair_mode": PAIR_MODE,
    "rotation_angles": ROTATION_ANGLES if DATASET_NAME == "RotatedMNIST" else
    ↪None,
    "permutation_seeds": PERMUTATION_SEEDS if DATASET_NAME == "PermutedMNIST"
    ↪else None,
    "rc2jf_method": RC3_METHOD,
    "rc2f_method": RC3_METHOD,
    "rc2b_compat_method_name": RC2B_METHOD,
    "rc2jf_candidate_variants": RC2B_CANDIDATE_VARIANTS,
    "rc2f_candidate_variants": RC2B_CANDIDATE_VARIANTS,

```



```

    "rc2jf_true_guard_safe_veto_enabled": bool(globals().
↪get("RC2JF_TRUE_GUARD_SAFE_VETO_ENABLED", False)),
    "rc2jg_generic_weak_boundary_latent_calibration_enabled": bool(globals().
↪get("ENABLE_RC2JG_GENERIC_WEAK_BOUNDARY_LATENT_CALIBRATION", False)),
    "rc2jg_variant": globals().get("RC2JG_VARIANT", None),
    "rc3_hydro_audit_enabled": RC3_HYDRO_AUDIT_ENABLED,
    "rc3_hydro_policy_mode": RC3_HYDRO_POLICY_MODE,
    "rc2g_tri_boundary_strict_passed": bool(globals().
↪get("RC2D_GATE_ALIGNED_STRICT_PASSED", False)),
    "notebook_filename": NOTEBOOK_FILENAME,
    "biometric_style_threshold_diagnostics_enabled": bool(globals().
↪get("ENABLE_BIOMETRIC_STYLE_THRESHOLD_DIAGNOSTICS", False)),
    "threshold_diagnostics_score_kind": globals().
↪get("THRESHOLD_DIAGNOSTICS_SCORE_KIND", None),
    "rotatedmnist_final_rotation_diagnostic_enabled": bool(globals().
↪get("ENABLE_ROTATEDMNIST_FINAL_ROTATION_DIAGNOSTIC", False)),
    "rotatedmnist_final_rotation_primary_diagnosis":
↪(rotated_final_rotation_diagnostic_df["primary_diagnosis"].iloc[0] if
↪"rotated_final_rotation_diagnostic_df" in globals() and
↪isinstance(rotated_final_rotation_diagnostic_df, pd.DataFrame) and not
↪rotated_final_rotation_diagnostic_df.empty else None),
    "rotatedmnist_final_rotation_gate_all_passed":
↪(bool(rotated_final_rotation_gate_df["passed"].all()) if
↪"rotated_final_rotation_gate_df" in globals() and
↪isinstance(rotated_final_rotation_gate_df, pd.DataFrame) and not
↪rotated_final_rotation_gate_df.empty else None),
    "results_dir": str(RESULTS_DIR),
    "zip_package": str(zip_path),
    "zip_sha256": zip_sha,
    "package_sha256_file": str(package_sha_path),
    "run_manifest": str(MANIFEST_PATH),
    "notebook_export_dir": str(NOTEBOOK_EXPORT_DIR),
    "created_utc": datetime.now(timezone.utc).isoformat(),
}
release_manifest_path.write_text(json.dumps(safe_jsonable(release_manifest),
↪indent=2, ensure_ascii=False), encoding="utf-8")

# Keep a human-friendly evidence bundle inside the run folder.
TRUSTED_EVIDENCE_DIR.mkdir(parents=True, exist_ok=True)
for evidence_file in sorted(RESULTS_DIR.glob("*.csv")):
    if any(token in evidence_file.name for token in ["scorecard",
↪"leaderboard", "gate", "selection", "summary", "eer", "score_dump"]):
        try:
            shutil.copy2(evidence_file, TRUSTED_EVIDENCE_DIR / evidence_file.
↪name)
        except Exception as e:

```

```

        print("TrustedEvidence CSV copy skipped:", evidence_file, repr(e))
# Copy threshold diagnostics into TrustedEvidence as a convenience.
for evidence_file in list((RESULTS_DIR / "threshold_curves").glob("*.csv")) if
↳ (RESULTS_DIR / "threshold_curves").exists() else []:
    try:
        dst = TRUSTED_EVIDENCE_DIR / "threshold_curves" / evidence_file.name
        dst.parent.mkdir(parents=True, exist_ok=True)
        shutil.copy2(evidence_file, dst)
    except Exception as e:
        print("TrustedEvidence threshold CSV copy skipped:", evidence_file,
↳ repr(e))
for evidence_file in list((PLOTS_DIR / "threshold_diagnostics").glob("*.png"))
↳ if (PLOTS_DIR / "threshold_diagnostics").exists() else []:
    try:
        dst = TRUSTED_EVIDENCE_DIR / "threshold_diagnostics" / evidence_file.
↳ name
        dst.parent.mkdir(parents=True, exist_ok=True)
        shutil.copy2(evidence_file, dst)
    except Exception as e:
        print("TrustedEvidence threshold figure copy skipped:", evidence_file,
↳ repr(e))

# Copy RotatedMNIST final-rotation diagnostic artifacts into TrustedEvidence.
for evidence_file in sorted(RESULTS_DIR.glob("rotatedmnist_final_rotation_*.
↳ csv")):
    try:
        shutil.copy2(evidence_file, TRUSTED_EVIDENCE_DIR / evidence_file.name)
    except Exception as e:
        print("TrustedEvidence RotatedMNIST final-rotation CSV copy skipped:",
↳ evidence_file, repr(e))
for evidence_file in list((PLOTS_DIR /
↳ "rotatedmnist_final_rotation_diagnostic").glob("*.png")) if (PLOTS_DIR /
↳ "rotatedmnist_final_rotation_diagnostic").exists() else []:
    try:
        dst = TRUSTED_EVIDENCE_DIR / "rotatedmnist_final_rotation_diagnostic" /
↳ evidence_file.name
        dst.parent.mkdir(parents=True, exist_ok=True)
        shutil.copy2(evidence_file, dst)
    except Exception as e:
        print("TrustedEvidence RotatedMNIST final-rotation figure copy skipped:
↳ ", evidence_file, repr(e))

for evidence_file in [MANIFEST_PATH, release_manifest_path, package_sha_path]:
    try:
        if evidence_file.exists():

```

```

        shutil.copy2(evidence_file, TRUSTED_EVIDENCE_DIR / evidence_file.
↪name)
        except Exception as e:
            print("TrustedEvidence manifest/hash copy skipped:", evidence_file,
↪repr(e))
try:
    shutil.copy2(zip_path, TRUSTED_EVIDENCE_DIR / zip_path.name)
except Exception as e:
    print("TrustedEvidence ZIP copy skipped:", repr(e))

print(f"Results zipped to: {zip_path}")
print(f"ZIP SHA-256: {zip_sha}")
print(f"Package SHA file: {package_sha_path}")
print(f"Release manifest: {release_manifest_path}")

# Add ZIP metadata to the run manifest after the ZIP exists.
write_manifest(extra={
    "zip_package": str(zip_path),
    "zip_sha256": zip_sha,
    "package_sha256_file": str(package_sha_path),
    "release_manifest": str(release_manifest_path),
})
print("Final manifest:", MANIFEST_PATH)

```

2.45 17. Next scientific step after RC3

If RC3 strict passes across the MNIST-family benchmarks, the next step is not simply to declare Hydro a performance winner.

The next step is:

MMALS v1.1-RC4 / v1.2 - RC2c / Hydro-audited policy selection across harder visual and class-i

Decision logic:

- If RC3 selects the same policy as RC2c but reports low Hydro turbulence, Hydro is validated as an audit layer.
- If RC3 changes the selected policy only when stability-aware weights are enabled, run an ablation before claiming improvement.
- If Hydro audit warns on the selected policy while accuracy remains high, treat this as a debuggability finding rather than a failure.
- Only after an aligned inferred-context/global-head protocol should Hydro be considered as a default learning regularizer.

2.46 17. Explanation for a 12-year-old kid

Before, the mushroom model was tested on normal images.

Now we can make the game harder in two ways:

1. rotate the handwritten digits (**RotatedMNIST**);
2. scramble the pixels differently for each lesson (**PermutedMNIST**).

The lessons still overlap:

- lesson 0: digits 0 and 1;
- lesson 1: digits 1 and 2;
- lesson 2: digits 2 and 3;
- lesson 3: digits 3 and 4;
- lesson 4: digits 4 and 5.

For **RotatedMNIST**, each lesson is turned a little differently:

- -30 degrees;
- -15 degrees;
- 0 degrees;
- +15 degrees;
- +30 degrees.

For **PermutedMNIST**, each lesson uses a different secret pixel shuffle:

- seed 100;
- seed 101;
- seed 102;
- seed 103;
- seed 104.

So the mushroom must not only remember old digits. It must also understand that the situation changed.

In this observability patch, we keep a black box recorder, like in an airplane.

It records:

- which situation the mushroom thought it was in;
- which memory/prototype it used;
- whether the head/readout was stable;
- whether one seed was weaker than the others;
- whether the model beat replay and other baselines.

The important idea is:

We are not changing the mushroom again. We are testing whether the same mushroom stays reliable on harder tracks.

2.47 Recommended RC2JG validation path

1. Run `RC2JG_GENERIC_WEAK_BOUNDARY_LATENT_CALIBRATION_HYDRO_AUDIT` in **smoke** mode on MNIST to verify execution, regime audit, Drive archive and artifact export.
2. Run `RC2JG_GENERIC_WEAK_BOUNDARY_LATENT_CALIBRATION_HYDRO_AUDIT` in **evidence** mode on MNIST to verify that the final checkpoint locks to the RC1b guarded context-gap family when ambiguity remains.

3. Only if evidence passes selector/final alignment, run RC2JG_GENERIC_WEAK_BOUNDARY_LATENT_CALIBRATION_HYDRO_AUDIT in robust mode on MNIST.
4. Then run robust on MNIST, RotatedMNIST and PermutedMNIST to verify that clean-context datasets still prefer proto/global no-repair.
5. Compare RC2Je against RC2Jb, RC2J, RC2I, fixed RC1b, proto/global no-repair, replay/balanced replay and the best deployable candidate per dataset.

RC2JG should be treated as a **candidate-only, validation-memory weak-boundary repair**, not as a new universal default or a test-triggered fix.

2.48 Export and Drive structure reminder

Notebook name:

MMALS_Prototype_1_1_RC2jh_Generic_Weak_Boundary_Latent_Calibration_FashionMNIST_Robust_LSA_PAN

Recommended execution for the patched RC2jh FashionMNIST robust run:

```
EXPERIMENT_PROFILE = "RC2JG_GENERIC_WEAK_BOUNDARY_LATENT_CALIBRATION_HYDRO_AUDIT"
RUN_MODE = "robust"
DATASET_NAME = "FashionMNIST"
ACTIVATE_ENERGY_CONTEXT_PROBE = True
RC2JF_DATASET_AGNOSTIC_TRUE_GUARD_HYGIENE_ENABLED = True
```

Expected Drive structure:

```
MMALS/
  v1_1_rc2jg_generic_weak_boundary_latent_calibration/
    RC2JG_GENERIC_WEAK_BOUNDARY_LATENT_CALIBRATION_HYDRO_AUDIT/
      MMALS_v11_RC2JG_GENERIC_WEAK_BOUNDARY_LATENT_CALIBRATION_HYDRO_AUDIT_FashionMNIST_robust.
```

```
[ ]: from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
[ ]: import os

!apt-get update
!apt-get install texlive texlive-xetex texlive-latex-extra pandoc
!pip install py pandoc
!pip -q install nbformat
```

```
Hit:1 https://cli.github.com/packages stable InRelease
Hit:2 http://archive.ubuntu.com/ubuntu jammy InRelease
Get:3 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Get:4 https://cloud.r-project.org/bin/linux/ubuntu jammy-cran40/ InRelease
[3,632 B]
Hit:5 https://r2u.stat.illinois.edu/ubuntu jammy InRelease
```

```

Get:6 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Hit:7 http://archive.ubuntu.com/ubuntu jammy-backports InRelease
Hit:8 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu jammy InRelease
Get:9 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages
[1,297 kB]
Get:10 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages
[3,959 kB]
Hit:11 https://ppa.launchpadcontent.net/ubuntugis/ppa/ubuntu jammy InRelease
Get:12 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [4,311
kB]
Get:13 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages
[1,605 kB]
Fetched 11.4 MB in 2s (6,373 kB/s)
Reading package lists... Done
W: Skipping acquire of configured file 'main/source/Sources' as repository
'https://r2u.stat.illinois.edu/ubuntu jammy InRelease' does not seem to provide
it (sources.list entry misspelt?)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
pandoc is already the newest version (2.9.2.1-3ubuntu2).
texlive is already the newest version (2021.20220204-1).
texlive-latex-extra is already the newest version (2021.20220204-1).
texlive-xetex is already the newest version (2021.20220204-1).
0 upgraded, 0 newly installed, 0 to remove and 9 not upgraded.
Requirement already satisfied: pypandoc in /usr/local/lib/python3.12/dist-
packages (1.17)

```

```

[ ]: # Explicitly setting the notebook filename for this execution,
# as there isn't a robust automatic detection method in Colab.
notebook_filename =
↳ "MMALS_Prototype_1_1_RC2M_beta_Dynamic_Boundary_Ambiguity_Lock_Benchmark_Harness.
↳ ipynb"
notebook_name_without_ext = os.path.splitext(notebook_filename)[0]

# Define the full path to the notebook in Google Drive
full_notebook_path_drive = f"/content/drive/MyDrive/Colab Notebooks/
↳ {notebook_filename}"
local_notebook_path = f"/content/{notebook_filename}"

# Copy the notebook to the local /content/ directory to avoid path issues with
↳ nbconvert
print(f"Executing: cp \"{full_notebook_path_drive}\" \"{local_notebook_path}\"")
!bash -c "cp \"{full_notebook_path_drive}\" \"{local_notebook_path}\""

```

```

Executing: cp "/content/drive/MyDrive/Colab Notebooks/MMALS_Prototype_1_1_RC2M_b
eta_Dynamic_Boundary_Ambiguity_Lock_Benchmark_Harness.ipynb" "/content/MMALS_Pro

```

```
totype_1_1_RC2M_beta_Dynamic_Boundary_Ambiguity_Lock_Benchmark_Harness.ipynb"
```

```
[ ]: [!]jupyter nbconvert \
      --to pdf \
      --output-dir='/content' \
      --PDFExporter.latex_command=["'xelatex', '{filename}',
      ↪'-interaction=nonstopmode'"]" \
      "/content/
      ↪MMALS_Prototype_1_1_RC2M_beta_Dynamic_Boundary_Ambiguity_Lock_Benchmark_Harness.
      ↪ipynb"
```

```
/usr/local/lib/python3.12/dist-packages/traitlets/traitlets.py:2915:
FutureWarning: --PDFExporter.latex_command=['xelatex', '{filename}',
'-interaction=nonstopmode'] for containers is deprecated in traitlets 5.0. You
can pass `--PDFExporter.latex_command item` ... multiple times to add items to a
list.
  warn(
[NbConvertApp] Converting notebook /content/MMALS_Prototype_1_1_RC2M_beta_Dynami
c_Boundary_Ambiguity_Lock_Benchmark_Harness.ipynb to pdf
[NbConvertApp] Support files will be in MMALS_Prototype_1_1_RC2M_beta_Dynamic_Bo
undary_Ambiguity_Lock_Benchmark_Harness_files/
[NbConvertApp] Making directory ./MMALS_Prototype_1_1_RC2M_beta_Dynamic_Boundary
_Ambiguity_Lock_Benchmark_Harness_files
[NbConvertApp] Writing 2362571 bytes to notebook.tex
[NbConvertApp] Building PDF
[NbConvertApp] Running xelatex 3 times: ['xelatex', 'notebook.tex',
'-interaction=nonstopmode']
[NbConvertApp] Running bibtex 1 time: ['bibtex', 'notebook']
[NbConvertApp] WARNING | bibtex had problems, most likely because there were no
citations
[NbConvertApp] PDF successfully created
[NbConvertApp] Writing 3439031 bytes to /content/MMALS_Prototype_1_1_RC2M_beta_D
ynamic_Boundary_Ambiguity_Lock_Benchmark_Harness.pdf
```